LLD Handbook — Vol 4: Case Studies II, Cheat Sheets, Question Bank

OOP · UML · SOLID · Clean Code · Refactoring · Design Patterns · Concurrency · 45+ case studies in Java

6 parts · one place to study LLD: DesignGurus courses + classic books + GitHub problem sets, combined

what it is | problem | how it works | trade-offs | real world | interview | self-check | TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 18 — Case Studies: Platforms & Feeds (Part 1)**
   - 18.0 What platform problems test
   - 18.1 Design Stack Overflow
   - 18.2 Design Cricinfo
   - 18.3 Design LinkedIn
   - 18.4 Design Facebook
   - 18.5 Chapter summary
2. **Chapter 19 — Case Studies: Commerce & Marketplaces (Part 2 of Platforms)**
   - Design Amazon — Online Shopping System
   - Design an Online Stock Brokerage System
   - Design an Online Auction System
   - Design a Music Streaming Service (Spotify)
   - Design an Online Food Delivery Service (Swiggy) + FoodKart
   - Design a Grocery Store (POS & Checkout)
   - Chapter summary
3. **Chapter 20 — Case Studies: Pattern-Driven Components**
   - 20.1 Design Splitwise
   - 20.2 Design a Notification Service
   - 20.3 Design an In-Memory Cache (and an LRU Cache from scratch)
   - 20.4 Design a Ride Sharing Service (Uber)
   - 20.5 Design a Logging Framework
   - 20.6 Design a Pub-Sub System
   - 20.7 Design a Rate Limiter
   - 20.8 Chapter summary
   - 20.9 Chapter self-check
   - 20.10 Chapter TL;DR
4. **Chapter 21 — Case Studies: Machine-Coding Round Problems**
   - 21.0 How the machine-coding round works
   - 21.1 Design a Task Management System
   - 21.2 Design a Digital Wallet Service
   - 21.3 Design an Event Calendar (Flipkart)
   - 21.4 Design a LeetCode-like Coding Platform — "Coding Blox" (Flipkart)
   - 21.5 Design a Property Listing Site — "PropertyHunt" (ClearTrip)
   - 21.6 Design a Loan Ledger — "Ledger Co" (Navi)
   - 21.7 Design an Inventory Management System
   - 21.8 Design a Unix File Search (`find`)
   - Chapter summary
5. **Chapter 22 — Revision: Cheat Sheets & Glossary**
   - 22.1 How to revise
   - 22.2 The LLD interview answer framework (one page)
   - 22.3 OOP & relationships cheat sheet
   - 22.4 SOLID one-pager
   - 22.5 Other design principles and Clean Code rules
   - 22.6 Code smells → refactorings (all 24)
   - 22.7 Design patterns: all 23 GoF + extras
   - 22.8 Concurrency cheat sheet
   - 22.9 Case-study map
   - 22.10 Machine-coding and edge-case checklists
   - 22.11 Glossary
   - Self-check
   - TL;DR
6. **Chapter 23 — Interview Question Bank**
   - How to use this bank
   - Part A — "Design X" (LLD and machine coding) questions
   - Everyday objects
   - Booking and reservations
   - Games
   - Platforms and marketplaces
   - Components and infrastructure
   - Machine-coding round favourites
   - Part B — Concept questions
   - OOP basics
   - Relationships and UML
   - SOLID
   - Design principles and clean code
   - Creational patterns
   - Structural patterns
   - Behavioral patterns
   - Concurrency
   - Java specifics for LLD rounds
   - Part C — How to run a 60–90 minute machine-coding round
   - Sources of the question lists

Chapter 18 — Case Studies: Platforms & Feeds (Part 1)

> **Where this fits.** Chapters 14–17 designed *things* (a parking lot, a vending machine), *bookings* (movies, hotels, flights) and *games*. This chapter and the next one design **platforms**: websites where many people create content, react to each other's content and build up a history. The objects are no longer machines with buttons; they are people, posts, votes, relationships and feeds. Chapter 19 continues with more platforms.

What you will learn

#	Problem	What makes it interesting	Main patterns and ideas
18.0	What platform problems test	The common toolkit you will reuse in every social or community design	Facade, Observer, Strategy, services for two-sided operations, inverted index
18.1	Design Stack Overflow	Reputation unlocks privileges; one vote per member; moderation; bounties	Shared `Post` base class, Strategy (reputation rules), Observer (badges), privilege table
18.2	Design Cricinfo (+ Udaan "Cricket Match Dashboard")	Modelling a real sport's rules; live scorecards; stats derived from balls	Template Method (formats and stat queries), Observer (live feeds), "derive, don't store"
18.3	Design LinkedIn	Profiles, connections and their degrees, invitations, messaging, jobs, search, feed	Service for two-sided operations, State (invitation), BFS for degrees, Strategy (feed ranking)
18.4	Design Facebook	Friends vs followers, privacy lists, pages, groups, news feed, friend suggestions	One visibility policy for feed + search, sealed `Audience`, pull-model feed, suggestions

How to run every example in this chapter

```
cd chapters/18-case-platforms-1/code
javac --release 21 -d /tmp/lld_18 $(find . -name '*.java')
java -cp /tmp/lld_18 lld.ch18.stackoverflow.Main       # or cricinfo, cricketdashboard, linkedin, facebook
```

`javac` is the Java **compiler** (it turns `.java` source files into `.class` files the Java runtime can execute); `--release 21` says which Java version to target, and `-d` names the output folder. `java -cp` runs a program, where the **classpath** (`-cp`) is the list of folders in which Java looks for compiled classes. The **standard library** is the set of classes that ships with Java itself, so nothing extra has to be downloaded.

All five `Main` classes compile with Java 21 (standard library only) and run. The output printed under each design is the real output of that program.

> **Code-style note.** To keep a whole design readable in one place, a file sometimes holds several small package-private types (for example `Posts.java` holds `Post`, `Question`, `Answer`, `Comment`, `Tag` and `Bounty`). That is legal Java. In a production codebase you would usually give each public top-level type its own file.

> **Key term — Java vocabulary used in this chapter:** A **package** is a named folder of related classes (`lld.ch18.stackoverflow`); a **package-private** type has no `public`/`private` keyword and can be used only by classes in the same package. A **top-level type** is a class, interface, enum or record not nested inside another one. An **interface** lists methods a class promises to provide, without the code; a class that provides them *implements* it. An **abstract class** can never be created directly — only its subclasses can. An **enum** is a type with a fixed list of named values (`OPEN`, `CLOSED`). A **record** is a short way to declare a class that just carries data that never changes. A **sealed** type names exactly which subclasses are allowed, so the compiler knows every possible kind.

18.0 What platform problems test

What it is

A **platform problem** asks you to design a website or app where *users produce the content* — questions and answers, posts and comments, match commentary, job postings. Stack Overflow, Cricinfo, LinkedIn and Facebook are the classic examples. Unlike a vending machine, there is no single "machine" object in the middle. Instead there is a **rich domain model**: many small classes (profile, post, comment, vote, tag, badge, invitation, group, page…) that each hold a little data and a little behaviour.

Everyday analogy: a vending machine is like a toaster — one box with a few buttons. A platform is like a town — many people, many buildings, rules about who may enter where, a notice board (the feed), a post office (notifications) and a library catalogue (search).

The problem it solves

Interviewers pick platform problems to check six skills at once. The table names each one, defines it, and shows where this chapter practises it.

Skill tested	What it means	Where you practise it
Rich domain model	Turning a long list of nouns into classes with clear responsibilities, and knowing which noun is *just data* (a record) and which has *behaviour*	Cricinfo's `Over` knows a wide is not one of its six balls
User roles	Different people may do different things: guest, member, moderator, admin, commentator	Stack Overflow's reputation-based privileges; Facebook's admin who blocks members
Feeds	A **feed** is a personalised, ranked list of recent content from the people and pages you are linked to	LinkedIn and Facebook feed services with a ranking Strategy
Notifications	Telling a user that something happened (new message, friend request, comment) through one or more **channels** (in-app, email, push — a pop-up alert sent to a phone or browser)	Observer + Strategy in every design
Search	Finding things by words. Built with an **inverted index**: a map from each word to the items that contain it	Question search, member search, privacy-aware post search
Reputation / trust	Numbers that summarise past behaviour and unlock abilities (reputation, badges, endorsements, ratings)	Stack Overflow reputation and badges; LinkedIn endorsements; Facebook page ratings

How it works — the reusable toolkit

These ideas repeat in every design in this chapter. Learn them once.

1. **A facade in front.** A **facade** is one class that offers the simple operations the UI (user interface — the screens and buttons people use) needs (`ask`, `vote`, `post`, `invite`) and hides the many classes behind it (Chapter 9). It is also the natural place to check "is this account active?" and "is this person allowed?" before anything changes.
2. **Behaviour shared by content goes in a base class.** Questions, answers and comments are all voted on; posts and comments are all liked. Write that once in a base class (`Post`, `SocialContent`) and let the content-specific parts stay in subclasses.
3. **Store *who*, not *how many*.** Keep likes and votes as a **set of member ids**, not an integer counter. A set blocks double voting, allows "unlike", and still gives the count as `set.size()`.
4. **Derive, don't store.** If a number can be computed from data you already keep (follower count, a batter's runs, a page's rating), compute it. A stored copy will one day disagree with the source and nobody will know which is right. If it is too slow to compute every time, keep a **cache** (a saved copy of a computed answer, kept so it can be read quickly instead of recomputed) that you can always rebuild from the source. See System Design Fundamentals for caches at scale.
5. **An operation that changes two objects lives in a service.** Accepting a connection or friend request changes *both* members. Put it on neither — put it in a small **service** class (a class whose job is to perform an operation, not to hold data) so there is exactly one place that keeps both sides consistent.

6. **Notifications through Observer + Strategy.** The domain *publishes* events ("answer posted"); **observers** (Chapter 10) react. The **Observer** pattern means objects *subscribe* to another object and are called back automatically whenever it announces an event, so the announcer does not need to know who is listening. The **Strategy** pattern means putting each interchangeable way of doing a job (here: each delivery method) behind one interface, so you can swap or add ways without editing the caller. How a message is *delivered* (email, push, in-app) is a **Strategy** (Chapter 11) so adding a channel means adding a class, not another `if`.
7. **Search is its own class.** An **inverted index** (word → items) belongs in a `SearchIndex`, not on the entity. Rule from the course: every key maps to a *collection*, even when only one item matches, so the second "Sharma" never breaks the index.
8. **Privacy and permissions are asked in one place.** If both the feed and the search must hide private posts, write `canSee(viewer, post)` once and call it from both.

Trade-offs

Choice	Benefit	Cost
Rich model (many small classes)	Each rule has one home; easy to test	More classes to explain in 45 minutes
Sets of voter ids instead of counters	Correct: no double votes, reversible	Memory grows with votes; real systems store votes in a table and cache the count
Derive stats from raw events	Never inconsistent	Slow for large data; needs a cache
Services for two-sided operations	Symmetry guaranteed in one place	One more class; entities become thinner ("anemic") if you overdo it

When not to use it

Do not bring the whole toolkit to a tiny problem. If the interviewer asks only for "a like button", a set on the post is enough — no services, no observers.

Common mistakes

- Putting search as a static method on `Question` ("`Question.search()`"), so the entity must know every other question.
- Storing `likeCount` as a constructor argument — a post can then be created "with 1,000 likes".
- Forgetting that the privacy rule also applies inside search and the feed.
- Writing `acceptInvitation` on the sender (or the receiver) and updating only one side.

Real-world examples

- **Kafka** (a system that stores streams of events and delivers them to any program that wants to read them) was created at LinkedIn to move activity events (page views, posts, connections) between systems — the industrial version of the Observer idea used here.
- **Facebook's TAO** is a graph store (a database built for a **graph** — items joined by links, like people joined by friendships) built to serve the social graph (users, friendships, likes) at very large scale — the production version of our in-memory friend graph.
- **Stack Overflow** publishes its reputation-to-privilege table on its help pages — exactly the data-driven `Privilege` enum used in 18.1.

In the interview

> **30-second answer:** "Platform problems test whether I can model a domain with many small classes, put each rule in one place, and handle roles, feeds, notifications and search. I'll start with requirements and actors, then the core entities, then the tricky flows — voting, accepting requests, privacy — and I'll say which operations need a service because they touch two objects."

Follow-ups:

- *Why a set instead of a counter for likes?* — A bare number does not record *who* liked, so it can neither block a repeat like nor undo one. The set gives the count for free.
- *Where does the notification code go?* — Not inside the domain methods directly; the domain raises an event and a notification service with pluggable channels delivers it.

Self-check

- **Q:** What is an inverted index? **A:** A map from each word to the items containing that word; used to answer "which items contain these words?" quickly.
- **Q:** Why is "accept a friend request" not a method on `Member`? **A:** It changes two members; putting it on one of them means one object edits another's state, and the two halves can drift. A service updates both together.
- **Q:** Name two ways to deliver a notification and the pattern that makes adding a third easy. **A:** Email and push; Strategy (a `NotificationChannel` interface).

TL;DR

Platform problems = rich domain model + roles + feed + notifications + search + reputation. Share behaviour in a content base class, store *who* not *how many*, derive numbers, put two-sided operations in services, notify via Observer + Strategy, keep search and privacy each in one place.

18.1 Design Stack Overflow

Figure — The Stack Overflow logo that opens this case study (course diagram)

Stack Overflow is a huge question-and-answer community for programmers. Members can **upvote** or **downvote** questions and answers, and anyone with enough standing can edit posts, much like a wiki (a website whose pages any permitted reader can edit). Members earn **reputation** (a points score) and **badges** (named awards) for useful contributions. Higher reputation unlocks **privileges** — voting, commenting anywhere, editing other people's posts, voting to close questions. For scale: an upvote on an answer is worth 10 points to its author. The course says a question upvote is worth 5; that was true until November 2019, when Stack Overflow raised it to 10 as well, and this chapter's code uses today's value.

Problem & clarifying questions

Ask these before drawing anything. The answers are the assumptions this design uses.

Question to ask	Answer we assume
Who uses the system?	Guests (not logged in), members, moderators, admins, and the system itself (notifications, badges)
What can a guest do?	Search and read only. Posting or voting requires membership
What can be voted on?	Questions, answers and comments (comments: upvote only, as on the real site)
How much reputation does a vote give?	Upvote +10 on a question or an answer (question upvotes were +5 before November 2019), downvote −2 to the author, −1 to whoever downvotes an answer (downvoting a question is free), accepted answer +15 (and +2 to the asker)
Does reputation unlock actions?	Yes — a privilege table (vote up at 15, comment everywhere at 50, set bounty at 75, vote down at 125, create tag at 1,500, edit at 2,000, close votes at 3,000, delete votes at 10,000)
How are questions closed?	3 close votes from members with the privilege, or one moderator. Moderators can also reopen, delete and undelete
What is a bounty?	Reputation a member (usually the asker) stakes, 50 to 500 points, to attract answers. It runs for 7 days (the real site adds a 24-hour grace period) and goes to the answer the sponsor picks. The staked points are taken at once and are never refunded; if the sponsor picks nobody, the real site may auto-award half to a qualifying new answer
Do we need search?	Yes: by keyword and by tag, plus "most used tags"
Scale?	Read-heavy (many more views than posts); votes must never double-count

Requirements

Functional (the course's thirteen, in our words)

1. Guests can search and view questions; to post or upvote they must become members.
2. Members can post new questions.
3. Members can answer an **open** question.
4. Members can comment on any question or answer.
5. Questions, answers and comments can all be upvoted by members.
6. Members can **flag** any question, answer or comment that has a serious problem or needs a moderator to look at it.
7. A member can attach a **bounty** to a question they asked.
8. Members earn **badges** for being helpful.
9. Members can cast votes to **close** a question; a moderator can close or reopen any question on their own.
10. Members can add **tags** (a word or phrase naming the topic) to questions.
11. Members can vote to **delete** far off-topic or very low-quality questions.
12. Moderators can close questions and restore (undelete) deleted ones.
13. The system can report the **most frequently used tags**.

Plus from the real site (added so the privilege rules have somewhere to live): downvotes, accepting an answer, reputation-gated privileges, blocking accounts.

Non-functional

- **Correctness of counts:** one member, one vote per post; reputation changes must be reversible when a vote changes.
- **Read-heavy performance:** search and "top tags" must not scan every question.
- **Concurrency** (several pieces of work running at the same time — for example two requests handled by two **threads**, the independent paths of execution inside one program)**:** two members voting on the same post at the same instant must both be counted exactly once.
- **Abuse resistance:** you cannot vote on your own post; blocked accounts can do nothing.
- **Extensibility** (how easily new features can be added without rewriting old code)**:** reputation rules, badge rules and notification channels change often — changing them should not touch voting code.

Use cases

The five actors:

- **Guest** — searches and views questions.
- **Member** — everything a guest can do, plus ask, answer, comment, vote, flag, add bounties and tags, and delete or undelete their own posts.
- **Moderator** — everything a member can do, plus close, reopen, delete and undelete *any* question.
- **Admin** — blocks and unblocks members.
- **System** — sends notifications and assigns badges.

Top use cases: search questions; ask a question (with tags and an optional bounty); write or edit answers; add comments; moderators close, delete and undelete questions.

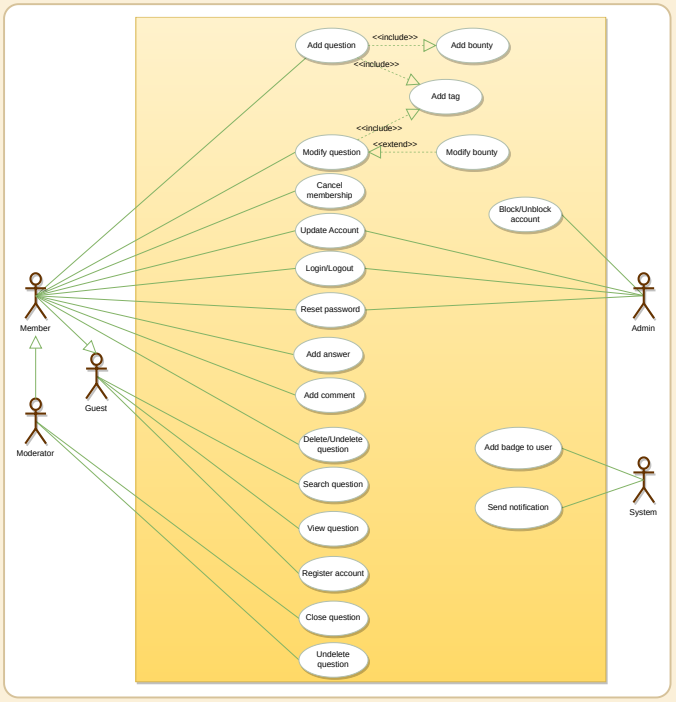


Figure — Use cases per actor: Guest (search, view, register), Member (add/modify question, answer, comment, bounty, tag), Moderator (close, delete/undelete), Admin (block/unblock), System (send notification, add badge) (course diagram)

A **use-case diagram** is a UML picture of the actors (stick figures) and the things each one can do (ovals). In the figure above, notice the `<<include>>` from "Add question" to "Add tag" (adding a tag is part of asking) and the `<<extend>>` for bounties (an optional extra on a question).

An **activity diagram** is a flowchart of the steps and yes/no decisions in one task; a **sequence diagram** shows, top to bottom in time order, the messages objects send to each other.

Activity — post a new question. The member clicks *Ask Question*, types a title and body, and adds tags. For each tag, the system checks whether it already exists; if not, it checks whether the member has **enough reputation to create a tag**. Then it validates the question and either saves and posts it or shows an error.

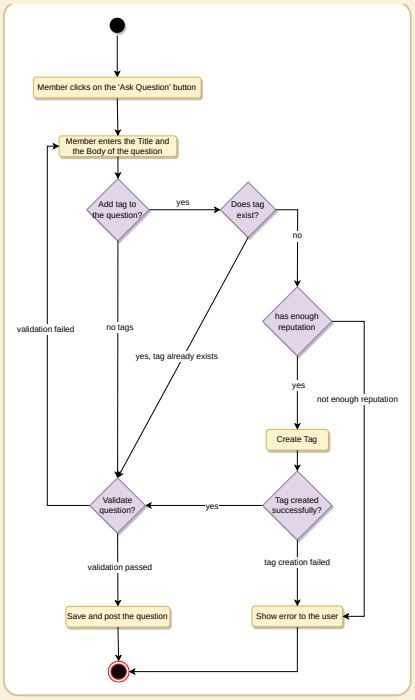


Figure — Activity diagram for posting a question, including the "does the tag exist?" and "has enough reputation?" decisions (course diagram)

Sequence — create a question. The member sets the title and body, looks up tags (creating one if missing), creates the question with a bounty value, the question creates its `Bounty`, and finally the question is published.

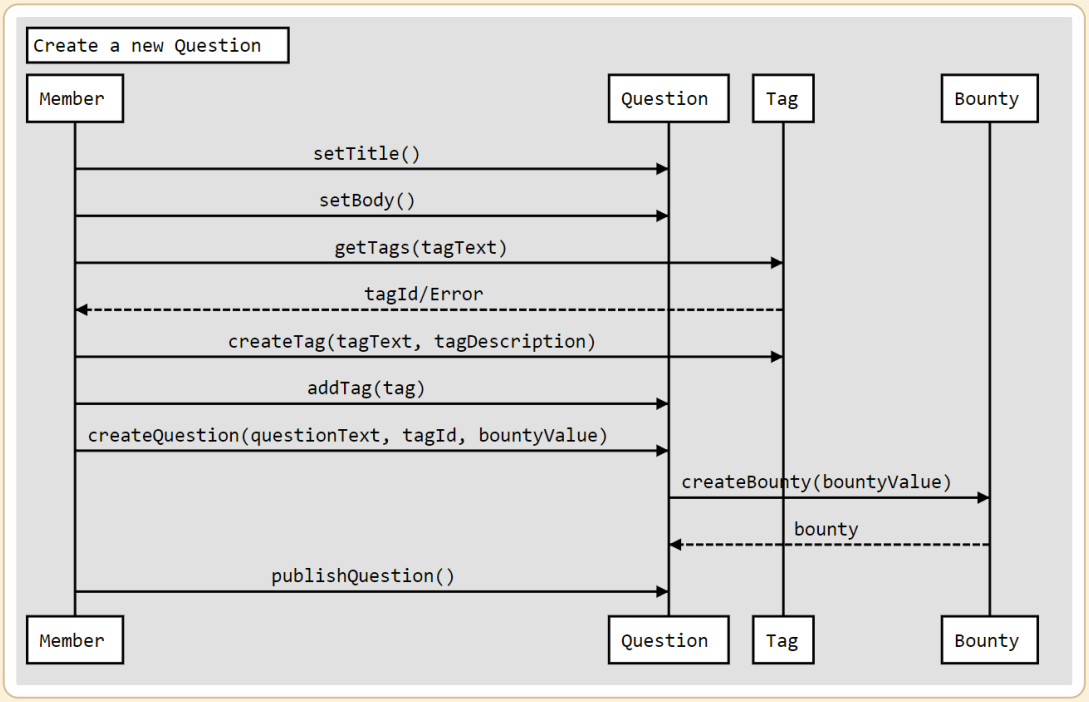


Figure — Sequence diagram: Member → Question (setTitle, setBody), Member → Tag (getTags / createTag), Question → Bounty (createBounty), then publishQuestion (course diagram)

Our Java code follows the same order: `ask()` resolves each tag, creating it only if the asker holds the `CREATE_TAG` privilege, and then builds the question; `offerBounty()` attaches the bounty.

Core entities

Class	Responsibility
`Member` (+ `Moderator`, `Admin`)	Identity, reputation (atomic — safe when updated by many threads at once), badges, account status; `hasPrivilege(p)`
`Privilege` (enum)	The reputation needed for each action — a data table, not scattered `if`s
`Post` (sealed, abstract)	Shared behaviour: author, body, votes per voter, flags, comments
`Question`	Title, tags, answers, status (OPEN / CLOSED / DELETED…), closing remark, close/delete voters, bounty, view count
`Answer`	Belongs to one question; can be *accepted*. (The course also gives answers and comments a view count and vote/flag counts; our code derives the vote and flag counts from the voter and flagger sets)
`Comment`	Belongs to a question or answer
`Tag`	Name + description; counts its own uses (for "top tags")
`Bounty`	Staked reputation, sponsor, expiry date, winner
`Badge` (record)	Name + description
`ReputationPolicy` / `DefaultReputationPolicy`	Strategy: how many points each event is worth
`ReputationListener` / `BadgeAwarder`	Observer: reacts to reputation changes (awards badges)
`Notifier` + `NotificationChannel` / `EmailChannel`	Sends notifications through pluggable channels
`Search` / `QuestionIndex`	Inverted index by word and by tag
`StackOverflow`	Facade: checks status + privilege, delegates, applies reputation

Class diagram

The course builds the class diagram in **five stages**, each adding a few classes for a few requirements. Classes from earlier stages are drawn in grey, so the picture fills in step by step.

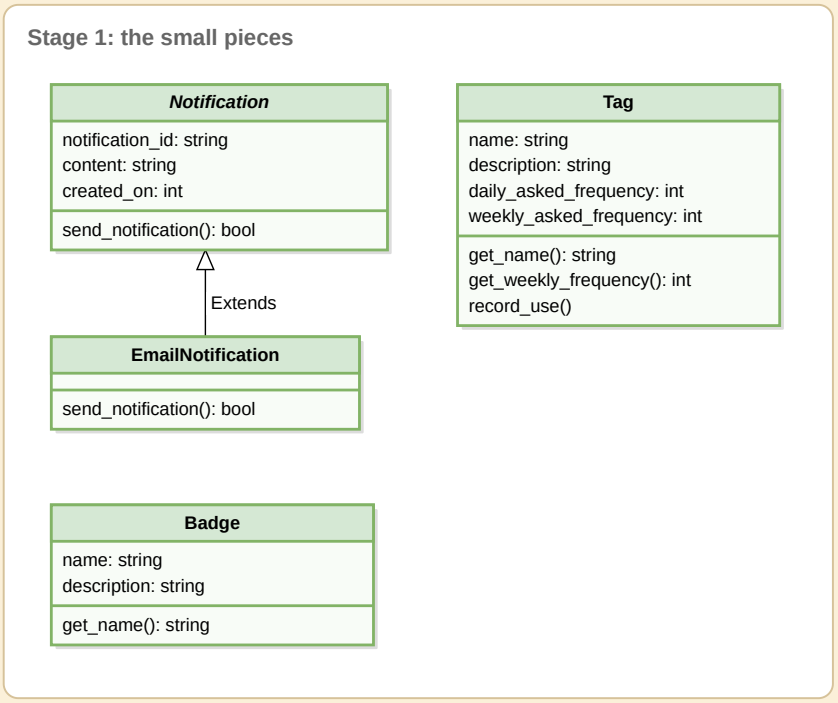


Figure — Stage 1: `Badge`, `Tag` (with daily and weekly usage counters and `record_use`) and abstract `Notification` with its `EmailNotification` subclass (course diagram)

Stage 1 covers requirements 8, 10 and 13. None of these classes depends on anything yet. The key idea is in `Tag`: it counts its own uses when attached to a question, so "most used tags" becomes a quick read rather than a scan of every question. `Notification` is abstract with one subclass per delivery method — an email notification genuinely is a notification that can be sent, so the inheritance passes the substitution test (the Liskov Substitution Principle: a subclass object must work correctly anywhere its parent class is expected).

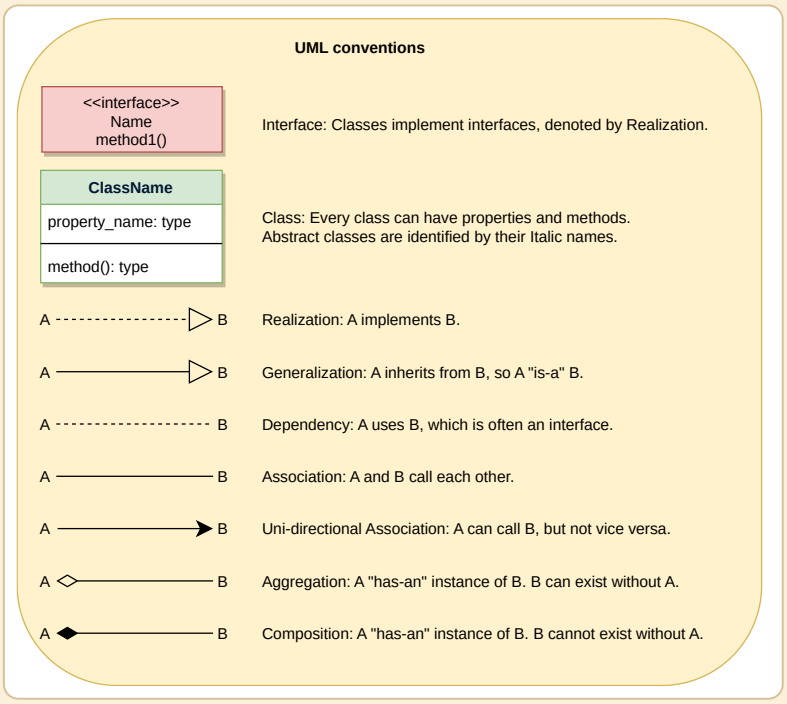


Figure — The UML legend used in these diagrams: class, interface, realization, generalization, dependency, association, aggregation and composition arrows (course diagram)

UML (Unified Modeling Language) is the standard notation for drawing classes and their links. In short: **generalization** is the inheritance arrow ("is a kind of"); **realization** means a class implements an interface; **dependency** means one class merely uses another for a moment; **aggregation** (hollow diamond) means "has, but does not own"; **composition** (filled diamond) and **association** are explained in Stage 3 below.

Figure — Stage 2 adds `Photo` (attachments on questions and answers) and `Bounty` (reputation held until awarded or expired) (course diagram)

Stage 2 covers requirement 7. The bounty keeps the staked amount itself rather than on the question, so the money-like value has one owner until it is settled. The course's reason for this is that the amount

can then go back to the sponsor if nobody earns it. Be careful in an interview: on the real site a bounty is *not* refundable, so our code deducts the points when the bounty starts and never returns them.

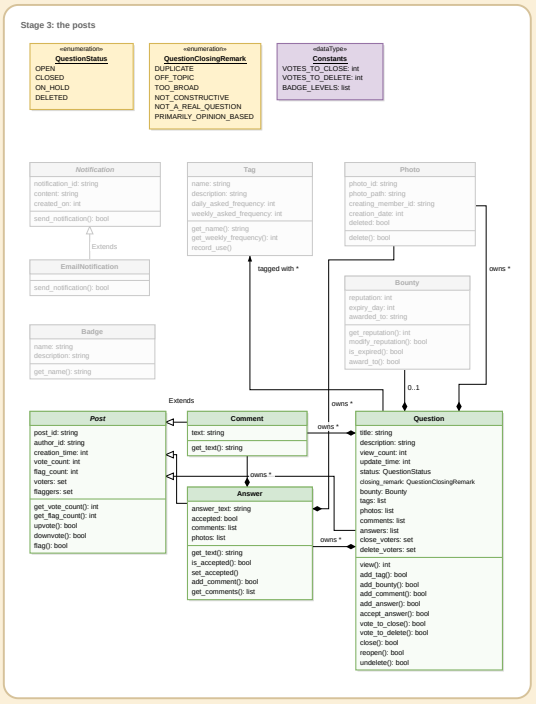


Figure — Stage 3: abstract `Post` with `Question`, `Answer` and `Comment` below it; filled diamonds from `Question` to its answers, comments, photos and bounty; a plain association from `Question` to `Tag` (course diagram)

Stage 3 is the heart of the design. Requirements 5 and 6 say that upvoting and flagging work the *same way* on three classes, so that behaviour is written once in `Post`. What *differs* is what each one contains, so they remain three separate classes instead of a single class plus a "kind" field. `Post` also remembers **who** voted — otherwise one member could vote ten times. The filled diamonds are **composition**: deleting a question also deletes its answers, comments, photos and bounty. `Tag` is linked by a plain **association** instead: many questions use the same tag, and the tag lives on after any one of them is deleted. Because `Post` keeps only the author's id, the diagram has no arrow from a post to a member.

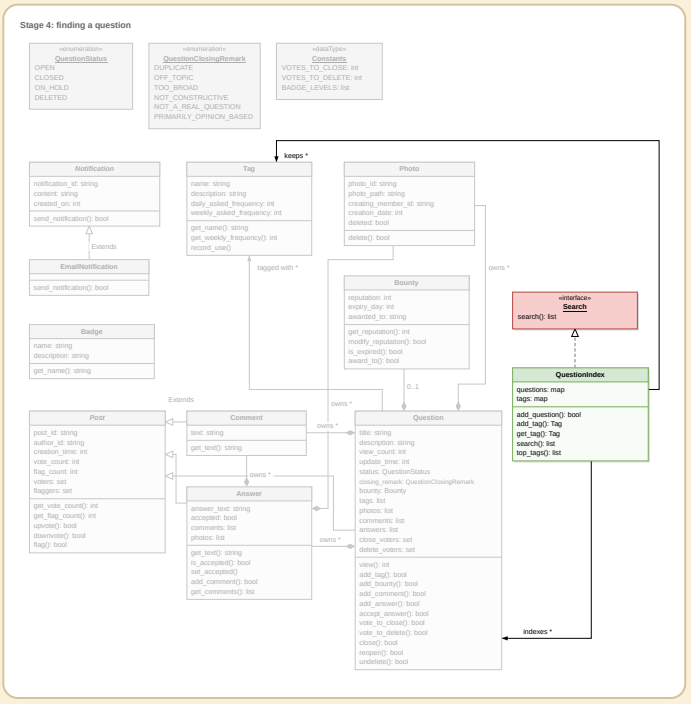


Figure — Stage 4: the `Search` interface and `QuestionIndex`, which holds (but does not own) questions and tags (course diagram)

Stage 4 covers search and requirement 13. A tempting first attempt makes search a static method (a method that belongs to the class itself rather than to any one object) on `Question`, which forces `Question` to know

every question in existence — not its job. The index implements `Search` (dashed line with hollow head = realization). Its arrows are plain associations: deleting the index deletes no question.

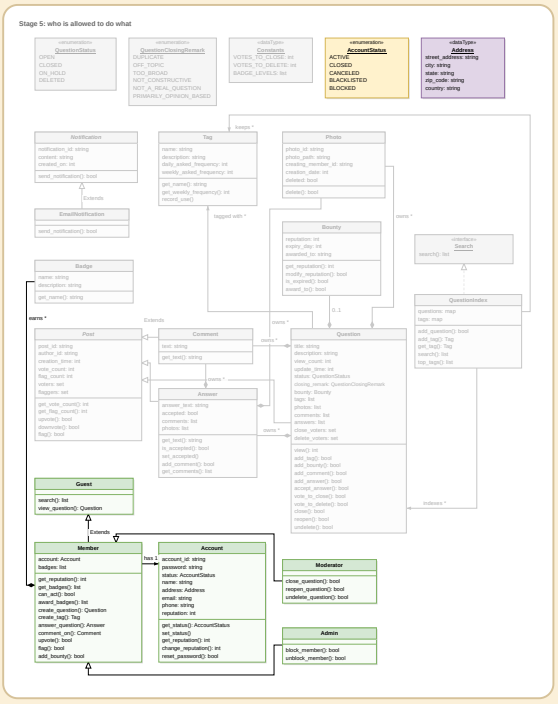


Figure — Stage 5: `Account`, `Guest`, `Member` (extends Guest), `Moderator` and `Admin` (extend Member); `Member` owns its badges (course diagram)

Stage 5 covers requirement 1 by the *shape* of the classes: the methods a guest may call sit on `Guest`, and `Member` inherits them and adds posting and voting. A guest simply has no `createQuestion` method, so no check can be forgotten. `Moderator` and `Admin` inherit from `Member` since each of them may do anything a member may (the substitution test passes). `Member` owns its badges (filled diamond): a badge is awarded to that one member. The link from a member to its `Account` is a plain association, because the account is created elsewhere and handed to the member.

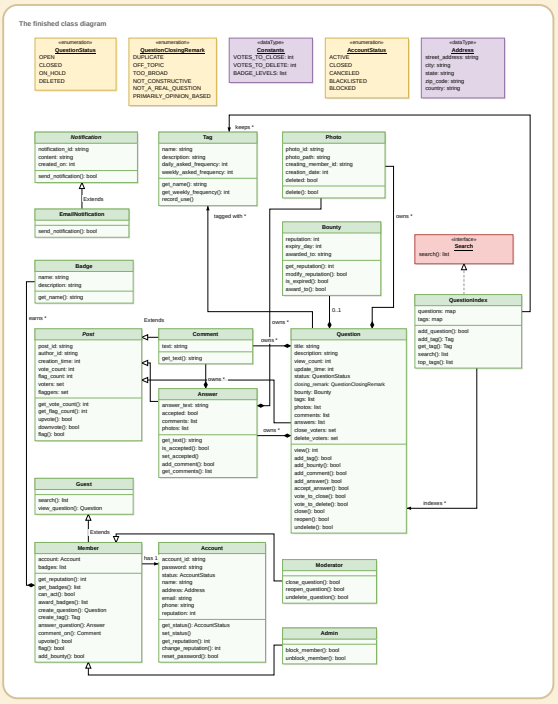
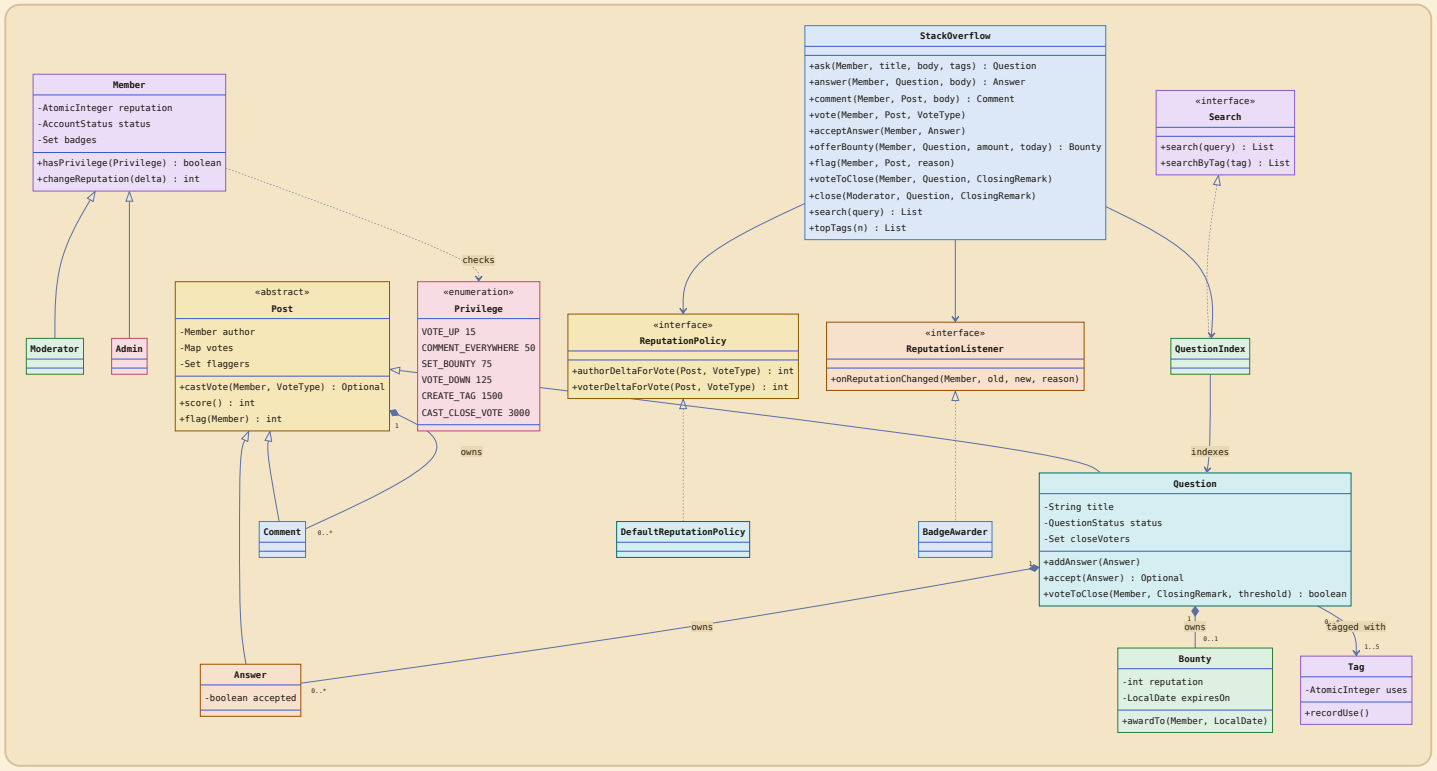


Figure — The finished Stack Overflow class diagram: enums (QuestionStatus, QuestionClosingRemark, AccountStatus), Constants, the posts, tags, badges, bounty, photo, notification, search index and the account hierarchy (course diagram)

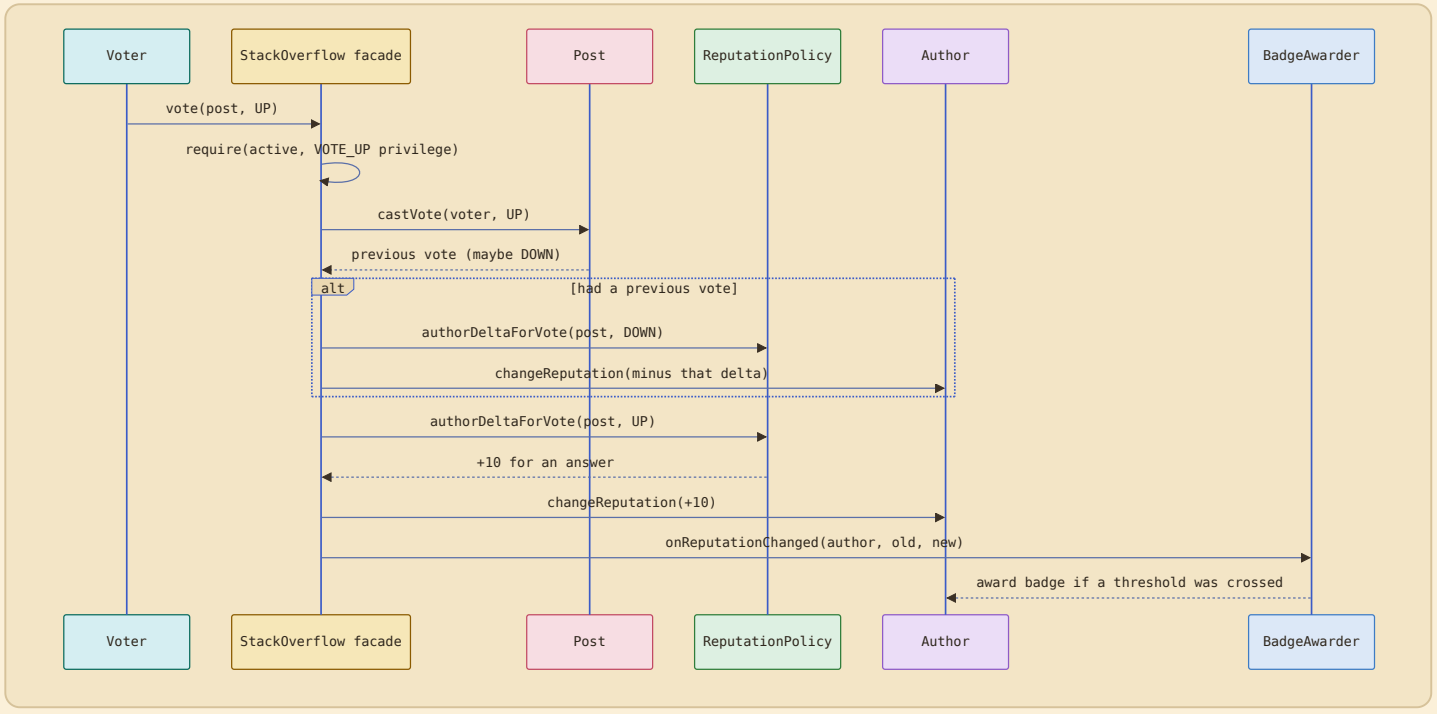
Our Java design. It keeps the course's structure and adds three things the course's "what we left out" section invites: a reputation **Strategy**, a badge **Observer**, and a data-driven **privilege table** (because on the real site privileges depend on reputation, not only on account type). A guest in our code is simply

"no `Member` object": every write operation on the facade requires a `Member` argument, so the type system still enforces requirement 1. The course also lists what it deliberately leaves out: edit history, revision review, linking duplicates, chat, and the reputation a downvoter pays. We add the downvote cost (it is one line in the policy); edit history and duplicate links are covered under Extensions below; chat stays out of scope.

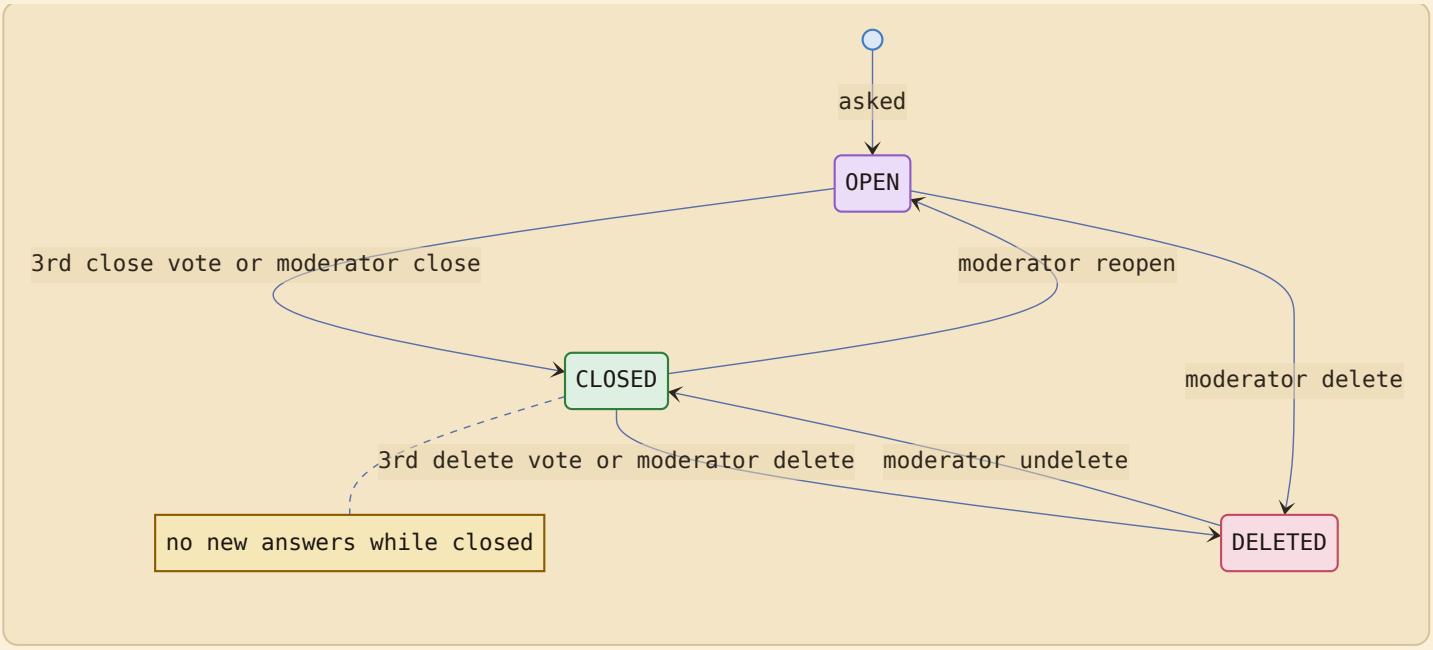


Key flows

Voting (with a changed vote). The facade checks privilege first, then asks the post to record the vote. The post returns the voter's *previous* vote, so the facade can reverse its reputation effect before applying the new one.



Question lifecycle.



Bounty. `offerBounty` checks the `SET_BOUNTY` privilege and the 50–500 range, attaches the bounty to the (open) question, and *immediately* deducts the amount from the sponsor. Later `awardBounty` (sponsor only, not to their own answer, before expiry) credits the winner.

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
`Post` base class for question, answer, comment	Inheritance (passes the substitution test), `sealed`	Voting, flagging and commenting are identical on all three	One `Post` class with a `type` field — then every method needs `if (type == …)`
Votes stored as `Map<voterId, VoteType>`	"Store who, not how many"	Blocks double votes; lets a vote be changed and its reputation reversed	An `int voteCount` — cannot tell who voted
`ReputationPolicy` interface	**Strategy**	Scoring rules change (Stack Overflow really did change them); voting code should not	Hard-coded numbers inside `vote()` — the course calls this out as a weakness
`ReputationListener` + `BadgeAwarder`	**Observer**	Badges, notifications, analytics all react to reputation without the facade knowing them	Calling `awardBadges()` directly inside `vote()`
`Privilege` enum with thresholds	Table-driven design	One place holds the rules; `hasPrivilege` is one comparison	`if (rep >= 125)` scattered across methods
Moderator-only methods take a `Moderator` parameter	Type-level permission	You cannot even call `close(…)` without a moderator object	A runtime role check that someone might forget
`QuestionIndex` separate from `Question`	Single Responsibility	Search can later be swapped for Elasticsearch (a popular search-engine server) behind `Search`	Static `Question.search()`
`Tag` counts its own uses	Maintain an aggregate at write time	"Top tags" becomes a sort of a small collection	Scanning all questions on every request
`StackOverflow` facade	**Facade**	One entry point that validates, delegates and logs	Letting the UI call `Question`, `Member`, `Bounty` directly
Pattern matching `switch` over the sealed `Post`	Exhaustive dispatch	The compiler forces us to handle each post kind in the policy	`instanceof` chains that silently miss a new type

Java implementation

Package `lld.ch18.stackoverflow`. Read the files in this order: the enums and privilege table (`Types.java`), accounts, posts, reputation Strategy/Observer, the search index, then the facade and the demo.

```java
// File: lld/ch18/stackoverflow/Types.java
package lld.ch18.stackoverflow;

/** Lifecycle of a question. ON_HOLD is the older name Stack Overflow used for a freshly closed question. */
enum QuestionStatus { OPEN, ON_HOLD, CLOSED, DELETED }

/** Why a question was closed. */
enum ClosingRemark {
    DUPLICATE, OFF_TOPIC, TOO_BROAD, NOT_CONSTRUCTIVE, NOT_A_REAL_QUESTION, PRIMARILY_OPINION_BASED
}

enum AccountStatus { ACTIVE, CLOSED, CANCELED, BLACKLISTED, BLOCKED }

enum VoteType { UP, DOWN }

/**
 * Reputation needed to unlock an action. The numbers follow the public
 * privileges page of stackoverflow.com, so "reputation unlocks privileges"
 * is data here, not if-statements spread over the code.
 */
enum Privilege {
    ASK_OR_ANSWER(1),
    VOTE_UP(15),
    FLAG(15),
    COMMENT_EVERYWHERE(50),
    SET_BOUNTY(75),
    VOTE_DOWN(125),
    CREATE_TAG(1500),
    EDIT_ANY_POST(2000),
    CAST_CLOSE_VOTE(3000),
    CAST_DELETE_VOTE(10000);

    final int minReputation;

    Privilege(int minReputation) { this.minReputation = minReputation; }
}

/** A badge is just a name and a description; members own the badges they earn. */
record Badge(String name, String description) {}
```

```java
// File: lld/ch18/stackoverflow/Accounts.java
package lld.ch18.stackoverflow;

import java.util.Collections;
import java.util.LinkedHashSet;
import java.util.Set;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicLong;

/**
 * A registered member. A guest has no Member object at all: every write
 * operation on the StackOverflow facade takes a Member, so "guests can only
 * search and view" is enforced by the method signatures.
 */
class Member {
    private static final AtomicLong IDS = new AtomicLong();

    private final long id = IDS.incrementAndGet();
    private final String name;
    private final String email;
    private final AtomicInteger reputation = new AtomicInteger(1);   // every account starts at 1
    private final Set<Badge> badges = Collections.synchronizedSet(new LinkedHashSet<>());
    private volatile AccountStatus status = AccountStatus.ACTIVE;

    Member(String name, String email) {
        this.name = name;
        this.email = email;
    }

    long id() { return id; }
    String name() { return name; }
    String email() { return email; }
    int reputation() { return reputation.get(); }
    AccountStatus status() { return status; }
    void setStatus(AccountStatus status) { this.status = status; }

    /** Applies a delta atomically; reputation never drops below 1 (the site's floor). Returns the new value. */
    int changeReputation(int delta) {
        return reputation.updateAndGet(r -> Math.max(1, r + delta));
    }

    /** Demo helper: pretend the member earned reputation before the demo started. */
    void seedReputation(int value) { reputation.set(value); }

    boolean hasPrivilege(Privilege p) { return reputation() >= p.minReputation; }

    boolean addBadge(Badge b) { return badges.add(b); }
    Set<Badge> badges() { synchronized (badges) { return Set.copyOf(badges); } }

    @Override public String toString() { return name + "(" + reputation() + ")"; }
}

/** Moderators can do everything a member can, whatever their reputation, plus close/reopen/delete/undelete any
question. */
class Moderator extends Member {
    Moderator(String name, String email) { super(name, email); }
    @Override boolean hasPrivilege(Privilege p) { return true; }
}

/** Admins block and unblock members. */
class Admin extends Member {
    Admin(String name, String email) { super(name, email); }
    @Override boolean hasPrivilege(Privilege p) { return true; }
}
```

```java
// File: lld/ch18/stackoverflow/Posts.java
package lld.ch18.stackoverflow;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.Set;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicLong;

/**
 * Behaviour shared by questions, answers and comments: votes, flags and comments.
 * Votes are stored per voter (not as a bare counter) so one member cannot vote twice
 * and a vote can be changed or reversed.
 */
abstract sealed class Post permits Question, Answer, Comment {
    private static final AtomicLong IDS = new AtomicLong(100);

    private final long id = IDS.incrementAndGet();
    private final Member author;
    private volatile String body;
    private final Map<Long, VoteType> votes = new HashMap<>();
    private final Set<Long> flaggers = new HashSet<>();
    private final List<Comment> comments = new ArrayList<>();

    Post(Member author, String body) {
        this.author = author;
        this.body = body;
    }

    long id() { return id; }
    Member author() { return author; }
    String body() { return body; }
    void edit(String newBody) { this.body = newBody; }

    /** Records or changes a vote and returns the voter's previous vote, if any. */
    synchronized Optional<VoteType> castVote(Member voter, VoteType type) {
        if (voter.id() == author.id()) throw new IllegalStateException("you cannot vote on your own post");
        VoteType previous = votes.put(voter.id(), type);
        if (previous == type) throw new IllegalStateException(voter.name() + " already voted " + type + " on this
post");
        return Optional.ofNullable(previous);
    }

    synchronized int score() {
        return votes.values().stream().mapToInt(v -> v == VoteType.UP ? 1 : -1).sum();
    }

    /** Returns the number of distinct flaggers after this flag. */
    synchronized int flag(Member by) {
        if (!flaggers.add(by.id())) throw new IllegalStateException(by.name() + " already flagged this post");
        return flaggers.size();
    }

    synchronized void addComment(Comment c) { comments.add(c); }
    synchronized List<Comment> comments() { return List.copyOf(comments); }

    abstract String label();
}

final class Question extends Post {
    private final String title;
    private final Set<Tag> tags;
    private final List<Answer> answers = new ArrayList<>();
    private final Set<Long> closeVoters = new HashSet<>();
    private final Set<Long> deleteVoters = new HashSet<>();
    private final AtomicInteger views = new AtomicInteger();
    private QuestionStatus status = QuestionStatus.OPEN;
```

```
    private ClosingRemark closingRemark;
    private Bounty bounty;

    Question(Member author, String title, String body, Set<Tag> tags) {
        super(author, body);
        this.title = title;
        this.tags = new LinkedHashSet<>(tags);
        tags.forEach(Tag::recordUse);            // a tag counts its own uses: "top tags" becomes a read
    }

    String title() { return title; }
    Set<Tag> tags() { return Set.copyOf(tags); }
    int view() { return views.incrementAndGet(); }
    int views() { return views.get(); }

    synchronized QuestionStatus status() { return status; }
    synchronized Optional<ClosingRemark> closingRemark() { return Optional.ofNullable(closingRemark); }
    synchronized List<Answer> answers() { return List.copyOf(answers); }
    synchronized Optional<Bounty> bounty() { return Optional.ofNullable(bounty); }

    synchronized void addAnswer(Answer a) {
        if (status != QuestionStatus.OPEN) throw new IllegalStateException("question is " + status + ", no new
answers");
        answers.add(a);
    }

    /** Marks one answer accepted and returns the previously accepted one (only one answer can win). */
    synchronized Optional<Answer> accept(Answer chosen) {
        if (!answers.contains(chosen)) throw new IllegalArgumentException("answer does not belong to this question");
        Optional<Answer> previous = answers.stream().filter(Answer::isAccepted).findFirst();
        previous.ifPresent(p -> p.setAccepted(false));
        chosen.setAccepted(true);
        return previous;
    }

    synchronized void attachBounty(Bounty b) {
        if (status != QuestionStatus.OPEN) throw new IllegalStateException("bounties need an open question");
        if (bounty != null && !bounty.isSettled()) throw new IllegalStateException("question already has an active
bounty");
        bounty = b;
    }

    /** One close vote per member; returns true when this vote closed the question. */
    synchronized boolean voteToClose(Member voter, ClosingRemark remark, int threshold) {
        if (status != QuestionStatus.OPEN) throw new IllegalStateException("question is already " + status);
        if (!closeVoters.add(voter.id())) throw new IllegalStateException(voter.name() + " already voted to close");
        if (closeVoters.size() >= threshold) { close(remark); return true; }
        return false;
    }

    synchronized boolean voteToDelete(Member voter, int threshold) {
        if (status != QuestionStatus.CLOSED) throw new IllegalStateException("only closed questions can be deleted by
vote");
        if (!deleteVoters.add(voter.id())) throw new IllegalStateException(voter.name() + " already voted to delete");
        if (deleteVoters.size() >= threshold) { status = QuestionStatus.DELETED; return true; }
        return false;
    }

    synchronized void close(ClosingRemark remark) { status = QuestionStatus.CLOSED; closingRemark = remark; }
    synchronized void reopen() { status = QuestionStatus.OPEN; closingRemark = null; closeVoters.clear(); }
    synchronized void delete() { status = QuestionStatus.DELETED; }
    synchronized void undelete() { status = QuestionStatus.CLOSED; deleteVoters.clear(); }

    @Override String label() { return "question #" + id() + " \"" + title + "\""; }
}

final class Answer extends Post {
    private final Question question;
    private volatile boolean accepted;

    Answer(Member author, Question question, String body) {
        super(author, body);
        this.question = question;
    }
```

```
    Question question() { return question; }
    boolean isAccepted() { return accepted; }
    void setAccepted(boolean accepted) { this.accepted = accepted; }

    @Override String label() { return "answer #" + id() + " by " + author().name(); }
}

final class Comment extends Post {
    private final Post parent;

    Comment(Member author, Post parent, String body) {
        super(author, body);
        this.parent = parent;
    }

    Post parent() { return parent; }
    @Override String label() { return "comment #" + id(); }
}

/** A tag is shared by many questions and outlives each of them (association, not composition). */
final class Tag {
    private final String name;
    private final String description;
    private final AtomicInteger uses = new AtomicInteger();

    Tag(String name, String description) { this.name = name; this.description = description; }

    String name() { return name; }
    String description() { return description; }
    void recordUse() { uses.incrementAndGet(); }
    int uses() { return uses.get(); }
    @Override public String toString() { return "[" + name + "]"; }
}

/** Reputation the asker has staked; it leaves the asker's account when the bounty starts. */
final class Bounty {
    private final Member sponsor;
    private final int reputation;
    private final LocalDate expiresOn;
    private Member awardedTo;
    private boolean expired;

    Bounty(Member sponsor, int reputation, LocalDate expiresOn) {
        this.sponsor = sponsor;
        this.reputation = reputation;
        this.expiresOn = expiresOn;
    }

    Member sponsor() { return sponsor; }
    int reputation() { return reputation; }
    LocalDate expiresOn() { return expiresOn; }
    synchronized boolean isSettled() { return awardedTo != null || expired; }

    synchronized void awardTo(Member winner, LocalDate today) {
        if (isSettled()) throw new IllegalStateException("bounty already settled");
        if (today.isAfter(expiresOn)) throw new IllegalStateException("bounty expired on " + expiresOn);
        awardedTo = winner;
    }

    synchronized void expire() { expired = true; }
}
```

```java
// File: lld/ch18/stackoverflow/Reputation.java
package lld.ch18.stackoverflow;

import java.util.List;

/**
 * Strategy: the scoring rules live behind an interface, so voting code does not
 * know the numbers. Changing a rule (e.g. the 2019 move of question upvotes
 * from +5 to +10) is a new
 * policy, not an edit to the voting code (Open/Closed Principle).
 */
interface ReputationPolicy {
    int authorDeltaForVote(Post post, VoteType type);
    int voterDeltaForVote(Post post, VoteType type);
    int acceptedAnswerBonus();
    int acceptorBonus();
}

/** Today's stackoverflow.com rules: +10 for any upvote, -2 to the author per downvote, -1 to whoever downvotes an answer. */
final class DefaultReputationPolicy implements ReputationPolicy {
    @Override public int authorDeltaForVote(Post post, VoteType type) {
        return switch (post) {
            case Question q -> type == VoteType.UP ? 10 : -2;   // +5 before November 2019
            case Answer a -> type == VoteType.UP ? 10 : -2;
            case Comment c -> 0;                        // comment votes earn no reputation
        };
    }
    @Override public int voterDeltaForVote(Post post, VoteType type) {
        return post instanceof Answer && type == VoteType.DOWN ? -1 : 0;
    }
    @Override public int acceptedAnswerBonus() { return 15; }
    @Override public int acceptorBonus() { return 2; }
}

/** Observer: anything that wants to react to a reputation change. */
interface ReputationListener {
    void onReputationChanged(Member member, int oldValue, int newValue, String reason);
}

/** Awards reputation-level badges (course thresholds: 15, 50, 125). */
final class BadgeAwarder implements ReputationListener {
    private record Level(int reputation, Badge badge) {}

    private final List<Level> levels = List.of(
            new Level(15, new Badge("Contributor", "reached 15 reputation")),
            new Level(50, new Badge("Helpful", "reached 50 reputation")),
            new Level(125, new Badge("Trusted", "reached 125 reputation")));
    private final Notifier notifier;

    BadgeAwarder(Notifier notifier) { this.notifier = notifier; }

    @Override public void onReputationChanged(Member m, int oldValue, int newValue, String reason) {
        for (Level level : levels) {
            if (oldValue < level.reputation() && newValue >= level.reputation() && m.addBadge(level.badge())) {
                notifier.send(m, "You earned the '" + level.badge().name() + "' badge");
            }
        }
    }
}

/** Delivery channel for notifications (Strategy: email today, push tomorrow). */
interface NotificationChannel {
    void deliver(Member to, String text);
}

final class EmailChannel implements NotificationChannel {
    @Override public void deliver(Member to, String text) {
        System.out.println("    [email to " + to.email() + "] " + text);
    }
}

/** The "System" actor: sends notifications through every configured channel. */
```

```
final class Notifier {
    private final List<NotificationChannel> channels;
    Notifier(List<NotificationChannel> channels) { this.channels = List.copyOf(channels); }
    void send(Member to, String text) { channels.forEach(c -> c.deliver(to, text)); }
}
```

```
// File: lld/ch18/stackoverflow/QuestionIndex.java
package lld.ch18.stackoverflow;

import java.util.Arrays;
import java.util.Comparator;
import java.util.HashSet;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;

/** Search contract, kept separate so a full-text engine could replace the in-memory index. */
interface Search {
    List<Question> search(String query);
    List<Question> searchByTag(String tagName);
}

/**
 * An inverted index: word -> questions containing it. It refers to questions
 * but does not own them (throwing the index away deletes no question).
 */
final class QuestionIndex implements Search {
    private final Map<String, Set<Question>> byWord = new ConcurrentHashMap<>();
    private final Map<String, Set<Question>> byTag = new ConcurrentHashMap<>();

    void add(Question q) {
        for (String word : words(q.title() + " " + q.body())) {
            byWord.computeIfAbsent(word, k -> ConcurrentHashMap.newKeySet()).add(q);
        }
        for (Tag t : q.tags()) {
            byTag.computeIfAbsent(t.name(), k -> ConcurrentHashMap.newKeySet()).add(q);
        }
    }

    /** Questions containing every word of the query, best score first; deleted questions are hidden. */
    @Override public List<Question> search(String query) {
        List<String> terms = words(query);
        if (terms.isEmpty()) return List.of();
        Set<Question> result = new HashSet<>(byWord.getOrDefault(terms.get(0), Set.of()));
        for (String t : terms.subList(1, terms.size())) result.retainAll(byWord.getOrDefault(t, Set.of()));
        return rank(result);
    }

    @Override public List<Question> searchByTag(String tagName) {
        return rank(byTag.getOrDefault(tagName.toLowerCase(Locale.ROOT), Set.of()));
    }

    private static List<Question> rank(Set<Question> qs) {
        return qs.stream()
                .filter(q -> q.status() != QuestionStatus.DELETED)
                .sorted(Comparator.comparingInt(Question::score).reversed().thenComparingLong(Question::id))
                .toList();
    }

    private static List<String> words(String text) {
        return Arrays.stream(text.toLowerCase(Locale.ROOT).split("[^a-z0-9#+]+"))
                .filter(w -> w.length() > 2)
                .distinct()
                .toList();
    }
}
```

```java
// File: lld/ch18/stackoverflow/StackOverflow.java
package lld.ch18.stackoverflow;

import java.time.LocalDate;
import java.util.Comparator;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.Optional;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;

/**
 * Facade: the one entry point the UI/controllers call. It checks who may do
 * what (account status + privilege), delegates state changes to the posts,
 * applies reputation through the policy, and tells listeners.
 */
public final class StackOverflow {
    static final int VOTES_TO_CLOSE = 3;
    static final int VOTES_TO_DELETE = 3;
    static final int FLAGS_FOR_MODERATOR_REVIEW = 3;
    static final int MIN_BOUNTY = 50;
    static final int MAX_BOUNTY = 500;
    static final int BOUNTY_DAYS = 7;

    private final Map<String, Tag> tags = new ConcurrentHashMap<>();
    private final Map<Long, Member> members = new ConcurrentHashMap<>();
    private final QuestionIndex index = new QuestionIndex();
    private final ReputationPolicy policy;
    private final Notifier notifier;
    private final List<ReputationListener> listeners = new CopyOnWriteArrayList<>();
    private final List<Moderator> moderators = new CopyOnWriteArrayList<>();

    StackOverflow(ReputationPolicy policy, Notifier notifier) {
        this.policy = policy;
        this.notifier = notifier;
    }

    void addReputationListener(ReputationListener l) { listeners.add(l); }

    // ---------- accounts ----------
    <M extends Member> M register(M member) {
        members.put(member.id(), member);
        if (member instanceof Moderator mod) moderators.add(mod);
        return member;
    }

    void block(Admin admin, Member target) { target.setStatus(AccountStatus.BLOCKED); log(admin.name() + " blocked " +
target.name()); }
    void unblock(Admin admin, Member target) { target.setStatus(AccountStatus.ACTIVE); log(admin.name() + " unblocked
" + target.name()); }

    // ---------- guest operations (no Member needed) ----------
    List<Question> search(String query) { return index.search(query); }
    List<Question> searchByTag(String tag) { return index.searchByTag(tag); }
    int view(Question q) { return q.view(); }

    /** Requirement 13: the most used tags; each tag counted itself, so this is a sort, not a scan of all questions.
*/
    List<Tag> topTags(int n) {
        return tags.values().stream()
                .sorted(Comparator.comparingInt(Tag::uses).reversed().thenComparing(Tag::name))
                .limit(n).toList();
    }

    // ---------- tags ----------
    Tag createTag(Member by, String name, String description) {
        require(by, Privilege.CREATE_TAG);
        return tags.computeIfAbsent(name.toLowerCase(Locale.ROOT), k -> new Tag(k, description));
    }
```

```
    // ---------- posting ----------
    Question ask(Member asker, String title, String body, List<String> tagNames) {
        require(asker, Privilege.ASK_OR_ANSWER);
        Set<Tag> resolved = new LinkedHashSet<>();
        for (String raw : tagNames) {
            String name = raw.toLowerCase(Locale.ROOT);
            Tag tag = tags.get(name);
            if (tag == null) tag = createTag(asker, name, "");    // throws if the asker may not create tags
            resolved.add(tag);
        }
        Question q = new Question(asker, title, body, resolved);
        index.add(q);
        log(asker.name() + " asked " + q.label() + " " + resolved);
        return q;
    }

    Answer answer(Member author, Question q, String body) {
        require(author, Privilege.ASK_OR_ANSWER);
        Answer a = new Answer(author, q, body);
        q.addAnswer(a);                                       // throws unless the question is OPEN
        log(author.name() + " posted " + a.label() + " on question #" + q.id());
        notifier.send(q.author(), author.name() + " answered \"" + q.title() + "\"");
        return a;
    }

    Comment comment(Member author, Post target, String body) {
        requireActive(author);
        boolean ownThread = target.author().id() == author.id()
                || (target instanceof Answer a && a.question().author().id() == author.id());
        if (!ownThread) require(author, Privilege.COMMENT_EVERYWHERE);
        Comment c = new Comment(author, target, body);
        target.addComment(c);
        log(author.name() + " commented on " + target.label() + ": \"" + body + "\"");
        if (target.author().id() != author.id()) notifier.send(target.author(), author.name() + " commented on your "
+ target.label());
        return c;
    }

    // ---------- voting & reputation ----------
    void vote(Member voter, Post post, VoteType type) {
        require(voter, type == VoteType.UP ? Privilege.VOTE_UP : Privilege.VOTE_DOWN);
        if (post instanceof Comment && type == VoteType.DOWN) throw new IllegalStateException("comments can only be
upvoted");
        Optional<VoteType> previous = post.castVote(voter, type);
        log(voter.name() + " voted " + type + " on " + post.label() + " -> score " + post.score());
        previous.ifPresent(p -> {                             // undo the old vote's effect first
            adjust(post.author(), -policy.authorDeltaForVote(post, p), "vote changed on " + post.label());
            adjust(voter, -policy.voterDeltaForVote(post, p), "vote changed");
        });
        adjust(post.author(), policy.authorDeltaForVote(post, type), type + " on " + post.label());
        adjust(voter, policy.voterDeltaForVote(post, type), "downvoted an answer");
    }

    void acceptAnswer(Member asker, Answer answer) {
        requireActive(asker);
        Question q = answer.question();
        if (q.author().id() != asker.id()) throw new IllegalStateException("only the asker can accept an answer");
        Optional<Answer> previous = q.accept(answer);
        log(asker.name() + " accepted " + answer.label());
        previous.ifPresent(p -> adjust(p.author(), -policy.acceptedAnswerBonus(), "acceptance moved away"));
        adjust(answer.author(), policy.acceptedAnswerBonus(), "answer accepted");
        if (previous.isEmpty() && answer.author().id() != asker.id()) adjust(asker, policy.acceptorBonus(), "accepted
an answer");
    }

    // ---------- bounties ----------
    Bounty offerBounty(Member sponsor, Question q, int amount, LocalDate today) {
        require(sponsor, Privilege.SET_BOUNTY);
        if (amount < MIN_BOUNTY || amount > MAX_BOUNTY) throw new IllegalArgumentException("bounty must be " +
MIN_BOUNTY + ".." + MAX_BOUNTY);
        if (sponsor.reputation() - amount < 1) throw new IllegalStateException("not enough reputation to stake " +
amount);
        Bounty b = new Bounty(sponsor, amount, today.plusDays(BOUNTY_DAYS));
        q.attachBounty(b);
```

```
        adjust(sponsor, -amount, "bounty staked");          // staked reputation leaves the sponsor now
        log(sponsor.name() + " offered a +" + amount + " bounty on question #" + q.id() + " (expires " + b.expiresOn()
+ ")");
        return b;
    }

    void awardBounty(Member sponsor, Answer winner, LocalDate today) {
        Bounty b = winner.question().bounty().orElseThrow(() -> new IllegalStateException("no bounty on this
question"));
        if (b.sponsor().id() != sponsor.id()) throw new IllegalStateException("only the sponsor awards the bounty");
        if (winner.author().id() == sponsor.id()) throw new IllegalStateException("you cannot award your own answer");
        b.awardTo(winner.author(), today);
        adjust(winner.author(), b.reputation(), "bounty awarded");
        log(sponsor.name() + " awarded the bounty to " + winner.label());
    }

    // ---------- moderation ----------
    void flag(Member by, Post post, String reason) {
        require(by, Privilege.FLAG);
        int count = post.flag(by);
        log(by.name() + " flagged " + post.label() + " (" + reason + "), flags=" + count);
        if (count == FLAGS_FOR_MODERATOR_REVIEW) {
            moderators.forEach(m -> notifier.send(m, post.label() + " needs moderator attention"));
        }
    }

    void voteToClose(Member voter, Question q, ClosingRemark remark) {
        require(voter, Privilege.CAST_CLOSE_VOTE);
        boolean closed = q.voteToClose(voter, remark, VOTES_TO_CLOSE);
        log(voter.name() + " voted to close question #" + q.id() + " as " + remark + (closed ? " -> CLOSED" : ""));
    }

    void voteToDelete(Member voter, Question q) {
        require(voter, Privilege.CAST_DELETE_VOTE);
        boolean deleted = q.voteToDelete(voter, VOTES_TO_DELETE);
        log(voter.name() + " voted to delete question #" + q.id() + (deleted ? " -> DELETED" : ""));
    }

    // Moderator-only operations: the parameter type is the permission check.
    void close(Moderator m, Question q, ClosingRemark remark) { q.close(remark); log(m.name() + " (mod) closed
question #" + q.id() + " as " + remark); }
    void reopen(Moderator m, Question q) { q.reopen(); log(m.name() + " (mod) reopened question #" + q.id()); }
    void delete(Moderator m, Question q) { q.delete(); log(m.name() + " (mod) deleted question #" + q.id()); }
    void undelete(Moderator m, Question q) { q.undelete(); log(m.name() + " (mod) undeleted question #" + q.id() + " -
> " + q.status()); }

    // ---------- helpers ----------
    private void adjust(Member m, int delta, String reason) {
        if (delta == 0) return;
        int before = m.reputation();
        int after = m.changeReputation(delta);
        log(String.format("    rep %-6s %+d -> %d (%s)", m.name(), delta, after, reason));
        listeners.forEach(l -> l.onReputationChanged(m, before, after, reason));
    }

    private static void requireActive(Member m) {
        if (m.status() != AccountStatus.ACTIVE) throw new IllegalStateException(m.name() + " is " + m.status());
    }

    private static void require(Member m, Privilege p) {
        requireActive(m);
        if (!m.hasPrivilege(p)) {
            throw new IllegalStateException(m.name() + " needs " + p.minReputation + " reputation for " + p + " (has "
+ m.reputation() + ")");
        }
    }

    private static void log(String s) { System.out.println(s); }
}
```

```java
// File: lld/ch18/stackoverflow/Main.java
package lld.ch18.stackoverflow;

import java.time.LocalDate;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        Notifier notifier = new Notifier(List.of(new EmailChannel()));
        StackOverflow so = new StackOverflow(new DefaultReputationPolicy(), notifier);
        so.addReputationListener(new BadgeAwarder(notifier));

        Member alice = so.register(new Member("alice", "alice@example.com"));
        Member bob = so.register(new Member("bob", "bob@example.com"));
        Member carol = so.register(new Member("carol", "carol@example.com"));
        Member erin = so.register(new Member("erin", "erin@example.com"));
        Member dave = so.register(new Member("dave", "dave@example.com"));
        Moderator mia = so.register(new Moderator("mia", "mia@example.com"));
        Admin root = so.register(new Admin("root", "root@example.com"));
        alice.seedReputation(200);
        bob.seedReputation(3500);
        carol.seedReputation(3200);
        erin.seedReputation(3100);
        LocalDate today = LocalDate.of(2026, 9, 28);

        section("1. Tags and asking");
        so.createTag(bob, "java", "The Java language");
        so.createTag(bob, "collections", "java.util collections");
        attempt(() -> so.ask(alice, "How does HashMap handle collisions?", "What happens when two keys land in the
same bucket?",
                List.of("java", "hashmap")));
        Question q1 = so.ask(alice, "How does HashMap handle collisions?", "What happens when two keys land in the
same bucket?",
                List.of("java", "collections"));

        section("2. Answers, votes, reputation");
        Answer bobs = so.answer(bob, q1, "Entries share a bucket as a linked list, turned into a tree when the bin
grows past 8.");
        Answer carols = so.answer(carol, q1, "It uses open addressing.");
        attempt(() -> so.vote(dave, q1, VoteType.UP));               // dave has 1 rep
        attempt(() -> so.vote(alice, q1, VoteType.UP));              // own post
        so.vote(bob, q1, VoteType.UP);
        so.vote(carol, bobs, VoteType.UP);
        so.vote(erin, carols, VoteType.DOWN);
        attempt(() -> so.vote(erin, carols, VoteType.DOWN));         // double vote
        so.vote(erin, carols, VoteType.UP);                          // changing a vote reverses the old effect
        so.comment(erin, carols, "HashMap uses chaining, not open addressing.");
        attempt(() -> so.comment(dave, carols, "+1"));               // needs 50 rep to comment everywhere
        so.acceptAnswer(alice, bobs);

        section("3. Bounty");
        Question q2 = so.ask(alice, "Why is ConcurrentHashMap faster than synchronizedMap?", "Under many threads.",
List.of("java"));
        so.offerBounty(alice, q2, 50, today);
        Answer carolsQ2 = so.answer(carol, q2, "Reads take no lock and writes lock only one bin.");
        so.awardBounty(alice, carolsQ2, today.plusDays(3));

        section("4. Badges for a new member");
        Question q3 = so.ask(dave, "Is Java pass-by-value?", "Objects seem to be passed by reference.",
List.of("java"));
        so.vote(bob, q3, VoteType.UP);
        so.vote(carol, q3, VoteType.UP);
        so.vote(erin, q3, VoteType.UP);
        System.out.println("dave's badges: " + dave.badges().stream().map(Badge::name).toList());

        section("5. Moderation");
        so.voteToClose(bob, q3, ClosingRemark.DUPLICATE);
        so.voteToClose(carol, q3, ClosingRemark.DUPLICATE);
        so.voteToClose(erin, q3, ClosingRemark.DUPLICATE);
        attempt(() -> so.answer(bob, q3, "Yes, always by value."));
        so.reopen(mia, q3);
        Comment rude = so.comment(dave, q3, "you all are wrong!!!");
        so.flag(bob, rude, "rude");
```

```
        so.flag(carol, rude, "rude");
        so.flag(erin, rude, "rude");
        so.delete(mia, q3);
        so.undelete(mia, q3);
        so.block(root, dave);
        attempt(() -> so.ask(dave, "Another one", "text", List.of("java")));

        section("6. Guest search and top tags");
        so.search("hashmap collisions").forEach(q -> System.out.println("found: " + q.label() + " score=" + q.score()
                + " answers=" + q.answers().size() + " status=" + q.status()));
        so.searchByTag("java").forEach(q -> System.out.println("[java] " + q.label()));
        System.out.println("top tags: " + so.topTags(2).stream().map(t -> t.name() + "x" + t.uses()).toList());
        System.out.println("final reputation: " + List.of(alice, bob, carol, erin, dave));
    }

    private static void section(String title) { System.out.println("\n== " + title + " =="); }

    private static void attempt(Runnable action) {
        try { action.run(); }
        catch (RuntimeException e) { System.out.println("  REJECTED: " + e.getMessage()); }
    }
}
```

Output:

```
== 1. Tags and asking ==
  REJECTED: alice needs 1500 reputation for CREATE_TAG (has 200)
alice asked question #101 "How does HashMap handle collisions?" [[java], [collections]]

== 2. Answers, votes, reputation ==
bob posted answer #102 by bob on question #101
    [email to alice@example.com] bob answered "How does HashMap handle collisions?"
carol posted answer #103 by carol on question #101
    [email to alice@example.com] carol answered "How does HashMap handle collisions?"
  REJECTED: dave needs 15 reputation for VOTE_UP (has 1)
  REJECTED: you cannot vote on your own post
bob voted UP on question #101 "How does HashMap handle collisions?" -> score 1
    rep alice  +10 -> 210 (UP on question #101 "How does HashMap handle collisions?")
carol voted UP on answer #102 by bob -> score 1
    rep bob    +10 -> 3510 (UP on answer #102 by bob)
erin voted DOWN on answer #103 by carol -> score -1
    rep carol  -2 -> 3198 (DOWN on answer #103 by carol)
    rep erin   -1 -> 3099 (downvoted an answer)
  REJECTED: erin already voted DOWN on this post
erin voted UP on answer #103 by carol -> score 1
    rep carol  +2 -> 3200 (vote changed on answer #103 by carol)
    rep erin   +1 -> 3100 (vote changed)
    rep carol  +10 -> 3210 (UP on answer #103 by carol)
erin commented on answer #103 by carol: "HashMap uses chaining, not open addressing."
    [email to carol@example.com] erin commented on your answer #103 by carol
  REJECTED: dave needs 50 reputation for COMMENT_EVERYWHERE (has 1)
alice accepted answer #102 by bob
    rep bob    +15 -> 3525 (answer accepted)
    rep alice  +2 -> 212 (accepted an answer)

== 3. Bounty ==
alice asked question #105 "Why is ConcurrentHashMap faster than synchronizedMap?" [[java]]
    rep alice  -50 -> 162 (bounty staked)
alice offered a +50 bounty on question #105 (expires 2026-10-05)
carol posted answer #106 by carol on question #105
    [email to alice@example.com] carol answered "Why is ConcurrentHashMap faster than synchronizedMap?"
    rep carol  +50 -> 3260 (bounty awarded)
alice awarded the bounty to answer #106 by carol

== 4. Badges for a new member ==
dave asked question #107 "Is Java pass-by-value?" [[java]]
bob voted UP on question #107 "Is Java pass-by-value?" -> score 1
    rep dave   +10 -> 11 (UP on question #107 "Is Java pass-by-value?")
carol voted UP on question #107 "Is Java pass-by-value?" -> score 2
    rep dave   +10 -> 21 (UP on question #107 "Is Java pass-by-value?")
    [email to dave@example.com] You earned the 'Contributor' badge
erin voted UP on question #107 "Is Java pass-by-value?" -> score 3
    rep dave   +10 -> 31 (UP on question #107 "Is Java pass-by-value?")
dave's badges: [Contributor]

== 5. Moderation ==
bob voted to close question #107 as DUPLICATE
carol voted to close question #107 as DUPLICATE
erin voted to close question #107 as DUPLICATE -> CLOSED
  REJECTED: question is CLOSED, no new answers
mia (mod) reopened question #107
dave commented on question #107 "Is Java pass-by-value?": "you all are wrong!!!"
bob flagged comment #109 (rude), flags=1
carol flagged comment #109 (rude), flags=2
erin flagged comment #109 (rude), flags=3
    [email to mia@example.com] comment #109 needs moderator attention
mia (mod) deleted question #107
mia (mod) undeleted question #107 -> CLOSED
root blocked dave
  REJECTED: dave is BLOCKED

== 6. Guest search and top tags ==
found: question #101 "How does HashMap handle collisions?" score=1 answers=2 status=OPEN
[java] question #107 "Is Java pass-by-value?"
[java] question #101 "How does HashMap handle collisions?"
[java] question #105 "Why is ConcurrentHashMap faster than synchronizedMap?"
```

```
top tags: [javax3, collectionsx1]
final reputation: [alice(162), bob(3525), carol(3260), erin(3100), dave(31)]
```

Walking through the important parts

- `Post.castVote` is `synchronized` (Java lets only one thread at a time run a synchronized method on the same object; others wait their turn) and uses `votes.put(...)`, which returns the previous vote. That single call both records the vote and tells us whether this is a *change* (so the facade can undo the old reputation effect). Voting the same way twice throws.
- `DefaultReputationPolicy` uses a Java 21 **pattern-matching switch** (a `switch` that picks a branch by the *type* of the value and hands you that typed value) over the sealed `Post`. Because `Post` permits exactly `Question`, `Answer` and `Comment`, the compiler rejects the switch if a case is missing.
- `Member.changeReputation` uses `AtomicInteger.updateAndGet` (an `AtomicInteger` is a number whose read-change-write happens as one indivisible step) so concurrent changes never lose an update, and it enforces the site's floor of 1.
- `BadgeAwarder` fires only when a threshold is *crossed* (`oldValue < level && newValue >= level`), so members who already had high reputation are not showered with badges, and each badge is added once (`Set.add` returns `false` for duplicates).
- `ask()` shows the activity diagram's decision "does the tag exist? has enough reputation?": in the output, Alice (200 rep) cannot create the new tag `hashmap`, so her first attempt is rejected.
- In section 2 of the output, Erin downvotes Carol's answer (Carol −2, Erin −1), then changes her vote to up: the old effect is reversed (+2, +1) and the new one applied (+10).
- `voteToClose` closes the question on the third vote; answering a closed question is then refused; a moderator reopens it.

Concurrency & edge cases

- **Two votes at the same time on one post.** `castVote`, `score` and `flag` are `synchronized` on the post, so the vote map is never corrupted and each voter is counted once. Reputation uses `AtomicInteger`, so two simultaneous reputation changes to the same author both apply.
- **Reputation updates are not in the same lock as the vote.** A reader could briefly see the new vote before the author's reputation changes. That is acceptable (**eventual consistency** for a score: the copies may disagree for a moment but will match shortly — see System Design Fundamentals). In a database you would put both in one **transaction** (a group of changes that either all succeed or all fail together), and enforce "one vote per member per post" with a **unique constraint** (a database rule that refuses a second row with the same values) on `(post_id, voter_id)`.
- **Reputation floor.** Because reputation cannot go below 1, reversing a vote on a member who was at the floor is not perfectly symmetric. The real site has the same quirk; if exact reversal matters, keep a ledger (an append-only list of every change, in order) of reputation events and recompute.
- **Accepting a different answer.** `Question.accept` returns the previously accepted answer so the facade removes its +15; the asker's +2 is given only once.
- **Self-actions.** Voting on your own post and awarding your own bounty are rejected.
- **Closed and deleted questions.** Closed: no new answers. Deleted: hidden from search results.
- **Blocked accounts.** Every write goes through `requireActive`, so a blocked member can do nothing.
- **Bounty expiry.** `awardTo` refuses after the expiry date. On the real site, bounty reputation is never refunded; if the sponsor does not award it, the system may auto-award half to a qualifying answer.

Extensions & follow-up questions

- **"How do you stop one person voting twice?"** — The per-post map of voter ids (the course's key follow-up). In a database: a unique key on `(post_id, voter_id)`.
- **"Add edit history."** — Store each version as an immutable `Revision` (author, time, body) in a list on the post; editing appends a revision. This is close to the **Memento** idea: the past states are kept so they can be shown or rolled back.
- **"Daily reputation cap."** — The real site caps reputation from votes at 200 per day. Wrap the policy: a `CappedReputationPolicy` decorator (the **Decorator** pattern: an object that wraps another object with the same interface and adds behaviour before or after passing the call on) that tracks today's total per member and trims the delta. No voting code changes.

- **"Mark as duplicate and link."** — Add `duplicateOf: Question` to `Question` and a `ClosingRemark.DUPLICATE`; closing as duplicate sets the link.
- **"Review queues."** — Flags already reach moderators at a threshold; turn that into a `ReviewQueue` that holds flagged posts, with reviewers voting to keep or delete.
- **"Hot questions."** — A ranking Strategy over recent votes, answers and views with time decay (older activity counts for less and less), just like the feed rankers in 18.3 and 18.4.
- **"Tag frequency per day and week."** — The course's `Tag` keeps daily and weekly counters. Keep a counter per time bucket and roll the buckets over with a scheduled job (a task the system runs automatically at set times, e.g. every midnight).

Real-world example

On stackoverflow.com, the numbers in this design are real product rules. The help centre lists privileges by reputation (vote up at 15, comment everywhere at 50, set bounties at 75, vote down at 125, create tags at 1,500, edit at 2,000, cast close and reopen votes at 3,000, moderator tools at 10,000). In November 2019 the company raised question upvotes from +5 to +10 ("rewarding the question askers") — a change that, in our design, touches only the `ReputationPolicy` (one number, or a new policy class) and nothing in the voting code. After a 2019 experiment the number of close votes needed was lowered from 5 to 3, which in our code is one constant.

In the interview

How to spend 30–45 minutes

Minutes	What to do
0–5	Clarify actors and the reputation rules; write the requirement list
5–10	Use cases per actor; say "guest has no posting method"
10–20	Class diagram: `Post` base with three subclasses, `Member` hierarchy, `Tag`, `Bounty`, `QuestionIndex`, facade
20–30	Walk through `vote()` and `acceptAnswer()` including reversal; question state machine
30–40	Code the core: `Post.castVote`, the policy, the facade's `vote`
40–45	Concurrency (per-post lock, atomic reputation, DB unique key) and extensions

30-second answer: "Questions, answers and comments share voting and flagging, so they extend one `Post` that stores votes per voter — that stops double voting and lets me reverse a changed vote. Reputation rules are a Strategy so they can change without touching voting; badges are an Observer on reputation changes. Privileges come from a reputation table; moderator actions take a `Moderator` parameter so a guest or member cannot call them. Search is a separate inverted index, and tags count their own uses for 'top tags'."

Follow-ups with answers

- *Why not store `voteCount` only?* — It cannot prevent duplicates or support vote changes; derive the count from the voter map (or cache it next to a votes table).
- *Where does reputation get awarded — in `upvote`?* — Not inside the post. The facade asks the policy; that keeps posts ignorant of scoring (the course names this as the improvement to make).
- *How would you scale search?* — Keep the `Search` interface and replace `QuestionIndex` with a search engine fed by events when questions are created or edited.

Self-check

- **Q:** Which requirement is enforced by the *shape* of the classes rather than by an `if`? **A:** Guests cannot post or vote: guests have no `Member` object, and every write method needs one (course version: `Guest` has no `createQuestion`).
- **Q:** Why is `Post` abstract and sealed? **A:** Abstract because a bare "post" is never created; sealed so switches over post kinds are checked for completeness.

- **Q:** What happens to reputation when Erin changes a downvote into an upvote on an answer? **A:** The old effect is reversed (author +2, Erin +1) and the new one applied (author +10).
- **Q:** Why does `Tag` count its own uses? **A:** So "most used tags" is a sort of the tags, not a scan of all questions.
- **Q:** What makes a comment different from an answer in voting? **A:** Comments can only be upvoted and earn no reputation (policy returns 0).

> **TL;DR**

Stack Overflow = `Post` base (votes per voter, flags, comments) + `Question`/`Answer`/`Comment` + reputation **Strategy** + badge **Observer** + reputation **privilege table** + moderator-typed methods + inverted-index search + a facade that checks, delegates and scores.

18.2 Design Cricinfo

Figure — A ball hitting the stumps under the ESPNcricinfo logo, the cover image of this case study (course diagram)

Cricinfo (today ESPNcricinfo) is a sports news site only for cricket. It gives **live ball-by-ball commentary** of matches, keeps a database of every historic match, and publishes news and articles.

Before we start, a two-minute cricket primer, because this design is graded on whether you model the *rules*, not just the nouns:

Term	Meaning
Delivery / ball	One throw ("bowl") of the ball by the **bowler** to the **batter** on strike
Over	Six **legal** deliveries by one bowler. A **wide** (too far from the batter) or a **no ball** (illegal delivery, e.g. overstepping the line) gives the batting side a penalty run and must be bowled again, so it does not count toward the six
Striker / non-striker	Two batters are in at once. The striker faces the ball; they change ends whenever they complete an odd number of runs (1, 3, 5), and again after every over
Runs	Off the bat (credited to the batter) or **extras** — wides, no balls, **byes** and **leg byes** (credited to the team, not the batter)
Wicket / dismissal	A batter is out: bowled, caught, LBW (leg before wicket), stumped, hit wicket, run out, obstructing the field. A **run out is not credited to the bowler**
Innings	One team's turn to bat, ending when 10 batters are out ("all out"), the overs run out, or the target is reached
Formats	**T20** (20 overs per side), **ODI** (One Day International, 50 overs per side), **Test** (no over limit, two innings per side, over several days)
Squad / playing eleven	Before a tournament each team names a **squad**; for every match it chooses exactly **eleven** of those players

Problem & clarifying questions

Question to ask	Answer we assume
Which formats?	T20, ODI and Test
Who enters the data?	An **Admin** (scorer) records every ball; a **Commentator** adds text for each ball
Do we follow real cricket rules?	Yes — the international rules, at least: six legal balls per over, extras, who gets credit for a wicket, strike rotation, bowler limits
Live updates?	Yes — the scorecard and commentary update after every ball (push to subscribers, i.e. readers who asked for updates)
Stats?	Per player, match and tournament, plus global questions such as "highest wicket taker of all time"
Tournaments?	Yes, with squads and a points table
Out of scope?	The course leaves these out on purpose: rain rules (Duckworth–Lewis–Stern, a formula that resets the target when rain shortens a match), the follow-on and declarations in a Test, powerplays and fielding restrictions, free hits after a no ball, telling a leg-side wide from an off-side one, partnerships, net run rate (a team's scoring rate minus its opponents', used to break ties in a points table), and the site's news articles. (Our code adds a simple `declare()`; several others appear under Extensions)

Requirements

Functional (the course's eight, in our words, plus the rules they imply)

1. Track every team that plays cricket, and every match it plays.
2. Show live ball-by-ball commentary.
3. Follow international cricket rules.
4. Each team announces a squad for each tournament.
5. For each match, each team announces its playing eleven from the tournament squad.
6. Keep statistics for players, for matches and for tournaments.
7. Answer global statistics questions, such as "which bowler has taken the most wickets ever?" or "which batter has made the most Test hundreds?"
8. Track ODI, Test and T20 matches.

Non-functional

- **Accuracy:** stats must match the ball-by-ball record exactly (a scorer's correction must fix every derived number).
- **Low-latency live updates** (**latency** = the delay between something happening and the reader seeing it)**:** thousands to millions of readers see a new ball within seconds.
- **Consistency of recording:** two scorers must not interleave balls in the same match.
- **Extensibility:** a new stat question, a new format, or a new subscriber (push alerts) should be a new class.

Use cases

Two actors: the **Admin** adds or modifies players, teams, tournaments and matches and records the ball-by-ball details; the **Commentator** writes the text commentary for each ball.

Top use cases: add/modify teams and players; add tournaments and matches; add ball; add stadium, umpire and referee; add/update stats (some generated by the system); add commentary.

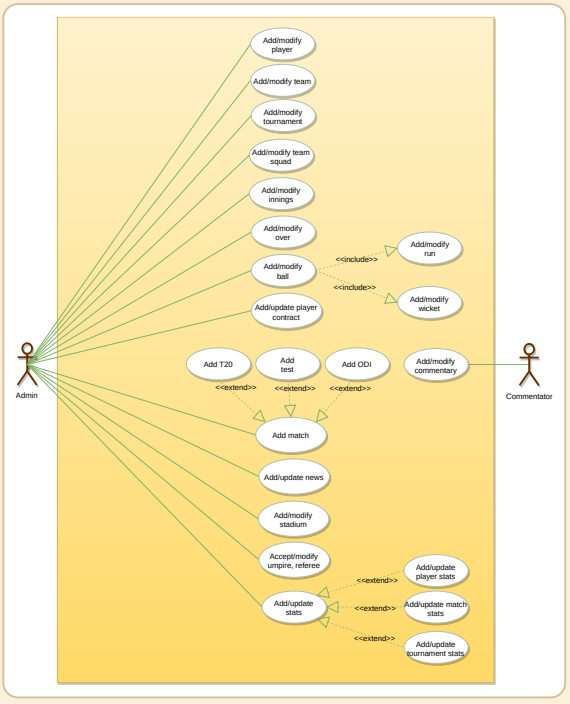


Figure — Cricinfo use cases: the Admin adds tournaments, matches (extended by Test, T20 and ODI), teams, squads, players, stadiums, umpires, innings, overs, balls (which include runs and wickets) and stats; the Commentator adds commentary (course diagram)

In the figure above, "Add match" is *extended* by the three formats, and "Add ball" *includes* adding its runs and wicket — a ball is recorded together with what happened on it.

Activity — record a ball. The scorer selects *Add Ball*; picks the ball type (normal, wide, no ball, or wicket-taking); if a wicket fell, adds wicket details; if runs were scored, adds a run record for each run type; adds commentary; saves the ball.

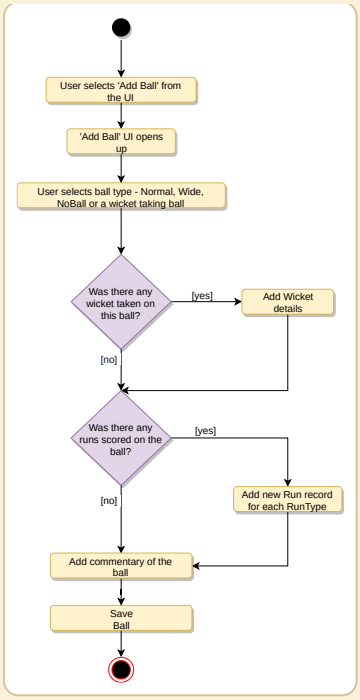


Figure — Activity diagram "record a ball of an over": choose ball type, add wicket details if any, add run records if any, add commentary, save (course diagram)

Core entities

Class	Responsibility
`Person` (record)	A human: name, country. Roles *hold* a person
`Player`	A role holding a `Person` (the course's `Player` also keeps a basic profile and contract details)
`MatchOfficial` (abstract) → `Umpire`, `Referee`, `Commentator`	Shared "assign to a match" behaviour; each says its role
`Team`	Every player the team has ever picked
`TournamentSquad`	Who a team may use in one tournament (must be team members)
`Playing11`	Exactly eleven, all from the squad, in batting order
`Tournament`	Owns squads, lists matches, keeps the points table
`BallOutcome` (record)	What happened: runs off the bat, extra type and runs, boundary?, wicket type, which batter was out, commentary text. Validates the laws (no bowled off a no ball)
`Ball` (record)	A recorded delivery: over, delivery number, bowler, striker, outcome, wicket
`Wicket` (record) + `WicketType`	Who was out, how, off whose bowling; `creditedToBowler()`
`Over`	One bowler's six **legal** deliveries; runs conceded; maiden?
`Innings`	Owns overs; decides striker, end of over, new batter, bowler limits, and when the innings ends
`Match` (abstract) → `T20`, `ODI`, `Test`	Holds everything; formats say only overs per innings and innings per side
`Result` (sealed) → `Win`, `Tie`, `Draw`	The outcome
`MatchObserver` → `CommentaryFeed`, `LiveScorecard`	Subscribers to ball / over / innings / match events
`PlayerStats`, `StatsService`	Numbers derived from balls
`StatQuery` (abstract) → `MostRuns`, `MostHundreds`, `MostWickets`, `MostSixes`	Template Method: one question per class

Class diagram

The course builds this design in **six stages**.

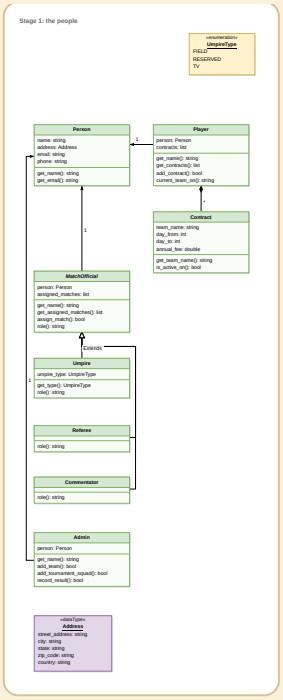


Figure — Stage 1: `Person`, `MatchOfficial` with `Umpire`, `Referee` and `Commentator`, `Admin`, and `Player` holding a `Person` (course diagram)

Stage 1 — the people. A player *holds* a person instead of extending one: one real human can play today and commentate after retiring, and composition lets them take both roles without copying. Umpires, referees and commentators all do one thing in common — get assigned to matches — so that lives once in `MatchOfficial`.

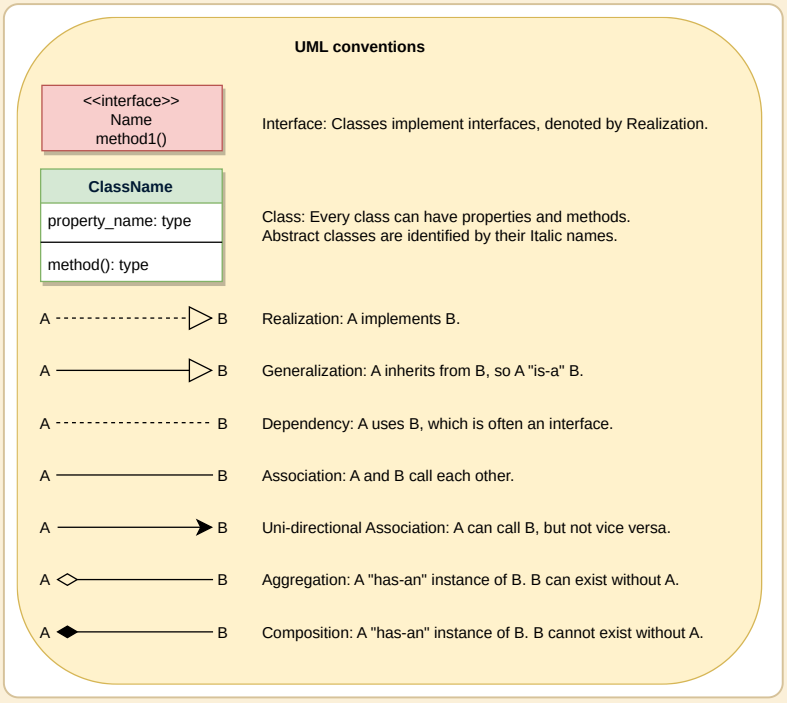


Figure — UML legend for reading the arrows in these diagrams (course diagram)

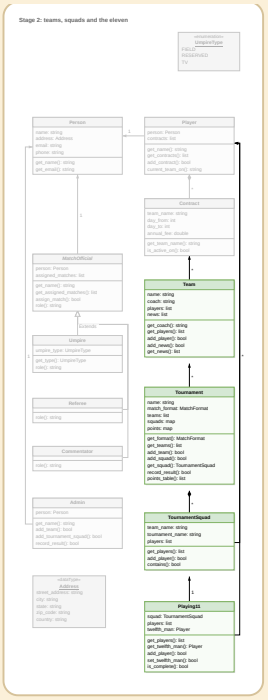


Figure — Stage 2: `Team`, `TournamentSquad`, `Playing11` and `Tournament`; only the tournament owns anything (its squads) (course diagram)

Stage 2 — teams, squads and the eleven. Requirements 4 and 5 look like one idea but are three nested sets: the team (everyone ever picked) ⊇ the squad (one tournament) ⊇ the eleven (one match). `Playing11` keeps a reference to its squad, which lets it reject anyone outside the squad, and it also rejects a twelfth pick. Requirement 5 thus lives in code rather than in a comment.

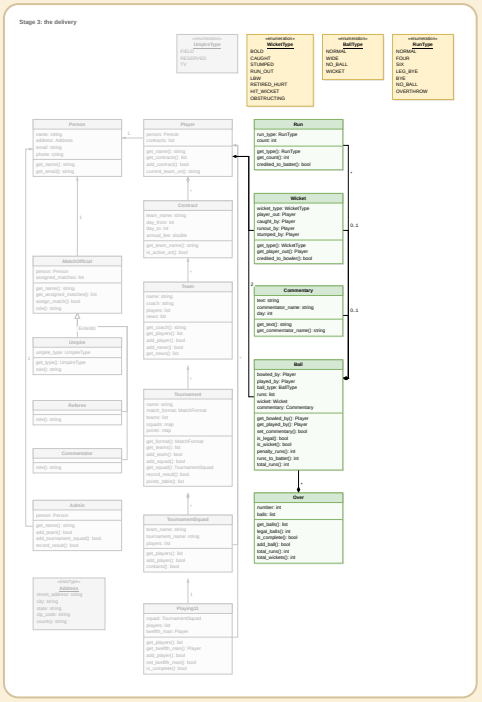


Figure — Stage 3: enums `WicketType`, `BallType`, `RunType`, and classes `Run`, `Wicket`, `Commentary`, `Ball` (owning its runs, wicket and commentary) and `Over` (course diagram)

Stage 3 — the delivery. This stage decides whether the design understands cricket. Three rules hide in it: (1) an over is six *legal* deliveries, so `Over.isComplete` counts only legal balls, and an over with two wides takes eight deliveries; (2) a run out is credited to the fielding side, *not the bowler* — get this wrong and "highest wicket taker" is silently wrong for years; (3) byes and leg byes belong to the team,

not the batter, so each run carries a type that says who gets the credit. `Ball` owns its runs, wicket and commentary (composition); it only *refers* to the bowler and batter.

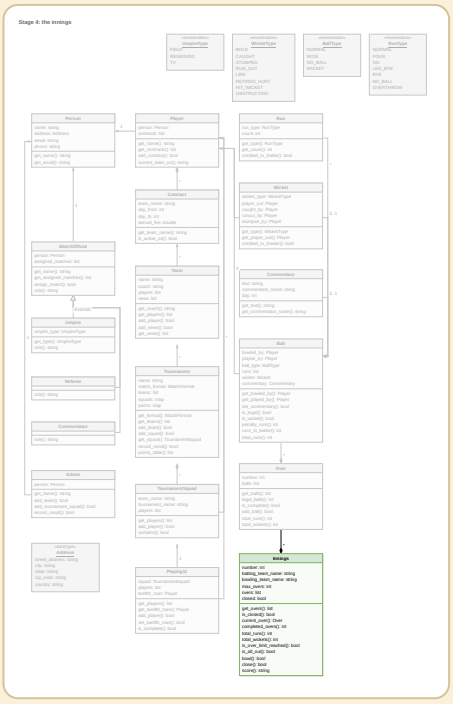


Figure — Stage 4: `Innings` owns its overs and prints the score (course diagram)

Stage 4 — the innings. Because the innings owns the overs, it alone decides when an over is finished and a new one starts; if that rule lived anywhere else, it would soon exist in two slightly different copies. Its `score` uses scorecard notation: runs/wickets, then full overs and the legal balls of the over in progress.

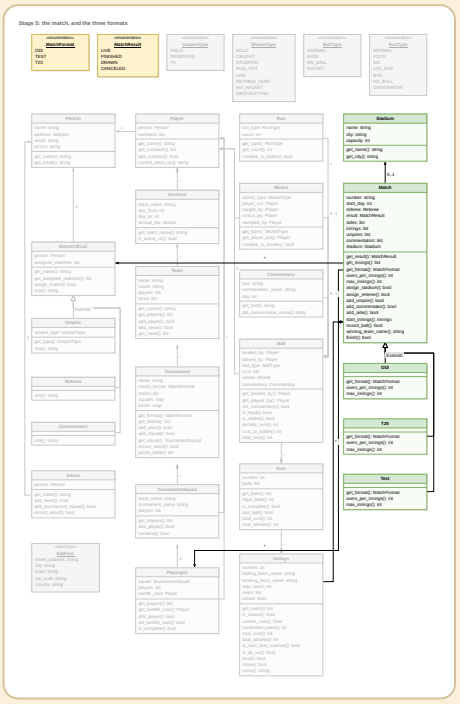


Figure — Stage 5: `Stadium`, abstract `Match` owning its innings, and the `ODI`, `T20` and `Test` subclasses (course diagram)

Stage 5 — the match and the formats. Before creating three classes, check what really changes between the formats. Starting an innings, recording a ball and deciding the winner are the same. Only two numbers

differ: overs per innings and innings per match. So `Match` is abstract and holds everything; each format overrides two short methods. A Test returns zero overs (meaning "no limit") and four innings in total.

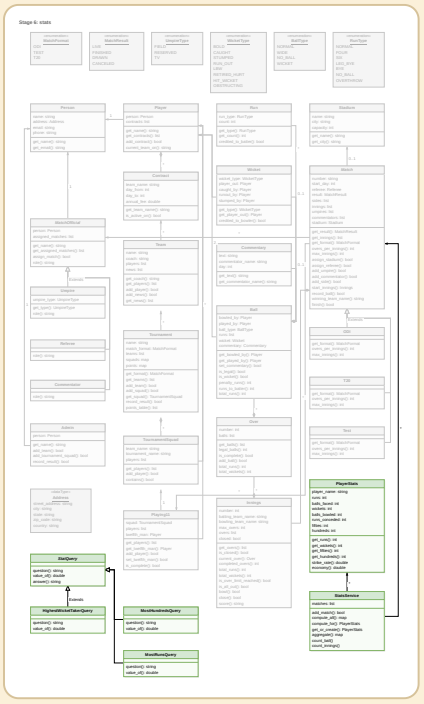


Figure — Stage 6: `PlayerStats`, `StatsService` and the `StatQuery` hierarchy (course diagram)

> **Key term — Template Method:** a design pattern where a base class writes the fixed outline of an algorithm in one method and leaves one or more steps as abstract methods for subclasses to fill in. Every subclass reuses the outline and changes only its own step. In this design `StatQuery.answer()` is the outline (loop over players, keep the best) and each subclass supplies only `valueOf()` — "runs", "wickets" and so on.

Stage 6 — stats. Stats are **worked out from the balls**, not stored beside them. Keep a separate counter and it will drift from the ball record as soon as a scorer fixes a mistake, and after that nobody can say which number is correct. Fifties and hundreds are per-innings milestones, so they are counted after each

innings (`countInnings`) next to the per-ball counting (`countBall`). Each global question is one small `StatQuery` subclass.

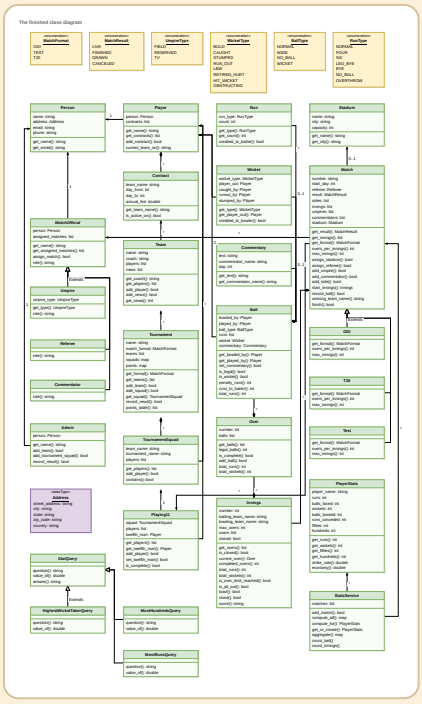
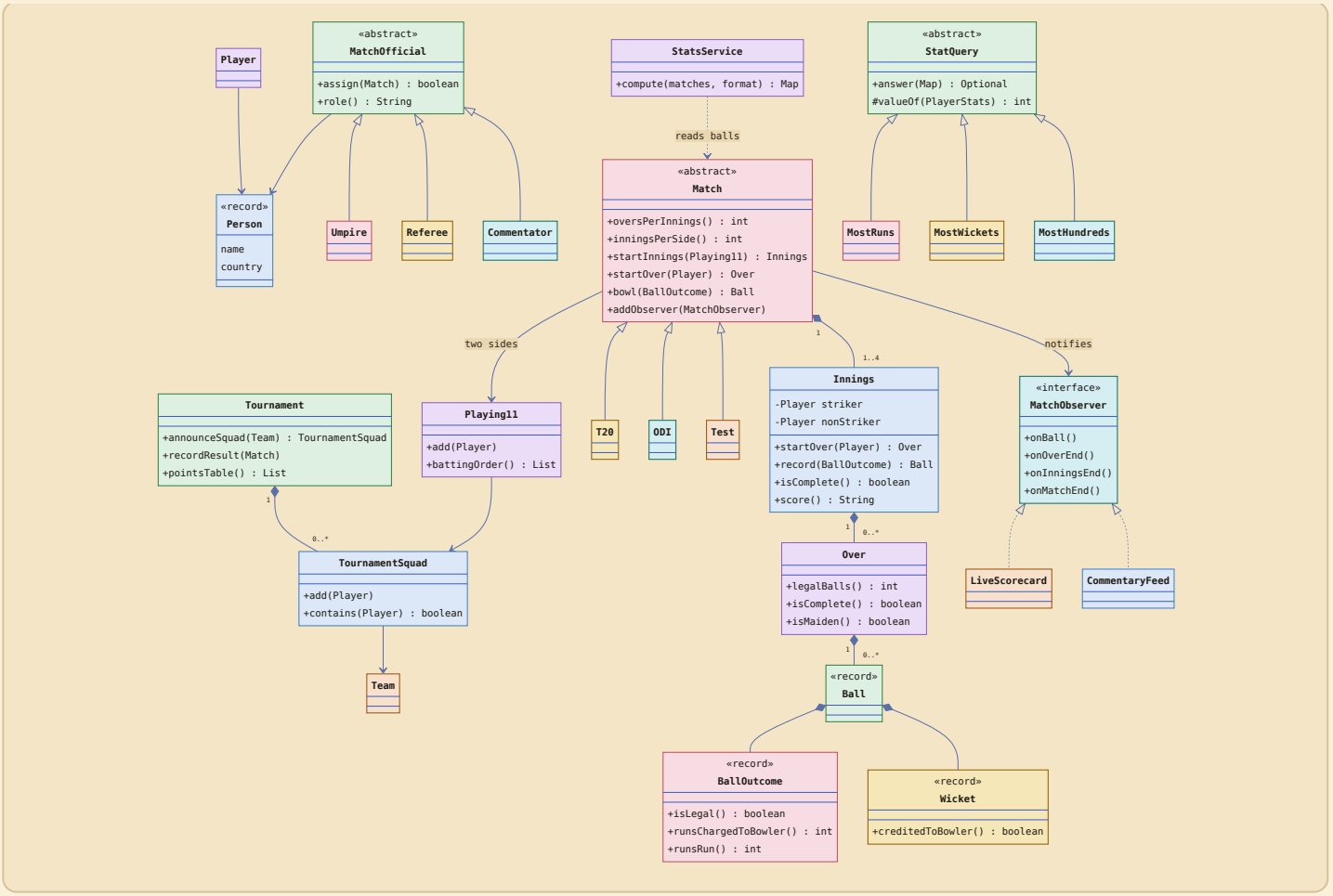


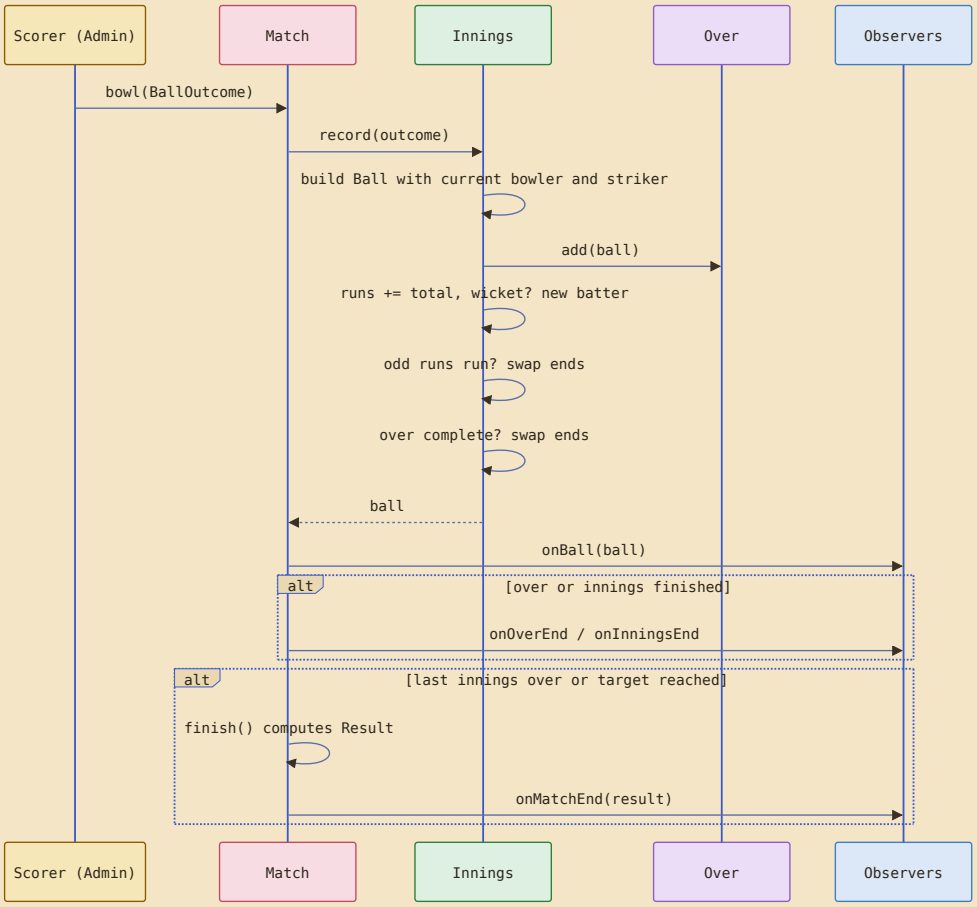
Figure — The finished Cricinfo class diagram, with every class from the six stages in full colour (course diagram)

Our Java design keeps all six stages and adds what the course flags as missing: **strike rotation** (who is on strike), bowler rules (not two overs in a row; 4 overs per bowler in a T20, 10 in an ODI), the laws about which dismissals are possible off a wide or no ball, a result calculation, and **Observers** for live updates. Instead of separate `Run` objects per run type, a `BallOutcome` record carries bat runs, extra type and extra runs — the same information in one immutable value (an object that can never be changed after it is created).

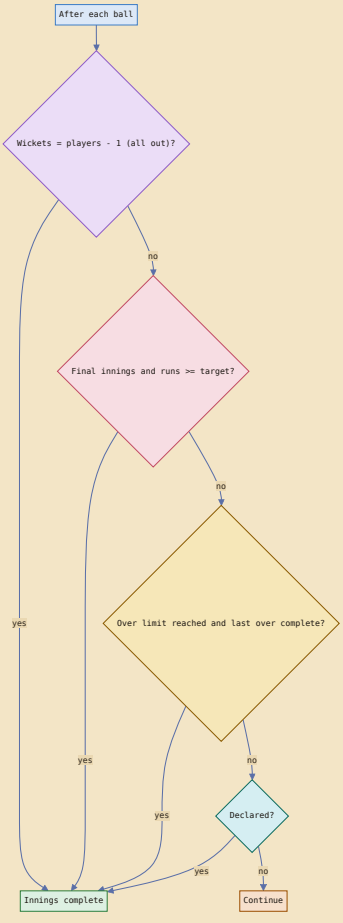


Key flows

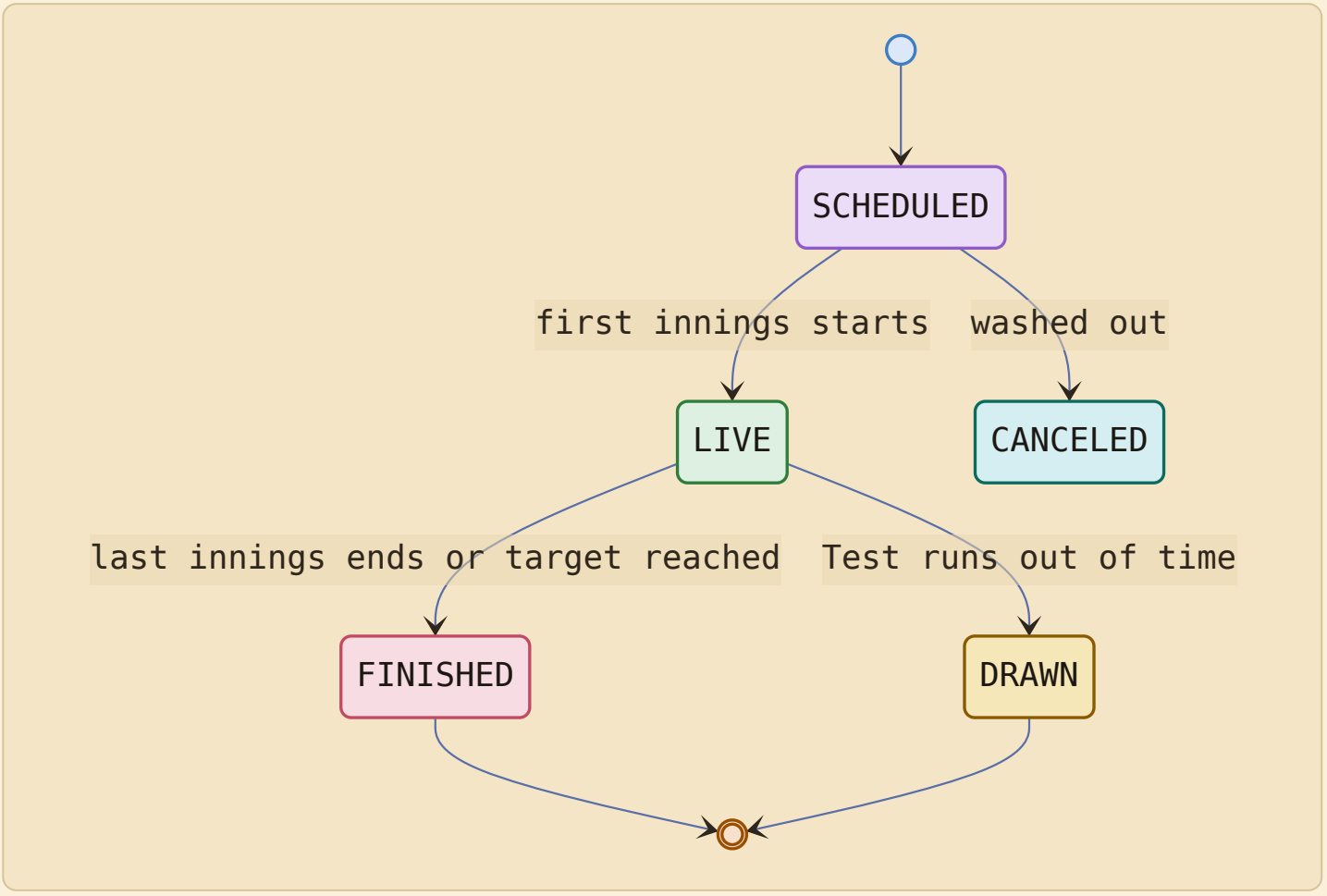
Recording one ball (the scorer's "Add Ball"):



Innings end conditions (checked by `Innings.isComplete`):



Match status:



Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
`Player` and `MatchOfficial` hold a `Person`	Composition over inheritance	One human, many roles over time	`Player extends Person` — a player who becomes a commentator must be copied
`Match` abstract; `T20` / `ODI` / `Test` override two numbers	Template-style base class (the course notes it is not a named pattern here)	Only what differs is overridden	Three full `Match` copies
`StatQuery.answer()` final, `valueOf()` abstract	**Template Method**	A new statistic is one tiny subclass; the search loop is written once (Open/Closed: open to new features, closed to editing old code)	A `switch (questionType)` inside the service
Stats derived from balls	Single source of truth	Corrections fix every number automatically	Stored counters that drift; a cache is fine if it is rebuildable
`Innings` owns the over rule and strike	Information Expert (GRASP — guidelines for deciding which class gets a job)	The class with the data makes the decision	Scorer UI deciding when an over ends
`BallOutcome` validates the laws in its constructor	Fail fast, immutable value	Impossible deliveries (bowled off a no ball) never enter the record	Checking later in the stats code
`MatchObserver` with default methods	**Observer**	Commentary feed, live scorecard, push alerts subscribe without the match knowing them	The match printing to the console itself
`Result` is a sealed interface of records	Algebraic data type (a value that is exactly one of a closed set of shapes)	`switch` over results is exhaustive (see `Tournament.recordResult`)	Strings like "Team A won"
`Match.bowl` is `synchronized`	Monitor lock (Java's built-in per-object lock) per match	Two scorers cannot interleave balls	Unsynchronised updates that corrupt the over

Java implementation

Package `lld.ch18.cricinfo`. The demo builds two teams, a tournament, squads and elevens (showing both refusals), then plays a scripted first over (a wide and a no ball make it **eight deliveries**; a bowled that counts for the bowler; a run out that does not), and simulates the rest with a fixed random seed (the starting number for the random generator; the same seed always gives the same "random" sequence) so the output is repeatable. An ODI is simulated too, so the global stat queries have data from two formats.

```java
// File: lld/ch18/cricinfo/People.java
package lld.ch18.cricinfo;

import java.util.ArrayList;
import java.util.List;

/** A human being. Roles (player, umpire, commentator) HOLD a person instead of extending it. */
record Person(String name, String country) {}

final class Player {
    private final Person person;
    Player(Person person) { this.person = person; }
    String name() { return person.name(); }
    Person person() { return person; }
    @Override public String toString() { return name(); }
}

enum UmpireType { FIELD, RESERVED, TV }

/** Umpires, referees and commentators share one behaviour: being assigned to matches. */
abstract class MatchOfficial {
    private final Person person;
    private final List<Match> assignedMatches = new ArrayList<>();

    MatchOfficial(Person person) { this.person = person; }

    String name() { return person.name(); }
    synchronized boolean assign(Match m) { return !assignedMatches.contains(m) && assignedMatches.add(m); }
    synchronized List<Match> assignedMatches() { return List.copyOf(assignedMatches); }
    abstract String role();
    @Override public String toString() { return role() + " " + name(); }
}

final class Umpire extends MatchOfficial {
    private final UmpireType type;
    Umpire(Person p, UmpireType type) { super(p); this.type = type; }
    @Override String role() { return type + " umpire"; }
}

final class Referee extends MatchOfficial {
    Referee(Person p) { super(p); }
    @Override String role() { return "referee"; }
}

final class Commentator extends MatchOfficial {
    Commentator(Person p) { super(p); }
    @Override String role() { return "commentator"; }
}
```

```java
// File: lld/ch18/cricinfo/Teams.java
package lld.ch18.cricinfo;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.LinkedHashMap;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

/** Everyone the team has ever picked. */
final class Team {
    private final String name;
    private final Set<Player> players = new LinkedHashSet<>();

    Team(String name) { this.name = name; }
    String name() { return name; }
    void addPlayer(Player p) { players.add(p); }
    List<Player> players() { return List.copyOf(players); }   // keeps insertion order
    @Override public String toString() { return name; }
}

/** Who a team may use in ONE tournament: a subset of the team. */
final class TournamentSquad {
    private final Team team;
    private final Set<Player> players = new LinkedHashSet<>();

    TournamentSquad(Team team) { this.team = team; }
    Team team() { return team; }
    void add(Player p) {
        if (!team.players().contains(p)) throw new IllegalArgumentException(p + " does not play for " + team);
        players.add(p);
    }
    boolean contains(Player p) { return players.contains(p); }
}

/** Who takes the field in ONE match: exactly eleven, all from the squad. The class enforces the rule. */
final class Playing11 {
    static final int SIZE = 11;
    private final TournamentSquad squad;
    private final List<Player> players = new ArrayList<>();   // batting order

    Playing11(TournamentSquad squad) { this.squad = squad; }

    void add(Player p) {
        if (!squad.contains(p)) throw new IllegalArgumentException(p + " is not in the " + squad.team() + " squad");
        if (players.size() == SIZE) throw new IllegalStateException("a playing eleven cannot have a twelfth player ("
+ p + ")");
        if (players.contains(p)) throw new IllegalArgumentException(p + " already picked");
        players.add(p);
    }

    Team team() { return squad.team(); }
    List<Player> battingOrder() { return List.copyOf(players); }
    boolean isComplete() { return players.size() == SIZE; }
    boolean contains(Player p) { return players.contains(p); }
}

/** A tournament owns its squads and keeps a points table (2 for a win, 1 each for a tie or no result). */
final class Tournament {
    private final String name;
    private final Map<Team, TournamentSquad> squads = new LinkedHashMap<>();
    private final Map<Team, Integer> points = new LinkedHashMap<>();
    private final List<Match> matches = new ArrayList<>();

    Tournament(String name) { this.name = name; }
    String name() { return name; }

    TournamentSquad announceSquad(Team team) {
        TournamentSquad s = new TournamentSquad(team);
        squads.put(team, s);
        points.putIfAbsent(team, 0);
```

```
        return s;
    }

    void addMatch(Match m) { matches.add(m); }
    List<Match> matches() { return List.copyOf(matches); }

    void recordResult(Match m) {
        Result r = m.result().orElseThrow(() -> new IllegalStateException("match not finished"));
        switch (r) {
            case Result.Win w -> points.merge(w.winner(), 2, Integer::sum);
            case Result.Tie t -> { points.merge(m.teamA().team(), 1, Integer::sum); points.merge(m.teamB().team(), 1,
Integer::sum); }
            case Result.Draw d -> { }                    // Tests are not part of a points table here
        }
    }

    List<Map.Entry<Team, Integer>> pointsTable() {
        return points.entrySet().stream()
                .sorted(Map.Entry.<Team, Integer>comparingByValue(Comparator.reverseOrder()))
                .toList();
    }
}
```

```java
// File: lld/ch18/cricinfo/Delivery.java
package lld.ch18.cricinfo;

import java.util.ArrayList;
import java.util.List;

enum Extra { NONE, WIDE, NO_BALL, BYE, LEG_BYE }

enum WicketType {
    BOWLED, CAUGHT, LBW, STUMPED, HIT_WICKET, RUN_OUT, OBSTRUCTING_THE_FIELD;

    /** A run out (or obstructing the field) is a wicket for the fielding side, NOT for the bowler. */
    boolean creditedToBowler() { return this != RUN_OUT && this != OBSTRUCTING_THE_FIELD; }
}

/**
 * What happened on one delivery, as typed by the scorer. Who bowled and who
 * faced is filled in by the Innings, which knows the state of play.
 *
 * @param batRuns    runs off the bat (credited to the batter)
 * @param extra      kind of extra, if any
 * @param extraRuns  runs credited to the team as extras (a wide or no ball is at least 1)
 * @param boundary   true when batRuns came from a 4 or 6 hit to the rope (batters do not cross)
 * @param wicket     type of dismissal, or null
 * @param nonStrikerOut for a run out: true when the batter at the bowler's end is out
 */
record BallOutcome(int batRuns, Extra extra, int extraRuns, boolean boundary,
                   WicketType wicket, boolean nonStrikerOut, String commentary) {

    BallOutcome {
        if (batRuns < 0 || extraRuns < 0) throw new IllegalArgumentException("runs cannot be negative");
        if ((extra == Extra.WIDE || extra == Extra.NO_BALL) && extraRuns < 1)
            throw new IllegalArgumentException(extra + " carries at least one penalty run");
        // Laws of cricket: off a no ball the bowler cannot take a wicket; off a wide only stumped/hit wicket/run out.
        if (wicket != null && extra == Extra.NO_BALL && wicket.creditedToBowler())
            throw new IllegalArgumentException(wicket + " is not possible off a no ball");
        if (wicket != null && extra == Extra.WIDE && !(wicket == WicketType.STUMPED || wicket == WicketType.HIT_WICKET
                || !wicket.creditedToBowler()))
            throw new IllegalArgumentException(wicket + " is not possible off a wide");
    }

    static BallOutcome dot(String c) { return new BallOutcome(0, Extra.NONE, 0, false, null, false, c); }
    static BallOutcome runs(int n, String c) { return new BallOutcome(n, Extra.NONE, 0, false, null, false, c); }
    static BallOutcome four(String c) { return new BallOutcome(4, Extra.NONE, 0, true, null, false, c); }
    static BallOutcome six(String c) { return new BallOutcome(6, Extra.NONE, 0, true, null, false, c); }
    static BallOutcome wide(String c) { return new BallOutcome(0, Extra.WIDE, 1, false, null, false, c); }
    static BallOutcome noBall(int batRuns, String c) { return new BallOutcome(batRuns, Extra.NO_BALL, 1, batRuns >= 4,
null, false, c); }
    static BallOutcome legBye(int n, String c) { return new BallOutcome(0, Extra.LEG_BYE, n, false, null, false, c); }
    static BallOutcome bye(int n, String c) { return new BallOutcome(0, Extra.BYE, n, false, null, false, c); }
    static BallOutcome out(WicketType w, String c) { return new BallOutcome(0, Extra.NONE, 0, false, w, false, c); }
    static BallOutcome runOut(int completedRuns, boolean nonStrikerOut, String c) {
        return new BallOutcome(completedRuns, Extra.NONE, 0, false, WicketType.RUN_OUT, nonStrikerOut, c);
    }

    /** A wide or a no ball is not one of the six balls of an over. */
    boolean isLegal() { return extra != Extra.WIDE && extra != Extra.NO_BALL; }
    boolean countsAsBallFaced() { return extra != Extra.WIDE; }
    int totalRuns() { return batRuns + extraRuns; }
    /** Byes and leg byes are not the bowler's fault; wides and no balls are. */
    int runsChargedToBowler() { return batRuns + (isLegal() ? 0 : extraRuns); }

    /** Runs physically run between the wickets: an odd number means the batters swapped ends. */
    int runsRun() {
        if (boundary) return 0;
        return switch (extra) {
            case NONE -> batRuns;
            case WIDE -> extraRuns - 1;
            case NO_BALL -> batRuns;
            case BYE, LEG_BYE -> extraRuns;
        };
    }
}
```

```java
record Wicket(WicketType type, Player batter, Player bowler) {
    boolean creditedToBowler() { return type.creditedToBowler(); }
}

/** One delivery as recorded: owns its outcome, its wicket and its commentary (composition). */
record Ball(int overNumber, int deliveryInOver, Player bowler, Player striker, BallOutcome outcome, Wicket wicket) {
    /** Scorecard style: over 1, ball 3 is "0.3" (completed overs . ball); an extra shows the number of the ball it replaces. */
    String label() { return (overNumber - 1) + "." + deliveryInOver; }
}

/** An over is SIX LEGAL deliveries, not six balls. */
final class Over {
    static final int LEGAL_BALLS = 6;
    private final int number;
    private final Player bowler;
    private final List<Ball> balls = new ArrayList<>();

    Over(int number, Player bowler) { this.number = number; this.bowler = bowler; }

    int number() { return number; }
    Player bowler() { return bowler; }
    void add(Ball b) {
        if (isComplete()) throw new IllegalStateException("over " + number + " is complete");
        balls.add(b);
    }
    List<Ball> balls() { return List.copyOf(balls); }
    int legalBalls() { return (int) balls.stream().filter(b -> b.outcome().isLegal()).count(); }
    boolean isComplete() { return legalBalls() == LEGAL_BALLS; }
    int runsConceded() { return balls.stream().mapToInt(b -> b.outcome().runsChargedToBowler()).sum(); }
    boolean isMaiden() { return isComplete() && runsConceded() == 0; }
}
```

```java
// File: lld/ch18/cricinfo/Innings.java
package lld.ch18.cricinfo;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;

/**
 * One side's turn to bat. The innings owns its overs, so it is the ONE place
 * that decides when an over ends, who is on strike, and when the innings is over.
 */
final class Innings {
    private final int number;
    private final Playing11 batting;
    private final Playing11 bowling;
    private final int maxOvers;              // 0 = no limit (Test)
    private final int maxOversPerBowler;     // 0 = no limit
    private final Integer target;            // runs needed to win, only for the final innings
    private final List<Over> overs = new ArrayList<>();
    private final List<Ball> balls = new ArrayList<>();
    private final Map<Player, Integer> oversByBowler = new HashMap<>();
    private Player striker;
    private Player nonStriker;
    private int nextBatter = 2;
    private int runs;
    private int wickets;
    private boolean closed;

    Innings(int number, Playing11 batting, Playing11 bowling, int maxOvers, int maxOversPerBowler, Integer target) {
        if (!batting.isComplete() || !bowling.isComplete()) throw new IllegalStateException("both sides need eleven
players");
        this.number = number;
        this.batting = batting;
        this.bowling = bowling;
        this.maxOvers = maxOvers;
        this.maxOversPerBowler = maxOversPerBowler;
        this.target = target;
        this.striker = batting.battingOrder().get(0);
        this.nonStriker = batting.battingOrder().get(1);
    }

    int number() { return number; }
    Team battingTeam() { return batting.team(); }
    Team bowlingTeam() { return bowling.team(); }
    Playing11 battingSide() { return batting; }
    int runs() { return runs; }
    int wickets() { return wickets; }
    Optional<Integer> target() { return Optional.ofNullable(target); }
    List<Ball> balls() { return List.copyOf(balls); }
    List<Over> overs() { return List.copyOf(overs); }
    Player striker() { return striker; }
    Player nonStriker() { return nonStriker; }

    Over startOver(Player bowler) {
        if (isComplete()) throw new IllegalStateException("innings is complete");
        Over current = currentOver();
        if (current != null && !current.isComplete()) throw new IllegalStateException("over " + current.number() + "
is still in progress");
        if (!bowling.contains(bowler)) throw new IllegalArgumentException(bowler + " is not in the fielding eleven");
        if (current != null && current.bowler().equals(bowler))
            throw new IllegalStateException(bowler + " cannot bowl two overs in a row");
        int bowled = oversByBowler.getOrDefault(bowler, 0);
        if (maxOversPerBowler > 0 && bowled >= maxOversPerBowler)
            throw new IllegalStateException(bowler + " has used all " + maxOversPerBowler + " overs");
        oversByBowler.put(bowler, bowled + 1);
        Over over = new Over(overs.size() + 1, bowler);
        overs.add(over);
        return over;
    }

    Ball record(BallOutcome o) {
```

```
        Over over = currentOver();
        if (over == null || over.isComplete()) throw new IllegalStateException("start a new over first");
        if (isComplete()) throw new IllegalStateException("innings is complete");

        int deliveryNo = over.legalBalls() + 1;          // a wide or no ball is re-bowled, so it shares the next
number
        Wicket wicket = null;
        if (o.wicket() != null) wicket = new Wicket(o.wicket(), o.nonStrikerOut() ? nonStriker : striker,
over.bowler());
        Ball ball = new Ball(over.number(), deliveryNo, over.bowler(), striker, o, wicket);
        over.add(ball);
        balls.add(ball);
        runs += o.totalRuns();

        if (o.runsRun() % 2 == 1) swapEnds();              // odd runs: batters crossed
        if (wicket != null) {
            wickets++;
            if (!allOut()) {
                Player newBatter = batting.battingOrder().get(nextBatter++);
                if (wicket.batter().equals(striker)) striker = newBatter; else nonStriker = newBatter;
            }
        }
        if (over.isComplete()) swapEnds();                  // ends change after every over
        return ball;
    }

    boolean isComplete() {
        if (closed || allOut()) return true;
        if (target != null && runs >= target) return true;
        Over current = currentOver();
        return maxOvers > 0 && overs.size() == maxOvers && current != null && current.isComplete();
    }

    /** A Test captain may declare; the demo also uses it to stop an unlimited innings. */
    void declare() { closed = true; }

    int wicketsInHand() { return batting.battingOrder().size() - 1 - wickets; }

    /** Scorecard style: runs/wickets (completed overs.legal balls). */
    String score() {
        return battingTeam() + " " + runs + "/" + wickets + " (" + oversText() + " ov)";
    }

    String oversText() {
        Over current = currentOver();
        if (current == null) return "0";
        int full = current.isComplete() ? overs.size() : overs.size() - 1;
        int part = current.isComplete() ? 0 : current.legalBalls();
        return part == 0 ? String.valueOf(full) : full + "." + part;
    }

    private boolean allOut() { return wickets == batting.battingOrder().size() - 1; }
    private Over currentOver() { return overs.isEmpty() ? null : overs.get(overs.size() - 1); }
    private void swapEnds() { Player t = striker; striker = nonStriker; nonStriker = t; }
}
```

```java
// File: lld/ch18/cricinfo/Match.java
package lld.ch18.cricinfo;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;
import java.util.concurrent.CopyOnWriteArrayList;

enum MatchFormat { ODI, T20, TEST }

enum MatchStatus { SCHEDULED, LIVE, FINISHED, DRAWN, CANCELED }

record Stadium(String name, String city, int capacity) {}

/** Outcome of a finished match. A sealed hierarchy lets callers switch over it exhaustively. */
sealed interface Result {
    record Win(Team winner, String margin) implements Result {
        @Override public String toString() { return winner + " won by " + margin; }
    }
    record Tie() implements Result {
        @Override public String toString() { return "Match tied"; }
    }
    record Draw() implements Result {
        @Override public String toString() { return "Match drawn"; }
    }
}

/**
 * Everything that is the same in every format lives here. The formats only
 * answer two questions: how many overs per innings, how many innings per side.
 * Observers (live scorecard, commentary feed, push alerts) subscribe to events.
 */
abstract class Match {
    private final Playing11 teamA;
    private final Playing11 teamB;
    private final Stadium stadium;
    private final LocalDate date;
    private final List<MatchOfficial> officials = new ArrayList<>();
    private final List<Innings> innings = new ArrayList<>();
    private final List<MatchObserver> observers = new CopyOnWriteArrayList<>();
    private MatchStatus status = MatchStatus.SCHEDULED;
    private Result result;

    Match(Playing11 teamA, Playing11 teamB, Stadium stadium, LocalDate date) {
        this.teamA = teamA;
        this.teamB = teamB;
        this.stadium = stadium;
        this.date = date;
    }

    abstract MatchFormat format();
    abstract int oversPerInnings();       // 0 means no limit
    abstract int inningsPerSide();
    /** ODIs allow 10 overs per bowler and T20s 4, i.e. one fifth of the innings. */
    int maxOversPerBowler() { return oversPerInnings() / 5; }

    Playing11 teamA() { return teamA; }
    Playing11 teamB() { return teamB; }
    Stadium stadium() { return stadium; }
    LocalDate date() { return date; }
    synchronized MatchStatus status() { return status; }
    synchronized Optional<Result> result() { return Optional.ofNullable(result); }
    synchronized List<Innings> innings() { return List.copyOf(innings); }

    void addObserver(MatchObserver o) { observers.add(o); }

    void assign(MatchOfficial official) {
        official.assign(this);
        officials.add(official);
    }

    synchronized Innings startInnings(Playing11 batting) {
```

```
        if (status == MatchStatus.FINISHED || status == MatchStatus.DRAWN) throw new IllegalStateException("match is
over");
        Innings current = currentInnings();
        if (current != null && !current.isComplete()) throw new IllegalStateException("current innings still in
progress");
        if (innings.size() == 2 * inningsPerSide()) throw new IllegalStateException("all innings played");
        Playing11 bowling = batting == teamA ? teamB : teamA;
        boolean finalInnings = innings.size() == 2 * inningsPerSide() - 1;
        Integer target = finalInnings ? totalFor(bowling.team()) - totalFor(batting.team()) + 1 : null;
        Innings in = new Innings(innings.size() + 1, batting, bowling, oversPerInnings(), maxOversPerBowler(),
target);
        innings.add(in);
        status = MatchStatus.LIVE;
        return in;
    }

    synchronized Over startOver(Player bowler) { return requireCurrent().startOver(bowler); }

    /** Records one delivery and fires the events; thread-safe so two scorers cannot interleave balls. */
    synchronized Ball bowl(BallOutcome outcome) {
        Innings in = requireCurrent();
        Ball ball = in.record(outcome);
        observers.forEach(o -> o.onBall(this, in, ball));
        Over over = in.overs().get(in.overs().size() - 1);
        if (over.isComplete() || in.isComplete()) observers.forEach(o -> o.onOverEnd(this, in, over));
        if (in.isComplete()) {
            observers.forEach(o -> o.onInningsEnd(this, in));
            boolean chased = in.target().map(t -> in.runs() >= t).orElse(false);
            if (innings.size() == 2 * inningsPerSide() || chased) finish(in);
        }
        return ball;
    }

    synchronized void declareDraw() {
        status = MatchStatus.DRAWN;
        result = new Result.Draw();
        observers.forEach(o -> o.onMatchEnd(this, result));
    }

    private void finish(Innings last) {
        Team chasing = last.battingTeam();
        Team other = last.bowlingTeam();
        int chasingRuns = totalFor(chasing);
        int otherRuns = totalFor(other);
        if (chasingRuns > otherRuns) result = new Result.Win(chasing, last.wicketsInHand() + " wickets");
        else if (chasingRuns < otherRuns) result = new Result.Win(other, (otherRuns - chasingRuns) + " runs");
        else result = new Result.Tie();
        status = MatchStatus.FINISHED;
        observers.forEach(o -> o.onMatchEnd(this, result));
    }

    private int totalFor(Team t) {
        return innings.stream().filter(i -> i.battingTeam() == t).mapToInt(Innings::runs).sum();
    }

    private Innings currentInnings() { return innings.isEmpty() ? null : innings.get(innings.size() - 1); }

    private Innings requireCurrent() {
        Innings in = currentInnings();
        if (in == null) throw new IllegalStateException("no innings started");
        return in;
    }

    @Override public String toString() { return format() + " " + teamA.team() + " v " + teamB.team() + " at " +
stadium.name(); }
}

final class ODI extends Match {
    ODI(Playing11 a, Playing11 b, Stadium s, LocalDate d) { super(a, b, s, d); }
    @Override MatchFormat format() { return MatchFormat.ODI; }
    @Override int oversPerInnings() { return 50; }
    @Override int inningsPerSide() { return 1; }
}
```

```
final class T20 extends Match {
    T20(Playing11 a, Playing11 b, Stadium s, LocalDate d) { super(a, b, s, d); }
    @Override MatchFormat format() { return MatchFormat.T20; }
    @Override int oversPerInnings() { return 20; }
    @Override int inningsPerSide() { return 1; }
}

final class Test extends Match {
    Test(Playing11 a, Playing11 b, Stadium s, LocalDate d) { super(a, b, s, d); }
    @Override MatchFormat format() { return MatchFormat.TEST; }
    @Override int oversPerInnings() { return 0; }    // no limit
    @Override int inningsPerSide() { return 2; }
}
```

```java
// File: lld/ch18/cricinfo/Observers.java
package lld.ch18.cricinfo;

import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/** Observer: the match publishes events; screens, feeds and alerts subscribe. */
interface MatchObserver {
    default void onBall(Match m, Innings in, Ball b) {}
    default void onOverEnd(Match m, Innings in, Over o) {}
    default void onInningsEnd(Match m, Innings in) {}
    default void onMatchEnd(Match m, Result r) {}
}

/** Ball-by-ball commentary written by a commentator (prints only the first few overs of each innings in the demo). */
final class CommentaryFeed implements MatchObserver {
    private final Commentator commentator;
    private final int oversToShow;

    CommentaryFeed(Commentator commentator, int oversToShow) {
        this.commentator = commentator;
        this.oversToShow = oversToShow;
    }

    @Override public void onBall(Match m, Innings in, Ball b) {
        if (b.overNumber() > oversToShow) return;
        String what = describe(b);
        String text = b.outcome().commentary() == null ? "" : "  [" + commentator.name() + ": " +
b.outcome().commentary() + "]";
        System.out.printf("  %-4s %s to %s, %s%s%n", b.label(), b.bowler(), b.striker(), what, text);
    }

    private static String describe(Ball b) {
        BallOutcome o = b.outcome();
        if (b.wicket() != null) return "OUT " + b.wicket().type() + " (" + b.wicket().batter() + ")"
                + (b.wicket().creditedToBowler() ? "" : " - not the bowler's wicket");
        return switch (o.extra()) {
            case WIDE -> "wide";
            case NO_BALL -> "no ball" + (o.batRuns() > 0 ? " + " + o.batRuns() : "");
            case BYE -> o.extraRuns() + " bye(s)";
            case LEG_BYE -> o.extraRuns() + " leg bye(s)";
            case NONE -> o.batRuns() == 0 ? "no run" : o.boundary() ? o.batRuns() == 4 ? "FOUR" : "SIX" : o.batRuns()
+ " run(s)";
        };
    }
}

/** Live score line at the end of an over (every n overs, to keep the demo short). */
final class LiveScorecard implements MatchObserver {
    private final int everyNOvers;
    LiveScorecard(int everyNOvers) { this.everyNOvers = everyNOvers; }

    @Override public void onOverEnd(Match m, Innings in, Over o) {
        if (o.number() % everyNOvers == 0 || in.isComplete()) {
            String chase = in.target().filter(t -> in.runs() < t).map(t -> "  need " + (t - in.runs()) + " to
win").orElse("");
            System.out.println("  [live] " + in.score() + chase);
        }
    }

    @Override public void onInningsEnd(Match m, Innings in) {
        System.out.println("  [innings " + in.number() + " over] " + in.score());
    }

    @Override public void onMatchEnd(Match m, Result r) {
        System.out.println("  [result] " + r);
    }
}

/** Builds batting and bowling cards from the recorded balls (nothing is stored twice). */
final class Scorecard {
    static void print(Innings in) {
```

```
        System.out.println("Batting - " + in.battingTeam());
        Map<Player, int[]> bat = new LinkedHashMap<>();   // runs, balls, 4s, 6s
        Map<Player, String> howOut = new LinkedHashMap<>();
        for (Ball b : in.balls()) {
            int[] s = bat.computeIfAbsent(b.striker(), k -> new int[4]);
            BallOutcome o = b.outcome();
            s[0] += o.batRuns();
            if (o.countsAsBallFaced()) s[1]++;
            if (o.boundary() && o.batRuns() == 4) s[2]++;
            if (o.boundary() && o.batRuns() == 6) s[3]++;
            if (b.wicket() != null) {
                bat.computeIfAbsent(b.wicket().batter(), k -> new int[4]);
                howOut.put(b.wicket().batter(), b.wicket().creditedToBowler()
                        ? (b.wicket().type() == WicketType.BOWLED ? "" :
b.wicket().type().name().toLowerCase().replace('_', ' ') + " ")
                                + "b " + b.bowler()
                        : "run out");
            }
        }
        bat.computeIfAbsent(in.striker(), k -> new int[4]);        // the two batters at the crease are listed
        bat.computeIfAbsent(in.nonStriker(), k -> new int[4]);     // even if one has not faced a ball yet
        List<Player> order = in.battingSide().battingOrder();
        for (Player p : order) {
            int[] s = bat.get(p);
            if (s == null) continue;
            System.out.printf("  %-10s %-22s %3d (%d)  4s:%d 6s:%d%n", p, howOut.getOrDefault(p, "not out"), s[0],
s[1], s[2], s[3]);
        }
        int extras = in.balls().stream().mapToInt(b -> b.outcome().extraRuns()).sum();
        System.out.println("  Extras " + extras + "   Total " + in.runs() + "/" + in.wickets() + " (" + in.oversText()
+ " ov)");

        System.out.println("Bowling - " + in.bowlingTeam());
        Map<Player, int[]> bowl = new LinkedHashMap<>();  // legal balls, maidens, runs, wickets
        for (Over o : in.overs()) {
            int[] s = bowl.computeIfAbsent(o.bowler(), k -> new int[4]);
            s[0] += o.legalBalls();
            if (o.isMaiden()) s[1]++;
            s[2] += o.runsConceded();
            s[3] += (int) o.balls().stream().filter(b -> b.wicket() != null && b.wicket().creditedToBowler()).count();
        }
        bowl.forEach((p, s) -> System.out.printf("  %-10s %d.%d-%d-%d-%d%n", p, s[0] / 6, s[0] % 6, s[1], s[2],
s[3]));
    }
}
```

```java
// File: lld/ch18/cricinfo/Stats.java
package lld.ch18.cricinfo;

import java.util.Collection;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;

/** Career numbers for one player, always DERIVED from balls, never typed in by hand. */
final class PlayerStats {
    final Player player;
    int runs, ballsFaced, fours, sixes, fifties, hundreds, highest;
    int wickets, ballsBowled, runsConceded;

    PlayerStats(Player player) { this.player = player; }

    /** Per-ball numbers. */
    void countBall(Ball b, boolean asBatter) {
        BallOutcome o = b.outcome();
        if (asBatter) {
            runs += o.batRuns();
            if (o.countsAsBallFaced()) ballsFaced++;
            if (o.boundary() && o.batRuns() == 4) fours++;
            if (o.boundary() && o.batRuns() == 6) sixes++;
        } else {
            if (o.isLegal()) ballsBowled++;
            runsConceded += o.runsChargedToBowler();
            if (b.wicket() != null && b.wicket().creditedToBowler()) wickets++;
        }
    }

    /** Milestones are per innings, so they are counted once the innings score is known. */
    void countInnings(int inningsRuns) {
        if (inningsRuns >= 100) hundreds++;
        else if (inningsRuns >= 50) fifties++;
        highest = Math.max(highest, inningsRuns);
    }
}

/** Recomputes stats from the recorded balls of the given matches, optionally for one format. */
final class StatsService {
    Map<Player, PlayerStats> compute(Collection<Match> matches, MatchFormat formatOrNull) {
        Map<Player, PlayerStats> stats = new HashMap<>();
        for (Match m : matches) {
            if (formatOrNull != null && m.format() != formatOrNull) continue;
            for (Innings in : m.innings()) {
                Map<Player, Integer> inningsRuns = new HashMap<>();
                for (Ball b : in.balls()) {
                    stats.computeIfAbsent(b.striker(), PlayerStats::new).countBall(b, true);
                    stats.computeIfAbsent(b.bowler(), PlayerStats::new).countBall(b, false);
                    inningsRuns.merge(b.striker(), b.outcome().batRuns(), Integer::sum);
                }
                inningsRuns.forEach((p, r) -> stats.get(p).countInnings(r));
            }
        }
        return stats;
    }
}

/**
 * Template Method: answer() is the fixed algorithm (walk all players, keep the best);
 * each question fills in one step, valueOf(). A new statistic = one new subclass.
 */
abstract class StatQuery {
    record Leader(Player player, int value) {}

    abstract String question();
    abstract int valueOf(PlayerStats s);

    final Optional<Leader> answer(Map<Player, PlayerStats> stats) {
        Leader best = null;
        for (PlayerStats s : stats.values()) {
```

```
            int v = valueOf(s);
            if (v > 0 && (best == null || v > best.value()
                    || (v == best.value() && s.player.name().compareTo(best.player().name()) < 0))) {
                best = new Leader(s.player, v);
            }
        }
        return Optional.ofNullable(best);
    }
}

final class MostRuns extends StatQuery {
    @Override String question() { return "Most runs"; }
    @Override int valueOf(PlayerStats s) { return s.runs; }
}

final class MostHundreds extends StatQuery {
    @Override String question() { return "Most hundreds"; }
    @Override int valueOf(PlayerStats s) { return s.hundreds; }
}

final class MostWickets extends StatQuery {
    @Override String question() { return "Highest wicket taker"; }
    @Override int valueOf(PlayerStats s) { return s.wickets; }
}

final class MostSixes extends StatQuery {
    @Override String question() { return "Most sixes"; }
    @Override int valueOf(PlayerStats s) { return s.sixes; }
}

final class StatQueries {
    static final List<StatQuery> ALL = List.of(new MostRuns(), new MostHundreds(), new MostWickets(), new
MostSixes());
    private StatQueries() {}
}
```

```java
// File: lld/ch18/cricinfo/Main.java
package lld.ch18.cricinfo;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.Random;

public class Main {
    public static void main(String[] args) {
        // ---- teams, squads, tournament ----
        Team lions = team("Lions", "Arun", "Bala", "Chetan", "Dev", "Eshan", "Farhan", "Gopal", "Hari", "Imran",
"Jai", "Kiran", "Lokesh", "Manav");
        Team tigers = team("Tigers", "Nabil", "Omar", "Pranav", "Qasim", "Rahul", "Sameer", "Tarun", "Uday", "Varun",
"Wasim", "Yash", "Zubin", "Ajay");
        Tournament cup = new Tournament("Demo Cup");
        TournamentSquad lionsSquad = cup.announceSquad(lions);
        TournamentSquad tigersSquad = cup.announceSquad(tigers);
        lions.players().stream().limit(12).forEach(lionsSquad::add);      // Manav is not in the squad
        tigers.players().stream().limit(12).forEach(tigersSquad::add);

        System.out.println("== Picking the elevens ==");
        Playing11 lionsXI = new Playing11(lionsSquad);
        Playing11 tigersXI = new Playing11(tigersSquad);
        lions.players().stream().limit(11).forEach(lionsXI::add);
        tigers.players().stream().limit(11).forEach(tigersXI::add);
        attempt(() -> new Playing11(lionsSquad).add(find(lions, "Manav")));
        attempt(() -> lionsXI.add(find(lions, "Lokesh")));

        // ---- the match ----
        Stadium stadium = new Stadium("Riverside Oval", "Pune", 40000);
        T20 match = new T20(lionsXI, tigersXI, stadium, LocalDate.of(2026, 9, 28));
        cup.addMatch(match);
        Commentator ravi = new Commentator(new Person("Ravi", "IN"));
        match.assign(new Umpire(new Person("Ken", "NZ"), UmpireType.FIELD));
        match.assign(new Umpire(new Person("Paul", "AU"), UmpireType.FIELD));
        match.assign(new Referee(new Person("Chris", "UK")));
        match.assign(ravi);
        match.addObserver(new CommentaryFeed(ravi, 1));
        match.addObserver(new LiveScorecard(5));

        System.out.println("\n== " + match + " ==");
        match.startInnings(lionsXI);
        List<Player> tigersBowlers = bowlers(tigersXI);
        match.startOver(tigersBowlers.get(0));
        match.bowl(BallOutcome.four("driven through cover"));
        match.bowl(BallOutcome.wide("sprayed down leg"));
        match.bowl(BallOutcome.runs(1, "pushed to mid-on"));
        match.bowl(BallOutcome.noBall(0, "overstepped, free hit next"));
        match.bowl(BallOutcome.out(WicketType.BOWLED, "through the gate!"));
        match.bowl(BallOutcome.runOut(1, false, "a second run was never there"));
        match.bowl(BallOutcome.dot("defended"));
        match.bowl(BallOutcome.six("launched over long-on"));
        attempt(() -> match.startOver(tigersBowlers.get(0)));             // same bowler twice in a row
        Random rnd = new Random(23);
        simulate(match, rnd, tigersBowlers);

        match.startInnings(tigersXI);
        simulate(match, rnd, bowlers(lionsXI));

        System.out.println("\n== Scorecards ==");
        match.innings().forEach(Scorecard::print);
        cup.recordResult(match);
        System.out.println("Points table: " + cup.pointsTable());

        // ---- a second match (ODI) so the global stats have more data ----
        ODI odi = new ODI(tigersXI, lionsXI, stadium, LocalDate.of(2026, 10, 2));
        odi.addObserver(new LiveScorecard(25));
        System.out.println("\n== " + odi + " ==");
        odi.startInnings(tigersXI);
        simulate(odi, rnd, bowlers(lionsXI));
        odi.startInnings(lionsXI);
```

```
        simulate(odi, rnd, bowlers(tigersXI));

        // ---- stats, derived from balls ----
        StatsService statsService = new StatsService();
        List<Match> all = List.of(match, odi);
        Map<Player, PlayerStats> career = statsService.compute(all, null);
        System.out.println("\n== Global stat queries (all formats) ==");
        for (StatQuery q : StatQueries.ALL) {
            System.out.println("  " + q.question() + ": " + q.answer(career)
                    .map(l -> l.player() + " (" + l.value() + ")").orElse("nobody yet"));
        }
        Map<Player, PlayerStats> odiOnly = statsService.compute(all, MatchFormat.ODI);
        System.out.println("  Most runs in ODIs: " + new MostRuns().answer(odiOnly).map(l -> l.player() + " (" +
l.value() + ")").orElse("-"));
        System.out.println("  Ravi is assigned to " + ravi.assignedMatches().size() + " match(es)");
    }

    /** Plays the rest of the current innings with random but legal deliveries. */
    static void simulate(Match m, Random rnd, List<Player> bowlers) {
        Innings in = m.innings().get(m.innings().size() - 1);
        int over = in.overs().size();
        while (!in.isComplete()) {
            m.startOver(bowlers.get(over % bowlers.size()));
            over++;
            while (!in.isComplete() && in.overs().get(in.overs().size() - 1).legalBalls() < Over.LEGAL_BALLS) {
                m.bowl(randomBall(rnd));
            }
        }
    }

    static BallOutcome randomBall(Random rnd) {
        int r = rnd.nextInt(1000);
        if (r < 330) return BallOutcome.dot(null);
        if (r < 640) return BallOutcome.runs(1, null);
        if (r < 720) return BallOutcome.runs(2, null);
        if (r < 730) return BallOutcome.runs(3, null);
        if (r < 810) return BallOutcome.four(null);                    // about 1 ball in 12 is a four
        if (r < 840) return BallOutcome.six(null);                     // and 1 in 33 a six: roughly 6.5 runs an over
        if (r < 870) return BallOutcome.wide(null);
        if (r < 880) return BallOutcome.noBall(0, null);
        if (r < 895) return BallOutcome.legBye(1, null);
        if (r < 905) return BallOutcome.out(WicketType.BOWLED, null);
        if (r < 920) return BallOutcome.out(WicketType.CAUGHT, null);
        if (r < 927) return BallOutcome.out(WicketType.LBW, null);
        if (r < 930) return BallOutcome.runOut(0, rnd.nextBoolean(), null);
        return BallOutcome.dot(null);
    }

    static List<Player> bowlers(Playing11 xi) {
        List<Player> order = xi.battingOrder();
        return new ArrayList<>(order.subList(order.size() - 5, order.size()));   // the last five are the bowlers
    }

    static Team team(String name, String... players) {
        Team t = new Team(name);
        for (String p : players) t.addPlayer(new Player(new Person(p, "IN")));
        return t;
    }

    static Player find(Team t, String name) {
        return t.players().stream().filter(p -> p.name().equals(name)).findFirst().orElseThrow();
    }

    static void attempt(Runnable r) {
        try { r.run(); } catch (RuntimeException e) { System.out.println("  REJECTED: " + e.getMessage()); }
    }
}
```

Output:

```
== Picking the elevens ==
  REJECTED: Manav is not in the Lions squad
  REJECTED: a playing eleven cannot have a twelfth player (Lokesh)

== T20 Lions v Tigers at Riverside Oval ==
  0.1  Tarun to Arun, FOUR  [Ravi: driven through cover]
  0.2  Tarun to Arun, wide  [Ravi: sprayed down leg]
  0.2  Tarun to Arun, 1 run(s)  [Ravi: pushed to mid-on]
  0.3  Tarun to Bala, no ball  [Ravi: overstepped, free hit next]
  0.3  Tarun to Bala, OUT BOWLED (Bala)  [Ravi: through the gate!]
  0.4  Tarun to Chetan, OUT RUN_OUT (Chetan) - not the bowler's wicket  [Ravi: a second run was never there]
  0.5  Tarun to Arun, no run  [Ravi: defended]
  0.6  Tarun to Arun, SIX  [Ravi: launched over long-on]
  REJECTED: Tarun cannot bowl two overs in a row
  [live] Lions 29/2 (5 ov)
  [live] Lions 76/2 (10 ov)
  [live] Lions 117/3 (15 ov)
  [live] Lions 151/5 (20 ov)
  [innings 1 over] Lions 151/5 (20 ov)
  0.1  Gopal to Nabil, wide
  0.1  Gopal to Nabil, 1 run(s)
  0.2  Gopal to Omar, 1 run(s)
  0.3  Gopal to Nabil, 2 run(s)
  0.4  Gopal to Nabil, no run
  0.5  Gopal to Nabil, wide
  0.5  Gopal to Nabil, no run
  0.6  Gopal to Nabil, no ball
  0.6  Gopal to Nabil, no run
  [live] Tigers 40/0 (5 ov)  need 112 to win
  [live] Tigers 76/0 (10 ov)  need 76 to win
  [live] Tigers 91/2 (15 ov)  need 61 to win
  [live] Tigers 128/4 (20 ov)  need 24 to win
  [innings 2 over] Tigers 128/4 (20 ov)
  [result] Lions won by 23 runs

== Scorecards ==
Batting - Lions
  Arun       b Uday                 45 (32)  4s:4 6s:1
  Bala       b Tarun                 0 (2)  4s:0 6s:0
  Chetan     run out                 1 (1)  4s:0 6s:0
  Dev        caught b Yash          51 (51)  4s:4 6s:2
  Eshan      caught b Wasim         42 (31)  4s:5 6s:1
  Farhan     not out                 5 (3)  4s:0 6s:0
  Gopal      not out                 1 (1)  4s:0 6s:0
  Extras 6   Total 151/5 (20 ov)
Bowling - Tigers
  Tarun      4.0-0-41-1
  Uday       4.0-0-27-1
  Varun      4.0-0-29-0
  Wasim      4.0-0-28-1
  Yash       4.0-0-24-1
Batting - Tigers
  Nabil      lbw b Kiran            55 (56)  4s:3 6s:2
  Omar       lbw b Jai              44 (41)  4s:5 6s:0
  Pranav     run out                 1 (4)  4s:0 6s:0
  Qasim      not out                20 (19)  4s:1 6s:1
  Rahul      caught b Kiran          0 (1)  4s:0 6s:0
  Sameer     not out                 0 (0)  4s:0 6s:0
  Extras 8   Total 128/4 (20 ov)
Bowling - Lions
  Gopal      4.0-0-21-0
  Hari       4.0-0-22-0
  Imran      4.0-0-38-0
  Jai        4.0-0-19-1
  Kiran      4.0-0-27-2
Points table: [Lions=2, Tigers=0]

== ODI Tigers v Lions at Riverside Oval ==
  [live] Tigers 181/3 (25 ov)
  [live] Tigers 348/9 (50 ov)
  [innings 1 over] Tigers 348/9 (50 ov)
  [live] Lions 159/1 (25 ov)  need 190 to win
  [live] Lions 349/5 (49.4 ov)
```

```
  [innings 2 over] Lions 349/5 (49.4 ov)
  [result] Lions won by 5 wickets

== Global stat queries (all formats) ==
  Most runs: Chetan (131)
  Most hundreds: Chetan (1)
  Highest wicket taker: Kiran (5)
  Most sixes: Nabil (5)
  Most runs in ODIs: Chetan (130)
  Ravi is assigned to 1 match(es)
```

Walking through the important parts

- Ball numbers follow the scorecard convention "completed overs . ball", so the first ball of the match is `0.1`. A wide or no ball carries the number of the ball that must be re-bowled, which is why `0.2` and `0.3` each appear twice.
- In over 1 of the output, there are eight lines but the over ends at `0.6`: the wide and the no ball did not count toward the six (`Over.isComplete` counts legal balls).
- Chetan is **run out** after completing one run: `Innings.record` first swaps ends for the completed single, then replaces the dismissed batter, so the new batter (Dev) comes in at the correct end and Arun faces the next ball. In the scorecard, Chetan's line says "run out" and Tarun's bowling figures show only **1** wicket (the bowled), because `WicketType.RUN_OUT.creditedToBowler()` is `false`.
- Trying to give Tarun the second over is rejected ("cannot bowl two overs in a row"). The simulation rotates five bowlers, so each bowls exactly 4 overs in the T20 (`maxOversPerBowler() = 20 / 5`).
- `LiveScorecard` prints every 5 overs and the chase line ("need 112 to win") comes from the target set automatically for the final innings (`first-innings total + 1`). The random ball generator is tuned to about 6.5 runs an over, so totals look like real matches (151 and 128 in the T20; 348 and 349 in the ODI, which the Lions win by 5 wickets with two balls to spare).
- The batting card lists every batter who faced a ball or was out, plus the two batters at the crease (Sameer is "not out 0 (0)": he walked in but never faced).
- The global queries (`MostRuns`, `MostHundreds`, `MostWickets`, `MostSixes`) are each a one-line subclass; "Most runs in ODIs" reuses the same service with a format filter.

Machine-coding variant: Cricket Match Dashboard (Udaan)

A **machine-coding round** asks you to write a *working, runnable* program in 60–90 minutes, judged on correctness, readability and extensibility. This problem was set by Udaan (an Indian B2B — business-to-business — e-commerce company) and appears in the kumaransg/LLD collection.

Problem statement (in our words)

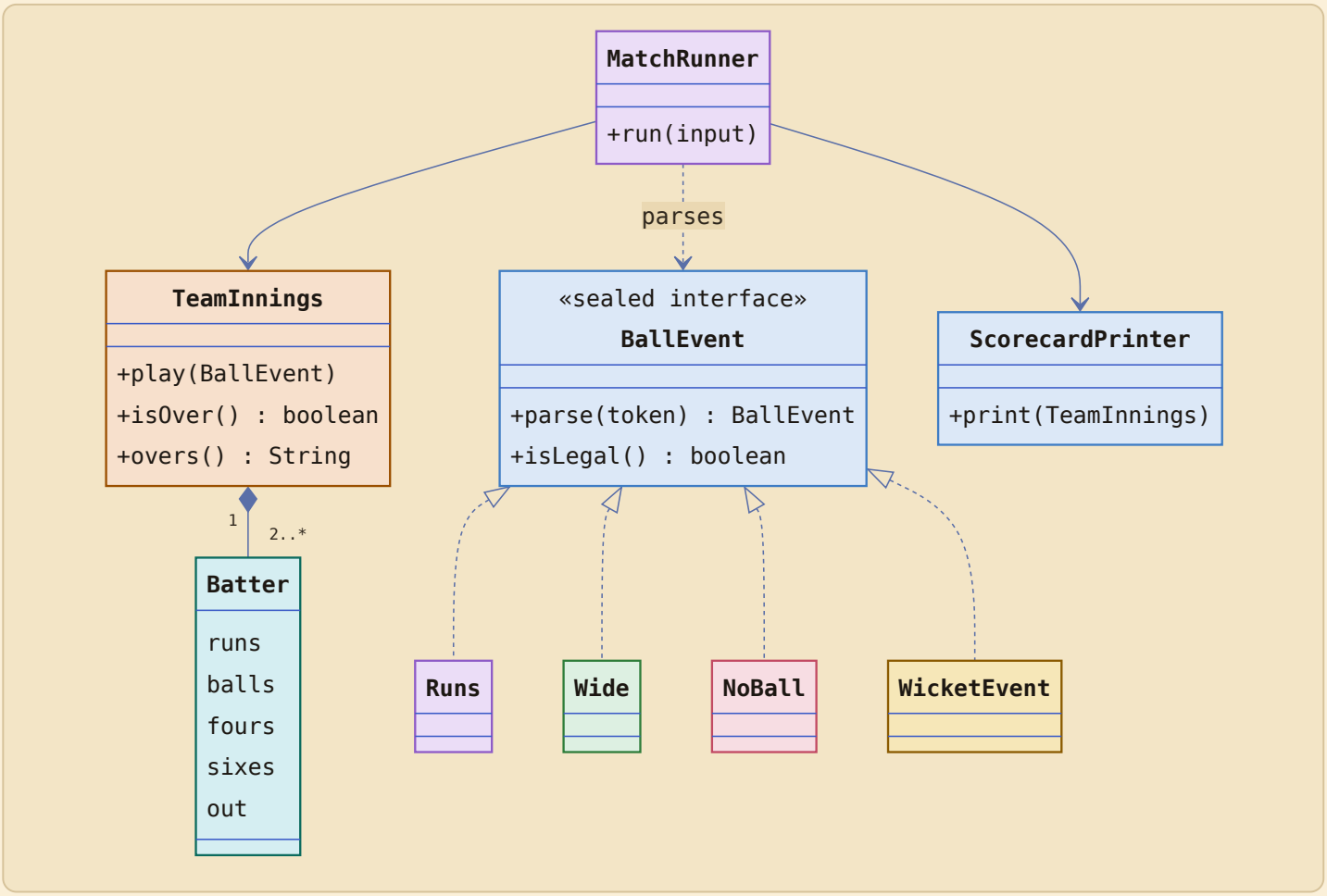
Build a cricket **scorecard** for a two-team, limited-overs match.

- **Input:** number of players per team, number of overs, then for each team its batting order, followed by the balls of each over, one per line: a number of runs (`0`–`6`), `W` (wicket), `Wd` (wide) or `Nb` (no ball).
- **Output after every over:** for each batter of the batting side — runs, 4s, 6s, balls faced — with a `*` on the two batters currently at the crease; then team total/wickets and overs.
- **Rules to handle:** strike changes on 1s and 3s and at the end of every over; wides and no balls add one run and are re-bowled (they do not count toward the six); the innings ends when the team is all out, the overs run out, or (second innings) the target is passed.
- **At the end:** print the winner and the margin (by runs or by wickets).
- **Good to have:** bowler figures (overs, runs, wickets, maidens, dot balls, economy), team extras, batter strike rates.

How the full Cricinfo design shrinks to a 90-minute solution

Cricinfo class	Machine-coding version	Why it is enough here
`BallOutcome` with extras, wickets, validation	`BallEvent` sealed interface: `Runs`, `Wide`, `NoBall`, `Wicket` with a `parse(token)` factory (a method that builds the right object from one input word)	Input has only four kinds of token
`Innings` (strike, over rule, end conditions)	`TeamInnings` — same responsibilities, ints instead of objects	Same rules, no bowler data in the input
`Scorecard` derived from balls	`Batter` counters + `ScorecardPrinter`	Only batting card needed; printer separated so another view could replace it
`Match` + `Result`	`MatchRunner` (reads input, plays two innings, prints result)	One format, two innings

This is the lesson of machine coding: keep the *same responsibilities in the same places* (the innings owns strike and the over rule; parsing is separate from scoring; printing is separate from both), but use the simplest data that satisfies the input.



(In the code the wicket record is called `BallEvent.Wicket`; it is shown as `WicketEvent` above only to keep the diagram label unique.)

Code

Package `lld.ch18.cricketdashboard`. `Main` feeds the problem's sample input as a text block.

```java
// File: lld/ch18/cricketdashboard/Model.java
package lld.ch18.cricketdashboard;

import java.util.ArrayList;
import java.util.List;

/** One input token, parsed once. Factory method: BallEvent.parse("Wd") etc. */
sealed interface BallEvent {
    record Runs(int runs) implements BallEvent {}
    record Wide() implements BallEvent {}
    record NoBall(int batRuns) implements BallEvent {}
    record Wicket() implements BallEvent {}

    static BallEvent parse(String token) {
        String t = token.trim();
        if (t.equalsIgnoreCase("W")) return new Wicket();
        if (t.equalsIgnoreCase("Wd")) return new Wide();
        if (t.regionMatches(true, 0, "Nb", 0, 2)) return new NoBall(t.length() > 2 ? Integer.parseInt(t.substring(2))
: 0);
        int runs = Integer.parseInt(t);
        if (runs < 0 || runs > 6) throw new IllegalArgumentException("bad runs: " + t);
        return new Runs(runs);
    }

    default boolean isLegal() { return !(this instanceof Wide || this instanceof NoBall); }
}

/** A batter's line on the scorecard. */
final class Batter {
    final String name;
    int runs, balls, fours, sixes;
    boolean out;

    Batter(String name) { this.name = name; }

    void faced(int batRuns) {
        runs += batRuns;
        balls++;
        if (batRuns == 4) fours++;
        if (batRuns == 6) sixes++;
    }

    String strikeRate() { return balls == 0 ? "-" : String.format("%.1f", 100.0 * runs / balls); }
}

/** One team's batting innings: owns strike rotation, extras, and the "is it over?" rule. */
final class TeamInnings {
    private final String team;
    private final List<Batter> order = new ArrayList<>();
    private final int maxOvers;
    private final Integer target;          // null for the first innings
    private int striker = 0, nonStriker = 1, next = 2;
    private int total, wickets, legalBalls, extras;

    TeamInnings(String team, List<String> battingOrder, int maxOvers, Integer target) {
        if (battingOrder.size() < 2) throw new IllegalArgumentException("need at least two batters");
        this.team = team;
        battingOrder.forEach(n -> order.add(new Batter(n)));
        this.maxOvers = maxOvers;
        this.target = target;
    }

    void play(BallEvent e) {
        if (isOver()) throw new IllegalStateException("innings already finished");
        Batter s = order.get(striker);
        switch (e) {
            case BallEvent.Wide w -> { total += 1; extras += 1; }
            case BallEvent.NoBall nb -> {
                total += 1 + nb.batRuns(); extras += 1;
                s.faced(nb.batRuns());              // a no ball counts as a ball faced by the batter
                if (nb.batRuns() % 2 == 1) swap();
            }
            case BallEvent.Wicket w -> {
```

```
                legalBalls++;
                s.balls++;
                s.out = true;
                wickets++;
                if (!allOut()) striker = next++;     // new batter takes strike
            }
            case BallEvent.Runs r -> {
                legalBalls++;
                total += r.runs();
                s.faced(r.runs());
                if (r.runs() % 2 == 1) swap();       // 1s and 3s change strike
            }
        }
        if (e.isLegal() && legalBalls % 6 == 0 && !isOver()) swap();  // end of over
    }

    boolean isOver() {
        return allOut() || legalBalls == maxOvers * 6 || (target != null && total >= target);
    }

    private boolean allOut() { return wickets == order.size() - 1; }
    private void swap() { int t = striker; striker = nonStriker; nonStriker = t; }

    String team() { return team; }
    int total() { return total; }
    int wickets() { return wickets; }
    int legalBalls() { return legalBalls; }
    int extras() { return extras; }
    int wicketsInHand() { return order.size() - 1 - wickets; }
    List<Batter> batters() { return List.copyOf(order); }
    boolean isAtCrease(Batter b) { return !b.out && (order.get(striker) == b || order.get(nonStriker) == b); }

    String overs() { return legalBalls / 6 + (legalBalls % 6 == 0 ? "" : "." + legalBalls % 6); }
}
```

```java
// File: lld/ch18/cricketdashboard/Main.java
package lld.ch18.cricketdashboard;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Deque;
import java.util.List;

/** Prints the scorecard; kept apart so a web or JSON view could replace it (Single Responsibility). */
final class ScorecardPrinter {
    void print(TeamInnings in) {
        System.out.println("Scorecard for " + in.team() + ":");
        System.out.printf("%-8s %5s %3s %3s %5s %6s%n", "Player", "Score", "4s", "6s", "Balls", "SR");
        for (Batter b : in.batters()) {
            String name = b.name + (in.isAtCrease(b) ? "*" : "");
            System.out.printf("%-8s %5d %3d %3d %5d %6s%n", name, b.runs, b.fours, b.sixes, b.balls, b.strikeRate());
        }
        System.out.println("Total: " + in.total() + "/" + in.wickets() + "   Extras: " + in.extras());
        System.out.println("Overs: " + in.overs());
    }
}

/** Drives the whole match from the text input (input format of the Udaan machine-coding round). */
final class MatchRunner {
    private final ScorecardPrinter printer = new ScorecardPrinter();

    void run(String input) {
        Deque<String> lines = new ArrayDeque<>();
        Arrays.stream(input.split("\\R")).map(String::trim).filter(l -> !l.isEmpty()).forEach(lines::add);
        int players = Integer.parseInt(after(lines.poll()));
        int overs = Integer.parseInt(after(lines.poll()));

        TeamInnings first = playInnings("Team 1", players, overs, null, lines);
        TeamInnings second = playInnings("Team 2", players, overs, first.total() + 1, lines);

        if (second.total() > first.total()) System.out.println("Result: Team 2 won the match by " +
second.wicketsInHand() + " wickets");
        else if (second.total() < first.total()) System.out.println("Result: Team 1 won the match by " +
(first.total() - second.total()) + " runs");
        else System.out.println("Result: Match tied");
    }

    private TeamInnings playInnings(String team, int players, int overs, Integer target, Deque<String> lines) {
        lines.poll();                                          // "Batting Order for team N:"
        List<String> order = new ArrayList<>();
        for (int i = 0; i < players; i++) order.add(lines.poll());
        TeamInnings in = new TeamInnings(team, order, overs, target);
        System.out.println("Batting order for " + team + ": " + order);
        while (!in.isOver() && !lines.isEmpty() && lines.peek().startsWith("Over")) {
            System.out.println(lines.poll());
            int legalThisOver = 0;
            while (legalThisOver < 6 && !in.isOver()) {          // an over = 6 LEGAL balls
                BallEvent e = BallEvent.parse(lines.poll());
                in.play(e);
                if (e.isLegal()) legalThisOver++;
            }
            printer.print(in);
        }
        // skip any balls left in the input after the innings ended (e.g. target reached mid-over)
        while (!lines.isEmpty() && !lines.peek().startsWith("Batting Order")) lines.poll();
        return in;
    }

    private static String after(String line) { return line.substring(line.indexOf(':') + 1).trim(); }
}

public class Main {
    static final String SAMPLE = """
            No. of players for each team: 5
            No. of overs: 2
            Batting Order for team 1:
            P1
```

```
            P2
            P3
            P4
            P5
            Over 1:
            1
            1
            1
            1
            1
            2
            Over 2:
            W
            4
            4
            Wd
            W
            1
            6
            Batting Order for team 2:
            P6
            P7
            P8
            P9
            P10
            Over 1:
            4
            6
            W
            W
            1
            1
            Over 2:
            6
            1
            W
            W
            """;

    public static void main(String[] args) {
        new MatchRunner().run(SAMPLE);
    }
}
```

Output:

```
Batting order for Team 1: [P1, P2, P3, P4, P5]
Over 1:
Scorecard for Team 1:
Player   Score  4s  6s Balls     SR
P1*          3   0   0     3  100.0
P2*          4   0   0     3  133.3
P3           0   0   0     0      -
P4           0   0   0     0      -
P5           0   0   0     0      -
Total: 7/0   Extras: 0
Overs: 1
Over 2:
Scorecard for Team 1:
Player   Score  4s  6s Balls     SR
P1           3   0   0     4   75.0
P2*         10   0   1     4  250.0
P3           8   2   0     3  266.7
P4*          1   0   0     1  100.0
P5           0   0   0     0      -
Total: 23/2   Extras: 1
Overs: 2
Batting order for Team 2: [P6, P7, P8, P9, P10]
Over 1:
Scorecard for Team 2:
Player   Score  4s  6s Balls     SR
P6          10   1   1     3  333.3
P7*          1   0   0     1  100.0
P8           0   0   0     1    0.0
P9*          1   0   0     1  100.0
P10          0   0   0     0      -
Total: 12/2   Extras: 0
Overs: 1
Over 2:
Scorecard for Team 2:
Player   Score  4s  6s Balls     SR
P6          10   1   1     3  333.3
P7*          8   0   1     3  266.7
P8           0   0   0     1    0.0
P9           1   0   0     2   50.0
P10          0   0   0     1    0.0
Total: 19/4   Extras: 0
Overs: 1.4
Result: Team 1 won the match by 4 runs
```

Notes on the sample and on correctness

- Leaving aside the strike-rate and extras columns we add, our output matches the published sample for team 1 exactly, and for team 2 in every line except two places where the published sample itself is inconsistent: after team 2's first over it prints `12/1`, but the over contains **two** `W` tokens (P6 and P8 were both out), so the correct total is `12/2`; and in the final card it shows P6 with 2 balls after 3 in the previous card. Following the stated rules gives `12/2` and 3 balls, which is what our program prints. Pointing out such inconsistencies calmly is a plus in a machine-coding round.
- Team 2's second over stops after four balls because the fourth wicket leaves only one batter (5 players → all out at 4 wickets), so "Overs: 1.4".
- The loop that reads an over counts **legal** balls (`legalThisOver < 6`), so a wide in team 1's second over makes that over seven tokens long.
- Extras and strike rate (two of the "good to have" items) are printed; bowler figures would need bowler names in the input — add a `Bowler` class with the same counters and a line per over naming the bowler.

Concurrency & edge cases

- **Two scorers, one match.** `Match.bowl`, `startOver` and `startInnings` are `synchronized` on the match, so balls are applied one at a time in order. Different matches never block each other.
- **Many readers.** Observers are held in a `CopyOnWriteArrayList` (a list that makes a fresh copy on every change, so readers never see it half-updated), which is safe to iterate while another thread subscribes. At web scale, the match would publish each ball to a **message broker** (a middleman service that receives messages and passes them on to every subscriber) and readers would receive it over **WebSockets** (a connection the browser keeps open so the server can send data at any time) or **server-sent**

events (a one-way stream of updates from server to browser); the scorecard would be a cached view rebuilt from balls.

- **Illegal sequences rejected:** starting an over while one is in progress; the same bowler twice in a row; a bowler over his quota; bowling after the innings ended; a bowler from the batting side; a twelfth player; a player outside the squad.
- **Impossible dismissals rejected:** bowled, caught, LBW, stumped or hit wicket off a no ball; bowled, caught or LBW off a wide.
- **Wicket on the last ball of an over:** the new batter comes in, then ends change for the new over — both handled in `Innings.record`.
- **Tie:** equal totals give `Result.Tie` (a super over would be an extension).
- **Test matches:** `oversPerInnings() = 0` means no limit; the demo's `declare()` lets a captain close an innings; `declareDraw()` ends a Test that runs out of time.

Extensions & follow-up questions

- **"Who is on strike?"** — The course leaves this out; our `Innings` tracks it (odd runs and end of over swap ends).
- **"Stats are slow to recompute from every ball."** — Keep a running total per player as a cache, updated by an Observer on each ball, and rebuild it from balls whenever a scorer corrects a ball. The balls stay the truth; the total is only a cache.
- **"Add Duckworth–Lewis–Stern."** — A `TargetCalculator` Strategy on the match: the default returns "opponent total + 1"; a DLS implementation recomputes the target from overs lost.
- **"Add a free hit after a no ball."** — The innings remembers "next legal ball is a free hit"; `BallOutcome` validation then allows only run-out-type dismissals on that ball.
- **"Add partnerships and fall of wickets."** — Derive from balls: split the ball list at each wicket and sum.
- **"Add net run rate to the points table."** — Derive from innings totals and overs faced/bowled; still no stored numbers.
- **"Push notifications for wickets and milestones."** — A new `MatchObserver` that sends a push when `ball.wicket() != null` or a batter crosses 50/100.

Real-world example

ESPNcricinfo (founded in 1993 and bought by ESPN in 2007) is the reference site for cricket statistics; its **Statsguru** tool answers questions like the ones in requirement 7 across every recorded international match. Live pages update ball by ball with commentary lines very much like our `CommentaryFeed`, and millions follow big matches on phones — which is why the push-based Observer design (and, at scale, message brokers and caches) matters.

In the interview

Minutes	What to do
0–5	Clarify formats, actors, and that the real rules matter; say early: **"an over ends after six *legal* balls, however many deliveries that takes"**
5–10	Team ⊇ squad ⊇ playing eleven; people hold a `Person`
10–20	Class diagram: match (abstract) → innings → over → ball; formats differ in two numbers
20–30	Ball-recording flow, strike rotation, **the bowler gets no credit for a run out**, byes are not the batter's runs
30–40	Stats derived from balls; Template Method for stat queries; Observers for live feed
40–45	Concurrency (one scorer lock per match, broadcast to readers) and extensions

30-second answer: "I model cricket's rules, not just its nouns. An over is six legal balls; a wide or no ball is re-bowled. Because the overs belong to the innings, only the innings decides when an over ends, who is on strike and when the innings is over. Formats differ only in overs per innings and

innings per match, so they override two methods. Stats are derived from the balls so corrections propagate, and each stat question is a Template Method subclass. Live pages subscribe as observers."

Follow-ups with answers

- *Why not store `wickets` on `Player`?* — It would drift from the ball record and cannot be filtered by format or tournament; derive it.
- *How do you add "most catches"?* — Add the fielder to `Wicket`, count it in `PlayerStats`, and write `MostCatches extends StatQuery` — no change to the service's loop.
- *Is Template Method the right name for `Match`/formats?* — The course says only `StatQuery` is a named pattern here; the formats are plain abstract-class specialisation. Be precise about that.

Self-check

- **Q:** A bowler bowls a wide and a no ball in an over. How many deliveries does the over have? **A:** Eight — six legal plus the two illegal ones.
- **Q:** Who is credited with a run-out wicket? **A:** The fielding side, not the bowler; it counts in the team's wickets but not in the bowler's figures.
- **Q:** Are leg byes added to the batter's score? **A:** No — they are extras for the team, and they are not charged to the bowler either.
- **Q:** Why does `Playing11` hold its `TournamentSquad`? **A:** So it can refuse a player not in the squad (requirement 5) — the rule lives in code.
- **Q:** What two methods distinguish `T20`, `ODI` and `Test`? **A:** `oversPerInnings()` (20, 50, 0 = unlimited) and `inningsPerSide()` (1, 1, 2).
- **Q:** In the Udaan variant, what is the correct team 2 score after over 1 of the sample? **A:** 12/2 — the over contains two wickets.

TL;DR

Cricinfo = people hold a `Person`; team ⊇ squad ⊇ eleven; `Match` (abstract, formats override two numbers) → `Innings` (owns overs, strike, end rules) → `Over` (six **legal** balls) → `Ball` (outcome + wicket). Run outs are not the bowler's; byes are not the batter's. Stats are derived from balls; stat questions are **Template Method** subclasses; live pages are **Observers**. The Udaan dashboard is the same responsibilities in a 90-minute, input-driven form.

18.3 Design LinkedIn

Figure — A tablet showing the LinkedIn logo surrounded by a network of professionals, the cover image of this case study (course diagram)

LinkedIn is a networking site for people's **working lives**. Members link up with colleagues and contacts they trust, and they use the site to find their next career move. A profile lists a member's skills, jobs and schooling; the home page shows a **feed** of updates from the member's network, arranged in sections (modules) the member can reorder. It works much like Facebook, with features specialised for careers: **connections** (mutual, both sides agree), **follows** (one-sided), **endorsements**, **recommendations**, **company pages** and **job postings**.

Problem & clarifying questions

Question to ask	Answer we assume
Is a connection mutual?	Yes: an **invitation** is sent, and the receiver accepts or rejects it. Following is one-sided and needs no approval
Do we show degrees of connection?	Yes: 1st (direct), 2nd (connection of a connection), 3rd; beyond that "out of network"
Who can message whom?	A member can message their connections (messaging most non-connections needs LinkedIn's paid *InMail*, out of scope)
Who can endorse a skill?	Only 1st-degree connections, once per skill
Recommendations?	A member *requests* one from a connection; only that person can write it
Profile stats?	Profile views, number of connections, search appearances
Companies and jobs?	Members create company pages; page admins post jobs; members apply once per job
Feed?	Posts from connections and followed members, ranked (the course leaves the feed out; we add a simple one)

Requirements

Functional (the course's twelve, in our words)

1. A member fills in a profile: basic details, jobs held, schooling, skills and accomplishments.
2. Any user can search for members or companies by name (and jobs by title).
3. Members send, accept (or reject) connection requests.
4. Members can request a recommendation from other members.
5. The system shows profile stats: profile views, total connections, search appearances.
6. Members create posts to share with their connections.
7. Members can comment on posts; they can like or share both posts **and comments**.
8. Members send messages to other members.
9. A member gets a notification for each new message, each connection invitation, and each comment left on one of their posts.
10. Members create company pages and add job postings.
11. Members create groups and join any group.
12. Members follow other members or companies.

Non-functional

- **Symmetry:** a connection must always appear on *both* members, never only one.
- **Fast reads:** feed, search and degree lookups are far more frequent than writes.
- **Privacy/abuse:** blocked members' content disappears; you cannot spam strangers with messages.
- **Extensibility:** new notification channels and feed-ranking rules plug in without edits.

Use cases

Three actors: **Member** (search, connect, post, message, follow…), **Admin** (block and unblock members) and **System** (sends notifications for new messages, invitations and so on).

Top use cases: add/update profile; search members, companies or jobs and send connection requests; follow or unfollow a member or company; send a message to connections; create a post, like and comment; send notifications.

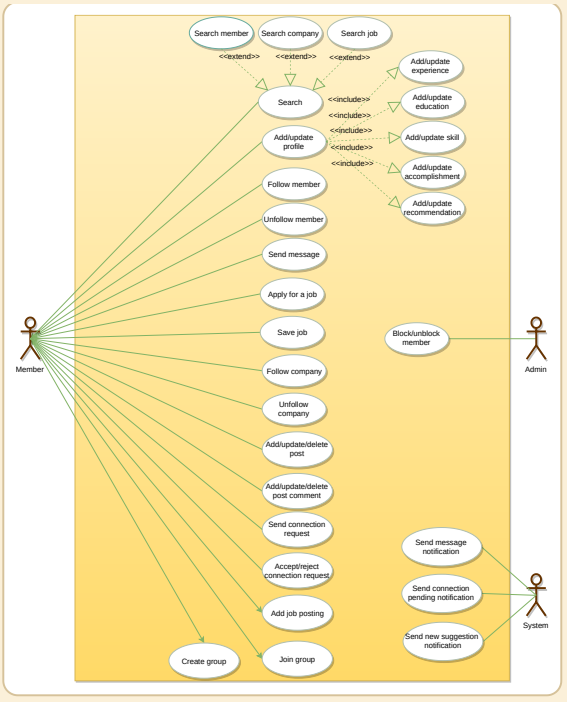


Figure — LinkedIn use cases for Member, Admin and System, including follow/unfollow member, with `<<include>>` and `<<extend>>` relationships (course diagram)

Activity — add experience to a profile. The member opens *Add Experience*, enters title, company, location, dates and description, optionally uploads a document or photo with a description, and presses save. The system verifies the data; if anything is missing it notifies the member, otherwise it saves the experience.

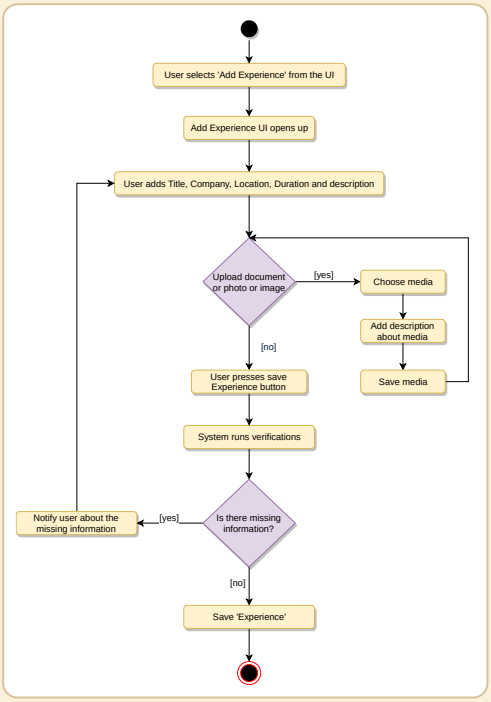


Figure — Activity diagram "add experience to profile" with optional media upload and a verification loop (course diagram)

Activity — send a message. The member composes a message, picks one or more recipients, adds a body and optional media, and presses send. After verification the message goes to every recipient, and each gets an email and a push notification.

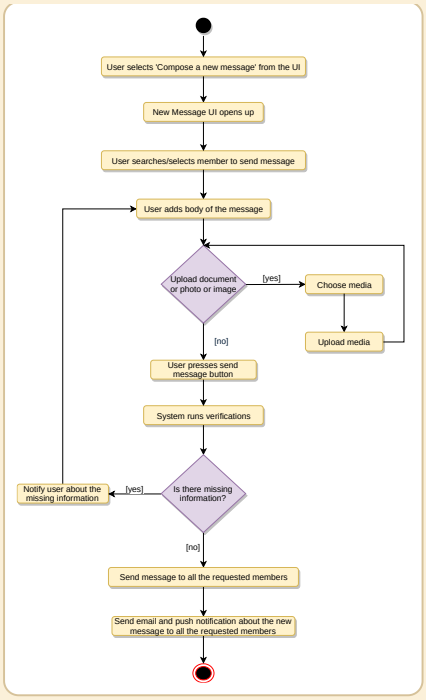


Figure — Activity diagram "send message": choose recipients, add body and media, verify, deliver to all recipients, notify them by email and push (course diagram)

Core entities

Class	Responsibility
`Member`, `Admin`	Identity, profile, connection set (changed only by the service), follows, posts, inbox, notifications
`Profile`	Owns `Experience`, `Education`, `Skill`, `Accomplishment`, `Recommendation`, `ProfileStats`
`Skill`	Name + set of endorsers (no double endorsement)
`Recommendation`	Created empty on request; only the named author writes it (REQUESTED → WRITTEN)
`ProfileStats`	View and search-appearance counters pushed in from where the events happen
`SocialContent` (abstract) → `Post`, `Comment`	Author, text, time, likes as a set; posts own comments and can be shares of another post
`Message` (record)	Sender, recipients, text — not "social content" (nobody likes a message). The course allows media on messages and posts too; our code keeps text only
`ConnectionInvitation` + `InvitationStatus`	State over time: PENDING → ACCEPTED / REJECTED / CANCELED
`ConnectionService`	Invite, accept, reject, withdraw; degree (BFS); mutual connections; people-you-may-know
`Company`, `JobPosting`, `JobApplication`, `Group`	Pages, jobs (apply once), groups (owner cannot leave)
`Notification`, `NotificationService`, channels	One notification class with a *type* field; delivery Strategy
`SearchIndex`	Word → set of members / companies / jobs
`FeedService` + `FeedRanker`	Pull-model feed with pluggable ranking
`LinkedIn`	Facade

Class diagram

The course builds LinkedIn in **six stages**.

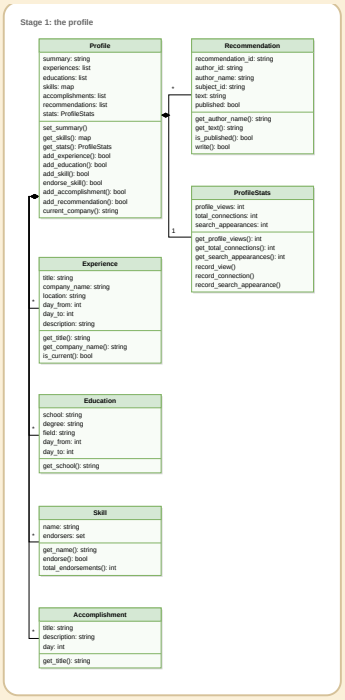


Figure — Stage 1: `Profile` owning `Experience`, `Education`, `Skill`, `Accomplishment`, `Recommendation` and `ProfileStats` (course diagram)

Stage 1 — the profile. Each of requirement 1's five parts is a small class, not a line of text, because other code uses them (search looks up the current employer; endorsements are counted on the skill). Two details matter. You cannot pass text when creating a `Recommendation`: requirement 4 is about *asking* for one, so the object is born empty and the author fills it in later. And the profile never updates `ProfileStats` itself: whoever witnesses the event bumps the counter — the code that shows a profile, the code that accepts an invitation, the code that returns search results — because an object cannot count events it never sees. All arrows out of `Profile` are composition: delete the profile and its parts go with it.

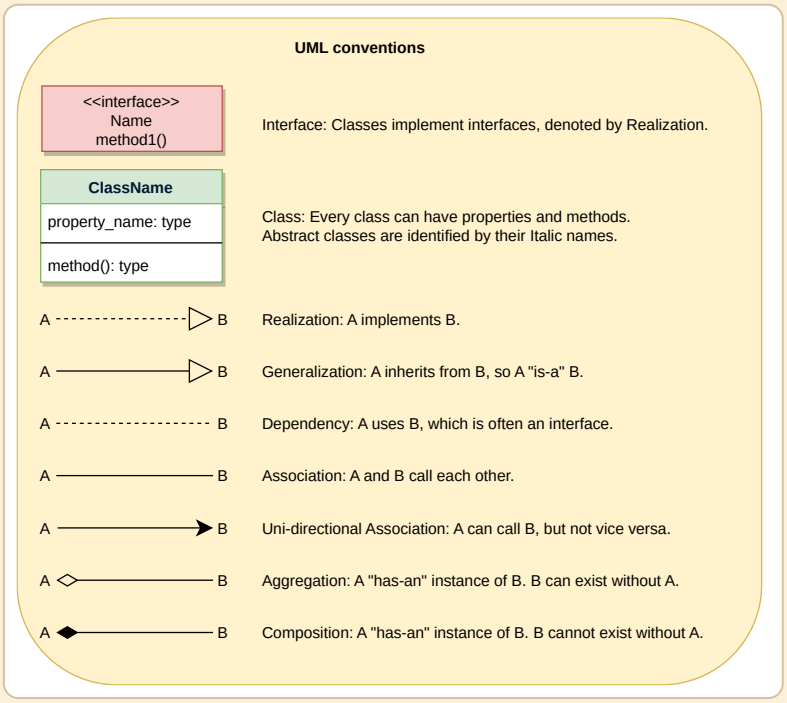


Figure — UML conventions: interface, class, abstract class, realization, generalization, dependency, association, aggregation and composition (course diagram)

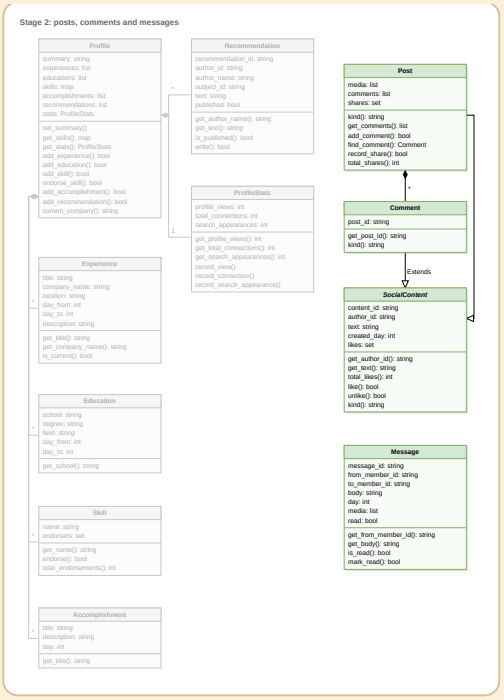


Figure — Stage 2: `SocialContent` *with* `Post` *and* `Comment`*;* `Post` *owns its comments;* `Message` *stands apart (course diagram)*

Stage 2 — posts, comments and messages. Requirement 7 hides a detail: *comments get likes as well.* Put likes only on `Post` and part of the requirement has no home. So both classes extend one parent, `SocialContent`, which holds the author, the text and the likes. The likes are kept as the **ids of the members who liked**, not as a number, which makes a second like by the same person impossible and lets a like be taken back. `Message` is not part of this family: it is delivered to people rather than published, and no one likes a message.

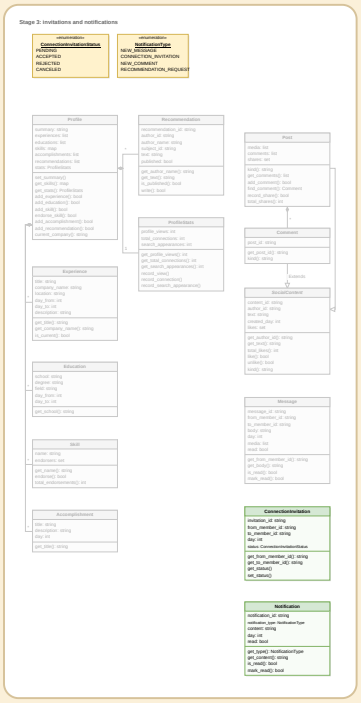


Figure — Stage 3: `ConnectionInvitation` *(with a status) and* `Notification` *(with a type) (course diagram)*

Stage 3 — invitations and notifications. An invitation to connect starts pending and later becomes accepted or rejected. Because it **changes state as time passes**, it deserves its own object (this is the idea behind the **State** pattern: an object's allowed actions depend on which state it is in, and only certain moves between states are legal) rather than a true/false field on a member, and that object can still record, afterwards, that the answer was no. Requirement 9 lists three notification events, but

delivering them is identical, so the kind is a **field**, not three subclasses. The general rule: **subclass when behaviour differs; use a field when only data differs.**

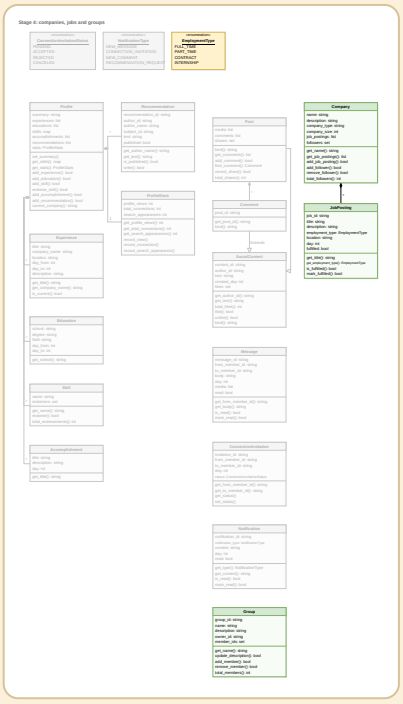


Figure — Stage 4: `Company` owning its `JobPosting`s, with followers as ids; `Group` with members and an owner (course diagram)

`Stage 4 — companies, jobs and groups.` A company page owns its job postings (a posting means nothing once the page is gone) and keeps followers as a set of ids. A group records who created it, and that creator is never allowed to leave or be removed.

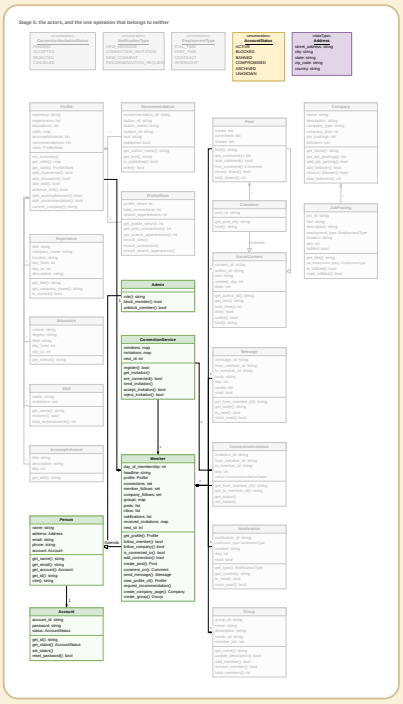


Figure — Stage 5: `Account`, `Person`, `Member`, `Admin` and `ConnectionService`, which points at both invitations and members (course diagram)

`Stage 5 — the actors, and an operation that fits neither of them.` Nearly every requirement becomes a method on `Member`. **Accepting an invitation does not**: it edits two members together. If the sender's class did it, the receiver's connection list would be changed by a different object; if each side did half, a bug would one day update only one half, leaving a one-way link. So the operation goes into a small `ConnectionService`, and that single class is responsible for keeping connections **symmetric** (A lists B

exactly when B lists A). Following is different: it is one-sided and needs no agreement, so it is just a set on the follower, no service needed.

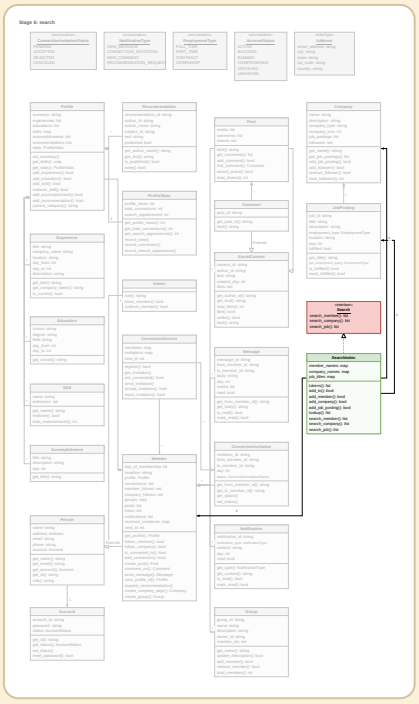


Figure — Stage 6: the `Search` interface and `SearchIndex` with three maps from key to list (members, companies, job postings) (course diagram)

Stage 6 — search. The course shows a real bug from an earlier version: on the first occurrence of a name the index saved *one* member object, and on the next occurrence it tried to "add" to that object — fine for the first Sharma, broken for the second. The cure is a rule: **an index value is always a collection, even when it holds one item.** Keys are single lowercase words, so first and last names both work.

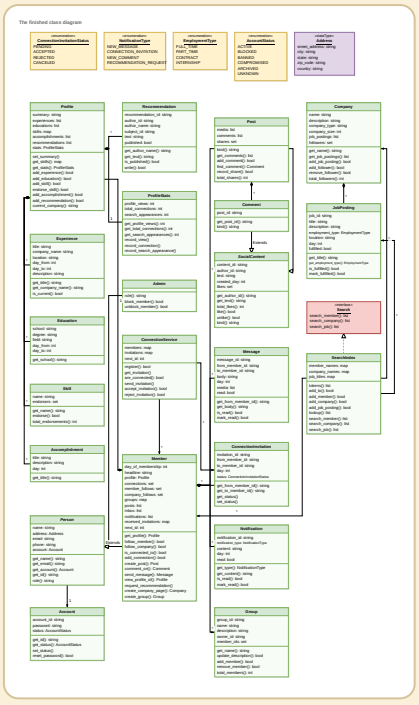
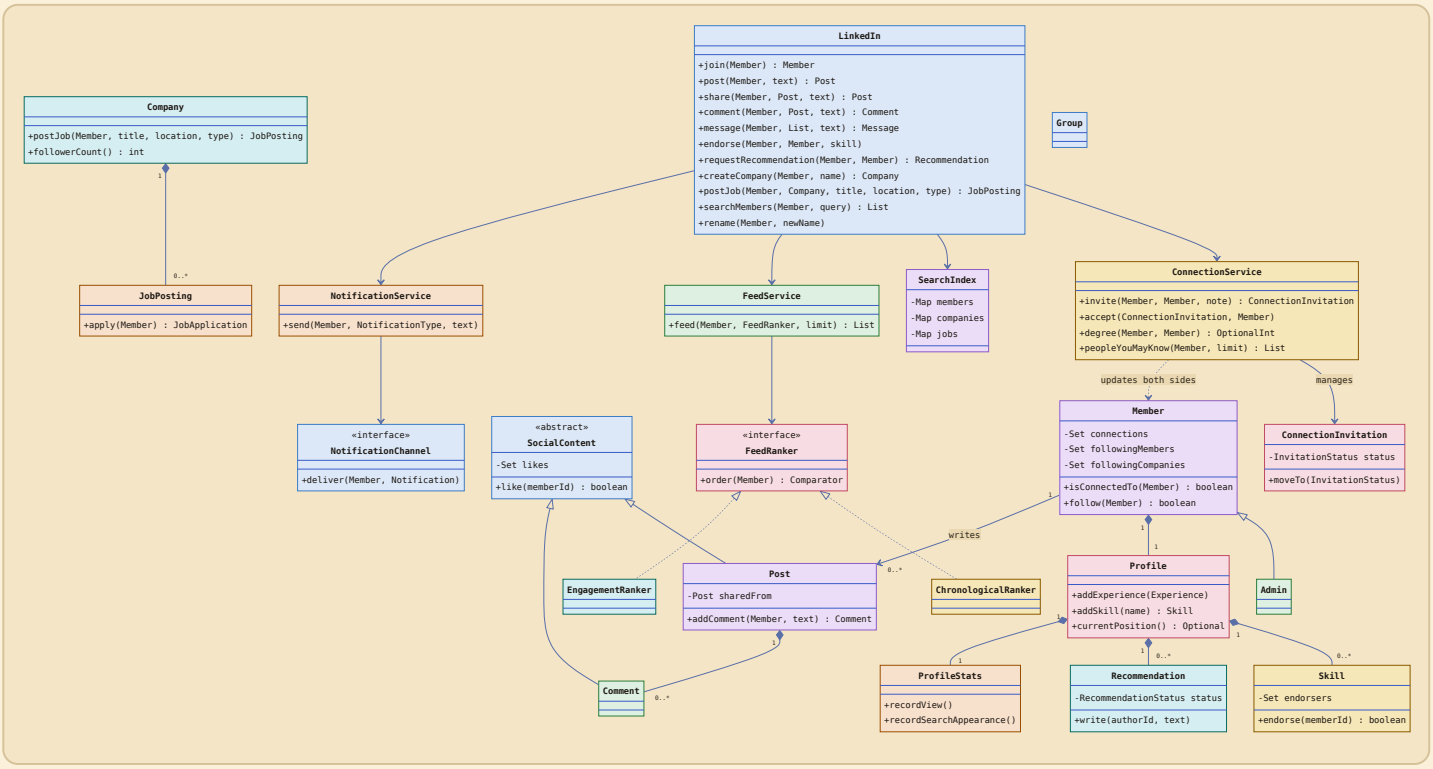


Figure — The finished LinkedIn class diagram, with all six stages in full colour (course diagram)

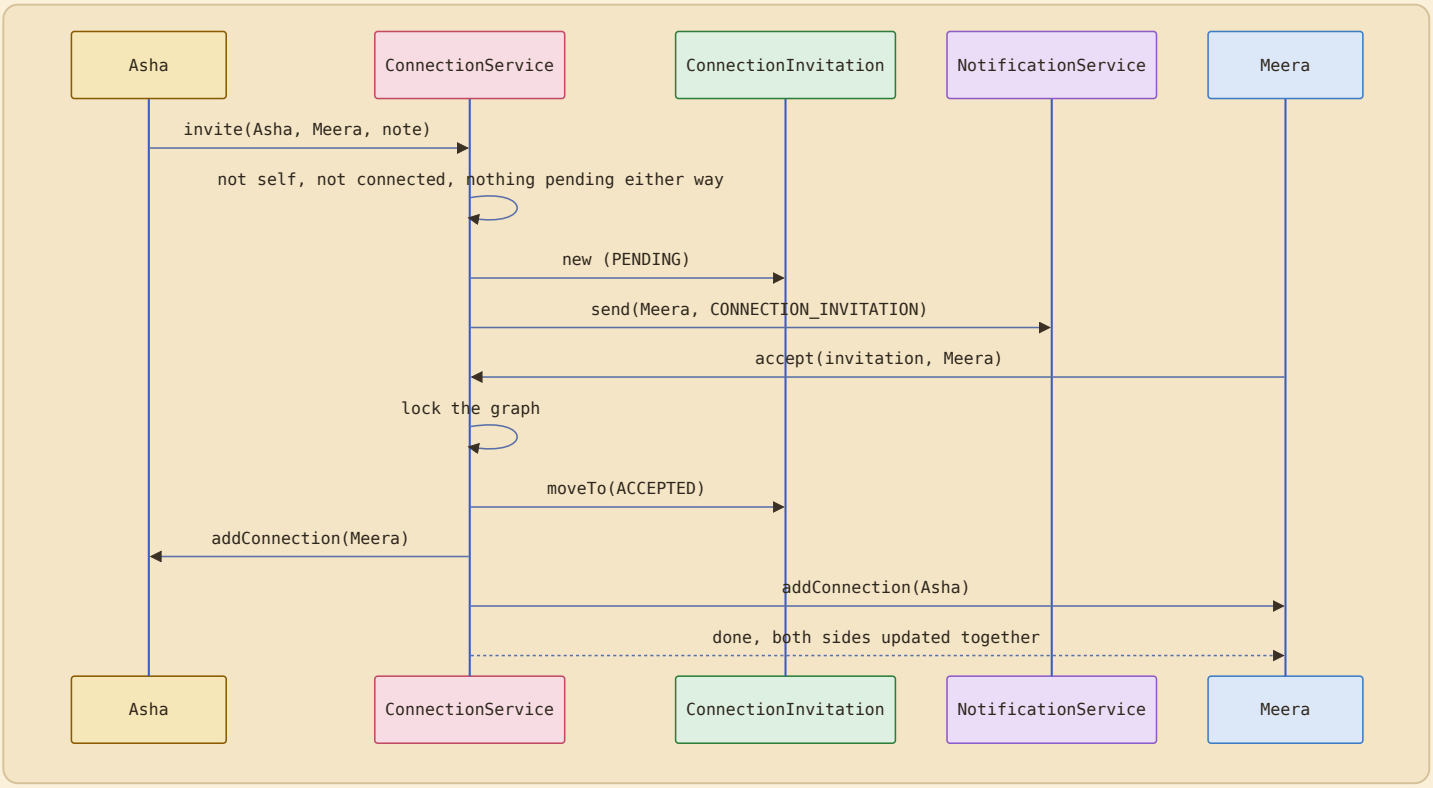
The course deliberately omits several features: the feed and how it is ranked, suggested connections, degrees of separation, applying for jobs and tracking applicants, privacy controls, and one member blocking another. It also admits two weak spots a reviewer will probe. First, there is no feed; building one means either gathering the connections' posts at the moment the page loads, or copying each new post into all the connections' feeds when it is written, and which is better depends on how many connections a typical member has. Second, the search index goes stale (out of date) when a member changes their name, because nothing informs it.

Our Java design follows the six stages and fills the gaps the course names: a **feed** (pull model — the feed is gathered when the member opens it — with a ranking Strategy), **degrees of connection** (breadth-first search, described under Key flows), **people you may know**, a **rename that updates the index**, job **applications**, and connection-count derived from the set instead of a stored counter.

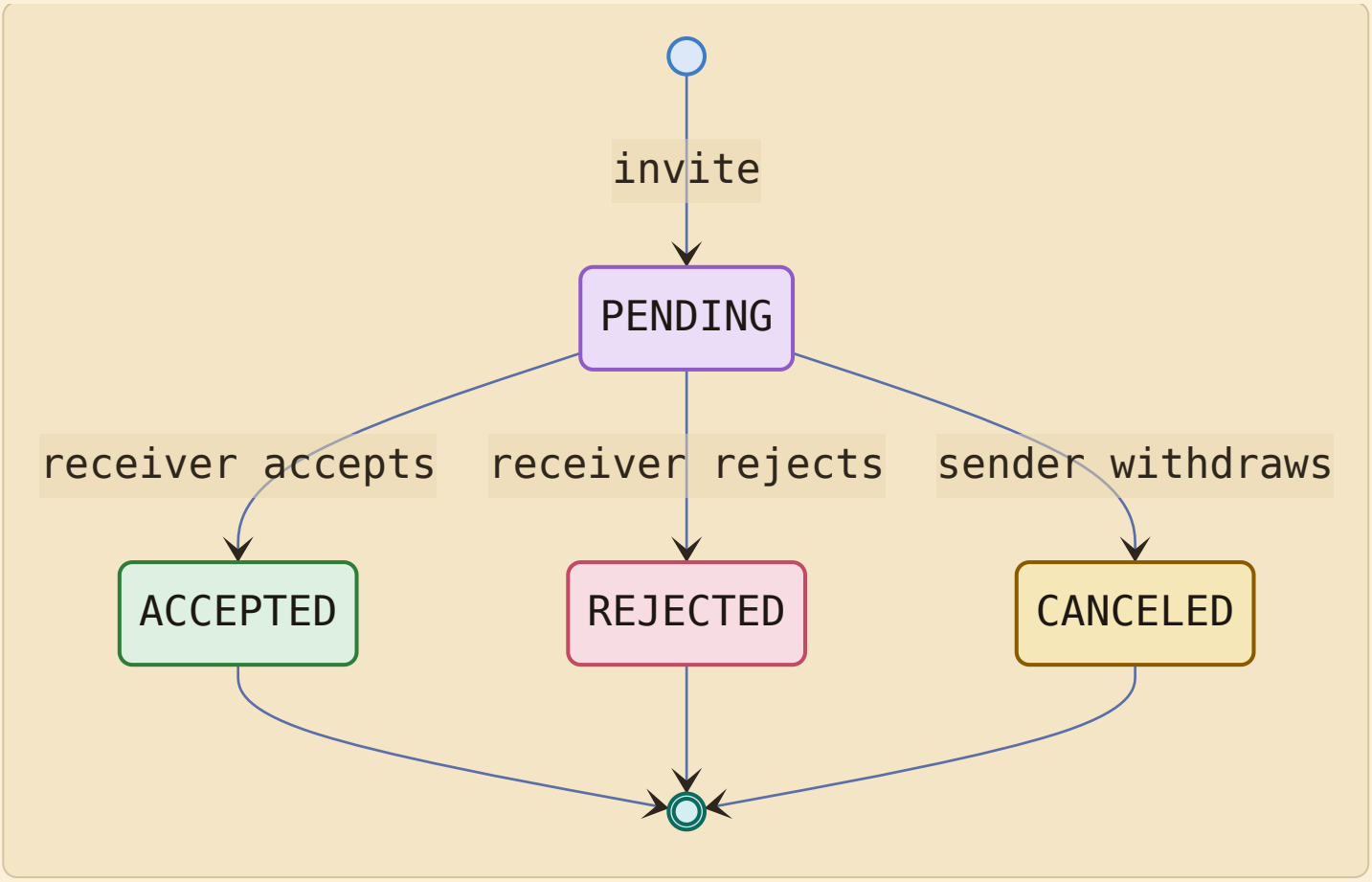


Key flows

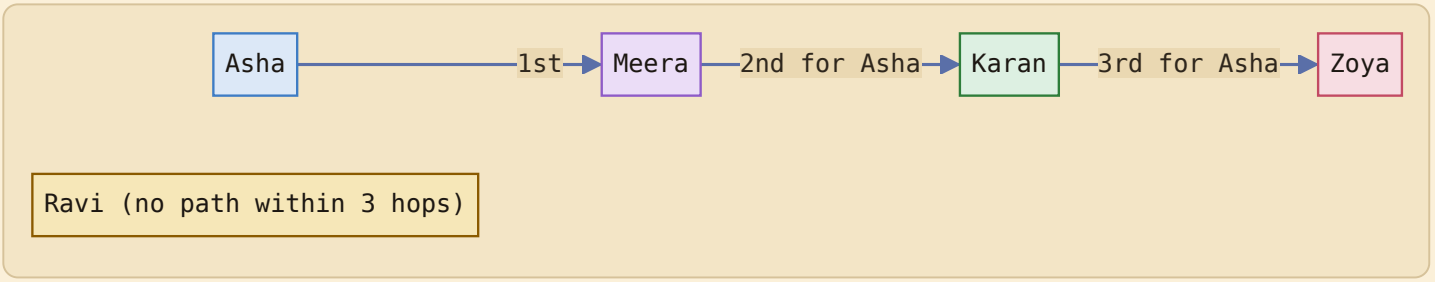
Invite and accept (the two-sided operation):



Invitation states:



Degree of connection — breadth-first search (**BFS**: explore all 1-hop neighbours, then all 2-hop, and so on) from the viewer, stopping at depth 3:



Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
`ConnectionService.accept` updates both members under one lock	Service for a two-object operation; Information Expert at the right level	Symmetry has one guardian	`member.accept(inv)` editing the other member
Following is a set on the follower	Simplicity	One-sided relationship needs no agreement or service	Reusing invitations for follows
`ConnectionInvitation` with explicit states	**State** (as an enum with allowed transitions)	Accept-twice and accept-after-reject are rejected	A boolean `accepted` flag
`SocialContent` base for `Post` and `Comment`	Inheritance for shared behaviour	Requirement 7: comments are liked too	Likes only on `Post`
One `Notification` class with a `type` field	"Field when only data differs"	Delivery is identical	A subclass per event
`NotificationChannel` interface	**Strategy**	In-app, email, push are interchangeable	`if (email) … else if (push) …`
`FeedRanker` interface	**Strategy**	"Recent" vs "Top" ranking swap at run time	Sorting code inside the feed service
Index maps key → set	Robust inverted index	Two people named Sharma both found	Key → single member (the course's bug)
`LinkedIn.rename` tells the index	Keep derived structures in sync at the write point	Search is never stale	Rebuilding the index nightly
Connection count = `connections().size()`	Derive, don't store	Cannot disagree with the set	A separate counter bumped on accept

Java implementation

Package `lld.ch18.linkedin`.

```java
// File: lld/ch18/linkedin/Profile.java
package lld.ch18.linkedin;

import java.time.YearMonth;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.Set;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicLong;

enum EmploymentType { FULL_TIME, PART_TIME, CONTRACT, INTERNSHIP }

/** A deterministic logical clock so the demo output is repeatable (a real system uses Instant.now()). */
final class Tick {
    private static final AtomicLong NOW = new AtomicLong();
    static long next() { return NOW.incrementAndGet(); }
    static long now() { return NOW.get(); }
    private Tick() {}
}

record Experience(String title, String company, String location, YearMonth from, YearMonth to, EmploymentType type) {
    boolean isCurrent() { return to == null; }
}

record Education(String school, String degree, int fromYear, int toYear) {}

record Accomplishment(String title, String description) {}

/** A skill keeps the set of members who endorsed it, so nobody endorses twice. */
final class Skill {
    private final String name;
    private final Set<Long> endorsers = new LinkedHashSet<>();
    Skill(String name) { this.name = name; }
    String name() { return name; }
    synchronized boolean endorse(long memberId) { return endorsers.add(memberId); }
    synchronized int endorsements() { return endorsers.size(); }
}

enum RecommendationStatus { REQUESTED, WRITTEN, DECLINED }

/** Created EMPTY when requested; only the named author may fill it in, once. */
final class Recommendation {
    private final long authorId;
    private final String authorName;
    private RecommendationStatus status = RecommendationStatus.REQUESTED;
    private String text;

    Recommendation(long authorId, String authorName) { this.authorId = authorId; this.authorName = authorName; }

    synchronized void write(long byMemberId, String text) {
        if (byMemberId != authorId) throw new IllegalStateException("only " + authorName + " can write this
recommendation");
        if (status != RecommendationStatus.REQUESTED) throw new IllegalStateException("recommendation is " + status);
        this.text = text;
        this.status = RecommendationStatus.WRITTEN;
    }

    long authorId() { return authorId; }
    String authorName() { return authorName; }
    synchronized RecommendationStatus status() { return status; }
    synchronized Optional<String> text() { return Optional.ofNullable(text); }
}

/**
 * Counters that are pushed in by whoever sees the event (a profile cannot
 * observe its own views or search appearances). The connection count is NOT
 * stored here: it is the size of the connection set, so it can never disagree.
 */
final class ProfileStats {
```

```
    private final AtomicInteger profileViews = new AtomicInteger();
    private final AtomicInteger searchAppearances = new AtomicInteger();
    void recordView() { profileViews.incrementAndGet(); }
    void recordSearchAppearance() { searchAppearances.incrementAndGet(); }
    int profileViews() { return profileViews.get(); }
    int searchAppearances() { return searchAppearances.get(); }
}

/** The profile owns all its parts (composition): delete the profile and they go too. */
final class Profile {
    private volatile String headline = "";
    private volatile String summary = "";
    private final List<Experience> experiences = new ArrayList<>();
    private final List<Education> educations = new ArrayList<>();
    private final Map<String, Skill> skills = new LinkedHashMap<>();
    private final List<Accomplishment> accomplishments = new ArrayList<>();
    private final List<Recommendation> recommendations = new ArrayList<>();
    private final ProfileStats stats = new ProfileStats();

    void setHeadline(String h) { headline = h; }
    void setSummary(String s) { summary = s; }
    String headline() { return headline; }
    String summary() { return summary; }

    synchronized void addExperience(Experience e) {
        if (e.to() != null && e.to().isBefore(e.from())) throw new IllegalArgumentException("experience ends before it
starts");
        experiences.add(e);
    }
    synchronized void addEducation(Education e) { educations.add(e); }
    synchronized void addAccomplishment(Accomplishment a) { accomplishments.add(a); }
    synchronized Skill addSkill(String name) { return skills.computeIfAbsent(name, Skill::new); }
    synchronized Optional<Skill> skill(String name) { return Optional.ofNullable(skills.get(name)); }
    synchronized void addRecommendation(Recommendation r) { recommendations.add(r); }

    synchronized List<Experience> experiences() { return List.copyOf(experiences); }
    synchronized List<Education> educations() { return List.copyOf(educations); }
    synchronized List<Skill> skills() { return List.copyOf(skills.values()); }
    synchronized List<Recommendation> recommendations() { return List.copyOf(recommendations); }
    ProfileStats stats() { return stats; }

    synchronized Optional<Experience> currentPosition() {
        return experiences.stream().filter(Experience::isCurrent).findFirst();
    }
}
```

```java
// File: lld/ch18/linkedin/Content.java
package lld.ch18.linkedin;

import java.util.ArrayList;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Optional;
import java.util.Set;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicLong;

/** Posts AND comments can be liked, so likes live in a shared base class, as a set of member ids. */
abstract class SocialContent {
    private static final AtomicLong IDS = new AtomicLong(1000);
    private final long id = IDS.incrementAndGet();
    private final Member author;
    private final String text;
    private final long createdAt = Tick.next();
    private final Set<Long> likes = new LinkedHashSet<>();

    SocialContent(Member author, String text) {
        if (text == null || text.isBlank()) throw new IllegalArgumentException("text cannot be empty");
        this.author = author;
        this.text = text;
    }

    long id() { return id; }
    Member author() { return author; }
    String text() { return text; }
    long createdAt() { return createdAt; }
    synchronized boolean like(long memberId) { return likes.add(memberId); }
    synchronized boolean unlike(long memberId) { return likes.remove(memberId); }
    synchronized int likeCount() { return likes.size(); }
}

final class Post extends SocialContent {
    private final List<Comment> comments = new ArrayList<>();
    private final Post sharedFrom;                  // null for an original post
    private final AtomicInteger shares = new AtomicInteger();

    Post(Member author, String text, Post sharedFrom) {
        super(author, text);
        this.sharedFrom = sharedFrom;
    }

    synchronized Comment addComment(Member by, String text) {
        Comment c = new Comment(by, text, this);
        comments.add(c);
        return c;
    }
    synchronized List<Comment> comments() { return List.copyOf(comments); }
    Optional<Post> sharedFrom() { return Optional.ofNullable(sharedFrom); }
    void recordShare() { shares.incrementAndGet(); }
    int shares() { return shares.get(); }

    @Override public String toString() {
        return "\"" + text() + "\" by " + author().name()
                + sharedFrom().map(p -> " (shared from " + p.author().name() + ")").orElse("");
    }
}

final class Comment extends SocialContent {
    private final Post post;
    Comment(Member author, String text, Post post) { super(author, text); this.post = post; }
    Post post() { return post; }
}

/** Sent to people, never posted; nobody likes a message, so it is not SocialContent. */
record Message(long id, Member sender, List<Member> recipients, String text, long sentAt) {}
```

```java
// File: lld/ch18/linkedin/Network.java
package lld.ch18.linkedin;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.Deque;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.OptionalInt;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicLong;
import java.util.function.LongFunction;

enum AccountStatus { ACTIVE, BLOCKED, BANNED, COMPROMISED, ARCHIVED, UNKNOWN }

class Member {
    private static final AtomicLong IDS = new AtomicLong();
    private final long id = IDS.incrementAndGet();
    private volatile String name;
    private final String email;
    private final Profile profile = new Profile();
    private volatile AccountStatus status = AccountStatus.ACTIVE;
    // Connections change only through ConnectionService, which keeps both sides symmetric.
    private final Set<Long> connections = ConcurrentHashMap.newKeySet();
    // Following is one-sided: nobody has to agree, so it is just a set on the follower.
    private final Set<Long> followingMembers = ConcurrentHashMap.newKeySet();
    private final Set<Long> followingCompanies = ConcurrentHashMap.newKeySet();
    private final List<Post> posts = new CopyOnWriteArrayList<>();
    private final List<Message> inbox = new CopyOnWriteArrayList<>();
    private final List<Notification> notifications = new CopyOnWriteArrayList<>();

    Member(String name, String email) { this.name = name; this.email = email; }

    long id() { return id; }
    String name() { return name; }
    void rename(String newName) { this.name = newName; }
    String email() { return email; }
    Profile profile() { return profile; }
    AccountStatus status() { return status; }
    void setStatus(AccountStatus s) { status = s; }

    Set<Long> connections() { return Set.copyOf(connections); }
    boolean isConnectedTo(Member other) { return connections.contains(other.id()); }
    void addConnection(long otherId) { connections.add(otherId); }        // package-private: only the service calls
it
    void removeConnection(long otherId) { connections.remove(otherId); }

    boolean follow(Member m) { return m.id != id && followingMembers.add(m.id()); }
    boolean unfollow(Member m) { return followingMembers.remove(m.id()); }
    boolean followCompany(Company c) { return followingCompanies.add(c.id()); }
    Set<Long> followingMembers() { return Set.copyOf(followingMembers); }
    Set<Long> followingCompanies() { return Set.copyOf(followingCompanies); }

    void addPost(Post p) { posts.add(p); }
    List<Post> posts() { return List.copyOf(posts); }
    void receive(Message m) { inbox.add(m); }
    List<Message> inbox() { return List.copyOf(inbox); }
    void notify(Notification n) { notifications.add(n); }
    List<Notification> notifications() { return List.copyOf(notifications); }

    @Override public String toString() { return name; }
}

final class Admin extends Member {
    Admin(String name, String email) { super(name, email); }
}

enum InvitationStatus {
```

```
    PENDING, ACCEPTED, REJECTED, CANCELED;
    boolean canMoveTo(InvitationStatus next) { return this == PENDING && next != PENDING; }
}

/** A connection request is an object with a state over time: it can remember that it was rejected. */
final class ConnectionInvitation {
    private static final AtomicLong IDS = new AtomicLong();
    private final long id = IDS.incrementAndGet();
    private final Member from;
    private final Member to;
    private final String note;
    private InvitationStatus status = InvitationStatus.PENDING;

    ConnectionInvitation(Member from, Member to, String note) { this.from = from; this.to = to; this.note = note; }

    long id() { return id; }
    Member from() { return from; }
    Member to() { return to; }
    String note() { return note; }
    synchronized InvitationStatus status() { return status; }

    synchronized void moveTo(InvitationStatus next) {
        if (!status.canMoveTo(next)) throw new IllegalStateException("invitation is already " + status);
        status = next;
    }
}

/**
 * Accepting an invitation changes TWO members, so it belongs to neither: it lives here.
 * This service is the only code that must be right for connections to stay symmetric.
 */
final class ConnectionService {
    private final Map<Long, ConnectionInvitation> invitations = new ConcurrentHashMap<>();
    private final NotificationService notifications;
    private final LongFunction<Member> lookup;

    ConnectionService(NotificationService notifications, LongFunction<Member> lookup) {
        this.notifications = notifications;
        this.lookup = lookup;
    }

    synchronized ConnectionInvitation invite(Member from, Member to, String note) {
        if (from.id() == to.id()) throw new IllegalArgumentException("cannot connect to yourself");
        if (from.isConnectedTo(to)) throw new IllegalStateException(from + " and " + to + " are already connected");
        if (pendingBetween(from, to)) throw new IllegalStateException("an invitation between " + from + " and " + to +
" is already pending");
        ConnectionInvitation inv = new ConnectionInvitation(from, to, note);
        invitations.put(inv.id(), inv);
        notifications.send(to, NotificationType.CONNECTION_INVITATION, from + " wants to connect: \"" + note + "\"");
        return inv;
    }

    /** One lock for the whole graph change: both sides are updated together or not at all. */
    synchronized void accept(ConnectionInvitation inv, Member by) {
        if (by.id() != inv.to().id()) throw new IllegalStateException("only " + inv.to() + " can accept");
        inv.moveTo(InvitationStatus.ACCEPTED);
        inv.from().addConnection(inv.to().id());
        inv.to().addConnection(inv.from().id());
    }

    synchronized void reject(ConnectionInvitation inv, Member by) {
        if (by.id() != inv.to().id()) throw new IllegalStateException("only " + inv.to() + " can reject");
        inv.moveTo(InvitationStatus.REJECTED);
    }

    synchronized void withdraw(ConnectionInvitation inv, Member by) {
        if (by.id() != inv.from().id()) throw new IllegalStateException("only " + inv.from() + " can withdraw");
        inv.moveTo(InvitationStatus.CANCELED);
    }

    synchronized void removeConnection(Member a, Member b) {
        a.removeConnection(b.id());
        b.removeConnection(a.id());
    }
```

```
    boolean pendingBetween(Member a, Member b) {
        return invitations.values().stream().anyMatch(i -> i.status() == InvitationStatus.PENDING
                && ((i.from() == a && i.to() == b) || (i.from() == b && i.to() == a)));
    }

    /** Degree of connection (1st, 2nd, 3rd) by breadth-first search; empty means "out of network". */
    OptionalInt degree(Member from, Member to) {
        if (from.id() == to.id()) return OptionalInt.of(0);
        Set<Long> seen = new HashSet<>(Set.of(from.id()));
        Deque<long[]> queue = new ArrayDeque<>();                // {memberId, depth}
        queue.add(new long[]{from.id(), 0});
        while (!queue.isEmpty()) {
            long[] cur = queue.poll();
            if (cur[1] == 3) continue;                           // LinkedIn stops at 3rd degree
            for (long next : lookup.apply(cur[0]).connections()) {
                if (next == to.id()) return OptionalInt.of((int) cur[1] + 1);
                if (seen.add(next)) queue.add(new long[]{next, cur[1] + 1});
            }
        }
        return OptionalInt.empty();
    }

    Set<Long> mutualConnections(Member a, Member b) {
        Set<Long> mutual = new HashSet<>(a.connections());
        mutual.retainAll(b.connections());
        return mutual;
    }

    /** "People you may know": 2nd-degree members ranked by mutual connections, then name. */
    List<Member> peopleYouMayKnow(Member me, int limit) {
        Map<Long, Integer> mutualCount = new HashMap<>();
        for (long friend : me.connections()) {
            for (long candidate : lookup.apply(friend).connections()) {
                if (candidate != me.id() && !me.connections().contains(candidate)) mutualCount.merge(candidate, 1,
Integer::sum);
            }
        }
        List<Member> result = new ArrayList<>();
        mutualCount.keySet().stream().map(lookup::apply)
                .filter(m -> !pendingBetween(me, m))
                .sorted(Comparator.comparing((Member m) -> -mutualCount.get(m.id())).thenComparing(Member::name))
                .limit(limit).forEach(result::add);
        return result;
    }
}
```

```java
// File: lld/ch18/linkedin/Pages.java
package lld.ch18.linkedin;

import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicLong;

final class Company {
    private static final AtomicLong IDS = new AtomicLong(500);
    private final long id = IDS.incrementAndGet();
    private final String name;
    private final Member createdBy;
    private final Set<Long> followers = ConcurrentHashMap.newKeySet();
    private final List<JobPosting> jobs = new CopyOnWriteArrayList<>();     // owned: gone with the page

    Company(String name, Member createdBy) { this.name = name; this.createdBy = createdBy; }

    long id() { return id; }
    String name() { return name; }
    Member createdBy() { return createdBy; }
    void addFollower(long memberId) { followers.add(memberId); }
    Set<Long> followers() { return Set.copyOf(followers); }
    int followerCount() { return followers.size(); }          // counted, never stored

    JobPosting postJob(Member by, String title, String location, EmploymentType type) {
        if (by.id() != createdBy.id()) throw new IllegalStateException(by + " is not an admin of " + name);
        JobPosting job = new JobPosting(this, title, location, type);
        jobs.add(job);
        return job;
    }
    List<JobPosting> jobs() { return List.copyOf(jobs); }
    @Override public String toString() { return name; }
}

enum ApplicationStatus { APPLIED, IN_REVIEW, REJECTED, HIRED }

record JobApplication(Member applicant, long appliedAt, ApplicationStatus status) {}

final class JobPosting {
    private static final AtomicLong IDS = new AtomicLong(700);
    private final long id = IDS.incrementAndGet();
    private final Company company;
    private final String title;
    private final String location;
    private final EmploymentType type;
    private final Map<Long, JobApplication> applications = new LinkedHashMap<>();
    private volatile boolean open = true;

    JobPosting(Company company, String title, String location, EmploymentType type) {
        this.company = company; this.title = title; this.location = location; this.type = type;
    }

    long id() { return id; }
    Company company() { return company; }
    String title() { return title; }
    String location() { return location; }
    EmploymentType type() { return type; }
    boolean isOpen() { return open; }
    void close() { open = false; }

    synchronized JobApplication apply(Member m) {
        if (!open) throw new IllegalStateException("job " + title + " is closed");
        if (applications.containsKey(m.id())) throw new IllegalStateException(m + " already applied to " + title);
        JobApplication a = new JobApplication(m, Tick.next(), ApplicationStatus.APPLIED);
        applications.put(m.id(), a);
        return a;
    }
    synchronized int applicantCount() { return applications.size(); }
    @Override public String toString() { return title + " @ " + company + " (" + location + ", " + type + ")"; }
```

```
}

final class Group {
    private static final AtomicLong IDS = new AtomicLong(900);
    private final long id = IDS.incrementAndGet();
    private final String name;
    private final long ownerId;
    private final Set<Long> members = ConcurrentHashMap.newKeySet();

    Group(String name, Member owner) {
        this.name = name;
        this.ownerId = owner.id();
        members.add(owner.id());
    }

    long id() { return id; }
    String name() { return name; }
    boolean join(Member m) { return members.add(m.id()); }
    boolean leave(Member m) {
        if (m.id() == ownerId) throw new IllegalStateException("the owner cannot leave their own group");
        return members.remove(m.id());
    }
    int memberCount() { return members.size(); }
}
```

```java
// File: lld/ch18/linkedin/Services.java
package lld.ch18.linkedin;

import java.util.Arrays;
import java.util.Comparator;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.stream.Stream;

enum NotificationType { NEW_MESSAGE, CONNECTION_INVITATION, NEW_COMMENT, RECOMMENDATION_REQUEST }

/** Only the data differs between notification kinds, so the kind is a field, not a subclass. */
record Notification(NotificationType type, String text, long at) {}

/** Strategy for delivery: in-app, email, push... */
interface NotificationChannel { void deliver(Member to, Notification n); }

final class InAppChannel implements NotificationChannel {
    @Override public void deliver(Member to, Notification n) { to.notify(n); }
}

final class EmailChannel implements NotificationChannel {
    @Override public void deliver(Member to, Notification n) {
        System.out.println("    [email to " + to.email() + "] " + n.type() + ": " + n.text());
    }
}

final class NotificationService {
    private final List<NotificationChannel> channels = new CopyOnWriteArrayList<>();
    NotificationService(List<NotificationChannel> channels) { this.channels.addAll(channels); }
    void send(Member to, NotificationType type, String text) {
        Notification n = new Notification(type, text, Tick.next());
        channels.forEach(c -> c.deliver(to, n));
    }
}

interface Search {
    List<Member> searchMembers(String name);
    List<Company> searchCompanies(String name);
    List<JobPosting> searchJobs(String title);
}

/**
 * Inverted indexes. RULE: every key maps to a LIST, even for one match,
 * so the second "Sharma" never breaks the index.
 */
final class SearchIndex implements Search {
    private final Map<String, Set<Member>> members = new ConcurrentHashMap<>();
    private final Map<String, Set<Company>> companies = new ConcurrentHashMap<>();
    private final Map<String, Set<JobPosting>> jobs = new ConcurrentHashMap<>();

    void addMember(Member m) { words(m.name()).forEach(w -> members.computeIfAbsent(w, k ->
ConcurrentHashMap.newKeySet()).add(m)); }
    void removeMember(Member m, String oldName) { words(oldName).forEach(w -> members.getOrDefault(w,
Set.of()).remove(m)); }
    void addCompany(Company c) { words(c.name()).forEach(w -> companies.computeIfAbsent(w, k ->
ConcurrentHashMap.newKeySet()).add(c)); }
    void addJob(JobPosting j) { words(j.title()).forEach(w -> jobs.computeIfAbsent(w, k ->
ConcurrentHashMap.newKeySet()).add(j)); }

    @Override public List<Member> searchMembers(String q) {
        return match(members, q).sorted(Comparator.comparing(Member::name)).toList();
    }
    @Override public List<Company> searchCompanies(String q) {
        return match(companies, q).sorted(Comparator.comparing(Company::name)).toList();
    }
    @Override public List<JobPosting> searchJobs(String q) {
        return match(jobs, q).filter(JobPosting::isOpen).sorted(Comparator.comparingLong(JobPosting::id)).toList();
    }
```

```
    /** Items matching ALL words of the query. */
    private static <T> Stream<T> match(Map<String, Set<T>> index, String query) {
        List<String> ws = words(query);
        if (ws.isEmpty()) return Stream.empty();
        return index.getOrDefault(ws.get(0), Set.of()).stream()
                .filter(item -> ws.stream().allMatch(w -> index.getOrDefault(w, Set.of()).contains(item)));
    }

    private static List<String> words(String s) {
        return Arrays.stream(s.toLowerCase(Locale.ROOT).split("\\W+")).filter(w -> !w.isEmpty()).toList();
    }
}

/** Strategy for ordering the feed. */
interface FeedRanker {
    String name();
    Comparator<Post> order(Member viewer);
}

final class ChronologicalRanker implements FeedRanker {
    @Override public String name() { return "latest"; }
    @Override public Comparator<Post> order(Member viewer) {
        return Comparator.comparingLong(Post::createdAt).reversed();
    }
}

/** Engagement score with a simple age decay; 1st-degree connections get a boost over follows. */
final class EngagementRanker implements FeedRanker {
    @Override public String name() { return "top"; }
    @Override public Comparator<Post> order(Member viewer) {
        return Comparator.comparingDouble((Post p) -> -score(p,
viewer)).thenComparing(Comparator.comparingLong(Post::createdAt).reversed());
    }
    private static double score(Post p, Member viewer) {
        double engagement = p.likeCount() + 2.0 * p.comments().size() + 3.0 * p.shares();
        double affinity = viewer.isConnectedTo(p.author()) ? 1.5 : 1.0;
        double age = Tick.now() - p.createdAt();
        return (1 + engagement) * affinity / (1 + age / 20.0);
    }
}

/**
 * Pull model ("fan-out on read"): build the feed when it is opened from the
 * posts of connections and followed members. Fine for small networks; see the
 * README for the push model and the hybrid real sites use.
 */
final class FeedService {
    private final java.util.function.LongFunction<Member> lookup;
    FeedService(java.util.function.LongFunction<Member> lookup) { this.lookup = lookup; }

    List<Post> feed(Member viewer, FeedRanker ranker, int limit) {
        Set<Long> sources = new java.util.HashSet<>(viewer.connections());
        sources.addAll(viewer.followingMembers());
        return sources.stream()
                .map(lookup::apply)
                .filter(m -> m.status() == AccountStatus.ACTIVE)
                .flatMap(m -> m.posts().stream())
                .sorted(ranker.order(viewer))
                .limit(limit)
                .toList();
    }
}
```

```java
// File: lld/ch18/linkedin/LinkedIn.java
package lld.ch18.linkedin;

import java.util.List;
import java.util.Map;
import java.util.NoSuchElementException;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;

/** Facade that wires the services and checks account status before every action. */
final class LinkedIn {
    private final Map<Long, Member> members = new ConcurrentHashMap<>();
    private final AtomicLong messageIds = new AtomicLong();
    final NotificationService notifications;
    final ConnectionService connections;
    final SearchIndex search = new SearchIndex();
    final FeedService feeds;

    LinkedIn(List<NotificationChannel> channels) {
        notifications = new NotificationService(channels);
        connections = new ConnectionService(notifications, this::member);
        feeds = new FeedService(this::member);
    }

    Member member(long id) {
        Member m = members.get(id);
        if (m == null) throw new NoSuchElementException("no member " + id);
        return m;
    }

    <M extends Member> M join(M m) {
        members.put(m.id(), m);
        search.addMember(m);
        return m;
    }

    /** The code that changes a name must tell the index, or search goes stale. */
    void rename(Member m, String newName) {
        String old = m.name();
        search.removeMember(m, old);
        m.rename(newName);
        search.addMember(m);
    }

    List<Member> searchMembers(Member viewer, String query) {
        List<Member> found = search.searchMembers(query);
        found.stream().filter(m -> m != viewer).forEach(m -> m.profile().stats().recordSearchAppearance());
        return found;
    }

    void viewProfile(Member viewer, Member owner) {
        if (viewer != owner) owner.profile().stats().recordView();
    }

    void endorse(Member by, Member owner, String skillName) {
        active(by);
        if (!by.isConnectedTo(owner)) throw new IllegalStateException("only 1st-degree connections can endorse " +
owner);
        Skill s = owner.profile().skill(skillName).orElseThrow(() -> new NoSuchElementException(owner + " has no skill
" + skillName));
        if (!s.endorse(by.id())) throw new IllegalStateException(by + " already endorsed " + skillName);
    }

    Recommendation requestRecommendation(Member requester, Member author) {
        active(requester);
        if (!requester.isConnectedTo(author)) throw new IllegalStateException("you can only ask your connections");
        Recommendation r = new Recommendation(author.id(), author.name());
        requester.profile().addRecommendation(r);
        notifications.send(author, NotificationType.RECOMMENDATION_REQUEST, requester + " asked you for a
recommendation");
        return r;
    }
```

```
    Post post(Member author, String text) {
        active(author);
        Post p = new Post(author, text, null);
        author.addPost(p);
        return p;
    }

    Post share(Member by, Post original, String text) {
        active(by);
        Post p = new Post(by, text, original);
        original.recordShare();
        by.addPost(p);
        return p;
    }

    Comment comment(Member by, Post post, String text) {
        active(by);
        Comment c = post.addComment(by, text);
        if (post.author() != by) notifications.send(post.author(), NotificationType.NEW_COMMENT, by + " commented: \""
+ text + "\"");
        return c;
    }

    Message message(Member from, List<Member> to, String text) {
        active(from);
        for (Member r : to) {
            if (!from.isConnectedTo(r)) throw new IllegalStateException(r + " is not a connection (that would need
InMail)");
        }
        Message m = new Message(messageIds.incrementAndGet(), from, List.copyOf(to), text, Tick.next());
        for (Member r : to) {
            r.receive(m);
            notifications.send(r, NotificationType.NEW_MESSAGE, "from " + from + ": " + text);
        }
        return m;
    }

    Company createCompany(Member by, String name) {
        active(by);
        Company c = new Company(name, by);
        search.addCompany(c);
        return c;
    }

    JobPosting postJob(Member by, Company c, String title, String location, EmploymentType type) {
        JobPosting j = c.postJob(by, title, location, type);
        search.addJob(j);
        return j;
    }

    void followCompany(Member m, Company c) {
        if (m.followCompany(c)) c.addFollower(m.id());
    }

    void block(Admin admin, Member target) { target.setStatus(AccountStatus.BLOCKED); }

    private static void active(Member m) {
        if (m.status() != AccountStatus.ACTIVE) throw new IllegalStateException(m + " is " + m.status());
    }
}
```

```java
// File: lld/ch18/linkedin/Main.java
package lld.ch18.linkedin;

import java.time.YearMonth;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        LinkedIn li = new LinkedIn(List.of(new InAppChannel(), new EmailChannel()));
        Member asha = li.join(new Member("Asha Sharma", "asha@example.com"));
        Member ravi = li.join(new Member("Ravi Sharma", "ravi@example.com"));
        Member meera = li.join(new Member("Meera Iyer", "meera@example.com"));
        Member karan = li.join(new Member("Karan Mehta", "karan@example.com"));
        Member zoya = li.join(new Member("Zoya Khan", "zoya@example.com"));
        Admin admin = li.join(new Admin("Site Admin", "admin@example.com"));

        section("1. Profile");
        Profile p = asha.profile();
        p.setHeadline("Backend engineer | Java | Distributed systems");
        p.addExperience(new Experience("Software Engineer", "Acme Cloud", "Bengaluru", YearMonth.of(2022, 7), null,
EmploymentType.FULL_TIME));
        p.addEducation(new Education("IIT Madras", "B.Tech CSE", 2018, 2022));
        p.addSkill("Java");
        p.addSkill("System Design");
        attempt(() -> p.addExperience(new Experience("Intern", "X", "Pune", YearMonth.of(2021, 6), YearMonth.of(2021,
1), EmploymentType.INTERNSHIP)));
        System.out.println(asha + ": " + p.headline() + ", now at " +
p.currentPosition().map(Experience::company).orElse("-"));

        section("2. Connections");
        ConnectionInvitation i1 = li.connections.invite(asha, meera, "We met at the Java meetup");
        li.connections.accept(i1, meera);
        attempt(() -> li.connections.accept(i1, meera));                // accepted once, never again
        li.connections.accept(li.connections.invite(meera, karan, "Ex-colleague"), karan);
        li.connections.accept(li.connections.invite(karan, zoya, "Hi"), zoya);
        ConnectionInvitation pending = li.connections.invite(asha, ravi, "Same surname!");
        attempt(() -> li.connections.invite(ravi, asha, "Hello"));      // pending the other way
        attempt(() -> li.connections.accept(pending, asha));            // only the receiver can accept
        System.out.println("degree asha->meera=" + li.connections.degree(asha, meera).getAsInt()
                + ", asha->karan=" + li.connections.degree(asha, karan).getAsInt()
                + ", asha->zoya=" + li.connections.degree(asha, zoya).getAsInt()
                + ", asha->ravi=" + (li.connections.degree(asha, ravi).isPresent() ? "?" : "out of network"));
        System.out.println("people asha may know: " + li.connections.peopleYouMayKnow(asha, 5));

        section("3. Endorsements and recommendations");
        li.endorse(meera, asha, "Java");
        attempt(() -> li.endorse(meera, asha, "Java"));
        attempt(() -> li.endorse(karan, asha, "Java"));
        Recommendation rec = li.requestRecommendation(asha, meera);
        attempt(() -> rec.write(karan.id(), "Great engineer"));
        rec.write(meera.id(), "Asha owns problems end to end.");
        System.out.println("Java endorsements=" + p.skill("Java").orElseThrow().endorsements()
                + ", recommendation from " + rec.authorName() + ": " + rec.text().orElse("") + " [" + rec.status() +
"]");

        section("4. Posts, comments, likes, feed");
        karan.follow(asha);                                             // follow = one-sided, no approval
        Post launch = li.post(meera, "We are hiring Java engineers at Acme Cloud!");
        Post tip = li.post(asha, "Tip: prefer composition over inheritance.");
        Comment c = li.comment(asha, launch, "Sharing with my network");
        launch.like(asha.id());
        c.like(meera.id());                                             // comments can be liked too
        System.out.println("double like accepted? " + launch.like(asha.id()));
        li.share(karan, tip, "Good advice");
        tip.like(meera.id());
        tip.like(zoya.id());
        System.out.println("feed(asha, latest): " + li.feeds.feed(asha, new ChronologicalRanker(), 5));
        System.out.println("feed(karan, top):   " + li.feeds.feed(karan, new EngagementRanker(), 5));

        section("5. Messaging");
        li.message(asha, List.of(meera), "Coffee on Friday?");
        attempt(() -> li.message(asha, List.of(karan), "Hi Karan"));
```

```
        section("6. Companies, jobs, groups");
        Company acme = li.createCompany(meera, "Acme Cloud");
        li.followCompany(asha, acme);
        JobPosting job = li.postJob(meera, acme, "Senior Java Engineer", "Bengaluru", EmploymentType.FULL_TIME);
        attempt(() -> li.postJob(asha, acme, "Fake job", "Nowhere", EmploymentType.CONTRACT));
        System.out.println("search jobs 'java engineer': " + li.search.searchJobs("java engineer"));
        job.apply(asha);
        attempt(() -> job.apply(asha));
        Group group = new Group("Java Developers India", meera);
        group.join(asha);
        attempt(() -> group.leave(meera));
        System.out.println(acme + " followers=" + acme.followerCount() + ", applicants=" + job.applicantCount()
                + ", group members=" + group.memberCount());

        section("7. Search and profile stats");
        System.out.println("search 'sharma' (by meera): " + li.searchMembers(meera, "sharma"));
        li.rename(zoya, "Zoya Ali");
        System.out.println("search 'khan' after rename: " + li.searchMembers(meera, "khan")
                + ", search 'ali': " + li.searchMembers(meera, "ali"));
        li.viewProfile(meera, asha);
        li.viewProfile(karan, asha);
        System.out.println("asha stats: views=" + p.stats().profileViews() + ", connections=" +
asha.connections().size()
                + ", search appearances=" + p.stats().searchAppearances());
        System.out.println("meera's in-app notifications: " +
meera.notifications().stream().map(Notification::type).toList());

        section("8. Admin");
        li.block(admin, karan);
        attempt(() -> li.post(karan, "spam spam"));
        System.out.println("feed(meera) hides blocked Karan's share: " + li.feeds.feed(meera, new
ChronologicalRanker(), 5));
    }

    static void section(String t) { System.out.println("\n== " + t + " =="); }

    static void attempt(Runnable r) {
        try { r.run(); } catch (RuntimeException e) { System.out.println("  REJECTED: " + e.getMessage()); }
    }
}
```

Output:

```
== 1. Profile ==
  REJECTED: experience ends before it starts
Asha Sharma: Backend engineer | Java | Distributed systems, now at Acme Cloud

== 2. Connections ==
    [email to meera@example.com] CONNECTION_INVITATION: Asha Sharma wants to connect: "We met at the Java meetup"
  REJECTED: invitation is already ACCEPTED
    [email to karan@example.com] CONNECTION_INVITATION: Meera Iyer wants to connect: "Ex-colleague"
    [email to zoya@example.com] CONNECTION_INVITATION: Karan Mehta wants to connect: "Hi"
    [email to ravi@example.com] CONNECTION_INVITATION: Asha Sharma wants to connect: "Same surname!"
  REJECTED: an invitation between Ravi Sharma and Asha Sharma is already pending
  REJECTED: only Ravi Sharma can accept
degree asha->meera=1, asha->karan=2, asha->zoya=3, asha->ravi=out of network
people asha may know: [Karan Mehta]

== 3. Endorsements and recommendations ==
  REJECTED: Meera Iyer already endorsed Java
  REJECTED: only 1st-degree connections can endorse Asha Sharma
    [email to meera@example.com] RECOMMENDATION_REQUEST: Asha Sharma asked you for a recommendation
  REJECTED: only Meera Iyer can write this recommendation
Java endorsements=1, recommendation from Meera Iyer: Asha owns problems end to end. [WRITTEN]

== 4. Posts, comments, likes, feed ==
    [email to meera@example.com] NEW_COMMENT: Asha Sharma commented: "Sharing with my network"
double like accepted? false
feed(asha, latest): ["We are hiring Java engineers at Acme Cloud!" by Meera Iyer]
feed(karan, top):   ["Tip: prefer composition over inheritance." by Asha Sharma, "We are hiring Java engineers at Acme
Cloud!" by Meera Iyer]

== 5. Messaging ==
    [email to meera@example.com] NEW_MESSAGE: from Asha Sharma: Coffee on Friday?
  REJECTED: Karan Mehta is not a connection (that would need InMail)

== 6. Companies, jobs, groups ==
  REJECTED: Asha Sharma is not an admin of Acme Cloud
search jobs 'java engineer': [Senior Java Engineer @ Acme Cloud (Bengaluru, FULL_TIME)]
  REJECTED: Asha Sharma already applied to Senior Java Engineer
  REJECTED: the owner cannot leave their own group
Acme Cloud followers=1, applicants=1, group members=2

== 7. Search and profile stats ==
search 'sharma' (by meera): [Asha Sharma, Ravi Sharma]
search 'khan' after rename: [], search 'ali': [Zoya Ali]
asha stats: views=2, connections=1, search appearances=1
meera's in-app notifications: [CONNECTION_INVITATION, RECOMMENDATION_REQUEST, NEW_COMMENT, NEW_MESSAGE]

== 8. Admin ==
  REJECTED: Karan Mehta is BLOCKED
feed(meera) hides blocked Karan's share: ["Tip: prefer composition over inheritance." by Asha Sharma]
```

Walking through the important parts

- `ConnectionService.invite` refuses a request when one is already pending *in either direction* — in the output Ravi's counter-invite to Asha is rejected.
- `accept` checks that the caller is the receiver, moves the invitation to ACCEPTED (a second accept fails because only PENDING can move), and adds each member to the other's set inside one `synchronized` block.
- `degree` is a BFS over connection sets that stops at depth 3: Asha → Meera is 1st, Karan 2nd, Zoya 3rd, and Ravi (only a pending invitation) is "out of network".
- `peopleYouMayKnow` counts mutual connections for every 2nd-degree member, skips anyone with a pending invitation, and sorts by count then name, so the list is stable.
- `endorse` requires a 1st-degree connection and uses the skill's endorser set to reject a second endorsement.
- `Recommendation.write` accepts only the named author and only once.
- `SearchIndex` maps each lowercase word to a *set*, so "sharma" returns both Asha and Ravi; `rename` removes the old words before adding the new ones, so "khan" finds nobody after Zoya's rename.
- `FeedService` is a **pull** feed: it gathers posts from connections and followed members when asked, drops blocked authors, then sorts with the chosen `FeedRanker`. Karan sees Asha's tip because he *follows* her,

even though they are not connected.

Concurrency & edge cases

- **Symmetric connections:** all graph changes (`invite`, `accept`, `reject`, `withdraw`, `removeConnection`) are `synchronized` on the single service, so two concurrent accepts or an accept racing a withdraw cannot leave a one-way edge. In a distributed system (one spread across many machines that talk over a network), write both edges (an **edge** is one link in a graph) in one database transaction (or one row keyed by the ordered pair of ids).
- **Double invitations** in either direction are refused while one is pending.
- **Likes, endorsements, applications:** sets and maps keyed by member id make repeats harmless or rejected; their mutators are `synchronized`.
- **Collections on `Member`** are concurrent sets or `CopyOnWriteArrayList`, and getters return copies, so callers cannot mutate internal state.
- **Stale search:** handled by `rename`; the general rule is "whatever changes the data tells the index" (in production, via an event).
- **Blocked members:** writes are refused and their posts vanish from feeds.
- **Validation:** an experience that ends before it starts is rejected; blank posts are rejected.

Extensions & follow-up questions

- **"Build the feed for 900 million members."** — Two classic strategies. **Fan-out on write (push):** when someone posts, append the post id to each connection's precomputed feed list — fast reads, expensive for members with huge followings. **Fan-out on read (pull):** assemble the feed when it is opened, as our `FeedService` does — cheap writes, slower reads. Large networks use a **hybrid**: push for ordinary members, pull for very popular accounts, merged at read time.
- **"People you may know at scale."** — Compute suggestions offline (in a background batch job, not while the user waits) on a schedule (friends-of-friends plus signals such as shared companies and schools) and store them; the live page reads the stored list.
- **"Connection limit."** — LinkedIn caps members at 30,000 connections; enforce in `accept`.
- **"InMail to non-connections."** — A premium credit balance on `Member`; `message` spends a credit instead of rejecting.
- **"Who viewed my profile."** — Store view events (viewer, time) instead of a bare counter, then derive the count.
- **"Job recommendations."** — A Strategy that matches a member's skills and current title against open postings, ranked by overlap.

Real-world example

LinkedIn shows 1st, 2nd and 3rd-degree badges next to names in search results — the exact BFS depth our `degree` computes — and its "People You May Know" panel is driven by shared connections and other signals. Messaging someone who is not a connection usually requires paid **InMail** (some members open their profile to free messages). And the event stream that feeds notifications, analytics and the feed at LinkedIn gave rise to **Apache Kafka**, which LinkedIn open-sourced (published free, with its source code, for anyone to use) in 2011.

In the interview

Minutes	What to do
0–5	Clarify connect vs follow, degrees, who can message and endorse
5–15	Profile composition, `SocialContent` (comments can be liked), invitation as an object with state
15–25	**Say it out loud:** accepting changes two objects, so it belongs to neither — `ConnectionService`; following is one-sided, so it needs no service
25–35	Search index (key → list), degrees by BFS, people you may know
35–45	Feed: pull vs push vs hybrid; notifications via Strategy channels; concurrency

30-second answer: "Members own profiles made of small parts. Posts and comments share a base class because both are liked, with likes as a set. A connection request is an object with states. Accepting changes both members, so it lives in a `ConnectionService` that updates both sides under one lock; following is one-sided, just a set. Search is an inverted index mapping each word to a list. Degrees come from a BFS to depth 3. The feed is a pull model with a pluggable ranking Strategy, and I'd move to a push/pull hybrid at scale."

Follow-ups with answers

- *Why not store `connectionCount` on the profile?* — It is `connections.size()`; a stored copy can drift.
- *How do you compute a 2nd-degree badge quickly for every search result?* — Precompute or cache each member's 1st-degree set; a result is 2nd-degree if its connection set intersects yours.
- *What if the same name appears twice?* — The index maps words to lists (sets), never to a single member.

Self-check

- **Q:** Why is accepting an invitation not a method on `Member`? **A:** It updates two members; a service keeps both sides consistent in one place.
- **Q:** What is the difference between connecting and following? **A:** Connecting is mutual and needs acceptance; following is one-sided and immediate.
- **Q:** Why does `Recommendation` start without text? **A:** Because it is *requested* first; the author fills it in later.
- **Q:** What is the rule for search index values? **A:** Always a collection, even for one match.
- **Q:** Pull vs push feed — which does `FeedService` implement, and what is its cost? **A:** Pull; cheap writes, but reading scans every connection's posts each time.

TL;DR

LinkedIn = profile composition + `SocialContent` (likes as sets) + invitation **State** + `ConnectionService` for the two-sided accept (following is just a set) + BFS degrees + people-you-may-know + inverted-index search (key → list) + notification and feed-ranking **Strategies** + a facade.

18.4 Design Facebook

Figure — A browser showing the Facebook home page, the cover image of this case study (course diagram)

Facebook is an online social network where users connect with **friends** (a mutual relationship), **follow** people and **pages**, join **groups**, and read and write posts through a website or mobile apps. Compared with

LinkedIn, the new challenges are **privacy** (who may see each post), a real **news feed**, **pages** with ratings, **groups** (public and private), and **friend suggestions**.

Problem & clarifying questions

Question to ask	Answer we assume
Friend vs follow?	Friendship is mutual (request + accept); following is one-sided
Who can see a post?	The author chooses an **audience**: Public, Friends, a custom **privacy list**, or Only me
What is a privacy list?	A named subset of the author's friends (e.g. "Close friends"); a post linked to it is visible only to those friends
Pages?	Created by members; others follow them and add **recommendations** (a 1–5 rating and text)
Groups?	Public (join at once) or private (owner approves); posts inside a private group are visible only to members
Search?	Members, groups and pages by name; posts by a word — respecting privacy
News feed?	Posts from friends, followed members, followed pages and joined groups, filtered by privacy, ranked
Extended requirement?	Suggest people you may know (friends of friends)

Requirements

Functional (the course's twelve plus the extended one, in our words)

1. A member fills in a profile: basic details, jobs, schooling and places lived.
2. Any user can search members, groups or pages by name.
3. Members send friend requests and accept or reject them.
4. Members can follow other members without becoming friends.
5. Members create groups and pages, join groups, and follow pages.
6. Members create posts to share with their friends.
7. Members can comment on posts; posts **and comments** can both be liked or shared.
8. Members create privacy lists of friends; a post linked to a list is visible only to that list.
9. Members send messages to other members.
10. Members add a recommendation to any page.
11. A member is notified about each new message, each friend request, and each comment on one of their posts.
12. Members can search posts for a word.
13. **Extended:** write a function that suggests connections for a member.

Non-functional

- **Privacy is never leaked** — through the feed, search, comments, likes or shares.
- **Feed latency** — the feed must load fast even for members with hundreds of friends.
- **Consistency of the friend graph** — friendship is symmetric.
- **Stable output** — suggestions and feeds must not reshuffle randomly between page loads.

Use cases

Three actors: **Member**, **Admin** (blocks/unblocks members) and **System** (sends notifications).

Top use cases: add/update profile; search members, groups, pages and posts; send friend requests; follow/unfollow a member or page; send messages to friends; create posts, like and comment on visible posts; send notifications; create a page; recommend a page.

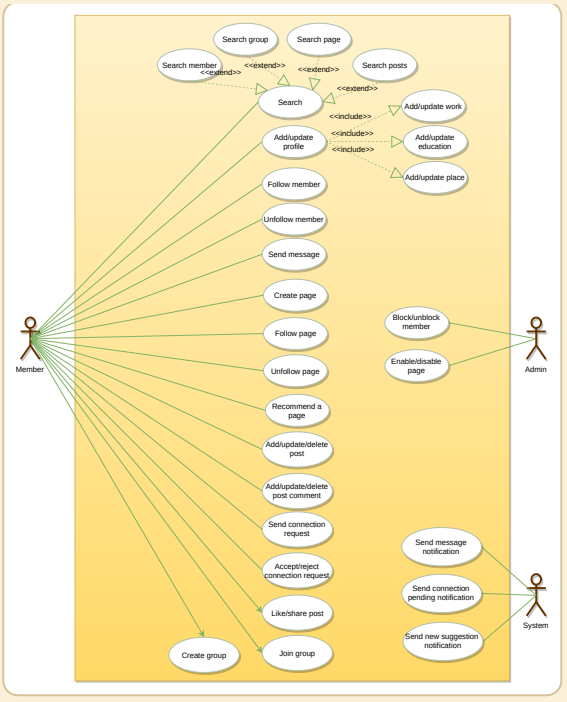


Figure — Facebook use cases for Member, Admin and System, including "Create page", "Recommend a page" and "Unfollow member" (course diagram)

Activity — add work experience. The member opens *Add Work*, enters position, company, location, dates and description, and presses save; the system verifies, asks for missing information if needed, then saves.

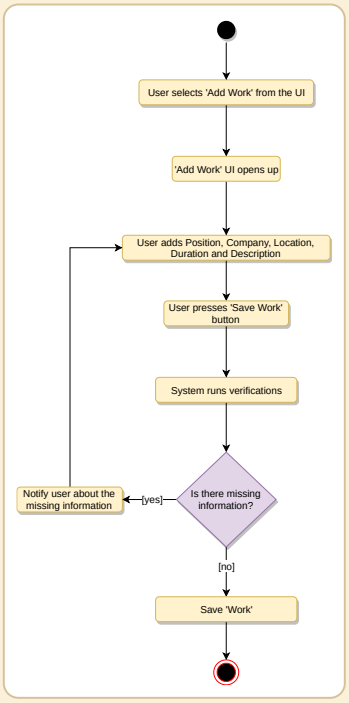


Figure — Activity diagram "add work to profile" with a verification loop (course diagram)

Activity — create a post. The member opens *New Post*, writes the text, optionally uploads a photo or video, **chooses the privacy list** that may see it, and presses publish. After verification the post is published to the chosen audience, who receive an email or push notification.

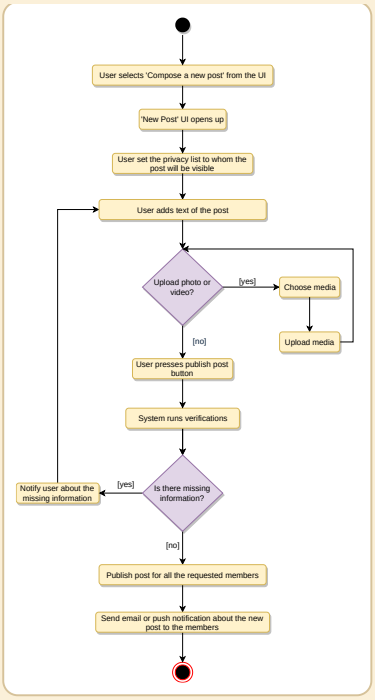


Figure — Activity diagram "create a new post": add text, optional media, set the privacy list, verify, publish, notify (course diagram)

Core entities

Class	Responsibility
`Member`, `Admin`	Identity, profile, follows (members and pages), groups, posts, privacy lists, inbox, notifications (the course's messages may carry photos and videos; ours are text only)
`Profile` (+ `Work`, `Education`, `Place`)	Defaults for every field, lists of parts
`PrivacyList`	Owner + named set of friend ids
`Audience` (sealed) → `Public`, `Friends`, `Custom(list)`, `OnlyMe`	Who may see a post
`SocialContent` → `Post`, `Comment`	Likes as a set; posts own comments; a post may belong to a group or page or be a share
`FriendRequest` + `FriendService`	Requests with both ends, the friend graph, `areFriends`, `pendingBetween` (both ways)
`VisibilityService`	The single `canSee(viewer, post)` rule, combining the post's privacy list and the friend graph
`Page` (+ `PageType`, `PageRecommendation`)	Followers (counted), one recommendation per member, average rating, page posts
`Group` (+ `GroupPrivacy`)	Members, join requests, owner approval, group posts
`NotificationService` + `PushChannel` / `EmailChannel`	Routes each notification type to channels
`SearchIndex`	Names of members/groups/pages; post words — `searchPosts` takes the viewer
`NewsFeedService` + `FeedRanker`	Gathers candidates, filters by `canSee`, ranks
`SuggestionService` + `Suggestion`	Friends-of-friends by mutual count
`Facebook`	Facade

Class diagram

The course builds Facebook in **five stages plus the extended requirement**.

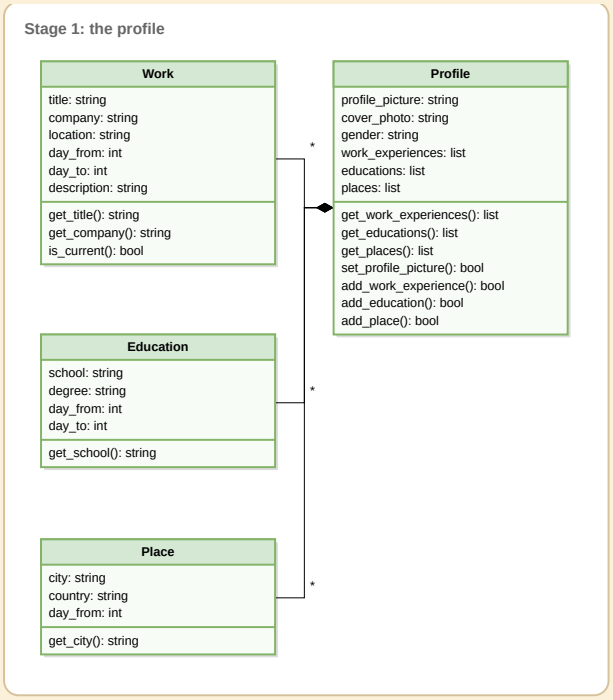


Figure — Stage 1: `Profile` holding lists of `Work`, `Education` and `Place` (course diagram)

`Stage 1 — the profile.` Work, education and places are small classes held in lists (composition). Every profile field starts with a **`default`** value, so a brand-new account already has a working profile, and no other code needs to make up a placeholder picture for someone who never uploaded one.

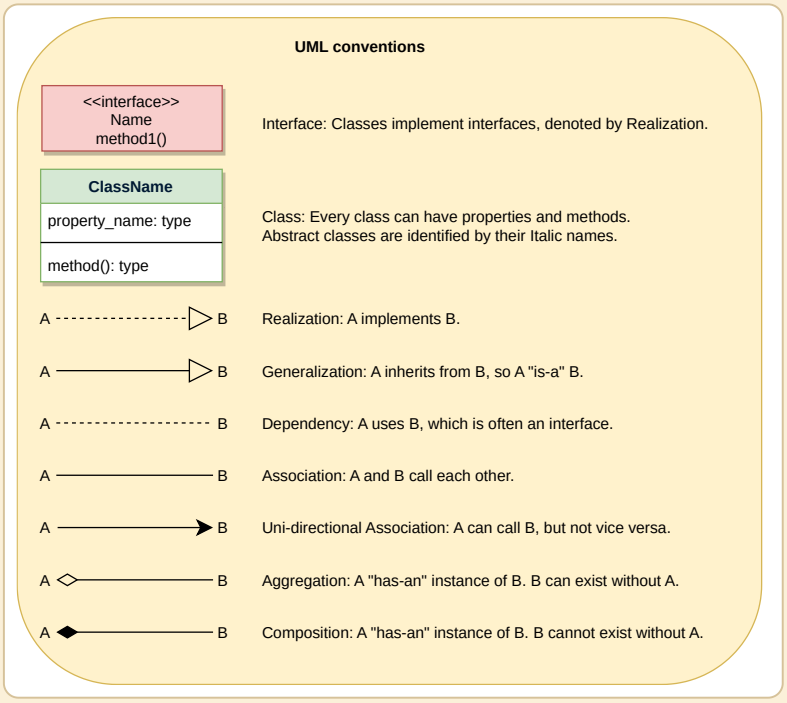


Figure — UML conventions used in these diagrams (course diagram)

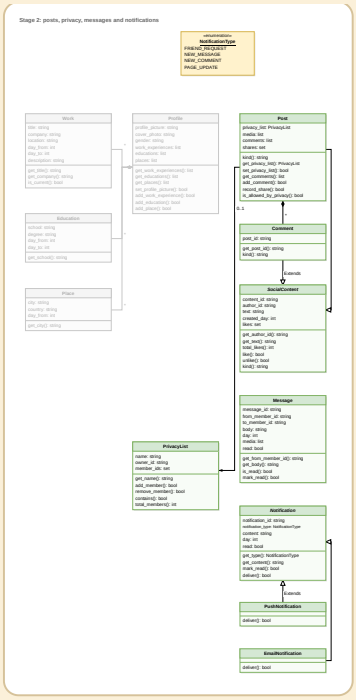


Figure — Stage 2: `SocialContent` with `Post` and `Comment`, a `0..1` association from `Post` to `PrivacyList`, `Message`, and abstract `Notification` (course diagram)

Stage 2 — posts, privacy, messages and notifications. Three choices here repair bugs in older versions. (1) **Store the likers, not a like count** — an old `Post` received its like count through the constructor, so a fresh post could start with a thousand likes, and there was no way to know whether you had liked it already. (2) **Comments get likes too**, so the likes set lives in the shared `SocialContent` parent. (3) **Privacy is checked by the post itself** — `isAllowedByPrivacy` is a method on the post, not something each screen must remember. Yet it can answer only part of the question, because a post has no idea who is friends with whom; stage 5 supplies the rest. The link from `Post` to `PrivacyList` is marked `0..1`: a post might have no list (it is public), and one list is reused by many posts and survives them. `Notification` is abstract, with a subclass for each delivery method.

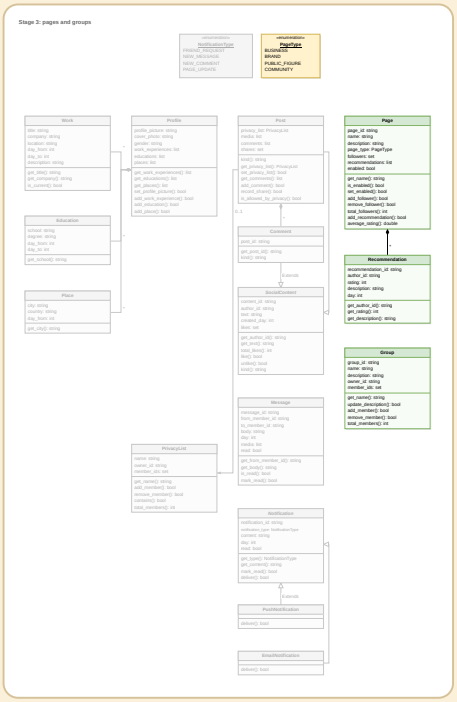


Figure — Stage 3: `Page` owning its `Recommendation`s, followers as ids, and `Group`; no member-count fields (course diagram)

Stage 3 — pages and groups. Look at what these classes do *not* have: a member-count field. An earlier version accepted the count in its constructor *and* also stored the members, so the two numbers could be

different from day one. The rule: **if a number can be computed by counting a collection, do not keep it as a separate field; keep the collection and count it when asked.**

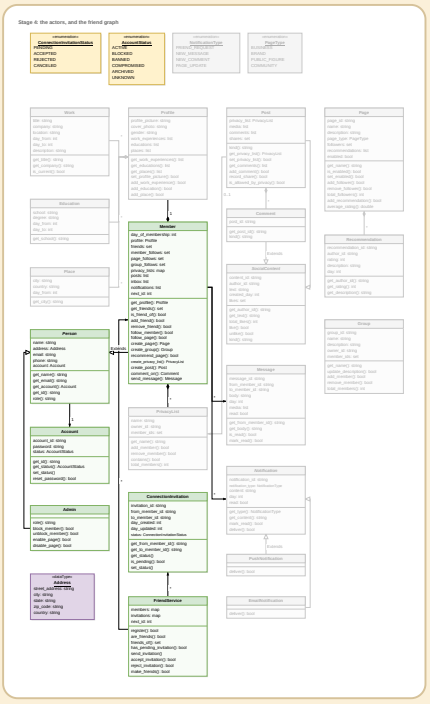


Figure — Stage 4: `Account`, `Person`, `Member`, `Admin`, `ConnectionInvitation` and `FriendService`, which owns the friend graph (course diagram)

Stage 4 — the actors and the friend graph. An earlier invitation class remembered only the person invited, not the person inviting, so there was no way to accept it properly. Since acceptance updates two members, it belongs in `FriendService`; that service therefore holds the **friend graph** (who is friends with whom) and is the obvious class to ask "are these two friends?" and "is a request already waiting between them?". Following (requirement 4) is deliberately different: one-sided, a set on the follower, no service.

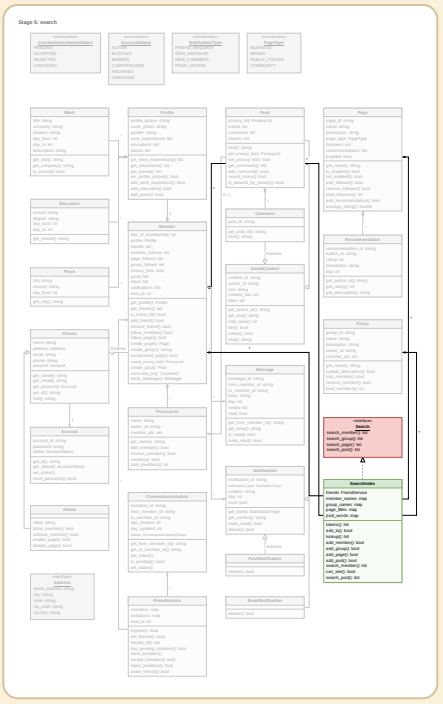


Figure — Stage 5: `Search` and `SearchIndex`, which holds a `FriendService` so post search can respect privacy (course diagram)

Stage 5 — search. The most important detail is small: **`searchPost` needs the viewer as a parameter.** The privacy rule (requirement 8) still applies when someone searches; a keyword search that skips it would let anyone read posts that were never shared with them. Deciding visibility therefore needs two sources — the

post's own privacy list, and the friendship data that only the friend service holds — and neither source can answer for the other. The index follows the same rule as LinkedIn's: each key maps to a collection.

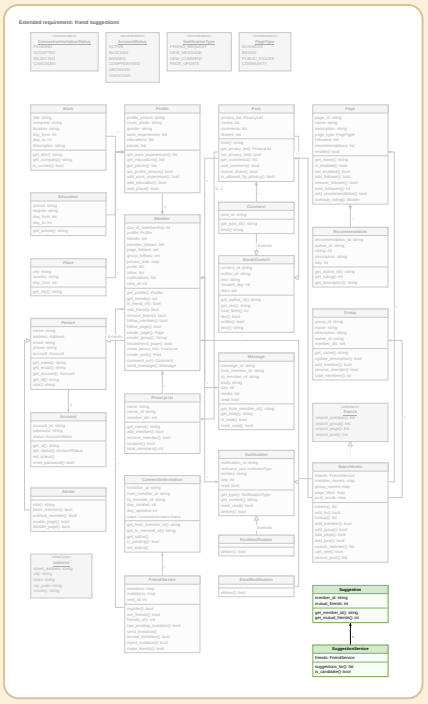


Figure — Extended requirement: `SuggestionService` reading the friend graph and producing `Suggestion` objects, with no change to existing classes (course diagram)

Extended requirement — friend suggestions. Friends of friends, **scored by mutual friends** (someone reached through three of your friends beats someone reached through one). Never suggest yourself, an existing friend, or anyone with a request pending **in either direction** (the one candidates forget — showing someone you invited yesterday is a visible bug). Sort by mutual count, then by id, so equal scores do not reshuffle between page loads. Notice that the feature arrives as two new classes with no edits to existing ones; if a design must be modified every time a new question is asked, it will keep being modified.

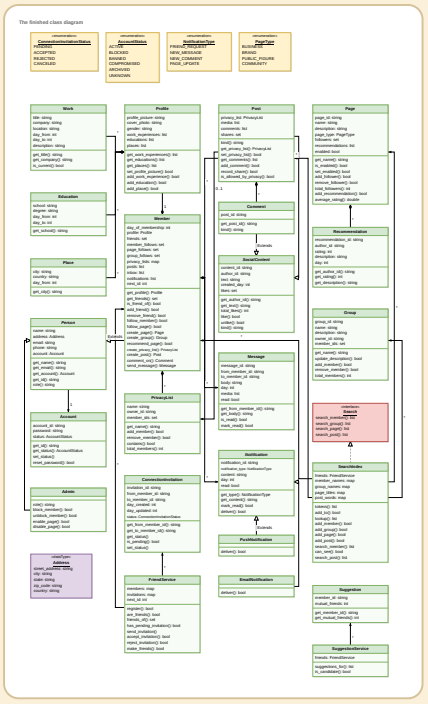
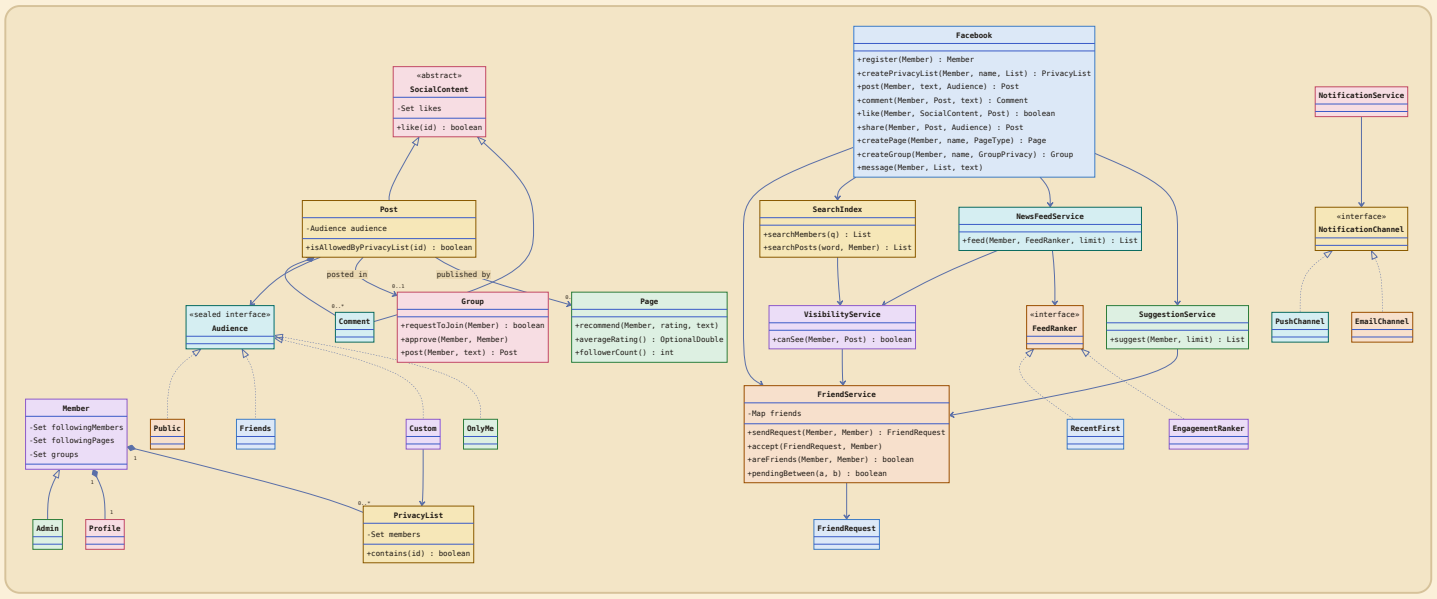


Figure — The finished Facebook class diagram with every class in full colour (course diagram)

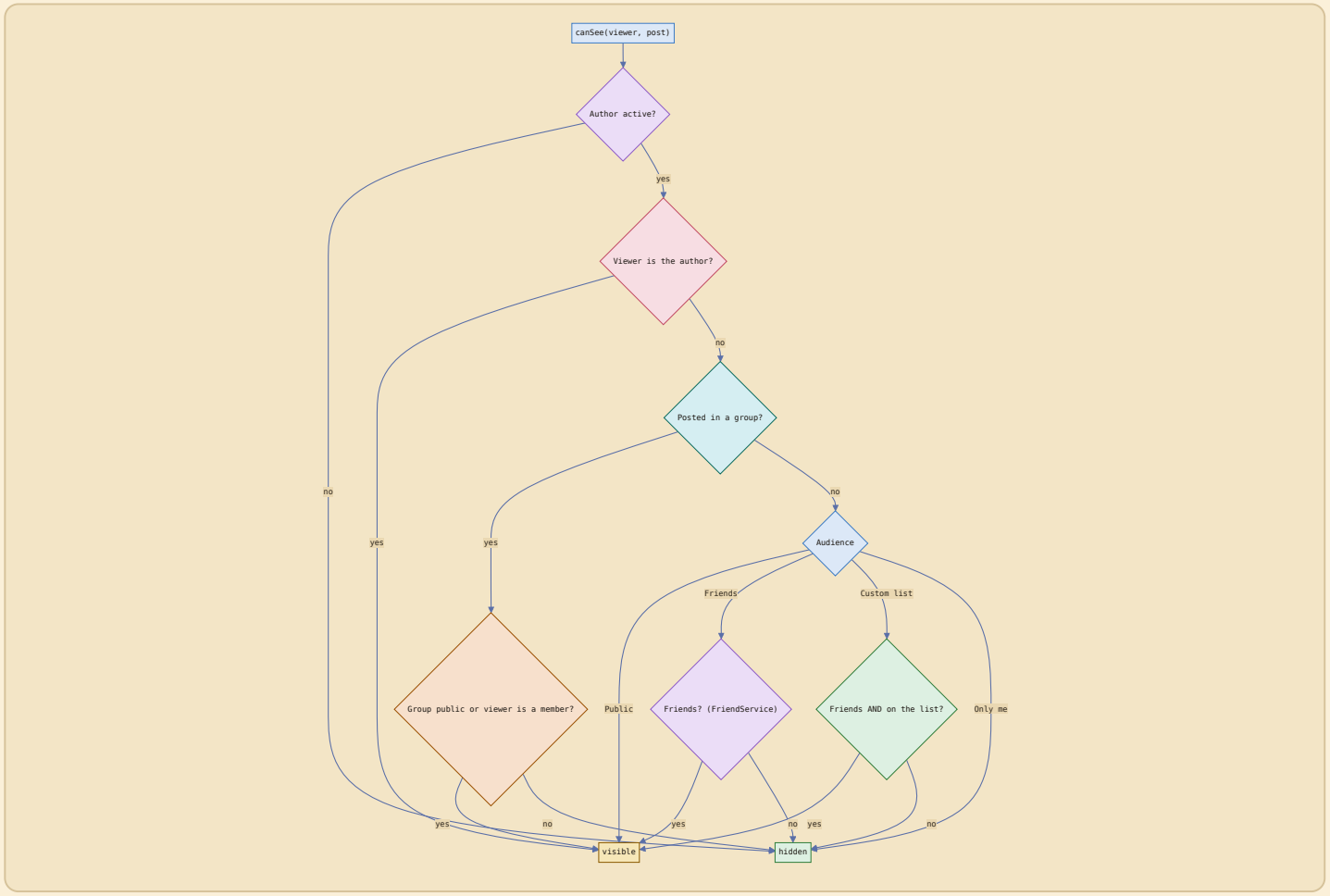
Our Java design keeps the course's structure and makes three things explicit: a sealed `Audience` type (Public / Friends / Custom list / Only me), a single `VisibilityService` used by the feed, search, comments, likes and shares, and a `NewsFeedService` (the course leaves the feed out) with ranking Strategies. Private groups with owner approval and page posts in the feed are added too. The course lists what it leaves out on purpose: the news feed and its ranking, one member blocking another, reactions other than a plain like,

tagging people, events, marketplace, stories, and photo albums that carry separate privacy settings. We add the feed; the rest are sketched under Extensions.

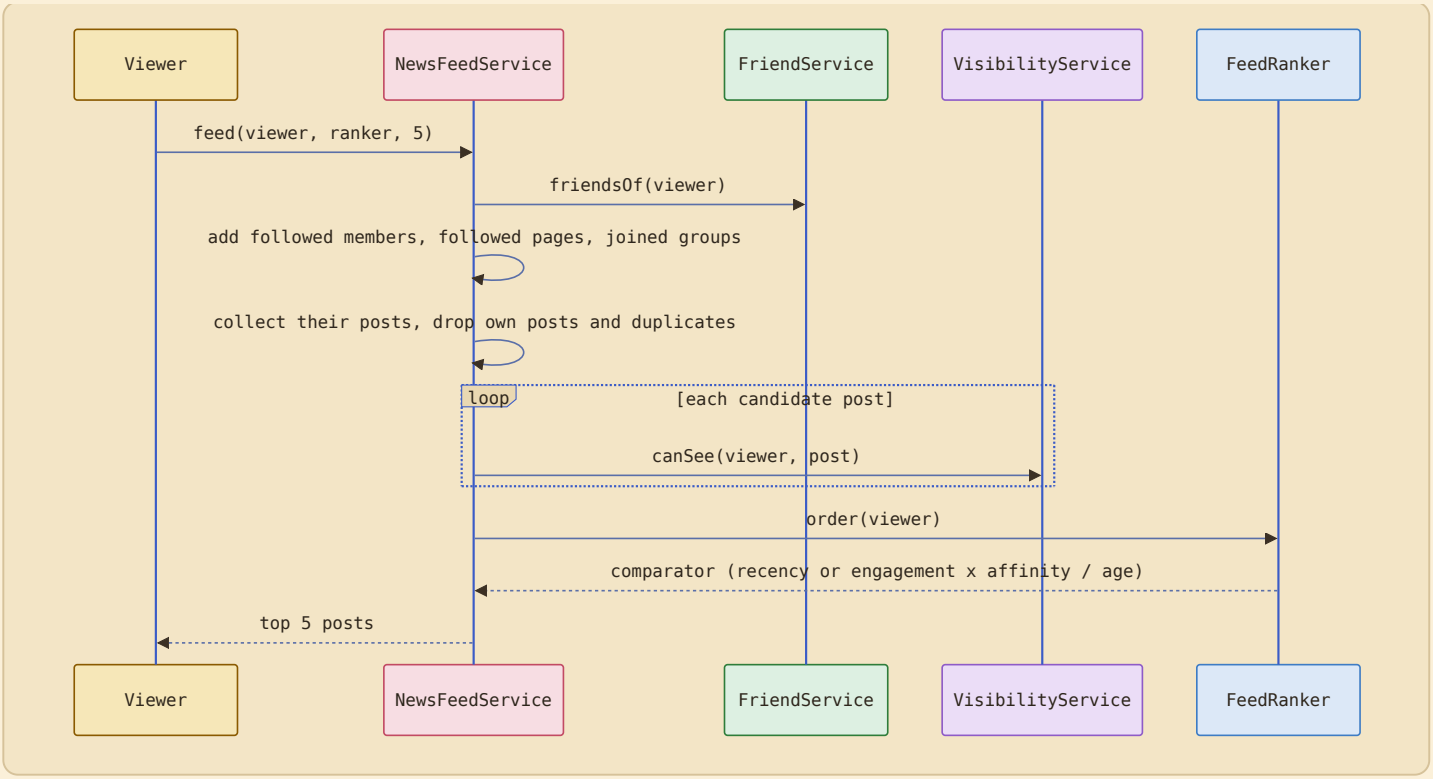


Key flows

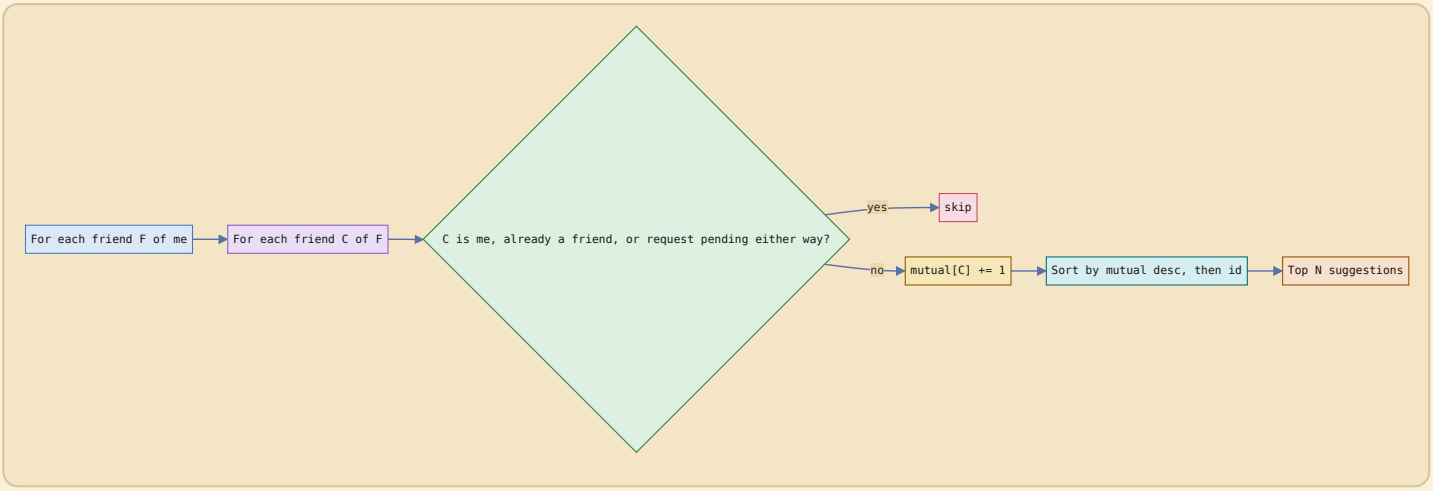
Can this viewer see this post? (used by feed, search, comment, like and share):



Building a news feed (pull model):



Friend suggestions:



Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
`VisibilityService.canSee` used everywhere	Single point of policy (a **Specification**-like rule object — one object that answers a yes/no business question)	The privacy rule is written once; search and feed cannot leak	Checking privacy in each screen
`Audience` sealed interface of records	Algebraic data type + exhaustive `switch`	Adding an audience forces every decision point to handle it	Integer or string codes
`Post.isAllowedByPrivacyList` + `FriendService.areFriends`	Each object answers what it knows (Information Expert)	Neither half pretends to know the other's data	Post holding a reference to the whole friend graph
`FriendService` owns the graph	Service for two-sided operations	Symmetric friendships; one place for "pending either way"	`member.acceptFriend()`
Likes as a set	"Store who, not how many"	No fake like counts, no double likes	`likeCount` field
No stored member/follower counts	Derive, don't store	Counts cannot disagree with lists	Constructor-supplied counts (the course's bug)
`NotificationService` routes types to channels	**Strategy** + configuration map	Friend requests go by push *and* email, comments by push only	`if/else` on type inside each domain method
`FeedRanker`	**Strategy**	"Most recent" vs ranked feed swap at run time	Ranking code inside the feed
`SuggestionService` added without editing others	Open/Closed Principle	The extended requirement is two new classes	Adding suggestion fields to `Member`
`Facebook` facade	**Facade**	One entry point, account-status checks	UI talking to every service

Java implementation

Package `lld.ch18.facebook`.

```java
// File: lld/ch18/facebook/Model.java
package lld.ch18.facebook;

import java.util.ArrayList;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Optional;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicLong;

/** Deterministic logical clock for repeatable demo output. */
final class Tick {
    private static final AtomicLong NOW = new AtomicLong();
    static long next() { return NOW.incrementAndGet(); }
    static long now() { return NOW.get(); }
    private Tick() {}
}

record Work(String title, String company, String location, int fromYear, Integer toYear) {
    boolean isCurrent() { return toYear == null; }
}
record Education(String school, String degree, int fromYear, int toYear) {}
record Place(String city, String country, int fromYear) {}

/** Every field has a default, so a profile is usable the moment the account exists. */
final class Profile {
    private volatile String picture = "default-avatar.png";
    private volatile String bio = "";
    private final List<Work> work = new CopyOnWriteArrayList<>();
    private final List<Education> education = new CopyOnWriteArrayList<>();
    private final List<Place> places = new CopyOnWriteArrayList<>();

    String picture() { return picture; }
    void setPicture(String p) { picture = p; }
    String bio() { return bio; }
    void setBio(String b) { bio = b; }
    void addWork(Work w) { work.add(w); }
    void addEducation(Education e) { education.add(e); }
    void addPlace(Place p) { places.add(p); }
    List<Work> work() { return List.copyOf(work); }
    List<Place> places() { return List.copyOf(places); }
}

enum AccountStatus { ACTIVE, BLOCKED, BANNED, COMPROMISED, ARCHIVED, UNKNOWN }

class Member {
    private static final AtomicLong IDS = new AtomicLong();
    private final long id = IDS.incrementAndGet();
    private final String name;
    private final String email;
    private final Profile profile = new Profile();
    private volatile AccountStatus status = AccountStatus.ACTIVE;
    private final Set<Long> followingMembers = ConcurrentHashMap.newKeySet();   // one-sided
    private final Set<Long> followingPages = ConcurrentHashMap.newKeySet();
    private final Set<Long> groups = ConcurrentHashMap.newKeySet();
    private final List<Post> posts = new CopyOnWriteArrayList<>();
    private final List<PrivacyList> privacyLists = new CopyOnWriteArrayList<>();
    private final List<Message> inbox = new CopyOnWriteArrayList<>();
    private final List<Notification> notifications = new CopyOnWriteArrayList<>();

    Member(String name, String email) { this.name = name; this.email = email; }

    long id() { return id; }
    String name() { return name; }
    String email() { return email; }
    Profile profile() { return profile; }
    AccountStatus status() { return status; }
    void setStatus(AccountStatus s) { status = s; }

    boolean follow(Member m) { return m.id != id && followingMembers.add(m.id); }
    boolean unfollow(Member m) { return followingMembers.remove(m.id); }
```

```
    Set<Long> followingMembers() { return Set.copyOf(followingMembers); }
    boolean followPage(Page p) { return followingPages.add(p.id()); }
    Set<Long> followingPages() { return Set.copyOf(followingPages); }
    void joinedGroup(Group g) { groups.add(g.id()); }
    Set<Long> groups() { return Set.copyOf(groups); }

    void addPost(Post p) { posts.add(p); }
    List<Post> posts() { return List.copyOf(posts); }
    void addPrivacyList(PrivacyList l) { privacyLists.add(l); }
    List<PrivacyList> privacyLists() { return List.copyOf(privacyLists); }
    void receive(Message m) { inbox.add(m); }
    List<Message> inbox() { return List.copyOf(inbox); }
    void notify(Notification n) { notifications.add(n); }
    List<Notification> notifications() { return List.copyOf(notifications); }

    @Override public String toString() { return name; }
}

final class Admin extends Member {
    Admin(String name, String email) { super(name, email); }
}

/** A named subset of the owner's friends, e.g. "Close friends". Outlives any single post. */
final class PrivacyList {
    private final Member owner;
    private final String name;
    private final Set<Long> members = ConcurrentHashMap.newKeySet();

    PrivacyList(Member owner, String name) { this.owner = owner; this.name = name; }
    Member owner() { return owner; }
    String name() { return name; }
    void add(long memberId) { members.add(memberId); }
    boolean contains(long memberId) { return members.contains(memberId); }
}

/** Who may see a post. A sealed type makes the visibility switch exhaustive. */
sealed interface Audience {
    record Public() implements Audience {}
    record Friends() implements Audience {}
    record Custom(PrivacyList list) implements Audience {}
    record OnlyMe() implements Audience {}
}

abstract class SocialContent {
    private static final AtomicLong IDS = new AtomicLong(1000);
    private final long id = IDS.incrementAndGet();
    private final Member author;
    private final String text;
    private final long createdAt = Tick.next();
    private final Set<Long> likes = new LinkedHashSet<>();      // a set: no double likes, can be undone

    SocialContent(Member author, String text) { this.author = author; this.text = text; }

    long id() { return id; }
    Member author() { return author; }
    String text() { return text; }
    long createdAt() { return createdAt; }
    synchronized boolean like(long memberId) { return likes.add(memberId); }
    synchronized boolean unlike(long memberId) { return likes.remove(memberId); }
    synchronized int likeCount() { return likes.size(); }
}

final class Post extends SocialContent {
    private final Audience audience;
    private final Group group;          // null unless posted in a group
    private final Page page;            // null unless published by a page
    private final Post sharedFrom;
    private final List<Comment> comments = new ArrayList<>();

    Post(Member author, String text, Audience audience, Group group, Page page, Post sharedFrom) {
        super(author, text);
        this.audience = audience;
        this.group = group;
        this.page = page;
```

```
        this.sharedFrom = sharedFrom;
    }

    Audience audience() { return audience; }
    Optional<Group> group() { return Optional.ofNullable(group); }
    Optional<Page> page() { return Optional.ofNullable(page); }
    Optional<Post> sharedFrom() { return Optional.ofNullable(sharedFrom); }

    /** The half of the privacy rule the post CAN answer on its own. */
    boolean isAllowedByPrivacyList(long viewerId) {
        return !(audience instanceof Audience.Custom c) || c.list().contains(viewerId);
    }

    synchronized Comment addComment(Member by, String text) {
        Comment c = new Comment(by, text, this);
        comments.add(c);
        return c;
    }
    synchronized List<Comment> comments() { return List.copyOf(comments); }

    @Override public String toString() {
        String where = page != null ? " [page " + page.name() + "]" : group != null ? " [group " + group.name() + "]"
: "";
        return "\"" + text() + "\" by " + author().name() + where;
    }
}

final class Comment extends SocialContent {
    private final Post post;
    Comment(Member author, String text, Post post) { super(author, text); this.post = post; }
    Post post() { return post; }
}

record Message(Member sender, List<Member> recipients, String text, long sentAt) {}
```

```java
// File: lld/ch18/facebook/PagesAndGroups.java
package lld.ch18.facebook;

import java.util.List;
import java.util.Map;
import java.util.OptionalDouble;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicLong;

enum PageType { BUSINESS, BRAND, PUBLIC_FIGURE, COMMUNITY }

/** One recommendation per member per page, with a 1-5 rating. */
record PageRecommendation(long memberId, int rating, String text) {
    PageRecommendation {
        if (rating < 1 || rating > 5) throw new IllegalArgumentException("rating must be 1..5");
    }
}

final class Page {
    private static final AtomicLong IDS = new AtomicLong(500);
    private final long id = IDS.incrementAndGet();
    private final String name;
    private final PageType type;
    private final Member owner;
    private final Set<Long> followers = ConcurrentHashMap.newKeySet();
    private final Map<Long, PageRecommendation> recommendations = new ConcurrentHashMap<>();   // owned
    private final List<Post> posts = new CopyOnWriteArrayList<>();

    Page(String name, PageType type, Member owner) { this.name = name; this.type = type; this.owner = owner; }

    long id() { return id; }
    String name() { return name; }
    PageType type() { return type; }
    Member owner() { return owner; }
    void addFollower(long memberId) { followers.add(memberId); }
    int followerCount() { return followers.size(); }            // never store what you can count

    void recommend(Member by, int rating, String text) {
        if (recommendations.putIfAbsent(by.id(), new PageRecommendation(by.id(), rating, text)) != null)
            throw new IllegalStateException(by + " already recommended " + name);
    }

    OptionalDouble averageRating() {
        return recommendations.values().stream().mapToInt(PageRecommendation::rating).average();
    }
    int recommendationCount() { return recommendations.size(); }

    Post publish(Member by, String text) {
        if (by.id() != owner.id()) throw new IllegalStateException(by + " does not manage page " + name);
        Post p = new Post(by, text, new Audience.Public(), null, this, null);
        posts.add(p);
        return p;
    }
    List<Post> posts() { return List.copyOf(posts); }
}

enum GroupPrivacy { PUBLIC, PRIVATE }

final class Group {
    private static final AtomicLong IDS = new AtomicLong(900);
    private final long id = IDS.incrementAndGet();
    private final String name;
    private final GroupPrivacy privacy;
    private final Member owner;
    private final Set<Long> members = ConcurrentHashMap.newKeySet();
    private final Set<Long> joinRequests = ConcurrentHashMap.newKeySet();
    private final List<Post> posts = new CopyOnWriteArrayList<>();

    Group(String name, GroupPrivacy privacy, Member owner) {
        this.name = name;
        this.privacy = privacy;
```

```
        this.owner = owner;
        members.add(owner.id());
        owner.joinedGroup(this);
    }

    long id() { return id; }
    String name() { return name; }
    boolean isPublic() { return privacy == GroupPrivacy.PUBLIC; }
    boolean isMember(long memberId) { return members.contains(memberId); }
    int memberCount() { return members.size(); }

    /** Public groups: join at once. Private groups: the request waits for the owner. Returns true if joined now. */
    boolean requestToJoin(Member m) {
        if (isPublic()) { members.add(m.id()); m.joinedGroup(this); return true; }
        joinRequests.add(m.id());
        return false;
    }

    void approve(Member by, Member requester) {
        if (by.id() != owner.id()) throw new IllegalStateException("only the owner approves members");
        if (!joinRequests.remove(requester.id())) throw new IllegalStateException(requester + " did not ask to join");
        members.add(requester.id());
        requester.joinedGroup(this);
    }

    Post post(Member by, String text) {
        if (!isMember(by.id())) throw new IllegalStateException(by + " is not a member of " + name);
        Post p = new Post(by, text, new Audience.Public(), this, null, null);
        posts.add(p);
        return p;
    }
    List<Post> posts() { return List.copyOf(posts); }
}
```

```java
// File: lld/ch18/facebook/Services.java
package lld.ch18.facebook;

import java.util.Arrays;
import java.util.Comparator;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;
import java.util.function.LongFunction;
import java.util.stream.Stream;

// ---------------------------------------------------------------- friends

enum RequestStatus { PENDING, ACCEPTED, REJECTED, CANCELED }

/** Records BOTH ends; an invitation that only knows the invitee cannot be accepted. */
final class FriendRequest {
    private static final AtomicLong IDS = new AtomicLong();
    private final long id = IDS.incrementAndGet();
    private final Member from;
    private final Member to;
    private RequestStatus status = RequestStatus.PENDING;

    FriendRequest(Member from, Member to) { this.from = from; this.to = to; }
    long id() { return id; }
    Member from() { return from; }
    Member to() { return to; }
    synchronized RequestStatus status() { return status; }
    synchronized void moveTo(RequestStatus next) {
        if (status != RequestStatus.PENDING) throw new IllegalStateException("request is already " + status);
        status = next;
    }
}

/** Owns the friend graph. Accepting changes two members, so it lives here, not on either member. */
final class FriendService {
    private final Map<Long, Set<Long>> friends = new ConcurrentHashMap<>();
    private final Map<Long, FriendRequest> requests = new ConcurrentHashMap<>();
    private final NotificationService notifications;

    FriendService(NotificationService notifications) { this.notifications = notifications; }

    synchronized FriendRequest sendRequest(Member from, Member to) {
        if (from.id() == to.id()) throw new IllegalArgumentException("cannot befriend yourself");
        if (areFriends(from, to)) throw new IllegalStateException(from + " and " + to + " are already friends");
        if (pendingBetween(from.id(), to.id())) throw new IllegalStateException("a request between " + from + " and "
+ to + " is pending");
        FriendRequest r = new FriendRequest(from, to);
        requests.put(r.id(), r);
        notifications.send(to, NotificationType.FRIEND_REQUEST, from + " sent you a friend request");
        return r;
    }

    synchronized void accept(FriendRequest r, Member by) {
        if (by.id() != r.to().id()) throw new IllegalStateException("only " + r.to() + " can accept");
        r.moveTo(RequestStatus.ACCEPTED);
        friends.computeIfAbsent(r.from().id(), k -> ConcurrentHashMap.newKeySet()).add(r.to().id());
        friends.computeIfAbsent(r.to().id(), k -> ConcurrentHashMap.newKeySet()).add(r.from().id());
    }

    synchronized void reject(FriendRequest r, Member by) {
        if (by.id() != r.to().id()) throw new IllegalStateException("only " + r.to() + " can reject");
        r.moveTo(RequestStatus.REJECTED);
    }

    synchronized void unfriend(Member a, Member b) {
        friends.getOrDefault(a.id(), Set.of()).remove(b.id());
        friends.getOrDefault(b.id(), Set.of()).remove(a.id());
```

```
    }

    boolean areFriends(Member a, Member b) { return friendsOf(a.id()).contains(b.id()); }
    Set<Long> friendsOf(long memberId) { return Set.copyOf(friends.getOrDefault(memberId, Set.of())); }

    /** Looks BOTH ways: a request from a to b or from b to a. */
    boolean pendingBetween(long a, long b) {
        return requests.values().stream().anyMatch(r -> r.status() == RequestStatus.PENDING
                && ((r.from().id() == a && r.to().id() == b) || (r.from().id() == b && r.to().id() == a)));
    }
}

// ---------------------------------------------------------------- privacy

/**
 * The ONE place that decides "can this viewer see this post?". The post knows
 * its privacy list; only the friend graph knows friendship; this asks both.
 * Feed, search and profile pages all call it, so the rule is written once.
 */
final class VisibilityService {
    private final FriendService friends;
    VisibilityService(FriendService friends) { this.friends = friends; }

    boolean canSee(Member viewer, Post post) {
        Member author = post.author();
        if (author.status() != AccountStatus.ACTIVE) return false;
        if (viewer.id() == author.id()) return true;
        if (post.group().isPresent()) {
            Group g = post.group().get();
            return g.isPublic() || g.isMember(viewer.id());
        }
        return switch (post.audience()) {
            case Audience.Public p -> true;
            case Audience.Friends f -> friends.areFriends(viewer, author);
            case Audience.Custom c -> friends.areFriends(viewer, author) && post.isAllowedByPrivacyList(viewer.id());
            case Audience.OnlyMe o -> false;
        };
    }
}

// ---------------------------------------------------------------- notifications

enum NotificationType { FRIEND_REQUEST, NEW_MESSAGE, NEW_COMMENT, PAGE_UPDATE }

record Notification(NotificationType type, String text, long at) {}

/** Strategy for delivery. A new channel is a new class, not a new if-branch. */
interface NotificationChannel { void deliver(Member to, Notification n); }

final class PushChannel implements NotificationChannel {
    @Override public void deliver(Member to, Notification n) {
        to.notify(n);
        System.out.println("    [push -> " + to + "] " + n.text());
    }
}

final class EmailChannel implements NotificationChannel {
    @Override public void deliver(Member to, Notification n) {
        System.out.println("    [email -> " + to.email() + "] " + n.text());
    }
}

final class NotificationService {
    private final Map<NotificationType, List<NotificationChannel>> routes;
    NotificationService(Map<NotificationType, List<NotificationChannel>> routes) { this.routes = Map.copyOf(routes); }
    void send(Member to, NotificationType type, String text) {
        Notification n = new Notification(type, text, Tick.next());
        routes.getOrDefault(type, List.of()).forEach(c -> c.deliver(to, n));
    }
}

// ---------------------------------------------------------------- search

/** Every key maps to a set of matches (never a single object), and post search respects privacy. */
```

```
final class SearchIndex {
    private final Map<String, Set<Member>> members = new ConcurrentHashMap<>();
    private final Map<String, Set<Group>> groups = new ConcurrentHashMap<>();
    private final Map<String, Set<Page>> pages = new ConcurrentHashMap<>();
    private final Map<String, Set<Post>> posts = new ConcurrentHashMap<>();
    private final VisibilityService visibility;

    SearchIndex(VisibilityService visibility) { this.visibility = visibility; }

    void add(Member m) { index(members, m.name(), m); }
    void add(Group g) { index(groups, g.name(), g); }
    void add(Page p) { index(pages, p.name(), p); }
    void add(Post p) { index(posts, p.text(), p); }

    List<Member> searchMembers(String q) { return find(members,
q).sorted(Comparator.comparing(Member::name)).toList(); }
    List<Group> searchGroups(String q) { return find(groups, q).toList(); }
    List<Page> searchPages(String q) { return find(pages, q).toList(); }

    /** Takes the viewer: privacy does not stop being true inside a search. */
    List<Post> searchPosts(String word, Member viewer) {
        return find(posts, word).filter(p -> visibility.canSee(viewer, p))
                .sorted(Comparator.comparingLong(Post::createdAt).reversed()).toList();
    }

    private static <T> void index(Map<String, Set<T>> idx, String text, T item) {
        for (String w : words(text)) idx.computeIfAbsent(w, k -> ConcurrentHashMap.newKeySet()).add(item);
    }

    private static <T> Stream<T> find(Map<String, Set<T>> idx, String q) {
        List<String> ws = words(q);
        if (ws.isEmpty()) return Stream.empty();
        return idx.getOrDefault(ws.get(0), Set.of()).stream()
                .filter(item -> ws.stream().allMatch(w -> idx.getOrDefault(w, Set.of()).contains(item)));
    }

    private static List<String> words(String s) {
        return Arrays.stream(s.toLowerCase(Locale.ROOT).split("\\W+")).filter(w -> !w.isEmpty()).toList();
    }
}

// ------------------------------------------------------------------ news feed

/** Strategy for ranking the news feed. */
interface FeedRanker { Comparator<Post> order(Member viewer); }

final class RecentFirst implements FeedRanker {
    @Override public Comparator<Post> order(Member viewer) { return
Comparator.comparingLong(Post::createdAt).reversed(); }
}

/** Engagement x affinity / age, a toy version of the idea behind ranked feeds. */
final class EngagementRanker implements FeedRanker {
    private final FriendService friends;
    EngagementRanker(FriendService friends) { this.friends = friends; }

    @Override public Comparator<Post> order(Member viewer) {
        return Comparator.comparingDouble((Post p) -> -score(viewer,
p)).thenComparing(Comparator.comparingLong(Post::createdAt).reversed());
    }

    private double score(Member viewer, Post p) {
        double engagement = 1 + p.likeCount() + 2.0 * p.comments().size();
        double affinity = friends.areFriends(viewer, p.author()) ? 2.0 : 1.0;
        return engagement * affinity / (1 + (Tick.now() - p.createdAt()) / 10.0);
    }
}

/**
 * Pull model: gather candidates (friends, followed members, followed pages,
 * joined groups), drop what the viewer may not see, rank, cut to size.
 */
final class NewsFeedService {
    private final FriendService friends;
```

```
    private final VisibilityService visibility;
    private final LongFunction<Member> members;
    private final LongFunction<Page> pages;
    private final LongFunction<Group> groups;

    NewsFeedService(FriendService f, VisibilityService v, LongFunction<Member> m, LongFunction<Page> p,
LongFunction<Group> g) {
        friends = f; visibility = v; members = m; pages = p; groups = g;
    }

    List<Post> feed(Member viewer, FeedRanker ranker, int limit) {
        Set<Long> people = new HashSet<>(friends.friendsOf(viewer.id()));
        people.addAll(viewer.followingMembers());
        Stream<Post> fromPeople = people.stream().map(members::apply).flatMap(m -> m.posts().stream());
        Stream<Post> fromPages = viewer.followingPages().stream().map(pages::apply).flatMap(pg ->
pg.posts().stream());
        Stream<Post> fromGroups = viewer.groups().stream().map(groups::apply).flatMap(g -> g.posts().stream());
        return Stream.of(fromPeople, fromPages, fromGroups).flatMap(s -> s)
                .distinct()
                .filter(p -> p.author().id() != viewer.id())
                .filter(p -> visibility.canSee(viewer, p))
                .sorted(ranker.order(viewer))
                .limit(limit)
                .toList();
    }
}

// ---------------------------------------------------------------------- suggestions (extended requirement)

record Suggestion(Member member, int mutualFriends) {
    @Override public String toString() { return member.name() + "(" + mutualFriends + " mutual)"; }
}

/** Friends of friends, scored by mutual friends; never yourself, a friend, or anyone with a pending request. */
final class SuggestionService {
    private final FriendService friends;
    private final LongFunction<Member> members;

    SuggestionService(FriendService friends, LongFunction<Member> members) { this.friends = friends; this.members =
members; }

    List<Suggestion> suggest(Member me, int limit) {
        Set<Long> mine = friends.friendsOf(me.id());
        Map<Long, Integer> mutual = new HashMap<>();
        for (long f : mine) {
            for (long candidate : friends.friendsOf(f)) {
                if (candidate == me.id() || mine.contains(candidate) || friends.pendingBetween(me.id(), candidate))
continue;
                mutual.merge(candidate, 1, Integer::sum);
            }
        }
        return mutual.entrySet().stream()
                .map(e -> new Suggestion(members.apply(e.getKey()), e.getValue()))
                .sorted(Comparator.comparingInt(Suggestion::mutualFriends).reversed()
                        .thenComparingLong(s -> s.member().id()))           // stable tie-break
                .limit(limit)
                .toList();
    }
}
```

```java
// File: lld/ch18/facebook/Facebook.java
package lld.ch18.facebook;

import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

/** Facade: wires the services and checks account status on every write. */
final class Facebook {
    private final Map<Long, Member> members = new ConcurrentHashMap<>();
    private final Map<Long, Page> pages = new ConcurrentHashMap<>();
    private final Map<Long, Group> groups = new ConcurrentHashMap<>();
    final NotificationService notifications;
    final FriendService friends;
    final VisibilityService visibility;
    final SearchIndex search;
    final NewsFeedService feed;
    final SuggestionService suggestions;

    Facebook(Map<NotificationType, List<NotificationChannel>> routes) {
        notifications = new NotificationService(routes);
        friends = new FriendService(notifications);
        visibility = new VisibilityService(friends);
        search = new SearchIndex(visibility);
        feed = new NewsFeedService(friends, visibility, members::get, pages::get, groups::get);
        suggestions = new SuggestionService(friends, members::get);
    }

    <M extends Member> M register(M m) {
        members.put(m.id(), m);
        search.add(m);
        return m;
    }

    PrivacyList createPrivacyList(Member owner, String name, List<Member> people) {
        PrivacyList list = new PrivacyList(owner, name);
        for (Member p : people) {
            if (!friends.areFriends(owner, p)) throw new IllegalStateException(p + " is not a friend of " + owner);
            list.add(p.id());
        }
        owner.addPrivacyList(list);
        return list;
    }

    Post post(Member author, String text, Audience audience) {
        active(author);
        if (audience instanceof Audience.Custom c && c.list().owner() != author)
            throw new IllegalArgumentException("you can only use your own privacy lists");
        Post p = new Post(author, text, audience, null, null, null);
        author.addPost(p);
        search.add(p);
        return p;
    }

    Post share(Member by, Post original, Audience audience) {
        active(by);
        if (!visibility.canSee(by, original)) throw new IllegalStateException(by + " cannot see that post");
        Post p = new Post(by, "shared: " + original.text(), audience, null, null, original);
        by.addPost(p);
        return p;
    }

    Comment comment(Member by, Post post, String text) {
        active(by);
        if (!visibility.canSee(by, post)) throw new IllegalStateException(by + " cannot see that post");
        Comment c = post.addComment(by, text);
        if (post.author() != by) notifications.send(post.author(), NotificationType.NEW_COMMENT, by + " commented on
your post");
        return c;
    }

    boolean like(Member by, SocialContent content, Post containingPost) {
        active(by);
```

```
        if (!visibility.canSee(by, containingPost)) throw new IllegalStateException(by + " cannot see that post");
        return content.like(by.id());
    }

    void message(Member from, List<Member> to, String text) {
        active(from);
        Message m = new Message(from, List.copyOf(to), text, Tick.next());
        for (Member r : to) {
            r.receive(m);
            notifications.send(r, NotificationType.NEW_MESSAGE, "new message from " + from);
        }
    }

    Page createPage(Member owner, String name, PageType type) {
        active(owner);
        Page p = new Page(name, type, owner);
        pages.put(p.id(), p);
        search.add(p);
        return p;
    }

    void followPage(Member m, Page p) {
        if (m.followPage(p)) p.addFollower(m.id());
    }

    Post publishOnPage(Member by, Page page, String text) {
        Post p = page.publish(by, text);
        search.add(p);
        return p;
    }

    Group createGroup(Member owner, String name, GroupPrivacy privacy) {
        active(owner);
        Group g = new Group(name, privacy, owner);
        groups.put(g.id(), g);
        search.add(g);
        return g;
    }

    Post postInGroup(Member by, Group g, String text) {
        active(by);
        Post p = g.post(by, text);
        search.add(p);
        return p;
    }

    void block(Admin admin, Member target) { target.setStatus(AccountStatus.BLOCKED); }

    private static void active(Member m) {
        if (m.status() != AccountStatus.ACTIVE) throw new IllegalStateException(m + " is " + m.status());
    }
}
```

```java
// File: lld/ch18/facebook/Main.java
package lld.ch18.facebook;

import java.util.List;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        NotificationChannel push = new PushChannel();
        NotificationChannel email = new EmailChannel();
        Facebook fb = new Facebook(Map.of(
                NotificationType.FRIEND_REQUEST, List.of(push, email),
                NotificationType.NEW_MESSAGE, List.of(push),
                NotificationType.NEW_COMMENT, List.of(push)));

        Member alice = fb.register(new Member("Alice Rao", "alice@example.com"));
        Member bob = fb.register(new Member("Bob Das", "bob@example.com"));
        Member carol = fb.register(new Member("Carol Nair", "carol@example.com"));
        Member dave = fb.register(new Member("Dave Paul", "dave@example.com"));
        Member erin = fb.register(new Member("Erin Shah", "erin@example.com"));
        Member frank = fb.register(new Member("Frank Lee", "frank@example.com"));
        Member grace = fb.register(new Member("Grace Lee", "grace@example.com"));
        Admin admin = fb.register(new Admin("Admin", "admin@example.com"));

        section("1. Profile and search by name");
        alice.profile().addWork(new Work("Designer", "Studio Nine", "Mumbai", 2021, null));
        alice.profile().addPlace(new Place("Mumbai", "India", 2019));
        System.out.println("Alice's picture before upload: " + alice.profile().picture());
        System.out.println("search 'lee': " + fb.search.searchMembers("lee"));

        section("2. Friend requests");
        FriendRequest r = fb.friends.sendRequest(alice, bob);
        fb.friends.accept(r, bob);
        attempt(() -> fb.friends.accept(r, bob));
        befriend(fb, alice, carol);
        befriend(fb, alice, erin);
        befriend(fb, bob, dave);
        befriend(fb, carol, dave);
        befriend(fb, erin, grace);
        frank.follow(alice);                                    // follow without being friends

        section("3. Friend suggestions (extended requirement)");
        System.out.println("before: " + fb.suggestions.suggest(alice, 5));
        fb.friends.sendRequest(alice, grace);
        System.out.println("after inviting Grace: " + fb.suggestions.suggest(alice, 5));
        attempt(() -> fb.friends.sendRequest(grace, alice));        // pending the other way

        section("4. Posts and privacy");
        PrivacyList close = fb.createPrivacyList(alice, "Close friends", List.of(bob, carol));
        attempt(() -> fb.createPrivacyList(alice, "Oops", List.of(frank)));
        Post pub = fb.post(alice, "New portfolio is live!", new Audience.Public());
        Post fr = fb.post(alice, "Moving to Pune next month", new Audience.Friends());
        Post secret = fb.post(alice, "Planning a surprise party for Erin", new Audience.Custom(close));
        for (Member viewer : List.of(alice, bob, erin, frank)) {
            System.out.printf("  %-10s public=%-5s friends=%-5s closeFriends=%s%n", viewer.name(),
                    fb.visibility.canSee(viewer, pub), fb.visibility.canSee(viewer, fr), fb.visibility.canSee(viewer,
secret));
        }
        System.out.println("search posts 'party' as Bob:  " + fb.search.searchPosts("party", bob));
        System.out.println("search posts 'party' as Erin: " + fb.search.searchPosts("party", erin));

        section("5. Comments, likes, shares");
        Comment cm = fb.comment(bob, secret, "I'll bring the cake");
        fb.like(carol, secret, secret);
        fb.like(alice, cm, secret);                                 // comments can be liked too
        System.out.println("Bob likes: " + fb.like(bob, secret, secret) + " / again: " + fb.like(bob, secret,
secret));
        attempt(() -> fb.comment(erin, secret, "What party?"));
        fb.share(erin, pub, new Audience.Friends());

        section("6. Pages and groups");
        Page pizza = fb.createPage(carol, "Joe's Pizza", PageType.BUSINESS);
        fb.followPage(alice, pizza);
```

```
        pizza.recommend(alice, 5, "Best crust in town");
        pizza.recommend(bob, 4, "Good, a bit slow");
        attempt(() -> pizza.recommend(alice, 1, "changed my mind"));
        fb.publishOnPage(carol, pizza, "Two-for-one pizza on Friday");
        System.out.printf("%s: followers=%d, rating=%.1f from %d recommendations%n",
                pizza.name(), pizza.followerCount(), pizza.averageRating().orElse(0), pizza.recommendationCount());
        Group chess = fb.createGroup(dave, "Chess Club", GroupPrivacy.PRIVATE);
        System.out.println("Frank joined at once? " + chess.requestToJoin(frank));
        chess.approve(dave, frank);
        fb.postInGroup(frank, chess, "Chess tournament on Sunday");
        attempt(() -> fb.postInGroup(alice, chess, "Can I play?"));
        System.out.println("search posts 'chess' as Alice: " + fb.search.searchPosts("chess", alice)
                + ", as Dave: " + fb.search.searchPosts("chess", dave));

        section("7. News feeds");
        System.out.println("Alice (recent): " + fb.feed.feed(alice, new RecentFirst(), 5));
        System.out.println("Bob   (ranked): " + fb.feed.feed(bob, new EngagementRanker(fb.friends), 5));
        System.out.println("Frank (recent): " + fb.feed.feed(frank, new RecentFirst(), 5));

        section("8. Messages and admin");
        System.out.println("Dave before Frank is blocked: " + fb.feed.feed(dave, new RecentFirst(), 5));
        fb.message(bob, List.of(alice, carol), "Party at 8?");
        fb.block(admin, frank);
        System.out.println("Dave after Frank is blocked: " + fb.feed.feed(dave, new RecentFirst(), 5));
        System.out.println("Alice's notifications: " +
alice.notifications().stream().map(Notification::type).toList());
    }

    static void befriend(Facebook fb, Member a, Member b) { fb.friends.accept(fb.friends.sendRequest(a, b), b); }

    static void section(String t) { System.out.println("\n== " + t + " =="); }

    static void attempt(Runnable r) {
        try { r.run(); } catch (RuntimeException e) { System.out.println("  REJECTED: " + e.getMessage()); }
    }
}
```

Output:

```
== 1. Profile and search by name ==
Alice's picture before upload: default-avatar.png
search 'lee': [Frank Lee, Grace Lee]

== 2. Friend requests ==
    [push -> Bob Das] Alice Rao sent you a friend request
    [email -> bob@example.com] Alice Rao sent you a friend request
  REJECTED: request is already ACCEPTED
    [push -> Carol Nair] Alice Rao sent you a friend request
    [email -> carol@example.com] Alice Rao sent you a friend request
    [push -> Erin Shah] Alice Rao sent you a friend request
    [email -> erin@example.com] Alice Rao sent you a friend request
    [push -> Dave Paul] Bob Das sent you a friend request
    [email -> dave@example.com] Bob Das sent you a friend request
    [push -> Dave Paul] Carol Nair sent you a friend request
    [email -> dave@example.com] Carol Nair sent you a friend request
    [push -> Grace Lee] Erin Shah sent you a friend request
    [email -> grace@example.com] Erin Shah sent you a friend request

== 3. Friend suggestions (extended requirement) ==
before: [Dave Paul(2 mutual), Grace Lee(1 mutual)]
    [push -> Grace Lee] Alice Rao sent you a friend request
    [email -> grace@example.com] Alice Rao sent you a friend request
after inviting Grace: [Dave Paul(2 mutual)]
  REJECTED: a request between Grace Lee and Alice Rao is pending

== 4. Posts and privacy ==
  REJECTED: Frank Lee is not a friend of Alice Rao
  Alice Rao  public=true  friends=true  closeFriends=true
  Bob Das    public=true  friends=true  closeFriends=true
  Erin Shah  public=true  friends=true  closeFriends=false
  Frank Lee  public=true  friends=false closeFriends=false
search posts 'party' as Bob:  ["Planning a surprise party for Erin" by Alice Rao]
search posts 'party' as Erin: []

== 5. Comments, likes, shares ==
    [push -> Alice Rao] Bob Das commented on your post
Bob likes: true / again: false
  REJECTED: Erin Shah cannot see that post

== 6. Pages and groups ==
  REJECTED: Alice Rao already recommended Joe's Pizza
Joe's Pizza: followers=1, rating=4.5 from 2 recommendations
Frank joined at once? false
  REJECTED: Alice Rao is not a member of Chess Club
search posts 'chess' as Alice: [], as Dave: ["Chess tournament on Sunday" by Frank Lee [group Chess Club]]

== 7. News feeds ==
Alice (recent): ["Two-for-one pizza on Friday" by Carol Nair [page Joe's Pizza], "shared: New portfolio is live!" by
Erin Shah]
Bob   (ranked): ["Planning a surprise party for Erin" by Alice Rao, "Moving to Pune next month" by Alice Rao, "New
portfolio is live!" by Alice Rao]
Frank (recent): ["New portfolio is live!" by Alice Rao]

== 8. Messages and admin ==
Dave before Frank is blocked: ["Chess tournament on Sunday" by Frank Lee [group Chess Club]]
    [push -> Alice Rao] new message from Bob Das
    [push -> Carol Nair] new message from Bob Das
Dave after Frank is blocked: []
Alice's notifications: [NEW_COMMENT, NEW_MESSAGE]
```

Walking through the important parts

- **The visibility table in section 4** is the course's key scenario, with four viewers. Alice is the author, so she sees all three of her posts. Bob is a friend *and* on the "Close friends" list, so he sees all three posts. Erin is a friend but *not* on the list, so she sees the public and friends posts but not the close-friends one. Frank is not Alice's friend, so only the public post is visible to him. Two rules, both applied, neither written twice.
- **Search respects privacy:** searching "party" returns Alice's close-friends post for Bob and nothing for Erin, because `searchPosts` takes the viewer and calls `canSee`.

- `createPrivacyList` refuses Frank because a privacy list may contain only friends.
- **Comments, likes and shares go through `canSee` too:** Erin cannot comment on the close-friends post. Without this, "comment" would be a side door into private content.
- **Suggestions:** before inviting Grace, Alice sees Dave (2 mutual: Bob and Carol) and Grace (1 mutual: Erin). After Alice invites Grace, Grace disappears (pending), and Grace cannot send a request back while one is pending.
- **Pages:** one recommendation per member (`putIfAbsent`), follower count and average rating computed from the stored data.
- **Private group:** Frank's join waits for Dave's approval; Alice (not a member) cannot post there and cannot find the group's post by search, while Dave can.
- **Feeds:** Alice's feed includes the page she follows and Erin's share; Bob's ranked feed puts the most-engaged, most recent friend posts first; Frank (who only follows Alice) sees only her public post.
- **Blocking:** once Frank is blocked, `canSee` hides his posts, so his group post disappears from Dave's feed.
- **Notification routing:** friend requests go to push *and* email; comments and messages go to push only — configured with a `Map<NotificationType, List<NotificationChannel>>`.

Concurrency & edge cases

- **Friend graph:** `sendRequest`, `accept`, `reject` and `unfriend` are `synchronized` on the service, and adjacency sets (each member's set of directly linked members) are concurrent, so an accept cannot leave a one-way friendship. In a database, a friendship is one row keyed by the ordered pair `(min(id), max(id))`, which makes symmetry automatic.
- **Pending in both directions:** checked by `pendingBetween(a, b)` for both requests and suggestions.
- **Privacy changes after posting:** because visibility is computed at read time, removing someone from a list or unfriending them hides the post immediately. (A push-model feed that copied posts into feeds would need to re-check `canSee` at read time anyway.)
- **Shares:** sharing requires seeing the original; a share of a friends-only post should not expose it to the sharer's audience — a production rule would cap the share's audience at the original's.
- **Deterministic ordering** (the same input always produces the same order)**:** suggestions tie-break by id; feeds tie-break by creation time; our logical `Tick` clock (a counter that goes up by one per event and stands in for real time) makes demo output repeatable.
- **Blocked or deleted authors:** `canSee` returns false for inactive authors.

Extensions & follow-up questions

- **"Scale the news feed."** — **Fan-out on write**: when a member posts, push the post id into each friend's feed cache (fast reads; costly for accounts with millions of followers). **Fan-out on read**: assemble at open time (our design). Real systems mix them: push for normal users, pull for celebrities, merge at read, and always re-check privacy.
- **"Rank the feed better."** — Add signals (affinity = how often the viewer interacts with the author, content type, recency decay) behind the same `FeedRanker` interface; that is the idea behind Facebook's early **EdgeRank** (affinity × weight × time decay).
- **"Suggestions at scale."** — Precompute friends-of-friends offline on a schedule and store them (the course notes the original diagram had a `memberSuggestions` field for exactly this), adding signals like shared groups, schools and workplaces.
- **"Blocking one member from another."** — Add a per-member block set; `canSee` and `sendRequest` check it both ways.
- **"Reactions beyond like."** — Change the like set to `Map<memberId, Reaction>`, exactly like Stack Overflow's vote map.
- **"Tagging people in posts."** — Add tagged ids to `Post`, notify them, and (a privacy decision) optionally widen the audience to their friends.
- **"Events, stories, marketplace, albums with their own privacy."** — Each is a new content type extending `SocialContent` with its own `Audience`.

Real-world example

Facebook launched **News Feed** in 2006, turning profiles you had to visit into a stream pushed to you — the feature this design adds. Posts carry an audience selector (Public, Friends, specific friends, Only me,

custom lists), and a personal profile is capped at **5,000 friends** (beyond that people use followers or pages). The social graph itself is served by **TAO**, a distributed store Facebook built for objects (users, posts) and associations (friendship, likes) — the production-scale version of our `FriendService` plus `SocialContent` likes.

In the interview

Minutes	What to do
0–5	Clarify friend vs follow, the audience options, pages, groups, feed
5–15	Profile, `SocialContent` (likes as set, comments liked too), `PrivacyList`, `Audience`
15–25	`FriendService` owns the graph (accept is two-sided); following is a set
25–35	**The strongest point:** the post holds its list and the friend service holds friendships, so `canSee` must consult both — and search, feed, comments and shares all call it
35–45	Feed (pull, push, hybrid), suggestions (mutual count, exclude pending both ways, stable sort), concurrency

30-second answer: "Posts and comments share a likes set. A post has an audience — public, friends, a custom privacy list, or only me. The post can check its own list, but only the friend service knows friendships, so one `VisibilityService.canSee` asks both, and every read path — feed, search, comment, like, share — calls it. The friend service owns the graph because accepting changes two members; following is one-sided. The feed gathers from friends, follows, pages and groups, filters by `canSee`, and ranks with a Strategy. Suggestions are friends of friends by mutual count, excluding pending requests either way, with a stable tie-break."

Follow-ups with answers

- *How could a private post leak?* — Through any read path that forgets privacy: search, feed, comment threads, shares, notifications. That is why `canSee` is one method used by all of them.
- *Why exclude pending requests in suggestions?* — Re-suggesting someone you already invited is a visible bug; and the check must look both ways.
- *Why the second sort key?* — Without it, people with the same mutual count can swap places between page loads, and a list that reshuffles itself looks broken.

Self-check

- **Q:** Erin is Alice's friend but not on "Close friends". Can she see a close-friends post? **A:** No — she must be a friend *and* on the list.
- **Q:** Why does `searchPosts` take a viewer? **A:** Because privacy applies inside search; otherwise search leaks private posts.
- **Q:** Where are follower counts stored? **A:** Nowhere — they are the size of the follower set.
- **Q:** Which three people are excluded from suggestions? **A:** Yourself, existing friends, and anyone with a pending request in either direction.
- **Q:** Name the two feed-building strategies and one weakness of each. **A:** Push (fan-out on write): expensive for accounts with huge audiences. Pull (fan-out on read): slower reads because it gathers at open time.

TL;DR

Facebook = LinkedIn's structure + **privacy**: a sealed `Audience`, privacy lists of friends, and one `VisibilityService.canSee` (post's list AND friend graph) called by feed, search, comments, likes and shares. `FriendService` owns the symmetric graph; following is a set. Pages and groups store collections, not counts. The news feed gathers, filters, and ranks with a Strategy; suggestions are friends-of-friends by mutual count, excluding pending requests both ways, with a stable tie-break.

18.5 Chapter summary

The four designs side by side

Aspect	Stack Overflow	Cricinfo	LinkedIn	Facebook
Core content	Question / answer / comment	Ball / over / innings / match	Post / comment / message	Post / comment / message / page post / group post
Relationship between users	None (votes and reputation)	Team membership, officials	Connection (mutual) + follow	Friendship (mutual) + follow
Two-sided operation → service	—	—	`ConnectionService.accept`	`FriendService.accept`
Role / permission model	Reputation privileges + Moderator/Admin types	Admin scorer, Commentator	Member / Admin	Member / Admin + audiences
"Derive, don't store"	Score from votes, tag counts	All stats from balls	Connection count, follower count	Member, follower counts, ratings
Strategy	Reputation policy, channels	(Target calculator as extension)	Feed ranker, channels	Feed ranker, channels
Observer	Reputation listeners (badges)	Match observers (live feed)	Notifications	Notifications
Template Method	—	`StatQuery`	—	—
Search	Words and tags	Stat queries	Members, companies, jobs	Members, groups, pages, privacy-aware posts

Common mistakes across all four

- Counters instead of sets (votes, likes, endorsements) — cannot stop duplicates.
- Stored counts next to the collections they summarise — they drift.
- Two-sided operations on one entity — asymmetric graphs.
- Privacy or permission checks in one read path but not another.
- Search index values that are a single object instead of a list.
- Modelling the nouns but not the rules (six balls vs six *legal* balls; run outs; byes).

Chapter TL;DR

Platforms are rich domain models. Share behaviour in a content base class; keep votes and likes as sets of ids; derive every number you can; put operations that change two objects in a service; notify through Observer with Strategy channels; rank feeds with a Strategy; keep search in an inverted index that maps words to lists; and write each permission or privacy rule exactly once, then call it from every read path.

Chapter 19 — Case Studies: Commerce & Marketplaces (Part 2 of Platforms)

> **Where this fits.** Chapter 18 designed platforms where people *talk* (Q&A sites, social networks, sports scores). This chapter designs platforms where people *buy, sell and trade*. Money moves, stock runs out, prices change every second, and two people often want the same last thing at the same moment. Those three pressures — **money must be exact, state must follow rules, and shared things must not be double-sold** — shape every design here.

What you will learn

#	Problem	The one idea that makes it work	Main patterns
19.1	Amazon — online shopping	An order is an immutable copy of the cart, moved through a **state machine**	State machine (enum), Strategy (payment), Observer (notifications), Facade
19.2	Online stock brokerage + stock-exchange matching engine	Four order types differ in only **two questions**; a book matches by **price, then time**	Template Method, Observer, Strategy (price policy)
19.3	Online auction	"Read highest bid, compare, replace" must be **one atomic step**	Monitor lock, Observer, Specification
19.4	Music streaming (Spotify)	Play order is a **pluggable strategy**; plan rules live **in the plan**	Strategy, Observer, enum-with-behaviour
19.5	Food delivery (Swiggy) + FoodKart	Capacity and riders are **claimed atomically**; selection is a **strategy**	Strategy, Observer, State machine, CAS
19.6	Grocery store (POS & checkout)	A **pure pricing engine**: best offer per product, then a coupon spread over items so tax and refunds stay exact	Strategy (offers, tax, payment), Facade, Observer, enum state machine

How to run every example in this chapter

```
cd chapters/19-case-platforms-2/code
javac --release 21 -d /tmp/lld_19 $(find . -name '*.java')
java -cp /tmp/lld_19 lld.ch19.amazon.Main          # or brokerage, stockexchange, auction, music, fooddelivery, foodkart, grocery
```

`javac` is the Java compiler (it turns `.java` files into runnable `.class` files); `-d` names the output folder, and `java -cp` runs a class, where the **classpath** (`-cp`) is the list of folders Java searches for compiled classes.

All eight `Main` classes compile with Java 21 (standard library only — only the classes that ship with Java) and run. The output under each implementation is the real output of that program. The stock-exchange `Main` also runs a small test suite (a set of automatic checks that confirm the code behaves as expected; no JUnit — the usual Java testing library — needed).

> **Code-style note.** To keep each case readable, one file sometimes holds several small package-private types (for example `Payments.java` holds `Payment`, its two subclasses and `Shipment`). That is legal Java. In production you would usually give each top-level type its own file.

> **Key term — Java vocabulary used in this chapter:** A **package-private** type has no access keyword and is visible only inside its own package (folder of classes). A **top-level type** is a class, interface, enum or record not nested in another. An **interface** lists methods a class promises to provide. An **enum** is a type with a fixed list of named values. A **record** is a compact class that just carries data that never changes. A **sealed** type lists exactly which subclasses are allowed (the `permits` clause), so the

compiler knows every possible kind. `synchronized` means only one thread at a time may run that code on the same object.

Key term — the design patterns named in this chapter: Strategy puts each interchangeable way of doing a job (each payment method, each play order) behind one interface so it can be swapped or added without editing the caller. **Observer** lets objects subscribe to events from another object and be called back automatically, so the publisher does not know who listens. **Facade** is one simple class that hides many classes behind a few easy methods. **Template Method** is a base-class method that fixes the outline of an algorithm and lets subclasses fill in one or two steps. **Specification** turns each business rule or search condition into a small yes/no object that can be combined with others.

Four rules that apply to every design in this chapter

1. **Money is `BigDecimal`, never `double`.** `double` is Java's floating-point number type: it stores numbers in binary, so most decimal fractions are only approximated. `BigDecimal` is Java's exact decimal-number class. A `double` cannot store 0.1 exactly, so `0.1 + 0.2` prints `0.30000000000000004`. With `BigDecimal` you choose the scale (number of decimal places) and the rounding mode yourself. Two traps: build it from a `String` (`new BigDecimal("0.10")`), not from a `double`; and compare with `compareTo`, not `equals` — `new BigDecimal("2.0").equals(new BigDecimal("2.00"))` is `false` because the scales differ.
2. **Status changes follow a state machine.** A **state machine** is a fixed list of states plus the allowed moves between them. We put the allowed moves *inside the enum*, so an illegal move (for example "cancel a delivered order") throws immediately instead of silently corrupting data.
3. **Check-then-act on shared stock must be atomic.** "Is there one left? Then take it" is two steps. If two threads (independent paths of execution running at the same time inside one program) run step 1 together, both see "yes" and both take it — a **race condition**, where the result depends on which thread happens to get there first. The fix is to do both steps under one lock (a guard that only one thread can hold at a time; the others wait), or with one atomic compare-and-set (CAS: "set to B only if it is still A").
4. **Copy prices into orders.** An order remembers the price that was paid, and is **immutable** (it cannot be changed after it is created). If the menu or catalog price changes later, old orders must not change.

Design Amazon — Online Shopping System

Problem & clarifying questions

Amazon started as an online bookshop and now sells almost every kind of consumer product and digital media. We design the **online retail part**: people list products, other people find them, put them in a cart, pay, and track the parcel.

Questions worth asking before you draw anything (with the answers we assume):

Question	Assumed answer
Can someone buy without an account?	No. Browsing and filling a cart are open to everyone; placing an order needs a **registered member** account.
Who sells?	Members can list products for sale (a marketplace, not only first-party — goods sold by the platform itself).
How do people pay?	Credit card or electronic bank transfer. Cards clear at once, transfers clear later.
Can an order be cancelled?	Yes, until it has shipped.
Do we notify people?	Yes, on every order or shipping status change.
Returns, wish lists, seller payouts, recommendations, stock spread over several warehouses?	Out of scope — a deliberate choice, and you should say so out loud.

Requirements

Functional

1. Members can add new products to sell.
2. Anyone can search products by **name** or **category**.
3. Anyone can search and view products; only registered members can buy.
4. Users can add, remove and change the quantity of items in their shopping cart.
5. Users can check out and buy everything in the cart.
6. Members can rate and review a product.
7. Members give a shipping address for delivery.
8. A member can cancel an order that has not shipped.
9. Members are notified each time the status of their order or of its shipment changes.
10. Members pay with a credit card or with an electronic bank transfer.
11. Members can track the shipment and see the current order state.

Non-functional

- **Correctness of stock:** two buyers can never both get the last unit.
- **Exact money:** totals are computed with `BigDecimal`.
- **Auditability** (being able to prove later exactly what happened and when)**:** every order and shipment keeps a history of its states, not just the latest one.
- **Extensibility** (how easily new features can be added without rewriting old code)**:** adding a payment method or a notification channel must not change the checkout code.

Use cases

Four actors:

- **Admin** — manages accounts (block / unblock) and adds or changes product categories.
- **Guest** — searches the catalog, adds and removes cart items, and can register to become a member.
- **Member** — everything a guest can do, plus placing orders and listing products to sell.
- **System** — sends order and shipment notifications.

Top use cases: add or update products (each change also updates the catalog), search products (by name, or by category), put items into the cart or take them out, check out, pay, add a product category, and send shipment notifications.

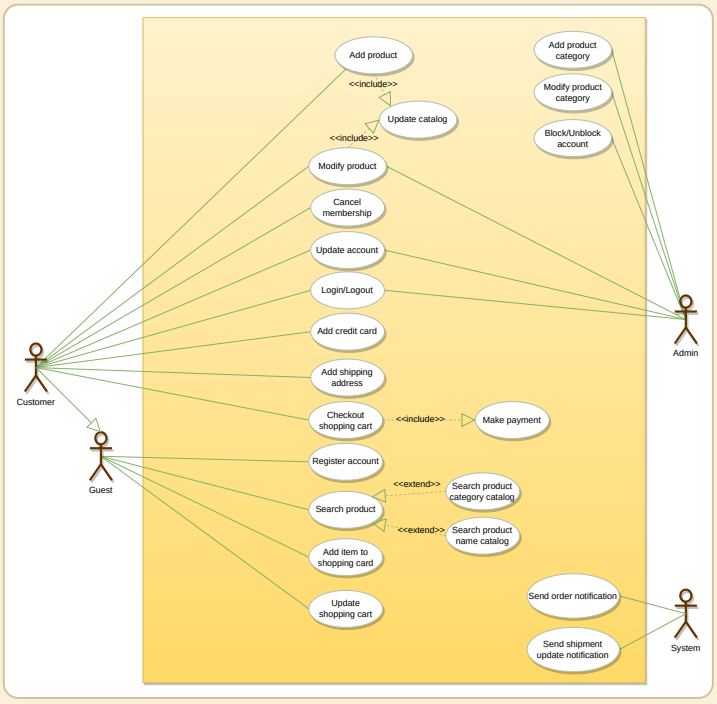


Figure — Use-case diagram for the online shopping system: Admin, Guest, Member and System and what each may do (course diagram)

In the figure above, notice that "Checkout" *includes* "Make payment" and "Add shipping address": you cannot check out without them. Search by name and search by category *extend* the general "Search product" use case.

Core entities

Class	Responsibility
`ProductCategory`	A named group such as Books or Electronics. Exists whether or not any product uses it.
`Product`	The thing sold: price, seller, stock count. **Owns** its reviews and guards its own stock (`reserve` / `release`).
`ProductReview`	Rating 1–5 plus text. Records the reviewer **by id**, so `Product` never depends on `Member`.
`Search` / `Catalog`	Interface with two search methods. The catalog keeps one index (a lookup map built in advance) per search (name words → products, category → products), so a lookup reads a map instead of scanning every product, and `addProduct` updates both indexes together so they cannot drift apart. The catalog only *points to* products; it never owns them.
`ShoppingCart`	Mutable (changeable) working area. **Owns** its items; clearing the cart destroys them.
`Item`	One line: product id, name, quantity, unit price (an immutable `record`). A pen is a *product*; ten pens in stock are ten product *items*.
`Order`	Immutable **copy** of the cart lines + address + a status that moves only along legal edges.
`OrderLog`	One entry per status change (unshipped, pending, complete, cancelled...).
`Payment` (`CreditCardPayment`, `BankTransferPayment`)	Takes the money. A card clears immediately; a bank transfer comes back `PENDING` from `process()` and clears later.
`Shipment` / `ShipmentLog`	A parcel and its tracking history (pending, shipped, delivered, on hold). The last log entry is "where is it now".
`Notification` channel (`EmailChannel`, `SmsChannel`)	One way of delivering a message. (The course draws an abstract `Notification` class with one subclass per channel; we use an interface — same idea.)
`Account`	Login data, addresses, status (active, blocked, banned...).
`Customer` → `Guest`, `Member`	`Customer` has only what *anyone* may do; ordering lives on `Member`.
`Admin`	Adds categories, blocks and unblocks members.
`OrderService`	Application service (a class that coordinates domain objects to carry out one user action) that turns a cart into an order and drives its state machine.

Class diagram

The course builds the model in five stages (products → catalog → payment/shipment/notification → cart/order → accounts). Here is the finished picture:

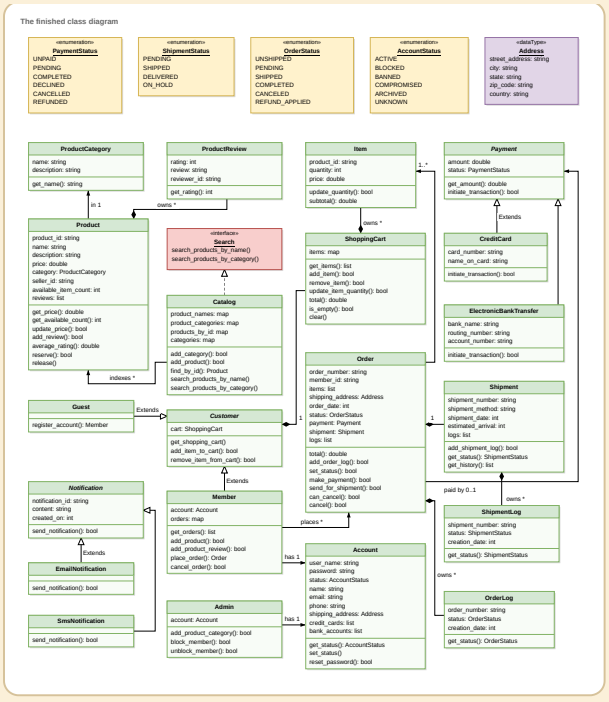


Figure — The finished class diagram of the online shopping system (course diagram)

In the figure above, read the arrows this way: a **filled diamond** means *composition* (the part belongs to the whole and is destroyed along with it — `Product` owns its reviews, `ShoppingCart` owns its items, `Order` owns its log and shipment). A **plain arrow** is *association* (a link without ownership — a product points at its category; an order only *refers to* the payment it was handed). The hollow triangle is inheritance (`Guest` and `Member` extend `Customer`).

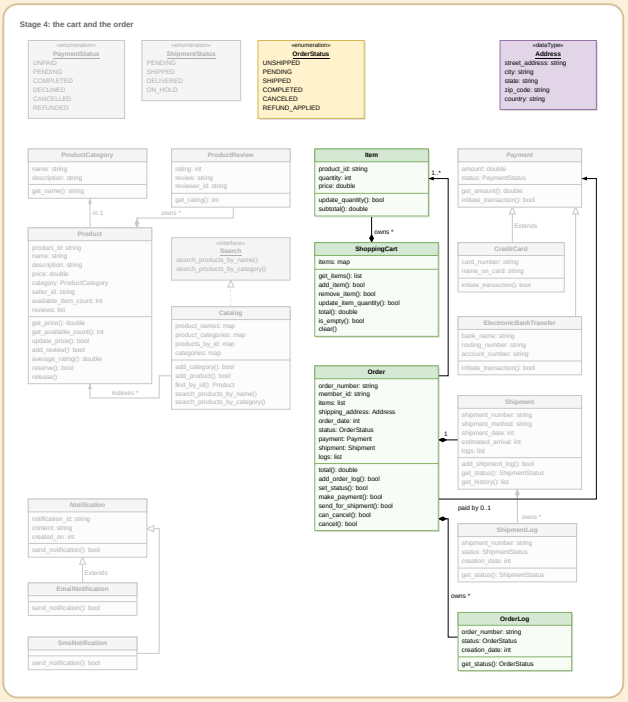
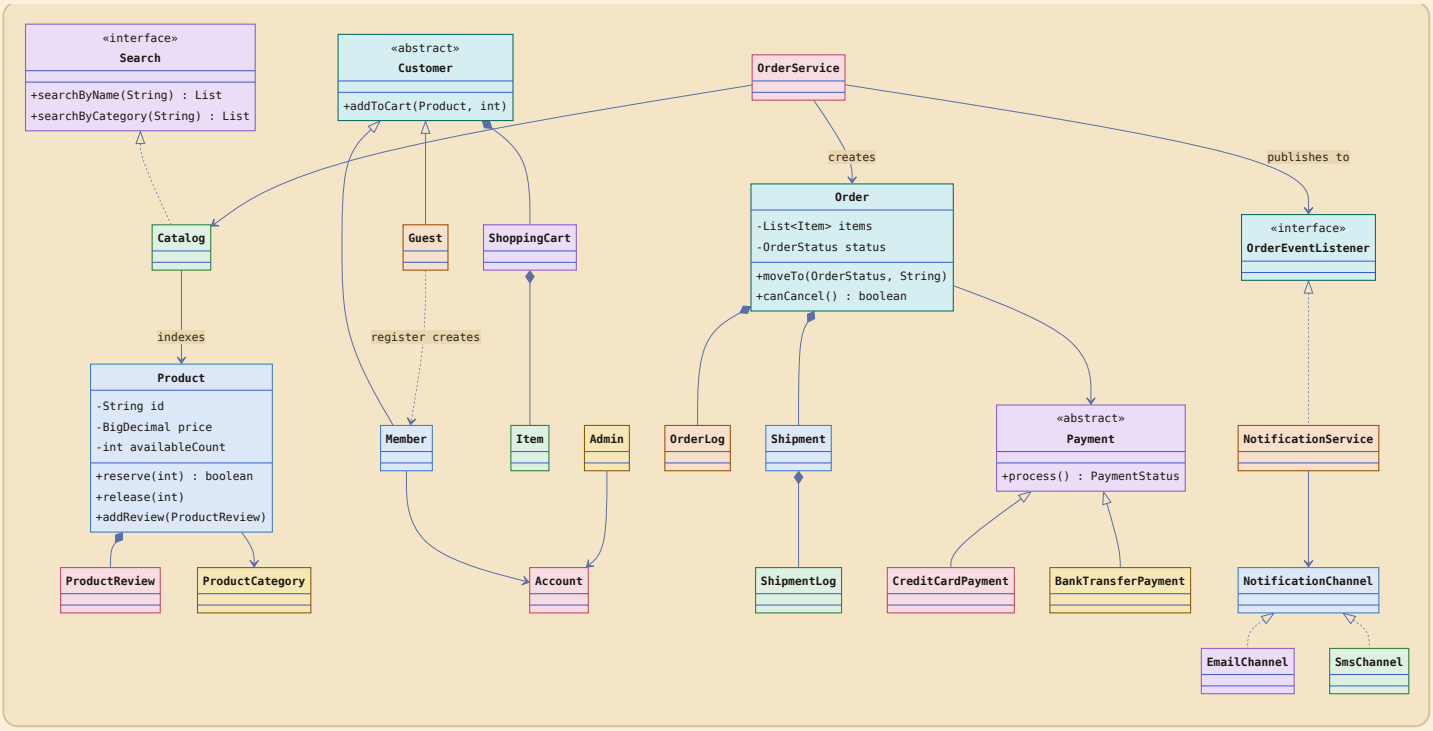


Figure — Stage 4: the cart owns its items, while the order holds its own copy of what was bought (course diagram)

Our Java design follows the same shape:



Key flows

Shopping activity. The course's activity diagram shows the loop a shopper goes round: search, view, add to cart, update the cart, "more shopping?", and finally checkout.

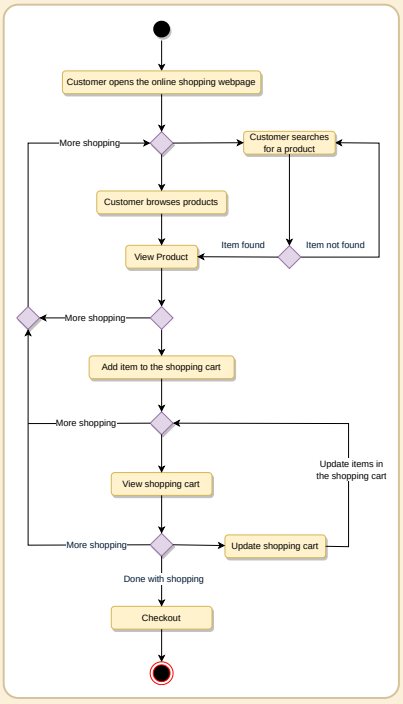


Figure — Activity diagram of a customer shopping online, from opening the site to checkout (course diagram)

Searching the catalog, and adding to the cart. The first two course sequence diagrams show the simple read path (customer → catalog → matching products) and the cart update path (customer → cart → item).

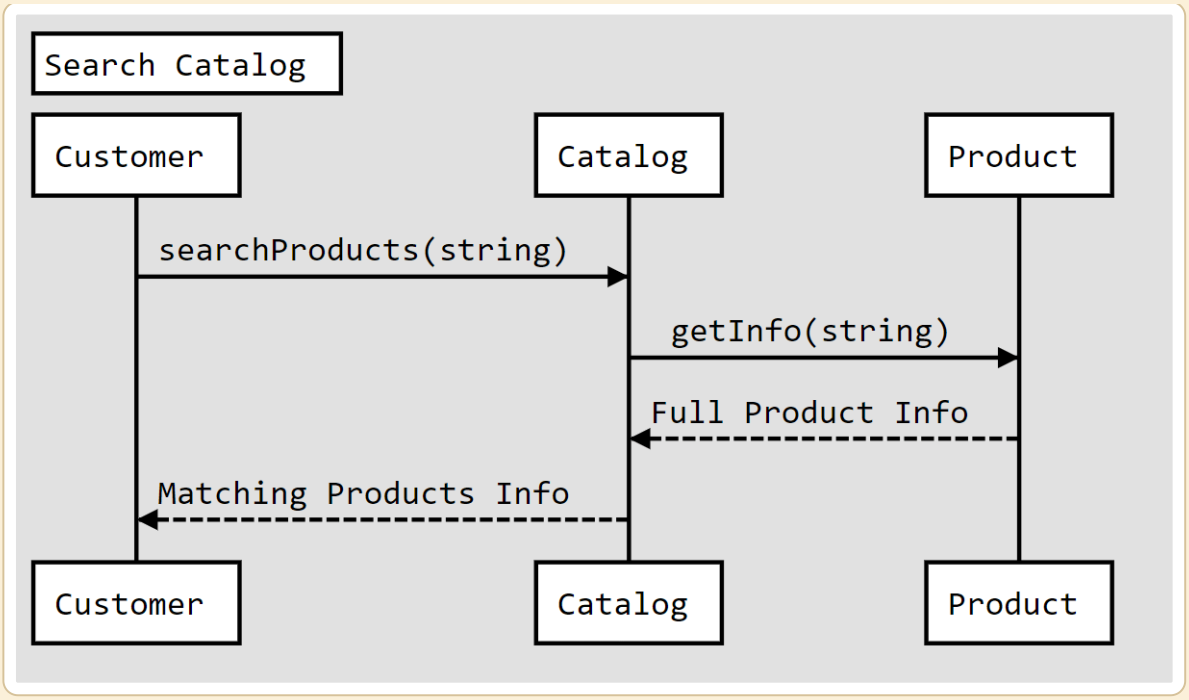


Figure — Sequence diagram for searching the catalog (course diagram)

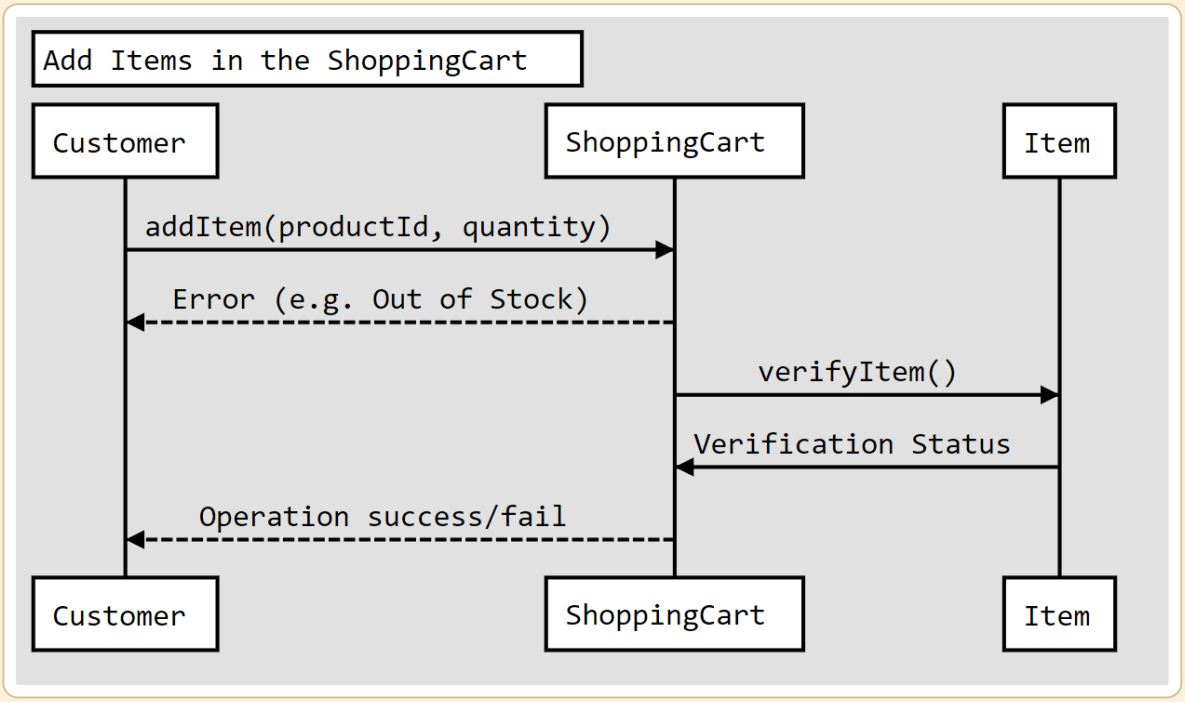


Figure — Sequence diagram for adding an item to the shopping cart (course diagram)

Checkout. The third course sequence diagram shows the important one: get the cart items, get billing and shipping info, create the order, verify every item, process the payment, create the shipment, and return the order status — with an error path after item verification and after payment.

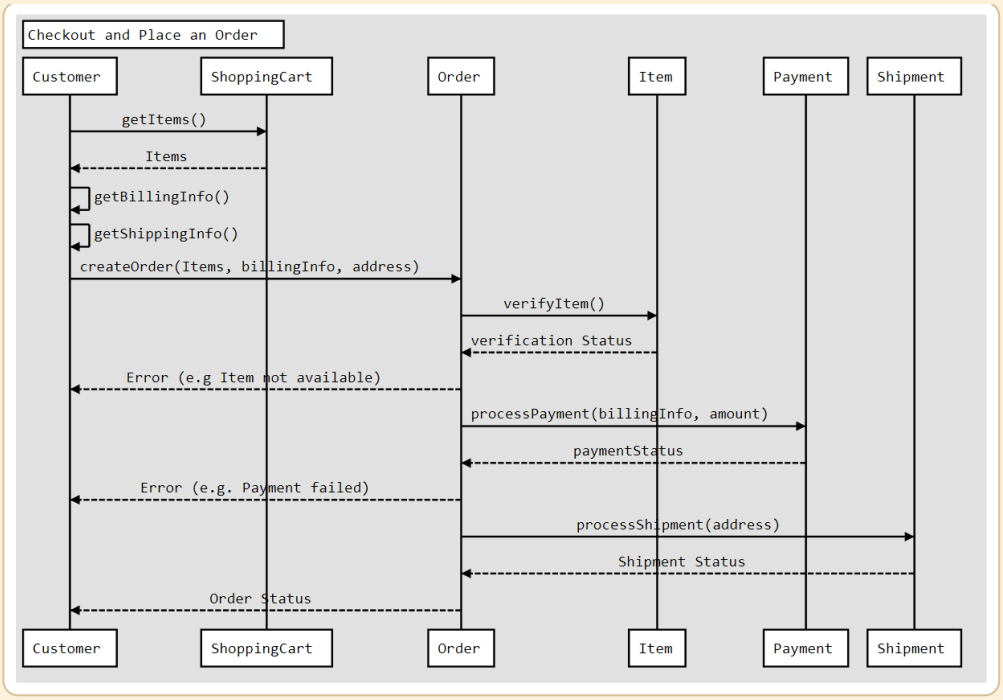
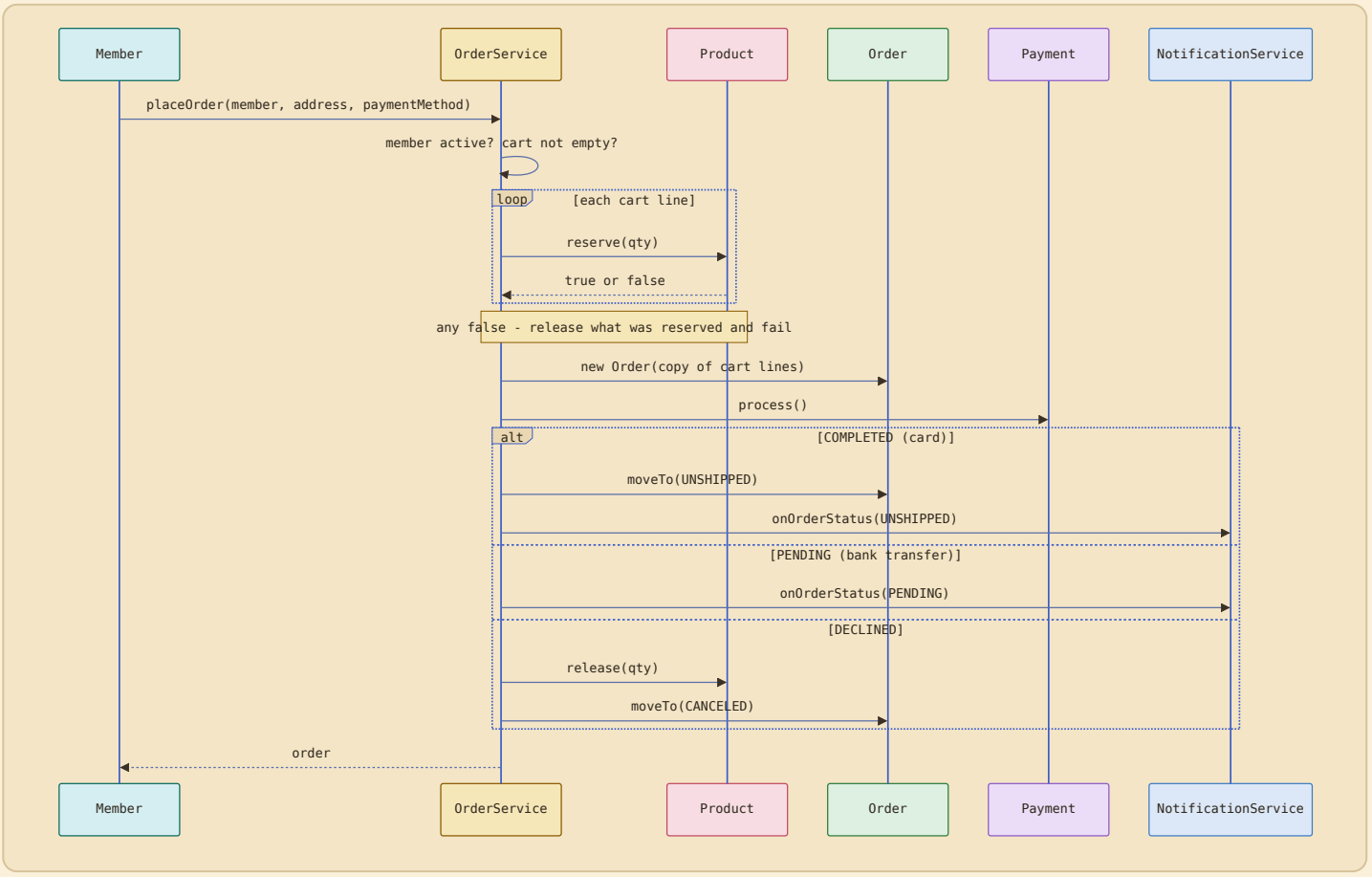
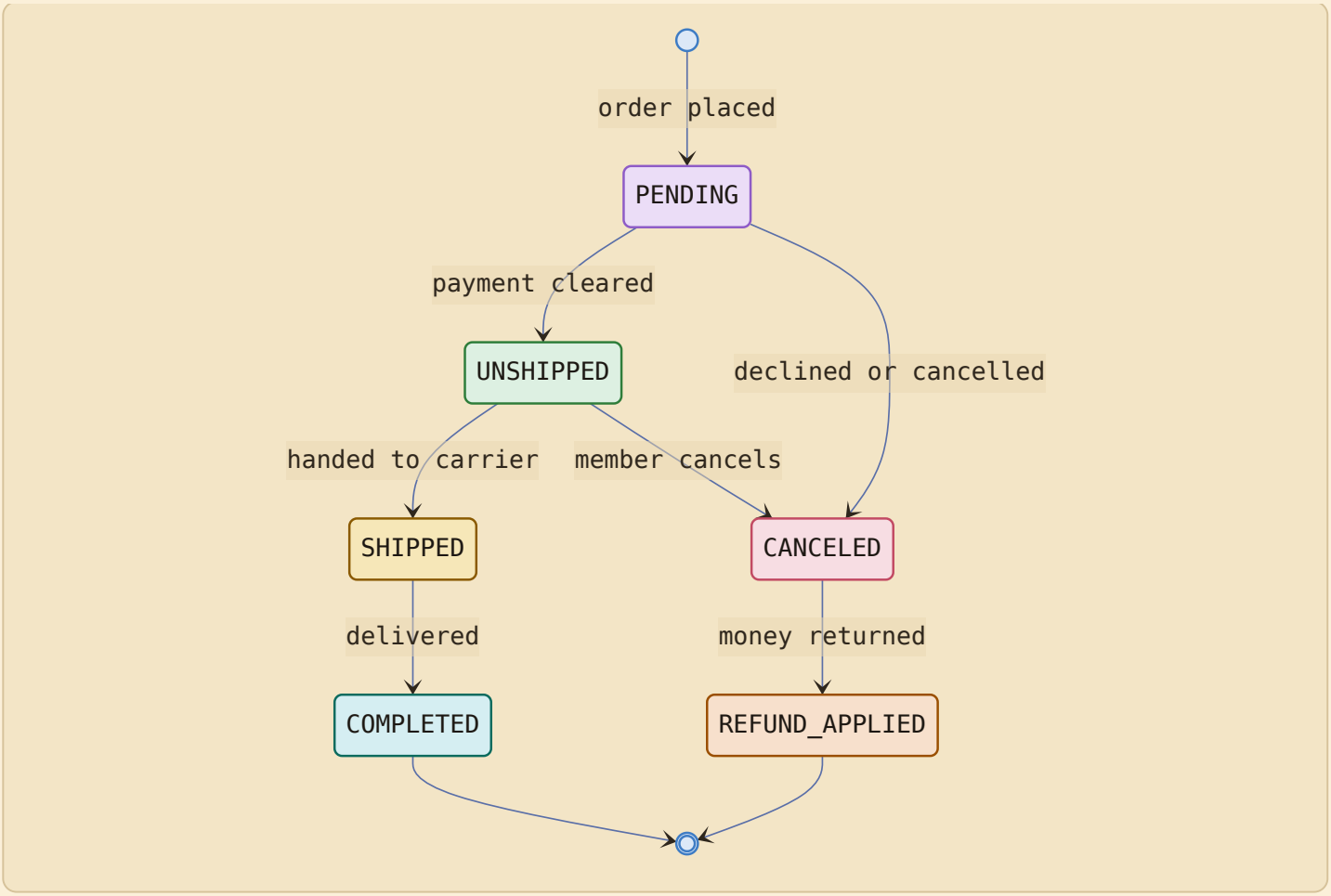


Figure — Sequence diagram for checkout: verify items, take payment, create shipment (course diagram)

Our `OrderService.placeOrder` follows that order, with one addition: stock is **reserved** during verification (not merely checked), so the check cannot go stale before payment.



The order state machine. Every legal move is listed in `OrderStatus.next()`. Anything else throws.



Requirement 8 ("cancel only if not shipped") is now one line: `canCancel()` returns `status.canMoveTo(CANCELED)`. There is no `SHIPPED → CANCELED` edge, so it is impossible by construction.

Design decisions & patterns used

Decision	Why	Alternative rejected
Ordering lives on `Member`, not `Customer`	A `Guest` *is a* `Customer`. If `Customer` had `placeOrder`, every guest would inherit an operation it has to reject — breaking the **Liskov Substitution Principle** (a subclass must be usable wherever its parent is, without surprises).	`Customer.placeOrder()` that throws for guests (the classic LSP smell).
`Guest.register()` returns a new `Member` and carries the cart across	Registration is a real-world moment where the cart must survive.	Mutating a guest into a member (Java objects cannot change class).
`Order` copies the cart lines	An order documents a *finished purchase*; it must stay fixed even if the cart is edited later.	Sharing `Item` objects — cheaper, but editing the cart would silently rewrite a finished order.
Stock reserved at order time, not at payment time	Prevents two shoppers from both getting the final unit. Cost: an abandoned order holds stock until cancelled; real systems release it on a timer.	Check stock, then pay, then decrement (a race window between check and decrement).
State machine inside `OrderStatus`	One place owns the rules. `moveTo` rejects illegal edges.	`if` checks scattered over `cancel`, `ship`, `deliver`.
`Payment` is an abstract (sealed) class, one subclass per method — a **Strategy**	The two methods *behave* differently (card clears now, transfer is pending). An **abstract class** cannot be instantiated itself — only its subclasses can — so every payment is a real card or transfer. Each subclass passes the substitution test (it can genuinely stand in for `Payment`: both really take money), and the same holds for the notification channels. A new method = a new subclass; checkout does not change (Open/Closed Principle: code should be open to extension but closed to modification). The course calls this out: it is why `Payment` is not an enum.	`enum PaymentType` + `switch` in checkout.
Payment passed in as `Function<BigDecimal, Payment>` (a function object: give it an amount, get a `Payment` back)	Checkout knows the total only after building the order, so it asks the caller's factory (a piece of code whose job is to create objects) for a payment of that amount.	Creating the payment before the total is known.
Observer for notifications	`OrderService` publishes events; `NotificationService` decides who to tell and how. Adding push notifications = a new `NotificationChannel`.	Calling email and SMS code directly inside `OrderService`.
`Shipment` keeps a log	Requirement 11 = read the last entry; the whole history stays for support and disputes.	A single `status` field (history lost).
Reviews store `reviewerId`	Avoids a two-way dependency `Product ↔ Member`; each class can be understood alone.	`ProductReview` holding a `Member` object.
`OrderService` as a Facade	One simple entry point hides stock, payment, shipment and notification steps.	Callers orchestrating five objects themselves.

Java implementation

Files: `Money.java` (BigDecimal helpers), `Catalog.java` (category, product, review, search, catalog), `Order.java` (address, item, cart, order status machine, order), `Payments.java` (payments, shipment), `Accounts.java` (account, customer, guest, member, admin), `OrderService.java` (listeners, notifications, service), `Main.java` (demo).

```java
// File: lld/ch19/amazon/Accounts.java
package lld.ch19.amazon;

import java.util.ArrayList;
import java.util.List;

enum AccountStatus { ACTIVE, BLOCKED, BANNED, COMPROMISED, ARCHIVED, UNKNOWN }

/** Login data shared by members and admins. */
final class Account {
    private final String userName;
    private final String email;
    private final String phone;
    private final List<Address> addresses = new ArrayList<>();
    private volatile AccountStatus status = AccountStatus.ACTIVE;

    Account(String userName, String email, String phone) {
        this.userName = userName;
        this.email = email;
        this.phone = phone;
    }

    String userName() { return userName; }
    String email() { return email; }
    String phone() { return phone; }
    AccountStatus status() { return status; }
    void setStatus(AccountStatus s) { status = s; }
    synchronized void addAddress(Address a) { addresses.add(a); }
    synchronized List<Address> addresses() { return List.copyOf(addresses); }
}

/**
 * Everything any shopper may do - browse and fill a cart. Placing an order is
 * deliberately NOT here, because a Guest is a Customer and must not inherit it (LSP).
 */
abstract sealed class Customer permits Guest, Member {
    private final ShoppingCart cart = new ShoppingCart();

    ShoppingCart cart() { return cart; }
    void addToCart(Product p, int qty) { cart.addItem(p, qty); }
    void removeFromCart(String productId) { cart.removeItem(productId); }
}

final class Guest extends Customer {
    /** Registering turns a guest into a member and carries the cart across. */
    Member register(String memberId, Account account) {
        Member m = new Member(memberId, account);
        m.cart().mergeFrom(cart());
        cart().clear();
        return m;
    }
}

final class Member extends Customer {
    private final String id;
    private final Account account;

    Member(String id, Account account) {
        this.id = id;
        this.account = account;
    }

    String id() { return id; }
    Account account() { return account; }
    boolean canBuy() { return account.status() == AccountStatus.ACTIVE; }
}

final class Admin {
    private final Account account;

    Admin(Account account) { this.account = account; }

    ProductCategory addCategory(String name, String description) { return new ProductCategory(name, description); }
```

```
    void block(Member m) { m.account().setStatus(AccountStatus.BLOCKED); }

    void unblock(Member m) { m.account().setStatus(AccountStatus.ACTIVE); }

    String name() { return account.userName(); }
}
```

```java
// File: lld/ch19/amazon/Catalog.java
package lld.ch19.amazon;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.HashSet;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.Optional;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;

/** A category such as "Books". Products point at it; it does not own them. */
record ProductCategory(String name, String description) { }

/** A review names its reviewer by id, so Product never depends on Member. */
record ProductReview(String reviewerId, int rating, String text) {
    ProductReview {
        if (rating < 1 || rating > 5) throw new IllegalArgumentException("rating must be 1..5");
    }
}

/** The thing being sold. Owns its reviews and guards its own stock count. */
final class Product {
    private final String id;
    private final String name;
    private final ProductCategory category;
    private final String sellerId;
    private volatile BigDecimal price;
    private int availableCount;                       // guarded by this
    private final List<ProductReview> reviews = new CopyOnWriteArrayList<>();

    Product(String id, String name, ProductCategory category, BigDecimal price,
            String sellerId, int availableCount) {
        this.id = id;
        this.name = name;
        this.category = category;
        this.price = Money.of(price);
        this.sellerId = sellerId;
        this.availableCount = availableCount;
    }

    String id() { return id; }
    String name() { return name; }
    ProductCategory category() { return category; }
    String sellerId() { return sellerId; }
    BigDecimal price() { return price; }
    synchronized int availableCount() { return availableCount; }

    void updatePrice(BigDecimal newPrice) {
        if (newPrice.signum() < 0) throw new IllegalArgumentException("negative price");
        price = Money.of(newPrice);
    }

    /** Take stock out of the pool. False means "not enough" - nothing changes. */
    synchronized boolean reserve(int quantity) {
        if (quantity <= 0 || quantity > availableCount) return false;
        availableCount -= quantity;
        return true;
    }

    synchronized void release(int quantity) { availableCount += quantity; }

    void addReview(ProductReview review) { reviews.add(review); }

    double averageRating() {
        return reviews.stream().mapToInt(ProductReview::rating).average().orElse(0.0);
    }
```

```
    @Override public String toString() { return name + " (" + Money.fmt(price) + ")"; }
}

/** Requirement: search by name or by category. */
interface Search {
    List<Product> searchByName(String text);
    List<Product> searchByCategory(String categoryName);
}

/**
 * The catalog points to products but never owns them. It keeps one index per search
 * (words of the name, category name), so a lookup is a map read, not a scan of every
 * product. Every index is updated in one method (addProduct) so they can never drift apart.
 */
final class Catalog implements Search {
    private final Map<String, Product> byId = new ConcurrentHashMap<>();
    private final Map<String, Set<Product>> byNameWord = new ConcurrentHashMap<>();
    private final Map<String, List<Product>> byCategory = new ConcurrentHashMap<>();

    void addProduct(Product p) {
        byId.put(p.id(), p);
        for (String word : words(p.name()))
            byNameWord.computeIfAbsent(word, k -> ConcurrentHashMap.newKeySet()).add(p);
        byCategory.computeIfAbsent(key(p.category().name()), k -> new CopyOnWriteArrayList<>()).add(p);
    }

    Optional<Product> find(String productId) { return Optional.ofNullable(byId.get(productId)); }

    /** Products whose name contains every word of the query (e.g. "clean code"). */
    @Override public List<Product> searchByName(String text) {
        List<String> query = words(text);
        if (query.isEmpty()) return List.of();
        Set<Product> hits = new HashSet<>(byNameWord.getOrDefault(query.get(0), Set.of()));
        for (String w : query.subList(1, query.size())) hits.retainAll(byNameWord.getOrDefault(w, Set.of()));
        List<Product> out = new ArrayList<>(hits);
        out.sort((a, b) -> a.name().compareTo(b.name()));
        return out;
    }

    @Override public List<Product> searchByCategory(String categoryName) {
        return Collections.unmodifiableList(byCategory.getOrDefault(key(categoryName), List.of()));
    }

    private static List<String> words(String s) {
        return Arrays.stream(key(s).split("[^\\p{Alnum}]+")).filter(w -> !w.isEmpty()).toList();
    }

    private static String key(String s) { return s.toLowerCase(Locale.ROOT).trim(); }
}
```

```java
// File: lld/ch19/amazon/Main.java
package lld.ch19.amazon;

import java.math.BigDecimal;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    public static void main(String[] args) throws Exception {
        Admin admin = new Admin(new Account("root", "admin@shop.test", "000"));
        ProductCategory books = admin.addCategory("Books", "Printed and e-books");
        ProductCategory electronics = admin.addCategory("Electronics", "Gadgets");

        Catalog catalog = new Catalog();
        catalog.addProduct(new Product("P1", "Clean Code", books, Money.of("32.50"), "seller-9", 5));
        catalog.addProduct(new Product("P2", "Refactoring", books, Money.of("41.00"), "seller-9", 2));
        catalog.addProduct(new Product("P3", "USB-C Cable", electronics, Money.of("9.99"), "seller-4", 100));
        catalog.addProduct(new Product("P4", "Design Patterns (last copy)", books, Money.of("55.00"), "seller-2", 1));

        System.out.println("== 1. Search ==");
        System.out.println("name 'code'      -> " + catalog.searchByName("code"));
        System.out.println("name 'usb cable' -> " + catalog.searchByName("usb cable"));
        System.out.println("category 'books' -> " + catalog.searchByCategory("books"));

        Map<String, Account> accounts = new ConcurrentHashMap<>();
        OrderService orders = new OrderService(catalog);
        orders.subscribe(new NotificationService(accounts::get, List.of(new EmailChannel(), new SmsChannel())));

        System.out.println("\n== 2. Guest fills a cart, then registers (cart carries over) ==");
        Guest guest = new Guest();
        guest.addToCart(catalog.find("P1").orElseThrow(), 1);
        guest.addToCart(catalog.find("P3").orElseThrow(), 3);
        Account aliceAcc = new Account("alice", "alice@mail.test", "+1-555-0101");
        Member alice = guest.register("M1", aliceAcc);
        accounts.put(alice.id(), aliceAcc);
        alice.cart().updateQuantity("P3", 2);
        alice.cart().snapshot().forEach(i -> System.out.println("  cart: " + i.quantity() + " x " + i.name()
                + " = " + Money.fmt(i.lineTotal())));

        System.out.println("\n== 3. Checkout with a credit card ==");
        Address home = new Address("1 Main St", "Seattle", "98101", "US");
        Order o1 = orders.placeOrder(alice, home,
                amt -> new CreditCardPayment(amt, "4242", Money.of("1000.00")));
        System.out.println("  " + o1.orderNumber() + " total " + Money.fmt(o1.total()) + ", status " + o1.status()
                + ", stock of Clean Code now " + catalog.find("P1").orElseThrow().availableCount());

        System.out.println("\n== 4. Ship, track, deliver ==");
        orders.ship(o1.orderNumber(), "UPS Ground");
        orders.updateShipment(o1.orderNumber(), ShipmentStatus.SHIPPED, "Portland hub");
        orders.updateShipment(o1.orderNumber(), ShipmentStatus.DELIVERED, "front door");
        System.out.println("  tracking now: " + o1.shipment().current() + ", order " + o1.status());
        System.out.println("  cancel after delivery? " + orders.cancel(o1.orderNumber()));

        System.out.println("\n== 5. Bank transfer (pending) then cancel before shipping -> refund ==");
        alice.addToCart(catalog.find("P2").orElseThrow(), 2);
        Order o2 = orders.placeOrder(alice, home, amt -> new BankTransferPayment(amt, "Chase"));
        System.out.println("  " + o2.orderNumber() + " status " + o2.status() + ", payment " + o2.payment().status());
        orders.onPaymentSettled(o2.orderNumber());
        System.out.println("  cancel before shipping? " + orders.cancel(o2.orderNumber())
                + " -> " + o2.status() + ", payment " + o2.payment().status()
                + ", Refactoring stock back to " + catalog.find("P2").orElseThrow().availableCount());
        System.out.println("  order log: " + o2.log().stream().map(OrderLog::status).toList());

        System.out.println("\n== 6. Illegal transition is rejected by the state machine ==");
        try {
            orders.ship(o2.orderNumber(), "FedEx");
        } catch (IllegalStateException e) {
            System.out.println("  " + e.getMessage());
        }
```

```
        System.out.println("\n== 7. Review ==");
        Product cleanCode = catalog.find("P1").orElseThrow();
        cleanCode.addReview(new ProductReview(alice.id(), 5, "Changed how I name things"));
        cleanCode.addReview(new ProductReview("M7", 4, "Good, a bit long"));
        System.out.println("  Clean Code average rating = " + cleanCode.averageRating());

        System.out.println("\n== 8. Blocked member cannot buy ==");
        admin.block(alice);
        alice.addToCart(catalog.find("P3").orElseThrow(), 1);
        try {
            orders.placeOrder(alice, home, amt -> new CreditCardPayment(amt, "4242", Money.of("1000")));
        } catch (IllegalStateException e) {
            System.out.println("  rejected: " + e.getMessage());
        }
        admin.unblock(alice);
        alice.cart().clear();

        System.out.println("\n== 9. Ten members race for the last copy ==");
        OrderService quiet = new OrderService(catalog);          // no listeners: keep output short
        Product last = catalog.find("P4").orElseThrow();
        CountDownLatch start = new CountDownLatch(1);
        AtomicInteger winners = new AtomicInteger();
        AtomicInteger losers = new AtomicInteger();
        Thread[] threads = new Thread[10];
        for (int t = 0; t < threads.length; t++) {
            Member m = new Member("R" + t, new Account("racer" + t, "r" + t + "@mail.test", "-"));
            m.addToCart(last, 1);
            threads[t] = new Thread(() -> {
                try {
                    start.await();
                    quiet.placeOrder(m, home, amt -> new CreditCardPayment(amt, "1111", new BigDecimal("500")));
                    winners.incrementAndGet();
                } catch (IllegalStateException e) {
                    losers.incrementAndGet();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            });
            threads[t].start();
        }
        start.countDown();
        for (Thread t : threads) t.join();
        System.out.println("  winners=" + winners + " losers=" + losers + " stock left=" + last.availableCount());
    }
}
```

```
// File: lld/ch19/amazon/Money.java
package lld.ch19.amazon;

import java.math.BigDecimal;
import java.math.RoundingMode;

/** Money helpers: always BigDecimal, always 2 decimal places, never double. */
final class Money {
    private Money() { }

    static final BigDecimal ZERO = of("0");

    static BigDecimal of(String amount) { return new BigDecimal(amount).setScale(2, RoundingMode.HALF_EVEN); }

    static BigDecimal of(BigDecimal amount) { return amount.setScale(2, RoundingMode.HALF_EVEN); }

    static String fmt(BigDecimal amount) { return "$" + amount.setScale(2, RoundingMode.HALF_EVEN).toPlainString(); }
}
```

```java
// File: lld/ch19/amazon/Order.java
package lld.ch19.amazon;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.Collections;
import java.util.EnumSet;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Set;

record Address(String street, String city, String zip, String country) {
    @Override public String toString() { return street + ", " + city + " " + zip + ", " + country; }
}

/** One line of a cart or an order: an immutable snapshot of product, quantity and unit price. */
record Item(String productId, String name, int quantity, BigDecimal unitPrice) {
    Item {
        if (quantity <= 0) throw new IllegalArgumentException("quantity must be positive");
    }
    BigDecimal lineTotal() { return unitPrice.multiply(BigDecimal.valueOf(quantity)); }
}

/** A mutable working area. Owns its items (they die when the cart is cleared). */
final class ShoppingCart {
    private final Map<String, Item> items = new LinkedHashMap<>();

    synchronized void addItem(Product p, int qty) {
        items.merge(p.id(), new Item(p.id(), p.name(), qty, p.price()),
                (old, add) -> new Item(old.productId(), old.name(), old.quantity() + add.quantity(), p.price()));
    }

    synchronized void updateQuantity(String productId, int qty) {
        if (qty == 0) { items.remove(productId); return; }
        Item old = items.get(productId);
        if (old == null) throw new IllegalArgumentException("not in cart: " + productId);
        items.put(productId, new Item(old.productId(), old.name(), qty, old.unitPrice()));
    }

    synchronized void removeItem(String productId) { items.remove(productId); }

    /** A copy, so an order never shares its lines with the cart. */
    synchronized List<Item> snapshot() { return List.copyOf(items.values()); }

    synchronized boolean isEmpty() { return items.isEmpty(); }

    synchronized void clear() { items.clear(); }

    synchronized void mergeFrom(ShoppingCart other) {
        for (Item i : other.snapshot()) items.merge(i.productId(), i,
                (a, b) -> new Item(a.productId(), a.name(), a.quantity() + b.quantity(), a.unitPrice()));
    }
}

/** The order state machine. Each state lists the states it may move to. */
enum OrderStatus {
    PENDING, UNSHIPPED, SHIPPED, COMPLETED, CANCELED, REFUND_APPLIED;

    private Set<OrderStatus> next() {
        return switch (this) {
            case PENDING -> EnumSet.of(UNSHIPPED, CANCELED);      // waiting for payment
            case UNSHIPPED -> EnumSet.of(SHIPPED, CANCELED);      // paid, not yet handed to carrier
            case SHIPPED -> EnumSet.of(COMPLETED);                // too late to cancel
            case CANCELED -> EnumSet.of(REFUND_APPLIED);          // money goes back if it was taken
            case COMPLETED, REFUND_APPLIED -> EnumSet.noneOf(OrderStatus.class);
        };
    }

    boolean canMoveTo(OrderStatus target) { return next().contains(target); }
}

record OrderLog(OrderStatus status, String note) { }
```

```java
/** A fixed copy of the purchased lines, plus a status that only moves along legal edges. */
final class Order {
    private final String orderNumber;
    private final String memberId;
    private final List<Item> items;              // copied from the cart, never shared
    private final Address shippingAddress;
    private final List<OrderLog> log = new ArrayList<>();
    private OrderStatus status = OrderStatus.PENDING;
    private Payment payment;
    private Shipment shipment;

    Order(String orderNumber, String memberId, List<Item> items, Address shippingAddress) {
        this.orderNumber = orderNumber;
        this.memberId = memberId;
        this.items = List.copyOf(items);
        this.shippingAddress = shippingAddress;
        log.add(new OrderLog(status, "order placed"));
    }

    String orderNumber() { return orderNumber; }
    String memberId() { return memberId; }
    List<Item> items() { return items; }
    Address shippingAddress() { return shippingAddress; }
    synchronized OrderStatus status() { return status; }
    synchronized Payment payment() { return payment; }
    synchronized Shipment shipment() { return shipment; }
    synchronized List<OrderLog> log() { return Collections.unmodifiableList(new ArrayList<>(log)); }

    BigDecimal total() {
        return items.stream().map(Item::lineTotal).reduce(Money.ZERO, BigDecimal::add);
    }

    synchronized void attachPayment(Payment p) { this.payment = p; }
    synchronized void attachShipment(Shipment s) { this.shipment = s; }

    /** Requirement 8 lives in exactly one place. */
    synchronized boolean canCancel() { return status.canMoveTo(OrderStatus.CANCELED); }

    synchronized void moveTo(OrderStatus target, String note) {
        if (!status.canMoveTo(target))
            throw new IllegalStateException(orderNumber + ": " + status + " -> " + target + " is not allowed");
        status = target;
        log.add(new OrderLog(target, note));
    }
}
```

```java
// File: lld/ch19/amazon/OrderService.java
package lld.ch19.amazon;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.function.Function;

/** Observer: anything that wants to hear about order and shipment changes. */
interface OrderEventListener {
    void onOrderStatus(Order order, OrderStatus status);
    void onShipmentUpdate(Order order, ShipmentLog entry);
}

/** One way of delivering a message (email, SMS, push...). */
interface NotificationChannel {
    void send(Account to, String message);
}

final class EmailChannel implements NotificationChannel {
    @Override public void send(Account to, String message) {
        System.out.println("    [email -> " + to.email() + "] " + message);
    }
}

final class SmsChannel implements NotificationChannel {
    @Override public void send(Account to, String message) {
        System.out.println("    [sms   -> " + to.phone() + "] " + message);
    }
}

/** Requirement 9: tell the member whenever order or shipping status changes. */
final class NotificationService implements OrderEventListener {
    private final Function<String, Account> accountLookup;
    private final List<NotificationChannel> channels;

    NotificationService(Function<String, Account> accountLookup, List<NotificationChannel> channels) {
        this.accountLookup = accountLookup;
        this.channels = List.copyOf(channels);
    }

    @Override public void onOrderStatus(Order order, OrderStatus status) {
        Account to = accountLookup.apply(order.memberId());
        String msg = "Order " + order.orderNumber() + " is now " + status;
        channels.forEach(c -> c.send(to, msg));
    }

    @Override public void onShipmentUpdate(Order order, ShipmentLog entry) {
        Account to = accountLookup.apply(order.memberId());
        channels.get(0).send(to, "Shipment for " + order.orderNumber() + ": " + entry.status() + " @ " +
entry.location());
    }
}

/** Application service (facade) that turns a cart into an order and drives the order state machine. */
final class OrderService {
    private final Catalog catalog;
    private final Map<String, Order> orders = new ConcurrentHashMap<>();
    private final List<OrderEventListener> listeners = new CopyOnWriteArrayList<>();
    private final AtomicInteger orderSeq = new AtomicInteger(1000);
    private final AtomicInteger shipSeq = new AtomicInteger(1);

    OrderService(Catalog catalog) { this.catalog = catalog; }

    void subscribe(OrderEventListener l) { listeners.add(l); }

    /**
     * Checkout. Stock is reserved when the order is placed (not when paid), all-or-nothing,
     * so two buyers can never both take the last copy.
```

```
     */
    Order placeOrder(Member member, Address address, Function<BigDecimal, Payment> paymentMethod) {
        if (!member.canBuy()) throw new IllegalStateException("account " + member.account().status());
        List<Item> items = member.cart().snapshot();
        if (items.isEmpty()) throw new IllegalStateException("cart is empty");

        List<Item> reserved = new ArrayList<>();
        for (Item i : items) {
            Product p = catalog.find(i.productId()).orElseThrow();
            if (!p.reserve(i.quantity())) {
                reserved.forEach(r -> catalog.find(r.productId()).orElseThrow().release(r.quantity()));
                throw new IllegalStateException("out of stock: " + i.name());
            }
            reserved.add(i);
        }

        Order order = new Order("ORD-" + orderSeq.incrementAndGet(), member.id(), items, address);
        orders.put(order.orderNumber(), order);

        Payment payment = paymentMethod.apply(order.total());
        order.attachPayment(payment);
        switch (payment.process()) {
            case COMPLETED -> { transition(order, OrderStatus.UNSHIPPED, "paid by " + payment.describe());
member.cart().clear(); }
            case PENDING -> { publish(order, OrderStatus.PENDING); member.cart().clear(); }
            default -> {                                  // declined: give the stock back, keep the cart
                releaseStock(order);
                transition(order, OrderStatus.CANCELED, "payment declined");
            }
        }
        return order;
    }

    /** Called when a bank transfer finally clears. */
    void onPaymentSettled(String orderNumber) {
        Order o = get(orderNumber);
        o.payment().markSettled();
        transition(o, OrderStatus.UNSHIPPED, "transfer cleared");
    }

    /** Requirement 8: cancel only if not shipped yet. */
    boolean cancel(String orderNumber) {
        Order o = get(orderNumber);
        synchronized (o) {                       // check-then-act must be atomic against ship()
            if (!o.canCancel()) return false;
            releaseStock(o);
            transition(o, OrderStatus.CANCELED, "cancelled by member");
        }
        if (o.payment().refund() == PaymentStatus.REFUNDED)
            transition(o, OrderStatus.REFUND_APPLIED, "refunded " + Money.fmt(o.total()));
        return true;
    }

    void ship(String orderNumber, String method) {
        Order o = get(orderNumber);
        Shipment s = new Shipment("SHP-" + shipSeq.getAndIncrement(), method);
        synchronized (o) {
            transition(o, OrderStatus.SHIPPED, "handed to " + method);
            o.attachShipment(s);
        }
        updateShipment(orderNumber, ShipmentStatus.SHIPPED, "sort centre");
    }

    void updateShipment(String orderNumber, ShipmentStatus status, String location) {
        Order o = get(orderNumber);
        o.shipment().addLog(status, location);
        ShipmentLog entry = o.shipment().current();
        listeners.forEach(l -> l.onShipmentUpdate(o, entry));
        if (status == ShipmentStatus.DELIVERED) transition(o, OrderStatus.COMPLETED, "delivered");
    }

    Order get(String orderNumber) {
        Order o = orders.get(orderNumber);
        if (o == null) throw new IllegalArgumentException("no order " + orderNumber);
```

```
        return o;
    }

    private void releaseStock(Order o) {
        o.items().forEach(i -> catalog.find(i.productId()).orElseThrow().release(i.quantity()));
    }

    private void transition(Order o, OrderStatus target, String note) {
        o.moveTo(target, note);
        publish(o, target);
    }

    private void publish(Order o, OrderStatus s) { listeners.forEach(l -> l.onOrderStatus(o, s)); }
}
```

```java
// File: lld/ch19/amazon/Payments.java
package lld.ch19.amazon;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;

enum PaymentStatus { UNPAID, PENDING, COMPLETED, DECLINED, CANCELLED, REFUNDED }

/**
 * Strategy for taking money. Each subclass behaves differently: a card clears at
 * once, a bank transfer comes back PENDING and only clears later.
 */
abstract sealed class Payment permits CreditCardPayment, BankTransferPayment {
    private final BigDecimal amount;
    protected PaymentStatus status = PaymentStatus.UNPAID;

    Payment(BigDecimal amount) { this.amount = Money.of(amount); }

    BigDecimal amount() { return amount; }
    synchronized PaymentStatus status() { return status; }

    /** Try to take the money. Returns the new status. */
    abstract PaymentStatus process();

    abstract String describe();

    synchronized void markSettled() {
        if (status == PaymentStatus.PENDING) status = PaymentStatus.COMPLETED;
    }

    synchronized PaymentStatus refund() {
        if (status == PaymentStatus.COMPLETED) status = PaymentStatus.REFUNDED;
        else if (status == PaymentStatus.PENDING || status == PaymentStatus.UNPAID) status = PaymentStatus.CANCELLED;
        return status;
    }
}

final class CreditCardPayment extends Payment {
    private final String last4;             // never store the full card number
    private final BigDecimal creditLimit;

    CreditCardPayment(BigDecimal amount, String last4, BigDecimal creditLimit) {
        super(amount);
        this.last4 = last4;
        this.creditLimit = creditLimit;
    }

    @Override synchronized PaymentStatus process() {
        status = amount().compareTo(creditLimit) <= 0 ? PaymentStatus.COMPLETED : PaymentStatus.DECLINED;
        return status;
    }

    @Override String describe() { return "card ****" + last4; }
}

final class BankTransferPayment extends Payment {
    private final String bankName;

    BankTransferPayment(BigDecimal amount, String bankName) {
        super(amount);
        this.bankName = bankName;
    }

    @Override synchronized PaymentStatus process() {
        status = PaymentStatus.PENDING;      // clears later, when the bank confirms
        return status;
    }

    @Override String describe() { return "bank transfer via " + bankName; }
}

enum ShipmentStatus { PENDING, SHIPPED, DELIVERED, ON_HOLD }
```

```
record ShipmentLog(ShipmentStatus status, String location) { }

/** Keeps a log, not one status field: the last entry is "now", the rest is history. */
final class Shipment {
    private final String shipmentNumber;
    private final String method;
    private final List<ShipmentLog> log = new ArrayList<>();

    Shipment(String shipmentNumber, String method) {
        this.shipmentNumber = shipmentNumber;
        this.method = method;
        log.add(new ShipmentLog(ShipmentStatus.PENDING, "warehouse"));
    }

    String shipmentNumber() { return shipmentNumber; }
    String method() { return method; }
    synchronized void addLog(ShipmentStatus s, String where) { log.add(new ShipmentLog(s, where)); }
    synchronized ShipmentLog current() { return log.get(log.size() - 1); }
    synchronized List<ShipmentLog> history() { return List.copyOf(log); }
}
```

Output:

```
== 1. Search ==
name 'code'      -> [Clean Code ($32.50)]
name 'usb cable' -> [USB-C Cable ($9.99)]
category 'books' -> [Clean Code ($32.50), Refactoring ($41.00), Design Patterns (last copy) ($55.00)]

== 2. Guest fills a cart, then registers (cart carries over) ==
  cart: 1 x Clean Code = $32.50
  cart: 2 x USB-C Cable = $19.98

== 3. Checkout with a credit card ==
    [email -> alice@mail.test] Order ORD-1001 is now UNSHIPPED
    [sms   -> +1-555-0101] Order ORD-1001 is now UNSHIPPED
  ORD-1001 total $52.48, status UNSHIPPED, stock of Clean Code now 4

== 4. Ship, track, deliver ==
    [email -> alice@mail.test] Order ORD-1001 is now SHIPPED
    [sms   -> +1-555-0101] Order ORD-1001 is now SHIPPED
    [email -> alice@mail.test] Shipment for ORD-1001: SHIPPED @ sort centre
    [email -> alice@mail.test] Shipment for ORD-1001: SHIPPED @ Portland hub
    [email -> alice@mail.test] Shipment for ORD-1001: DELIVERED @ front door
    [email -> alice@mail.test] Order ORD-1001 is now COMPLETED
    [sms   -> +1-555-0101] Order ORD-1001 is now COMPLETED
  tracking now: ShipmentLog[status=DELIVERED, location=front door], order COMPLETED
  cancel after delivery? false

== 5. Bank transfer (pending) then cancel before shipping -> refund ==
    [email -> alice@mail.test] Order ORD-1002 is now PENDING
    [sms   -> +1-555-0101] Order ORD-1002 is now PENDING
  ORD-1002 status PENDING, payment PENDING
    [email -> alice@mail.test] Order ORD-1002 is now UNSHIPPED
    [sms   -> +1-555-0101] Order ORD-1002 is now UNSHIPPED
    [email -> alice@mail.test] Order ORD-1002 is now CANCELED
    [sms   -> +1-555-0101] Order ORD-1002 is now CANCELED
    [email -> alice@mail.test] Order ORD-1002 is now REFUND_APPLIED
    [sms   -> +1-555-0101] Order ORD-1002 is now REFUND_APPLIED
  cancel before shipping? true -> REFUND_APPLIED, payment REFUNDED, Refactoring stock back to 2
  order log: [PENDING, UNSHIPPED, CANCELED, REFUND_APPLIED]

== 6. Illegal transition is rejected by the state machine ==
  ORD-1002: REFUND_APPLIED -> SHIPPED is not allowed

== 7. Review ==
  Clean Code average rating = 4.5

== 8. Blocked member cannot buy ==
  rejected: account BLOCKED

== 9. Ten members race for the last copy ==
  winners=1 losers=9 stock left=0
```

Reading the output. Step 1 answers both searches from indexes (a name search matches every query word, so `usb cable` finds "USB-C Cable"). Step 2 shows the guest's cart surviving registration. Step 3 prints two notifications (email + SMS) because both channels are registered. Step 5 walks the full cancel path `PENDING → UNSHIPPED → CANCELED → REFUND_APPLIED`, and the stock returns to 2. Step 6 proves the state machine refuses `REFUND_APPLIED → SHIPPED`. Step 9 starts ten threads at the same instant (a `CountDownLatch` — a gate that keeps threads waiting until a counter reaches zero — releases them together) and exactly one wins the last copy.

Concurrency & edge cases

Risk	What could go wrong	How the code prevents it
Last unit, two buyers	Both see `availableCount == 1`, both decrement → stock −1	`Product.reserve` is `synchronized`: check and decrement are one step. Step 9 proves it.
Multi-line cart, one line out of stock	Lines 1–2 reserved, line 3 fails → stock leaked	`placeOrder` remembers what it reserved and releases it all on failure (all-or-nothing).
Cancel and ship at the same time	Order shipped *and* stock released	`cancel` and `ship` both lock the `Order` object around check-then-move.
Payment declined	Stock stays reserved forever	Declined path releases stock and moves the order to `CANCELED`; the cart is kept so the member can retry.
Bank transfer never clears	Stock held forever	Mention a timeout job that cancels `PENDING` orders after N hours.
Price changes after adding to cart	Customer charged a price they never saw	`Item` stores the unit price at add time; production systems re-validate prices at checkout and show changes.
Blocked member	Places orders anyway	`Member.canBuy()` checks `AccountStatus.ACTIVE` first (step 8).
Deadlock (two threads each waiting forever for a lock the other holds)	Two locks taken in different orders	Each lock (product, order, cart) is held briefly and never while waiting for another object's lock in the opposite order.

Real-world example

On Amazon's "Your Orders" page, the *Cancel items* button is offered only while an order has not yet entered the shipping process — the same edge our state machine removes after `SHIPPED`. Amazon also treated the shopping cart as something that must *always* accept writes: its 2007 Dynamo paper (the key-value store — a database that simply maps a key to a value — behind services such as the cart) chose **availability** (staying usable even when some machines fail; see System Design Fundamentals) so that "add to cart" works even during failures, and merges conflicting cart versions later. Our in-memory design shows the object model; at Amazon's scale the cart, catalog and orders live in separate services.

Extensions & follow-up questions

- **"Add a new payment type, e.g. gift cards or UPI (India's instant bank-to-bank mobile payment system)."** Add a `GiftCardPayment extends Payment` (and list it in `permits`). `OrderService` is untouched, because it only calls `process()`.
- **"How do you prevent two shoppers from both buying the final unit?"** Reserve at order time inside a lock (`Product.reserve`). In a distributed system (one spread over many machines): a conditional database update (`UPDATE ... SET stock = stock - 1 WHERE id = ? AND stock >= 1`) or **optimistic locking** with a version column (no lock is held; each row carries a version number, and an update succeeds only if the version is still the one you read — otherwise you retry).
- **"Abandoned orders hold stock."** Give each reservation an expiry. A scheduled job cancels `PENDING` orders older than, say, 30 minutes and releases stock.
- **"Returns after delivery."** Add `COMPLETED → RETURN_REQUESTED → RETURNED → REFUND_APPLIED` edges; the enum change is local.
- **"Multiple warehouses."** Replace `Product.availableCount` with an `Inventory` per warehouse; checkout picks a warehouse per line (a Strategy: nearest, cheapest shipping).
- **"Split shipments."** One `Order` owns many `Shipment`s, each with its own item subset and log.
- **"Search at scale."** The in-memory `Catalog` becomes a search index (for example Elasticsearch/OpenSearch, popular search-engine servers) fed by product change events; the `Search` interface stays.
- **"Seller payouts, recommendations, wish lists."** Separate modules listening to order events (Observer again).

In the interview

30–45 minute plan

1. (3 min) Clarify: guest vs member, payment types, cancel rule, notifications, what is out of scope.
2. (5 min) Actors and use cases; name the four actors.
3. (10 min) Class diagram. **Say the LSP sentence out loud**: "Ordering is on `Member`, not `Customer`, otherwise every guest inherits an ordering method it is not allowed to use."
4. (5 min) The order state machine; show that "cancel after ship" has no edge.
5. (10 min) Code `placeOrder` (reserve all-or-nothing, copy items, pay, notify) and `cancel`.
6. (5 min) Concurrency: last-item race; abandoned-order timeout; extensions.

> **30-second answer:** "Catalog with name and category indexes; a cart per customer; ordering only on `Member`; checkout reserves stock for every line atomically, copies the lines into an immutable order, then takes payment through a `Payment` strategy. The order moves through a state machine that forbids cancelling after shipping, and every status change is published to a notification observer."

Follow-ups with answers

- *Why is `Payment` abstract rather than an enum?* Because the methods behave differently (clears now vs pending). New methods are new classes — Open/Closed.
- *Why does `Order` copy items?* So later cart edits or price changes cannot rewrite history.
- *Where would you put tax and discounts?* A `PricingRule` chain (Chain of Responsibility — a pattern where a request is passed along a line of handlers, each doing its part — or a list of Strategies) applied when building the order; the order stores the final line prices.
- *How do you make checkout idempotent (safe to repeat: doing it twice has the same effect as doing it once) if the user double-clicks?* Accept a client-generated idempotency key (a unique id the browser sends with the request); if an order with that key exists, return it instead of creating a second one.

Self-check

1. **Q:** Why can't `placeOrder` live on `Customer`? **A:** `Guest extends Customer` and guests must not order; inheriting a method you must refuse violates LSP.
2. **Q:** What does the filled diamond between `Order` and `OrderLog` mean? **A:** Composition — the order owns its log entries; they are created and destroyed with it.
3. **Q:** Name two things wrong with "check stock, take payment, then decrement stock". **A:** A race window lets two buyers pass the check; and a payment can succeed for goods that are no longer there.
4. **Q:** Which status edges allow a cancel? **A:** `PENDING → CANCELED` and `UNSHIPPED → CANCELED`.
5. **Q:** Why `compareTo` and not `equals` on `BigDecimal`? **A:** `equals` also compares scale, so `2.0` and `2.00` are "not equal".
6. **Q:** What pattern lets you add push notifications without touching `OrderService`? **A:** Observer (listener) plus a new `NotificationChannel` implementation.

TL;DR

- Guests browse and fill carts; **only members order** (LSP).
- Checkout = **reserve all lines atomically → copy into an immutable order → pay (Strategy) → publish (Observer)**.
- `OrderStatus` is a **state machine**; "cancel after ship" is impossible because the edge does not exist.
- Money in `BigDecimal`; histories in logs; stock guarded by a lock.

Design an Online Stock Brokerage System

This problem has two halves, and interviews ask both:

- **Part A — the brokerage** (what the course designs): members, money in and out, watchlists, the four order types, lots and positions, statements.

- **Part B — the exchange's matching engine** (a machine-coding problem — write a working program in about 90 minutes — originally from Geektrust, an Indian coding-assessment platform, kept in kumaransg/LLD under a project named "Navi"; we call it "the Navi statement" below): an **order book** (the list of all waiting buy and sell orders for one stock) that matches buy and sell orders by **price-time priority**, implemented and tested.

A **broker** is the company you trade *through* (it holds your cash and shares and forwards your orders). An **exchange** is the market where buy and sell orders from all brokers *meet* and are matched. Keep the two apart in your head; the design keeps them apart in code.

Problem & clarifying questions

An online brokerage lets people buy and sell stocks without a human middleman. Members track and carry out their transactions, see charts of how the stocks they hold are performing, and get an alert when a stock moves by an amount they chose. The platform must keep those transactions secure. Because computers replace the phone call to a broker, each trade is quicker and cheaper, and members get fast access to real-time (updated the moment they change) prices, market trends and stock reports.

Question	Assumed answer
Which order types?	Market, limit, stop-loss, stop-limit (explained in the table under Key flows).
Can one order fill at several prices?	Yes — an order can be filled in parts.
Do we track separate purchases of the same stock?	Yes — each purchase is a **lot**, because tax depends on which lot you sell.
How does money come in and out?	Check, wire (a direct bank-to-bank transfer), or electronic bank transfer, each with a per-transfer limit.
Reports?	Quarterly statements and a yearly tax statement.
Notifications?	When an order executes.
Margin, short selling, options, dividends and stock splits, real-time streaming quotes, the settlement delay, the wash-sale tax rule?	Out of scope — a deliberate decision, say so.

For reference, those out-of-scope terms: **margin** is trading with money borrowed from the broker; **short selling** is selling borrowed shares, hoping to buy them back cheaper; **options** are contracts giving the right to buy or sell later at a fixed price; **dividends** are profits a company pays to its shareholders; a **stock split** turns each share into several cheaper ones; **settlement** is the actual swap of cash and shares, which happens a day or so after the trade; the **wash-sale rule** is a US tax rule that blocks claiming a loss if you rebuy the same stock within 30 days.

Requirements

Functional (brokerage)

1. Any member can buy and sell stocks.
2. A member can have **multiple watchlists** (named lists of stocks to keep an eye on), each with multiple stock quotes (a quote is a stock's latest price).
3. Orders can be **market, limit, stop-loss, stop-limit**.
4. A member can hold **multiple lots** of the same stock; the system keeps them apart.
5. The system generates **quarterly** updates and **yearly tax** statements.
6. Money goes in and out of the account in three ways: **check, wire, or electronic bank transfer**.
7. A notification is sent whenever an order executes.

Functional (matching engine, Navi variant)

8. Orders arrive as `<id> <time> <symbol> <buy/sell> <price> <qty>`, e.g. `#1 09:45 BAC sell 240.12 100`. They are read from a text file whose path is given as the program's argument; trades are printed to the console.
9. On every new order, match against the opposite side. Lower sell prices and higher buy prices go first; at the same price, **earlier orders go first** (price-time priority).

10. A trade happens when buy price ≥ sell price. In the Navi statement the trade is recorded at the **sell order's price**.
11. Print each trade as `<buy_id> <price> <qty> <sell_id>`; unmatched quantity stays in the book.

Non-functional

- Money is never over-committed: ten $1,000 orders cannot all pass a $1,000 balance check.
- Matching is deterministic (no randomness or timing luck): same input, same trades.
- The engine is safe under concurrent submissions and conserves quantity (shares bought = shares sold).

Use cases

Three actors:

- **Admin** — administrative jobs such as blocking or unblocking members.
- **Member** — searches the stock inventory, buys and sells, manages watchlists.
- **System** — sends order notifications and periodically fetches quotes from the exchange.

Top use cases: register / cancel membership; add, remove or edit a watchlist; search stocks by symbol; place an order; cancel an order; deposit or withdraw money.

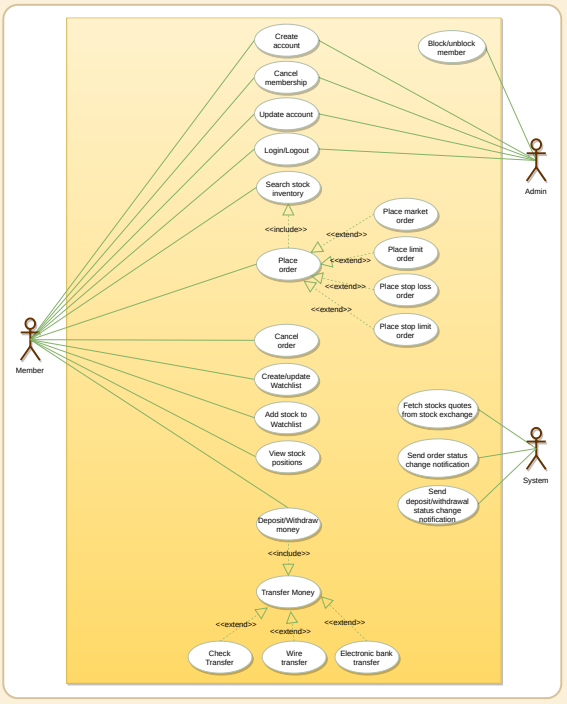


Figure — Use-case diagram for the stock brokerage: Admin, Member and System (course diagram)

Core entities

Class	Responsibility
`MoneyTransfer` → `CheckTransfer`, `WireTransfer`, `ElectronicBankTransfer`	Move money in or out. They differ only in name and clearing days (5 / 1 / 3 in our code); the per-transfer limit (100,000) is a single rule, so it lives only in the base class.
`Notifier` / `EmailNotifier`	Delivers "your order filled" messages. The course draws one base with two delivery ways (email and SMS); an `SmsNotifier` would be a second implementation.
`Order` (abstract)	Shared by all order types: member, symbol, side (buy or sell), quantity, time-in-force (how long the order stays live), status, list of fills. Declares the **two questions**.
`MarketOrder`, `LimitOrder`, `StopLossOrder`, `StopLimitOrder`	Each answers "may I trade now?" and "what is my worst price?". `StopLimitOrder extends StopLossOrder`.
`OrderPart`	One fill: quantity, price, day. A 100-share order might be filled as 70 @ 10.00 plus 30 @ 10.05.
`Stock`, `StockInventory`	Symbol + latest price; the inventory is the **single source of truth** (the one place whose value counts if copies disagree) for prices.
`StockExchange`	Holds open orders; on each new order or price tick tries to fill them; calls back through `OrderListener`.
`OrderListener`	Interface the exchange calls (`onFill`, `onCancelled`). The exchange never learns what a member is.
`StockLot`	One purchase at one price.
`StockPosition`	All lots of one symbol; sells from the oldest lot first (FIFO — first in, first out) unless a lot is named. Returns a `LotSale` per lot touched.
`LotSale`	Shares taken from one lot, buy price, sell price → realised gain (profit actually locked in by selling).
`Watchlist`	A named set of **symbols** (not `Stock` objects, so it never goes stale).
`Trade`	One fill as the member sees it (sells carry their `LotSale`s).
`Statement` → `QuarterlyStatement`, `TaxStatement`	Base class filters trades by day range; subclasses write their own lines.
`Member`	Cash, **reserved funds**, positions, watchlists, trades. Implements `OrderListener`. (The course's `Account` also stores name, address, e-mail and phone; we keep only what the demo uses.)
`Admin`	Blocks / unblocks members.
(engine) `OrderBook`, `MatchingEngine`, `Trade`, `ExecutionPricePolicy`	Price levels with FIFO queues per symbol; routing and locking; who sets the trade price.

Class diagram

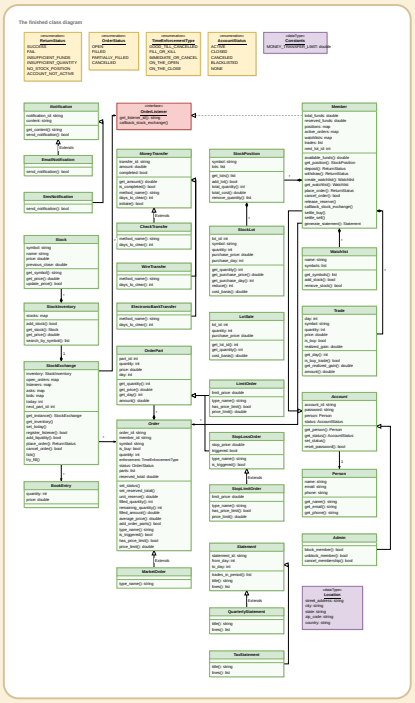


Figure — The finished class diagram of the online stock brokerage (course diagram)

The course builds this model in six stages: money transfers and notifications → the order hierarchy → stock, inventory and exchange → lots and positions → watchlists, trades and statements → members and admins. In the figure above, `Member` extends `Account` and implements `OrderListener` (dashed arrow): the exchange reports fills through that interface and never needs the `Member` type. The filled diamonds from `Member` mean each position, watchlist and trade is owned by exactly one member; the plain arrow to `Order` says the exchange shares those orders.

The course singles out stage 2 — the order hierarchy — as the part that earns the most credit in an interview:

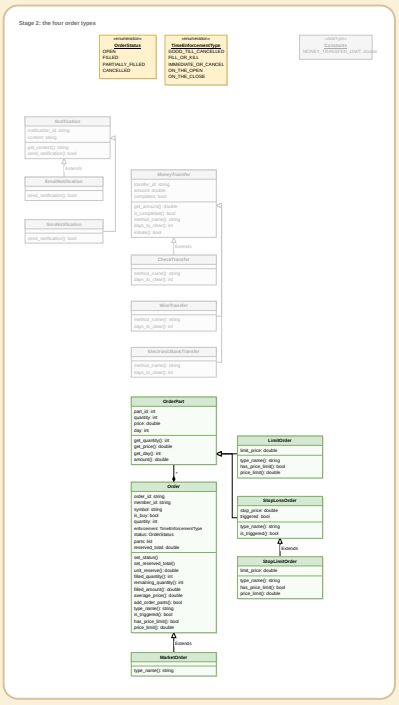
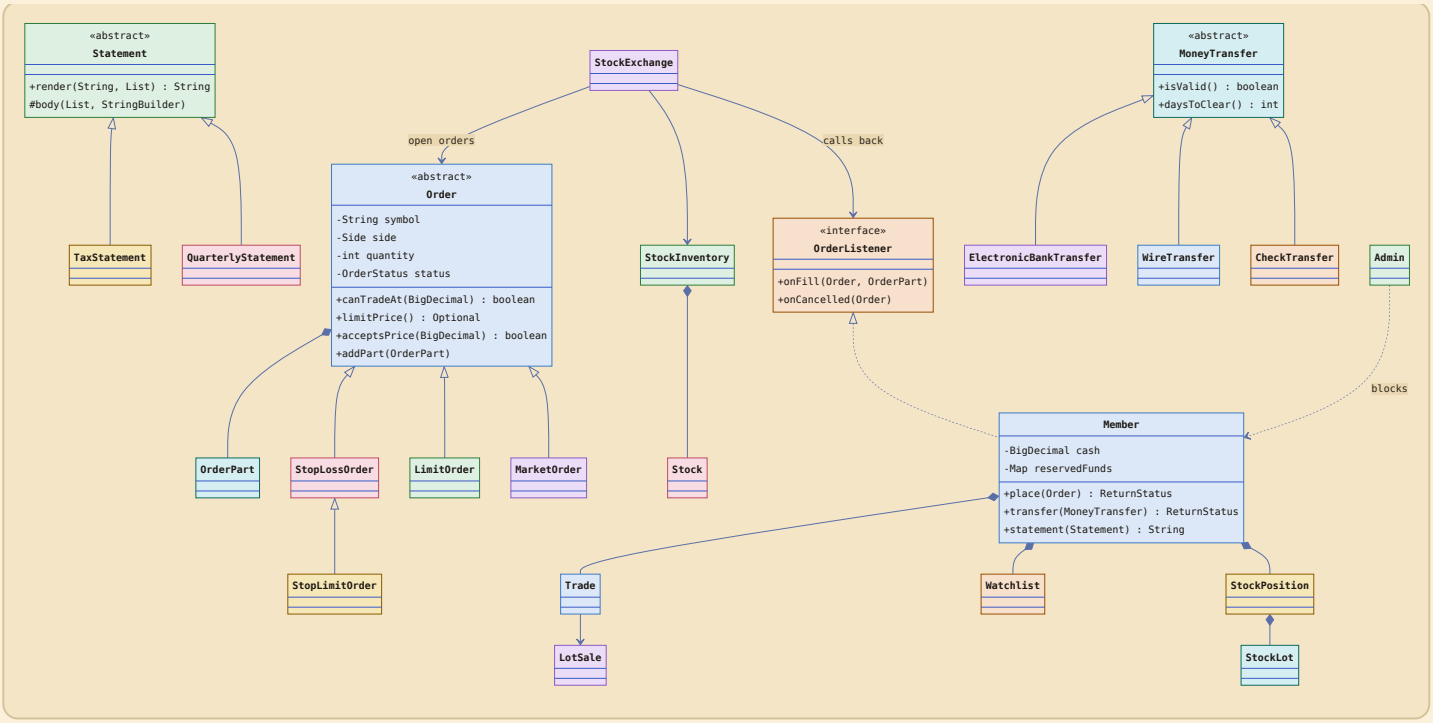
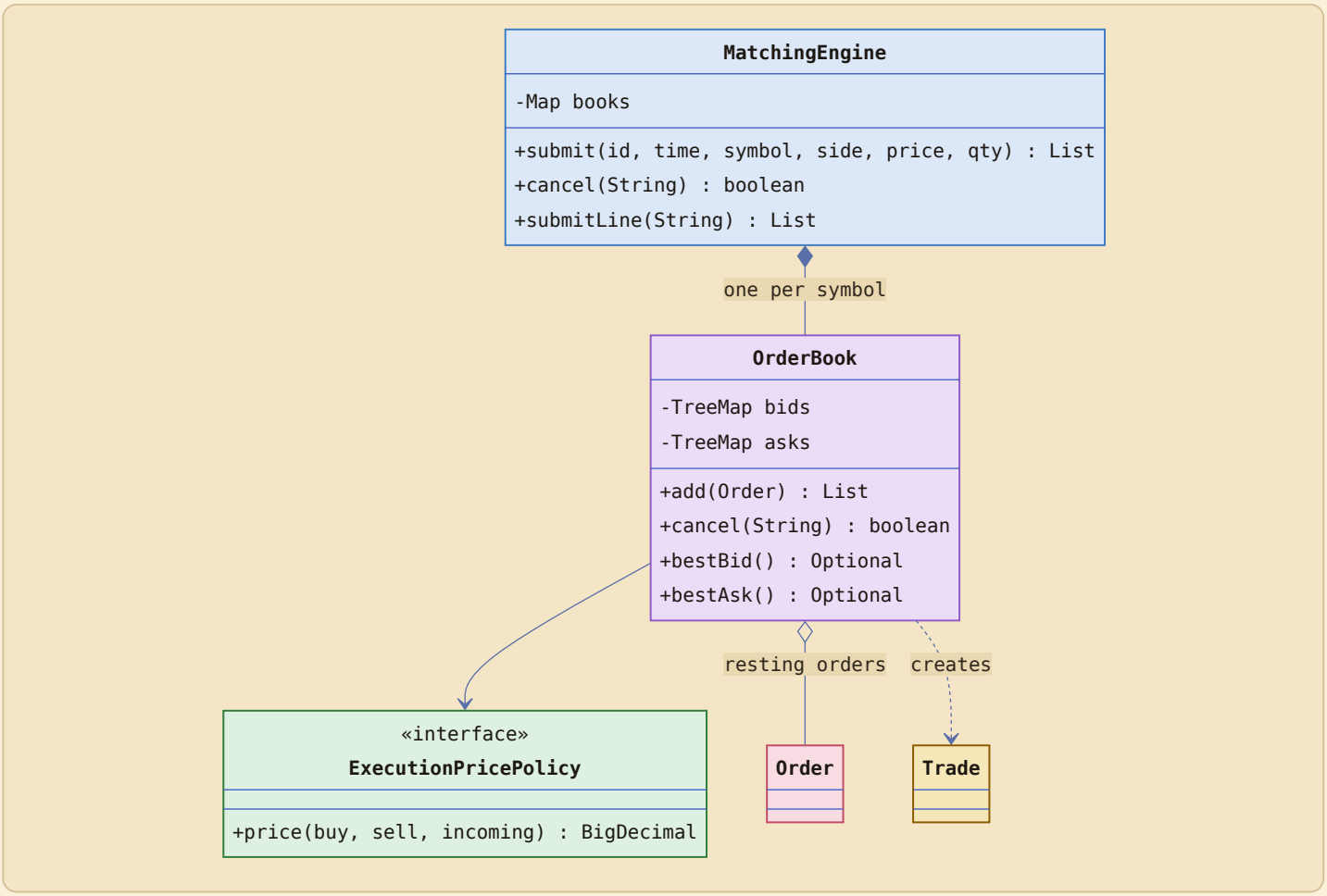


Figure — Stage 2: one abstract Order and four small subclasses that differ only in two answers (course diagram)

Our brokerage code:



And the matching engine (Part B):



Key flows

Placing a buy order. The course's activity diagram: pick a stock, pick the order type (market, limit, stop-loss, stop-limit), enter quantity, price limit and time enforcement; the system checks there is enough money, then deducts it (we *reserve* it) and sends the order to the exchange; when the exchange acknowledges, the member sees "order placed". The member can then update the order.

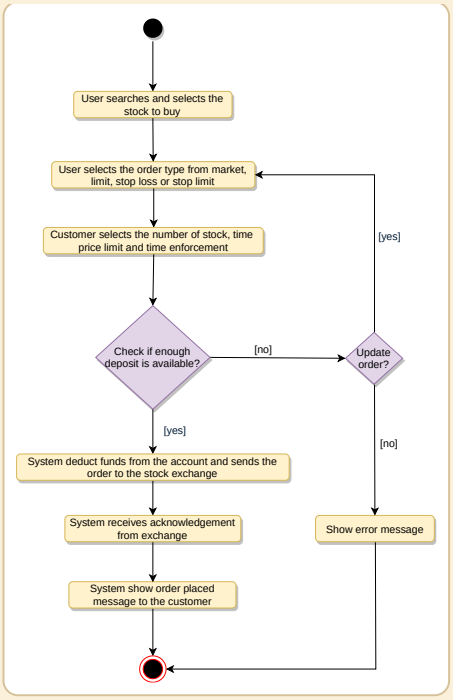


Figure — Activity diagram for placing a buy order (course diagram)

Placing a sell order is the mirror image, with one extra step: the member picks **which lots** to sell, and the check is "do I own enough shares?" rather than "do I have enough money?".

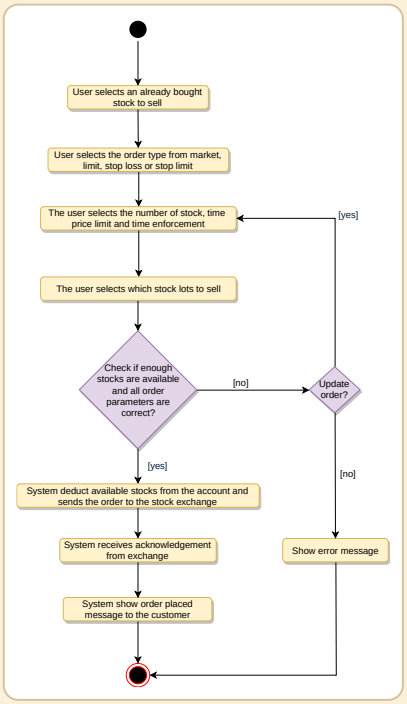


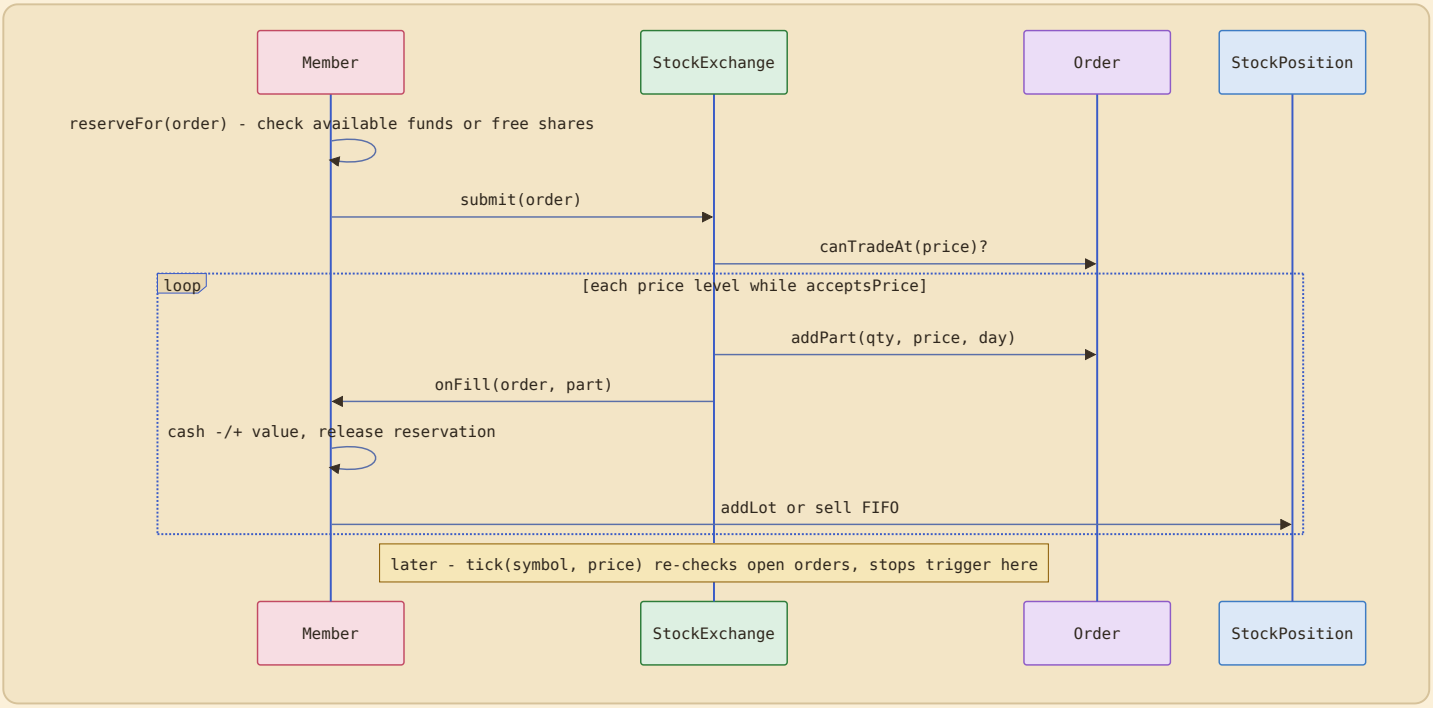
Figure — Activity diagram for placing a sell order, including choosing lots (course diagram)

The four order types in one table. They share everything except two answers:

Type	Q1: may it trade at this market price?	Q2: worst acceptable price	Plain words
Market	Always	None	"Buy/sell now at whatever the market gives."
Limit	Always	The limit	"Buy at 9.50 or less / sell at 10.90 or more."
Stop-loss (stop)	Only after the price **reaches the stop**, then forever	None (becomes a market order)	"If it falls to 10.00, get me out."
Stop-limit	Only after the stop is reached	The limit	"If it rises to 305, buy — but pay no more than 306."

A SELL stop triggers when price ≤ stop; a BUY stop triggers when price ≥ stop. Once triggered it stays triggered (the code "latches" a boolean).

Fill → money and lots move (Observer callback).

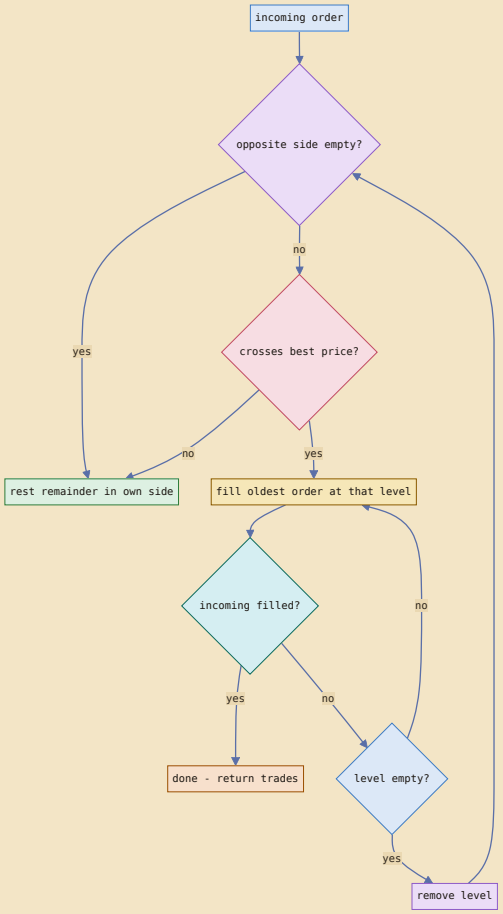


Matching engine: price-time priority. Each symbol has an `OrderBook` with two sides. Each side is a `TreeMap` (a map that keeps its keys sorted) from price to a FIFO queue (`ArrayDeque`, Java's fast double-ended queue). **Bids** (buy orders) are sorted highest-first, **asks** (sell orders) lowest-first. For an incoming order:

1. Look at the best price on the opposite side.
2. If it does not **cross** (buy price < best ask, or sell price > best bid), stop.
3. Otherwise fill against that level's queue, **oldest order first**, for `min(incoming remaining, resting remaining)`.
4. Remove fully filled resting orders; drop empty price levels; repeat from step 1.
5. If the incoming order still has quantity, add it to the back of its own price level's queue.

Walk-through of the sample input:

Order	Book before (asks / bids)	What happens
#1 sell 240.12 × 100	empty	rests on asks
#2 sell 237.45 × 90	asks 240.12×100	rests; best ask now 237.45
#3 buy 238.10 × 110	asks 237.45×90, 240.12×100	crosses 237.45 → trade `#3 237.45 90 #2`; 20 left; 240.12 > 238.10 so stop; rests 20 @ 238.10
#4 buy 237.80 × 10	best ask 240.12	no cross; rests
#5 buy 237.80 × 40	same	no cross; rests **behind #4** at 237.80
#6 sell 236.00 × 50	bids 238.10×20, 237.80×(10, 40)	fills #3 for 20, then #4 for 10, then #5 for 20 — all at 236.00 (the sell price)



Design decisions & patterns used

Decision	Why	Alternative rejected
One abstract `Order` + two abstract questions	The four types differ only in "may I trade?" and "worst price?". Each subclass is a few lines. The course stresses this is just a type hierarchy (plain polymorphism — calling the same method on different subclasses and each running its own version), not a named pattern: every subclass answers for itself whether the current price lets it trade, and no order ever turns into another kind — so it is not the State pattern either. (The only named patterns the course lists are Observer and Singleton — a class that allows exactly one instance; the Template Method and Strategy labels in this table are our additions.)	Four unrelated classes with copied fields; or one class with a `type` field and `switch`es.
`StopLimitOrder extends StopLossOrder`	A stop-limit is a stop that becomes a *limit* instead of a *market* order: it inherits the trigger, overrides only `limitPrice()`.	Duplicating the trigger logic.
`OrderPart` per fill	One order can fill at several prices/days; each fill needs its own price and quantity (and becomes its own lot).	Storing only an average price (loses tax information).
Observer: `OrderListener`	The exchange tells whoever placed the order; it does not import `Member`. Exchange and account code change independently.	The exchange updating member balances directly.
Reserved funds / reserved shares	Without it, a member with $1,000 could place ten $1,000 orders and all would pass the check. Money is reserved at placement and released as fills or cancellations arrive. Money and shares actually move only in `onFill`, which the **exchange** calls (the course's `callbackStockExchange`) — so the moment an order is *placed* and the moment it is *paid for* are different.	Checking the balance only at fill time (over-commitment).
Lots, FIFO by default	Tax depends on *which* shares you sold; each `LotSale` records it.	Merging all purchases into one average-cost position.
Template Method: `Statement`	Filtering by day range is shared; only the lines differ. `render` is `final`; subclasses fill in `body`.	Two statement classes each re-implementing the filter.
`MoneyTransfer` base class holds the limit rule	One rule, three delivery methods.	Copying the limit check into each subclass.
Singleton?	The course makes `StockExchange` a Singleton (one gateway to the real exchange). We pass one instance into `Member` instead (**dependency injection**: an object is handed the things it needs from outside rather than creating or looking them up itself) — same "only one" in practice, but testable with a fake exchange.	A global `getInstance()` that tests cannot replace.
Engine: `TreeMap<price, ArrayDeque>` per side	Best price in O(log n) (Big-O notation: how the work grows with n items — here, very slowly); FIFO inside a level gives time priority for free.	One `PriorityQueue` (a structure that always hands back the smallest item first) sorted by (price, time) — works, but cancel is O(n) (time grows in step with the number of orders) and levels are not visible.
Engine: lock per symbol	Orders for different symbols match in parallel; same-symbol orders are strictly serialised (handled one at a time, in order), so "time priority" is well defined.	One global lock (slower) or no lock (corrupt book).

Decision	Why	Alternative rejected
Engine: `ExecutionPricePolicy` (Strategy)	The Navi statement prices trades at the **sell** order's price; real continuous-trading exchanges use the **resting** (already-in-book) order's price. Both are one-line policies.	Hard-coding one rule.

Java implementation

Part A — Brokerage (`lld.ch19.brokerage`)

Files: `Orders.java` (enums, money helper, `OrderPart`, the order hierarchy), `Market.java` (stock, inventory, listener, simulated exchange), `Holdings.java` (lots, positions, watchlists, trades, statements), `Accounts.java` (transfers, notifier, member, admin), `Main.java`.

The simulated exchange offers a fixed depth of shares per price level (70 shares at each level, 5 cents apart, 3 levels) so fills come back in parts, like a real book. Part B builds a real matching engine.

```java
// File: lld/ch19/brokerage/Accounts.java
package lld.ch19.brokerage;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;

enum AccountStatus { ACTIVE, CLOSED, CANCELED, BLACKLISTED, NONE }

enum ReturnStatus { SUCCESS, FAIL, INSUFFICIENT_FUNDS, INSUFFICIENT_QUANTITY, NO_STOCK_POSITION, ACCOUNT_NOT_ACTIVE }

enum Direction { DEPOSIT, WITHDRAW }

/** Three ways to move money. They differ only in name and clearing time; the limit rule lives here once. */
abstract class MoneyTransfer {
    static final BigDecimal TRANSFER_LIMIT = Money.of("100000");
    private final BigDecimal amount;
    private final Direction direction;
    MoneyTransfer(BigDecimal amount, Direction direction) { this.amount = amount; this.direction = direction; }
    BigDecimal amount() { return amount; }
    Direction direction() { return direction; }
    boolean isValid() { return amount.signum() > 0 && amount.compareTo(TRANSFER_LIMIT) <= 0; }
    abstract String methodName();
    abstract int daysToClear();
}

final class CheckTransfer extends MoneyTransfer {
    CheckTransfer(BigDecimal a, Direction d) { super(a, d); }
    @Override String methodName() { return "check"; }
    @Override int daysToClear() { return 5; }
}

final class WireTransfer extends MoneyTransfer {
    WireTransfer(BigDecimal a, Direction d) { super(a, d); }
    @Override String methodName() { return "wire"; }
    @Override int daysToClear() { return 1; }
}

final class ElectronicBankTransfer extends MoneyTransfer {
    ElectronicBankTransfer(BigDecimal a, Direction d) { super(a, d); }
    @Override String methodName() { return "electronic bank transfer"; }
    @Override int daysToClear() { return 3; }
}

interface Notifier { void send(String to, String message); }

final class EmailNotifier implements Notifier {
    @Override public void send(String to, String message) { System.out.println("    [email " + to + "] " + message); }
}

/**
 * A member. Implements OrderListener so the exchange can call back without knowing
 * what a member is. Holds reserved funds so ten $1,000 orders cannot all pass a $1,000 check.
 * Lock order: the member never calls the exchange while holding its own lock.
 */
final class Member implements OrderListener {
    private final String id;
    private final String name;
    private final String email;
    private final StockExchange exchange;
    private final StockInventory inventory;
    private final Notifier notifier;
    private volatile AccountStatus status = AccountStatus.ACTIVE;

    private BigDecimal cash = Money.of("0");
    private final Map<String, BigDecimal> reservedFunds = new HashMap<>();   // buy orderId -> money still reserved
    private final Map<String, Order> pendingSells = new HashMap<>();            // sell orderId -> open sell order
    private final Map<String, StockPosition> positions = new LinkedHashMap<>();
    private final Map<String, Watchlist> watchlists = new LinkedHashMap<>();
```

```
    private final List<Trade> trades = new ArrayList<>();
    private int lotSeq = 1;

    Member(String id, String name, String email, StockExchange ex, StockInventory inv, Notifier n) {
        this.id = id; this.name = name; this.email = email;
        this.exchange = ex; this.inventory = inv; this.notifier = n;
        ex.register(id, this);
    }

    String id() { return id; }
    String name() { return name; }
    void setStatus(AccountStatus s) { status = s; }

    // ---- money ----
    synchronized ReturnStatus transfer(MoneyTransfer t) {
        if (!t.isValid()) return ReturnStatus.FAIL;
        if (t.direction() == Direction.WITHDRAW) {
            if (t.amount().compareTo(availableFunds()) > 0) return ReturnStatus.INSUFFICIENT_FUNDS;
            cash = cash.subtract(t.amount());
        } else {
            cash = cash.add(t.amount());
        }
        return ReturnStatus.SUCCESS;
    }

    synchronized BigDecimal cash() { return cash; }

    synchronized BigDecimal reserved() {
        BigDecimal r = Money.of("0");
        for (Map.Entry<String, BigDecimal> e : reservedFunds.entrySet()) r = r.add(e.getValue());
        return r;
    }

    synchronized BigDecimal availableFunds() { return cash.subtract(reserved()); }

    // ---- orders ----
    ReturnStatus place(Order o) {
        ReturnStatus check = reserveFor(o);                // under our lock
        if (check == ReturnStatus.SUCCESS) exchange.submit(o);   // outside our lock
        return check;
    }

    private synchronized ReturnStatus reserveFor(Order o) {
        if (status != AccountStatus.ACTIVE) return ReturnStatus.ACCOUNT_NOT_ACTIVE;
        if (o.side() == Side.BUY) {
            BigDecimal perShare = o.limitPrice().orElse(inventory.priceOf(o.symbol()));
            BigDecimal need = Money.times(perShare, o.quantity());
            if (need.compareTo(availableFunds()) > 0) return ReturnStatus.INSUFFICIENT_FUNDS;
            reservedFunds.put(o.id(), need);           // released part by part as fills arrive
            return ReturnStatus.SUCCESS;
        }
        StockPosition pos = positions.get(o.symbol());
        if (pos == null) return ReturnStatus.NO_STOCK_POSITION;
        int promised = pendingSells.values().stream()               // shares already promised to other
                .filter(p -> p.symbol().equals(o.symbol()))             // open sells of this symbol
                .mapToInt(Order::remaining).sum();
        if (o.quantity() > pos.quantity() - promised) return ReturnStatus.INSUFFICIENT_QUANTITY;
        pendingSells.put(o.id(), o);
        return ReturnStatus.SUCCESS;
    }

    ReturnStatus cancel(Order o) { return exchange.cancel(o) ? ReturnStatus.SUCCESS : ReturnStatus.FAIL; }

    /** Called by the exchange: this is where money and shares actually move. */
    @Override public synchronized void onFill(Order o, OrderPart part) {
        BigDecimal value = Money.times(part.price(), part.quantity());
        List<LotSale> sales = List.of();
        if (o.side() == Side.BUY) {
            cash = cash.subtract(value);
            BigDecimal left = reservedFunds.getOrDefault(o.id(), Money.of("0")).subtract(value);
            if (o.remaining() == 0 || left.signum() <= 0) reservedFunds.remove(o.id());
            else reservedFunds.put(o.id(), left);
            positions.computeIfAbsent(o.symbol(), StockPosition::new)
                     .addLot(new StockLot("L" + lotSeq++, part.quantity(), part.price(), part.day()));
```

```
        } else {
            cash = cash.add(value);
            if (o.remaining() == 0) pendingSells.remove(o.id());
            sales = positions.get(o.symbol()).sell(part.quantity(), part.price(), Optional.empty());
            if (positions.get(o.symbol()).quantity() == 0) positions.remove(o.symbol());
        }
        trades.add(new Trade(part.day(), o.symbol(), o.side(), part.quantity(), part.price(), sales));
        notifier.send(email, "filled " + part.quantity() + " " + o.symbol() + " @ " + part.price()
                + " (" + o.id() + " " + o.status() + ")");
    }

    @Override public synchronized void onCancelled(Order o) {
        reservedFunds.remove(o.id());
        pendingSells.remove(o.id());
        notifier.send(email, o.id() + " cancelled, " + o.remaining() + " shares unfilled");
    }

    // ---- watchlists, positions, reports ----
    synchronized Watchlist watchlist(String name) { return watchlists.computeIfAbsent(name, Watchlist::new); }
    synchronized List<Watchlist> watchlists() { return List.copyOf(watchlists.values()); }
    synchronized Map<String, List<StockLot>> holdings() {
        Map<String, List<StockLot>> m = new LinkedHashMap<>();
        positions.forEach((s, p) -> m.put(s, p.lots()));
        return m;
    }
    synchronized String statement(Statement s) { return s.render(name, List.copyOf(trades)); }
}

final class Admin {
    void block(Member m) { m.setStatus(AccountStatus.BLACKLISTED); }
    void unblock(Member m) { m.setStatus(AccountStatus.ACTIVE); }
}
```

```java
// File: lld/ch19/brokerage/Holdings.java
package lld.ch19.brokerage;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Optional;
import java.util.Set;

/** One purchase at one price. Kept apart from other purchases because tax depends on it. */
final class StockLot {
    private final String lotId;
    private final BigDecimal buyPrice;
    private final int day;
    private int quantity;

    StockLot(String lotId, int quantity, BigDecimal buyPrice, int day) {
        this.lotId = lotId; this.quantity = quantity; this.buyPrice = buyPrice; this.day = day;
    }
    String lotId() { return lotId; }
    int quantity() { return quantity; }
    BigDecimal buyPrice() { return buyPrice; }
    int day() { return day; }
    void take(int q) { quantity -= q; }
    @Override public String toString() { return lotId + ":" + quantity + "@" + buyPrice; }
}

/** What came out of one lot in one sale - exactly what a tax statement needs. */
record LotSale(String lotId, int quantity, BigDecimal buyPrice, BigDecimal sellPrice) {
    BigDecimal gain() { return Money.times(sellPrice.subtract(buyPrice), quantity); }
}

/** All lots of one symbol. Sells from the named lot first, then oldest first (FIFO). */
final class StockPosition {
    private final String symbol;
    private final List<StockLot> lots = new ArrayList<>();

    StockPosition(String symbol) { this.symbol = symbol; }

    String symbol() { return symbol; }
    void addLot(StockLot lot) { lots.add(lot); }
    int quantity() { return lots.stream().mapToInt(StockLot::quantity).sum(); }
    List<StockLot> lots() { return List.copyOf(lots); }

    List<LotSale> sell(int qty, BigDecimal price, Optional<String> preferredLot) {
        if (qty > quantity()) throw new IllegalStateException("not enough shares");
        List<LotSale> out = new ArrayList<>();
        List<StockLot> order = new ArrayList<>(lots);
        preferredLot.flatMap(id -> lots.stream().filter(l -> l.lotId().equals(id)).findFirst())
                    .ifPresent(l -> { order.remove(l); order.add(0, l); });
        int left = qty;
        for (StockLot lot : order) {
            if (left == 0) break;
            int take = Math.min(left, lot.quantity());
            if (take == 0) continue;
            lot.take(take);
            out.add(new LotSale(lot.lotId(), take, lot.buyPrice(), price));
            left -= take;
        }
        for (Iterator<StockLot> it = lots.iterator(); it.hasNext(); ) if (it.next().quantity() == 0) it.remove();
        return out;
    }
}

/** A watchlist holds symbols, not Stock objects, so it never goes stale. */
final class Watchlist {
    private final String name;
    private final Set<String> symbols = new LinkedHashSet<>();
    Watchlist(String name) { this.name = name; }
    String name() { return name; }
    void add(String symbol) { symbols.add(symbol); }
```

```
    void remove(String symbol) { symbols.remove(symbol); }
    Set<String> symbols() { return Set.copyOf(symbols); }
    String render(StockInventory inv) {
        StringBuilder sb = new StringBuilder(name + ":");
        symbols.forEach(s -> sb.append(' ').append(s).append('=').append(inv.priceOf(s)));
        return sb.toString();
    }
}

/** One executed fill as the member sees it. Sells carry the lots they consumed. */
record Trade(int day, String symbol, Side side, int quantity, BigDecimal price, List<LotSale> sales) { }

/** Template Method: the base class filters trades by day range, subclasses write the lines. */
abstract class Statement {
    private final int fromDay;
    private final int toDay;
    Statement(int fromDay, int toDay) { this.fromDay = fromDay; this.toDay = toDay; }

    final String render(String memberName, List<Trade> all) {
        List<Trade> inRange = all.stream().filter(t -> t.day() >= fromDay && t.day() <= toDay).toList();
        StringBuilder sb = new StringBuilder(title() + " for " + memberName + " (days " + fromDay + "-" + toDay +
")\n");
        body(inRange, sb);
        return sb.toString();
    }
    abstract String title();
    abstract void body(List<Trade> trades, StringBuilder sb);
}

final class QuarterlyStatement extends Statement {
    QuarterlyStatement(int from, int to) { super(from, to); }
    @Override String title() { return "Quarterly statement"; }
    @Override void body(List<Trade> trades, StringBuilder sb) {
        for (Trade t : trades)
            sb.append(String.format("  day %d %-4s %-4s %4d @ %s%n", t.day(), t.side(), t.symbol(), t.quantity(),
t.price()));
    }
}

final class TaxStatement extends Statement {
    TaxStatement(int from, int to) { super(from, to); }
    @Override String title() { return "Tax statement"; }
    @Override void body(List<Trade> trades, StringBuilder sb) {
        BigDecimal total = Money.of("0");
        for (Trade t : trades) for (LotSale s : t.sales()) {
            sb.append(String.format("  %s %s: %d sh bought @ %s sold @ %s gain %s%n",
                    t.symbol(), s.lotId(), s.quantity(), s.buyPrice(), s.sellPrice(), Money.fmt(s.gain())));
            total = total.add(s.gain());
        }
        sb.append("  realised gain: ").append(Money.fmt(total)).append('\n');
    }
}
```

```java
// File: lld/ch19/brokerage/Main.java
package lld.ch19.brokerage;

import java.math.BigDecimal;

public class Main {
    public static void main(String[] args) {
        StockInventory inv = new StockInventory();
        inv.list("AAPL", Money.of("10.00"));
        inv.list("MSFT", Money.of("300.00"));
        inv.list("AMZN", Money.of("120.00"));
        // 70 shares offered per price level, levels 5 cents apart, 3 levels deep
        StockExchange ex = new StockExchange(inv, 70, new BigDecimal("0.05"), 3);
        Member alice = new Member("M1", "Alice", "alice@mail.test", ex, inv, new EmailNotifier());
        Admin admin = new Admin();

        System.out.println("== 1. Fund the account ==");
        System.out.println("  wire 5,000        -> " + alice.transfer(new WireTransfer(Money.of("5000"),
Direction.DEPOSIT)));
        System.out.println("  check 200,000     -> " + alice.transfer(new CheckTransfer(Money.of("200000"),
Direction.DEPOSIT))
                + " (over the 100,000 limit)");
        System.out.println("  cash = " + Money.fmt(alice.cash()));

        System.out.println("\n== 2. Watchlists and search ==");
        alice.watchlist("tech").add("AAPL");
        alice.watchlist("tech").add("MSFT");
        alice.watchlist("retail").add("AMZN");
        alice.watchlists().forEach(w -> System.out.println("  " + w.render(inv)));
        System.out.println("  search 'A' -> " + inv.search("A"));

        System.out.println("\n== 3. Market buy 100 AAPL fills in two parts (two lots) ==");
        Order m1 = new MarketOrder("O1", "M1", "AAPL", Side.BUY, 100);
        System.out.println("  place -> " + alice.place(m1));
        System.out.println("  " + m1 + " parts=" + m1.parts());

        System.out.println("\n== 4. Next day: limit buy 50 AAPL @ 9.50 waits, then fills when price drops ==");
        ex.setDay(2);
        Order l1 = new LimitOrder("O2", "M1", "AAPL", Side.BUY, 50, Money.of("9.50"),
TimeInForce.GOOD_TILL_CANCELLED);
        System.out.println("  place -> " + alice.place(l1) + ", " + l1 + ", reserved " + Money.fmt(alice.reserved()));
        ex.tick("AAPL", Money.of("9.40"));
        System.out.println("  after tick 9.40: " + l1 + ", reserved " + Money.fmt(alice.reserved()));
        System.out.println("  holdings " + alice.holdings());

        System.out.println("\n== 5. Day 3: price rises, sell 120 AAPL (FIFO across lots) ==");
        ex.setDay(3);
        ex.tick("AAPL", Money.of("11.00"));
        Order s1 = new LimitOrder("O3", "M1", "AAPL", Side.SELL, 120, Money.of("10.90"),
TimeInForce.GOOD_TILL_CANCELLED);
        System.out.println("  place -> " + alice.place(s1) + ", " + s1);
        System.out.println("  holdings " + alice.holdings());

        System.out.println("\n== 6. Protect the rest with a stop-loss @ 10.00, then the price falls ==");
        ex.setDay(4);
        Order stop = new StopLossOrder("O4", "M1", "AAPL", Side.SELL, 30, Money.of("10.00"));
        System.out.println("  place -> " + alice.place(stop) + ", " + stop);
        ex.tick("AAPL", Money.of("10.50"));
        System.out.println("  tick 10.50: " + stop);
        ex.tick("AAPL", Money.of("9.95"));
        System.out.println("  tick 9.95: " + stop + " parts=" + stop.parts());

        System.out.println("\n== 7. Stop-limit buy triggers but its limit is not met ==");
        Order sl = new StopLimitOrder("O5", "M1", "MSFT", Side.BUY, 5, Money.of("305.00"), Money.of("306.00"));
        System.out.println("  place -> " + alice.place(sl) + ", " + sl);
        ex.tick("MSFT", Money.of("310.00"));
        System.out.println("  tick 310 (stop hit, above limit): " + sl);
        ex.tick("MSFT", Money.of("305.50"));
        System.out.println("  tick 305.50 (within limit): " + sl);

        System.out.println("\n== 8. Reserved funds block over-spending, cancel releases them ==");
        System.out.println("  cash " + Money.fmt(alice.cash()) + ", available " + Money.fmt(alice.availableFunds()));
        Order big1 = new LimitOrder("O6", "M1", "AMZN", Side.BUY, 20, Money.of("100.00"),
```

```
TimeInForce.GOOD_TILL_CANCELLED);
        Order big2 = new LimitOrder("O7", "M1", "AMZN", Side.BUY, 20, Money.of("100.00"),
TimeInForce.GOOD_TILL_CANCELLED);
        System.out.println("  O6 buy 20 AMZN @ 100 -> " + alice.place(big1) + ", available " +
Money.fmt(alice.availableFunds()));
        System.out.println("  O7 buy 20 AMZN @ 100 -> " + alice.place(big2));
        System.out.println("  cancel O6 -> " + alice.cancel(big1) + ", available " +
Money.fmt(alice.availableFunds()));
        System.out.println("  sell 10 AMZN (none owned) -> "
                + alice.place(new MarketOrder("O8", "M1", "AMZN", Side.SELL, 10)));

        System.out.println("\n== 9. Blocked account cannot trade ==");
        admin.block(alice);
        System.out.println("  -> " + alice.place(new MarketOrder("O9", "M1", "AAPL", Side.BUY, 1)));
        admin.unblock(alice);

        System.out.println("\n== 10. Reports ==");
        System.out.print(alice.statement(new QuarterlyStatement(1, 90)));
        System.out.print(alice.statement(new TaxStatement(1, 365)));
    }
}
```

```java
// File: lld/ch19/brokerage/Market.java
package lld.ch19.brokerage;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;

/** A symbol and its latest price. */
final class Stock {
    private final String symbol;
    private volatile BigDecimal price;
    Stock(String symbol, BigDecimal price) { this.symbol = symbol; this.price = price; }
    String symbol() { return symbol; }
    BigDecimal price() { return price; }
    void setPrice(BigDecimal p) { price = p; }
}

/** The one place prices are read from, so no two parts of the system disagree. */
final class StockInventory {
    private final Map<String, Stock> stocks = new ConcurrentHashMap<>();
    void list(String symbol, BigDecimal price) { stocks.put(symbol, new Stock(symbol, price)); }
    Optional<Stock> find(String symbol) { return Optional.ofNullable(stocks.get(symbol)); }
    BigDecimal priceOf(String symbol) {
        return find(symbol).orElseThrow(() -> new IllegalArgumentException("unknown " + symbol)).price();
    }
    List<String> search(String prefix) {
        return stocks.keySet().stream().filter(s -> s.startsWith(prefix.toUpperCase())).sorted().toList();
    }
}

/** Observer: how the exchange calls back without knowing what a Member is. */
interface OrderListener {
    void onFill(Order order, OrderPart part);
    void onCancelled(Order order);
}

/**
 * A simulated exchange. At each price level it offers a fixed depth of shares
 * (e.g. 70 @ price, 70 @ price+0.05 ...) so that fills come back in parts, like a
 * real order book. (A real matching engine is built in lld.ch19.stockexchange.)
 * All methods are synchronized: one lock orders every event on the exchange.
 */
final class StockExchange {
    private final StockInventory inventory;
    private final int depthPerLevel;
    private final BigDecimal tickSize;
    private final int levels;
    private final Map<String, List<Order>> openOrders = new ConcurrentHashMap<>();
    private final Map<String, OrderListener> listeners = new ConcurrentHashMap<>();
    private int day = 1;

    StockExchange(StockInventory inventory, int depthPerLevel, BigDecimal tickSize, int levels) {
        this.inventory = inventory;
        this.depthPerLevel = depthPerLevel;
        this.tickSize = tickSize;
        this.levels = levels;
    }

    synchronized void register(String memberId, OrderListener l) { listeners.put(memberId, l); }
    synchronized void setDay(int d) { day = d; }
    synchronized int day() { return day; }

    synchronized void submit(Order o) {
        openOrders.computeIfAbsent(o.symbol(), k -> new ArrayList<>()).add(o);
        tryFill(o, inventory.priceOf(o.symbol()));
        prune(o.symbol());
    }

    /** A new price arrives: every open order on that symbol gets another chance (stops trigger here). */
    synchronized void tick(String symbol, BigDecimal newPrice) {
```

```
        inventory.find(symbol).orElseThrow().setPrice(newPrice);
        for (Order o : List.copyOf(openOrders.getOrDefault(symbol, List.of()))) tryFill(o, newPrice);
        prune(symbol);
    }

    synchronized boolean cancel(Order o) {
        if (!o.cancel()) return false;
        prune(o.symbol());
        listeners.get(o.memberId()).onCancelled(o);
        return true;
    }

    private void tryFill(Order o, BigDecimal market) {
        if (!o.isOpen() || !o.canTradeAt(market)) return;
        for (int k = 0; k < levels && o.remaining() > 0; k++) {
            BigDecimal step = tickSize.multiply(BigDecimal.valueOf(k));
            BigDecimal levelPrice = o.side() == Side.BUY ? market.add(step) : market.subtract(step);
            if (!o.acceptsPrice(levelPrice)) break;          // limit reached: rest stays open
            OrderPart part = new OrderPart(Math.min(depthPerLevel, o.remaining()), levelPrice, day);
            o.addPart(part);
            listeners.get(o.memberId()).onFill(o, part);
        }
    }

    private void prune(String symbol) {
        openOrders.getOrDefault(symbol, new ArrayList<>()).removeIf(o -> !o.isOpen());
    }
}
```

```java
// File: lld/ch19/brokerage/Orders.java
package lld.ch19.brokerage;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

enum Side { BUY, SELL }

enum OrderStatus { OPEN, PARTIALLY_FILLED, FILLED, CANCELLED }

/** Time in force: how long an order may stay alive. Modelled, only GTC used in the demo. */
enum TimeInForce { GOOD_TILL_CANCELLED, FILL_OR_KILL, IMMEDIATE_OR_CANCEL, ON_THE_OPEN, ON_THE_CLOSE }

final class Money {
    private Money() { }
    static BigDecimal of(String s) { return new BigDecimal(s).setScale(2, RoundingMode.HALF_EVEN); }
    static BigDecimal times(BigDecimal price, int qty) {
        return price.multiply(BigDecimal.valueOf(qty)).setScale(2, RoundingMode.HALF_EVEN);
    }
    static String fmt(BigDecimal b) { return "$" + b.setScale(2, RoundingMode.HALF_EVEN).toPlainString(); }
}

/** One fill. A 100-share order might be filled as 70 @ 10.00 plus 30 @ 10.05. */
record OrderPart(int quantity, BigDecimal price, int day) { }

/**
 * Everything the four order types share. They differ in exactly two answers:
 * (1) may I trade at this market price? (2) what is the worst price I accept?
 */
abstract class Order {
    private final String id;
    private final String memberId;
    private final String symbol;
    private final Side side;
    private final int quantity;
    private final TimeInForce timeInForce;
    private final List<OrderPart> parts = new ArrayList<>();
    private OrderStatus status = OrderStatus.OPEN;

    Order(String id, String memberId, String symbol, Side side, int quantity, TimeInForce tif) {
        if (quantity <= 0) throw new IllegalArgumentException("quantity must be positive");
        this.id = id;
        this.memberId = memberId;
        this.symbol = symbol;
        this.side = side;
        this.quantity = quantity;
        this.timeInForce = tif;
    }

    /** Question 1. Market and limit orders always say yes; stop orders wait for the trigger. */
    abstract boolean canTradeAt(BigDecimal marketPrice);

    /** Question 2. Empty means "any price" (market behaviour). */
    abstract Optional<BigDecimal> limitPrice();

    abstract String type();

    /** Is a fill at this price acceptable? BUY: price <= limit. SELL: price >= limit. */
    final boolean acceptsPrice(BigDecimal price) {
        return limitPrice().map(l -> side == Side.BUY ? price.compareTo(l) <= 0 : price.compareTo(l) >= 0)
                           .orElse(true);
    }

    final synchronized void addPart(OrderPart p) {
        if (p.quantity() > remaining()) throw new IllegalStateException("overfill");
        parts.add(p);
        status = remaining() == 0 ? OrderStatus.FILLED : OrderStatus.PARTIALLY_FILLED;
    }

    final synchronized boolean cancel() {
```

```
        if (status == OrderStatus.FILLED || status == OrderStatus.CANCELLED) return false;
        status = OrderStatus.CANCELLED;
        return true;
    }

    final synchronized int filled() { return parts.stream().mapToInt(OrderPart::quantity).sum(); }
    final synchronized int remaining() { return quantity - filled(); }
    final synchronized OrderStatus status() { return status; }
    final synchronized boolean isOpen() { return status == OrderStatus.OPEN || status == OrderStatus.PARTIALLY_FILLED;
}
    final synchronized List<OrderPart> parts() { return List.copyOf(parts); }

    final String id() { return id; }
    final String memberId() { return memberId; }
    final String symbol() { return symbol; }
    final Side side() { return side; }
    final int quantity() { return quantity; }
    final TimeInForce timeInForce() { return timeInForce; }

    @Override public String toString() {
        return id + " " + type() + " " + side + " " + quantity + " " + symbol + " [" + status() + "]";
    }
}

final class MarketOrder extends Order {
    MarketOrder(String id, String memberId, String symbol, Side side, int qty) {
        super(id, memberId, symbol, side, qty, TimeInForce.GOOD_TILL_CANCELLED);
    }
    @Override boolean canTradeAt(BigDecimal p) { return true; }
    @Override Optional<BigDecimal> limitPrice() { return Optional.empty(); }
    @Override String type() { return "MARKET"; }
}

final class LimitOrder extends Order {
    private final BigDecimal limit;
    LimitOrder(String id, String memberId, String symbol, Side side, int qty, BigDecimal limit, TimeInForce tif) {
        super(id, memberId, symbol, side, qty, tif);
        this.limit = limit;
    }
    @Override boolean canTradeAt(BigDecimal p) { return true; }
    @Override Optional<BigDecimal> limitPrice() { return Optional.of(limit); }
    @Override String type() { return "LIMIT@" + limit; }
}

/**
 * Stop (stop-loss) order: sleeps until the market reaches the stop price, then trades
 * like a market order. SELL stop fires when price <= stop, BUY stop when price >= stop.
 * Once triggered it stays triggered.
 */
class StopLossOrder extends Order {
    private final BigDecimal stopPrice;
    private boolean triggered;

    StopLossOrder(String id, String memberId, String symbol, Side side, int qty, BigDecimal stopPrice) {
        super(id, memberId, symbol, side, qty, TimeInForce.GOOD_TILL_CANCELLED);
        this.stopPrice = stopPrice;
    }

    @Override final synchronized boolean canTradeAt(BigDecimal p) {
        if (!triggered) {
            triggered = side() == Side.SELL ? p.compareTo(stopPrice) <= 0 : p.compareTo(stopPrice) >= 0;
        }
        return triggered;
    }
    @Override Optional<BigDecimal> limitPrice() { return Optional.empty(); }
    @Override String type() { return "STOP@" + stopPrice; }
    BigDecimal stopPrice() { return stopPrice; }
}

/** A stop order that becomes a LIMIT order (not a market order) once triggered. */
final class StopLimitOrder extends StopLossOrder {
    private final BigDecimal limit;
    StopLimitOrder(String id, String memberId, String symbol, Side side, int qty, BigDecimal stop, BigDecimal limit) {
        super(id, memberId, symbol, side, qty, stop);
```

```
        this.limit = limit;
    }
    @Override Optional<BigDecimal> limitPrice() { return Optional.of(limit); }
    @Override String type() { return "STOP@" + stopPrice() + "/LIMIT@" + limit; }
}
```

Output:

```
== 1. Fund the account ==
  wire 5,000        -> SUCCESS
  check 200,000     -> FAIL (over the 100,000 limit)
  cash = $5000.00

== 2. Watchlists and search ==
  tech: AAPL=10.00 MSFT=300.00
  retail: AMZN=120.00
  search 'A' -> [AAPL, AMZN]

== 3. Market buy 100 AAPL fills in two parts (two lots) ==
    [email alice@mail.test] filled 70 AAPL @ 10.00 (O1 PARTIALLY_FILLED)
    [email alice@mail.test] filled 30 AAPL @ 10.05 (O1 FILLED)
  place -> SUCCESS
  O1 MARKET BUY 100 AAPL [FILLED] parts=[OrderPart[quantity=70, price=10.00, day=1], OrderPart[quantity=30,
price=10.05, day=1]]

== 4. Next day: limit buy 50 AAPL @ 9.50 waits, then fills when price drops ==
  place -> SUCCESS, O2 LIMIT@9.50 BUY 50 AAPL [OPEN], reserved $475.00
    [email alice@mail.test] filled 50 AAPL @ 9.40 (O2 FILLED)
  after tick 9.40: O2 LIMIT@9.50 BUY 50 AAPL [FILLED], reserved $0.00
  holdings {AAPL=[L1:70@10.00, L2:30@10.05, L3:50@9.40]}

== 5. Day 3: price rises, sell 120 AAPL (FIFO across lots) ==
    [email alice@mail.test] filled 70 AAPL @ 11.00 (O3 PARTIALLY_FILLED)
    [email alice@mail.test] filled 50 AAPL @ 10.95 (O3 FILLED)
  place -> SUCCESS, O3 LIMIT@10.90 SELL 120 AAPL [FILLED]
  holdings {AAPL=[L3:30@9.40]}

== 6. Protect the rest with a stop-loss @ 10.00, then the price falls ==
  place -> SUCCESS, O4 STOP@10.00 SELL 30 AAPL [OPEN]
  tick 10.50: O4 STOP@10.00 SELL 30 AAPL [OPEN]
    [email alice@mail.test] filled 30 AAPL @ 9.95 (O4 FILLED)
  tick 9.95: O4 STOP@10.00 SELL 30 AAPL [FILLED] parts=[OrderPart[quantity=30, price=9.95, day=4]]

== 7. Stop-limit buy triggers but its limit is not met ==
  place -> SUCCESS, O5 STOP@305.00/LIMIT@306.00 BUY 5 MSFT [OPEN]
  tick 310 (stop hit, above limit): O5 STOP@305.00/LIMIT@306.00 BUY 5 MSFT [OPEN]
    [email alice@mail.test] filled 5 MSFT @ 305.50 (O5 FILLED)
  tick 305.50 (within limit): O5 STOP@305.00/LIMIT@306.00 BUY 5 MSFT [FILLED]

== 8. Reserved funds block over-spending, cancel releases them ==
  cash $3617.00, available $3617.00
  O6 buy 20 AMZN @ 100 -> SUCCESS, available $1617.00
  O7 buy 20 AMZN @ 100 -> INSUFFICIENT_FUNDS
    [email alice@mail.test] O6 cancelled, 20 shares unfilled
  cancel O6 -> SUCCESS, available $3617.00
  sell 10 AMZN (none owned) -> NO_STOCK_POSITION

== 9. Blocked account cannot trade ==
  -> ACCOUNT_NOT_ACTIVE

== 10. Reports ==
Quarterly statement for Alice (days 1-90)
  day 1 BUY  AAPL   70 @ 10.00
  day 1 BUY  AAPL   30 @ 10.05
  day 2 BUY  AAPL   50 @ 9.40
  day 3 SELL AAPL   70 @ 11.00
  day 3 SELL AAPL   50 @ 10.95
  day 4 SELL AAPL   30 @ 9.95
  day 4 BUY  MSFT    5 @ 305.50
Tax statement for Alice (days 1-365)
  AAPL L1: 70 sh bought @ 10.00 sold @ 11.00 gain $70.00
  AAPL L2: 30 sh bought @ 10.05 sold @ 10.95 gain $27.00
  AAPL L3: 20 sh bought @ 9.40 sold @ 10.95 gain $31.00
  AAPL L3: 30 sh bought @ 9.40 sold @ 9.95 gain $16.50
  realised gain: $144.50
```

Reading the output. Step 1: the 200,000 check fails the 100,000 transfer limit. Step 3 reproduces the course's example: a market order for 100 fills as 70 @ 10.00 and 30 @ 10.05 and creates **two lots**. Notice the course's warning in action: the order reserved $1,000.00 (100 × last price 10.00) but cost $1,001.50 — a market order can spend more than was reserved. Step 4: a limit buy waits until the price tick to 9.40.

Step 5 sells 120 shares FIFO across three lots. Step 6: the stop does nothing at 10.50 and fires at 9.95. Step 7: the stop-limit triggers at 310 but its 306 limit blocks the fill until 305.50. Step 8: the reserve blocks a second $2,000 order; cancelling returns the money. Step 10: the tax statement lists the gain **per lot**.

Part B — Matching engine (`lld.ch19.stockexchange`)

Files: `OrderBook.java` (side, order, trade, price policy, book), `MatchingEngine.java` (routing, per-symbol locking, line parser), `MatchingEngineTests.java` (nine tests, standard library only), `Main.java` (runs the sample — or a file given as the first argument — then the tests).

```
// File: lld/ch19/stockexchange/Main.java
package lld.ch19.stockexchange;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;

/** Usage: java lld.ch19.stockexchange.Main [orders.txt]  (no file = the sample from the problem statement). */
public class Main {
    static final List<String> SAMPLE = List.of(
            "#1 09:45 BAC sell 240.12 100",
            "#2 09:46 BAC sell 237.45 90",
            "#3 09:47 BAC buy 238.10 110",
            "#4 09:48 BAC buy 237.80 10",
            "#5 09:49 BAC buy 237.80 40",
            "#6 09:50 BAC sell 236.00 50");

    public static void main(String[] args) throws IOException {
        List<String> lines = args.length > 0 ? Files.readAllLines(Path.of(args[0])) : SAMPLE;

        System.out.println("== Trades (execution price = sell order's price) ==");
        MatchingEngine engine = new MatchingEngine(ExecutionPricePolicy.SELL_PRICE, System.out::println);
        lines.stream().filter(l -> !l.isBlank()).forEach(engine::submitLine);
        System.out.println("book after: " + engine.book("BAC").snapshot());

        System.out.println("\n== Same input, real-exchange rule (price of the resting order) ==");
        MatchingEngine real = new MatchingEngine(ExecutionPricePolicy.RESTING_PRICE, System.out::println);
        lines.stream().filter(l -> !l.isBlank()).forEach(real::submitLine);

        System.out.println("\n== Tests ==");
        int failed = MatchingEngineTests.runAll();
        if (failed > 0) System.exit(1);
    }
}
```

```java
// File: lld/ch19/stockexchange/MatchingEngine.java
package lld.ch19.stockexchange;

import java.math.BigDecimal;
import java.time.LocalTime;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;
import java.util.function.Consumer;

/**
 * Routes each order to its symbol's book. One lock per book: orders for different
 * symbols match in parallel, orders for the same symbol are strictly serialised
 * (which is what makes "time priority" well defined).
 */
final class MatchingEngine {
    private final Map<String, OrderBook> books = new ConcurrentHashMap<>();
    private final Map<String, String> symbolOfOrder = new ConcurrentHashMap<>();
    private final AtomicLong sequence = new AtomicLong();
    private final ExecutionPricePolicy policy;
    private final Consumer<Trade> tradeSink;

    MatchingEngine(ExecutionPricePolicy policy, Consumer<Trade> tradeSink) {
        this.policy = policy;
        this.tradeSink = tradeSink;
    }

    List<Trade> submit(String id, LocalTime time, String symbol, Side side, BigDecimal price, int qty) {
        if (qty <= 0 || price.signum() <= 0) throw new IllegalArgumentException("bad quantity/price: " + id);
        if (symbolOfOrder.putIfAbsent(id, symbol) != null) throw new IllegalArgumentException("duplicate order id " +
id);
        OrderBook book = books.computeIfAbsent(symbol, s -> new OrderBook(s, policy));
        List<Trade> trades;
        synchronized (book) {
            // the sequence is taken inside the lock, so arrival order == matching order
            Order o = new Order(id, time, symbol, side, price, qty, sequence.incrementAndGet());
            trades = book.add(o);
        }
        trades.forEach(tradeSink);        // publish outside the lock
        return trades;
    }

    boolean cancel(String orderId) {
        String symbol = symbolOfOrder.get(orderId);
        if (symbol == null) return false;
        OrderBook book = books.get(symbol);
        synchronized (book) { return book.cancel(orderId); }
    }

    OrderBook book(String symbol) { return books.get(symbol); }

    /** Parses "#1 09:45 BAC sell 240.12 100" and submits it. */
    List<Trade> submitLine(String line) {
        String[] f = line.trim().split("\\s+");
        if (f.length != 6) throw new IllegalArgumentException("bad line: " + line);
        return submit(f[0], LocalTime.parse(f[1]), f[2], Side.valueOf(f[3].toUpperCase()),
                new BigDecimal(f[4]), Integer.parseInt(f[5]));
    }
}
```

```java
// File: lld/ch19/stockexchange/MatchingEngineTests.java
package lld.ch19.stockexchange;

import java.math.BigDecimal;
import java.time.LocalTime;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;

/** Tiny test harness (no JUnit - standard library only). Each test throws AssertionError on failure. */
final class MatchingEngineTests {
    private MatchingEngineTests() { }

    interface Test { void run() throws Exception; }

    record Case(String name, Test body) { }

    static int runAll() {
        List<Case> tests = List.of(
                new Case("sample input gives the expected four trades", MatchingEngineTests::sample),
                new Case("better price wins over earlier time", MatchingEngineTests::pricePriority),
                new Case("same price: earlier order fills first", MatchingEngineTests::timePriority),
                new Case("no trade when prices do not cross", MatchingEngineTests::noCross),
                new Case("partial fill leaves the remainder resting", MatchingEngineTests::partialFill),
                new Case("cancelled order is never matched", MatchingEngineTests::cancel),
                new Case("symbols never match each other", MatchingEngineTests::symbolIsolation),
                new Case("duplicate id and bad quantity rejected", MatchingEngineTests::validation),
                new Case("concurrent submits conserve quantity", MatchingEngineTests::concurrency));
        int failed = 0;
        for (Case t : tests) {
            try {
                t.body().run();
                System.out.println("  PASS  " + t.name());
            } catch (Exception | AssertionError e) {
                failed++;
                System.out.println("  FAIL  " + t.name() + " -> " + e.getMessage());
            }
        }
        System.out.println("  " + (tests.size() - failed) + "/" + tests.size() + " passed");
        return failed;
    }

    // ---------- helpers ----------
    private static MatchingEngine engine(List<Trade> sink) {
        return new MatchingEngine(ExecutionPricePolicy.SELL_PRICE, sink::add);
    }

    private static void eq(Object expected, Object actual) {
        if (!expected.equals(actual)) throw new AssertionError("expected <" + expected + "> but was <" + actual +
">");
    }

    private static List<String> str(List<Trade> trades) { return trades.stream().map(Trade::toString).toList(); }

    private static final LocalTime T = LocalTime.of(10, 0);

    // ---------- tests ----------
    static void sample() {
        List<Trade> out = new ArrayList<>();
        MatchingEngine e = engine(out);
        Main.SAMPLE.forEach(e::submitLine);
        eq(List.of("#3 237.45 90 #2", "#3 236.00 20 #6", "#4 236.00 10 #6", "#5 236.00 20 #6"), str(out));
    }

    static void pricePriority() {
        List<Trade> out = new ArrayList<>();
        MatchingEngine e = engine(out);
        e.submit("s1", T, "X", Side.SELL, new BigDecimal("101"), 10);   // earlier but worse
```

```
        e.submit("s2", T, "X", Side.SELL, new BigDecimal("100"), 10);   // later but cheaper
        e.submit("b1", T, "X", Side.BUY, new BigDecimal("105"), 10);
        eq(List.of("b1 100 10 s2"), str(out));
    }

    static void timePriority() {
        List<Trade> out = new ArrayList<>();
        MatchingEngine e = engine(out);
        e.submit("b1", T, "X", Side.BUY, new BigDecimal("50"), 5);
        e.submit("b2", T, "X", Side.BUY, new BigDecimal("50"), 5);
        e.submit("s1", T, "X", Side.SELL, new BigDecimal("49"), 7);
        eq(List.of("b1 49 5 s1", "b2 49 2 s1"), str(out));
        eq(3, e.book("X").restingQty(Side.BUY));
    }

    static void noCross() {
        List<Trade> out = new ArrayList<>();
        MatchingEngine e = engine(out);
        e.submit("b1", T, "X", Side.BUY, new BigDecimal("99.99"), 5);
        e.submit("s1", T, "X", Side.SELL, new BigDecimal("100.00"), 5);
        eq(0, out.size());
        eq(5, e.book("X").restingQty(Side.BUY));
        eq(5, e.book("X").restingQty(Side.SELL));
    }

    static void partialFill() {
        List<Trade> out = new ArrayList<>();
        MatchingEngine e = engine(out);
        e.submit("s1", T, "X", Side.SELL, new BigDecimal("10"), 100);
        e.submit("b1", T, "X", Side.BUY, new BigDecimal("10"), 30);
        e.submit("b2", T, "X", Side.BUY, new BigDecimal("10"), 30);
        eq(40, e.book("X").restingQty(Side.SELL));
        eq(0, e.book("X").restingQty(Side.BUY));
    }

    static void cancel() {
        List<Trade> out = new ArrayList<>();
        MatchingEngine e = engine(out);
        e.submit("s1", T, "X", Side.SELL, new BigDecimal("10"), 10);
        e.submit("s2", T, "X", Side.SELL, new BigDecimal("10"), 10);
        eq(true, e.cancel("s1"));
        eq(false, e.cancel("s1"));
        e.submit("b1", T, "X", Side.BUY, new BigDecimal("10"), 10);
        eq(List.of("b1 10 10 s2"), str(out));
    }

    static void symbolIsolation() {
        List<Trade> out = new ArrayList<>();
        MatchingEngine e = engine(out);
        e.submit("s1", T, "AAA", Side.SELL, new BigDecimal("1"), 10);
        e.submit("b1", T, "BBB", Side.BUY, new BigDecimal("5"), 10);
        eq(0, out.size());
    }

    static void validation() {
        MatchingEngine e = engine(new ArrayList<>());
        e.submit("x1", T, "X", Side.BUY, new BigDecimal("1"), 1);
        try { e.submit("x1", T, "X", Side.BUY, new BigDecimal("1"), 1); throw new AssertionError("dup accepted"); }
        catch (IllegalArgumentException expected) { /* ok */ }
        try { e.submit("x2", T, "X", Side.BUY, new BigDecimal("1"), 0); throw new AssertionError("qty 0 accepted"); }
        catch (IllegalArgumentException expected) { /* ok */ }
    }

    /** 8 threads x 500 orders on one symbol. Every share bought must equal every share sold. */
    static void concurrency() throws InterruptedException {
        List<Trade> out = Collections.synchronizedList(new ArrayList<>());
        MatchingEngine e = engine(out);
        ExecutorService pool = Executors.newFixedThreadPool(8);
        CountDownLatch go = new CountDownLatch(1);
        AtomicInteger buys = new AtomicInteger();
        AtomicInteger sells = new AtomicInteger();
        for (int t = 0; t < 8; t++) {
            int thread = t;
            pool.submit(() -> {
```

```
                go.await();
                for (int i = 0; i < 500; i++) {
                    Side side = (i + thread) % 2 == 0 ? Side.BUY : Side.SELL;
                    int qty = 1 + (i % 7);
                    (side == Side.BUY ? buys : sells).addAndGet(qty);
                    e.submit("t" + thread + "-" + i, T, "X", side, new BigDecimal(100 + (i % 3)), qty);
                }
                return null;
            });
        }
        go.countDown();
        pool.shutdown();
        if (!pool.awaitTermination(30, TimeUnit.SECONDS)) throw new AssertionError("timeout");
        int traded = out.stream().mapToInt(Trade::quantity).sum();
        eq(buys.get(), traded + e.book("X").restingQty(Side.BUY));
        eq(sells.get(), traded + e.book("X").restingQty(Side.SELL));
        // the book must never be left crossed: best bid < best ask
        OrderBook book = e.book("X");
        if (book.bestBid().isPresent() && book.bestAsk().isPresent()
                && book.bestBid().get().compareTo(book.bestAsk().get()) >= 0)
            throw new AssertionError("book left crossed: " + book.snapshot());
    }
}
```

```java
// File: lld/ch19/stockexchange/OrderBook.java
package lld.ch19.stockexchange;

import java.math.BigDecimal;
import java.time.LocalTime;
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Deque;
import java.util.HashMap;
import java.util.Iterator;
import java.util.List;
import java.util.Map;
import java.util.NavigableMap;
import java.util.Optional;
import java.util.TreeMap;

enum Side { BUY, SELL }

/** An order. Only remainingQty changes, and only under the book's lock. */
final class Order {
    private final String id;
    private final LocalTime time;
    private final String symbol;
    private final Side side;
    private final BigDecimal price;
    private final long sequence;          // arrival order: breaks ties when times are equal
    private int remainingQty;

    Order(String id, LocalTime time, String symbol, Side side, BigDecimal price, int qty, long sequence) {
        if (qty <= 0) throw new IllegalArgumentException("quantity must be positive: " + id);
        if (price.signum() <= 0) throw new IllegalArgumentException("price must be positive: " + id);
        this.id = id; this.time = time; this.symbol = symbol; this.side = side;
        this.price = price; this.remainingQty = qty; this.sequence = sequence;
    }

    String id() { return id; }
    LocalTime time() { return time; }
    String symbol() { return symbol; }
    Side side() { return side; }
    BigDecimal price() { return price; }
    long sequence() { return sequence; }
    int remainingQty() { return remainingQty; }
    void fill(int qty) { remainingQty -= qty; }
    boolean isDone() { return remainingQty == 0; }
}

/** Output line: "<buy_order_id> <execution_price> <quantity> <sell_order_id>". */
record Trade(String buyOrderId, BigDecimal price, int quantity, String sellOrderId) {
    @Override public String toString() { return buyOrderId + " " + price.toPlainString() + " " + quantity + " " +
sellOrderId; }
}

/** Who sets the trade price. The Navi problem uses the SELL order's price; real exchanges use the resting order's. */
interface ExecutionPricePolicy {
    BigDecimal price(Order buy, Order sell, Order incoming);

    ExecutionPricePolicy SELL_PRICE = (buy, sell, incoming) -> sell.price();
    ExecutionPricePolicy RESTING_PRICE = (buy, sell, incoming) -> incoming == buy ? sell.price() : buy.price();
}

/**
 * The order book for ONE symbol. Each side is a sorted map of price level -> FIFO queue.
 * Bids: highest price first. Asks: lowest price first. Inside a level: first come, first served.
 * That is price-time priority. Not thread-safe by itself: the engine locks it.
 */
final class OrderBook {
    private final String symbol;
    private final NavigableMap<BigDecimal, Deque<Order>> bids = new TreeMap<>(Collections.reverseOrder());
    private final NavigableMap<BigDecimal, Deque<Order>> asks = new TreeMap<>();
    private final Map<String, Order> resting = new HashMap<>();
    private final ExecutionPricePolicy pricePolicy;
```

```
    OrderBook(String symbol, ExecutionPricePolicy pricePolicy) {
        this.symbol = symbol;
        this.pricePolicy = pricePolicy;
    }

    /** Match the incoming order against the other side, then rest whatever is left. */
    List<Trade> add(Order incoming) {
        List<Trade> trades = new ArrayList<>();
        NavigableMap<BigDecimal, Deque<Order>> opposite = incoming.side() == Side.BUY ? asks : bids;

        while (!incoming.isDone() && !opposite.isEmpty()) {
            Map.Entry<BigDecimal, Deque<Order>> best = opposite.firstEntry();
            if (!crosses(incoming, best.getKey())) break;          // best price does not meet ours: stop
            Deque<Order> queue = best.getValue();
            Iterator<Order> it = queue.iterator();
            while (!incoming.isDone() && it.hasNext()) {
                Order restingOrder = it.next();                    // oldest first = time priority
                int qty = Math.min(incoming.remainingQty(), restingOrder.remainingQty());
                Order buy = incoming.side() == Side.BUY ? incoming : restingOrder;
                Order sell = incoming.side() == Side.SELL ? incoming : restingOrder;
                trades.add(new Trade(buy.id(), pricePolicy.price(buy, sell, incoming), qty, sell.id()));
                incoming.fill(qty);
                restingOrder.fill(qty);
                if (restingOrder.isDone()) {
                    it.remove();
                    resting.remove(restingOrder.id());
                }
            }
            if (queue.isEmpty()) opposite.pollFirstEntry();
        }
        if (!incoming.isDone()) rest(incoming);
        return trades;
    }

    boolean cancel(String orderId) {
        Order o = resting.remove(orderId);
        if (o == null) return false;
        NavigableMap<BigDecimal, Deque<Order>> side = o.side() == Side.BUY ? bids : asks;
        Deque<Order> q = side.get(o.price());
        q.remove(o);
        if (q.isEmpty()) side.remove(o.price());
        return true;
    }

    int restingQty(Side side) {
        return (side == Side.BUY ? bids : asks).values().stream()
                .flatMap(Deque::stream).mapToInt(Order::remainingQty).sum();
    }

    Optional<BigDecimal> bestBid() { return bids.isEmpty() ? Optional.empty() : Optional.of(bids.firstKey()); }

    Optional<BigDecimal> bestAsk() { return asks.isEmpty() ? Optional.empty() : Optional.of(asks.firstKey()); }

    String snapshot() {
        return symbol + " bids=" + levels(bids) + " asks=" + levels(asks);
    }

    private static boolean crosses(Order incoming, BigDecimal bestOpposite) {
        return incoming.side() == Side.BUY
                ? incoming.price().compareTo(bestOpposite) >= 0     // buy meets or beats the ask
                : incoming.price().compareTo(bestOpposite) <= 0;    // sell meets or undercuts the bid
    }

    private void rest(Order o) {
        (o.side() == Side.BUY ? bids : asks).computeIfAbsent(o.price(), p -> new ArrayDeque<>()).addLast(o);
        resting.put(o.id(), o);
    }

    private static String levels(NavigableMap<BigDecimal, Deque<Order>> side) {
        List<String> out = new ArrayList<>();
        side.forEach((p, q) -> out.add(p.toPlainString() + "x" + q.stream().mapToInt(Order::remainingQty).sum()));
        return out.toString();
    }
}
```

Output:

```
== Trades (execution price = sell order's price) ==
#3 237.45 90 #2
#3 236.00 20 #6
#4 236.00 10 #6
#5 236.00 20 #6
book after: BAC bids=[237.80x20] asks=[240.12x100]

== Same input, real-exchange rule (price of the resting order) ==
#3 237.45 90 #2
#3 238.10 20 #6
#4 237.80 10 #6
#5 237.80 20 #6

== Tests ==
  PASS  sample input gives the expected four trades
  PASS  better price wins over earlier time
  PASS  same price: earlier order fills first
  PASS  no trade when prices do not cross
  PASS  partial fill leaves the remainder resting
  PASS  cancelled order is never matched
  PASS  symbols never match each other
  PASS  duplicate id and bad quantity rejected
  PASS  concurrent submits conserve quantity
  9/9 passed
```

Reading the output. The first block matches the expected output in the problem statement exactly. The second block runs the same orders with the real-exchange rule: #6 (the incoming sell) trades at each *resting* buyer's price (238.10, 237.80), which is how a continuous limit order book normally prices trades. The tests cover price priority, time priority, no-cross, partial fills, cancel, symbol isolation, validation, and a concurrency test (8 threads × 500 orders) that checks quantity is conserved and the book is never left crossed (best bid must stay below best ask).

Concurrency & edge cases

Risk	Where	Prevention
Over-spending with many open orders	Brokerage	`reserveFor` (synchronized on the member) subtracts all reservations from cash before accepting a new buy.
Selling the same shares twice	Brokerage	Open sell orders are tracked; free shares = position − remaining quantity of open sells *of that symbol*.
Deadlock between member and exchange	Brokerage	Lock order is fixed: the member never calls the exchange while holding its own lock (`place` reserves under the lock, then submits outside it). The exchange may call the member (exchange → member), never the reverse.
Market order costs more than reserved	Brokerage	Known gap (shown in step 3). Fix: reserve a buffer (e.g. +5%) or reject prices too far above the last price ("price band").
Time in force not enforced	Brokerage	The enum has five values (GTC, fill-or-kill, immediate-or-cancel, on-the-open, on-the-close) and no code reads them — every order behaves as good-till-cancelled within the demo. A GTC order should survive into the next trading day; FOK must be cancelled if it cannot be filled in full; IOC cancels whatever is left after one pass. All of this belongs in the exchange, next to the fill loop.
Two orders at the same price arrive "together"	Engine	Whichever thread gets the book's lock first is appended to the level's queue first, so lock-acquisition order = time priority. The `sequence` number is taken inside the same lock, so it records exactly that order (useful for logs and audits).
Cancel racing a match	Engine	Cancel takes the same per-symbol lock; an order is either matched or cancelled, never both.
Duplicate id / zero quantity / negative price	Engine	Validated before the id is registered (test `validation`).
Trade publishing slowing matching	Engine	Trades are collected under the lock and published after releasing it.
Floating-point prices	Both	`BigDecimal` everywhere; `TreeMap` keys compare with `compareTo`, so `237.8` and `237.80` land on the same level.

Real-world example

Indian brokers such as Zerodha offer the same stop order types to retail users (individual investors): **SL** (stop-loss *limit*) and **SL-M** (stop-loss *market*) — our `StopLimitOrder` and `StopLossOrder`. (Which segments allow SL-M has changed over time, so check the broker's current rules.) Their orders go to exchanges such as NSE (National Stock Exchange of India), whose continuous-trading order book matches on price-time priority, like our `OrderBook`. At the extreme end of performance, the LMAX exchange (described in Martin Fowler's article "The LMAX Architecture") runs its business logic, including matching, on a **single thread** that reads input events from a ring buffer (a fixed-size array used as a circular queue; LMAX's version is called the Disruptor) — the same idea as our "one lock per symbol, strictly serialised", taken to its logical end. On the tax side, US rules treat shares as sold first-in-first-out unless you identify specific lots, and FIFO is a common broker default — the reason `StockPosition.sell` takes an optional lot id.

Extensions & follow-up questions

- **"Add a trailing-stop order."** New subclass: it tracks the best price seen and moves its stop by a fixed distance. `canTradeAt` changes; nothing else does.
- **"Implement IOC and FOK."** IOC: after the fill loop, cancel any remainder. FOK: before filling, sum the quantity available at acceptable prices; if short, cancel without filling.
- **"Price alerts (the 'notify when a stock moves by X%' feature)."** A `PriceAlert` observer registered on `StockInventory`/`tick`; fires once when the move crosses the threshold.

- **"Modify an order."** On many exchanges, reducing quantity keeps time priority, but changing price (or increasing quantity) loses it — implement as cancel + new order.
- **"Make the engine faster."** Single writer thread per symbol fed by a queue (no locks on the hot path — the code that runs for every single order), pre-allocated objects, prices as `long` ticks instead of `BigDecimal`.
- **"Market orders in the engine."** Treat as a limit at +∞ (buy) or 0 (sell); never rest the remainder — cancel it.
- **"Settlement."** Trades settle later (for example T+1: one business day after the trade); add `unsettledCash` to the member so sale proceeds are not withdrawable immediately.

In the interview

30–45 minute plan (brokerage)

1. Clarify order types, lots, transfers, reports (3 min).
2. Actors and use cases (3 min).
3. Say the key sentence: *"The four order types differ in exactly two things: whether they may trade yet, and the worst price they accept."* Draw `Order` + four tiny subclasses (8 min).
4. Explain **lots** and why they exist: tax depends on which shares you sold (4 min).
5. Explain **reserved funds** (3 min).
6. Code `canTradeAt` / `limitPrice` for the four types and `StockPosition.sell` (10 min).
7. Observer callback, statements, extensions (5 min).

Machine-coding (Navi) plan, 90 minutes: parse lines → `Order`; `OrderBook` with two `TreeMap`s of `ArrayDeque`; `add()` matching loop; print trades; then tests for the sample, price priority, time priority, partial fill and cancel. Get the sample passing first, then refactor (restructure the code without changing what it does).

Follow-ups with answers

- *Why not four unrelated order classes?* They share member, symbol, quantity, side, status and fills; duplicating that invites bugs. The hierarchy isolates the two real differences.
- *Why is `StockInventory` separate from `StockExchange`?* One place to read prices, so no two components disagree about a price.
- *What data structure for the book, and the complexity?* `TreeMap` of price levels (a red-black tree — a self-balancing sorted tree): finding the best level with `firstEntry` and inserting a new level are both O(log L) for L price levels; FIFO queue per level: append O(1), match O(1) per fill. Cancel: O(log L) to find the level + O(k) in that level (use a linked structure with a handle for O(1)).
- *At what price does a trade execute?* In the Navi statement: the sell order's price. In real continuous trading: the resting order's price — the incoming order may get a better price than it asked for ("price improvement").

Self-check

1. **Q:** A SELL stop-loss at 10.00; the price goes 10.50 → 9.95 → 10.20. When does it trade? **A:** At the 9.95 tick. It stays triggered even when the price goes back up.
2. **Q:** What does a stop-limit add to a stop-loss? **A:** A worst acceptable price after triggering — it becomes a limit order, not a market order.
3. **Q:** Why does one market order create two lots in step 3? **A:** It filled at two prices, and a lot is defined by its purchase price.
4. **Q:** Two bids at 50.00 arrive, b1 then b2; a sell for 7 at 49 arrives. Trades? **A:** b1 gets 5, b2 gets 2 — time priority within the level (test `timePriority`).
5. **Q:** What does "the book is crossed" mean and why is it a bug? **A:** Best bid ≥ best ask; those two orders should already have traded.
6. **Q:** Which pattern produces the tax and quarterly statements? **A:** Template Method — shared filtering in `Statement.render`, specific lines in `body`.

TL;DR

> - **Brokerage:** one abstract `Order`, four tiny subclasses answering *may I trade now?* and *worst price?*; fills are `OrderPart`s; purchases are **lots**; money is **reserved** at placement; the exchange calls back

via **Observer**; statements are a **Template Method**.

- **Matching engine:** per symbol, two `TreeMap<price, FIFO queue>`; match best price first, oldest first; rest the remainder; one lock per symbol; trade price is a pluggable policy (Navi: sell price; real exchanges: resting price).

Design an Online Auction System

Problem & clarifying questions

An online auction (think eBay auctions) lets a **seller** list an item with a starting price and an end time. **Bidders** offer higher and higher amounts. When time runs out, the highest bidder wins — if the seller's hidden minimum (the **reserve price**) was reached.

Question	Assumed answer
What does a listing contain?	Item name, description, category, starting price, minimum bid increment, optional reserve price, duration.
What is a valid bid?	The first bid must be at least the starting price; later bids must beat the current highest by at least the increment.
Can the seller bid? Can the leader outbid themselves?	No and no.
When does an auction end?	At its end time. A late bid is refused even if the closing job has not run yet.
Who is told what?	The outbid bidder immediately; the winner and seller at close; the seller if unsold.
Proxy (automatic) bidding, anti-sniping (stopping last-second bids from stealing a win), payments?	Extensions — mention them.

Requirements

Functional

1. Users register and log in. (We model registration; login itself is the usual authentication (checking who a user is) service and is not coded here.)
2. Users create listings with item name, description, starting price and duration.
3. Users browse and search listings by name, category and price range.
4. Users place bids on **active** listings.
5. The system keeps the **current highest bid** up to date and notifies bidders.
6. When the duration ends, the auction closes and the **highest bidder wins** (if the reserve is met).

Non-functional

7. Many users bid on the same listing at the same moment; the system must stay consistent (one true highest bid).
8. Extensible: new bid rules, notification channels or search criteria without rewriting the core.

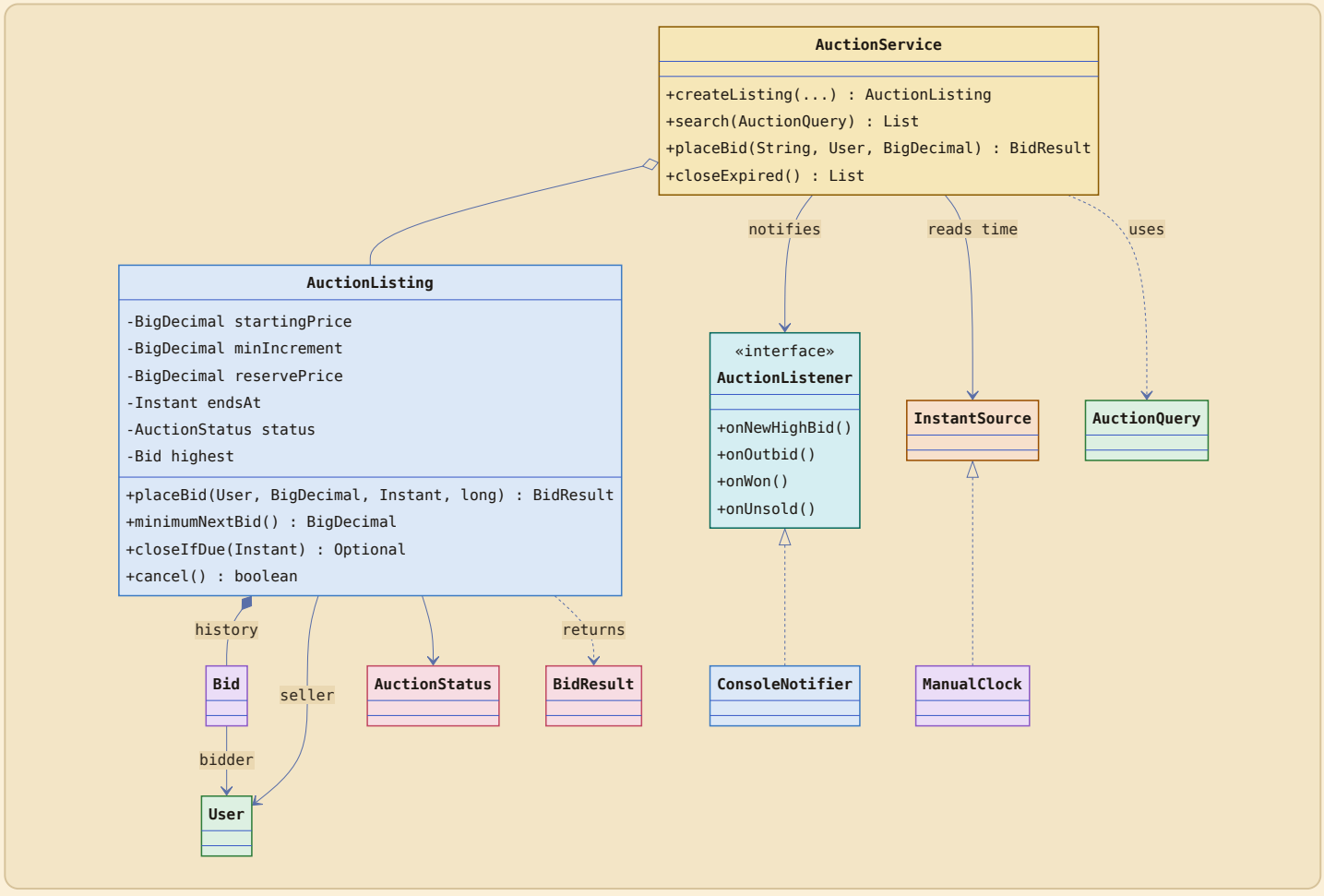
Use cases

Actor	Use cases
Seller	Create listing, cancel listing (only while nobody has bid), see result
Bidder	Search, view listing and bid history, place bid, be told when outbid or when won
System (scheduler)	Activate scheduled auctions, close expired ones, declare winner, send notifications

Core entities

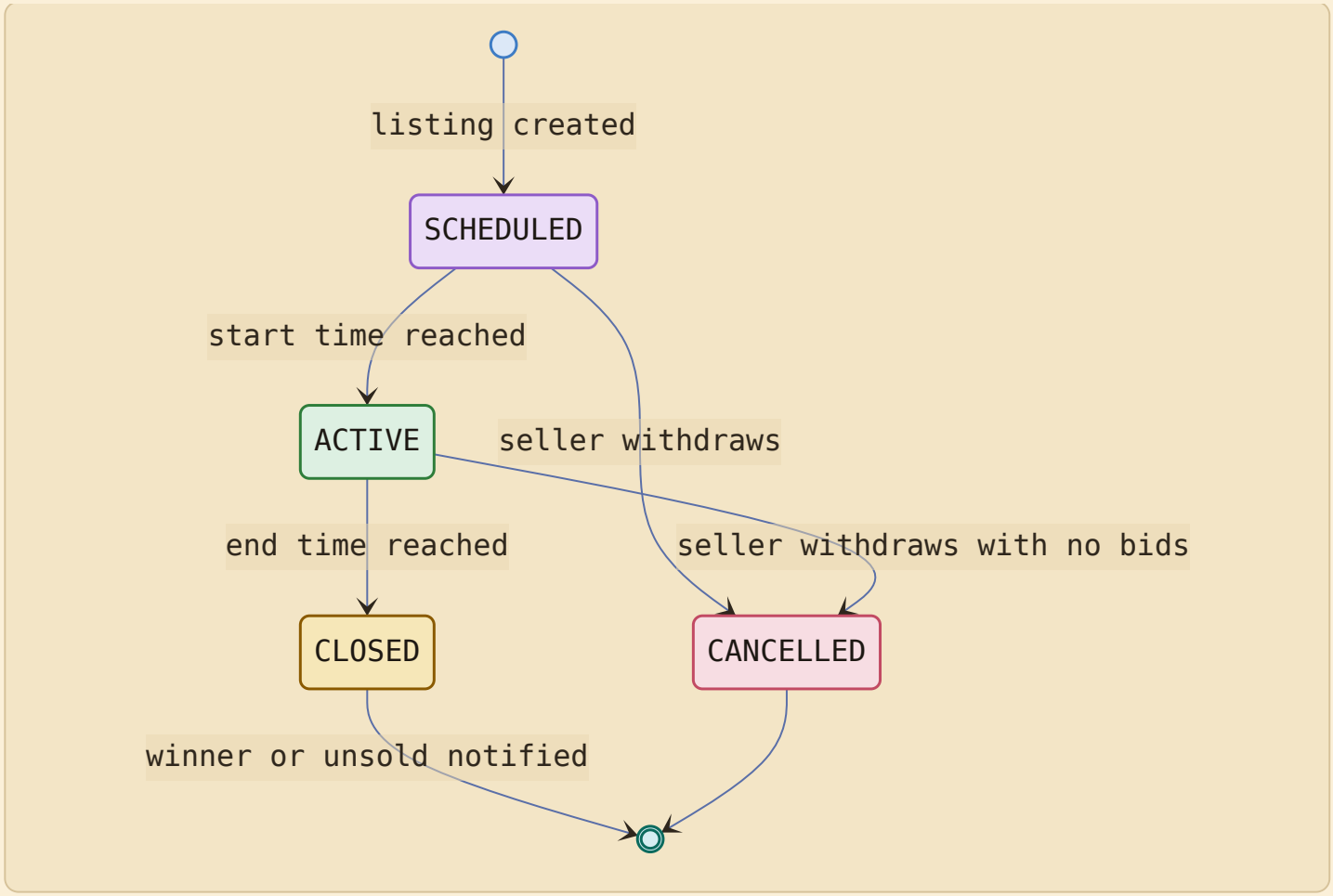
Class	Responsibility
`User`	id, name, email (a `record`).
`Bid`	bidder, amount (`BigDecimal`), time, sequence number. Immutable.
`AuctionStatus`	`SCHEDULED → ACTIVE → CLOSED`, or `CANCELLED`.
`AuctionListing`	Item details, seller, prices, times, bid history, highest bid. **Owns the bidding rules** and guards them with its own lock.
`BidOutcome` / `BidResult`	Why a bid was accepted or refused, the next minimum bid, the accepted bid, who was outbid.
`CloseResult`	Listing, winning bid (if any), whether the reserve was met.
`AuctionQuery`	Fluent (calls chained one after another) search criteria; each criterion is a `Predicate` (a function that answers true/false; Specification pattern).
`AuctionListener` / `ConsoleNotifier`	Observer: new high bid, outbid, won, unsold.
`AuctionService`	Facade: register, create, search, bid, close expired. Calls listeners **outside** listing locks.
`ManualClock`	An `InstantSource` (Java's "what time is it now?" interface) we can move forward in demos and tests.

Class diagram



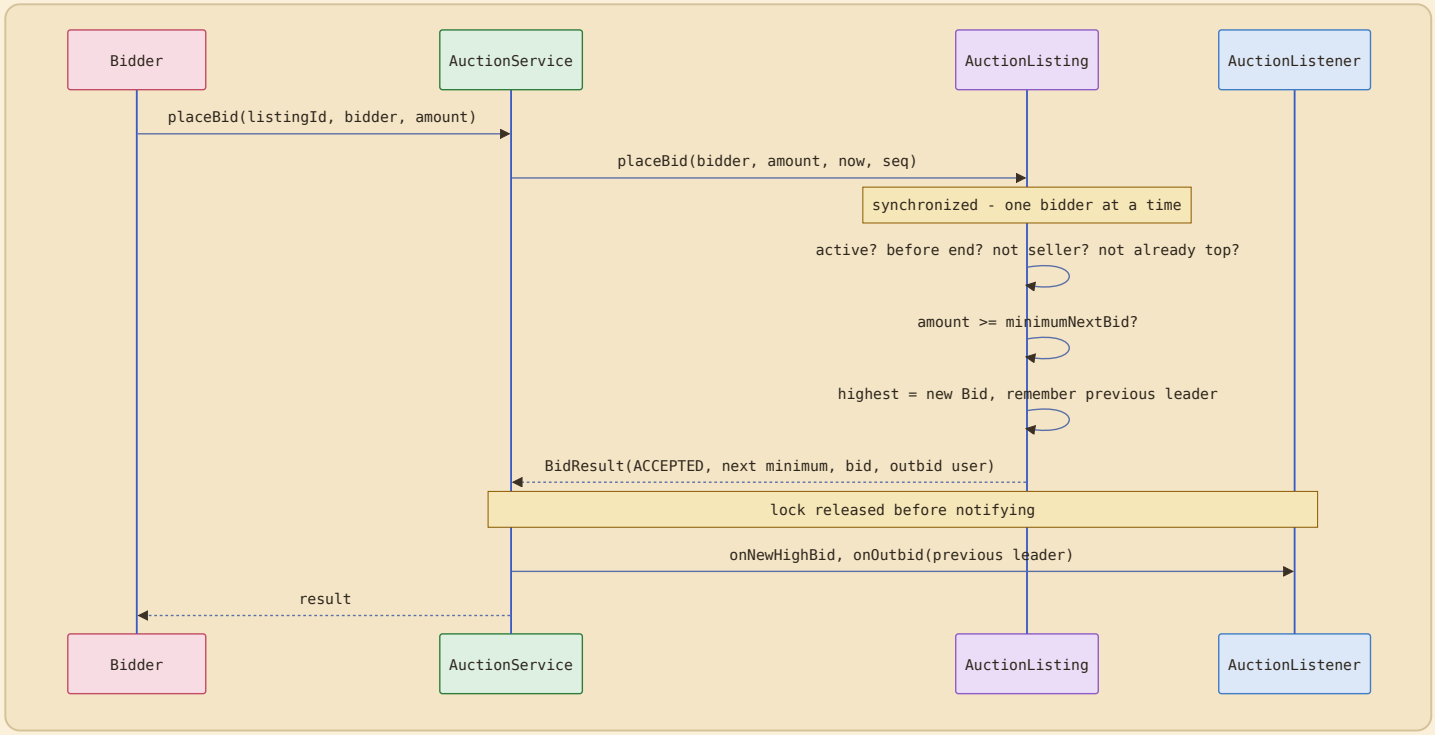
Key flows

Auction lifecycle



`SCHEDULED → ACTIVE` happens **lazily** (only when someone next touches the listing, not at the exact moment): every method first calls `refresh(now)`. No background thread is needed just to flip the state.

Placing a bid (the critical section) — a **critical section** is a piece of code that only one thread may run at a time because it reads and changes shared data.



Closing. A scheduler (in production a `ScheduledExecutorService` — Java's built-in timer that runs a task repeatedly — running every few seconds) calls `closeExpired()`. Each listing's `closeIfDue(now)` atomically

moves `ACTIVE → CLOSED` and returns the winner only if the reserve is met. Notifications go out after all locks are released.

Design decisions & patterns used

Decision	Why	Alternative rejected
Bid rules live inside `AuctionListing` and run under its lock (a *monitor*: an object whose methods are all `synchronized`)	"Read highest, compare, replace" must be one atomic step. The lock is per listing, so bids on different auctions never wait for each other.	A global lock in the service (all auctions serialised); or `AtomicReference<Bid>` (a reference that can be swapped atomically) + CAS loop (retry compare-and-set until it succeeds) — valid, but the rule set (seller, leader, end time, increment) is easier to get right under one lock.
Refusals are returned (`BidOutcome`), not thrown	A too-low bid is normal business, not an exceptional error (an **exception** is Java's way of aborting a method to signal that something went wrong). The caller also gets the next minimum to show in the UI.	Throwing an exception for every low bid.
Observer for notifications, called outside the lock	Slow email or push calls must not block other bidders. `BidResult` carries a **snapshot** (a copy frozen at one moment) of the accepted bid taken inside the lock, so listeners never read a newer, unrelated value.	Notifying inside `synchronized` (slow, and risks deadlock if a listener calls back).
Minimum increment	Stops 1-cent "wars" and makes the next minimum predictable.	Any higher amount accepted.
Reserve price	Lets a seller refuse to sell too cheaply; checked only at close.	Rejecting bids below the reserve (would reveal it).
`InstantSource` injected	Time-dependent code becomes testable: the demo moves a `ManualClock` forward instead of sleeping.	Calling `Instant.now()` everywhere.
Specification pattern for search	Each criterion is a `Predicate<AuctionListing>`; they combine with `and`. New criteria = one method.	One `search(name, category, min, max, ...)` method whose parameter list keeps growing.
The service is not a Singleton	awesome-low-level-design's version uses a Singleton `AuctionSystem`; we pass one instance around instead so tests can create fresh ones.	Global state shared by all tests.

Java implementation

Files: `Auction.java` (user, bid, status, outcome records, listing, manual clock), `AuctionService.java` (listener, notifier, query, service), `Main.java`.

```java
// File: lld/ch19/auction/Auction.java
package lld.ch19.auction;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.Duration;
import java.time.Instant;
import java.time.InstantSource;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

record User(String id, String name, String email) { }

/** One bid. seq breaks ties between bids with the same timestamp. */
record Bid(User bidder, BigDecimal amount, Instant at, long seq) { }

enum AuctionStatus { SCHEDULED, ACTIVE, CLOSED, CANCELLED }

/** Why a bid was refused - returned, not thrown, because refusals are normal business. */
enum BidOutcome { ACCEPTED, AUCTION_NOT_ACTIVE, AUCTION_ENDED, SELLER_CANNOT_BID, TOO_LOW, ALREADY_HIGHEST }

/** Result of placeBid: what happened, the bid that was accepted, who (if anyone) was just outbid. */
record BidResult(BidOutcome outcome, BigDecimal minimumNextBid, Optional<Bid> accepted, Optional<User> outbid) { }

/** Result of closing: sold to a winner, or unsold. */
record CloseResult(AuctionListing listing, Optional<Bid> winningBid, boolean reserveMet) { }

/**
 * One auction. All state changes go through synchronized methods, so the
 * "read highest bid, compare, replace" sequence is atomic: two bidders can never
 * both believe they are the highest.
 */
final class AuctionListing {
    private final String id;
    private final String itemName;
    private final String description;
    private final String category;
    private final User seller;
    private final BigDecimal startingPrice;
    private final BigDecimal minIncrement;
    private final BigDecimal reservePrice;          // hidden minimum; ZERO = no reserve
    private final Instant startsAt;
    private final Instant endsAt;
    private AuctionStatus status = AuctionStatus.SCHEDULED;
    private final List<Bid> bids = new ArrayList<>();
    private Bid highest;

    AuctionListing(String id, String itemName, String description, String category, User seller,
                   BigDecimal startingPrice, BigDecimal minIncrement, BigDecimal reservePrice,
                   Instant startsAt, Duration duration) {
        if (startingPrice.signum() <= 0 || minIncrement.signum() <= 0) throw new IllegalArgumentException("prices must
be > 0");
        this.id = id; this.itemName = itemName; this.description = description; this.category = category;
        this.seller = seller; this.startingPrice = money(startingPrice); this.minIncrement = money(minIncrement);
        this.reservePrice = money(reservePrice); this.startsAt = startsAt; this.endsAt = startsAt.plus(duration);
    }

    /** Lazily moves SCHEDULED -> ACTIVE when the start time has passed. */
    private void refresh(Instant now) {
        if (status == AuctionStatus.SCHEDULED && !now.isBefore(startsAt)) status = AuctionStatus.ACTIVE;
    }

    synchronized BidResult placeBid(User bidder, BigDecimal rawAmount, Instant now, long seq) {
        refresh(now);
        BigDecimal amount = money(rawAmount);
        if (status != AuctionStatus.ACTIVE) return refuse(BidOutcome.AUCTION_NOT_ACTIVE);
        if (!now.isBefore(endsAt)) return refuse(BidOutcome.AUCTION_ENDED);  // closing job may not have run yet
        if (bidder.id().equals(seller.id())) return refuse(BidOutcome.SELLER_CANNOT_BID);
        if (highest != null && highest.bidder().id().equals(bidder.id())) return refuse(BidOutcome.ALREADY_HIGHEST);
        if (amount.compareTo(minimumNextBid()) < 0) return refuse(BidOutcome.TOO_LOW);

        Optional<User> outbid = Optional.ofNullable(highest).map(Bid::bidder);
```

```
        highest = new Bid(bidder, amount, now, seq);
        bids.add(highest);
        return new BidResult(BidOutcome.ACCEPTED, minimumNextBid(), Optional.of(highest), outbid);
    }

    /** First bid must reach the starting price; later bids must beat the leader by the increment. */
    synchronized BigDecimal minimumNextBid() {
        return highest == null ? startingPrice : highest.amount().add(minIncrement);
    }

    /** ACTIVE -> CLOSED once time is up. Returns empty if not yet due (or already closed). */
    synchronized Optional<CloseResult> closeIfDue(Instant now) {
        refresh(now);
        if (status != AuctionStatus.ACTIVE || now.isBefore(endsAt)) return Optional.empty();
        status = AuctionStatus.CLOSED;
        boolean reserveMet = highest != null && highest.amount().compareTo(reservePrice) >= 0;
        return Optional.of(new CloseResult(this, reserveMet ? Optional.of(highest) : Optional.empty(), reserveMet));
    }

    /** A seller may withdraw only while nobody has bid. */
    synchronized boolean cancel() {
        if (status == AuctionStatus.CLOSED || status == AuctionStatus.CANCELLED || highest != null) return false;
        status = AuctionStatus.CANCELLED;
        return true;
    }

    private BidResult refuse(BidOutcome why) { return new BidResult(why, minimumNextBid(), Optional.empty(),
Optional.empty()); }

    private static BigDecimal money(BigDecimal b) { return b.setScale(2, RoundingMode.HALF_EVEN); }

    String id() { return id; }
    String itemName() { return itemName; }
    String description() { return description; }
    String category() { return category; }
    User seller() { return seller; }
    BigDecimal startingPrice() { return startingPrice; }
    synchronized AuctionStatus status(Instant now) { refresh(now); return status; }
    Instant endsAt() { return endsAt; }
    synchronized Optional<Bid> highestBid() { return Optional.ofNullable(highest); }
    synchronized BigDecimal currentPrice() { return highest == null ? startingPrice : highest.amount(); }
    synchronized List<Bid> bidHistory() { return List.copyOf(bids); }

    @Override public String toString() { return id + " '" + itemName + "' now " + currentPrice(); }
}

/** A controllable clock for demos and tests (production code passes InstantSource.system()). */
final class ManualClock implements InstantSource {
    private Instant now;
    ManualClock(Instant start) { now = start; }
    @Override public synchronized Instant instant() { return now; }
    synchronized void advance(Duration d) { now = now.plus(d); }
}
```

```java
// File: lld/ch19/auction/AuctionService.java
package lld.ch19.auction;

import java.math.BigDecimal;
import java.time.Duration;
import java.time.InstantSource;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicLong;
import java.util.function.Predicate;

/** Observer: who wants to hear about bids and results. */
interface AuctionListener {
    default void onNewHighBid(AuctionListing a, Bid bid) { }
    default void onOutbid(AuctionListing a, User outbid, BigDecimal newPrice) { }
    default void onWon(AuctionListing a, Bid winning) { }
    default void onUnsold(AuctionListing a, boolean hadBids) { }
}

final class ConsoleNotifier implements AuctionListener {
    private volatile boolean muted;
    void mute(boolean m) { muted = m; }
    private void say(String to, String msg) { if (!muted) System.out.println("    [notify " + to + "] " + msg); }

    @Override public void onOutbid(AuctionListing a, User u, BigDecimal p) {
        say(u.name(), "you were outbid on " + a.itemName() + ", now " + p);
    }
    @Override public void onWon(AuctionListing a, Bid w) {
        say(w.bidder().name(), "you WON " + a.itemName() + " for " + w.amount());
        say(a.seller().name(), a.itemName() + " sold to " + w.bidder().name());
    }
    @Override public void onUnsold(AuctionListing a, boolean hadBids) {
        say(a.seller().name(), a.itemName() + " unsold" + (hadBids ? " (reserve not met)" : " (no bids)"));
    }
}

/** Search criteria, built up fluently. Each criterion is a Predicate (Specification pattern). */
final class AuctionQuery {
    private final List<Predicate<AuctionListing>> parts = new ArrayList<>();
    AuctionQuery nameContains(String s) {
        parts.add(a -> a.itemName().toLowerCase(Locale.ROOT).contains(s.toLowerCase(Locale.ROOT))); return this;
    }
    AuctionQuery category(String c) { parts.add(a -> a.category().equalsIgnoreCase(c)); return this; }
    AuctionQuery priceBetween(BigDecimal lo, BigDecimal hi) {
        parts.add(a -> a.currentPrice().compareTo(lo) >= 0 && a.currentPrice().compareTo(hi) <= 0); return this;
    }
    Predicate<AuctionListing> toPredicate() { return parts.stream().reduce(x -> true, Predicate::and); }
}

/** Facade over listings, bidding and closing. Listeners are called OUTSIDE listing locks. */
final class AuctionService {
    private final InstantSource clock;
    private final Map<String, User> users = new ConcurrentHashMap<>();
    private final Map<String, AuctionListing> listings = new ConcurrentHashMap<>();
    private final List<AuctionListener> listeners = new CopyOnWriteArrayList<>();
    private final AtomicInteger listingSeq = new AtomicInteger();
    private final AtomicInteger userSeq = new AtomicInteger();
    private final AtomicLong bidSeq = new AtomicLong();

    AuctionService(InstantSource clock) { this.clock = clock; }

    void subscribe(AuctionListener l) { listeners.add(l); }

    User register(String name, String email) {
        User u = new User("U" + userSeq.incrementAndGet(), name, email);
        if (users.putIfAbsent(u.id(), u) != null) throw new IllegalStateException("id clash");
```

```
        return u;
    }

    AuctionListing createListing(User seller, String item, String description, String category,
                                 BigDecimal startingPrice, BigDecimal increment, BigDecimal reserve, Duration
duration) {
        AuctionListing a = new AuctionListing("A" + listingSeq.incrementAndGet(), item, description, category, seller,
                startingPrice, increment, reserve, clock.instant(), duration);
        listings.put(a.id(), a);
        return a;
    }

    List<AuctionListing> search(AuctionQuery q) {
        return listings.values().stream()
                .filter(a -> a.status(clock.instant()) == AuctionStatus.ACTIVE)
                .filter(q.toPredicate())
                .sorted(Comparator.comparing(AuctionListing::endsAt))
                .toList();
    }

    BidResult placeBid(String listingId, User bidder, BigDecimal amount) {
        AuctionListing a = listings.get(listingId);
        if (a == null) throw new IllegalArgumentException("no listing " + listingId);
        BidResult r = a.placeBid(bidder, amount, clock.instant(), bidSeq.incrementAndGet());
        r.accepted().ifPresent(top -> {                 // snapshot taken inside the lock
            listeners.forEach(l -> l.onNewHighBid(a, top));
            r.outbid().ifPresent(u -> listeners.forEach(l -> l.onOutbid(a, u, top.amount())));
        });
        return r;
    }

    /** Run periodically (e.g. by a ScheduledExecutorService). Closes every auction whose time is up. */
    List<CloseResult> closeExpired() {
        List<CloseResult> closed = new ArrayList<>();
        for (AuctionListing a : listings.values()) {
            Optional<CloseResult> r = a.closeIfDue(clock.instant());
            r.ifPresent(closed::add);
        }
        for (CloseResult r : closed) {
            if (r.winningBid().isPresent()) listeners.forEach(l -> l.onWon(r.listing(), r.winningBid().get()));
            else listeners.forEach(l -> l.onUnsold(r.listing(), r.listing().highestBid().isPresent()));
        }
        return closed;
    }
}
```

```java
// File: lld/ch19/auction/Main.java
package lld.ch19.auction;

import java.math.BigDecimal;
import java.time.Duration;
import java.time.Instant;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    public static void main(String[] args) throws Exception {
        ManualClock clock = new ManualClock(Instant.parse("2026-09-28T10:00:00Z"));
        AuctionService svc = new AuctionService(clock);
        ConsoleNotifier notifier = new ConsoleNotifier();
        svc.subscribe(notifier);

        User sam = svc.register("Sam", "sam@mail.test");
        User ana = svc.register("Ana", "ana@mail.test");
        User bob = svc.register("Bob", "bob@mail.test");

        System.out.println("== 1. Create listings ==");
        AuctionListing guitar = svc.createListing(sam, "Vintage Guitar", "1965 hollow body", "Music",
                bd("100"), bd("10"), bd("0"), Duration.ofHours(2));
        AuctionListing watch = svc.createListing(sam, "Pocket Watch", "Silver, working", "Collectibles",
                bd("50"), bd("5"), bd("500"), Duration.ofHours(1));
        System.out.println("  " + guitar + ", " + watch);

        System.out.println("\n== 2. Search (name 'watch', price 0-100) ==");
        System.out.println("  " + svc.search(new AuctionQuery().nameContains("watch").priceBetween(bd("0"),
bd("100"))));

        System.out.println("\n== 3. Bidding rules ==");
        show("Ana bids 90 (below start)", svc.placeBid(guitar.id(), ana, bd("90")));
        show("Ana bids 100", svc.placeBid(guitar.id(), ana, bd("100")));
        show("Ana bids 120 (already top)", svc.placeBid(guitar.id(), ana, bd("120")));
        show("Bob bids 105 (increment is 10)", svc.placeBid(guitar.id(), bob, bd("105")));
        show("Bob bids 110", svc.placeBid(guitar.id(), bob, bd("110")));
        show("Sam bids on own item", svc.placeBid(guitar.id(), sam, bd("500")));
        show("Ana bids 150", svc.placeBid(guitar.id(), ana, bd("150")));
        show("Bob bids 60 on the watch", svc.placeBid(watch.id(), bob, bd("60")));

        System.out.println("\n== 4. One hour later: the watch closes, reserve (500) not met ==");
        clock.advance(Duration.ofHours(1));
        svc.closeExpired().forEach(r -> System.out.println("  closed " + r.listing().id() + " reserveMet=" +
r.reserveMet()));

        System.out.println("\n== 5. 50 threads bid on the guitar at once ==");
        List<User> crowd = new ArrayList<>();
        for (int i = 0; i < 50; i++) crowd.add(svc.register("bidder" + i, "b" + i + "@mail.test"));
        notifier.mute(true);                // ~50 outbid messages would drown the output
        AtomicInteger accepted = new AtomicInteger();
        CountDownLatch go = new CountDownLatch(1);
        ExecutorService pool = Executors.newFixedThreadPool(16);
        for (int i = 0; i < crowd.size(); i++) {
            User u = crowd.get(i);
            BigDecimal amount = bd(String.valueOf(160 + i * 10));
            pool.submit(() -> {
                go.await();
                if (svc.placeBid(guitar.id(), u, amount).outcome() == BidOutcome.ACCEPTED) accepted.incrementAndGet();
                return null;
            });
        }
        go.countDown();
        pool.shutdown();
        pool.awaitTermination(10, TimeUnit.SECONDS);
        notifier.mute(false);
        Bid top = guitar.highestBid().orElseThrow();
        boolean increasing = true;
```

```
        List<Bid> history = guitar.bidHistory();
        for (int i = 1; i < history.size(); i++)
            increasing &= history.get(i).amount().compareTo(history.get(i - 1).amount().add(bd("10"))) >= 0;
        System.out.println("  accepted=" + accepted + " (varies run to run), highest=" + top.amount() + " by " +
top.bidder().name()
                + ", history strictly increasing by >= increment: " + increasing);

        System.out.println("\n== 6. Time is up: the guitar closes and the winner is told ==");
        clock.advance(Duration.ofHours(1));
        show("late bid", svc.placeBid(guitar.id(), ana, bd("5000")));
        svc.closeExpired();
        System.out.println("  status " + guitar.status(clock.instant()) + ", final price " + guitar.currentPrice());
    }

    private static BigDecimal bd(String s) { return new BigDecimal(s); }

    private static void show(String what, BidResult r) {
        System.out.println("  " + what + " -> " + r.outcome() + " (next minimum " + r.minimumNextBid() + ")");
    }
}
```

Output:

```
== 1. Create listings ==
  A1 'Vintage Guitar' now 100.00, A2 'Pocket Watch' now 50.00

== 2. Search (name 'watch', price 0-100) ==
  [A2 'Pocket Watch' now 50.00]

== 3. Bidding rules ==
  Ana bids 90 (below start) -> TOO_LOW (next minimum 100.00)
  Ana bids 100 -> ACCEPTED (next minimum 110.00)
  Ana bids 120 (already top) -> ALREADY_HIGHEST (next minimum 110.00)
  Bob bids 105 (increment is 10) -> TOO_LOW (next minimum 110.00)
    [notify Ana] you were outbid on Vintage Guitar, now 110.00
  Bob bids 110 -> ACCEPTED (next minimum 120.00)
  Sam bids on own item -> SELLER_CANNOT_BID (next minimum 120.00)
    [notify Bob] you were outbid on Vintage Guitar, now 150.00
  Ana bids 150 -> ACCEPTED (next minimum 160.00)
  Bob bids 60 on the watch -> ACCEPTED (next minimum 65.00)

== 4. One hour later: the watch closes, reserve (500) not met ==
    [notify Sam] Pocket Watch unsold (reserve not met)
  closed A2 reserveMet=false

== 5. 50 threads bid on the guitar at once ==
  accepted=28 (varies run to run), highest=650.00 by bidder49, history strictly increasing by >= increment: true

== 6. Time is up: the guitar closes and the winner is told ==
  late bid -> AUCTION_ENDED (next minimum 660.00)
    [notify bidder49] you WON Vintage Guitar for 650.00
    [notify Sam] Vintage Guitar sold to bidder49
  status CLOSED, final price 650.00
```

Reading the output. Step 3 exercises every rule: below start, leader re-bidding, below increment, seller bidding. The outbid notices print *before* the bidder's own result line because the service notifies before returning. Step 4 closes the watch as unsold because 60 < the 500 reserve. Step 5 fires 50 concurrent bids: how many are accepted depends on thread timing (a bid that arrives after a higher one is refused as too low), but the final highest is always the maximum, 650.00, and the history is always strictly increasing by at least the increment — the invariant (a rule that must always be true) the lock protects. Step 6 refuses a bid after the end time even though the closing job has not yet run.

Concurrency & edge cases

Case	Handling
Two bidders submit the same amount at the same time	The first one through the lock wins; the second is now below `minimumNextBid()` and gets `TOO_LOW`.
Bid arrives exactly at the end time	`!now.isBefore(endsAt)` → `AUCTION_ENDED`. The end instant is exclusive.
Close job runs late	Bids after `endsAt` are refused anyway, so a late job cannot change the winner.
Close and bid race	Both are `synchronized` on the listing; the bid either lands before the close (and counts) or after (and is refused).
Notification listener is slow or throws	Called outside the lock; in production, publish to a queue so failures are retried without affecting bidding.
Seller cancels after bids	Refused (`cancel()` returns false) — protects bidders.
Money precision	Amounts normalised to scale 2 with `BigDecimal`; compared with `compareTo`.
Very hot auction (thousands of bids per second)	One lock per listing is fine for one JVM (Java Virtual Machine — one running Java process); distributed: a single-writer per auction (partition bids by auction id — split them into separate streams so all bids for one auction go to the same place, e.g. a Kafka partition, Kafka being a widely used event-streaming system), or a conditional update `UPDATE ... SET high = ? WHERE id = ? AND high < ?`.

Real-world example

eBay auctions use **proxy bidding**: you enter the most you are willing to pay, and eBay bids on your behalf only as much as needed to stay ahead, by the current increment. eBay also supports hidden **reserve prices**, and its auctions end at a fixed time, which is why "sniping" (bidding in the last seconds) is common there. Some other auction sites use a **soft close** — a late bid extends the end time — precisely to defeat sniping.

Extensions & follow-up questions

- **"Add proxy (automatic) bidding."** Each bidder may register a `maxAmount`. After any accepted bid, inside the same lock, let the strongest proxy respond with `min(its max, competitor + increment)`. Ties go to the earlier proxy (use `Bid.seq`).
- **"Stop sniping."** Soft close: in `placeBid`, if an accepted bid arrives within the last N minutes, set `endsAt = endsAt + N` (make `endsAt` non-final, change under the same lock).
- **"Buy It Now."** A fixed price that closes the auction immediately if paid before the first bid.
- **"Dutch / sealed-bid / Vickrey auctions."** (Dutch: the price starts high and drops until someone accepts; sealed-bid: bidders cannot see each other's bids.) Extract an `AuctionRules` strategy: `validate(bid)`, `winner(bids)`, `price(bids)`. Vickrey (second-price sealed bid): highest bidder wins but pays the second-highest bid.
- **"Payment after winning."** Winner has 48 hours to pay (new state `AWAITING_PAYMENT`); if not, offer to the second-highest bidder.
- **"Scale the close job."** Keep a priority queue (or a sorted index) by `endsAt` so the scheduler only touches auctions that are due.

In the interview

30–45 minute plan

1. Clarify the bid rules (start price, increment, reserve, seller/leader restrictions) — 4 min.
2. Entities and the lifecycle state machine — 8 min.
3. Code `placeBid` under the listing lock and **say why the lock is per listing** — 12 min.
4. Closing + notifications outside the lock — 6 min.
5. Concurrency discussion and extensions (proxy bidding, soft close) — 10 min.

> **30-second answer:** "Each listing owns its bid rules and its highest bid, and `placeBid` is synchronized on the listing, so compare-and-replace is atomic and different auctions never contend. Refusals return a reason and the next minimum. The service notifies outbid users and, at close, the winner and seller, always after releasing the lock. Time comes from an injected clock, so closing and late bids are testable."

Follow-ups with answers

- *Why not `AtomicReference` + CAS?* Possible for the amount alone, but the rule also checks seller, leader and time. A tiny critical section under one lock is simpler and still fast because the lock is per listing.
- *How do you close auctions on time across servers?* A scheduler that owns a time-ordered index of end times; the close itself is idempotent (`closeIfDue` returns empty the second time), so running it twice is harmless.
- *How do you show a live price to thousands of viewers?* Push `onNewHighBid` events to a pub/sub channel (publish/subscribe: senders post messages to a topic and every subscriber receives them) and WebSockets (a connection the browser keeps open so the server can push data at any time); viewers read a cache (a fast saved copy of data — see System Design Fundamentals), not the lock-holding object.

Self-check

1. **Q:** What two conditions make a bid `TOO_LOW`? **A:** Below the starting price when there is no bid, or below highest + increment otherwise.
2. **Q:** Why are listeners called after `placeBid` returns from the listing? **A:** To keep the lock short and avoid deadlock or slowdown from notification code.
3. **Q:** A bid arrives at 12:00:00.000 and the auction ends at 12:00:00.000. Accepted? **A:** No — the end time is exclusive.
4. **Q:** Where does the reserve price matter? **A:** Only at close: if the highest bid is below it, the item is unsold.
5. **Q:** What is the Specification pattern here? **A:** Search criteria as composable `Predicate`s combined with `and`.

TL;DR

- One **lock per listing**; all bid rules inside it; refusals are values, not exceptions.
- Lifecycle `SCHEDULED → ACTIVE → CLOSED | CANCELLED`; activation is lazy, closing is idempotent.
- **Observer** notifications after the lock; **injected clock** for testable time; **Specification** for search.

Design a Music Streaming Service (Spotify)

Problem & clarifying questions

A music streaming service lets users search a huge catalog, build playlists, and play songs with the usual controls (play, pause, skip, previous, seek, shuffle, repeat). Free and paid plans behave differently. The service suggests new music.

Question	Assumed answer
What can users search?	Songs, albums, artists.
Playlists?	Users create, reorder and edit their own playlists.
Playback controls?	Play, pause, resume, skip, previous, seek; shuffle on/off; repeat off / all / one.
Plans?	FREE (ads, limited skips per hour, no downloads) and PREMIUM (no ads, unlimited skips, offline).
Recommendations?	A stub (a simple placeholder) based on listening history; real ones are an ML (machine learning — software that learns patterns from data) system.
Actual audio streaming, DRM (copy protection), CDNs (content delivery networks — servers near users that hold copies of files)?	Out of scope for LLD (low-level design: classes and their code) — the player models *what* plays, not the bytes.

Requirements

Functional

1. Browse and search songs, albums and artists.
2. Create and manage playlists.
3. User accounts and authentication (we model users and plans; login itself is the usual auth service).
4. Playback controls: play, pause, skip, seek (plus previous, shuffle, repeat).
5. Recommendations based on preferences and listening history.
6. Subscriptions: FREE vs PREMIUM with different limits.

Non-functional

7. Many users stream at once; the shared catalog is read-mostly and thread-safe (works correctly when many threads use it at once).
8. Scales to a large catalog and user base (indexes, not scans, in production).
9. Extensible: social sharing, offline mode, new play orders, new recommenders.

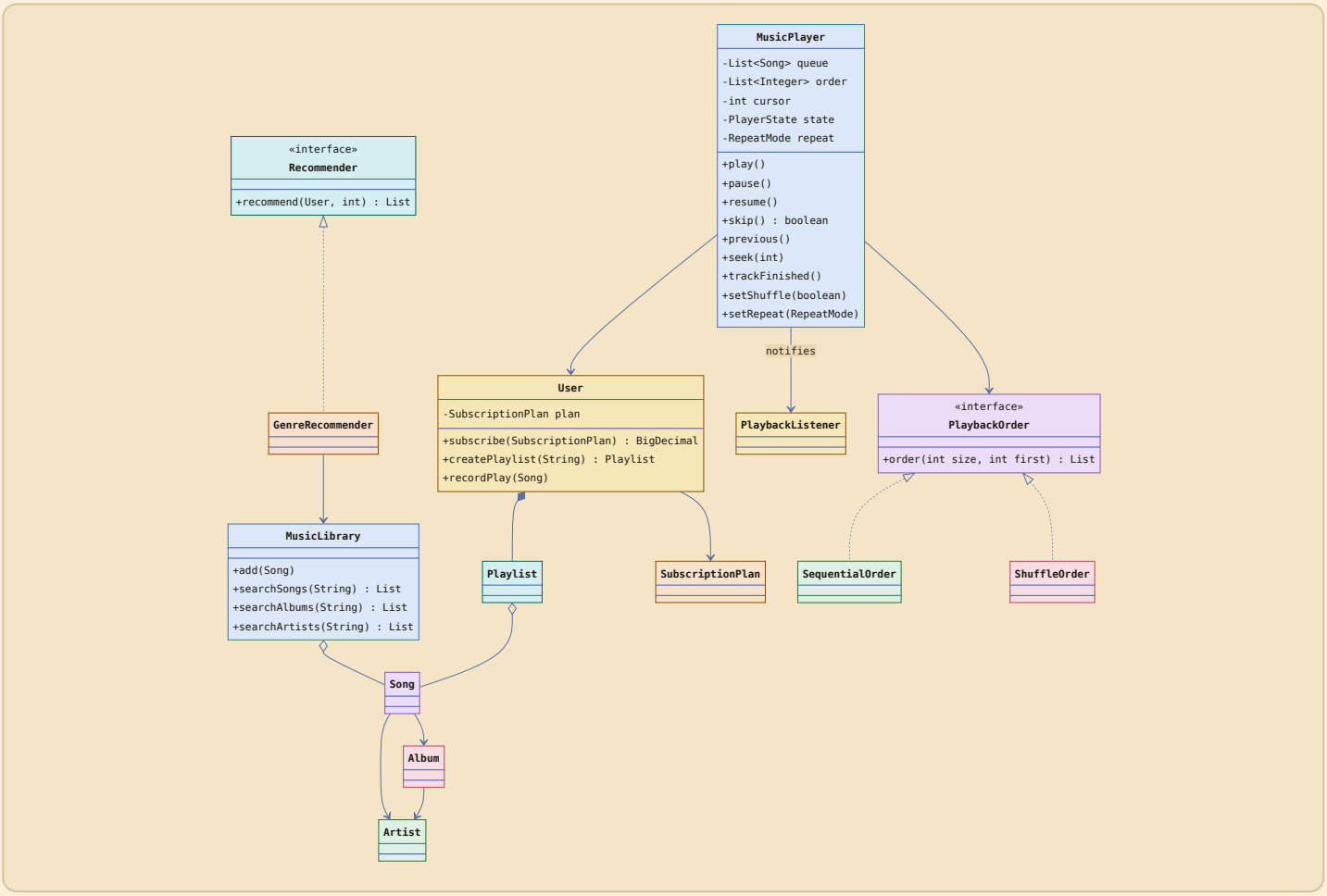
Use cases

Actor	Use cases
Listener	Search, play/pause/skip/seek, shuffle/repeat, create and edit playlists, upgrade plan
Artist / label (content admin)	Add songs and albums to the catalog
System	Record plays (history, royalties — money owed to artists per play), insert ads for free users, recommend songs

Core entities

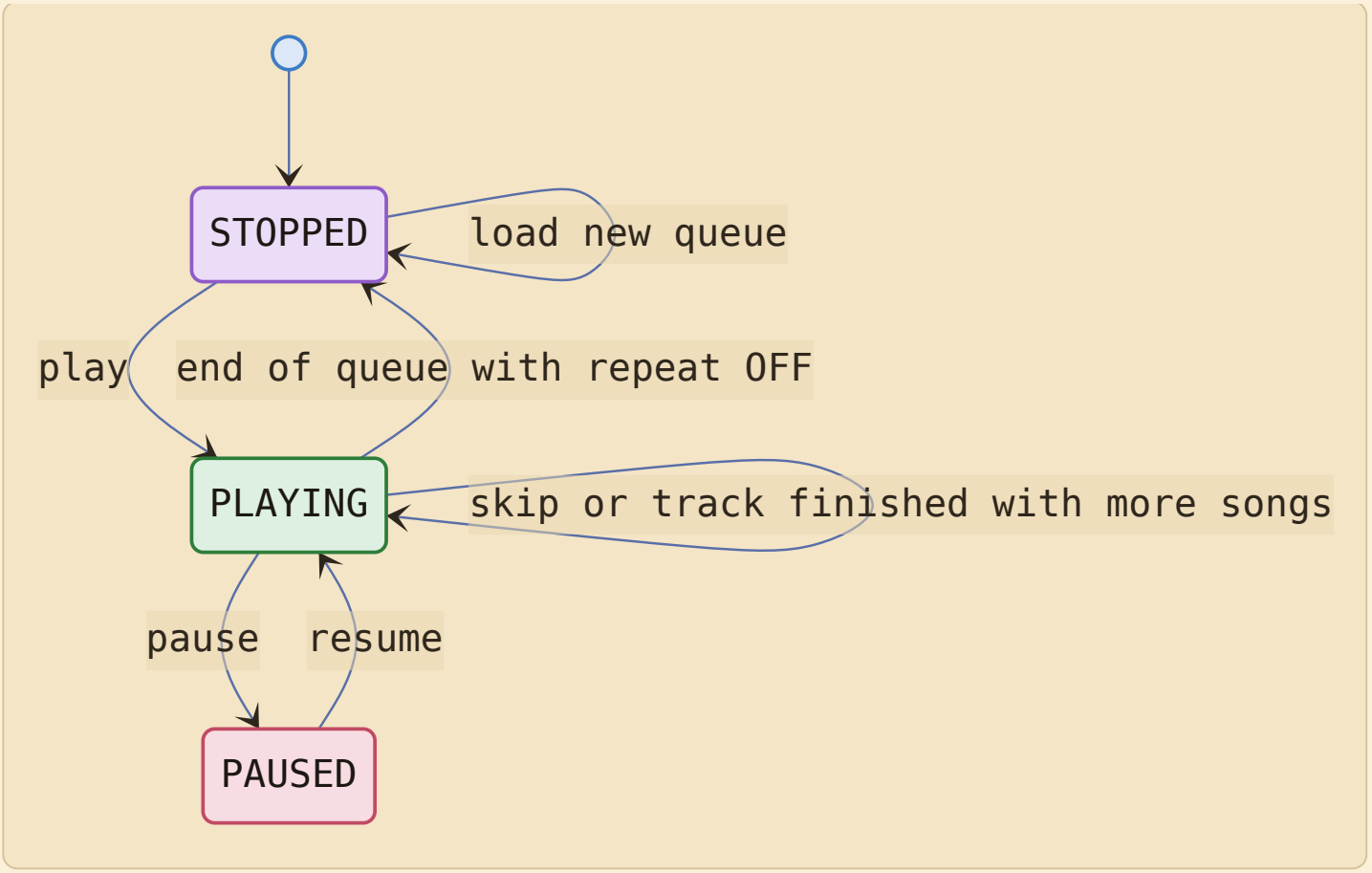
Class	Responsibility
`Artist`, `Album`, `Song`, `Genre`	Immutable catalog records.
`MusicLibrary`	Thread-safe catalog with search by song title, album title, artist name.
`SubscriptionPlan`	Enum that **carries the rules**: monthly price, skips per hour, ads, offline downloads.
`User`	Name, current plan, playlists, listening history.
`Playlist`	Ordered, user-owned song list: add, remove, move.
`PlaybackOrder` → `SequentialOrder`, `ShuffleOrder`	Strategy: produces the play order (a permutation — every song exactly once, in some order) starting from the current song.
`RepeatMode`, `PlayerState`	`OFF / ALL / ONE`; `STOPPED / PLAYING / PAUSED`.
`MusicPlayer`	One user's session: queue, play order, cursor (where we are in the play order), position, state, skip window, ad counter.
`PlaybackListener`	Observer on "song started" (history, royalties, recommendations).
`Recommender` → `GenreRecommender`	Strategy: suggest unplayed songs from the user's favourite genres.

Class diagram

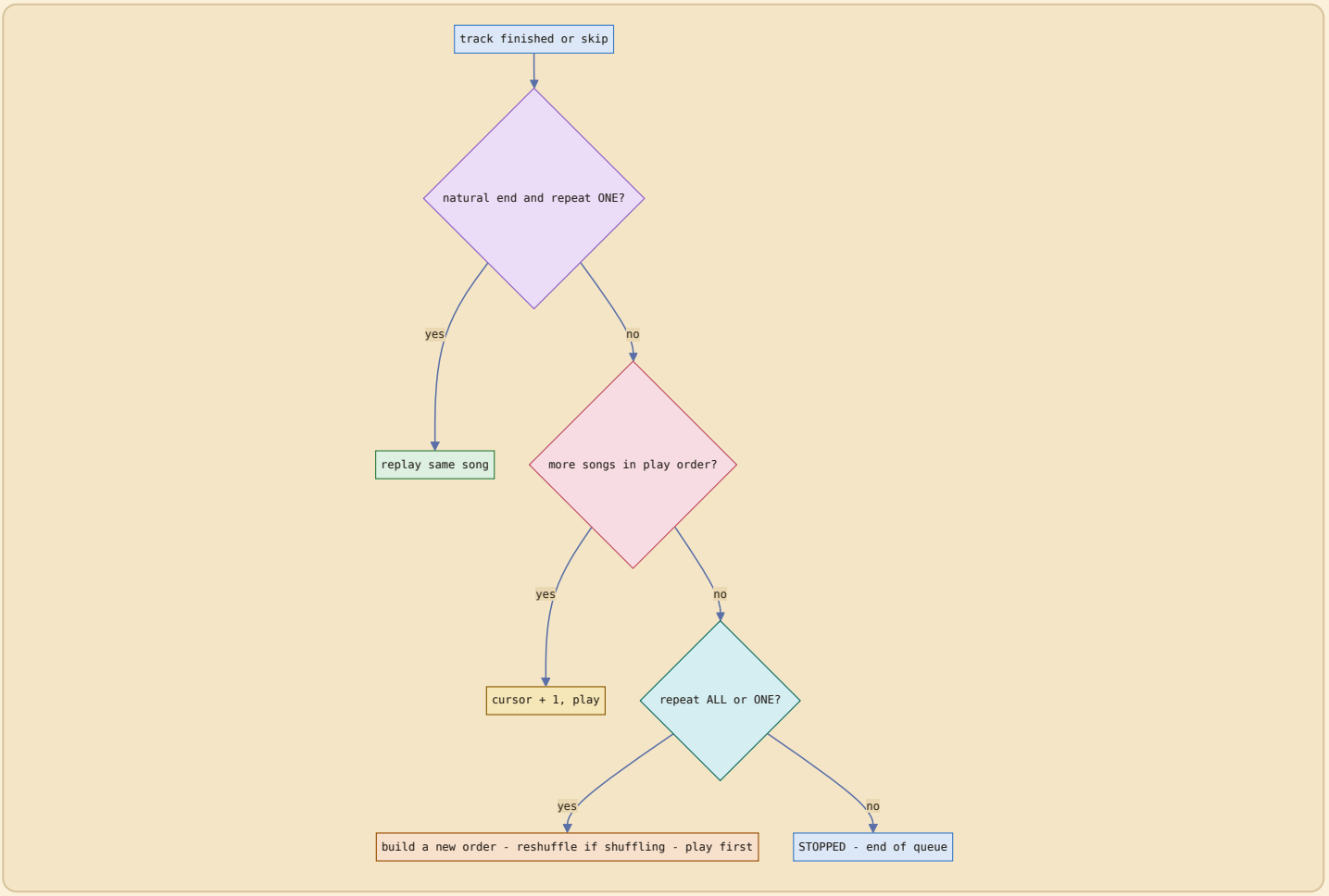


Key flows

Player states



What happens when a song ends or the user presses Next



Skip first checks the plan's rolling (always the last 60 minutes, not a fixed clock hour) one-hour limit (a deque — a queue you can add to or remove from at both ends — of skip timestamps; drop those older than an hour, compare the size with `skipsPerHour`). Pressing Next on repeat-ONE moves on — like real players, repeat-one only affects songs that end naturally.

Shuffle keeps the current song. Turning shuffle on builds a new permutation whose first element is the song playing now, and shuffles the rest with `Collections.shuffle(list, random)`. Turning it off rebuilds the sequential order starting from the current song. The queue itself never changes — only the **order** of indexes into it — so switching back and forth loses nothing.

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for play order (`PlaybackOrder`)	Shuffle, sequential, "smart shuffle" (spread the same artist apart) are interchangeable algorithms. The player just asks for an order.	`if (shuffle) ... else ...` inside `skip`, `previous`, `wrap`.
Order as a list of indexes into an unchanged queue	Turning shuffle off/on never loses the original order; `upNext` is a simple walk.	Physically shuffling the queue list (original order lost).
Seeded `Random`	Repeatable demos and tests. Production uses an unseeded (or per-session) seed.	`Math.random()` (not testable).
Plan rules inside `SubscriptionPlan` enum	`user.plan().skipsPerHour` and `.ads` — no `if (plan == FREE)` scattered around; a new plan is one enum constant (or a class if rules get complex).	Boolean flags on `User`.
Observer for plays (`PlaybackListener`)	History, royalty counting and recommendation features all react to "song started" without the player knowing them.	Player calling each subsystem directly.
Strategy for recommendations	`Recommender` can be genre-based today, collaborative filtering ("people who liked what you like also liked…") tomorrow.	Hard-coding one algorithm in `User`.
One `MusicPlayer` per user session, all methods `synchronized`	Commands can arrive from two devices (phone + desktop "connect"). The shared `MusicLibrary` uses concurrent maps.	A single shared player object.
State as an enum, not the State pattern	Three states with trivial behaviour; the GoF ("Gang of Four", the authors of the classic 1994 *Design Patterns* book) State pattern (one class per state) pays off only when each state has substantial, different behaviour.	Three state classes for four one-line methods.
Singletons?	awesome-low-level-design makes `MusicLibrary`, `UserManager` and `MusicRecommender` Singletons. We inject single instances instead — same sharing, easier tests.	Global `getInstance()`.

Java implementation

Files: `Catalog.java` (genre, artist, album, song, library, plans, user, playlist), `Player.java` (play-order strategies, repeat/state enums, listener, player, recommender), `Main.java`.

```java
// File: lld/ch19/music/Catalog.java
package lld.ch19.music;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.function.Function;

enum Genre { ROCK, POP, JAZZ, CLASSICAL, HIPHOP }

record Artist(String id, String name) { }

record Album(String id, String title, Artist artist, int year) { }

record Song(String id, String title, Artist artist, Album album, Genre genre, int durationSec) {
    @Override public String toString() { return title + " - " + artist.name(); }
}

/** The shared, read-mostly catalog. Thread-safe collections because every user reads it at once. */
final class MusicLibrary {
    private final Map<String, Song> songs = new ConcurrentHashMap<>();
    private final Map<String, Album> albums = new ConcurrentHashMap<>();
    private final Map<String, Artist> artists = new ConcurrentHashMap<>();

    void add(Song s) {
        songs.put(s.id(), s);
        albums.putIfAbsent(s.album().id(), s.album());
        artists.putIfAbsent(s.artist().id(), s.artist());
    }

    Song song(String id) { return songs.get(id); }
    List<Song> allSongs() { return sorted(songs.values(), Song::id); }

    List<Song> searchSongs(String q) { return match(songs.values(), q, Song::title); }
    List<Album> searchAlbums(String q) { return match(albums.values(), q, Album::title); }
    List<Artist> searchArtists(String q) { return match(artists.values(), q, Artist::name); }
    List<Song> byArtist(Artist a) { return songs.values().stream().filter(s -> s.artist().equals(a)).sorted(
            (x, y) -> x.id().compareTo(y.id())).toList(); }

    private static <T> List<T> match(Iterable<T> items, String q, Function<T, String> field) {
        String needle = q.toLowerCase(Locale.ROOT);
        List<T> out = new ArrayList<>();
        for (T t : items) if (field.apply(t).toLowerCase(Locale.ROOT).contains(needle)) out.add(t);
        out.sort((a, b) -> field.apply(a).compareTo(field.apply(b)));
        return out;
    }

    private static <T> List<T> sorted(Iterable<T> items, Function<T, String> key) {
        List<T> out = new ArrayList<>();
        items.forEach(out::add);
        out.sort((a, b) -> key.apply(a).compareTo(key.apply(b)));
        return out;
    }
}

/** What each plan allows. The enum carries the rules, so no if (plan == FREE) scattered around. */
enum SubscriptionPlan {
    FREE(new BigDecimal("0.00"), 6, true, false),
    PREMIUM(new BigDecimal("10.99"), Integer.MAX_VALUE, false, true);

    final BigDecimal monthlyPrice;
    final int skipsPerHour;
    final boolean ads;
    final boolean offlineDownloads;

    SubscriptionPlan(BigDecimal price, int skipsPerHour, boolean ads, boolean offline) {
        this.monthlyPrice = price; this.skipsPerHour = skipsPerHour; this.ads = ads; this.offlineDownloads = offline;
    }
```

```
}

final class User {
    private final String id;
    private final String name;
    private volatile SubscriptionPlan plan = SubscriptionPlan.FREE;
    private final List<Playlist> playlists = new CopyOnWriteArrayList<>();
    private final List<Song> history = new CopyOnWriteArrayList<>();

    User(String id, String name) { this.id = id; this.name = name; }
    String id() { return id; }
    String name() { return name; }
    SubscriptionPlan plan() { return plan; }

    /** Returns the amount to charge now. */
    BigDecimal subscribe(SubscriptionPlan p) { plan = p; return p.monthlyPrice; }

    Playlist createPlaylist(String name) { Playlist p = new Playlist(name, this); playlists.add(p); return p; }
    List<Playlist> playlists() { return List.copyOf(playlists); }
    void recordPlay(Song s) { history.add(s); }
    List<Song> history() { return Collections.unmodifiableList(history); }
}

/** An ordered, user-owned list of songs. Duplicates are allowed, as in real apps. */
final class Playlist {
    private final String name;
    private final User owner;
    private final List<Song> songs = new ArrayList<>();

    Playlist(String name, User owner) { this.name = name; this.owner = owner; }
    String name() { return name; }
    User owner() { return owner; }
    synchronized Playlist add(Song s) { songs.add(s); return this; }
    synchronized void remove(Song s) { songs.remove(s); }
    synchronized void move(int from, int to) { songs.add(to, songs.remove(from)); }
    synchronized List<Song> songs() { return List.copyOf(songs); }
}
```

```java
// File: lld/ch19/music/Main.java
package lld.ch19.music;

import java.time.Instant;
import java.time.InstantSource;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        MusicLibrary lib = new MusicLibrary();
        Artist queen = new Artist("AR1", "Queen");
        Artist miles = new Artist("AR2", "Miles Davis");
        Artist adele = new Artist("AR3", "Adele");
        Album opera = new Album("AL1", "A Night at the Opera", queen, 1975);
        Album blue = new Album("AL2", "Kind of Blue", miles, 1959);
        Album twentyFive = new Album("AL3", "25", adele, 2015);
        lib.add(new Song("S1", "Bohemian Rhapsody", queen, opera, Genre.ROCK, 354));
        lib.add(new Song("S2", "Love of My Life", queen, opera, Genre.ROCK, 219));
        lib.add(new Song("S3", "So What", miles, blue, Genre.JAZZ, 562));
        lib.add(new Song("S4", "Blue in Green", miles, blue, Genre.JAZZ, 337));
        lib.add(new Song("S5", "Hello", adele, twentyFive, Genre.POP, 295));
        lib.add(new Song("S6", "Death on Two Legs", queen, opera, Genre.ROCK, 223));
        lib.add(new Song("S7", "Freddie Freeloader", miles, blue, Genre.JAZZ, 589));
        lib.add(new Song("S8", "All Blues", miles, blue, Genre.JAZZ, 693));
        lib.add(new Song("S9", "When We Were Young", adele, twentyFive, Genre.POP, 291));
        lib.add(new Song("S10", "You're My Best Friend", queen, opera, Genre.ROCK, 172));

        System.out.println("== 1. Search ==");
        System.out.println("  songs 'blue'    -> " + lib.searchSongs("blue"));
        System.out.println("  albums 'opera'  -> " + lib.searchAlbums("opera").stream().map(Album::title).toList());
        System.out.println("  artists 'da'    -> " + lib.searchArtists("da").stream().map(Artist::name).toList());

        System.out.println("\n== 2. Playlist ==");
        User mia = new User("U1", "Mia");
        Playlist mix = mia.createPlaylist("Road trip")
                .add(lib.song("S1")).add(lib.song("S3")).add(lib.song("S5")).add(lib.song("S2")).add(lib.song("S4"));
        mix.move(4, 1);
        System.out.println("  " + mix.name() + ": " + mix.songs());

        System.out.println("\n== 3. Play in order, pause, seek, repeat-one ==");
        MusicPlayer player = new MusicPlayer(mia, InstantSource.fixed(Instant.parse("2026-09-28T08:00:00Z")));
        player.addListener((u, s) -> u.recordPlay(s));          // history is just another listener
        player.load(mix.songs());
        player.play();
        player.seek(120);
        player.pause();
        System.out.println("  state " + player.state() + " at " + player.position() + "s");
        player.resume();
        player.setRepeat(RepeatMode.ONE);
        player.trackFinished();                                 // replays the same song
        player.setRepeat(RepeatMode.OFF);
        player.trackFinished();

        System.out.println("\n== 4. Shuffle keeps the current song, reorders the rest ==");
        player.setShuffle(true);
        System.out.println("  now " + player.current() + ", up next " + player.upNext(4));

        System.out.println("\n== 5. FREE plan: skip limit (6/hour) and ads, repeat-all wraps the shuffled queue ==");
        player.setRepeat(RepeatMode.ALL);
        for (int i = 0; i < 7; i++) player.skip();

        System.out.println("\n== 6. Upgrade to PREMIUM: no ads, unlimited skips ==");
        System.out.println("  charged $" + mia.subscribe(SubscriptionPlan.PREMIUM));
        player.setShuffle(false);
        player.load(List.of(lib.song("S6"), lib.song("S7")));
        player.setRepeat(RepeatMode.ALL);
        player.play();
        player.skip();
        player.skip();                                          // wraps to the start
        player.setRepeat(RepeatMode.OFF);
        player.trackFinished();
        player.trackFinished();                                 // end of queue
```

```
        System.out.println("\n== 7. Recommendations from listening history ==");
        System.out.println("  plays so far: " + mia.history().size());
        System.out.println("  recommended: " + new GenreRecommender(lib).recommend(mia, 3));
    }
}
```

```java
// File: lld/ch19/music/Player.java
package lld.ch19.music;

import java.time.Duration;
import java.time.Instant;
import java.time.InstantSource;
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Deque;
import java.util.EnumMap;
import java.util.List;
import java.util.Map;
import java.util.Random;
import java.util.stream.IntStream;

/** Strategy: in what order should the queue be played? */
interface PlaybackOrder {
    /** A permutation of 0..size-1 that starts with {@code first}. */
    List<Integer> order(int size, int first);
}

final class SequentialOrder implements PlaybackOrder {
    @Override public List<Integer> order(int size, int first) {
        List<Integer> out = new ArrayList<>(IntStream.range(0, size).boxed().toList());
        Collections.rotate(out, -first);        // start at the current track, keep the rest in order
        return out;
    }
}

final class ShuffleOrder implements PlaybackOrder {
    private final Random random;
    ShuffleOrder(long seed) { this.random = new Random(seed); }   // seeded: repeatable demo and tests
    @Override public List<Integer> order(int size, int first) {
        List<Integer> rest = new ArrayList<>(IntStream.range(0, size).filter(i -> i != first).boxed().toList());
        Collections.shuffle(rest, random);
        rest.add(0, first);                     // the song you are hearing keeps playing
        return rest;
    }
}

enum RepeatMode { OFF, ALL, ONE }

enum PlayerState { STOPPED, PLAYING, PAUSED }

/** Observer hook: history, royalties, recommendations all listen to plays. */
interface PlaybackListener { void onStart(User user, Song song); }

/**
 * One user's player. Not shared between users, but a phone and a web tab could send
 * commands at once, so every public method is synchronized.
 */
final class MusicPlayer {
    private final User user;
    private final InstantSource clock;
    private final List<PlaybackListener> listeners = new ArrayList<>();
    private final Deque<Instant> recentSkips = new ArrayDeque<>();
    private List<Song> queue = List.of();
    private List<Integer> order = List.of();
    private int cursor;                         // index into order
    private int positionSec;
    private int songsSinceAd;
    private PlayerState state = PlayerState.STOPPED;
    private RepeatMode repeat = RepeatMode.OFF;
    private PlaybackOrder playbackOrder = new SequentialOrder();

    MusicPlayer(User user, InstantSource clock) { this.user = user; this.clock = clock; }

    synchronized void addListener(PlaybackListener l) { listeners.add(l); }

    synchronized void load(List<Song> songs) {
        queue = List.copyOf(songs);
        order = playbackOrder.order(queue.size(), 0);
```

```
        cursor = 0;
        state = PlayerState.STOPPED;
    }

    synchronized void play() {
        if (queue.isEmpty()) throw new IllegalStateException("queue is empty");
        start();
    }

    synchronized void pause() { if (state == PlayerState.PLAYING) state = PlayerState.PAUSED; }

    synchronized void resume() { if (state == PlayerState.PAUSED) state = PlayerState.PLAYING; }

    synchronized void seek(int sec) {
        positionSec = Math.max(0, Math.min(sec, current().durationSec()));
    }

    /** User pressed "next". Free users have a skip limit per rolling hour. */
    synchronized boolean skip() {
        if (state == PlayerState.STOPPED) return false;          // nothing to skip
        Instant now = clock.instant();
        while (!recentSkips.isEmpty() && recentSkips.peekFirst().isBefore(now.minus(Duration.ofHours(1))))
recentSkips.pollFirst();
        if (recentSkips.size() >= user.plan().skipsPerHour) {
            System.out.println("  [player] skip limit reached on " + user.plan() + " plan");
            return false;
        }
        recentSkips.addLast(now);
        advance(false);
        return true;
    }

    /** The current song ended naturally. Repeat-one replays it; skip does not honour repeat-one. */
    synchronized void trackFinished() { advance(true); }

    synchronized void previous() {
        if (positionSec > 3 || cursor == 0) { positionSec = 0; return; }   // like most players: restart first
        cursor--;
        start();
    }

    synchronized void setShuffle(boolean on) {
        int now = order.isEmpty() ? 0 : order.get(cursor);
        playbackOrder = on ? new ShuffleOrder(42) : new SequentialOrder();
        order = playbackOrder.order(queue.size(), now);
        cursor = 0;
    }

    synchronized void setRepeat(RepeatMode m) { repeat = m; }

    synchronized Song current() { return queue.get(order.get(cursor)); }
    synchronized PlayerState state() { return state; }
    synchronized int position() { return positionSec; }

    /** The next few songs in play order (what the "Up next" panel shows). */
    synchronized List<Song> upNext(int n) {
        List<Song> out = new ArrayList<>();
        for (int i = cursor + 1; i < order.size() && out.size() < n; i++) out.add(queue.get(order.get(i)));
        return out;
    }

    private void advance(boolean natural) {
        if (natural && repeat == RepeatMode.ONE) { start(); return; }
        if (cursor + 1 < order.size()) { cursor++; start(); return; }
        if (repeat == RepeatMode.ALL || repeat == RepeatMode.ONE) {   // wrap around (reshuffle if shuffling)
            order = playbackOrder.order(queue.size(), order.get(0));
            cursor = 0;
            start();
            return;
        }
        state = PlayerState.STOPPED;
        System.out.println("  [player] end of queue");
    }
```

```
    private void start() {
        if (user.plan().ads && ++songsSinceAd > 3) {
            System.out.println("  [player] (30-second ad for FREE plan)");
            songsSinceAd = 1;
        }
        positionSec = 0;
        state = PlayerState.PLAYING;
        Song s = current();
        System.out.println("  [player] > " + s);
        listeners.forEach(l -> l.onStart(user, s));
    }
}

/** Strategy for "what should this user hear next?" - a simple stub based on favourite genre. */
interface Recommender { List<Song> recommend(User user, int n); }

final class GenreRecommender implements Recommender {
    private final MusicLibrary library;
    GenreRecommender(MusicLibrary library) { this.library = library; }

    @Override public List<Song> recommend(User user, int n) {
        Map<Genre, Integer> counts = new EnumMap<>(Genre.class);
        user.history().forEach(s -> counts.merge(s.genre(), 1, Integer::sum));
        List<Genre> favourite = counts.entrySet().stream()
                .sorted((a, b) -> b.getValue() - a.getValue()).map(Map.Entry::getKey).toList();
        List<Song> heard = user.history();
        List<Song> out = new ArrayList<>();
        for (Genre g : favourite)
            for (Song s : library.allSongs())
                if (out.size() < n && s.genre() == g && !heard.contains(s)) out.add(s);
        return out;
    }
}
```

Output:

```
== 1. Search ==
  songs 'blue'    -> [All Blues - Miles Davis, Blue in Green - Miles Davis]
  albums 'opera'  -> [A Night at the Opera]
  artists 'da'    -> [Miles Davis]

== 2. Playlist ==
  Road trip: [Bohemian Rhapsody - Queen, Blue in Green - Miles Davis, So What - Miles Davis, Hello - Adele, Love of My
Life - Queen]

== 3. Play in order, pause, seek, repeat-one ==
  [player] > Bohemian Rhapsody - Queen
  state PAUSED at 120s
  [player] > Bohemian Rhapsody - Queen
  [player] > Blue in Green - Miles Davis

== 4. Shuffle keeps the current song, reorders the rest ==
  now Blue in Green - Miles Davis, up next [Love of My Life - Queen, So What - Miles Davis, Bohemian Rhapsody - Queen,
Hello - Adele]

== 5. FREE plan: skip limit (6/hour) and ads, repeat-all wraps the shuffled queue ==
  [player] (30-second ad for FREE plan)
  [player] > Love of My Life - Queen
  [player] > So What - Miles Davis
  [player] > Bohemian Rhapsody - Queen
  [player] (30-second ad for FREE plan)
  [player] > Hello - Adele
  [player] > Blue in Green - Miles Davis
  [player] > Hello - Adele
  [player] skip limit reached on FREE plan

== 6. Upgrade to PREMIUM: no ads, unlimited skips ==
  charged $10.99
  [player] > Death on Two Legs - Queen
  [player] > Freddie Freeloader - Miles Davis
  [player] > Death on Two Legs - Queen
  [player] > Freddie Freeloader - Miles Davis
  [player] end of queue

== 7. Recommendations from listening history ==
  plays so far: 13
  recommended: [You're My Best Friend - Queen, All Blues - Miles Davis, When We Were Young - Adele]
```

Reading the output. Step 3: repeat-ONE replays "Bohemian Rhapsody" when it ends naturally. Step 4: shuffle keeps "Blue in Green" playing and reorders the rest. Step 5: on the FREE plan an ad plays after every three songs, repeat-ALL wraps and reshuffles the queue, and the seventh skip within the hour is refused. Step 6: PREMIUM has no ads; with repeat-ALL the second skip wraps to the start; with repeat OFF the queue ends. Step 7: the recommender suggests songs the user has **not** heard, from the genres they played most.

Concurrency & edge cases

Case	Handling
Two devices send "next" at the same instant	Player methods are `synchronized`; the two skips apply one after the other.
Catalog updated while users search	`ConcurrentHashMap` (a thread-safe hash map) for songs/albums/artists; search iterates a weakly consistent view (it may or may not show changes made during the loop, but never breaks).
Playlist edited while it is playing	`load` copies the playlist (`List.copyOf`), so the running queue is a snapshot; a real app would sync edits into "up next".
Seek past the end or before 0	Clamped to `[0, duration]`.
Previous after 3 seconds	Restarts the current song (common player behaviour); within 3 seconds goes back one song.
Skip when stopped	Refused (and does not count against the limit).
Plan changes mid-session	`plan` is `volatile` (a Java keyword that makes every thread see the latest written value); the next skip or ad check sees it.
Empty queue	`play()` throws `IllegalStateException`.

Real-world example

Spotify's free tier has historically limited skips (for example, six per hour on mobile) and inserts ads between songs, while Premium removes both and allows downloads — the rules our `SubscriptionPlan` enum carries. Spotify engineers also wrote (2014) that users complained a truly random shuffle "felt" non-random when the same artist came up twice in a row, so they changed the algorithm to spread songs by the same artist across the list. That is exactly the kind of change the `PlaybackOrder` strategy absorbs: a new `ArtistSpreadShuffle` class, no player changes. Personalised playlists such as Discover Weekly are the industrial version of our `Recommender` stub.

Extensions & follow-up questions

- **"Add 'play next' and 'add to queue'."** Keep a separate *user queue* that is consumed before the context (playlist/album) order.
- **"Offline mode."** `SubscriptionPlan.offlineDownloads`; a `DownloadManager` stores encrypted files; the player chooses a local or remote `AudioSource` (Strategy).
- **"Social sharing / collaborative playlists."** Playlist gets a collaborator set and an edit log; listeners (Observer) push changes to other users.
- **"Crossfade, gapless playback."** Player behaviour around `trackFinished`; a `TransitionStrategy`.
- **"Royalties."** A `PlaybackListener` that records a play once the user has listened for a threshold time.
- **"Search at scale."** Inverted index (a map from each word to the items containing it) / search service with prefix (typeahead — suggestions as you type) and fuzzy matching (tolerating typos); the `MusicLibrary` interface stays.
- **"Smart shuffle."** New `PlaybackOrder` that spaces out songs by the same artist.

In the interview

30–45 minute plan

1. Clarify controls, plans and scope (skip audio bytes) — 4 min.
2. Catalog entities + library search — 6 min.
3. User, playlist, subscription plan — 5 min.
4. **Player**: queue + order + cursor; state machine; shuffle/repeat as strategy/enum — 15 min.
5. Observer for history; recommender strategy; concurrency; extensions — 10 min.

> **30-second answer:** "Immutable catalog records in a thread-safe library. Each user has a player session holding the queue, a play order produced by a `PlaybackOrder` strategy, a cursor and a repeat mode.

> Shuffle rebuilds the order around the current song. Plan limits live in the plan enum. A playback listener records history, which feeds a pluggable recommender."

Follow-ups with answers

- *How does shuffle avoid repeating a song before all have played?* It is a permutation (each index once), not "pick a random next song".
- *Where do ads fit?* The player asks the plan whether ads apply and inserts one every N songs; in production an ad service picks which ad.
- *How would you implement "previous" in shuffle mode?* Walk back along the same order list (the cursor), so previous is the song you actually heard before.

Self-check

1. **Q:** Why store the play order as indexes rather than reordering the queue? **A:** So switching shuffle off restores the original order and nothing is lost.
2. **Q:** What does repeat-ONE do when the user presses Next? **A:** Moves to the next song; it only replays on natural end.
3. **Q:** Which pattern lets you add a smarter shuffle without touching the player? **A:** Strategy (`PlaybackOrder`).
4. **Q:** How is "6 skips per hour" enforced? **A:** A deque of skip times; drop entries older than one hour; refuse if the size has reached the limit.
5. **Q:** Why is the player's state an enum rather than the State pattern? **A:** The states have almost no distinct behaviour; the pattern would add classes without benefit.

TL;DR

> - Catalog = immutable records in a concurrent library.
> - Player = **queue + order (Strategy) + cursor + repeat mode + state**; shuffle keeps the current song.
> - Plan rules in the **enum**; plays published via **Observer**; recommendations via **Strategy**.

Design an Online Food Delivery Service (Swiggy) + FoodKart

This problem also comes in two shapes:

- **Part A — Swiggy-style delivery platform** (the awesome-low-level-design problem): restaurants with menus, a cart, orders with tracking states, payments, and **assigning a delivery rider** by a pluggable strategy. We add **kitchen capacity** (a restaurant can only cook so many items at once), the key twist in a commonly reported Flipkart food-ordering variant (it is not part of the FoodKart statement below).
- **Part B — FoodKart** (Flipkart machine-coding round, from kumaransg/LLD): a smaller, command-driven problem — users and pincodes (Indian postal codes), one-dish restaurants, ratings, `show_restaurant` sorted by rating or price, `place_order` — plus a **restaurant selection strategy** (highest rating / lowest price) filtered by remaining capacity.

Problem & clarifying questions

Question	Assumed answer
Can one cart mix restaurants?	No — one restaurant per cart (as in Swiggy/Zomato).
Who confirms an order?	The restaurant; it may refuse if its kitchen is full.
When is capacity freed?	When the food is ready (handed over), not when it is delivered.
How is a rider chosen?	By a strategy: nearest to the restaurant, or fair share (fewest deliveries today).
What if no rider is free?	The order waits in a queue; the next freed rider takes the oldest waiting order.
Can a customer cancel?	Only before the kitchen starts cooking (PENDING or CONFIRMED).
Payment types?	UPI, card, cash on delivery (COD: nothing to refund).
Live GPS, ETA (estimated time of arrival) prediction, batching two orders per rider, surge fees (higher fees at busy times)?	Extensions.

Requirements

Functional (Part A)

1. Customers browse restaurants, view menus and place orders.
2. Restaurants manage menus, prices and availability.
3. Delivery agents accept and fulfil orders.
4. Customers track order status and get notified of changes.
5. Several payment methods.
6. Real-time notifications to customers, restaurants and delivery agents.

Functional (Part B — FoodKart)

7. `register_user(name, gender, phone, pincode)` — phone is unique.
8. `login_user(phone)` — sets the context; logging in someone else logs out the previous user.
9. `register_restaurant(name, pincodes "A/B", dish, price, quantity)` — in the context of the logged-in user (the owner). A restaurant serves **one** dish to **many** pincodes.
10. `update_quantity(name, add)`, `update_location(name, pincodes)` — owner only.
11. `rate_restaurant(name, 1..5, optional comment)`; rating = average of all ratings.
12. `show_restaurant(rating|price)` — **serviceable** restaurants (deliver to my pincode **and** quantity > 0), sorted **descending** by the chosen field.
13. `place_order(name, qty)`; bonus: order history.
14. *(Variant)* place an order without naming a restaurant: pick one by a **selection strategy**, among those that deliver here **and** have capacity for the whole quantity.

Non-functional

- Handle many concurrent orders without double-booking kitchen capacity or riders.
- Keep data consistent (no negative stock, no rider on two orders).
- In-memory only for the machine-coding round; demoable driver (a small `Main` program that feeds in commands and prints results); graceful failures.

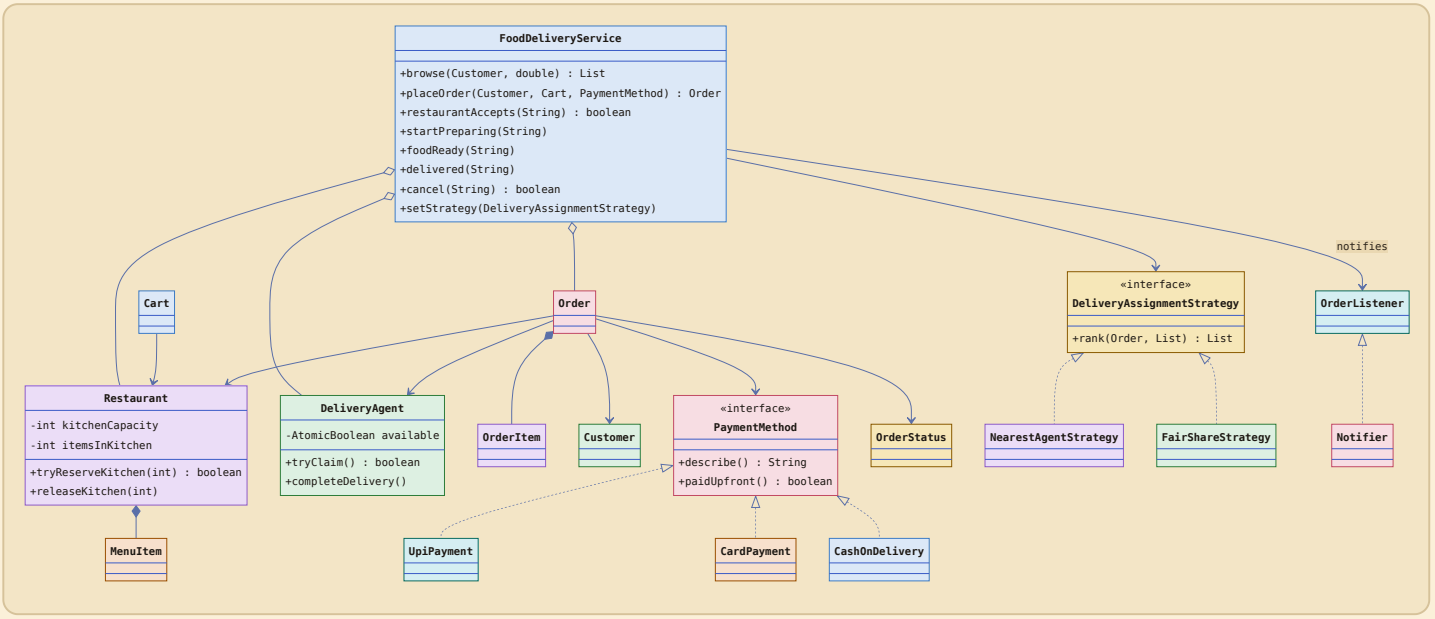
Use cases

Actor	Use cases
Customer	Browse nearby restaurants, build cart, pay, place order, track, cancel, rate
Restaurant	Manage menu/prices/availability, accept or refuse order, mark preparing, mark ready
Delivery agent	Get assigned, pick up, deliver
System	Compute fees, assign riders, queue orders when no rider is free, notify everyone

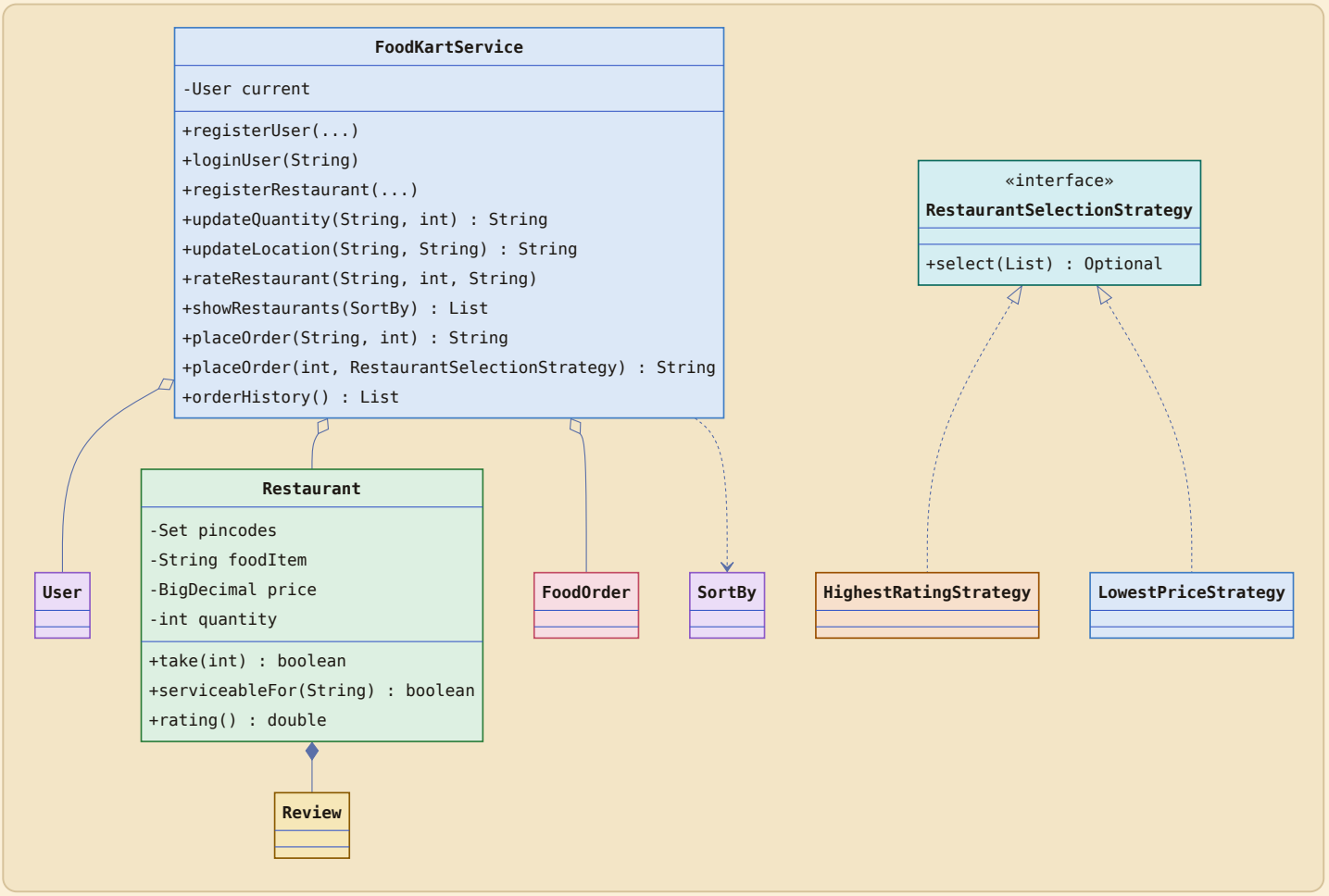
Core entities

Class	Responsibility
`Location`	A point; `distanceTo`.
`Customer`	id, name, phone, address.
`MenuItem`	Name, price, available flag (both changeable by the restaurant).
`Restaurant`	Menu, open flag, **kitchen capacity** (max items cooking at once), `tryReserveKitchen` / `releaseKitchen`.
`Cart`	One restaurant + item quantities; refuses items from a second restaurant.
`OrderItem`	Snapshot of item name, **unit price at order time**, quantity.
`OrderStatus`	State machine: PENDING → CONFIRMED → PREPARING → READY_FOR_PICKUP → OUT_FOR_DELIVERY → DELIVERED, or CANCELLED.
`Order`	Customer, restaurant, items, delivery fee, payment, status history, assigned agent.
`PaymentMethod` → `UpiPayment`, `CardPayment`, `CashOnDelivery`	Sealed interface; COD is not paid upfront.
`DeliveryAgent`	Location, `AtomicBoolean available` (a true/false flag that can be checked-and-changed atomically), deliveries today; `tryClaim()` is a CAS.
`DeliveryAssignmentStrategy` → `NearestAgentStrategy`, `FairShareStrategy`	Ranks free riders for an order.
`OrderListener` / `Notifier`	Observer for status changes.
`FoodDeliveryService`	Facade: browse, place, accept, prepare, ready, deliver, cancel; the waiting-for-rider queue.
(FoodKart) `User`, `Restaurant`, `Review`, `FoodOrder`, `SortBy`, `RestaurantSelectionStrategy`, `FoodKartService`	The machine-coding model; `Restaurant.take(q)` is the atomic capacity check.

Class diagram

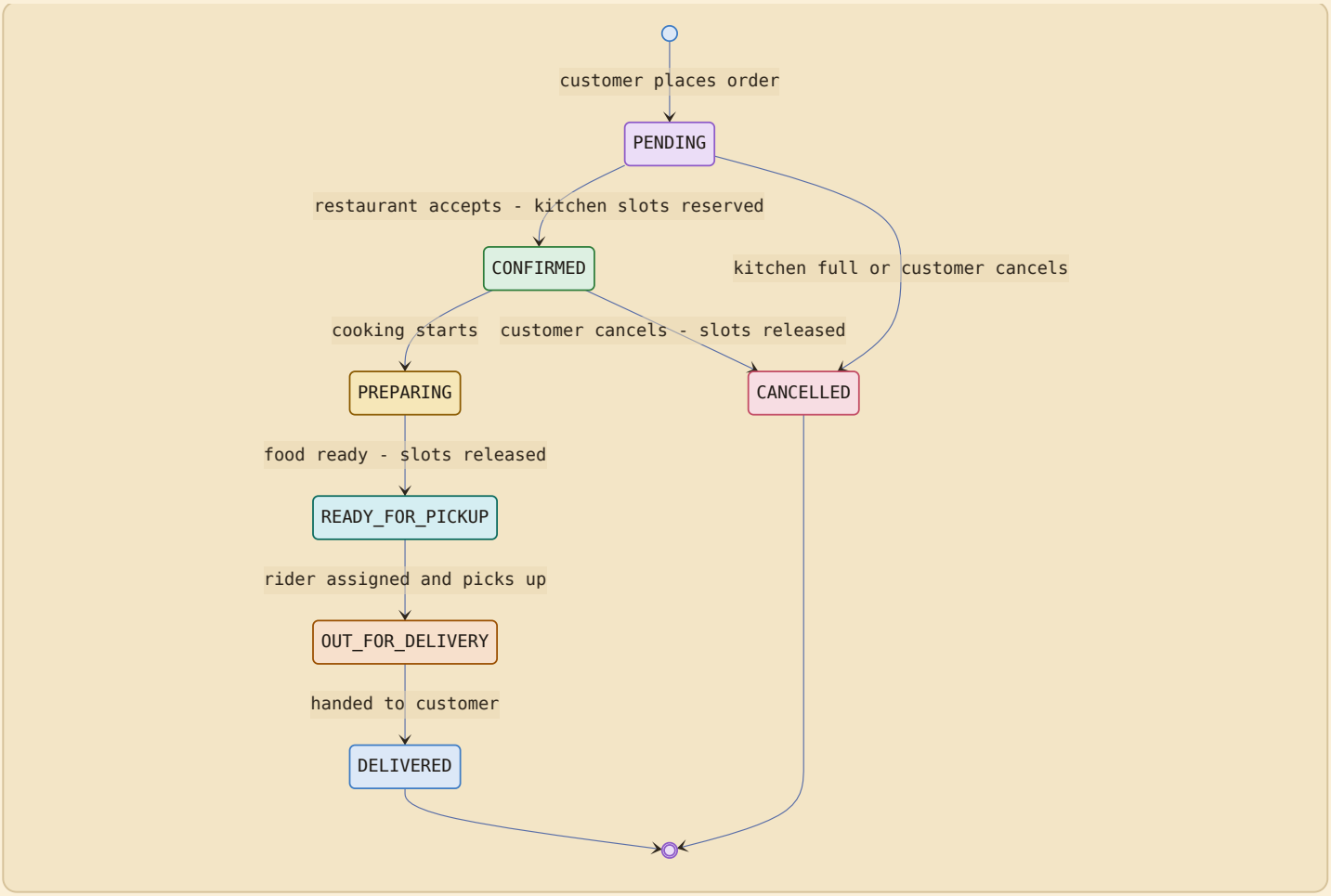


FoodKart (Part B):



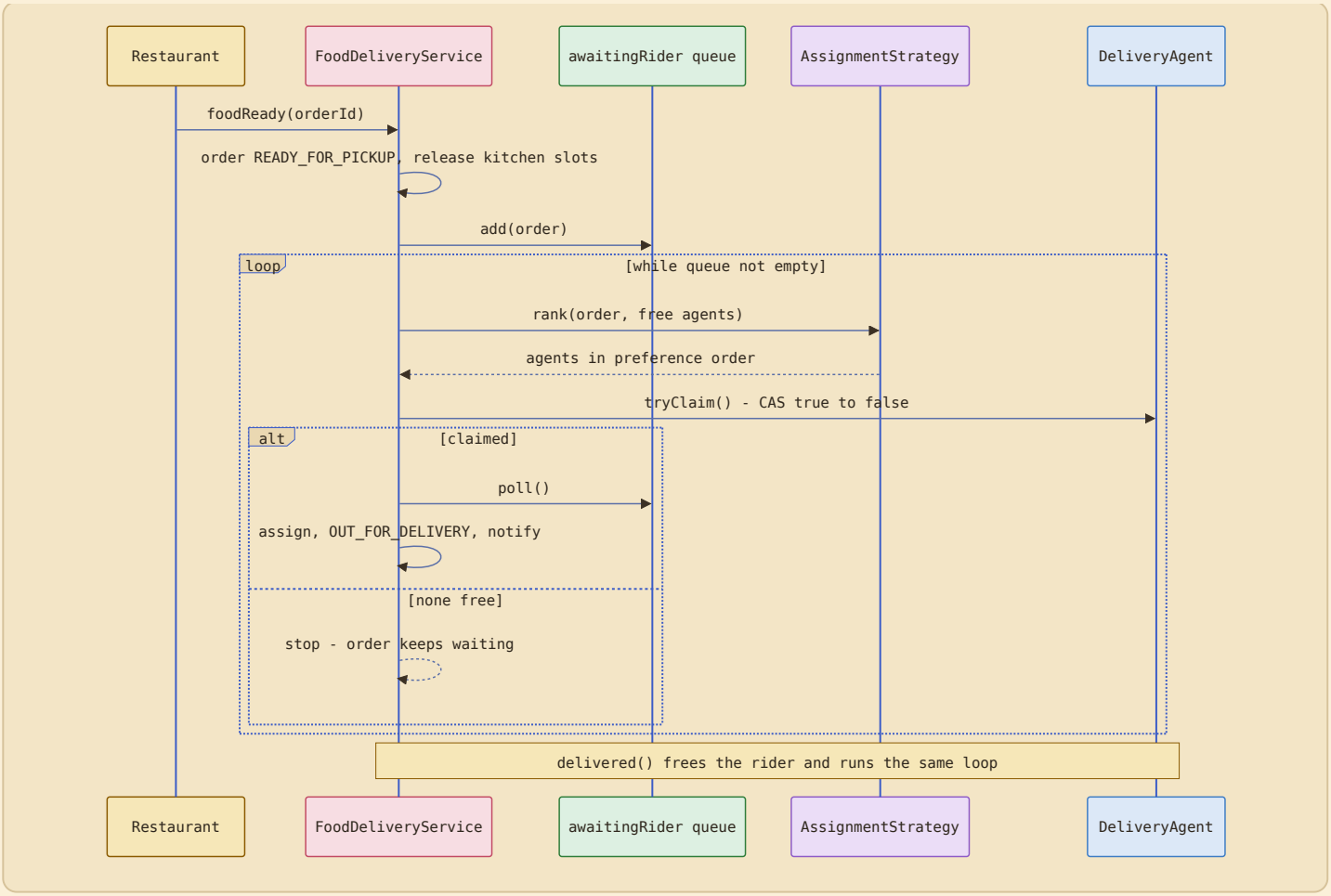
Key flows

Order tracking state machine



(Real apps often split `READY_FOR_PICKUP → OUT_FOR_DELIVERY` into "rider assigned", "arrived at restaurant", "picked up". Adding them is a local enum change.)

Food ready → rider assignment



FoodKart `place_order(name, qty)`: user logged in? → restaurant exists? → delivers to the user's pincode? → `take(qty)` atomically (quantity ≥ qty, then subtract) → record in history → "Order Placed Successfully." Otherwise "Cannot place order".

FoodKart selection variant: filter restaurants that deliver to the user's pincode and have `quantity ≥ qty` → ask the strategy to pick one → `take(qty)`; if another user took the capacity in between, drop that restaurant and ask again (bounded retries).

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for rider assignment (`rank` returns a preference list)	Nearest vs fair-share vs rating-based are swappable at run time (`setStrategy`, step 7). Returning a *ranked list* lets the service try the next rider if the first was just taken (the free list is read before claiming, so it can be stale).	Returning one rider (fails if that rider was claimed a moment earlier).
CAS on the rider (`AtomicBoolean.compareAndSet(true,false)`)	The rider guards itself: whoever calls `tryClaim()` — this dispatcher, a second dispatcher instance, or a future "rider accepts" button — exactly one `compareAndSet` succeeds. In this code `dispatchWaiting` is also `synchronized` (to drain the queue in FIFO order), so the CAS is the safety net that keeps working even when claims come from outside that lock.	Check `isAvailable()` then set (a race as soon as two callers exist).
Kitchen capacity reserved on accept, released on ready	Models the rule "a restaurant cannot accept more items while its kitchen is full" from the capacity-based food-ordering variant.	Unlimited orders per restaurant.
Waiting queue for riders	No free rider is normal at peak; FIFO keeps it fair; freeing a rider re-runs dispatch.	Failing the order.
State machine in `OrderStatus`	Tracking states and legal moves in one place; cancel after cooking starts is impossible.	Scattered `if`s.
`OrderItem` copies the price	A later menu price change (step 3) cannot alter a placed order.	Referencing `MenuItem` directly.
Sealed `PaymentMethod` + records	A closed, known set; COD overrides `paidUpfront()` so refunds skip it.	`String paymentType`.
Observer (`OrderListener`)	Customer app, restaurant tablet and rider app all react to the same events.	The service printing/sending messages itself.
One restaurant per cart	Matches real apps; a single kitchen, single pickup, single fee.	Multi-restaurant carts (needs split orders).
FoodKart: `SortBy` enum holds its comparator (an object that decides which of two items comes first)	`show_restaurant(rating	price)` becomes `sorted(sortBy.comparator)`; both descending as the statement says; name as tie-breaker for stable output.

Decision	Why	Alternative rejected
FoodKart: owner recorded at registration	The statement says restaurants are registered "in the context of the logged-in user"; only that owner may update quantity or location.	Anyone updating any restaurant.
FoodKart: `Restaurant.take(q)` synchronized	"Check quantity then subtract" is atomic — two users cannot both take the last plates.	Separate `getQuantity()` + `setQuantity()`.

Java implementation

Part A — Food delivery platform (`lld.ch19.fooddelivery`)

Files: `Model.java` (location, customer, menu item, restaurant, cart, order item, status machine, payments, order, delivery agent), `FoodDeliveryService.java` (strategies, listener, notifier, service), `Main.java`.

```java
// File: lld/ch19/fooddelivery/FoodDeliveryService.java
package lld.ch19.fooddelivery;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.Queue;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicInteger;

/** Strategy: which free rider gets this order? Returns riders in preference order. */
interface DeliveryAssignmentStrategy {
    List<DeliveryAgent> rank(Order order, List<DeliveryAgent> free);
}

/** Closest rider to the restaurant - fastest pickup. */
final class NearestAgentStrategy implements DeliveryAssignmentStrategy {
    @Override public List<DeliveryAgent> rank(Order o, List<DeliveryAgent> free) {
        Location r = o.restaurant().location();
        return free.stream().sorted(Comparator.comparingDouble(a -> a.location().distanceTo(r))).toList();
    }
}

/** Fewest deliveries today first (spreads earnings), nearest as tie-breaker. */
final class FairShareStrategy implements DeliveryAssignmentStrategy {
    @Override public List<DeliveryAgent> rank(Order o, List<DeliveryAgent> free) {
        Location r = o.restaurant().location();
        return free.stream().sorted(Comparator.comparingInt(DeliveryAgent::deliveriesToday)
                .thenComparingDouble(a -> a.location().distanceTo(r))).toList();
    }
}

/** Observer: customer app, restaurant tablet and rider app all track the order. */
interface OrderListener { void onStatus(Order order, OrderStatus status); }

final class Notifier implements OrderListener {
    @Override public void onStatus(Order o, OrderStatus s) {
        String who = switch (s) {
            case PENDING -> "restaurant " + o.restaurant().name() + ": new order";
            case OUT_FOR_DELIVERY -> "customer " + o.customer().name() + ": " +
o.agent().map(DeliveryAgent::name).orElse("?") + " is on the way";
            default -> "customer " + o.customer().name();
        };
        System.out.println("    [" + o.id() + " " + s + "] -> " + who);
    }
}

/** Facade: browse, order, accept, prepare, dispatch, deliver, cancel. */
final class FoodDeliveryService {
    private static final BigDecimal BASE_FEE = new BigDecimal("20.00");
    private static final BigDecimal PER_KM = new BigDecimal("8.00");

    private final Map<String, Restaurant> restaurants = new ConcurrentHashMap<>();
    private final List<DeliveryAgent> agents = new CopyOnWriteArrayList<>();
    private final Map<String, Order> orders = new ConcurrentHashMap<>();
    private final Queue<Order> awaitingRider = new ConcurrentLinkedQueue<>();
    private final List<OrderListener> listeners = new CopyOnWriteArrayList<>();
    private final AtomicInteger seq = new AtomicInteger(100);
    private volatile DeliveryAssignmentStrategy strategy;

    FoodDeliveryService(DeliveryAssignmentStrategy strategy) { this.strategy = strategy; }

    void setStrategy(DeliveryAssignmentStrategy s) { strategy = s; }
    void subscribe(OrderListener l) { listeners.add(l); }
    void addRestaurant(Restaurant r) { restaurants.put(r.id(), r); }
    void addAgent(DeliveryAgent a) { agents.add(a); }
```

```
    /** Open restaurants within the radius, nearest first. */
    List<Restaurant> browse(Customer c, double radiusKm) {
        return restaurants.values().stream().filter(Restaurant::isOpen)
                .filter(r -> r.location().distanceTo(c.address()) <= radiusKm)
                .sorted(Comparator.comparingDouble(r -> r.location().distanceTo(c.address()))).toList();
    }

    Order placeOrder(Customer c, Cart cart, PaymentMethod pay) {
        if (cart.isEmpty()) throw new IllegalStateException("empty cart");
        Restaurant r = cart.restaurant();
        if (!r.isOpen()) throw new IllegalStateException(r.name() + " is closed");
        List<OrderItem> items = new ArrayList<>();
        cart.quantities().forEach((itemId, qty) -> {
            MenuItem m = r.item(itemId).orElseThrow();
            if (!m.available()) throw new IllegalStateException(m.name() + " is sold out");
            items.add(new OrderItem(itemId, m.name(), m.price(), qty));
        });
        BigDecimal km = BigDecimal.valueOf(r.location().distanceTo(c.address())).setScale(1, RoundingMode.HALF_UP);
        BigDecimal fee = BASE_FEE.add(PER_KM.multiply(km)).setScale(2, RoundingMode.HALF_EVEN);
        Order o = new Order("OD" + seq.incrementAndGet(), c, r, items, fee, pay);
        orders.put(o.id(), o);
        cart.clear();
        publish(o, OrderStatus.PENDING);
        return o;
    }

    /** The restaurant accepts only if its kitchen has room; otherwise the order is auto-cancelled. */
    boolean restaurantAccepts(String orderId) {
        Order o = orders.get(orderId);
        synchronized (o) {                              // same lock as cancel(): capacity can never leak
            if (o.status() != OrderStatus.PENDING) return false;
            if (!o.restaurant().tryReserveKitchen(o.itemCount())) {
                System.out.println("    (" + o.restaurant().name() + " kitchen full: needs " + o.itemCount()
                        + " item slots, free " + o.restaurant().freeCapacity() + ")");
                transition(o, OrderStatus.CANCELLED);
                return false;
            }
            transition(o, OrderStatus.CONFIRMED);
            return true;
        }
    }

    void startPreparing(String orderId) { transition(orders.get(orderId), OrderStatus.PREPARING); }

    /** Food is ready: kitchen capacity comes back, and we look for a rider. */
    void foodReady(String orderId) {
        Order o = orders.get(orderId);
        transition(o, OrderStatus.READY_FOR_PICKUP);
        o.restaurant().releaseKitchen(o.itemCount());
        awaitingRider.add(o);
        dispatchWaiting();
    }

    void delivered(String orderId) {
        Order o = orders.get(orderId);
        transition(o, OrderStatus.DELIVERED);
        DeliveryAgent a = o.agent().orElseThrow();
        a.moveTo(o.customer().address());
        a.completeDelivery();
        dispatchWaiting();                              // a freed rider takes the oldest waiting order
    }

    boolean cancel(String orderId) {
        Order o = orders.get(orderId);
        synchronized (o) {
            if (!o.status().canMoveTo(OrderStatus.CANCELLED)) return false;
            boolean kitchenHeld = o.status() == OrderStatus.CONFIRMED;
            transition(o, OrderStatus.CANCELLED);
            if (kitchenHeld) o.restaurant().releaseKitchen(o.itemCount());
        }
        if (o.payment().paidUpfront()) System.out.println("    refund " + o.total() + " to " +
o.payment().describe());
        return true;
    }
```

```
    Order order(String id) { return orders.get(id); }

    /** Hand waiting orders (oldest first) to free riders. CAS on the rider prevents double booking. */
    private synchronized void dispatchWaiting() {
        Order o;
        while ((o = awaitingRider.peek()) != null) {
            List<DeliveryAgent> free = agents.stream().filter(DeliveryAgent::isAvailable).toList();
            Optional<DeliveryAgent> got = Optional.empty();
            for (DeliveryAgent a : strategy.rank(o, free)) if (a.tryClaim()) { got = Optional.of(a); break; }
            if (got.isEmpty()) {
                System.out.println("    (" + o.id() + " waiting for a free rider)");
                return;
            }
            awaitingRider.poll();
            o.assign(got.get());
            transition(o, OrderStatus.OUT_FOR_DELIVERY);
        }
    }

    private void transition(Order o, OrderStatus s) {
        o.moveTo(s);
        listeners.forEach(l -> l.onStatus(o, s));
    }

    private void publish(Order o, OrderStatus s) { listeners.forEach(l -> l.onStatus(o, s)); }
}
```

```java
// File: lld/ch19/fooddelivery/Main.java
package lld.ch19.fooddelivery;

import java.math.BigDecimal;

public class Main {
    public static void main(String[] args) {
        FoodDeliveryService app = new FoodDeliveryService(new NearestAgentStrategy());
        app.subscribe(new Notifier());

        Restaurant dosa = new Restaurant("R1", "Dosa Corner", new Location(1, 1), 4);
        dosa.addItem(new MenuItem("D1", "Masala Dosa", new BigDecimal("120.00")));
        dosa.addItem(new MenuItem("D2", "Filter Coffee", new BigDecimal("40.00")));
        Restaurant pizza = new Restaurant("R2", "Pizza Hub", new Location(5, 5), 10);
        pizza.addItem(new MenuItem("P1", "Margherita", new BigDecimal("299.00")));
        Restaurant far = new Restaurant("R3", "Far Away Grill", new Location(30, 30), 10);
        app.addRestaurant(dosa);
        app.addRestaurant(pizza);
        app.addRestaurant(far);
        DeliveryAgent ravi = new DeliveryAgent("A1", "Ravi", new Location(0, 0));
        DeliveryAgent meena = new DeliveryAgent("A2", "Meena", new Location(6, 6));
        app.addAgent(ravi);
        app.addAgent(meena);

        Customer asha = new Customer("C1", "Asha", "98000", new Location(2, 2));
        Customer vik = new Customer("C2", "Vikram", "98001", new Location(4, 4));

        System.out.println("== 1. Browse within 10 km ==");
        app.browse(asha, 10).forEach(r -> System.out.println("  " + r.name()));

        System.out.println("\n== 2. One restaurant per cart ==");
        Cart cart = new Cart();
        cart.add(dosa, "D1", 2);
        cart.add(dosa, "D2", 1);
        try { cart.add(pizza, "P1", 1); } catch (IllegalStateException e) { System.out.println("  " + e.getMessage());
}

        System.out.println("\n== 3. Place, accept, prepare, ready, deliver ==");
        Order o1 = app.placeOrder(asha, cart, new UpiPayment("asha@upi"));
        System.out.println("  subtotal " + o1.subtotal() + " + delivery " + o1.deliveryFee() + " = " + o1.total());
        dosa.item("D1").orElseThrow().setPrice(new BigDecimal("150.00"));    // later price change...
        System.out.println("  price changed after ordering; order still " + o1.total());
        app.restaurantAccepts(o1.id());
        app.startPreparing(o1.id());
        app.foodReady(o1.id());
        System.out.println("  rider: " + o1.agent().orElseThrow() + " (nearest to Dosa Corner)");

        System.out.println("\n== 4. Kitchen capacity: 4 items max at Dosa Corner ==");
        Cart c2 = new Cart();
        c2.add(dosa, "D1", 3);
        Order o2 = app.placeOrder(vik, c2, new CashOnDelivery());
        app.restaurantAccepts(o2.id());                                    // 3 of 4 slots used
        Cart c3 = new Cart();
        c3.add(dosa, "D2", 2);
        Order o3 = app.placeOrder(asha, c3, new CardPayment("4242"));
        app.restaurantAccepts(o3.id());                                    // needs 2, only 1 free

        System.out.println("\n== 5. Only one rider free: second ready order waits ==");
        app.startPreparing(o2.id());
        app.foodReady(o2.id());                                            // Meena takes it
        Cart c4 = new Cart();
        c4.add(pizza, "P1", 1);
        Order o4 = app.placeOrder(vik, c4, new UpiPayment("vik@upi"));
        app.restaurantAccepts(o4.id());
        app.startPreparing(o4.id());
        app.foodReady(o4.id());                                            // nobody free
        app.delivered(o1.id());                                            // Ravi frees up -> takes o4
        System.out.println("  o4 rider: " + o4.agent().orElseThrow());

        System.out.println("\n== 6. Cancel rules ==");
        System.out.println("  cancel o2 (out for delivery)? " + app.cancel(o2.id()));
        Cart c5 = new Cart();
        c5.add(pizza, "P1", 2);
```

```
        Order o5 = app.placeOrder(asha, c5, new CardPayment("4242"));
        app.restaurantAccepts(o5.id());
        System.out.println("  cancel o5 (confirmed)? " + app.cancel(o5.id()));

        System.out.println("\n== 7. Swap strategy at run time: fair share ==");
        app.delivered(o2.id());
        app.delivered(o4.id());
        System.out.println("  deliveries today: Ravi=" + ravi.deliveriesToday() + ", Meena=" + meena.deliveriesToday()
                + " (both now at Vikram's address)");
        app.setStrategy(new FairShareStrategy());
        Cart c6 = new Cart();
        c6.add(dosa, "D2", 1);
        Order o6 = app.placeOrder(asha, c6, new UpiPayment("asha@upi"));
        app.restaurantAccepts(o6.id());
        app.startPreparing(o6.id());
        app.foodReady(o6.id());
        System.out.println("  o6 rider: " + o6.agent().orElseThrow() + " (fewer deliveries today)");
        System.out.println("  o1 history: " + o1.history());
    }
}
```

```java
// File: lld/ch19/fooddelivery/Model.java
package lld.ch19.fooddelivery;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.Collection;
import java.util.EnumSet;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicBoolean;
import java.util.concurrent.atomic.AtomicInteger;

/** A point on a simple km grid (real systems use lat/long and a road-distance service). */
record Location(double x, double y) {
    double distanceTo(Location o) { return Math.hypot(x - o.x, y - o.y); }
}

record Customer(String id, String name, String phone, Location address) { }

final class MenuItem {
    private final String id;
    private final String name;
    private volatile BigDecimal price;
    private volatile boolean available = true;

    MenuItem(String id, String name, BigDecimal price) { this.id = id; this.name = name; this.price = price; }
    String id() { return id; }
    String name() { return name; }
    BigDecimal price() { return price; }
    boolean available() { return available; }
    void setPrice(BigDecimal p) { price = p; }
    void setAvailable(boolean a) { available = a; }
}

/**
 * A restaurant manages its own menu and has a kitchen capacity: at most N items being
 * prepared at once. Capacity is taken when an order is accepted and returned when the
 * food is ready (dispatched).
 */
final class Restaurant {
    private final String id;
    private final String name;
    private final Location location;
    private final int kitchenCapacity;
    private final Map<String, MenuItem> menu = new ConcurrentHashMap<>();
    private volatile boolean open = true;
    private int itemsInKitchen;                          // guarded by this

    Restaurant(String id, String name, Location location, int kitchenCapacity) {
        this.id = id; this.name = name; this.location = location; this.kitchenCapacity = kitchenCapacity;
    }

    String id() { return id; }
    String name() { return name; }
    Location location() { return location; }
    boolean isOpen() { return open; }
    void setOpen(boolean o) { open = o; }
    void addItem(MenuItem m) { menu.put(m.id(), m); }
    Optional<MenuItem> item(String itemId) { return Optional.ofNullable(menu.get(itemId)); }
    Collection<MenuItem> menu() { return menu.values(); }

    synchronized boolean tryReserveKitchen(int items) {
        if (itemsInKitchen + items > kitchenCapacity) return false;
        itemsInKitchen += items;
        return true;
    }

    synchronized void releaseKitchen(int items) { itemsInKitchen -= items; }
```

```
    synchronized int freeCapacity() { return kitchenCapacity - itemsInKitchen; }
}

/** A customer's cart. One restaurant per cart, as in Swiggy/Zomato. */
final class Cart {
    private Restaurant restaurant;
    private final Map<String, Integer> quantities = new LinkedHashMap<>();

    synchronized void add(Restaurant r, String itemId, int qty) {
        if (restaurant != null && !restaurant.id().equals(r.id()))
            throw new IllegalStateException("cart has items from " + restaurant.name() + " - clear it first");
        r.item(itemId).orElseThrow(() -> new IllegalArgumentException("no item " + itemId));
        restaurant = r;
        quantities.merge(itemId, qty, Integer::sum);
    }

    synchronized void clear() { restaurant = null; quantities.clear(); }
    synchronized Restaurant restaurant() { return restaurant; }
    synchronized Map<String, Integer> quantities() { return Map.copyOf(quantities); }
    synchronized boolean isEmpty() { return quantities.isEmpty(); }
}

/** Price is copied at order time: a later menu price change never alters a placed order. */
record OrderItem(String itemId, String name, BigDecimal unitPrice, int quantity) {
    BigDecimal total() { return unitPrice.multiply(BigDecimal.valueOf(quantity)); }
}

/** The order-tracking state machine. */
enum OrderStatus {
    PENDING, CONFIRMED, PREPARING, READY_FOR_PICKUP, OUT_FOR_DELIVERY, DELIVERED, CANCELLED;

    private Set<OrderStatus> next() {
        return switch (this) {
            case PENDING -> EnumSet.of(CONFIRMED, CANCELLED);
            case CONFIRMED -> EnumSet.of(PREPARING, CANCELLED);
            case PREPARING -> EnumSet.of(READY_FOR_PICKUP);
            case READY_FOR_PICKUP -> EnumSet.of(OUT_FOR_DELIVERY);
            case OUT_FOR_DELIVERY -> EnumSet.of(DELIVERED);
            case DELIVERED, CANCELLED -> EnumSet.noneOf(OrderStatus.class);
        };
    }

    boolean canMoveTo(OrderStatus t) { return next().contains(t); }
}

/** Strategy: how the customer pays. */
sealed interface PaymentMethod permits UpiPayment, CardPayment, CashOnDelivery {
    String describe();
    default boolean paidUpfront() { return true; }
}

record UpiPayment(String vpa) implements PaymentMethod {
    public String describe() { return "UPI " + vpa; }
}

record CardPayment(String last4) implements PaymentMethod {
    public String describe() { return "card ****" + last4; }
}

record CashOnDelivery() implements PaymentMethod {
    public String describe() { return "cash on delivery"; }
    public boolean paidUpfront() { return false; }
}

final class Order {
    private final String id;
    private final Customer customer;
    private final Restaurant restaurant;
    private final List<OrderItem> items;
    private final BigDecimal deliveryFee;
    private final PaymentMethod payment;
    private final List<OrderStatus> history = new ArrayList<>();
    private OrderStatus status = OrderStatus.PENDING;
```

```
    private DeliveryAgent agent;

    Order(String id, Customer c, Restaurant r, List<OrderItem> items, BigDecimal fee, PaymentMethod payment) {
        this.id = id; this.customer = c; this.restaurant = r; this.items = List.copyOf(items);
        this.deliveryFee = fee; this.payment = payment;
        history.add(status);
    }

    String id() { return id; }
    Customer customer() { return customer; }
    Restaurant restaurant() { return restaurant; }
    List<OrderItem> items() { return items; }
    PaymentMethod payment() { return payment; }
    int itemCount() { return items.stream().mapToInt(OrderItem::quantity).sum(); }
    BigDecimal subtotal() { return items.stream().map(OrderItem::total).reduce(BigDecimal.ZERO, BigDecimal::add); }
    BigDecimal total() { return subtotal().add(deliveryFee).setScale(2, RoundingMode.HALF_EVEN); }
    BigDecimal deliveryFee() { return deliveryFee; }
    synchronized OrderStatus status() { return status; }
    synchronized List<OrderStatus> history() { return List.copyOf(history); }
    synchronized Optional<DeliveryAgent> agent() { return Optional.ofNullable(agent); }
    synchronized void assign(DeliveryAgent a) { agent = a; }

    synchronized void moveTo(OrderStatus t) {
        if (!status.canMoveTo(t)) throw new IllegalStateException(id + ": " + status + " -> " + t + " not allowed");
        status = t;
        history.add(t);
    }
}

/** A rider. available is an AtomicBoolean so two orders can never claim the same rider. */
final class DeliveryAgent {
    private final String id;
    private final String name;
    private volatile Location location;
    private final AtomicBoolean available = new AtomicBoolean(true);
    private final AtomicInteger deliveriesToday = new AtomicInteger();

    DeliveryAgent(String id, String name, Location location) { this.id = id; this.name = name; this.location =
location; }
    String id() { return id; }
    String name() { return name; }
    Location location() { return location; }
    void moveTo(Location l) { location = l; }
    boolean isAvailable() { return available.get(); }
    int deliveriesToday() { return deliveriesToday.get(); }

    /** Compare-and-set: exactly one caller wins. */
    boolean tryClaim() { return available.compareAndSet(true, false); }

    void completeDelivery() { deliveriesToday.incrementAndGet(); available.set(true); }

    @Override public String toString() { return name; }
}
```

Output:

```
== 1. Browse within 10 km ==
  Dosa Corner
  Pizza Hub

== 2. One restaurant per cart ==
  cart has items from Dosa Corner - clear it first

== 3. Place, accept, prepare, ready, deliver ==
    [OD101 PENDING] -> restaurant Dosa Corner: new order
  subtotal 280.00 + delivery 31.20 = 311.20
  price changed after ordering; order still 311.20
    [OD101 CONFIRMED] -> customer Asha
    [OD101 PREPARING] -> customer Asha
    [OD101 READY_FOR_PICKUP] -> customer Asha
    [OD101 OUT_FOR_DELIVERY] -> customer Asha: Ravi is on the way
  rider: Ravi (nearest to Dosa Corner)

== 4. Kitchen capacity: 4 items max at Dosa Corner ==
    [OD102 PENDING] -> restaurant Dosa Corner: new order
    [OD102 CONFIRMED] -> customer Vikram
    [OD103 PENDING] -> restaurant Dosa Corner: new order
    (Dosa Corner kitchen full: needs 2 item slots, free 1)
    [OD103 CANCELLED] -> customer Asha

== 5. Only one rider free: second ready order waits ==
    [OD102 PREPARING] -> customer Vikram
    [OD102 READY_FOR_PICKUP] -> customer Vikram
    [OD102 OUT_FOR_DELIVERY] -> customer Vikram: Meena is on the way
    [OD104 PENDING] -> restaurant Pizza Hub: new order
    [OD104 CONFIRMED] -> customer Vikram
    [OD104 PREPARING] -> customer Vikram
    [OD104 READY_FOR_PICKUP] -> customer Vikram
    (OD104 waiting for a free rider)
    [OD101 DELIVERED] -> customer Asha
    [OD104 OUT_FOR_DELIVERY] -> customer Vikram: Ravi is on the way
  o4 rider: Ravi

== 6. Cancel rules ==
  cancel o2 (out for delivery)? false
    [OD105 PENDING] -> restaurant Pizza Hub: new order
    [OD105 CONFIRMED] -> customer Asha
    [OD105 CANCELLED] -> customer Asha
    refund 651.60 to card ****4242
  cancel o5 (confirmed)? true

== 7. Swap strategy at run time: fair share ==
    [OD102 DELIVERED] -> customer Vikram
    [OD104 DELIVERED] -> customer Vikram
  deliveries today: Ravi=2, Meena=1 (both now at Vikram's address)
    [OD106 PENDING] -> restaurant Dosa Corner: new order
    [OD106 CONFIRMED] -> customer Asha
    [OD106 PREPARING] -> customer Asha
    [OD106 READY_FOR_PICKUP] -> customer Asha
    [OD106 OUT_FOR_DELIVERY] -> customer Asha: Meena is on the way
  o6 rider: Meena (fewer deliveries today)
  o1 history: [PENDING, CONFIRMED, PREPARING, READY_FOR_PICKUP, OUT_FOR_DELIVERY, DELIVERED]
```

Reading the output. Step 1 hides the far-away grill (outside 10 km). Step 2 refuses a second restaurant in one cart. Step 3: the delivery fee is 20 + 8 × 1.4 km = 31.20; changing the dosa price afterwards does not change the order. Step 4: Dosa Corner holds 4 item slots; an order for 3 fits, the next order for 2 is auto-cancelled. Step 5: Meena takes OD102; OD104 waits because Ravi is still out; as soon as OD101 is delivered, Ravi is claimed for OD104. Step 6: an order out for delivery cannot be cancelled; a confirmed card order can, and is refunded. Step 7: with both riders at the same spot, the fair-share strategy picks Meena (1 delivery) over Ravi (2).

Part B — FoodKart (`lld.ch19.foodkart`)

Files: `FoodKart.java` (gender, user, review, order, restaurant, sort, selection strategies, service), `Main.java` (driver replaying the problem's sample test case, then extras).

```java
// File: lld/ch19/foodkart/FoodKart.java
package lld.ch19.foodkart;

import java.math.BigDecimal;
import java.util.Comparator;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.Set;
import java.util.TreeSet;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicInteger;

enum Gender { M, F, OTHER }

record User(String name, Gender gender, String phone, String pincode) { }

record Review(String userPhone, int rating, String comment) {
    Review {
        if (rating < 1 || rating > 5) throw new IllegalArgumentException("rating must be 1..5");
    }
}

record FoodOrder(int id, String userPhone, String restaurant, String item, int quantity, BigDecimal total) { }

/** A restaurant serves ONE dish to MANY pincodes. Quantity is its remaining capacity. */
final class Restaurant {
    private final String name;
    private final String ownerPhone;
    private final String foodItem;
    private final BigDecimal price;
    private volatile Set<String> pincodes;
    private int quantity;                                        // guarded by this
    private final List<Review> reviews = new CopyOnWriteArrayList<>();

    Restaurant(String name, String ownerPhone, Set<String> pincodes, String foodItem, BigDecimal price, int quantity)
{
        this.name = name; this.ownerPhone = ownerPhone; this.pincodes = Set.copyOf(pincodes);
        this.foodItem = foodItem; this.price = price; this.quantity = quantity;
    }

    String name() { return name; }
    String ownerPhone() { return ownerPhone; }
    String foodItem() { return foodItem; }
    BigDecimal price() { return price; }
    Set<String> pincodes() { return pincodes; }
    void setPincodes(Set<String> p) { pincodes = Set.copyOf(p); }
    synchronized int quantity() { return quantity; }
    synchronized void addQuantity(int q) { quantity += q; }

    /** Check-and-take in one step, so two users cannot both get the last plates. */
    synchronized boolean take(int q) {
        if (q <= 0 || q > quantity) return false;
        quantity -= q;
        return true;
    }

    /** Serviceable = delivers to this pincode AND has non-zero quantity (from the problem statement). */
    boolean serviceableFor(String pincode) { return pincodes.contains(pincode) && quantity() > 0; }

    void addReview(Review r) { reviews.add(r); }

    double rating() { return reviews.stream().mapToInt(Review::rating).average().orElse(0.0); }

    String describe() { return name + ", " + String.join("/", new TreeSet<>(pincodes)) + ", " + foodItem + " - " +
quantity(); }
}

/** show_restaurant(rating|price): both sorted in DESCENDING order, as the problem states. */
enum SortBy {
    RATING(Comparator.comparingDouble(Restaurant::rating).reversed()),
```

```
    PRICE(Comparator.comparing(Restaurant::price).reversed());

    final Comparator<Restaurant> comparator;
    SortBy(Comparator<Restaurant> c) { comparator = c.thenComparing(Restaurant::name); }
}

/** Strategy: when the user does not name a restaurant, which one should we pick? */
interface RestaurantSelectionStrategy {
    Optional<Restaurant> select(List<Restaurant> canFulfil);
}

final class HighestRatingStrategy implements RestaurantSelectionStrategy {
    @Override public Optional<Restaurant> select(List<Restaurant> c) {
        return c.stream().max(Comparator.comparingDouble(Restaurant::rating)
                .thenComparing(Restaurant::quantity));              // tie: the one with most capacity left
    }
}

final class LowestPriceStrategy implements RestaurantSelectionStrategy {
    @Override public Optional<Restaurant> select(List<Restaurant> c) {
        return c.stream().min(Comparator.comparing(Restaurant::price)
                .thenComparing(Comparator.comparingDouble(Restaurant::rating).reversed()));
    }
}

/** The service behind every command in the machine-coding round. In-memory only. */
final class FoodKartService {
    private final Map<String, User> users = new ConcurrentHashMap<>();
    private final Map<String, Restaurant> restaurants = new ConcurrentHashMap<>();
    private final Map<String, List<FoodOrder>> history = new ConcurrentHashMap<>();
    private final AtomicInteger orderSeq = new AtomicInteger();
    private volatile User current;                                  // one logged-in user at a time

    void registerUser(String name, Gender g, String phone, String pincode) {
        if (users.putIfAbsent(phone, new User(name, g, phone, pincode)) != null)
            throw new IllegalArgumentException("phone already registered: " + phone);
    }

    void loginUser(String phone) {
        User u = users.get(phone);
        if (u == null) throw new IllegalArgumentException("unknown user " + phone);
        current = u;                                                // previous user is logged out
    }

    void registerRestaurant(String name, String pincodesSlashSeparated, String item, BigDecimal price, int qty) {
        User owner = requireLogin();
        Restaurant r = new Restaurant(name, owner.phone(), parsePincodes(pincodesSlashSeparated), item, price, qty);
        if (restaurants.putIfAbsent(name, r) != null) throw new IllegalArgumentException("restaurant exists: " +
name);
    }

    String updateQuantity(String name, int toAdd) {
        Restaurant r = ownedByCurrentUser(name);
        if (toAdd <= 0) throw new IllegalArgumentException("quantity to add must be positive");
        r.addQuantity(toAdd);
        return r.describe();
    }

    String updateLocation(String name, String pincodesSlashSeparated) {
        Restaurant r = ownedByCurrentUser(name);
        r.setPincodes(parsePincodes(pincodesSlashSeparated));
        return r.describe();
    }

    void rateRestaurant(String name, int rating, String comment) {
        restaurant(name).addReview(new Review(requireLogin().phone(), rating, comment == null ? "" : comment));
    }

    List<String> showRestaurants(SortBy sortBy) {
        String pin = requireLogin().pincode();
        return restaurants.values().stream().filter(r -> r.serviceableFor(pin))
                .sorted(sortBy.comparator).map(r -> r.name() + ", " + r.foodItem()).toList();
    }
```

```
    String placeOrder(String name, int qty) {
        User u = requireLogin();
        Restaurant r = restaurants.get(name);
        if (r == null || !r.pincodes().contains(u.pincode()) || !r.take(qty)) return "Cannot place order";
        record(u, r, qty);
        return "Order Placed Successfully.";
    }

    /** Variant: the system picks the restaurant. Only those that deliver here AND have capacity for qty qualify. */
    String placeOrder(int qty, RestaurantSelectionStrategy strategy) {
        User u = requireLogin();
        List<Restaurant> canFulfil = restaurants.values().stream()
                .filter(r -> r.pincodes().contains(u.pincode()) && r.quantity() >= qty).toList();
        for (int attempt = 0; attempt < 3; attempt++) {           // retry if a racer took the capacity
            Optional<Restaurant> pick = strategy.select(canFulfil);
            if (pick.isEmpty()) return "Cannot place order";
            if (pick.get().take(qty)) {
                record(u, pick.get(), qty);
                return "Order Placed Successfully from " + pick.get().name() + ".";
            }
            canFulfil = canFulfil.stream().filter(r -> r != pick.get()).toList();
        }
        return "Cannot place order";
    }

    List<FoodOrder> orderHistory() { return List.copyOf(history.getOrDefault(requireLogin().phone(), List.of())); }

    private void record(User u, Restaurant r, int qty) {
        FoodOrder o = new FoodOrder(orderSeq.incrementAndGet(), u.phone(), r.name(), r.foodItem(), qty,
                r.price().multiply(BigDecimal.valueOf(qty)));
        history.computeIfAbsent(u.phone(), k -> new CopyOnWriteArrayList<>()).add(o);
    }

    private User requireLogin() {
        User u = current;
        if (u == null) throw new IllegalStateException("no user logged in");
        return u;
    }

    private Restaurant restaurant(String name) {
        Restaurant r = restaurants.get(name);
        if (r == null) throw new IllegalArgumentException("no restaurant " + name);
        return r;
    }

    private Restaurant ownedByCurrentUser(String name) {
        Restaurant r = restaurant(name);
        if (!r.ownerPhone().equals(requireLogin().phone())) throw new IllegalStateException("only the owner can update
" + name);
        return r;
    }

    private static Set<String> parsePincodes(String s) {
        Set<String> out = new LinkedHashSet<>();
        for (String p : s.split("/")) if (!p.isBlank()) out.add(p.trim());
        if (out.isEmpty()) throw new IllegalArgumentException("at least one pincode needed");
        return out;
    }

}
```

```java
// File: lld/ch19/foodkart/Main.java
package lld.ch19.foodkart;

import java.math.BigDecimal;

/** Driver: replays the sample test case from the problem statement, then the extras. */
public class Main {
    public static void main(String[] args) {
        FoodKartService fk = new FoodKartService();

        System.out.println("== Sample test case ==");
        fk.registerUser("Pralove", Gender.M, "phoneNumber-1", "HSR");
        fk.registerUser("Nitesh", Gender.M, "phoneNumber-2", "BTM");
        fk.registerUser("Vatsal", Gender.M, "phoneNumber-3", "BTM");

        fk.loginUser("phoneNumber-1");
        fk.registerRestaurant("Food Court-1", "BTM/HSR", "NI Thali", new BigDecimal("100"), 5);
        fk.registerRestaurant("Food Court-2", "BTM", "Burger", new BigDecimal("120"), 3);
        fk.loginUser("phoneNumber-2");
        fk.registerRestaurant("Food Court-3", "HSR", "SI Thali", new BigDecimal("150"), 1);
        fk.loginUser("phoneNumber-3");

        System.out.println("show_restaurant(price):");
        fk.showRestaurants(SortBy.PRICE).forEach(s -> System.out.println("  " + s));
        System.out.println("place_order(Food Court-1, 2): " + fk.placeOrder("Food Court-1", 2));
        System.out.println("place_order(Food Court-2, 7): " + fk.placeOrder("Food Court-2", 7));
        fk.rateRestaurant("Food Court-2", 3, "Good Food");
        fk.rateRestaurant("Food Court-1", 5, "Nice Food");
        System.out.println("show_restaurant(rating):");
        fk.showRestaurants(SortBy.RATING).forEach(s -> System.out.println("  " + s));

        fk.loginUser("phoneNumber-1");
        System.out.println("update_quantity(Food Court-2, 5): " + fk.updateQuantity("Food Court-2", 5));
        System.out.println("update_location(Food Court-2, BTM/HSR): " + fk.updateLocation("Food Court-2", "BTM/HSR"));

        System.out.println("\n== Extras ==");
        fk.loginUser("phoneNumber-2");
        try { fk.updateQuantity("Food Court-1", 10); } catch (IllegalStateException e) { System.out.println("non-owner
update: " + e.getMessage()); }
        try { fk.rateRestaurant("Food Court-1", 6, null); } catch (IllegalArgumentException e) {
System.out.println("rating 6: " + e.getMessage()); }
        fk.rateRestaurant("Food Court-1", 4, null);             // rating without a comment
        System.out.println("order from a restaurant that does not deliver to BTM: " + fk.placeOrder("Food Court-3",
1));
        fk.registerRestaurant("Food Court-4", "BTM", "Dosa", new BigDecimal("80"), 4);

        System.out.println("\n== Selection strategy variant (user in BTM, wants 4 plates) ==");
        fk.loginUser("phoneNumber-3");
        System.out.println("highest rating: " + fk.placeOrder(4, new HighestRatingStrategy()));
        System.out.println("lowest price  : " + fk.placeOrder(4, new LowestPriceStrategy()));
        System.out.println("lowest price  : " + fk.placeOrder(4, new LowestPriceStrategy()));
        System.out.println("lowest price  : " + fk.placeOrder(4, new LowestPriceStrategy()) + "  (no one has 4
left)");

        System.out.println("\n== Order history of phoneNumber-3 ==");
        fk.orderHistory().forEach(o -> System.out.println("  " + o));
    }
}
```

Output:

```
== Sample test case ==
show_restaurant(price):
  Food Court-2, Burger
  Food Court-1, NI Thali
place_order(Food Court-1, 2): Order Placed Successfully.
place_order(Food Court-2, 7): Cannot place order
show_restaurant(rating):
  Food Court-1, NI Thali
  Food Court-2, Burger
update_quantity(Food Court-2, 5): Food Court-2, BTM, Burger - 8
update_location(Food Court-2, BTM/HSR): Food Court-2, BTM/HSR, Burger - 8

== Extras ==
non-owner update: only the owner can update Food Court-1
rating 6: rating must be 1..5
order from a restaurant that does not deliver to BTM: Cannot place order

== Selection strategy variant (user in BTM, wants 4 plates) ==
highest rating: Order Placed Successfully from Food Court-2.
lowest price  : Order Placed Successfully from Food Court-4.
lowest price  : Order Placed Successfully from Food Court-2.
lowest price  : Cannot place order  (no one has 4 left)

== Order history of phoneNumber-3 ==
  FoodOrder[id=1, userPhone=phoneNumber-3, restaurant=Food Court-1, item=NI Thali, quantity=2, total=200]
  FoodOrder[id=2, userPhone=phoneNumber-3, restaurant=Food Court-2, item=Burger, quantity=4, total=480]
  FoodOrder[id=3, userPhone=phoneNumber-3, restaurant=Food Court-4, item=Dosa, quantity=4, total=320]
  FoodOrder[id=4, userPhone=phoneNumber-3, restaurant=Food Court-2, item=Burger, quantity=4, total=480]
```

Reading the output. The "Sample test case" block reproduces every expected line from the problem statement: price-sorted list (Burger 120 before Thali 100), the successful and failed orders, the rating-sorted list, and the owner's quantity and location updates (`Burger - 8`). The extras show owner-only updates, rating validation, a rating without comment, and a restaurant that does not deliver to the user's pincode. The variant shows the two strategies choosing **different** restaurants (highest rating → Food Court-2; lowest price → Food Court-4), capacity filtering (Food Court-1 has only 3 plates, so it is never picked for 4), and a clean refusal once no one has 4 plates left.

Concurrency & edge cases

Case	Handling
Two claims for the same rider	`tryClaim()` CAS: only one succeeds; the loser moves to the next rider in its ranked list. (Inside one service the synchronized dispatcher already serialises claims; the CAS also covers claims made outside it.)
Two dispatch loops run at once	`dispatchWaiting` is `synchronized`, so the waiting queue is drained by one thread at a time (FIFO fairness preserved).
Kitchen over-booked	`tryReserveKitchen` is `synchronized` (check + add in one step).
Cancel races with accept	Both lock the `Order`; accept re-checks `PENDING`, cancel releases slots only if they were reserved (`CONFIRMED`). No leak, no double release.
Item sold out after it was added to the cart	`placeOrder` re-checks `available` and rejects.
Restaurant closes	`browse` hides it; `placeOrder` refuses.
COD cancelled	No refund (nothing was paid).
FoodKart: no user logged in	`requireLogin()` throws a clear `IllegalStateException`.
FoodKart: duplicate phone / restaurant name	Rejected with `putIfAbsent`.
FoodKart: rating outside 1–5	The `Review` record's compact constructor (a record's short validation block that runs on creation) rejects it.
FoodKart: strategy picks a restaurant whose capacity was just taken	`take()` fails; that restaurant is dropped and the strategy asked again (bounded retries).

Real-world example

In Swiggy and Zomato the order screen walks through the same states — order placed, restaurant confirmed, food being prepared, rider assigned / picked up, delivered — and if you add a dish from a second restaurant, the app asks whether to **reset your cart**, which is our one-restaurant `Cart` rule. The rider side is a dispatch problem: these platforms assign riders using distance, rider load and fairness signals, and may batch two orders on one trip — all variations of `DeliveryAssignmentStrategy`. FoodKart itself is a Flipkart machine-coding question: a 90-minute, in-memory, driver-class exercise that rewards clean separation (service, model, strategy) over clever algorithms.

Extensions & follow-up questions

- **"Batch two orders for one rider."** A strategy that accepts a rider already carrying one order if the second pickup is on the way; the agent then has a list of orders and a capacity (e.g. 2).
- **"ETA."** An `EtaEstimator` strategy: prep time (per restaurant load) + travel time (distance / speed, or a maps API — an online service other programs call).
- **"Surge / distance-based fees."** Extract `DeliveryFeePolicy` (Strategy): base + per-km today, add surge when waiting orders exceed free riders.
- **"Rider rejects an assignment."** Put the rider back to available, add the order back to the head of the queue, and exclude that rider for this order.
- **"Multiple dishes per restaurant in FoodKart."** Replace `foodItem/price/quantity` with a `Map<String, MenuItem>`; `take` becomes per item and all-or-nothing for multi-item orders.
- **"Order split across restaurants (Flipkart variant)."** The selection strategy returns a *set* of restaurants that together cover all items; reserve capacity in each, roll back all on any failure.
- **"Live tracking."** The rider app pushes location updates; an Observer forwards them to the customer's app (WebSocket).

In the interview

Design round (45 min): clarify (5) → entities + one-restaurant cart (8) → **order state machine** (7) → **assignment strategy + CAS claim + waiting queue** (12) → payments/notifications (5) → concurrency and

extensions (8).

Machine-coding round (FoodKart, 90 min): model classes first (User, Restaurant, Review, Order) → service with one method per command → the driver replaying the sample → sort enum → owner checks and validation → selection strategy → order history. Keep it runnable at every step; print exactly the expected lines.

> **30-second answer:** "Restaurants own their menus and a kitchen capacity reserved on accept and released when food is ready. Orders copy item prices and move through a state machine from PENDING to DELIVERED. When food is ready the order joins a FIFO queue; a pluggable assignment strategy ranks free riders and we claim one with a compare-and-set, so a rider can never get two orders. Every status change is published to observers for the customer, restaurant and rider apps."

Follow-ups with answers

- *Why release kitchen capacity on "ready" and not on "delivered"?* The kitchen is free once the food leaves it; delivery time is the rider's constraint, not the kitchen's.
- *Why does the strategy return a list?* Because between ranking and claiming, the top rider may be claimed by someone else (another dispatcher or the rider's own app); the list lets us fall through without re-ranking.
- *How would you scale assignment across cities?* Partition by city/zone; each zone has its own dispatcher (single writer), so there is no cross-zone contention (threads or servers fighting over the same lock or data).

Self-check

1. **Q:** Which states allow a customer cancel, and why not later? **A:** PENDING and CONFIRMED; once cooking starts the food cost is already spent.
2. **Q:** What stops two orders getting the same rider? **A:** `AtomicBoolean.compareAndSet(true, false)` in `tryClaim()`.
3. **Q:** In FoodKart, what makes a restaurant "serviceable"? **A:** It delivers to the user's pincode and has quantity > 0.
4. **Q:** Why does `show_restaurant(price)` list Burger (120) before NI Thali (100)? **A:** The statement asks for descending order.
5. **Q:** How does the selection variant avoid choosing a restaurant that cannot fulfil the order? **A:** It filters by `quantity ≥ qty` before the strategy chooses, and `take()` re-checks atomically.
6. **Q:** Which patterns appear? **A:** Strategy (assignment, selection, payment), Observer (notifications), state machine (order status), Facade (service).

TL;DR

- One-restaurant cart; order copies prices; **state machine** PENDING → … → DELIVERED.
- **Kitchen capacity** reserved on accept, released on ready; **riders** claimed with **CAS**, chosen by a swappable **strategy**, with a FIFO queue when none are free.
- FoodKart: pincode-based serviceability, one dish per restaurant, average rating, `SortBy` enum, atomic `take`, and a **selection strategy** filtered by capacity.

Design a Grocery Store (POS & Checkout)

A **point-of-sale (POS) system** is the software behind a supermarket till: the cashier (or a self-checkout kiosk) scans items, the screen shows a running total, offers are applied, the customer pays in one or more ways, a receipt is printed, and stock goes down. Amazon (§19.1) was about an online cart that becomes an order; a grocery till is different in three ways that make it a good interview question:

- **Items are weighed as well as counted.** Loose produce is priced per kilogram; the scale sends a weight and the price is computed at the till.
- **Offers interact.** "Buy 2 get 1 free", "3 for ₹80", "10% off produce" and a basket coupon can all apply to one basket — and the rules for combining them decide what the customer pays, what tax is due, and how much a return refunds.

- **The till must never block a physical sale.** If the system says milk stock is 0 but a customer is holding a carton, the *count* is wrong, not the customer.

Money in this section is Indian rupees (₹; written `Rs` in the program output) and uses `BigDecimal`, following the chapter's four rules.

Problem & clarifying questions

Question	Assumed answer
Who uses it?	Cashiers at several registers (lanes) in one store; a supervisor for voids and refunds. Self-checkout uses the same register API.
How are items identified?	Packaged goods by **barcode** (EAN-13, the standard 13-digit product barcode); loose produce by a 4-digit **PLU** (price look-up) code plus the scale's weight.
Are shelf prices tax-inclusive?	Yes (as in India, the UK and the EU): the receipt shows how much tax the price *contains*. A US-style store adds sales tax at the till — the design must support both.
Which offers?	Buy X get Y free, multi-buy ("3 for ₹80"), percent off a department, and a basket coupon ("₹50 off when you spend ₹500").
Can offers stack?	Each product gets at most **one** item offer — the best for the customer. A basket coupon applies on top, to what is still owed.
How do people pay?	Cash (with change), card, UPI — and **split tender** (part UPI, part cash).
Can the customer change their mind?	Void a line while scanning; go back to scanning if nothing is paid yet; cancel the whole sale (card/UPI payments reversed).
Returns?	Yes, with the receipt: refund what the customer **actually paid** for that item, never more units than bought.
Stock?	Decrease on sale, increase on return; alert when low; never refuse a sale because of the stock count.

Requirements

Functional

1. Catalog lookup by barcode or PLU; weighed items priced by weight.
2. A **sale** collects line items; lines can be voided (kept on the receipt, marked VOID).
3. Item offers and basket coupons, with clear stacking rules.
4. Totals with tax per tax category (inclusive or exclusive policy).
5. Split payment: cash (with change), card, UPI; a declined card changes nothing.
6. Immutable **receipt** with lines, discounts, tax breakdown, tenders and change.
7. Returns against a receipt, refunding the net price paid, at most the quantity bought.
8. Stock updates and **low-stock / negative-stock alerts**.

Non-functional

- **Exact money**: `BigDecimal`, 2 decimal places, `HALF_UP` rounding (a 5 or more rounds up), and parts (per-product discounts, refunds) that always add up to the total.
- **Several registers at once** share the catalog, the stock and the receipts safely.
- **Deterministic pricing**: the same basket always prices the same way (a **pure function**: its result depends only on its inputs, and it changes nothing outside itself) — needed for audits and for returns.
- **Auditable**: voids stay on the receipt; prices are copied into lines at scan time.

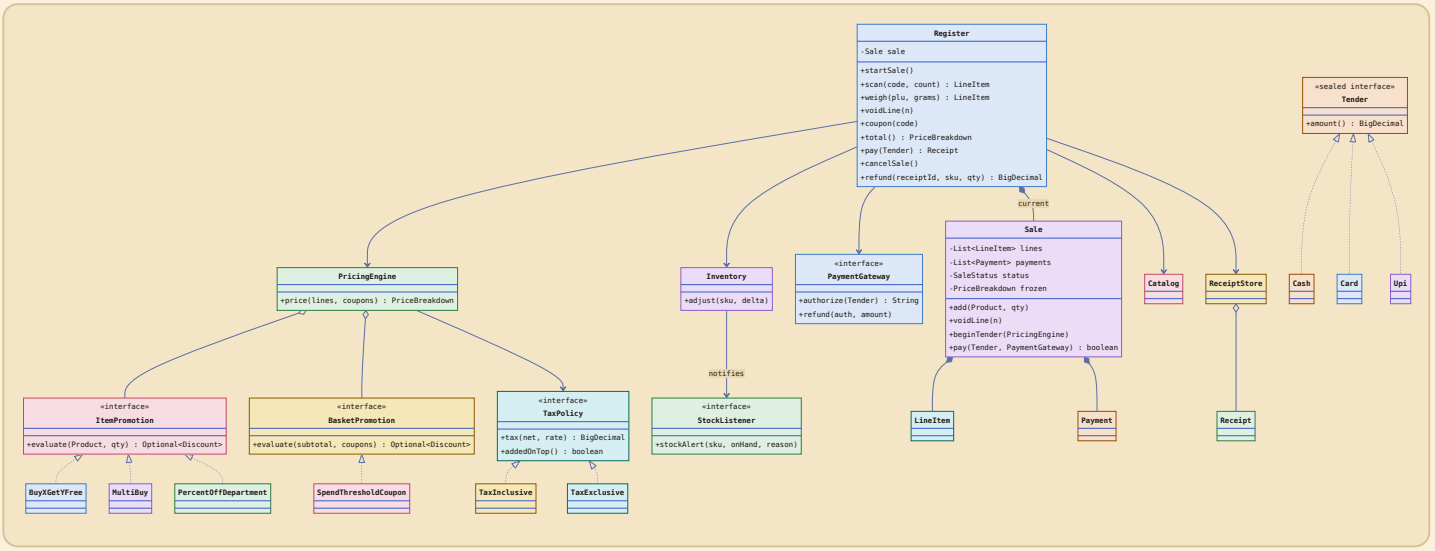
Use cases

Actor	Use cases
Cashier / self-checkout	start sale, scan, weigh, void line, add coupon, total, take payment, cancel sale
Customer	pay by cash / card / UPI, return items with the receipt
Supervisor	approve voids and returns (not modelled), reprice products
Payment network (system)	authorise and reverse card / UPI payments
Purchasing / store manager (subscriber)	receive low-stock and negative-stock alerts

Core entities

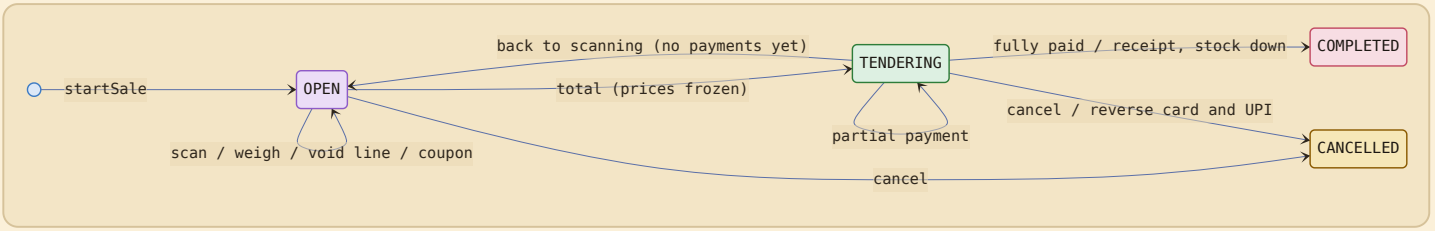
Class	Kind	Responsibility
`Product`	record	SKU (stock-keeping unit — the store's id for one distinct product), barcode/PLU, name, department, `PricingUnit` (EACH or PER_KG), unit price, `TaxCategory`.
`Catalog`	class	Lookup by code; `reprice` replaces the record (old sales keep old prices).
`LineItem`	record	One scan: product copy, quantity (count or kg), gross amount, voided flag.
`Sale` + `SaleStatus`	class + enum	OPEN → TENDERING → COMPLETED, or → CANCELLED; lines, coupons, payments, frozen price breakdown.
`ItemPromotion` + `BuyXGetYFree`, `MultiBuy`, `PercentOffDepartment`	interface + records	**Strategy** for item offers.
`BasketPromotion` + `SpendThresholdCoupon`	interface + record	**Strategy** for basket offers.
`TaxPolicy` + `TaxInclusive`, `TaxExclusive`	interface + classes	**Strategy** for how tax relates to the shelf price.
`PricingEngine` → `PriceBreakdown`	class → record	Pure function: lines + coupons → discounts, net per SKU, tax per category, total.
`Tender` (sealed) `Cash`, `Card`, `Upi`; `Payment`	records	How the customer pays; the part applied to the bill.
`PaymentGateway`	interface	Port (an interface through which the core code talks to an outside system) to the card / UPI networks (authorise, refund).
`Receipt`, `ReceiptStore`	record, class	Immutable proof of purchase; returns ledger per receipt and SKU.
`Inventory`, `StockListener`	class, interface	Shared stock; **Observer** for edge-triggered alerts (fired once when a line is crossed, not on every sale below it).
`Register`	class	**Facade** for one lane: the only API the till UI calls.

Class diagram

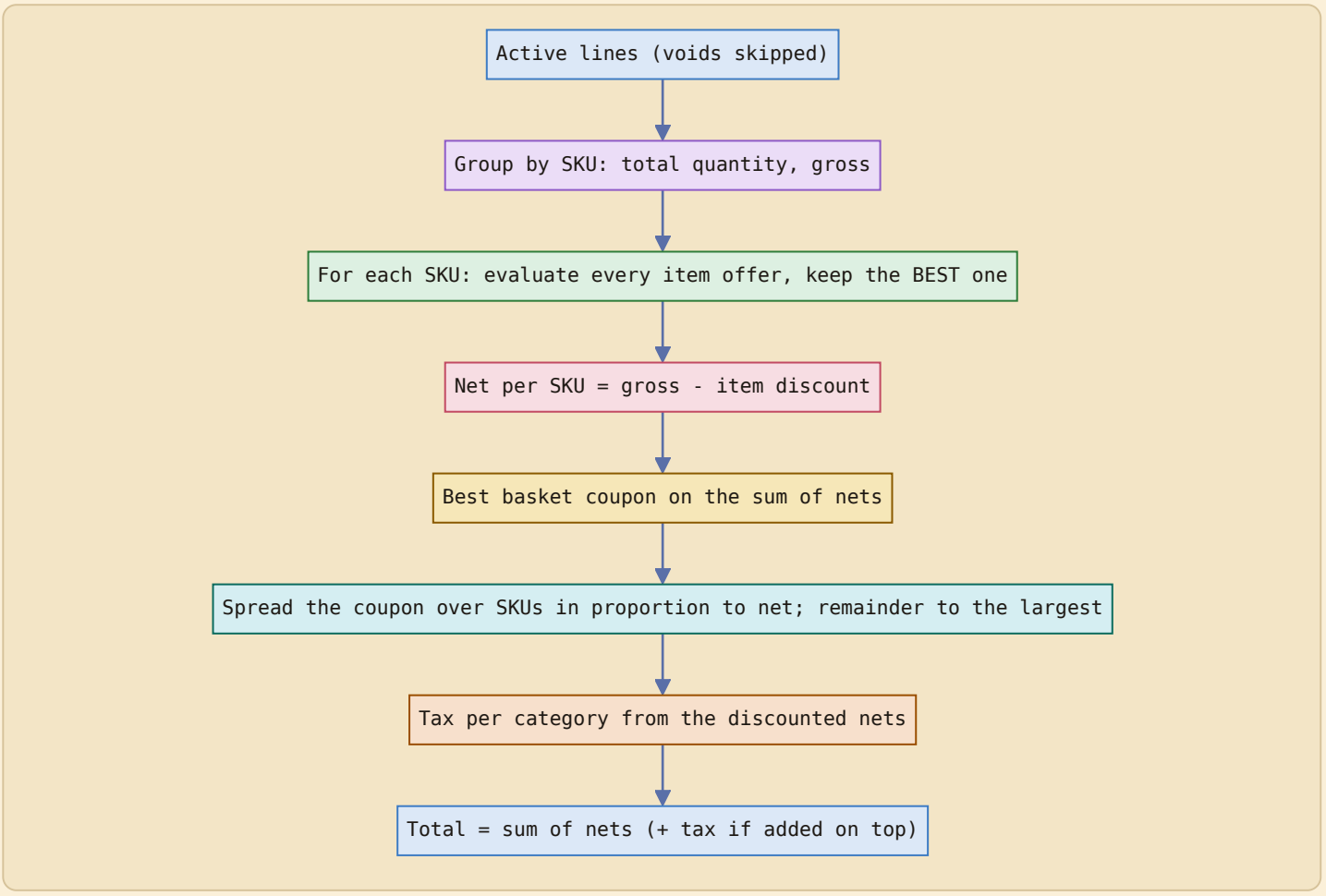


Key flows

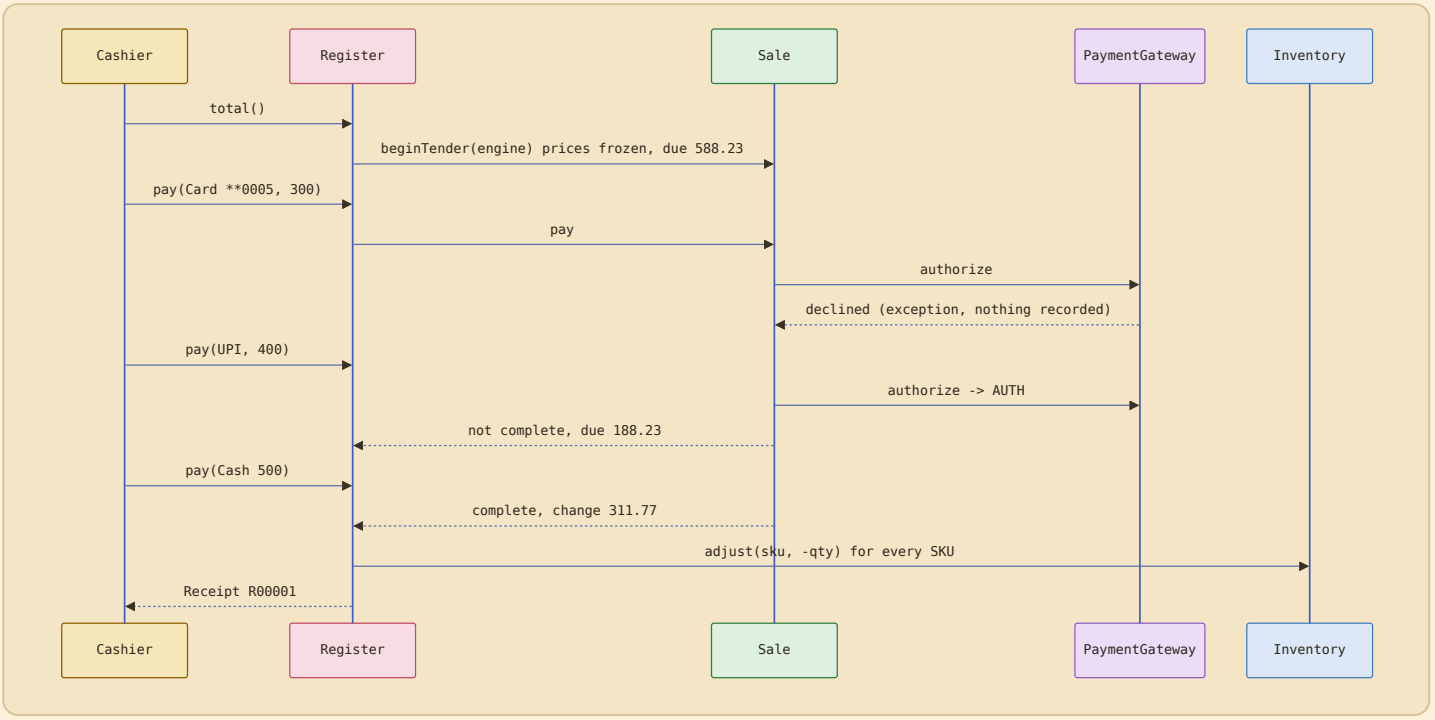
Sale lifecycle.



How one basket is priced (the hard method, `PricingEngine.price`):



Split payment.



Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Each offer is a small class behind `ItemPromotion` / `BasketPromotion`	**Strategy**	Marketing adds offers weekly; a new offer is one new class, the engine does not change	A `switch` over offer types inside the pricing code
Best offer per SKU wins; basket coupon after item offers	Explicit stacking policy	Stacking rules are business decisions that must be written down once; "best for the customer" is simple to explain at the till	Applying every offer in list order (results depend on the order; can go below zero)
`PricingEngine` is a **pure function** of lines and coupons	Stateless service	Same basket → same price; one engine shared by all registers (stateless: it keeps no data between calls); easy to test	Mutating discounts into the line items as they are scanned
Prices **frozen** when payment starts	Snapshot	A price change or an offer expiring mid-payment must not change the amount due	Re-pricing on every payment
Coupon **spread over SKUs**, remainder to the largest	Allocation with exact rounding	Tax is per category and returns refund per item, so every rupee of discount must belong to some item — and the parts must add up exactly	Keeping the coupon as one unallocated number
Tax as a `TaxPolicy`	**Strategy**	Inclusive (India/EU) vs exclusive (US) is a store-level choice	Two versions of the engine
Tenders as a **sealed** interface	Sealed types	The receipt printer's `switch` is checked by the compiler	`String paymentType`
`Register` as the only API the UI calls	**Facade**	The UI does not need to know about engines, stores and gateways	UI code calling each part directly
Sale status in an enum with `canMoveTo`	Enum state machine	"Scan after payment started" and "pay a cancelled sale" fail loudly	Boolean flags (`isPaying`, `isDone`)
Stock may go **negative**, alerts are **edge-triggered**	**Observer**	The shelf is the truth; negative stock is a signal for a recount. Each crossing alerts once	Refusing the sale; alerting on every sale below the line
Voided lines are kept and printed	Audit trail (a permanent record of every action)	Voids are a common way to hide theft; supervisors review them	Deleting the line

Java implementation

Package `lld.ch19.grocery`.

```java
// File: lld/ch19/grocery/Catalog.java
package lld.ch19.grocery;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;

/** Money helpers: BigDecimal, 2 decimal places, HALF_UP (the usual retail rounding). Never double. */
final class Money {
    private Money() { }
    static final BigDecimal ZERO = of("0");
    static BigDecimal of(String s) { return new BigDecimal(s).setScale(2, RoundingMode.HALF_UP); }
    static BigDecimal round(BigDecimal b) { return b.setScale(2, RoundingMode.HALF_UP); }
    static String fmt(BigDecimal b) { return "Rs " + round(b).toPlainString(); }
}

enum Department { PRODUCE, DAIRY, SNACKS, HOUSEHOLD }

/** How the shelf price is quoted: per piece, or per kilogram (weighed at the till). */
enum PricingUnit { EACH, PER_KG }

/** Tax slab. Rates are illustrative (loosely modelled on India's GST slabs), not tax advice. */
enum TaxCategory {
    EXEMPT("0"), REDUCED("5"), STANDARD("18");
    final BigDecimal ratePercent;
    TaxCategory(String r) { ratePercent = new BigDecimal(r); }
}

/** code = barcode (EAN-13) for packaged goods, or a 4-digit PLU for loose produce. */
record Product(String sku, String code, String name, Department dept, PricingUnit unit,
               BigDecimal unitPrice, TaxCategory tax) {
    Product {
        if (unitPrice.signum() <= 0) throw new PosException("Price must be positive: " + name);
        unitPrice = Money.round(unitPrice);
    }
}

/** Look-up by scanned code. Read far more than written, so a ConcurrentHashMap shared by all registers. */
final class Catalog {
    private final Map<String, Product> byCode = new ConcurrentHashMap<>();

    void add(Product p) {
        if (byCode.putIfAbsent(p.code(), p) != null) throw new PosException("Duplicate code " + p.code());
    }

    /** Price changes replace the record; open sales keep the price they already scanned. */
    void reprice(String code, BigDecimal newPrice) {
        byCode.computeIfPresent(code, (c, p) -> new Product(p.sku(), c, p.name(), p.dept(), p.unit(), newPrice,
p.tax()));
    }

    Optional<Product> find(String code) { return Optional.ofNullable(byCode.get(code)); }
}

final class PosException extends RuntimeException {
    PosException(String message) { super(message); }
}
```

```java
// File: lld/ch19/grocery/Promotions.java
package lld.ch19.grocery;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.Optional;
import java.util.Set;

/** One discount line on the receipt. sku == null means a basket-level discount. */
record Discount(String description, String sku, BigDecimal amount) { }

/** Strategy for an item-level offer. Called once per SKU with the total quantity of that SKU in the basket. */
interface ItemPromotion {
    Optional<Discount> evaluate(Product p, BigDecimal quantity);
}

/** "Buy 2, get 1 free": in every group of (buy + free) units, the free units cost nothing. */
record BuyXGetYFree(String sku, int buy, int free) implements ItemPromotion {
    @Override public Optional<Discount> evaluate(Product p, BigDecimal qty) {
        if (!p.sku().equals(sku)) return Optional.empty();
        int groups = qty.intValue() / (buy + free);
        if (groups == 0) return Optional.empty();
        BigDecimal off = p.unitPrice().multiply(BigDecimal.valueOf((long) groups * free));
        return Optional.of(new Discount("Buy " + buy + " get " + free + " free: " + p.name(), sku, Money.round(off)));
    }
}

/** "3 for Rs 100": each full group of n units costs a fixed price. */
record MultiBuy(String sku, int n, BigDecimal groupPrice) implements ItemPromotion {
    @Override public Optional<Discount> evaluate(Product p, BigDecimal qty) {
        if (!p.sku().equals(sku)) return Optional.empty();
        int groups = qty.intValue() / n;
        BigDecimal saving = p.unitPrice().multiply(BigDecimal.valueOf(n)).subtract(groupPrice);
        if (groups == 0 || saving.signum() <= 0) return Optional.empty();
        return Optional.of(new Discount(n + " for " + Money.fmt(groupPrice) + ": " + p.name(), sku,
                Money.round(saving.multiply(BigDecimal.valueOf(groups)))));
    }
}

/** "10% off all produce" — applies to weighed items too. */
record PercentOffDepartment(Department dept, int percent) implements ItemPromotion {
    @Override public Optional<Discount> evaluate(Product p, BigDecimal qty) {
        if (p.dept() != dept) return Optional.empty();
        BigDecimal gross = Money.round(p.unitPrice().multiply(qty));
        BigDecimal off = gross.multiply(BigDecimal.valueOf(percent)).divide(BigDecimal.valueOf(100), 2,
RoundingMode.HALF_UP);
        return Optional.of(new Discount(percent + "% off " + dept + ": " + p.name(), p.sku(), off));
    }
}

/** Strategy for a basket-level offer, evaluated after item offers, on what the customer still owes. */
interface BasketPromotion {
    Optional<Discount> evaluate(BigDecimal subtotalAfterItemOffers, Set<String> couponsPresented);
}

/** "Rs 50 off when you spend Rs 500 or more" — needs the coupon code to be scanned. */
record SpendThresholdCoupon(String code, BigDecimal minSpend, BigDecimal amountOff) implements BasketPromotion {
    @Override public Optional<Discount> evaluate(BigDecimal subtotal, Set<String> coupons) {
        if (!coupons.contains(code) || subtotal.compareTo(minSpend) < 0) return Optional.empty();
        return Optional.of(new Discount("Coupon " + code, null, amountOff.min(subtotal)));
    }
}
```

```java
// File: lld/ch19/grocery/Pricing.java
package lld.ch19.grocery;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.EnumMap;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.Set;

/** Strategy: are shelf prices tax-inclusive (India, EU, UK) or is tax added at the till (US sales tax)? */
interface TaxPolicy {
    BigDecimal tax(BigDecimal net, BigDecimal ratePercent);
    boolean addedOnTop();
}

final class TaxInclusive implements TaxPolicy {
    /** The price already contains the tax: tax = price x r / (100 + r). */
    @Override public BigDecimal tax(BigDecimal net, BigDecimal r) {
        return net.multiply(r).divide(BigDecimal.valueOf(100).add(r), 2, RoundingMode.HALF_UP);
    }
    @Override public boolean addedOnTop() { return false; }
}

final class TaxExclusive implements TaxPolicy {
    @Override public BigDecimal tax(BigDecimal net, BigDecimal r) {
        return net.multiply(r).divide(BigDecimal.valueOf(100), 2, RoundingMode.HALF_UP);
    }
    @Override public boolean addedOnTop() { return true; }
}

/** All scanned lines of one SKU added together. */
record SkuTotal(Product product, BigDecimal quantity, BigDecimal gross) { }

/** Everything the receipt needs. netBySku = what the customer really paid per SKU, after every discount. */
record PriceBreakdown(List<SkuTotal> items, List<Discount> discounts, Map<String, BigDecimal> netBySku,
                      BigDecimal gross, BigDecimal discountTotal, Map<TaxCategory, BigDecimal> taxByCategory,
                      BigDecimal total) { }

/**
 * Pure function of (lines, coupons): no state, so every register can share one engine and
 * the same basket always prices the same way. Rules:
 *  1. each SKU gets at most ONE item offer — the best one for the customer;
 *  2. then the best basket coupon, on what is still owed;
 *  3. basket discounts are spread over SKUs in proportion to their value (needed for tax and refunds);
 *  4. tax is computed per tax category on the discounted amounts.
 */
final class PricingEngine {
    private final List<ItemPromotion> itemOffers;
    private final List<BasketPromotion> basketOffers;
    private final TaxPolicy taxPolicy;

    PricingEngine(List<ItemPromotion> itemOffers, List<BasketPromotion> basketOffers, TaxPolicy taxPolicy) {
        this.itemOffers = List.copyOf(itemOffers);
        this.basketOffers = List.copyOf(basketOffers);
        this.taxPolicy = taxPolicy;
    }

    PriceBreakdown price(List<LineItem> lines, Set<String> coupons) {
        Map<String, SkuTotal> bySku = new LinkedHashMap<>();
        for (LineItem l : lines) {
            if (l.voided()) continue;
            bySku.merge(l.product().sku(), new SkuTotal(l.product(), l.quantity(), l.gross()),
                    (a, b) -> new SkuTotal(a.product(), a.quantity().add(b.quantity()), a.gross().add(b.gross())));
        }
        List<Discount> discounts = new ArrayList<>();
        Map<String, BigDecimal> net = new LinkedHashMap<>();
        for (SkuTotal t : bySku.values()) {
            Optional<Discount> best = itemOffers.stream()
```

```
                    .map(p -> p.evaluate(t.product(), t.quantity()))
                    .flatMap(Optional::stream)
                    .max(Comparator.comparing(Discount::amount));
            BigDecimal off = best.map(d -> d.amount().min(t.gross())).orElse(Money.ZERO);
            best.ifPresent(d -> discounts.add(new Discount(d.description(), d.sku(), off)));
            net.put(t.product().sku(), t.gross().subtract(off));
        }
        BigDecimal subtotal = sum(net.values());
        Optional<Discount> basket = basketOffers.stream()
                .map(b -> b.evaluate(subtotal, coupons))
                .flatMap(Optional::stream)
                .max(Comparator.comparing(Discount::amount));
        basket.ifPresent(d -> { discounts.add(d); spread(d.amount(), net); });

        Map<TaxCategory, BigDecimal> netByTax = new EnumMap<>(TaxCategory.class);
        bySku.values().forEach(t -> netByTax.merge(t.product().tax(), net.get(t.product().sku()), BigDecimal::add));
        Map<TaxCategory, BigDecimal> taxByCategory = new EnumMap<>(TaxCategory.class);
        netByTax.forEach((cat, amt) -> taxByCategory.put(cat, taxPolicy.tax(amt, cat.ratePercent)));

        BigDecimal gross = sum(bySku.values().stream().map(SkuTotal::gross).toList());
        BigDecimal netTotal = sum(net.values());
        BigDecimal total = taxPolicy.addedOnTop() ? netTotal.add(sum(taxByCategory.values())) : netTotal;
        return new PriceBreakdown(List.copyOf(bySku.values()), List.copyOf(discounts), Map.copyOf(net),
                gross, gross.subtract(netTotal), taxByCategory, total);
    }

    /** Proportional allocation; the rounding remainder goes to the largest SKU so the parts add up exactly. */
    private static void spread(BigDecimal amount, Map<String, BigDecimal> net) {
        BigDecimal base = sum(net.values());
        if (base.signum() == 0) return;
        BigDecimal given = Money.ZERO;
        String largest = null;
        for (var e : net.entrySet()) {
            BigDecimal share = amount.multiply(e.getValue()).divide(base, 2, RoundingMode.HALF_UP);
            e.setValue(e.getValue().subtract(share));
            given = given.add(share);
            if (largest == null || e.getValue().compareTo(net.get(largest)) > 0) largest = e.getKey();
        }
        net.merge(largest, given.subtract(amount), BigDecimal::add);
    }

    private static BigDecimal sum(Iterable<BigDecimal> xs) {
        BigDecimal s = Money.ZERO;
        for (BigDecimal x : xs) s = s.add(x);
        return s;
    }
}
```

```java
// File: lld/ch19/grocery/Sale.java
package lld.ch19.grocery;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Set;

/** One scan. The product (and so its price) is copied in at scan time; later reprices do not change it. */
record LineItem(int lineNo, Product product, BigDecimal quantity, BigDecimal gross, boolean voided) {
    LineItem asVoided() { return new LineItem(lineNo, product, quantity, gross, true); }
}

enum SaleStatus {
    OPEN, TENDERING, COMPLETED, CANCELLED;

    boolean canMoveTo(SaleStatus next) {
        return switch (this) {
            case OPEN -> next == TENDERING || next == CANCELLED;
            case TENDERING -> next == COMPLETED || next == OPEN || next == CANCELLED;
            case COMPLETED, CANCELLED -> false;
        };
    }
}

/** How the customer pays. Sealed: the compiler knows every kind of tender. */
sealed interface Tender permits Cash, Card, Upi { BigDecimal amount(); }
record Cash(BigDecimal amount) implements Tender { }
record Card(String last4, BigDecimal amount) implements Tender { }
record Upi(String vpa, BigDecimal amount) implements Tender { }

/** applied = the part of the tender that went towards the bill (cash can be more; the rest is change). */
record Payment(Tender tender, BigDecimal applied, String authCode) { }

/** Strategy / port to the card and UPI networks. Cash needs no gateway. */
interface PaymentGateway {
    String authorize(Tender t);          // returns an auth code or throws PosException("declined ...")
    void refund(String authCode, BigDecimal amount);
}

/** One customer's basket at one register, from first scan to receipt. */
final class Sale {
    final String id;
    private final List<LineItem> lines = new ArrayList<>();
    private final Set<String> coupons = new LinkedHashSet<>();
    private final List<Payment> payments = new ArrayList<>();
    private SaleStatus status = SaleStatus.OPEN;
    private PriceBreakdown frozen;       // prices are frozen when payment starts
    private BigDecimal change = Money.ZERO;

    Sale(String id) { this.id = id; }

    SaleStatus status() { return status; }
    List<LineItem> lines() { return List.copyOf(lines); }
    List<Payment> payments() { return List.copyOf(payments); }
    Set<String> coupons() { return Set.copyOf(coupons); }
    BigDecimal change() { return change; }
    PriceBreakdown breakdown() { return frozen; }

    LineItem add(Product p, BigDecimal qty) {
        require(SaleStatus.OPEN, "scan");
        if (qty.signum() <= 0) throw new PosException("Quantity must be positive");
        LineItem l = new LineItem(lines.size() + 1, p, qty, Money.round(p.unitPrice().multiply(qty)), false);
        lines.add(l);
        return l;
    }

    void voidLine(int lineNo) {
        require(SaleStatus.OPEN, "void a line");
        if (lineNo < 1 || lineNo > lines.size()) throw new PosException("No line " + lineNo);
        LineItem l = lines.get(lineNo - 1);
        if (l.voided()) throw new PosException("Line " + lineNo + " is already void");
```

```
        lines.set(lineNo - 1, l.asVoided());          // keep the line: the audit trail shows the void
    }

    void addCoupon(String code) { require(SaleStatus.OPEN, "add a coupon"); coupons.add(code); }

    PriceBreakdown beginTender(PricingEngine engine) {
        require(SaleStatus.OPEN, "start payment");
        if (lines.stream().allMatch(LineItem::voided)) throw new PosException("Nothing to pay for");
        frozen = engine.price(lines, coupons);
        move(SaleStatus.TENDERING);
        return frozen;
    }

    /** Back to scanning ("I forgot the milk") — only while nothing has been paid. */
    void reopen() {
        if (!payments.isEmpty()) throw new PosException("Payments already taken; cancel them first");
        frozen = null;
        move(SaleStatus.OPEN);
    }

    BigDecimal due() {
        BigDecimal paid = payments.stream().map(Payment::applied).reduce(Money.ZERO, BigDecimal::add);
        return frozen.total().subtract(paid);
    }

    /** Returns true when the bill is fully paid and the sale is complete. */
    boolean pay(Tender t, PaymentGateway gateway) {
        require(SaleStatus.TENDERING, "pay");
        BigDecimal due = due();
        if (t.amount().signum() <= 0) throw new PosException("Tender must be positive");
        String auth = "CASH";
        BigDecimal applied = t.amount();
        if (t instanceof Cash) {
            if (t.amount().compareTo(due) > 0) { change = t.amount().subtract(due); applied = due; }
        } else {
            if (t.amount().compareTo(due) > 0) throw new PosException("Card/UPI cannot be more than the amount due " +
Money.fmt(due));
            auth = gateway.authorize(t);                // throws if declined: nothing recorded
        }
        payments.add(new Payment(t, applied, auth));
        if (due().signum() == 0) { move(SaleStatus.COMPLETED); return true; }
        return false;
    }

    /** Cancel the whole sale; card/UPI payments already taken are reversed. */
    void cancel(PaymentGateway gateway) {
        if (!status.canMoveTo(SaleStatus.CANCELLED)) throw new PosException("Cannot cancel a " + status + " sale");
        for (Payment p : payments) if (!(p.tender() instanceof Cash)) gateway.refund(p.authCode(), p.applied());
        move(SaleStatus.CANCELLED);
    }

    private void require(SaleStatus s, String action) {
        if (status != s) throw new PosException("Cannot " + action + " when the sale is " + status);
    }

    private void move(SaleStatus next) {
        if (!status.canMoveTo(next)) throw new PosException(status + " -> " + next + " not allowed");
        status = next;
    }
}
```

```java
// File: lld/ch19/grocery/Register.java
package lld.ch19.grocery;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicLong;

/** Observer: purchasing, the store manager's dashboard, shrinkage reports. */
interface StockListener {
    void stockAlert(String sku, BigDecimal onHand, String reason);
}

/**
 * Store stock shared by every register. A till never refuses a sale because the SYSTEM says zero:
 * the item is physically in the customer's hand, so the count is what is wrong. Stock may go
 * negative, and that is reported as a discrepancy for a recount.
 */
final class Inventory {
    private final Map<String, BigDecimal> onHand = new ConcurrentHashMap<>();
    private final Map<String, BigDecimal> reorderPoint = new ConcurrentHashMap<>();
    private final List<StockListener> listeners = new CopyOnWriteArrayList<>();

    void set(String sku, String qty, String reorderAt) {
        onHand.put(sku, new BigDecimal(qty));
        reorderPoint.put(sku, new BigDecimal(reorderAt));
    }
    void addListener(StockListener l) { listeners.add(l); }
    BigDecimal onHand(String sku) { return onHand.getOrDefault(sku, BigDecimal.ZERO); }

    /** delta < 0 for a sale, > 0 for a return. merge() is atomic per key, so registers never lose updates. */
    void adjust(String sku, BigDecimal delta) {
        BigDecimal after = onHand.merge(sku, delta, BigDecimal::add);
        BigDecimal before = after.subtract(delta);
        BigDecimal point = reorderPoint.getOrDefault(sku, BigDecimal.ZERO);
        // edge-triggered: alert only when THIS change crosses the line, so each crossing alerts exactly once
        if (before.compareTo(point) > 0 && after.compareTo(point) <= 0)
            listeners.forEach(l -> l.stockAlert(sku, after, "LOW - reorder"));
        if (before.signum() >= 0 && after.signum() < 0)
            listeners.forEach(l -> l.stockAlert(sku, after, "NEGATIVE - recount the shelf"));
    }
}

/** Immutable proof of purchase. */
record Receipt(String id, String registerId, List<LineItem> lines, PriceBreakdown breakdown,
               List<Payment> payments, BigDecimal change) {

    String render() {
        StringBuilder sb = new StringBuilder();
        sb.append("  ---------------- RECEIPT ").append(id).append(" (").append(registerId).append(") ----------------
\n");
        for (LineItem l : lines) {
            String qty = l.product().unit() == PricingUnit.PER_KG
                    ? l.quantity().setScale(3, RoundingMode.UNNECESSARY) + " kg @ " +
Money.fmt(l.product().unitPrice()) + "/kg"
                    : l.quantity().toPlainString() + " x " + Money.fmt(l.product().unitPrice());
            sb.append("  %2d %-18s %-24s %12s%s%n".formatted(l.lineNo(), l.product().name(), qty,
Money.fmt(l.gross()),
                    l.voided() ? "  VOID" : ""));
        }
        sb.append("  %-46s %12s%n".formatted("Gross", Money.fmt(breakdown.gross())));
        for (Discount d : breakdown.discounts())
            sb.append("  %-46s %12s%n".formatted("  " + d.description(), "-" + Money.fmt(d.amount())));
        sb.append("  %-46s %12s%n".formatted("TOTAL", Money.fmt(breakdown.total())));
        breakdown.taxByCategory().forEach((cat, tax) ->
                sb.append("  %-46s %12s%n".formatted("  incl. tax " + cat + " @" + cat.ratePercent + "%",
Money.fmt(tax))));
        for (Payment p : payments) sb.append("  %-46s %12s%n".formatted("Paid " + describe(p.tender()),
Money.fmt(p.tender().amount())));
        if (change.signum() > 0) sb.append("  %-46s %12s%n".formatted("Change", Money.fmt(change)));
```

```
        return sb.toString();
    }

    private static String describe(Tender t) {
        return switch (t) {
            case Cash c -> "cash";
            case Card c -> "card **" + c.last4();
            case Upi u -> "UPI " + u.vpa();
        };
    }
}

/** Receipts from every register, plus what has already been returned against each (shared, thread-safe). */
final class ReceiptStore {
    private final Map<String, Receipt> receipts = new ConcurrentHashMap<>();
    private final Map<String, BigDecimal> returned = new ConcurrentHashMap<>();   // key receiptId/sku
    private final AtomicLong nextId = new AtomicLong(1);

    String newId() { return "R%05d".formatted(nextId.getAndIncrement()); }
    void save(Receipt r) { receipts.put(r.id(), r); }
    Receipt get(String id) {
        Receipt r = receipts.get(id);
        if (r == null) throw new PosException("Unknown receipt " + id);
        return r;
    }

    /** Atomically claims qty units for return (two counters cannot refund the same unit twice); returns the total
claimed so far. */
    BigDecimal claimReturn(Receipt r, String sku, BigDecimal qty, BigDecimal bought) {
        return returned.compute(r.id() + "/" + sku, (k, already) -> {
            BigDecimal next = (already == null ? BigDecimal.ZERO : already).add(qty);
            if (next.compareTo(bought) > 0)
                throw new PosException("Only " +
bought.subtract(next.subtract(qty)).stripTrailingZeros().toPlainString()
                        + " of " + sku + " left to return on " + r.id());
            return next;
        });
    }
}

/**
 * Facade for one checkout lane. A register is driven by one cashier (one thread); the things it
 * shares with other registers — catalog, inventory, receipts — are the thread-safe ones.
 */
final class Register {
    private final String id;
    private final Catalog catalog;
    private final PricingEngine engine;
    private final Inventory inventory;
    private final ReceiptStore receipts;
    private final PaymentGateway gateway;
    private Sale sale;
    private int saleNo = 1;

    Register(String id, Catalog catalog, PricingEngine engine, Inventory inventory, ReceiptStore receipts,
PaymentGateway gateway) {
        this.id = id; this.catalog = catalog; this.engine = engine;
        this.inventory = inventory; this.receipts = receipts; this.gateway = gateway;
    }

    void startSale() {
        if (sale != null && (sale.status() == SaleStatus.OPEN || sale.status() == SaleStatus.TENDERING))
            throw new PosException("Finish or cancel sale " + sale.id + " first");
        sale = new Sale(id + "-S" + saleNo++);
    }

    LineItem scan(String code) { return scan(code, 1); }

    LineItem scan(String code, int count) {
        Product p = lookup(code);
        if (p.unit() != PricingUnit.EACH) throw new PosException(p.name() + " is sold by weight: put it on the
scale");
        return current().add(p, BigDecimal.valueOf(count));
    }
```

```
    /** Loose produce: the cashier keys the PLU and the scale sends the weight in grams. */
    LineItem weigh(String plu, int grams) {
        Product p = lookup(plu);
        if (p.unit() != PricingUnit.PER_KG) throw new PosException(p.name() + " is not sold by weight");
        if (grams <= 0) throw new PosException("Scale reading must be positive");
        return current().add(p, BigDecimal.valueOf(grams).movePointLeft(3));
    }

    void voidLine(int lineNo) { current().voidLine(lineNo); }
    void coupon(String code) { current().addCoupon(code); }
    PriceBreakdown total() { return current().beginTender(engine); }
    void backToScanning() { current().reopen(); }
    void cancelSale() { current().cancel(gateway); }

    /** Returns the receipt when the bill is fully paid, else null (more tenders needed). */
    Receipt pay(Tender t) {
        Sale s = current();
        if (!s.pay(t, gateway)) return null;
        for (SkuTotal item : s.breakdown().items()) inventory.adjust(item.product().sku(), item.quantity().negate());
        Receipt r = new Receipt(receipts.newId(), id, s.lines(), s.breakdown(), s.payments(), s.change());
        receipts.save(r);
        return r;
    }

    /** Refund what the customer actually paid per unit (after offers and coupons), not the shelf price. */
    BigDecimal refund(String receiptId, String sku, int qty) {
        Receipt r = receipts.get(receiptId);
        SkuTotal bought = r.breakdown().items().stream().filter(t -> t.product().sku().equals(sku)).findFirst()
                .orElseThrow(() -> new PosException(sku + " is not on receipt " + receiptId));
        BigDecimal q = BigDecimal.valueOf(qty);
        BigDecimal after = receipts.claimReturn(r, sku, q, bought.quantity());
        // refund = share(after) - share(before): the parts always add up to exactly what was paid
        BigDecimal net = r.breakdown().netBySku().get(sku);
        BigDecimal refund = share(net, after, bought.quantity()).subtract(share(net, after.subtract(q),
bought.quantity()));
        inventory.adjust(sku, q);
        return refund;
    }

    private static BigDecimal share(BigDecimal net, BigDecimal units, BigDecimal of) {
        return net.multiply(units).divide(of, 2, RoundingMode.HALF_UP);
    }

    private Product lookup(String code) {
        if (code.length() == 13 && !Ean13.valid(code)) throw new PosException("Bad barcode " + code + ": check digit
wrong, rescan");
        return catalog.find(code).orElseThrow(() -> new PosException("Unknown code " + code + ": call a supervisor"));
    }

    private Sale current() {
        if (sale == null) throw new PosException("No sale in progress");
        return sale;
    }
}

/** EAN-13 check digit: weights 1,3,1,3... on the first 12 digits; the 13th makes the sum a multiple of 10. */
final class Ean13 {
    private Ean13() { }
    static String withCheckDigit(String first12) {
        int sum = 0;
        for (int i = 0; i < 12; i++) sum += (first12.charAt(i) - '0') * (i % 2 == 0 ? 1 : 3);
        return first12 + (10 - sum % 10) % 10;
    }
    static boolean valid(String code) {
        return code.chars().allMatch(Character::isDigit) && withCheckDigit(code.substring(0, 12)).equals(code);
    }
}
```

```java
// File: lld/ch19/grocery/Main.java
package lld.ch19.grocery;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    /** Fake card/UPI network: declines one blocked card, approves everything else. */
    static final class FakeGateway implements PaymentGateway {
        private final AtomicInteger seq = new AtomicInteger(1000);
        @Override public String authorize(Tender t) {
            if (t instanceof Card c && c.last4().equals("0005")) throw new PosException("Card **0005 declined by
issuer");
            return "AUTH" + seq.incrementAndGet();
        }
        @Override public void refund(String auth, BigDecimal amount) {
            System.out.println("   gateway: reversed " + Money.fmt(amount) + " on " + auth);
        }
    }

    static final String MILK = Ean13.withCheckDigit("890100000001"), PANEER = Ean13.withCheckDigit("890100000002"),
            CHIPS = Ean13.withCheckDigit("890100000003"), BISCUITS = Ean13.withCheckDigit("890100000004"),
            DETERGENT = Ean13.withCheckDigit("890100000005"), BANANAS = "4011", TOMATOES = "4664";

    public static void main(String[] args) throws Exception {
        Catalog catalog = new Catalog();
        catalog.add(new Product("BAN", BANANAS, "Bananas", Department.PRODUCE, PricingUnit.PER_KG, Money.of("60"),
TaxCategory.EXEMPT));
        catalog.add(new Product("TOM", TOMATOES, "Tomatoes", Department.PRODUCE, PricingUnit.PER_KG, Money.of("40"),
TaxCategory.EXEMPT));
        catalog.add(new Product("MLK", MILK, "Milk 1L", Department.DAIRY, PricingUnit.EACH, Money.of("68"),
TaxCategory.EXEMPT));
        catalog.add(new Product("PNR", PANEER, "Paneer 200g", Department.DAIRY, PricingUnit.EACH, Money.of("90"),
TaxCategory.REDUCED));
        catalog.add(new Product("CHP", CHIPS, "Chips 50g", Department.SNACKS, PricingUnit.EACH, Money.of("20"),
TaxCategory.STANDARD));
        catalog.add(new Product("BIS", BISCUITS, "Biscuits", Department.SNACKS, PricingUnit.EACH, Money.of("30"),
TaxCategory.REDUCED));
        catalog.add(new Product("DET", DETERGENT, "Detergent 1kg", Department.HOUSEHOLD, PricingUnit.EACH,
Money.of("245"), TaxCategory.STANDARD));

        List<ItemPromotion> offers = List.of(
                new BuyXGetYFree("CHP", 2, 1),                       // saves 20 on 3 packs
                new MultiBuy("CHP", 3, Money.of("50")),               // saves 10 on 3 packs: loses to the one above
                new MultiBuy("BIS", 3, Money.of("80")),
                new PercentOffDepartment(Department.PRODUCE, 10));
        List<BasketPromotion> coupons = List.of(new SpendThresholdCoupon("SAVE50", Money.of("500"), Money.of("50")));
        PricingEngine engine = new PricingEngine(offers, coupons, new TaxInclusive());

        Inventory stock = new Inventory();
        for (String sku : List.of("BAN", "TOM", "MLK", "CHP", "BIS", "DET")) stock.set(sku, "200", "20");
        stock.set("PNR", "100", "20");
        List<String> alerts = Collections.synchronizedList(new ArrayList<>());
        stock.addListener((sku, left, why) -> alerts.add(sku + " " + why + " (on hand " +
left.stripTrailingZeros().toPlainString() + ")"));
        ReceiptStore receipts = new ReceiptStore();
        FakeGateway gateway = new FakeGateway();
        Register lane1 = new Register("LANE-1", catalog, engine, stock, receipts, gateway);

        System.out.println("== 1. Scan a basket ==");
        lane1.startSale();
        lane1.weigh(BANANAS, 1245);
        lane1.weigh(TOMATOES, 500);
        lane1.scan(MILK);
        lane1.scan(MILK);
        lane1.scan(PANEER);
        lane1.scan(CHIPS, 3);
```

```
        lane1.scan(BISCUITS, 4);
        lane1.scan(DETERGENT);
        lane1.voidLine(4);                                      // customer puts one milk back
        for (String bad : List.of(Ean13.withCheckDigit("890100000009"), "8901000000018", BANANAS)) {
            try { lane1.scan(bad); } catch (PosException e) { System.out.println("   error: " + e.getMessage()); }
        }
        lane1.coupon("SAVE50");

        System.out.println("\n== 2. Total, then split payment: declined card, UPI, cash with change ==");
        PriceBreakdown bill = lane1.total();
        System.out.println("   amount due " + Money.fmt(bill.total()));
        try { lane1.scan(MILK); } catch (PosException e) { System.out.println("   error: " + e.getMessage()); }
        try { lane1.pay(new Card("0005", Money.of("300"))); } catch (PosException e) { System.out.println("   error: "
+ e.getMessage()); }
        try { lane1.pay(new Upi("asha@okbank", Money.of("5000"))); } catch (PosException e) { System.out.println("
error: " + e.getMessage()); }
        System.out.println("   UPI 400 -> receipt? " + lane1.pay(new Upi("asha@okbank", Money.of("400"))));
        Receipt r1 = lane1.pay(new Cash(Money.of("500")));
        System.out.print(r1.render());

        System.out.println("\n== 3. Returns refund what was really paid ==");
        System.out.println("   return 1 of 3 chips -> refund " + Money.fmt(lane1.refund(r1.id(), "CHP", 1)));
        System.out.println("   return 2 more chips -> refund " + Money.fmt(lane1.refund(r1.id(), "CHP", 2))
                + "  (all three add up to the " + Money.fmt(r1.breakdown().netBySku().get("CHP")) + " paid)");
        try { lane1.refund(r1.id(), "CHP", 1); } catch (PosException e) { System.out.println("   error: " +
e.getMessage()); }
        try { lane1.refund(r1.id(), "PNR", 2); } catch (PosException e) { System.out.println("   error: " +
e.getMessage()); }

        System.out.println("\n== 4. Price change during a sale; going back to scanning; cancelling ==");
        lane1.startSale();
        lane1.scan(MILK);
        catalog.reprice(MILK, Money.of("72"));
        lane1.scan(MILK);
        SkuTotal milk = lane1.total().items().get(0);
        System.out.println("   2 milk scanned around a reprice: gross " + Money.fmt(milk.gross()) + " = Rs 68 (old
price kept) + Rs 72 (new)");
        lane1.backToScanning();
        lane1.scan(PANEER);
        System.out.println("   after adding paneer, due " + Money.fmt(lane1.total().total()));
        lane1.pay(new Card("4242", Money.of("100")));
        try { lane1.backToScanning(); } catch (PosException e) { System.out.println("   error: " + e.getMessage()); }
        lane1.cancelSale();
        try { lane1.pay(new Cash(Money.of("10"))); } catch (PosException e) { System.out.println("   error: " +
e.getMessage()); }

        System.out.println("\n== 5. Same basket, US-style tax added at the till ==");
        Sale same = new Sale("X");
        r1.lines().forEach(l -> { if (!l.voided()) same.add(l.product(), l.quantity()); });
        same.addCoupon("SAVE50");
        PriceBreakdown us = same.beginTender(new PricingEngine(offers, coupons, new TaxExclusive()));
        System.out.println("   inclusive total " + Money.fmt(r1.breakdown().total()) + " vs exclusive total " +
Money.fmt(us.total())
                + " (tax " + us.taxByCategory() + ")");

        System.out.println("\n== 6. Four registers sell 30 paneer each at once; stock was 100 ==");
        CountDownLatch go = new CountDownLatch(1);
        try (ExecutorService pool = Executors.newFixedThreadPool(4)) {
            for (int i = 1; i <= 4; i++) {
                Register lane = new Register("LANE-" + (i + 1), catalog, engine, stock, receipts, gateway);
                pool.submit(() -> {
                    go.await();
                    for (int k = 0; k < 30; k++) {
                        lane.startSale();
                        lane.scan(PANEER);
                        lane.total();
                        lane.pay(new Cash(Money.of("100")));
                    }
                    return null;
                });
            }
            go.countDown();
        }
        System.out.println("   paneer on hand: " + stock.onHand("PNR") + "  (sold 120 + 1 earlier; the till never
```

```
refuses a real item)");
        alerts.stream().sorted().forEach(a -> System.out.println("   alert: " + a));
    }
}
```

How the code works, part by part

- **Scanning.** `Register.scan` checks the EAN-13 check digit (a last digit computed from the others, so a misread code fails the sum) first (a misread barcode is caught before lookup), then refuses weighed products; `weigh` converts grams to kilograms with `movePointLeft(3)` so 1245 g becomes exactly `1.245`. `Sale.add` copies the product — and therefore its price — into the new `LineItem`.
- **Pricing.** `PricingEngine.price` skips voided lines, groups by SKU, asks every item offer for a discount and keeps the largest (capped at the gross), applies the best basket coupon to what is left, spreads that coupon across SKUs, and computes tax per category on the discounted amounts. With `TaxInclusive`, tax = amount × r ÷ (100 + r); with `TaxExclusive`, tax = amount × r ÷ 100 and is added to the total.
- **Paying.** `beginTender` freezes the breakdown. `pay` treats cash specially: if it is more than the amount due, the extra becomes change. Card and UPI may not exceed the amount due and must be authorised; a declined authorisation throws before anything is recorded. When the amount due reaches zero, the sale completes and the register writes the receipt and reduces stock.
- **Returns.** `ReceiptStore.claimReturn` uses `ConcurrentHashMap.compute` so "check how many were already returned, then add" is atomic. The refund for a return is `share(after) − share(before)`, where `share(k)` is the net paid for the first *k* units; that way three separate one-item returns add up to exactly what was paid.
- **Stock.** `Inventory.adjust` uses `merge`, which is atomic per key; it compares the value before and after *this* change, so each crossing of the reorder point — the stock level at which more should be ordered — (or of zero) alerts exactly once, however many registers sell at the same time.

Output:

```
== 1. Scan a basket ==
   error: Unknown code 8901000000095: call a supervisor
   error: Bad barcode 8901000000018: check digit wrong, rescan
   error: Bananas is sold by weight: put it on the scale

== 2. Total, then split payment: declined card, UPI, cash with change ==
   amount due Rs 588.23
   error: Cannot scan when the sale is TENDERING
   error: Card **0005 declined by issuer
   error: Card/UPI cannot be more than the amount due Rs 588.23
   UPI 400 -> receipt? null
  ---------------- RECEIPT R00001 (LANE-1) ----------------
   1 Bananas            1.245 kg @ Rs 60.00/kg       Rs 74.70
   2 Tomatoes           0.500 kg @ Rs 40.00/kg       Rs 20.00
   3 Milk 1L            1 x Rs 68.00                 Rs 68.00
   4 Milk 1L            1 x Rs 68.00                 Rs 68.00  VOID
   5 Paneer 200g        1 x Rs 90.00                 Rs 90.00
   6 Chips 50g          3 x Rs 20.00                 Rs 60.00
   7 Biscuits           4 x Rs 30.00                Rs 120.00
   8 Detergent 1kg      1 x Rs 245.00               Rs 245.00
  Gross                                            Rs 677.70
    10% off PRODUCE: Bananas                        -Rs 7.47
    10% off PRODUCE: Tomatoes                       -Rs 2.00
    Buy 2 get 1 free: Chips 50g                    -Rs 20.00
    3 for Rs 80.00: Biscuits                       -Rs 10.00
    Coupon SAVE50                                  -Rs 50.00
  TOTAL                                            Rs 588.23
    incl. tax EXEMPT @0%                            Rs 0.00
    incl. tax REDUCED @5%                           Rs 8.78
    incl. tax STANDARD @18%                        Rs 40.07
  Paid UPI asha@okbank                             Rs 400.00
  Paid cash                                        Rs 500.00
  Change                                           Rs 311.77

== 3. Returns refund what was really paid ==
   return 1 of 3 chips -> refund Rs 12.29
   return 2 more chips -> refund Rs 24.58  (all three add up to the Rs 36.87 paid)
   error: Only 0 of CHP left to return on R00001
   error: Only 1 of PNR left to return on R00001

== 4. Price change during a sale; going back to scanning; cancelling ==
   2 milk scanned around a reprice: gross Rs 140.00 = Rs 68 (old price kept) + Rs 72 (new)
   after adding paneer, due Rs 230.00
   error: Payments already taken; cancel them first
   gateway: reversed Rs 100.00 on AUTH1002
   error: Cannot pay when the sale is CANCELLED

== 5. Same basket, US-style tax added at the till ==
   inclusive total Rs 588.23 vs exclusive total Rs 644.73 (tax {EXEMPT=0.00, REDUCED=9.22, STANDARD=47.28})

== 6. Four registers sell 30 paneer each at once; stock was 100 ==
   paneer on hand: -21  (sold 120 + 1 earlier; the till never refuses a real item)
   alert: PNR LOW - reorder (on hand 20)
   alert: PNR NEGATIVE - recount the shelf (on hand -1)
```

Reading the output: the misread barcode (wrong check digit), an unknown code and scanning bananas without the scale are all refused. On the receipt, chips qualify for both "buy 2 get 1 free" (saves ₹20) and "3 for ₹50" (saves ₹10); only the better one is applied. The ₹50 coupon applied because the basket was still over ₹500 after item offers. The total ₹588.23 contains ₹8.78 + ₹40.07 of tax; the same basket under a US-style policy costs ₹644.73. The declined card and the over-large UPI amount changed nothing; UPI ₹400 plus cash ₹500 gave ₹311.77 change. Returning the chips one, then two, refunded ₹12.29 + ₹24.58 = ₹36.87 — exactly the net paid after the offer and the coupon share, not the shelf price of ₹60. The re-priced milk shows that a line keeps the price it was scanned at. Four registers selling 30 paneer each drove stock from 99 to −21 without refusing anyone, and the low and negative alerts each fired once.

Concurrency & edge cases

Case	Handling
Four registers sell the same product at once	`Inventory.adjust` uses `ConcurrentHashMap.merge` (atomic per key); no lost updates
Stock count says zero but the item is in hand	Sale goes through; stock goes negative; one "recount" alert
Alert spam when stock is low	Edge-triggered: alert only on the change that crosses the line
Two service desks refund the same receipt line at once	`ReceiptStore.claimReturn` uses `compute`: check-and-add is atomic; the second over-claim is refused
Price changes during a sale	Lines keep the price copied at scan; the frozen breakdown fixes the amount due
Offer changes during payment	Breakdown frozen at `total()`; going back to scanning re-prices
Card declined	Exception from the gateway before anything is recorded
Cancel after a card payment	`cancel` reverses every non-cash payment through the gateway
Rounding pennies in coupon allocation and refunds	Remainder to the largest SKU; refunds as differences of cumulative shares
One register used by one cashier	`Register` and `Sale` are single-threaded by design; shared objects (`Catalog`, `Inventory`, `ReceiptStore`) are the thread-safe ones

Real-world example

Supermarket tills identify loose produce by **PLU codes** — 4011 is the standard code for yellow bananas, and tomatoes have their own codes in the 4000 range — and read the weight from an integrated scale, exactly like `weigh(plu, grams)`. Packaged goods carry EAN-13 barcodes whose last digit is a check digit, which is why a smudged barcode is rejected rather than misread as another product. Receipts in tax-inclusive countries (for example GST — Goods and Services Tax — invoices in India or VAT — value-added tax — receipts in the UK) print how much tax the total *contains*, while US receipts add sales tax below the subtotal — our two `TaxPolicy` strategies. Retailers also run periodic "cycle counts" to fix the kind of negative stock our `Inventory` reports.

Extensions & follow-up questions

- **"Loyalty points and member prices."** A `CustomerContext` passed to the engine; member-only offers check it. Points are earned on the *net* paid (after discounts), from the receipt.
- **"Weighed items with a pre-packed label."** In-store printed barcodes that encode the price or weight in the digits (a variable-measure barcode); the scanner decodes them into a `LineItem` directly.
- **"Offers that span products (buy pasta + sauce, save ₹20)."** A `BundlePromotion` that looks at several SKUs; the "one offer per unit" rule then needs a small optimiser (try the combinations, keep the best) instead of best-per-SKU.
- **"Refund to the original payment method."** Store tender splits on the receipt; refund card/UPI parts through the gateway first, then cash.
- **"The network is down."** Offline mode: queue card authorisations (with a floor limit — a maximum amount accepted without online approval), keep selling, reconcile (match up the records) later; the catalog and offers are cached locally in each register.
- **"End-of-day."** A `Shift` per cashier: opening float (the cash put in the drawer at the start), cash counted at close, expected cash = float + cash sales − change − cash refunds.
- **"Self-checkout."** The same `Register` facade with a weight-check step at the bagging area (weight mismatch → assistance needed).

In the interview

Plan for 45 minutes: clarify offers, tax, tenders, returns (5) → entities and the Register facade (8) → sale state machine (5) → **pricing engine** with the stacking rules (12) → split payment and receipt (7) → returns, stock and concurrency (8).

> **30-second answer:** "A Register facade drives one Sale through OPEN, TENDERING, COMPLETED or CANCELLED. Lines copy the price at scan time; produce is priced by weight from the scale. A pure pricing engine groups lines by SKU, gives each SKU its single best offer, applies a basket coupon, spreads it over SKUs so tax and refunds are exact, and computes tax through a pluggable inclusive or exclusive policy. The price is frozen at total; tenders are a sealed type, cash can give change, card and UPI go through a gateway. On completion an immutable receipt is stored and shared stock is reduced atomically, with edge-triggered alerts — and the till never refuses a real item."

Follow-ups with answers

- *Why not apply all offers that match?* Stacking can give away more than intended or even a negative price; the business has to choose a rule, and "best single offer per product, then one basket coupon" is simple and explainable.
- *Why spread the coupon over items?* Tax differs per category and returns must refund the price actually paid for that item.
- *Why is the pricing engine stateless?* So every register can share it and the same basket always gives the same answer — also what makes returns reproducible.

Self-check

1. **Q:** Three chips qualify for "buy 2 get 1 free" and "3 for ₹50". What is applied? **A:** Only the larger discount (₹20), because each SKU gets its single best item offer.
2. **Q:** Why does a return of one chip refund ₹12.29, not ₹20? **A:** The refund is the net price paid — after the offer and the item's share of the coupon.
3. **Q:** What stops two service desks refunding the same unit? **A:** `ConcurrentHashMap.compute` makes "check already returned, add" one atomic step.
4. **Q:** Why is stock allowed to go negative? **A:** The physical item is in the customer's hand; the count is wrong, so we sell and raise a recount alert.
5. **Q:** When are prices frozen, and why? **A:** At `total()`, so price or offer changes during payment cannot change the amount due.
6. **Q:** Which patterns appear? **A:** Strategy (offers, tax, payment gateway), Observer (stock alerts), Facade (Register), enum state machine (sale), sealed types (tenders).

TL;DR

- **Register facade** → one **Sale** state machine; lines copy prices; produce priced by **weight** (PLU + scale).
- **Pure pricing engine**: best item offer per SKU → basket coupon → spread over SKUs → tax per category via a **TaxPolicy** strategy.
- **Freeze** prices at total; **split tender** with a sealed `Tender`; declined cards change nothing; cancel reverses non-cash payments.
- Returns refund the **net paid**, claimed atomically; stock updates are atomic, may go negative, alerts are **edge-triggered**.

Chapter summary

Problem	Hard part	The move that solves it
Amazon	Guests vs members; last-item races; cancel rules	Ordering only on `Member` (LSP); atomic all-or-nothing stock reservation; order state machine
Brokerage	Four order types; tax lots; over-spending	Two abstract questions on `Order`; `StockLot` + FIFO `LotSale`; reserved funds
Stock exchange	Fair matching	Per-symbol `TreeMap<price, FIFO queue>`; price then time; one lock per symbol
Auction	Concurrent bids	All bid rules in one per-listing critical section; notify after unlocking
Music streaming	Shuffle/repeat without losing order	Play order as a strategy-produced permutation of indexes; plan rules in the enum
Food delivery / FoodKart	Capacity and riders under concurrency	Reserve kitchen slots atomically; CAS-claim riders; strategies for assignment and selection
Grocery POS	Weighed items, stacking offers, split tender, returns	Pure pricing engine (best offer per SKU, coupon spread, tax per category); frozen price; atomic return claims; edge-triggered stock alerts

Patterns that kept coming back: Strategy (payments, play order, rider assignment, restaurant selection, trade-price policy, grocery offers and tax), Observer (notifications, fills, plays), state machines in enums (orders, auctions, player), Template Method (statements), Facade (every `*Service`), and one concurrency rule — **make check-then-act atomic** (lock, or compare-and-set).

Sources combined in this chapter: the DesignGurus *Grokking the Object-Oriented Design Interview* lessons on Amazon and the stock brokerage (requirements, actors, staged class design and diagrams); the awesome-low-level-design problem statements for the auction system, music streaming service and food delivery service; and the kumaransg/LLD machine-coding statements for the Navi stock exchange and Flipkart's FoodKart. All Java code is original. The grocery point-of-sale case is designed from first principles, using public retail conventions (GS1 EAN-13 check digits, IFPS PLU codes for produce, tax-inclusive vs sales-tax receipts).

Chapter 20 — Case Studies: Pattern-Driven Components

> **Goal.** The problems in this chapter are not whole products. They are **building blocks** that almost every real system contains: a ledger that splits money, a service that sends notifications, a cache, a matcher that pairs riders with drivers, a logger, a message broker and a rate limiter. Each one is small enough to finish in an interview, and each one is **anchored on one or two design patterns**. If you can name the pattern and say what change it absorbs, you have already answered half of the question.

Why "components" deserve their own chapter. A component is a part that other parts of a system *use*. It has a small public surface (a few methods), and a lot of hidden rules behind it. That makes it the perfect interview question: the interviewer can keep asking "what if this rule changes?" and a good design answers with "one new class, nothing else touched".

#	Component	Anchor pattern(s)	The change the pattern absorbs
20.1	Splitwise (expense sharing)	**Strategy** (split rules)	A new way to split a bill
20.2	Notification service	**Template Method** (retry loop), **Strategy** (send rules), **Observer** (outcomes), Simple Factory (channels)	A new channel, a new reason not to send, a new thing that watches results
20.3	In-memory cache (+ LRU from scratch)	**Strategy** (eviction policy), **Observer** (removal listener), Builder	A new eviction rule
20.4	Ride sharing (Uber)	**Strategy** (matching, pricing, payment), **Decorator** (surge), State machine (trip)	A new matching rule, surge on/off, a new payment method
20.5	Logging framework	**Chain of Responsibility** (logger hierarchy), **Singleton** (LogManager), Strategy (formatter), Decorator (async appender)	A new sink, a new format, async writing
20.6	Pub-Sub system	**Observer / Publish-Subscribe**, append-only log with offsets	New subscribers, replay, slow consumers
20.7	Rate limiter	**Strategy** (algorithm), Simple Factory, Composite	A new algorithm, a new tier, combined limits

Key terms — the patterns named above. A **design pattern** is a named, reusable shape for solving a design problem that keeps coming back. In one line each: **Strategy** puts each interchangeable rule in its own class behind one interface (a list of methods with no code, which several classes implement in their own way); **Template Method** writes the fixed steps of an algorithm once in a parent class and lets subclasses fill in one or two steps; **Observer** lets objects sign up to be told when something happens; **Simple Factory** is one place that decides which class to create; **Builder** assembles an object with many optional settings step by step; **Decorator** wraps an object in another object of the same type to add behaviour; a **state machine** is a fixed set of states plus the allowed moves between them; **Chain of Responsibility** passes a request along a line of handlers; **Singleton** guarantees exactly one instance of a class; **Composite** lets a group of objects be used as if it were one; **Publish-Subscribe** sends messages through a middleman so senders and receivers never know each other. An **append-only log with offsets** is a list that is only ever added to at the end, where each entry is found by its position number (its offset).

How every case is laid out (the case-study template used in Chapters 14–21): *Problem & clarifying questions → Requirements → Use cases → Core entities → Class diagram → Key flows → Design decisions & patterns used → Java implementation (+ real output) → Concurrency & edge cases → Extensions & follow-up questions → Real-world example → In the interview → Self-check → TL;DR.* (*Concurrency* means several threads running at the same time, which is where most subtle bugs hide.)

Running the code. All code is Java 21, standard library only (only the classes that ship with Java itself, no extra downloads), under `code/lld/ch20/<name>/`. Every package (Java's folder-like grouping of related

classes) has a `Main` (a class with the `main` method the program starts from). From `chapters/20-case-components/code`:

```
javac --release 21 -d /tmp/lld_20 $(find . -name '*.java')
java -cp /tmp/lld_20 lld.ch20.splitwise.Main      # also: notification, cache, lrucache,
                                                  # ridesharing, carpool, logging, pubsub, ratelimiter
```

Money is always `BigDecimal`. A `double` (Java's standard binary floating-point number type, fast but approximate for decimals) cannot hold 0.1 exactly, so sums drift (`0.1 + 0.2` prints `0.30000000000000004`). `BigDecimal` stores decimal digits exactly. Two rules to remember: compare amounts with `compareTo` (not `equals`, because `equals` also compares the scale (the number of digits after the decimal point), so `2.0` and `2.00` are "different"), and always say how to round when you divide (`divide(x, 2, RoundingMode.HALF_EVEN)`), or Java throws an `ArithmeticException` for results like 1/3. (An **exception** is Java's way of signalling an error: normal execution stops and the error travels up to whoever catches it.)

Reading the course diagrams. The figures marked "(course diagram)" are UML diagrams (UML, the Unified Modeling Language, is the standard box-and-arrow notation for drawing classes and how they relate) and use one arrow legend throughout:

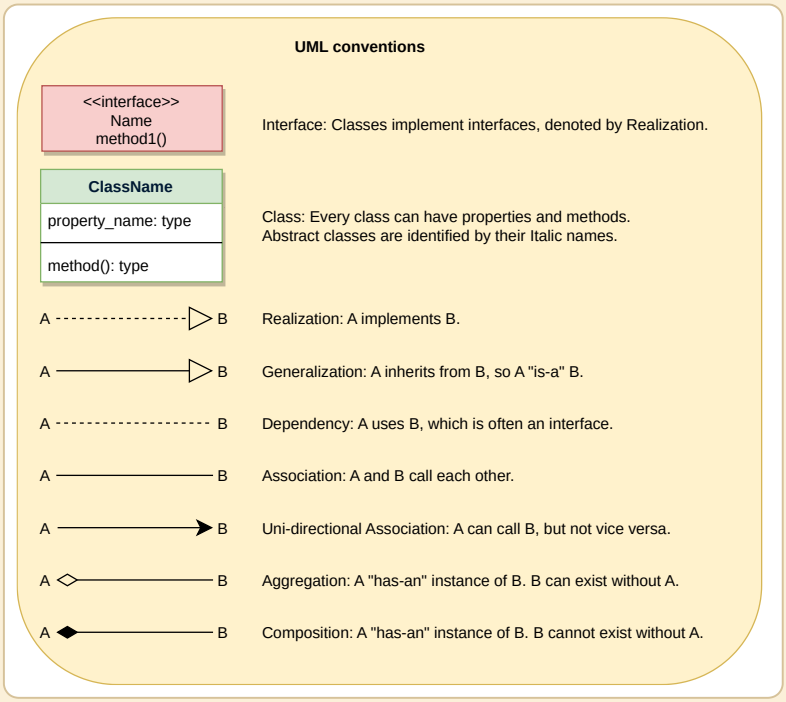


Figure — UML conventions used in the course class diagrams: interface, class, realization (implements), generalization (inherits), dependency, association, one-way association, aggregation and composition (course diagram)

The mermaid diagrams (Mermaid is a text format that a renderer turns into pictures) are drawn from **this chapter's Java code**, so they match the code line by line.

20.1 Design Splitwise

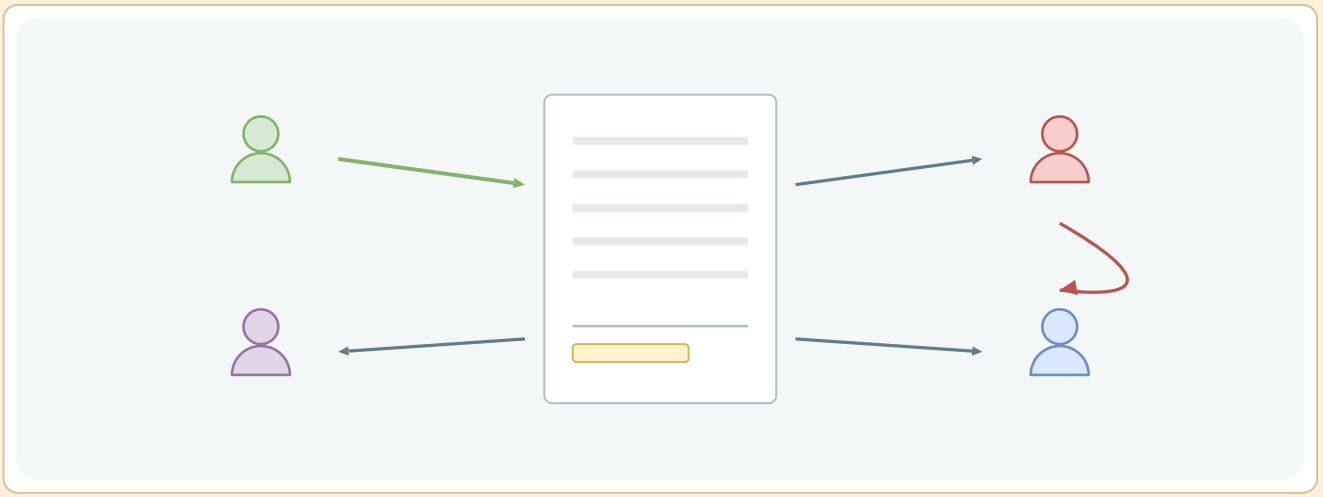


Figure — Splitwise in one picture: people send expenses into one shared record, and the record tells each person what they owe (course diagram)

Problem & clarifying questions

What it is. Splitwise is an app that remembers who paid for what when friends share costs, and tells everyone who owes whom. Everyday analogy: a weekend away with four friends. The hotel goes on Alice's card, Bob covers dinner, Carol gets the cab. By Sunday evening no one can reconstruct the totals. Splitwise is the shared notebook that does the maths.

The hard part is **not** the UI (user interface: the screens people tap). The real decision is *which class owns the money arithmetic*, so the books still balance after every single operation.

Questions to ask the interviewer (with the answers we assume):

Question	Assumed answer
Which split types?	Equal, exact amounts, percentage. More may come later.
Must shares add up to the total exactly?	Yes. If they don't, the whole expense is rejected and no balance changes.
Rounding?	Amounts are money with 2 decimal places. No cent may be lost.
Groups, or only one-to-one?	Both. A group is just a name plus a member list (a trip, a shared flat).
Can a person pay for only themselves?	No. That is not shared spending: at least two people (the payer counts) must be involved.
Settle up?	Yes: record a real payment that reduces a debt.
Simplify debts?	Yes: on request, replace many small debts with a shorter list of transfers.
Multiple currencies, receipts, comments, undo?	Out of scope (say so explicitly).

Requirements

Functional

1. Every user is identified by a name and an email.
2. Users form groups; a group has a name and members.
3. An expense records a description, total, who paid and who shares it.
4. Split types: equal, exact, percent.
5. Shares must add up to the total; otherwise the expense is refused and **nothing changes**.
6. At least two distinct people (the payer plus the sharers) must be involved in an expense.
7. Balances between every pair of people are tracked continuously.

8. A user can query their balance with one specific person, and their overall net balance.
9. Settle up: a real payment reduces a debt.
10. A group can request a simplified settlement plan with fewer transfers.
11. Rounding never loses or creates a cent.
12. Expenses are stored permanently, so a group's history can be listed.

Non-functional

- Correctness of money above everything (exact decimals, no drift).
- Adding a new split type must not change existing classes (Open/Closed Principle).
- Thread-safe: two people adding expenses at the same time must not corrupt the ledger. (A **thread** is one independent line of execution inside a running program; code is **thread-safe** when several threads can use it at the same moment without corrupting its data.)

Use cases

Actors:

- **Member** — adds expenses, settles up, asks what they owe.
- **Group creator** — sets up a group and invites its members.
- **System** — works out each share, checks shares add up, updates the ledger, simplifies debts.

Main use cases: *Add an expense* (which includes *Work out each share*, *Check the shares add up*, *Update the ledger*), *Settle up*, *See what I owe*, *Create a group / add people*, *Simplify the debts*.

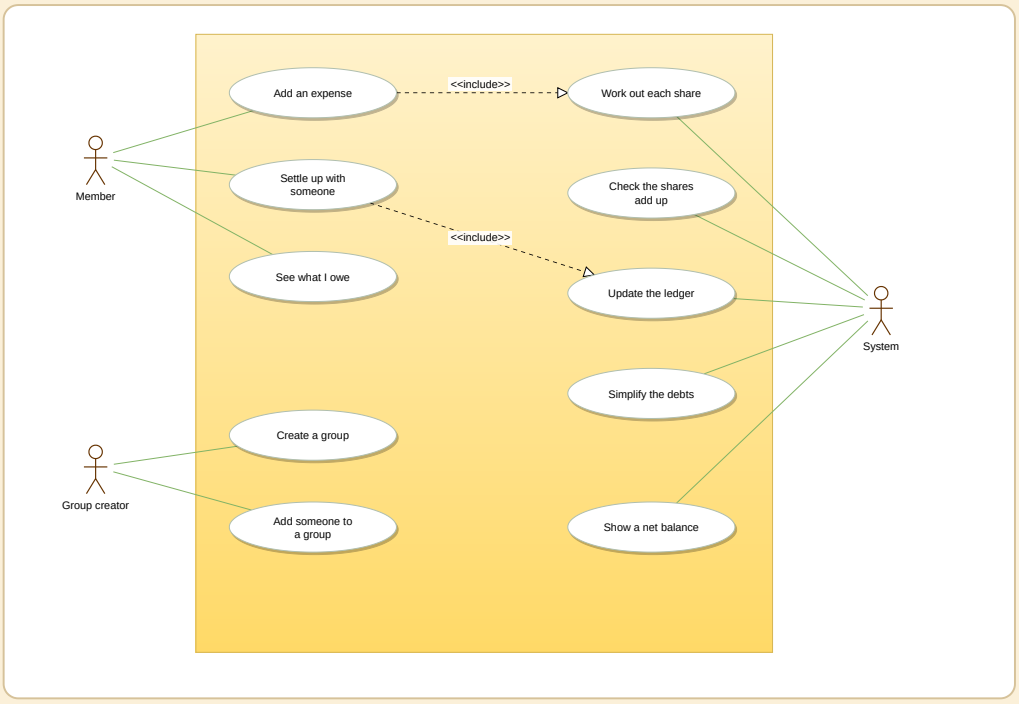


Figure — Splitwise use cases: members add expenses and settle up; the system computes shares, validates them, updates the ledger and simplifies debts (course diagram)

In the figure above, "Add an expense" *includes* the three system steps. "Include" in a use-case diagram means the step always happens as part of the bigger use case.

Core entities

Class	Responsibility
`User` (record)	Identity: id, name, email. No money.
`Group`	A named set of member ids + the list of its expense ids. **Holds no balances.**
`SplitStrategy` (interface)	Turns a total into one `Share` per person.
`EqualSplit`, `ExactSplit`, `PercentSplit`	The three rules.
`Share` (record)	A (person, amount) pair for a single expense (a value object).
`Expense`	Description, total, payer, shares. Validates itself once, when created.
`Ledger`	Who owes whom: exactly **one signed number per pair** of people. One mutator: `adjust`.
`DebtSimplifier`	Greedy minimum-cash-flow: turns net positions into a short list of payments.
`Payment` (record)	One suggested transfer (a suggestion, not a record of money moving).
`Splitwise`	Facade: the one class the app talks to. Orders the steps, contains no arithmetic.
`Money`	Tiny helper: 2-decimal `BigDecimal` values.

A few words from the table: a **record** is Java's short syntax for a class that only carries data, whose fields cannot change after creation; a **value object** is a small object defined only by its values (two shares with the same person and amount are interchangeable); a **mutator** is a method that changes an object's state; a **Facade** is one simple front class that hides several classes behind it; and a **greedy** algorithm makes the best-looking choice at each step without reconsidering earlier choices.

Why the group holds no money. If each group kept its own balances, a person in two groups would have "what Bob owes Alice" stored in two places. Sooner or later the two copies disagree. One ledger, one number per pair, is the single source of truth.

Why `Group → User` is association, not composition. Association = one object refers to another that lives independently. Composition = the part is owned and dies with the whole. Deleting a group must not delete its people, so it is association.

Class diagram

The course builds the model in five stages, and the order is worth copying in an interview because every stage leaves a slice that already works (run out of time and you still have something that runs, not half a diagram):

Stage	Adds	Works afterwards
1	`User`, `Group`	You can say who shares costs
2	`SplitStrategy` + three rules, `Share`	A total becomes one amount per person
3	`Expense` (self-checking)	A bill that does not add up is rejected
4	`Ledger`	Pairwise balances are tracked
5	`Splitwise` facade + `DebtSimplifier`	A whole trip can be recorded, settled and simplified

Here is the finished course diagram:

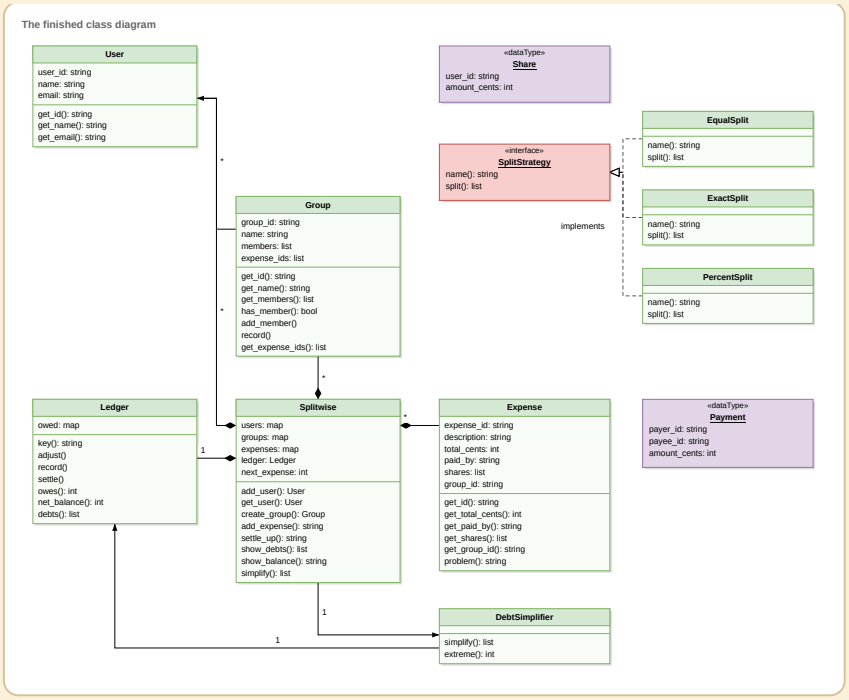
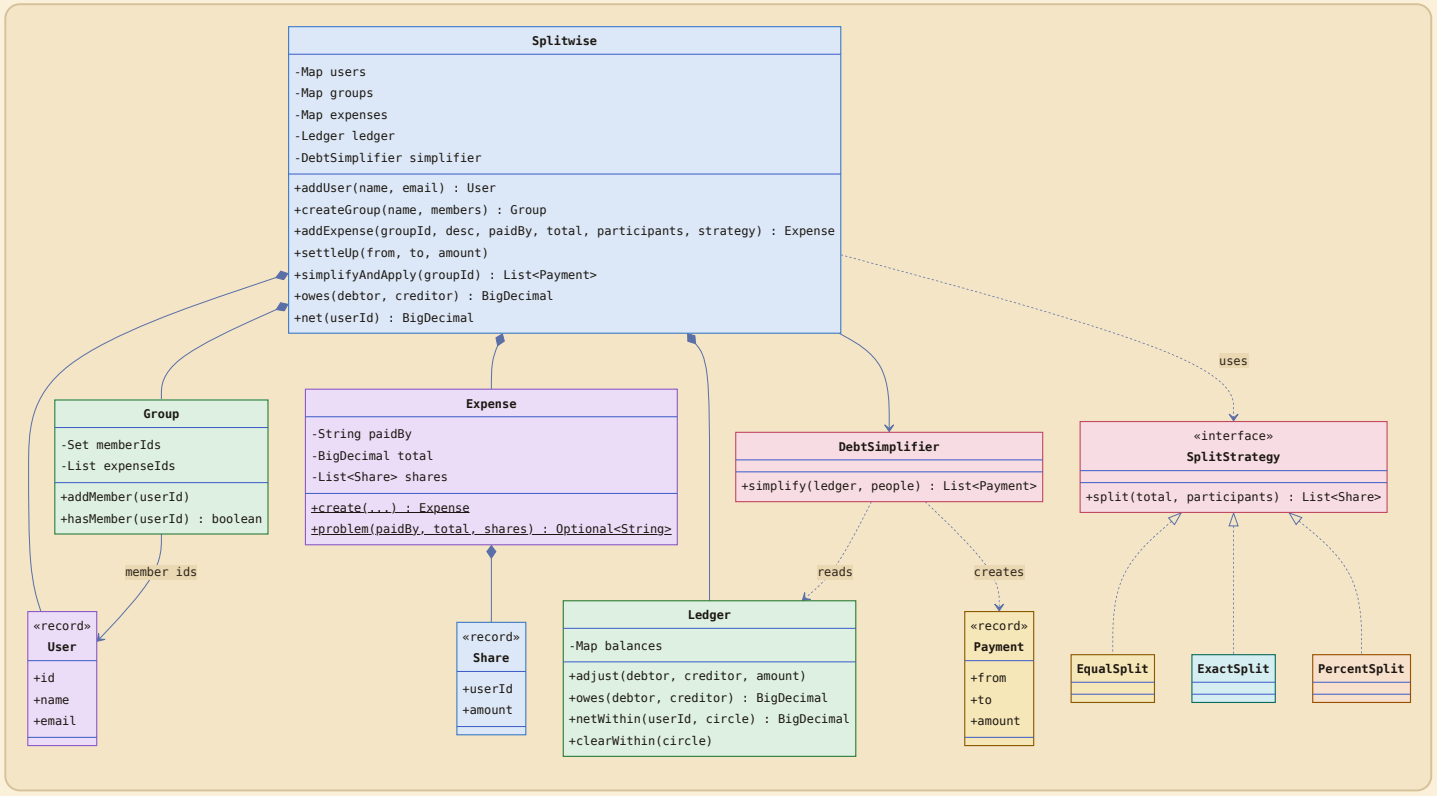


Figure — The finished Splitwise class diagram: service owning users, groups, expenses and the ledger, with the split strategies and the simplifier (course diagram)

In the figure above, the three split rules meet in one hollow triangle: they all realize the same interface. The service's filled diamonds are composition: it owns users, groups, expenses and the ledger.

Our Java code, as a class diagram:



Key flows

Adding an expense. Nothing touches the ledger until both checks pass.

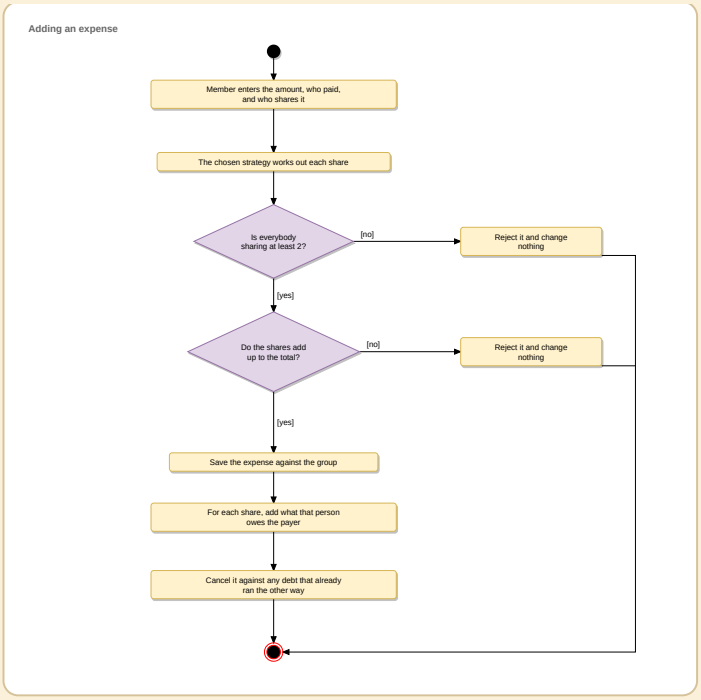
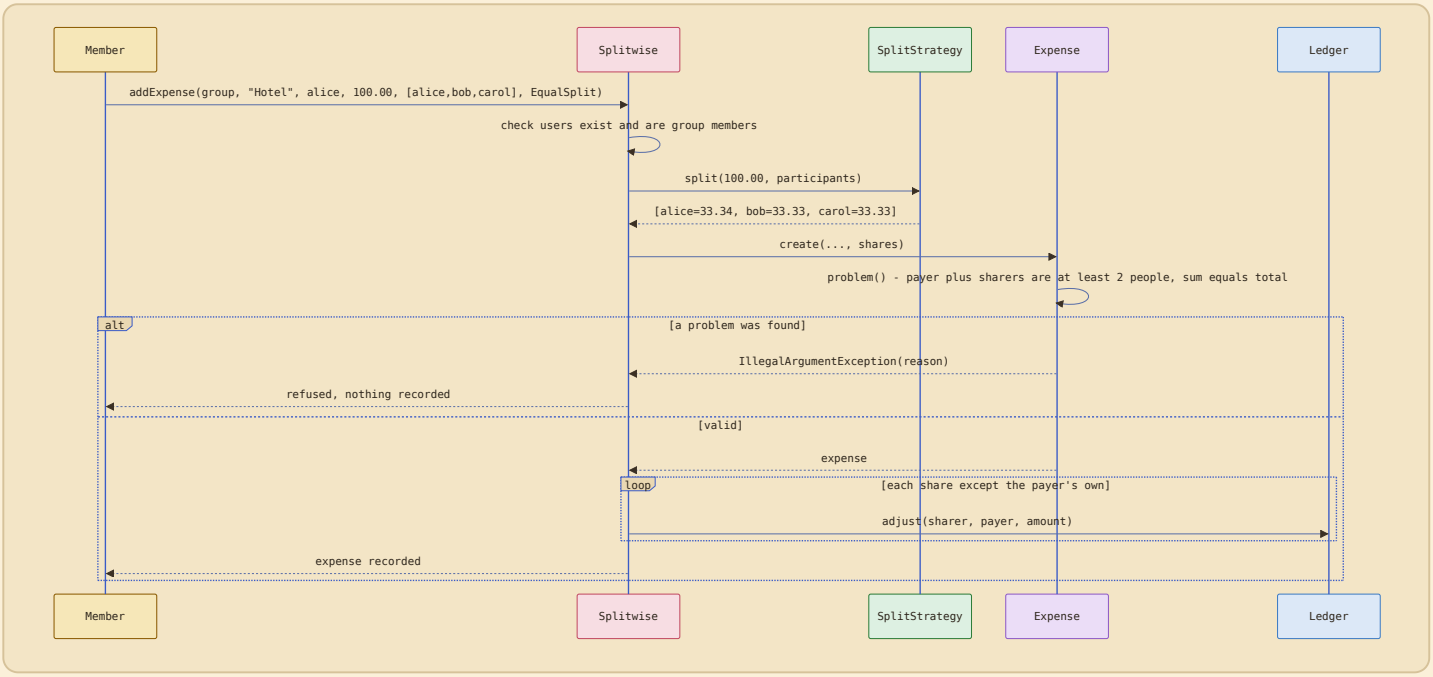


Figure — Adding an expense: the strategy computes shares, the expense checks itself, and only then is the ledger updated (course diagram)



Simplifying a group's debts (minimum cash flow, greedy).

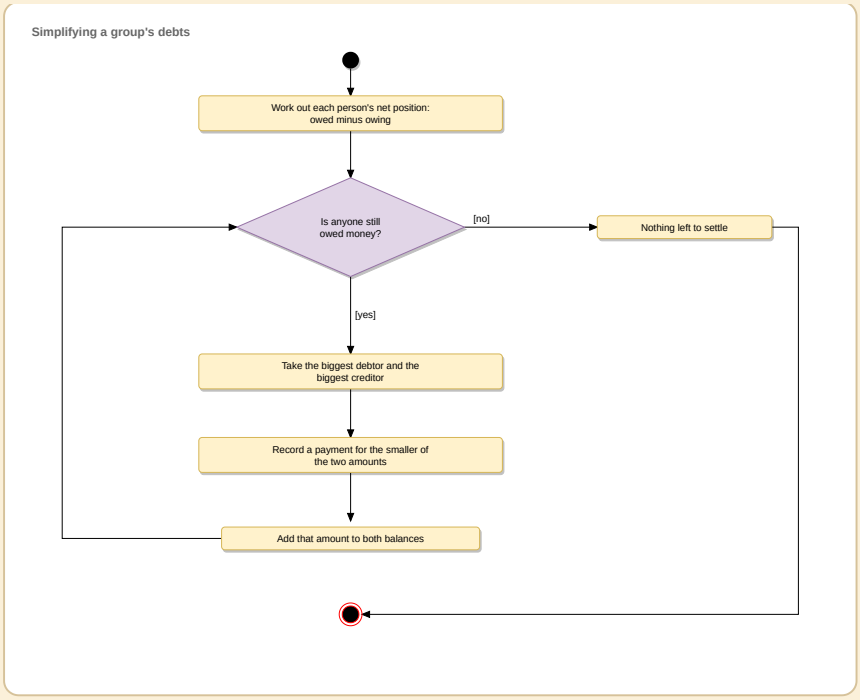


Figure — Simplifying debts: compute net positions, then the largest debtor repeatedly pays the largest creditor until every balance is zero (course diagram)

How it works, step by step:

1. For each member, compute the **net position** = (what others owe them) − (what they owe others). Positive = creditor, negative = debtor. The nets always sum to zero.
2. Put creditors in one max-heap and debtors in another (by amount; ties by name, so the output is deterministic). A **max-heap** is a priority queue that always hands back its largest item first; **deterministic** means the same input always gives the same output.
3. Pop the biggest creditor C and biggest debtor D. Let x = min(C, D). Suggest "D pays C x".
4. Push back whichever side still has a remainder.
5. Stop when a heap is empty (everyone is at zero).

Each round brings at least one person to zero, and the final round brings two to zero, so there are **at most n − 1 payments** for n people. It is **not always the true minimum**: finding the minimum number of transfers is NP-hard (no fast algorithm for it is known: the work needed explodes as the number of people grows; it is related to splitting people into as many zero-sum subgroups as possible). Say that before you are asked.

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for split rules (`SplitStrategy`)	The rule arrives from outside and is interchangeable. A fourth rule (by shares, by adjustment) is a new class. Every rule returns the same `List<Share>`, so nothing downstream knows which rule ran.	`if (type == EQUAL) ... else if (type == EXACT)` in the service: every new type edits old, tested code.
Expense validates itself in a **static factory** (a static method that builds the object instead of `new`; here `Expense.create`)	Checked once, when created; after that every class can trust its shares. An invalid expense can never exist.	Checking in the ledger: re-checks on every read and still lets bad objects exist.
`problem()` returns the **reason**, not a boolean	The user sees *why* ("shares add up to 15.00 but the total is 30.00").	`boolean isValid()`: loses the reason.
Ledger stores **one signed number per unordered pair**	A debt is never stored twice, so it can never disagree with itself. Money moving the other way cancels automatically.	Two maps "A owes B" and "B owes A": two places for the truth.
Ledger has **one mutator** (`adjust`)	Expenses, settle-ups and simplification all go through it, so the cancel rule exists in exactly one place.	Separate `addDebt`, `settle`, `reduce` methods each with their own arithmetic.
`DebtSimplifier` separate from `Ledger`	Single Responsibility (each class has one reason to change): *keeping* balances and *planning* settlements change for different reasons.	Putting the algorithm inside the ledger.
`BigDecimal` with explicit rounding; spare cents handed out one at a time	100.00 / 3 = 33.33 × 3 = 99.99 would lose a cent. Equal split gives the extra cent to the first people; percent split lets the last person absorb the remainder.	`double`: 0.1 + 0.2 ≠ 0.3. (The course keeps whole cents in a `long`; that is equally correct. We use `BigDecimal` as this book's standard.)
`Splitwise` is a **Facade**	One entry point; it orders the steps and decides nothing.	UI code calling ledger and strategies directly.

Only one named GoF pattern here (Strategy). (GoF, the "Gang of Four", are the four authors of the 1994 book that catalogued 23 classic design patterns.) That is fine. Good designs do not collect patterns; they use the one that absorbs the change that is likely.

Java implementation

```java
// File: code/lld/ch20/splitwise/Model.java
package lld.ch20.splitwise;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.Collections;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Objects;
import java.util.Set;

/** Money helpers: every amount is a BigDecimal with exactly 2 decimal places. */
final class Money {
    static final BigDecimal ZERO = new BigDecimal("0.00");
    static final BigDecimal CENT = new BigDecimal("0.01");
    static final BigDecimal HUNDRED = new BigDecimal("100");

    private Money() {}

    static BigDecimal of(String amount) {
        return new BigDecimal(amount).setScale(2, RoundingMode.UNNECESSARY); // refuses 10.005
    }

    static BigDecimal normalize(BigDecimal amount) {
        return amount.setScale(2, RoundingMode.HALF_EVEN);
    }
}

/** A person. Identity only - no money lives here. */
record User(String id, String name, String email) {
    User {
        Objects.requireNonNull(id);
        Objects.requireNonNull(name);
    }
}

/** A named set of people. Holds member ids and expense ids, never balances. */
final class Group {
    private final String id;
    private final String name;
    private final Set<String> memberIds = new LinkedHashSet<>();
    private final List<String> expenseIds = new ArrayList<>();

    Group(String id, String name) {
        this.id = id;
        this.name = name;
    }

    String id() { return id; }
    String name() { return name; }
    void addMember(String userId) { memberIds.add(userId); }
    boolean hasMember(String userId) { return memberIds.contains(userId); }
    Set<String> members() { return Collections.unmodifiableSet(memberIds); }
    void recordExpense(String expenseId) { expenseIds.add(expenseId); }
    List<String> expenseIds() { return Collections.unmodifiableList(expenseIds); }
}

/** One person's part of one expense (a value object). */
record Share(String userId, BigDecimal amount) {
    @Override
    public String toString() { return userId + "=" + amount; }
}

/** A suggested transfer produced by the simplifier - a suggestion, not a record. */
record Payment(String from, String to, BigDecimal amount) {
    @Override
    public String toString() {
        return from + " pays " + to + " " + amount;
```

```
    }
}
```

```java
// File: code/lld/ch20/splitwise/SplitStrategy.java
package lld.ch20.splitwise;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Map;

/** Strategy: turns one total into one Share per participant. */
interface SplitStrategy {
    List<Share> split(BigDecimal total, List<String> participants);
}

/** Equal split. Spare cents are handed out one at a time, in participant order. */
final class EqualSplit implements SplitStrategy {
    @Override
    public List<Share> split(BigDecimal total, List<String> participants) {
        if (participants.isEmpty()) throw new IllegalArgumentException("nobody to split between");
        BigDecimal n = BigDecimal.valueOf(participants.size());
        BigDecimal base = total.divide(n, 2, RoundingMode.DOWN);          // 100.00/3 -> 33.33
        BigDecimal leftover = total.subtract(base.multiply(n));           // 0.01
        int spareCents = leftover.divide(Money.CENT).intValueExact();     // 1
        List<Share> shares = new ArrayList<>();
        for (int i = 0; i < participants.size(); i++) {
            BigDecimal amount = i < spareCents ? base.add(Money.CENT) : base;
            shares.add(new Share(participants.get(i), amount));
        }
        return shares;
    }
}

/** Exact split: the caller states each person's amount. Expense checks the sum. */
final class ExactSplit implements SplitStrategy {
    private final Map<String, BigDecimal> amounts;

    ExactSplit(Map<String, BigDecimal> amounts) { this.amounts = Map.copyOf(amounts); }

    @Override
    public List<Share> split(BigDecimal total, List<String> participants) {
        if (!new HashSet<>(participants).equals(amounts.keySet()))
            throw new IllegalArgumentException("exact amounts must name exactly the participants");
        List<Share> shares = new ArrayList<>();
        for (String id : participants) shares.add(new Share(id, Money.normalize(amounts.get(id))));
        return shares;
    }
}

/** Percent split. Percentages must add to 100; the last person absorbs rounding. */
final class PercentSplit implements SplitStrategy {
    private final Map<String, BigDecimal> percents;

    PercentSplit(Map<String, BigDecimal> percents) { this.percents = Map.copyOf(percents); }

    @Override
    public List<Share> split(BigDecimal total, List<String> participants) {
        if (!new HashSet<>(participants).equals(percents.keySet()))
            throw new IllegalArgumentException("percentages must name exactly the participants");
        BigDecimal sum = percents.values().stream().reduce(BigDecimal.ZERO, BigDecimal::add);
        if (sum.compareTo(Money.HUNDRED) != 0)
            throw new IllegalArgumentException("percentages add up to " + sum + ", not 100");
        List<Share> shares = new ArrayList<>();
        BigDecimal given = Money.ZERO;
        for (int i = 0; i < participants.size(); i++) {
            String id = participants.get(i);
            BigDecimal amount = (i == participants.size() - 1)
                    ? total.subtract(given)                                   // absorbs the remainder
                    : total.multiply(percents.get(id)).divide(Money.HUNDRED, 2, RoundingMode.HALF_EVEN);
            given = given.add(amount);
            shares.add(new Share(id, amount));
        }
        return shares;
```

```
    }
}
```

```java
// File: code/lld/ch20/splitwise/Expense.java
package lld.ch20.splitwise;

import java.math.BigDecimal;
import java.util.HashSet;
import java.util.List;
import java.util.Optional;
import java.util.Set;

/** What was bought, what it cost, who paid, and each share. Validated once, at creation. */
final class Expense {
    private final String id;
    private final String description;
    private final String paidBy;
    private final BigDecimal total;
    private final List<Share> shares;
    private final String groupId; // null for an expense between friends outside any group

    private Expense(String id, String description, String paidBy, BigDecimal total,
                    List<Share> shares, String groupId) {
        this.id = id;
        this.description = description;
        this.paidBy = paidBy;
        this.total = total;
        this.shares = List.copyOf(shares);
        this.groupId = groupId;
    }

    /** Factory: builds a valid expense or throws with the reason it is invalid. */
    static Expense create(String id, String description, String paidBy, BigDecimal total,
                          List<Share> shares, String groupId) {
        problem(paidBy, total, shares).ifPresent(p -> { throw new IllegalArgumentException(p); });
        return new Expense(id, description, paidBy, total, shares, groupId);
    }

    /** Why these shares cannot form an expense, or empty if they can. */
    static Optional<String> problem(String paidBy, BigDecimal total, List<Share> shares) {
        if (total.signum() <= 0) return Optional.of("total must be positive");
        Set<String> sharers = new HashSet<>();
        for (Share s : shares) {
            if (s.amount().signum() < 0) return Optional.of("negative share for " + s.userId());
            if (!sharers.add(s.userId())) return Optional.of("duplicate share for " + s.userId());
        }
        Set<String> people = new HashSet<>(sharers);
        people.add(paidBy); // the payer counts as one of the two people even if they owe no share
        if (people.size() < 2) return Optional.of("an expense needs at least two people");
        BigDecimal sum = shares.stream().map(Share::amount).reduce(BigDecimal.ZERO, BigDecimal::add);
        if (sum.compareTo(total) != 0)
            return Optional.of("shares add up to " + sum + " but the total is " + total);
        return Optional.empty();
    }

    String id() { return id; }
    String description() { return description; }
    String paidBy() { return paidBy; }
    BigDecimal total() { return total; }
    List<Share> shares() { return shares; }
    String groupId() { return groupId; }
}
```

```java
// File: code/lld/ch20/splitwise/Ledger.java
package lld.ch20.splitwise;

import java.math.BigDecimal;
import java.util.Collection;
import java.util.Map;
import java.util.TreeMap;

/**
 * Who owes whom. Exactly ONE signed number per unordered pair of people.
 * Key (lo, hi) with lo < hi alphabetically; a positive value means "hi owes lo".
 */
final class Ledger {
    private record Pair(String lo, String hi) {
        static Pair of(String a, String b) { return a.compareTo(b) < 0 ? new Pair(a, b) : new Pair(b, a); }
    }

    private final Map<Pair, BigDecimal> balances = new TreeMap<>(
            (p, q) -> p.lo().equals(q.lo()) ? p.hi().compareTo(q.hi()) : p.lo().compareTo(q.lo()));

    /** The ONLY mutator: 'debtor' now owes 'creditor' 'amount' more. Opposite debts cancel. */
    void adjust(String debtor, String creditor, BigDecimal amount) {
        if (debtor.equals(creditor) || amount.signum() == 0) return;
        Pair key = Pair.of(debtor, creditor);
        BigDecimal delta = creditor.equals(key.lo()) ? amount : amount.negate();
        BigDecimal updated = balances.getOrDefault(key, Money.ZERO).add(delta);
        if (updated.signum() == 0) balances.remove(key); else balances.put(key, updated);
    }

    /** Drops every debt where both people are in 'circle' (used when a simplified plan replaces them). */
    void clearWithin(Collection<String> circle) {
        balances.keySet().removeIf(p -> circle.contains(p.lo()) && circle.contains(p.hi()));
    }

    /** How much 'debtor' owes 'creditor' (never negative). */
    BigDecimal owes(String debtor, String creditor) {
        if (debtor.equals(creditor)) return Money.ZERO;
        Pair key = Pair.of(debtor, creditor);
        BigDecimal v = balances.getOrDefault(key, Money.ZERO);
        BigDecimal debtorOwes = creditor.equals(key.lo()) ? v : v.negate();
        return debtorOwes.signum() > 0 ? debtorOwes : Money.ZERO;
    }

    /** Positive = others owe this person; negative = this person owes others. */
    BigDecimal net(String userId) { return netWithin(userId, null); }

    /** Net position counting only debts with people in 'circle' (null = everybody). */
    BigDecimal netWithin(String userId, Collection<String> circle) {
        BigDecimal net = Money.ZERO;
        for (var e : balances.entrySet()) {
            Pair p = e.getKey();
            String other = p.lo().equals(userId) ? p.hi() : p.hi().equals(userId) ? p.lo() : null;
            if (other == null || (circle != null && !circle.contains(other))) continue;
            // positive value: hi owes lo, so lo is up and hi is down
            net = p.lo().equals(userId) ? net.add(e.getValue()) : net.subtract(e.getValue());
        }
        return net;
    }

    /** Human-readable list of every open debt: "B owes A 10.00". */
    void print() {
        if (balances.isEmpty()) System.out.println("  (everyone is settled up)");
        balances.forEach((p, v) -> {
            if (v.signum() > 0) System.out.println("  " + p.hi() + " owes " + p.lo() + " " + v);
            else System.out.println("  " + p.lo() + " owes " + p.hi() + " " + v.negate());
        });
    }
}
```

```java
// File: code/lld/ch20/splitwise/DebtSimplifier.java
package lld.ch20.splitwise;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.Collection;
import java.util.Comparator;
import java.util.List;
import java.util.PriorityQueue;

/**
 * Greedy "minimum cash flow": largest debtor pays largest creditor, repeat.
 * Uses at most (people - 1) payments. Not always the true minimum (that problem is NP-hard).
 */
final class DebtSimplifier {
    private record Position(String userId, BigDecimal amount) {}

    List<Payment> simplify(Ledger ledger, Collection<String> people) {
        // Biggest amount first; ties broken by name so results are deterministic.
        Comparator<Position> biggestFirst = Comparator.comparing(Position::amount).reversed()
                .thenComparing(Position::userId);
        PriorityQueue<Position> creditors = new PriorityQueue<>(biggestFirst);
        PriorityQueue<Position> debtors = new PriorityQueue<>(biggestFirst);
        for (String id : people) {
            BigDecimal net = ledger.netWithin(id, people);
            if (net.signum() > 0) creditors.add(new Position(id, net));
            else if (net.signum() < 0) debtors.add(new Position(id, net.negate()));
        }
        List<Payment> payments = new ArrayList<>();
        while (!creditors.isEmpty() && !debtors.isEmpty()) {
            Position c = creditors.poll();
            Position d = debtors.poll();
            BigDecimal x = c.amount().min(d.amount());
            payments.add(new Payment(d.userId(), c.userId(), x));
            if (c.amount().compareTo(x) > 0) creditors.add(new Position(c.userId(), c.amount().subtract(x)));
            if (d.amount().compareTo(x) > 0) debtors.add(new Position(d.userId(), d.amount().subtract(x)));
        }
        return payments;
    }
}
```

```java
// File: code/lld/ch20/splitwise/Splitwise.java
package lld.ch20.splitwise;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.concurrent.atomic.AtomicInteger;

/** Facade the app talks to. Orders the steps; the arithmetic lives in the other classes. */
final class Splitwise {
    private final Map<String, User> users = new LinkedHashMap<>();
    private final Map<String, Group> groups = new LinkedHashMap<>();
    private final Map<String, Expense> expenses = new LinkedHashMap<>();
    private final Ledger ledger = new Ledger();
    private final DebtSimplifier simplifier = new DebtSimplifier();
    private final AtomicInteger ids = new AtomicInteger();

    synchronized User addUser(String name, String email) {
        User u = new User(name.toLowerCase(), name, email);
        users.put(u.id(), u);
        return u;
    }

    synchronized Group createGroup(String name, String... memberIds) {
        Group g = new Group("g" + ids.incrementAndGet(), name);
        for (String m : memberIds) g.addMember(requireUser(m));
        groups.put(g.id(), g);
        return g;
    }

    /** Build shares -> build (and validate) expense -> only then touch the ledger. */
    synchronized Expense addExpense(String groupId, String description, String paidBy, BigDecimal total,
                                    List<String> participants, SplitStrategy strategy) {
        requireUser(paidBy);
        participants.forEach(this::requireUser);
        Group group = groupId == null ? null : groups.get(groupId);
        if (group != null) {
            for (String p : participants)
                if (!group.hasMember(p)) throw new IllegalArgumentException(p + " is not in " + group.name());
            if (!group.hasMember(paidBy)) throw new IllegalArgumentException(paidBy + " is not in " + group.name());
        }
        List<Share> shares = strategy.split(total, participants);       // may throw
        Expense e = Expense.create("e" + ids.incrementAndGet(), description, paidBy, total, shares, groupId);
        for (Share s : e.shares()) ledger.adjust(s.userId(), paidBy, s.amount()); // payer's own share is a no-op
        expenses.put(e.id(), e);
        if (group != null) group.recordExpense(e.id());
        return e;
    }

    /** A real payment from 'from' to 'to'. Cannot pay more than is owed. */
    synchronized void settleUp(String from, String to, BigDecimal amount) {
        BigDecimal owed = ledger.owes(from, to);
        if (amount.signum() <= 0 || amount.compareTo(owed) > 0)
            throw new IllegalArgumentException(from + " owes " + to + " only " + owed + ", cannot pay " + amount);
        ledger.adjust(to, from, amount); // the reverse debt cancels the existing one
    }

    synchronized BigDecimal owes(String debtor, String creditor) { return ledger.owes(debtor, creditor); }
    synchronized BigDecimal net(String userId) { return ledger.net(userId); }
    synchronized List<Payment> simplify(String groupId) {
        return simplifier.simplify(ledger, groups.get(groupId).members());
    }
    /** Replaces the group's tangle of debts with the simplified plan. Every net position is unchanged. */
    synchronized List<Payment> simplifyAndApply(String groupId) {
        var members = groups.get(groupId).members();
        List<Payment> plan = simplifier.simplify(ledger, members);
        ledger.clearWithin(members);
        for (Payment p : plan) ledger.adjust(p.from(), p.to(), p.amount());
        return plan;
    }
```

```
    synchronized void printLedger() { ledger.print(); }

    synchronized List<String> history(String groupId) {
        List<String> lines = new ArrayList<>();
        for (String id : groups.get(groupId).expenseIds()) {
            Expense e = expenses.get(id);
            lines.add(e.id() + " " + e.description() + " " + e.total() + " paid by " + e.paidBy() + " " + e.shares());
        }
        return lines;
    }

    private String requireUser(String id) {
        if (!users.containsKey(id)) throw new IllegalArgumentException("unknown user " + id);
        return id;
    }
}
```

```java
// File: code/lld/ch20/splitwise/Main.java
package lld.ch20.splitwise;

import java.math.BigDecimal;
import java.util.List;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        Splitwise app = new Splitwise();
        for (String n : List.of("Alice", "Bob", "Carol", "Dave")) app.addUser(n, n.toLowerCase() + "@mail.com");
        Group trip = app.createGroup("Goa trip", "alice", "bob", "carol", "dave");

        System.out.println("== 1. Equal split with a spare cent");
        Expense hotel = app.addExpense(trip.id(), "Hotel", "alice", Money.of("100.00"),
                List.of("alice", "bob", "carol"), new EqualSplit());
        System.out.println("Hotel shares: " + hotel.shares());

        System.out.println("== 2. Exact split");
        app.addExpense(trip.id(), "Dinner", "bob", Money.of("60.00"), List.of("alice", "bob", "carol"),
                new ExactSplit(Map.of("alice", Money.of("10.00"), "bob", Money.of("20.00"), "carol",
Money.of("30.00"))));

        System.out.println("== 3. Percent split");
        Expense taxi = app.addExpense(trip.id(), "Taxi", "carol", Money.of("50.00"), List.of("alice", "bob", "dave"),
                new PercentSplit(Map.of("alice", new BigDecimal("33.33"), "bob", new BigDecimal("33.33"),
                        "dave", new BigDecimal("33.34"))));
        System.out.println("Taxi shares: " + taxi.shares());

        System.out.println("== 4. Invalid expenses are refused, ledger untouched");
        tryIt(() -> app.addExpense(trip.id(), "Snacks", "dave", Money.of("30.00"), List.of("alice", "dave"),
                new ExactSplit(Map.of("alice", Money.of("10.00"), "dave", Money.of("5.00")))));
        tryIt(() -> app.addExpense(trip.id(), "Coffee", "dave", Money.of("4.00"), List.of("dave"), new EqualSplit()));
        tryIt(() -> app.addExpense(trip.id(), "Boat", "dave", Money.of("50.00"), List.of("alice", "dave"),
                new PercentSplit(Map.of("alice", new BigDecimal("60"), "dave", new BigDecimal("30")))));

        System.out.println("== 5. Who owes whom");
        app.printLedger();
        for (String u : List.of("alice", "bob", "carol", "dave")) System.out.println("  net " + u + " = " +
app.net(u));
        System.out.println("  bob owes alice: " + app.owes("bob", "alice"));

        System.out.println("== 6. Simplify the group's debts (min cash flow, greedy)");
        List<Payment> plan = app.simplifyAndApply(trip.id());
        plan.forEach(p -> System.out.println("  suggested: " + p));
        app.printLedger();
        for (String u : List.of("alice", "bob", "carol", "dave")) System.out.println("  net " + u + " = " +
app.net(u));

        System.out.println("== 7. Settle up");
        tryIt(() -> app.settleUp("dave", "alice", Money.of("999.00")));
        app.settleUp("bob", "alice", Money.of("5.00"));
        System.out.println("  after a partial payment of 5.00, bob owes alice: " + app.owes("bob", "alice"));
        app.settleUp("bob", "alice", app.owes("bob", "alice"));
        for (Payment p : plan) if (!p.from().equals("bob")) app.settleUp(p.from(), p.to(), p.amount());
        app.printLedger();

        System.out.println("== 8. Group history");
        app.history(trip.id()).forEach(h -> System.out.println("  " + h));
    }

    private static void tryIt(Runnable r) {
        try {
            r.run();
            System.out.println("  accepted");
        } catch (IllegalArgumentException e) {
            System.out.println("  refused: " + e.getMessage());
        }
    }
}
```

Walking through the code.

- `Money.of("10.005")` throws, because `RoundingMode.UNNECESSARY` refuses to silently round. Inputs with more than 2 decimals are a bug, not something to hide.
- `EqualSplit` divides with `RoundingMode.DOWN` (33.33), computes the leftover (0.01), and gives one extra cent to the first participant(s). The shares always add up exactly.
- `PercentSplit` rejects percentages that do not total 100, rounds each share with `HALF_EVEN` (banker's rounding: halves go to the even neighbour, which avoids a systematic upward bias), and lets the **last** person absorb whatever remains.
- `Ledger.Pair.of(a, b)` sorts the two ids so (alice, bob) and (bob, alice) are the same key. The sign of the stored value says who owes whom.
- `Splitwise.addExpense` builds shares, then the expense (which validates), and **only then** updates the ledger. That order is requirement 5.
- `settleUp` refuses to pay more than is owed, then calls `adjust(to, from, amount)`: a debt in the reverse direction that cancels the existing one.
- `simplifyAndApply` replaces all debts between the group's members with the simplified plan. Every net position stays the same (the output proves it).

Output:

```
== 1. Equal split with a spare cent
Hotel shares: [alice=33.34, bob=33.33, carol=33.33]
== 2. Exact split
== 3. Percent split
Taxi shares: [alice=16.66, bob=16.66, dave=16.68]
== 4. Invalid expenses are refused, ledger untouched
  refused: shares add up to 15.00 but the total is 30.00
  refused: an expense needs at least two people
  refused: percentages add up to 90, not 100
== 5. Who owes whom
  bob owes alice 23.33
  carol owes alice 16.67
  carol owes bob 13.34
  dave owes carol 16.68
  net alice = 40.00
  net bob = -9.99
  net carol = -13.33
  net dave = -16.68
  bob owes alice: 23.33
== 6. Simplify the group's debts (min cash flow, greedy)
  suggested: dave pays alice 16.68
  suggested: carol pays alice 13.33
  suggested: bob pays alice 9.99
  bob owes alice 9.99
  carol owes alice 13.33
  dave owes alice 16.68
  net alice = 40.00
  net bob = -9.99
  net carol = -13.33
  net dave = -16.68
== 7. Settle up
  refused: dave owes alice only 16.68, cannot pay 999.00
  after a partial payment of 5.00, bob owes alice: 4.99
  (everyone is settled up)
== 8. Group history
  e2 Hotel 100.00 paid by alice [alice=33.34, bob=33.33, carol=33.33]
  e3 Dinner 60.00 paid by bob [alice=10.00, bob=20.00, carol=30.00]
  e4 Taxi 50.00 paid by carol [alice=16.66, bob=16.66, dave=16.68]
```

Look at step 5 and step 6: four pairwise debts became three payments, and every net (alice +40.00, bob −9.99, carol −13.33, dave −16.68) is identical before and after.

Related machine-coding problem: The Ledger Co. (Navi)

Splitwise is not the only "ledger" question you will meet. The Geektrust / Navi machine-coding round (an interview round where you write a complete, runnable program in one or two hours) **"Ledger Co."** also keeps money records, but for **bank loans** (simple interest — interest charged only on the original amount borrowed; a fixed monthly instalment; lump-sum prepayments — extra one-off payments made early) rather than bills shared between friends. It is fully designed, coded and run in **Chapter 21, §21.6**, so it is not

repeated here. The comparison is the useful part: both keep **one authoritative record** and compute every answer from it (Splitwise: one signed balance per pair of people; Ledger Co.: one loan per bank and borrower, holding its lump-sum payments), and both use `BigDecimal` with an explicitly named rounding mode. What differs is the rule set: Splitwise's rules are about *who owes whom*, Ledger Co.'s are about *how much has been repaid by a given month*.

Concurrency & edge cases

- **Two expenses at once.** Every public method on `Splitwise` is `synchronized` (a Java keyword that lets only one thread at a time run such methods on the same object), so "build → validate → adjust ledger" is atomic (it happens as one indivisible step that no other thread can see half-done). Simple and correct for one process. At scale you would lock per group or per pair of users (a **lock** is an object only one thread can hold at a time; others wait for it), or store balances in a database and update them in one transaction (a group of database changes that either all succeed or all fail together).
- **Rounding.** Covered by spare-cent distribution and "last person absorbs". Test: 100.00 / 3 must give 33.34 + 33.33 + 33.33.
- **Payer not among the sharers** (Alice pays for Bob and Carol only, or for Bob alone): allowed. Every share becomes a debt to Alice. `Expense.problem` counts the payer as one of the "at least two people", so Alice buying Bob's lunch is a valid expense, while Dave paying only for Dave is refused.
- **Payer's own share:** `adjust(alice, alice, …)` is a no-op (does nothing) by design.
- **Overpaying on settle-up:** refused, so a debt can never flip direction by accident.
- **Unknown user / not a group member:** refused before any computation.
- **Duplicate person in the shares list, negative share, zero total:** refused by `Expense.problem`.
- **Simplify and other groups.** Because the ledger has one number per *pair*, simplifying a group rewrites all debts *between its members*, even ones that came from another group with the same two people. That is the price of the single source of truth. If per-group simplification must not touch other groups, keep the ledger per group *and* a global view derived from them.

Extensions & follow-up questions

- **"Add a split by shares (e.g. 2:1:1)."** A new `ShareRatioSplit implements SplitStrategy`. Nothing else changes.
- **"Delete or edit an expense."** Our ledger stores totals, not history, so undo means re-adding the inverse (`adjust(payer, sharer, amount)` for each share). Cleaner: make the ledger an **append-only list of entries** (like accounting journals / event sourcing — storing every change as an event and rebuilding the current state from them) and compute balances on demand or in a cache (a saved copy of computed results kept for quick reuse; see 20.3). Then delete = append a reversing entry.
- **"Multiple currencies."** Add a `Currency` to every amount (a `Money(amount, currency)` value object) and keep one ledger per currency, or convert at the time of the expense and store the rate used.
- **"Is the greedy always optimal?"** No. Counter-example: nets A −6, B −5, C +2, D +4, E +5. Greedy: A pays E 5, B pays D 4, then two more small payments to C = **4 payments**. Optimal: B pays E 5 (they cancel exactly), A pays D 4 and C 2 = **3 payments**. The optimum groups people into as many zero-sum subsets as possible, which needs an exponential search; greedy is the accepted interview answer.
- **"Isn't `Splitwise` doing too much?"** A fair criticism: it holds users, groups, expenses *and* the ledger. At larger scale, give each of the four its own repository (store) and let `Splitwise` shrink to the part that orders the steps. Say this before the interviewer does.
- **"Show a user's activity feed."** Keep expenses and settle-ups as events; the feed is a query over them.
- **"Notify people when an expense is added."** Publish an `ExpenseAdded` event to an Observer / the notification service in 20.2.

Real-world example

Splitwise really does offer a "simplify debts" option for groups: when it is on, the app shows each member a small number of payments to make instead of every pairwise debt, exactly the net-position idea above. Payment apps with "split the bill" features face the same rounding problem and solve it the same way: one person absorbs the leftover cent.

In the interview

30-second answer: "Users and groups hold no money. Each split rule is a `SplitStrategy` that returns shares. An `Expense` validates itself once: at least two people, shares add to the total. The `Ledger` keeps one signed number per pair of people and has a single `adjust` method, so debts cancel automatically. Settling up is just an adjust in the other direction. Simplification uses net positions and a greedy biggest-debtor-pays-biggest-creditor loop: at most n − 1 payments, not always the true minimum."

Tip: reach the split rules early and say the phrase "one class per split rule" while you draw them; that is the part of the question with a pattern name. Then invest your time in the ledger, because the one-number-per-pair decision is what keeps everything else short.

Suggested 40-minute plan: clarify (5) → entities + the "no money in groups" decision (5) → Strategy for splits and the cents problem (8) → Expense validation (5) → Ledger with one number per pair (7) → simplify algorithm (5) → concurrency and extensions (5).

Likely follow-ups (with answers):

- *"What if 100 does not divide by 3?"* → Hand out spare cents one by one; the total is always exact.
- *"How do you simplify debts?"* → Net positions + greedy; mention n − 1 bound and that optimal is NP-hard.
- *"Why not store balances in the group?"* → A person in two groups would have two copies of the truth.

Self-check

1. Why does `Group` hold no balances? → So each debt lives in exactly one place (the ledger); two copies would eventually disagree.
2. Split 10.00 equally among 3 people. → 3.34, 3.33, 3.33.
3. Why does `Expense.problem` return a `String` instead of a `boolean`? → So the caller can show the reason it was refused.
4. What is the maximum number of payments the greedy simplifier suggests for n people? → n − 1.
5. Why compare `BigDecimal` with `compareTo` rather than `equals`? → `equals` also compares scale, so `2.0` does not equal `2.00`.
6. Alice pays 20.00 for Bob's lunch and Bob owes the whole 20.00. Is that a valid expense? → Yes. Two people are involved (payer and sharer), even though only one of them has a share.

TL;DR

- **Strategy** for split rules; every rule returns shares.
- **Expense validates itself** once; invalid expenses never exist.
- **Ledger = one signed number per pair**, one mutator; settle-up is a reverse adjust.
- **Simplify** = net positions + greedy; ≤ n − 1 payments; optimal is NP-hard.
- **BigDecimal**, explicit rounding, no lost cents.

20.2 Design a Notification Service

Problem & clarifying questions

What it is. A notification service is the component of a product responsible for emails, text messages (SMS) and push alerts ("Your order has shipped", "Your login code is 482913"). Everyday analogy: a post office counter. Many departments hand it letters; the counter picks the right delivery method, respects "do not disturb" notes, retries if the courier fails, and keeps a receipt for every letter.

Why it is a design question. Every product grows one. It starts as one `sendEmail()` function. Then someone adds SMS. Then "don't wake me at 3 a.m.". Then marketing wants "max 2 promos per hour". Six months later

one method has fourteen `if`s and nobody dares touch it. The whole exercise is: **where should each of those additions have gone instead?**

Question	Assumed answer
Channels?	Email, SMS, push. More later (WhatsApp, Slack).
Who chooses the channel?	The user, **per kind** of notification (SMS for deliveries, email for newsletters), in order of preference.
Wording?	Templates with placeholders, e.g. `Hi {name}, it arrives {eta}.`
Quiet hours?	Yes, per user; they can wrap past midnight (22:00–07:00). Urgent messages (login codes) ignore them.
Rate limits?	At most N notifications of one kind per user per hour.
Failures?	Retry a channel up to a limit, then fall back to the next accepted channel; if all fail, record FAILED.
Audit?	Every attempt recorded: SENT, FAILED or SUPPRESSED, and why.
Sync or async? (synchronous = caller waits; asynchronous = caller moves on)	The caller should not wait for a slow SMS gateway (the provider service that actually sends texts).
Bulk sends / translations / open tracking / unsubscribe links?	Out of scope.

Requirements

Functional

1. A user has a name, email, phone number and device token (the address a push service uses for one phone).
2. Channels: email, SMS, push.
3. Each user chooses accepted channels **per kind** of notification, in order.
4. The text for each kind comes from a template with placeholders ("holes") filled in per request.
5. Users can set quiet hours; nothing normal is sent during them.
6. Urgent notifications ignore quiet hours.
7. A user receives no more than N notifications of a given kind in any hour.
8. A channel that fails is retried, but only a bounded number of times.
9. When a channel runs out of retries, the service falls back to the user's next accepted channel.
10. When no channel succeeds, the outcome is stored as FAILED.
11. Every attempt is recorded with its outcome and reason.
12. Adding a channel, **or** a new reason to hold a message back, requires no edit to the service class.

Non-functional: the caller is not blocked by slow providers (async option); thread-safe under concurrent sends; testable without waiting for real time (injected clock); a broken listener must never break delivery. An **injected** clock is one handed to the class from outside (instead of the class reading the system time itself), so a test can pass in a fake clock it moves by hand; a **listener** is an object that has asked to be told when something happens.

Use cases

Actors: **Product code** (asks for a notification), **Person** (chooses channels, sets quiet hours), **Operator** (adds channels, watches outcomes), **Service** (renders, checks rules, tries channels, retries, records).

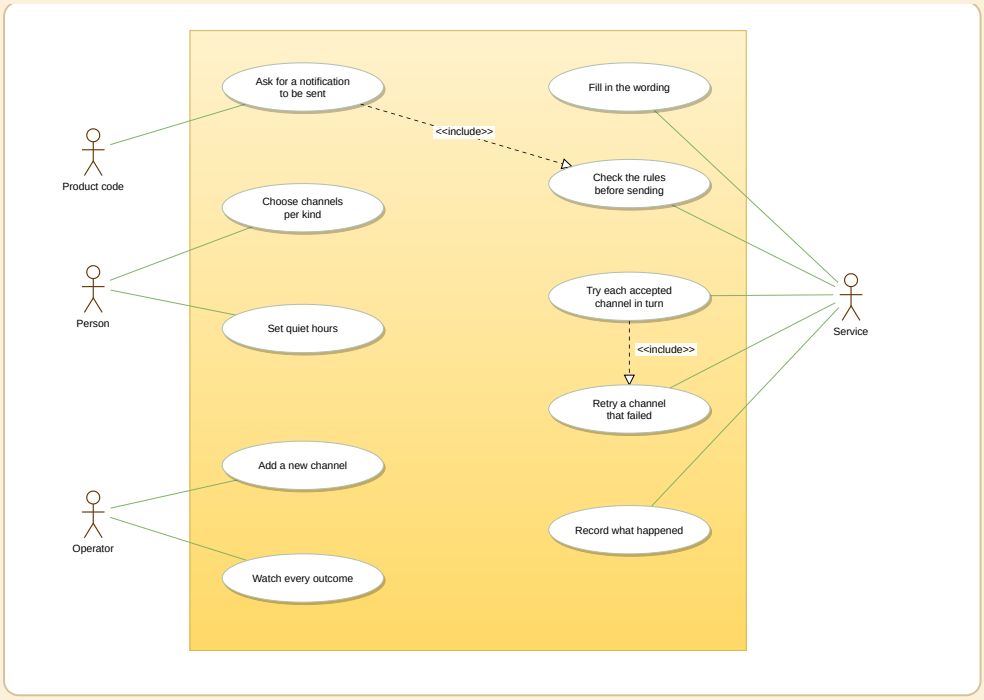


Figure — Notification service use cases: product code asks for a notification; the service checks rules, fills in the wording, tries each channel with retries and records the outcome (course diagram)

Core entities

Class	Responsibility
`User` (record)	Who someone is: id, name, email, phone, device token.
`Preferences`	What they accept: ordered channels per kind; quiet hours (with midnight wrap logic in **one** place).
`Template` (record)	Subject + body with `{holes}`; `render(values)` → `Message`.
`Message` (record)	The finished subject and body, identical on every channel.
`NotificationRequest` (record)	One ask: kind, user, values, priority (NORMAL / URGENT).
`Gateway` (interface)	The external provider (SMTP server, SMS gateway, push service). Swappable and fakeable.
`NotificationChannel` (abstract)	**Template Method**: the final `deliver()` holds the retry-with-backoff loop; subclasses supply only `addressOf(user)`.
`EmailChannel`, `SmsChannel`, `PushChannel`	Three tiny subclasses.
`ChannelFactory`	**Simple Factory**: channel type → channel object.
`RetryPolicy` (record)	Max attempts + initial backoff (doubles each time).
`SendRule` (interface)	**Strategy**: one reason not to send. Returns the reason text, or empty.
`QuietHoursRule`, `RateLimitRule`	Two rules. Quiet hours knows urgent bypasses it; rate limit counts only *successful* sends.
`DeliveryRecord` (record)	What happened: status, channel, attempts, reason.
`DeliveryListener` (interface)	**Observer**: told every outcome. `LoggingListener`, `MetricsListener`.
`TimeSource`	Where "now" comes from (a manual clock in tests).
`NotificationService`	Thin coordinator: rules → preferences → render → channels in order → `finish()`. Decides nothing itself.

Terms from the table: **SMTP** is the protocol mail servers use to pass email along; a **fake** is a pretend version of a real dependency used in tests; an **abstract** class cannot be created on its own and exists to

be extended; a `final` method cannot be overridden by subclasses; **backoff** means waiting a little longer before each new retry, so a struggling provider gets time to recover.

Why `User` and `Preferences` are separate classes. Who you are rarely changes; what you are willing to receive changes often. Separate reasons to change → separate classes (Single Responsibility Principle).

Class diagram

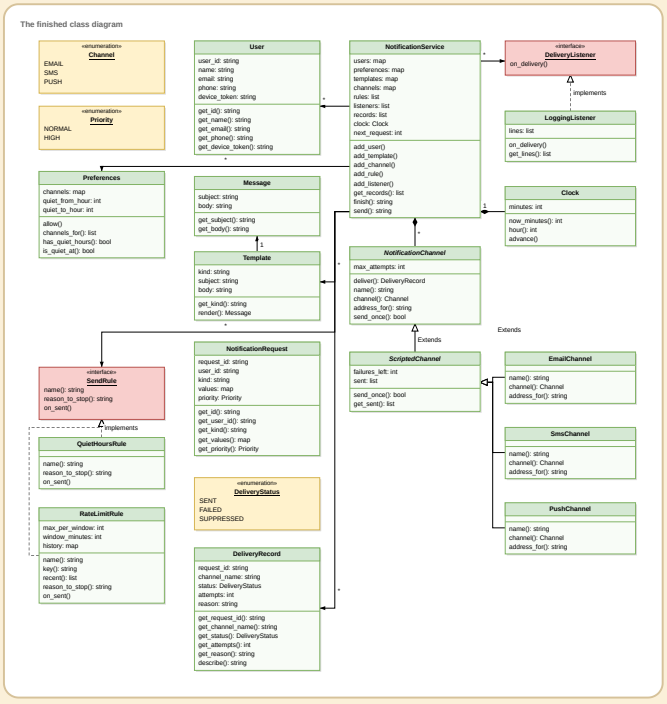
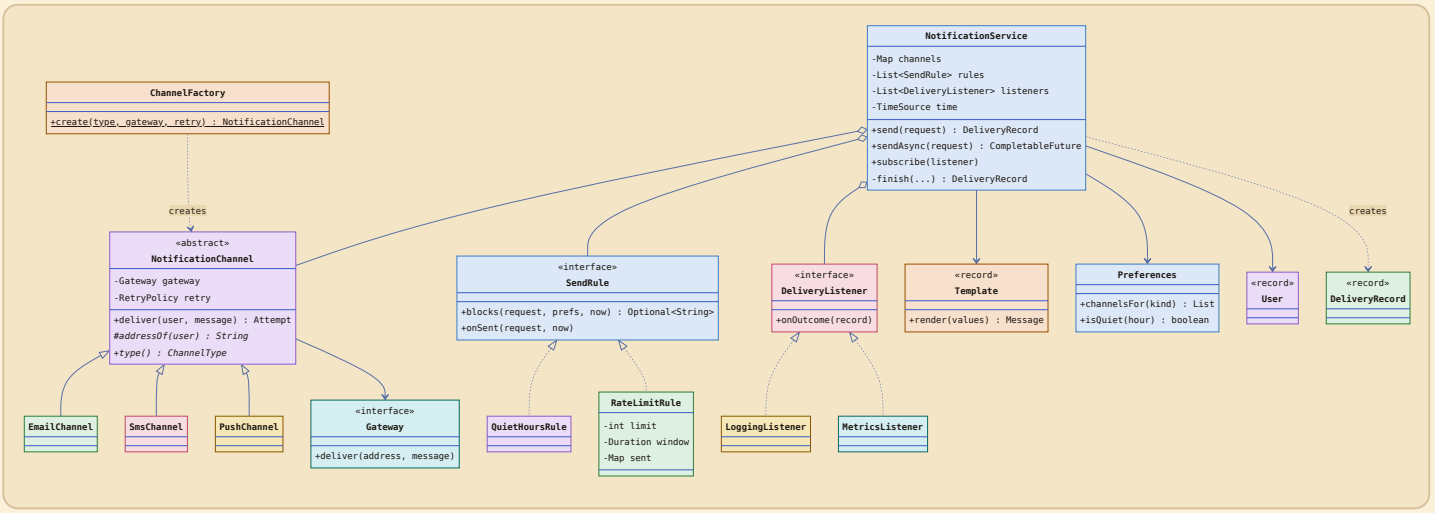


Figure — The finished notification service class diagram: the service in the middle, the channel hierarchy with the retry in the parent, the send rules, templates, preferences and the delivery listeners (course diagram)

In the figure above, notice that the three channel classes are tiny: all the behaviour that makes sending reliable lives in the abstract parent.

The course reaches this diagram in five build stages, each leaving something that already works: (1) `User` + `Preferences` (you can ask what a person accepts, and when); (2) `Template` + `Message` + `NotificationRequest` (wording filled in once); (3) the channel hierarchy (delivery with retries and an outcome); (4) the `SendRule`s (a notification can be stopped before sending); (5) `NotificationService` + listeners (end-to-end sending, observable by any listener). Reuse this order when you interview.



Key flows

Sending one notification. A rule (or a missing channel preference) may stop it before any work is done; otherwise channels are tried in the user's order until one succeeds.

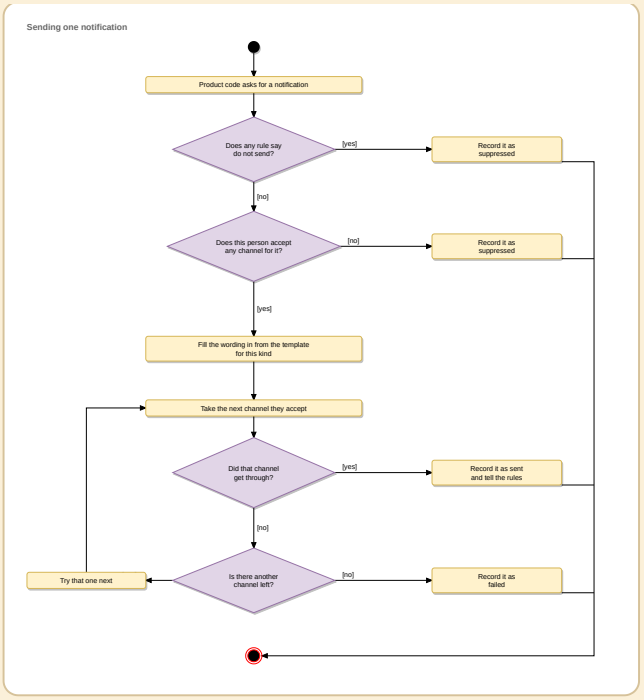


Figure — Sending one notification: ask every rule, render once, try each accepted channel in turn, and record sent / failed / suppressed (course diagram)

Inside one channel — the retry loop, which exists exactly once, in the abstract parent:

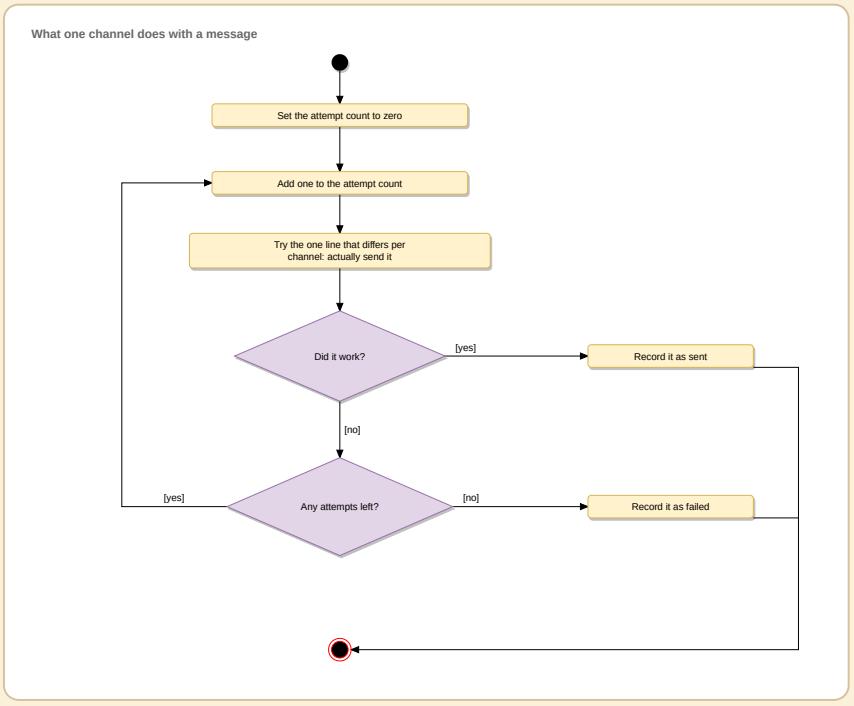
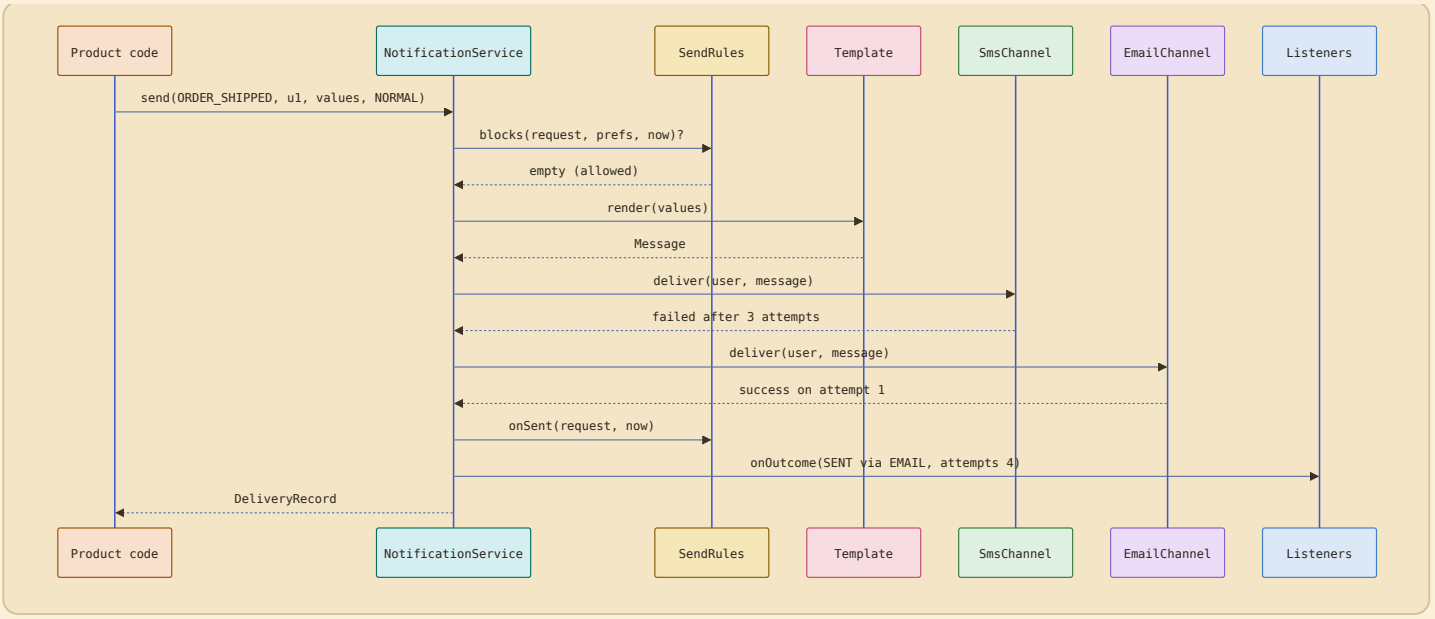


Figure — One channel's delivery loop: attempt, and on failure retry while attempts remain; then report success or give up (course diagram)


Product code
NotificationService
SendRules
Template
SmsChannel
EmailChannel
Listeners
send(ORDER_SHIPPED, u1, values, NORMAL)
blocks(request, prefs, now)?
empty (allowed)
render(values)
Message
deliver(user, message)
failed after 3 attempts
deliver(user, message)
success on attempt 1
onSent(request, now)
onOutcome(SENT via EMAIL, attempts 4)
DeliveryRecord
Product code
NotificationService
SendRules
Template
SmsChannel
EmailChannel
Listeners

Design decisions & patterns used

Pattern / decision	Where	Why	Alternative rejected
Template Method	`NotificationChannel.deliver()` is `final`; subclasses fill `addressOf()`	The retry loop is identical for all channels. Write it once; new channels inherit reliability. Only a *step* varies, not the whole algorithm.	Copying the loop into every channel (they drift apart), or Strategy (would replace the whole method, not one step).
Strategy	`SendRule`	Every "reason not to send" is its own class. A new rule (e.g. "user unsubscribed", "duplicate within 5 minutes") is a new class plus one line in the wiring code.	`if (quiet) … if (tooMany) …` inside `send`.
Observer	`DeliveryListener`	Logs, dashboards, alerts all want outcomes. The service announces; it doesn't know who listens.	Hard-coding logging/metrics in the service; it grows forever.
Simple Factory	`ChannelFactory`	One place maps `ChannelType` → class; config can list channel names.	`new SmsChannel(...)` scattered through wiring code.
Adapter (target interface; an adapter converts one API into the shape your code expects)	`Gateway`	Each real provider SDK (the vendor's client library) has its own API (the set of calls a library or service offers to programs); an adapter per provider implements `Gateway`. Also lets tests use a fake.	Channels calling vendor SDKs directly.
Rule returns **reason text**, not boolean	`SendRule.blocks`	The reason is recorded (requirement 11).	`boolean allowed()` loses why.
Urgency handled **inside** the rule	`QuietHoursRule`	The service doesn't need to know which rules urgent messages ignore.	A special case in the service.
Rate limit counts only **successful** sends	`onSent` hook	A suppressed/failed message shouldn't use up the user's hourly allowance.	Counting at check time.
Single exit `finish()`	Service	Every path stores a record and notifies listeners. A second exit that forgot would leave invisible gaps.	Returning from many places.
Template rendered **once**	Service	Every channel gets the same text; a wording change is one edit.	Wording inside each channel.

Using three patterns here is not decoration: each one soaks up a *separate* axis of change — another channel (Template Method + Factory), another reason to hold a message back (Strategy), another party interested in outcomes (Observer).

Java implementation

```java
// File: code/lld/ch20/notification/Model.java
package lld.ch20.notification;

import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

enum ChannelType { EMAIL, SMS, PUSH }

enum Priority { NORMAL, URGENT }

enum DeliveryStatus { SENT, FAILED, SUPPRESSED }

/** Who a person is: identity and addresses. */
record User(String id, String name, String email, String phone, String deviceToken) {}

/** What a person is willing to receive, and when. Changes far more often than User. */
final class Preferences {
    private final Map<String, List<ChannelType>> channelsByKind = new HashMap<>();
    private Integer quietStart; // hour of day, inclusive
    private Integer quietEnd;   // hour of day, exclusive

    Preferences accept(String kind, ChannelType... inOrder) {
        channelsByKind.put(kind, List.of(inOrder));
        return this;
    }

    Preferences quietHours(int startHour, int endHour) {
        this.quietStart = startHour;
        this.quietEnd = endHour;
        return this;
    }

    List<ChannelType> channelsFor(String kind) { return channelsByKind.getOrDefault(kind, List.of()); }

    /** Quiet hours may wrap past midnight (22 -> 7), so the comparison lives here, once. */
    boolean isQuiet(int hour) {
        if (quietStart == null) return false;
        return quietStart <= quietEnd
                ? hour >= quietStart && hour < quietEnd
                : hour >= quietStart || hour < quietEnd;
    }
}

/** The finished wording handed to every channel. */
record Message(String subject, String body) {}

/** Wording for one kind of notification, with {holes} to fill in. */
record Template(String kind, String subject, String body) {
    private static final Pattern HOLE = Pattern.compile("\\{(\\w+)}");

    Message render(Map<String, String> values) {
        return new Message(fill(subject, values), fill(body, values));
    }

    private static String fill(String text, Map<String, String> values) {
        Matcher m = HOLE.matcher(text);
        StringBuilder out = new StringBuilder();
        while (m.find()) {
            String v = values.get(m.group(1));
            if (v == null) throw new IllegalArgumentException("missing value for {" + m.group(1) + "}");
            m.appendReplacement(out, Matcher.quoteReplacement(v));
        }
        return m.appendTail(out).toString();
    }
}
```

```java
/** One ask from product code. Values are kept sorted so the same request renders the same text. */
record NotificationRequest(String kind, String userId, Map<String, String> values, Priority priority) {
    NotificationRequest {
        values = new TreeMap<>(values);
    }
}

/** What happened to one notification. */
record DeliveryRecord(String kind, String userId, DeliveryStatus status, ChannelType channel,
                      int attempts, String reason) {}

/** Where "now" comes from, so quiet hours and rate limits are testable without waiting. */
interface TimeSource { LocalDateTime now(); }

final class ManualTime implements TimeSource {
    private LocalDateTime now;
    ManualTime(LocalDateTime start) { this.now = start; }
    public synchronized LocalDateTime now() { return now; }
    synchronized void set(LocalDateTime t) { now = t; }
}
```

```java
// File: code/lld/ch20/notification/Channels.java
package lld.ch20.notification;

import java.time.Duration;

/** Adapter target: the outside provider (SMTP server, SMS gateway, FCM/APNs). */
interface Gateway {
    /** Throws on failure. */
    void deliver(String address, Message message) throws Exception;
}

/** How many times to try and how long to wait between tries (exponential backoff). */
record RetryPolicy(int maxAttempts, Duration initialBackoff) {}

/** The outcome of one channel's delivery loop. */
record Attempt(boolean success, int attempts, String error) {}

/**
 * Template Method: deliver() is the fixed algorithm (try, back off, retry) and is final.
 * Subclasses fill in only the steps that differ: which address to use.
 */
abstract class NotificationChannel {
    private final Gateway gateway;
    private final RetryPolicy retry;

    NotificationChannel(Gateway gateway, RetryPolicy retry) {
        this.gateway = gateway;
        this.retry = retry;
    }

    abstract ChannelType type();

    /** Hook: where this channel reaches the user, or null if the user has no such address. */
    protected abstract String addressOf(User user);

    final Attempt deliver(User user, Message message) {
        String address = addressOf(user);
        if (address == null || address.isBlank()) return new Attempt(false, 0, "no " + type() + " address");
        String lastError = null;
        long backoff = retry.initialBackoff().toMillis();
        for (int attempt = 1; attempt <= retry.maxAttempts(); attempt++) {
            try {
                gateway.deliver(address, message);
                return new Attempt(true, attempt, null);
            } catch (Exception e) {
                lastError = e.getMessage();
                if (attempt < retry.maxAttempts()) sleep(backoff);
                backoff *= 2;
            }
        }
        return new Attempt(false, retry.maxAttempts(), lastError);
    }

    private static void sleep(long ms) {
        try {
            Thread.sleep(ms);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}

final class EmailChannel extends NotificationChannel {
    EmailChannel(Gateway g, RetryPolicy r) { super(g, r); }
    ChannelType type() { return ChannelType.EMAIL; }
    protected String addressOf(User u) { return u.email(); }
}

final class SmsChannel extends NotificationChannel {
    SmsChannel(Gateway g, RetryPolicy r) { super(g, r); }
    ChannelType type() { return ChannelType.SMS; }
    protected String addressOf(User u) { return u.phone(); }
}
```

```
final class PushChannel extends NotificationChannel {
    PushChannel(Gateway g, RetryPolicy r) { super(g, r); }
    ChannelType type() { return ChannelType.PUSH; }
    protected String addressOf(User u) { return u.deviceToken(); }
}

/** Factory: the one place that knows which class implements which channel type. */
final class ChannelFactory {
    private ChannelFactory() {}

    static NotificationChannel create(ChannelType type, Gateway gateway, RetryPolicy retry) {
        return switch (type) {
            case EMAIL -> new EmailChannel(gateway, retry);
            case SMS -> new SmsChannel(gateway, retry);
            case PUSH -> new PushChannel(gateway, retry);
        };
    }
}
```

```java
// File: code/lld/ch20/notification/Rules.java
package lld.ch20.notification;

import java.time.Duration;
import java.time.LocalDateTime;
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;

/** Strategy: one reason not to send. Returns the reason, or empty to let it through. */
interface SendRule {
    Optional<String> blocks(NotificationRequest request, Preferences prefs, LocalDateTime now);

    /** Told after a successful send (so suppressed messages do not count). */
    default void onSent(NotificationRequest request, LocalDateTime now) {}
}

/** Nothing normal during quiet hours. The rule itself knows that URGENT ignores it. */
final class QuietHoursRule implements SendRule {
    @Override
    public Optional<String> blocks(NotificationRequest r, Preferences prefs, LocalDateTime now) {
        if (r.priority() == Priority.URGENT) return Optional.empty();
        return prefs.isQuiet(now.getHour()) ? Optional.of("quiet hours") : Optional.empty();
    }
}

/** At most 'limit' notifications of one kind per user per sliding window. */
final class RateLimitRule implements SendRule {
    private final int limit;
    private final Duration window;
    private final Map<String, Deque<LocalDateTime>> sent = new ConcurrentHashMap<>();

    RateLimitRule(int limit, Duration window) {
        this.limit = limit;
        this.window = window;
    }

    private static String key(NotificationRequest r) { return r.userId() + "|" + r.kind(); }

    @Override
    public Optional<String> blocks(NotificationRequest r, Preferences prefs, LocalDateTime now) {
        Deque<LocalDateTime> times = sent.computeIfAbsent(key(r), k -> new ArrayDeque<>());
        synchronized (times) {
            prune(times, now); // old timestamps are dropped, so one key's deque never exceeds 'limit'
            return times.size() >= limit
                    ? Optional.of("rate limit " + limit + " per " + window.toMinutes() + " min")
                    : Optional.empty();
        }
    }

    @Override
    public void onSent(NotificationRequest r, LocalDateTime now) {
        Deque<LocalDateTime> times = sent.computeIfAbsent(key(r), k -> new ArrayDeque<>());
        synchronized (times) {
            times.addLast(now);
        }
    }

    private void prune(Deque<LocalDateTime> times, LocalDateTime now) {
        LocalDateTime cutoff = now.minus(window);
        while (!times.isEmpty() && !times.peekFirst().isAfter(cutoff)) times.pollFirst();
    }
}

/** Observer: told the outcome of every notification. */
interface DeliveryListener {
    void onOutcome(DeliveryRecord record);
}

final class LoggingListener implements DeliveryListener {
    @Override
```

```
    public void onOutcome(DeliveryRecord r) {
        System.out.printf("  [log] %-10s %-13s user=%s channel=%s attempts=%d%s%n", r.status(), r.kind(),
                r.userId(), r.channel() == null ? "-" : r.channel(), r.attempts(),
                r.reason() == null ? "" : " reason=" + r.reason());
    }
}

final class MetricsListener implements DeliveryListener {
    private final Map<DeliveryStatus, Integer> counts = new ConcurrentHashMap<>();

    @Override
    public void onOutcome(DeliveryRecord r) { counts.merge(r.status(), 1, Integer::sum); }

    Map<DeliveryStatus, Integer> counts() { return new java.util.TreeMap<>(counts); }
}
```

```java
// File: code/lld/ch20/notification/NotificationService.java
package lld.ch20.notification;

import java.time.LocalDateTime;
import java.util.EnumMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

/** The class product code calls. It orders the steps and decides nothing itself. */
final class NotificationService implements AutoCloseable {
    private final Map<ChannelType, NotificationChannel> channels = new EnumMap<>(ChannelType.class);
    private final Map<String, User> users = new ConcurrentHashMap<>();
    private final Map<String, Preferences> preferences = new ConcurrentHashMap<>();
    private final Map<String, Template> templates = new ConcurrentHashMap<>();
    private final List<SendRule> rules;
    private final List<DeliveryListener> listeners = new CopyOnWriteArrayList<>();
    private final List<DeliveryRecord> history = new CopyOnWriteArrayList<>();
    private final TimeSource time;
    private final ExecutorService workers = Executors.newFixedThreadPool(4);

    NotificationService(List<NotificationChannel> channelList, List<SendRule> rules, TimeSource time) {
        channelList.forEach(c -> channels.put(c.type(), c));
        this.rules = List.copyOf(rules);
        this.time = time;
    }

    void register(User u, Preferences p) { users.put(u.id(), u); preferences.put(u.id(), p); }
    void addTemplate(Template t) { templates.put(t.kind(), t); }
    void subscribe(DeliveryListener l) { listeners.add(l); }
    List<DeliveryRecord> history() { return List.copyOf(history); }

    /** Synchronous send: rules -> render -> try each accepted channel in the user's order. */
    DeliveryRecord send(NotificationRequest req) {
        User user = users.get(req.userId());
        if (user == null) return finish(req, DeliveryStatus.FAILED, null, 0, "unknown user");
        Preferences prefs = preferences.get(req.userId());
        LocalDateTime now = time.now();

        for (SendRule rule : rules) {
            Optional<String> reason = rule.blocks(req, prefs, now);
            if (reason.isPresent()) return finish(req, DeliveryStatus.SUPPRESSED, null, 0, reason.get());
        }
        List<ChannelType> accepted = prefs.channelsFor(req.kind());
        if (accepted.isEmpty()) return finish(req, DeliveryStatus.SUPPRESSED, null, 0, "user accepts no channel");

        Template template = templates.get(req.kind());
        if (template == null) return finish(req, DeliveryStatus.FAILED, null, 0, "no template");
        Message message;
        try {
            message = template.render(req.values()); // rendered ONCE, same text on every channel
        } catch (IllegalArgumentException e) {        // e.g. a {hole} with no value: still leave via finish()
            return finish(req, DeliveryStatus.FAILED, null, 0, "template: " + e.getMessage());
        }

        int totalAttempts = 0;
        String lastError = null;
        for (ChannelType type : accepted) {
            NotificationChannel channel = channels.get(type);
            if (channel == null) continue;
            Attempt a = channel.deliver(user, message);
            totalAttempts += a.attempts();
            if (a.success()) {
                rules.forEach(r -> r.onSent(req, now));
                return finish(req, DeliveryStatus.SENT, type, totalAttempts, null);
            }
            lastError = type + ": " + a.error();
        }
```

```
        return finish(req, DeliveryStatus.FAILED, null, totalAttempts, "all channels failed, last " + lastError);
    }

    /** Asynchronous send: the caller (e.g. checkout) is not blocked by a slow provider. */
    CompletableFuture<DeliveryRecord> sendAsync(NotificationRequest req) {
        return CompletableFuture.supplyAsync(() -> send(req), workers);
    }

    /** The single exit: every outcome is stored and announced here. */
    private DeliveryRecord finish(NotificationRequest req, DeliveryStatus status, ChannelType channel,
                                  int attempts, String reason) {
        DeliveryRecord record = new DeliveryRecord(req.kind(), req.userId(), status, channel, attempts, reason);
        history.add(record);
        for (DeliveryListener l : listeners) {
            try {
                l.onOutcome(record);
            } catch (RuntimeException e) {
                // a broken listener must not break delivery
            }
        }
        return record;
    }

    @Override
    public void close() { workers.shutdown(); }
}
```

```java
// File: code/lld/ch20/notification/Main.java
package lld.ch20.notification;

import java.time.Duration;
import java.time.LocalDateTime;
import java.util.List;
import java.util.Map;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {

    /** A fake provider that fails the next N calls, then succeeds, printing what it "sent". */
    static final class FakeGateway implements Gateway {
        private final String name;
        private final AtomicInteger failuresLeft = new AtomicInteger();
        FakeGateway(String name) { this.name = name; }
        void failNext(int n) { failuresLeft.set(n); }

        @Override
        public void deliver(String address, Message m) throws Exception {
            if (failuresLeft.getAndUpdate(x -> Math.max(0, x - 1)) > 0) throw new Exception(name + " timeout");
            System.out.println("    " + name + " -> " + address + " : " + m.subject() + " | " + m.body());
        }
    }

    public static void main(String[] args) throws Exception {
        FakeGateway smtp = new FakeGateway("SMTP"), twilio = new FakeGateway("SMS-GW"), fcm = new FakeGateway("FCM");
        RetryPolicy retry = new RetryPolicy(3, Duration.ofMillis(1));
        ManualTime clock = new ManualTime(LocalDateTime.of(2026, 9, 28, 14, 0));

        MetricsListener metrics = new MetricsListener();
        try (NotificationService svc = new NotificationService(
                List.of(ChannelFactory.create(ChannelType.EMAIL, smtp, retry),
                        ChannelFactory.create(ChannelType.SMS, twilio, retry),
                        ChannelFactory.create(ChannelType.PUSH, fcm, retry)),
                List.of(new QuietHoursRule(), new RateLimitRule(2, Duration.ofHours(1))),
                clock)) {
            svc.subscribe(new LoggingListener());
            svc.subscribe(metrics);

            svc.addTemplate(new Template("ORDER_SHIPPED", "Order {orderId} shipped", "Hi {name}, it arrives {eta}."));
            svc.addTemplate(new Template("OTP", "Your code", "{code} is your login code."));
            svc.addTemplate(new Template("PROMO", "Sale!", "{name}, {pct}% off today."));

            svc.register(new User("u1", "Asha", "asha@mail.com", "+91-98450", "tok-asha"),
                    new Preferences().accept("ORDER_SHIPPED", ChannelType.SMS, ChannelType.EMAIL)
                            .accept("OTP", ChannelType.PUSH, ChannelType.SMS)
                            .accept("PROMO", ChannelType.EMAIL)
                            .quietHours(22, 7));
            svc.register(new User("u2", "Ben", "ben@mail.com", null, "tok-ben"),
                    new Preferences().accept("ORDER_SHIPPED", ChannelType.PUSH));

            Map<String, String> order = Map.of("orderId", "A-17", "name", "Asha", "eta", "Tuesday");

            System.out.println("== 1. Normal send over the first accepted channel (SMS)");
            svc.send(new NotificationRequest("ORDER_SHIPPED", "u1", order, Priority.NORMAL));

            System.out.println("== 2. SMS fails 3 times -> retried, then falls back to EMAIL");
            twilio.failNext(3);
            svc.send(new NotificationRequest("ORDER_SHIPPED", "u1", order, Priority.NORMAL));

            System.out.println("== 3. Quiet hours at 23:00: normal is suppressed, urgent goes through");
            clock.set(LocalDateTime.of(2026, 9, 28, 23, 0));
            svc.send(new NotificationRequest("PROMO", "u1", Map.of("name", "Asha", "pct", "20"), Priority.NORMAL));
            svc.send(new NotificationRequest("OTP", "u1", Map.of("code", "482913"), Priority.URGENT));

            System.out.println("== 4. Rate limit: 2 PROMO per hour, the 3rd is suppressed");
            clock.set(LocalDateTime.of(2026, 9, 29, 10, 0));
            for (int i = 0; i < 3; i++)
                svc.send(new NotificationRequest("PROMO", "u1", Map.of("name", "Asha", "pct", "" + (10 + i)),
Priority.NORMAL));
            clock.set(LocalDateTime.of(2026, 9, 29, 11, 1));
            System.out.println("  ...61 minutes later the window has moved:");
```

```
            svc.send(new NotificationRequest("PROMO", "u1", Map.of("name", "Asha", "pct", "50"), Priority.NORMAL));

            System.out.println("== 5. Every channel gives up -> FAILED");
            fcm.failNext(3);
            svc.send(new NotificationRequest("ORDER_SHIPPED", "u2",
                    Map.of("orderId", "B-2", "name", "Ben", "eta", "Friday"), Priority.NORMAL));

            System.out.println("== 6. No accepted channel for this kind -> SUPPRESSED");
            svc.send(new NotificationRequest("PROMO", "u2", Map.of("name", "Ben", "pct", "5"), Priority.NORMAL));

            System.out.println("== 7. Template value missing -> FAILED, still recorded through finish()");
            svc.send(new NotificationRequest("OTP", "u1", Map.of(), Priority.URGENT));

            System.out.println("== 8. Async send (caller not blocked)");
            DeliveryRecord r = svc.sendAsync(new NotificationRequest("ORDER_SHIPPED", "u2",
                    Map.of("orderId", "B-3", "name", "Ben", "eta", "Monday"), Priority.NORMAL)).get();
            System.out.println("  future completed with " + r.status() + " via " + r.channel());

            System.out.println("== Metrics: " + metrics.counts() + ", history size = " + svc.history().size());
        }
    }
}
```

Walking through the code.

- `Preferences.isQuiet` handles the midnight wrap: if start ≤ end (09–17) it is a simple range; otherwise (22–07) it is "after start **or** before end".
- `Template.render` uses a regex (regular expression: a small pattern language for finding text) `\{(\w+)}` and `Matcher.appendReplacement` with `Matcher.quoteReplacement`, so a value containing `$` or `\` is inserted literally. A missing value is an error, not a silent `{name}` in a customer's inbox.
- `NotificationChannel.deliver` is `final`: subclasses cannot break the retry loop. Backoff doubles after each failure (exponential backoff). The course's version retries immediately and names that as a weakness (three back-to-back tries against a server that is down are three quick failures); the waiting belongs in this one parent loop, which is exactly where we put it.
- `RateLimitRule` keeps a deque (double-ended queue: items can be added or removed at either end) of send times per (user, kind) and **prunes** old ones on every check, so one key's deque never holds more than `limit` entries (the course's version never discarded old timestamps and flagged that as a known leak — a **memory leak** is memory a program keeps holding although it no longer needs it). The *map of keys* still grows with every (user, kind) pair ever seen; a production version adds a periodic sweep that drops empty or stale deques, or uses a fixed-size window counter per key.
- `NotificationService.send` has one exit, `finish()`. Even a template with a missing value is caught and recorded as FAILED (step 7 of the output) instead of escaping as an exception that no listener would ever hear about. A listener that throws is caught so it cannot break delivery.
- `NotificationRequest` copies its values into a `TreeMap` (a map that keeps its keys sorted), so the values are always in the same order and the same request always renders the same text: handy when comparing test output.
- `sendAsync` runs `send` on a worker pool (a fixed set of threads that pick up tasks as they arrive) and returns a `CompletableFuture` (a handle to a result that will be ready later); in production a durable queue (Kafka, SQS) would sit here instead.

> **Key term — message queue:** a queue is a waiting line for work. Producers put messages in at one end; workers take them out at the other, at their own pace. A **durable** queue stores its messages on disk, so they survive a crash or restart. **Kafka** (an open-source log-based messaging system) and **Amazon SQS** (a managed queue service in AWS, Amazon's cloud) are two widely used examples. See System Design Fundamentals for queues in larger systems.

Output:

```
== 1. Normal send over the first accepted channel (SMS)
    SMS-GW -> +91-98450 : Order A-17 shipped | Hi Asha, it arrives Tuesday.
  [log] SENT       ORDER_SHIPPED user=u1 channel=SMS attempts=1
== 2. SMS fails 3 times -> retried, then falls back to EMAIL
    SMTP -> asha@mail.com : Order A-17 shipped | Hi Asha, it arrives Tuesday.
  [log] SENT       ORDER_SHIPPED user=u1 channel=EMAIL attempts=4
== 3. Quiet hours at 23:00: normal is suppressed, urgent goes through
  [log] SUPPRESSED PROMO         user=u1 channel=- attempts=0 reason=quiet hours
    FCM -> tok-asha : Your code | 482913 is your login code.
  [log] SENT       OTP           user=u1 channel=PUSH attempts=1
== 4. Rate limit: 2 PROMO per hour, the 3rd is suppressed
    SMTP -> asha@mail.com : Sale! | Asha, 10% off today.
  [log] SENT       PROMO         user=u1 channel=EMAIL attempts=1
    SMTP -> asha@mail.com : Sale! | Asha, 11% off today.
  [log] SENT       PROMO         user=u1 channel=EMAIL attempts=1
  [log] SUPPRESSED PROMO         user=u1 channel=- attempts=0 reason=rate limit 2 per 60 min
  ...61 minutes later the window has moved:
    SMTP -> asha@mail.com : Sale! | Asha, 50% off today.
  [log] SENT       PROMO         user=u1 channel=EMAIL attempts=1
== 5. Every channel gives up -> FAILED
  [log] FAILED     ORDER_SHIPPED user=u2 channel=- attempts=3 reason=all channels failed, last PUSH: FCM timeout
== 6. No accepted channel for this kind -> SUPPRESSED
  [log] SUPPRESSED PROMO         user=u2 channel=- attempts=0 reason=user accepts no channel
== 7. Template value missing -> FAILED, still recorded through finish()
  [log] FAILED     OTP           user=u1 channel=- attempts=0 reason=template: missing value for {code}
== 8. Async send (caller not blocked)
    FCM -> tok-ben : Order B-3 shipped | Hi Ben, it arrives Monday.
  [log] SENT       ORDER_SHIPPED user=u2 channel=PUSH attempts=1
  future completed with SENT via PUSH
== Metrics: {SENT=7, FAILED=2, SUPPRESSED=3}, history size = 12
```

Step 2 shows Template Method + fallback: 3 failed SMS attempts + 1 successful email = 4 attempts. Step 4 shows that the suppressed third promo did not count: an hour later the window has moved and sending works again.

Concurrency & edge cases

- **Shared state is thread-safe:** users/preferences/templates in `ConcurrentHashMap` (a map built for many threads to read and write at once); listeners and history in `CopyOnWriteArrayList` (a list that copies itself on every write, so reads need no lock: cheap reads, rare writes); the rate-limit deque is locked per key.
- **Check-then-act gap in rate limiting.** (Check-then-act: code first checks a condition, then acts on it, and another thread can change things in between.) Two concurrent (running at the same time) sends for the same user and kind can both pass `blocks()` before either calls `onSent()`, so the limit can be exceeded by the number of concurrent sends. Fixes: reserve a slot atomically at check time and release it if the send fails, or serialize sends per user (run them strictly one after another, e.g. a per-user queue, which also keeps their order).
- **Retries block a worker thread.** With `Thread.sleep` backoff, a slow provider ties up threads. Real systems re-enqueue the message with a delay instead (a "retry topic" — a topic is a named channel that messages are published to — or delayed queue, which hands a message out only after a set delay).
- **Idempotency.** A retry after a timeout (giving up on waiting for a reply after a set time) may deliver twice (the provider may have sent it but the reply was lost). Pass an idempotency key / message id to providers that support it, and make receivers tolerate duplicates.

> **Key term — idempotency:** an operation is **idempotent** when doing it twice has the same effect as doing it once ("set the balance to 10" is; "add 10 to the balance" is not). Retries are only safe on idempotent operations. An **idempotency key** is a unique id sent with a request, so the receiver can recognise a repeat and skip it. More in System Design Fundamentals.

- **Missing address** (user has no phone): the channel returns a failed attempt with 0 tries and the next channel is used.
- **Unknown kind / no template:** FAILED with a reason; **no accepted channel:** SUPPRESSED.
- **Time zones:** quiet hours should use the *user's* time zone; store a `ZoneId` (Java's type for a time zone) in preferences and convert `Instant` (an exact moment on the global timeline) → local hour.

Extensions & follow-up questions

- **"Add WhatsApp."** One `WhatsAppChannel extends NotificationChannel` (3 small methods), one enum value (an enum is a type with a fixed list of named values), one `Gateway` adapter. Nothing else changes.
- **"Someone is getting too many messages."** A new `SendRule` (e.g. a global daily cap, or "dedupe identical messages within 10 minutes"), not a new `if`.
- **"Send to a million users (a campaign)."** Split into a fan-out stage (fan-out: one input turned into many copies or tasks) that expands the audience into individual requests on a queue, and many workers that each run this `send`. Rate-limit per provider too (SMS gateways have throughput limits — **throughput** is how much work gets done per unit of time, e.g. messages per second).
- **"Priorities."** Separate queues per priority (OTP — one-time password, the login-code kind — queue drained first), or a `PriorityBlockingQueue` (a thread-safe queue that always hands out the highest-priority item next).
- **"Localization."** (Localization: adapting text to a user's language and region.) Key templates by (kind, locale — a language-plus-region code such as `fr-CA`) and store the user's locale in preferences.
- **"Delivery receipts / opened tracking."** Providers call back via webhooks (an HTTP request — HTTP being the protocol web clients and servers talk — that the provider sends to a URL you registered, when something happens on their side); record them as further events on the `DeliveryRecord`.
- **"Circuit breaker."** If a provider fails repeatedly, stop calling it for a while (Resilience4j-style; Resilience4j is a Java library of such fault-tolerance tools) and go straight to the next channel. Like an electrical breaker, it "trips" after too many failures and lets calls through again after a pause.

Real-world example

Push notifications on phones go through platform services: **Firebase Cloud Messaging (FCM)** for Android and **Apple Push Notification service (APNs)** for iOS; the app's server sends to a *device token*, exactly the `deviceToken` field above. Email is usually sent through providers such as Amazon SES or SendGrid, and SMS through gateways such as Twilio; each has its own API, which is why the `Gateway` adapter exists. **AWS SNS** is a managed (run for you by the cloud provider) pub-sub that fans one message out to email, SMS, mobile push and queues. LinkedIn's engineering blog describes **Air Traffic Controller (ATC)**, a central service that almost all LinkedIn notifications now pass through and that decides whether, when and how to notify each member, to stop excessive, repetitive notifications: the `SendRule` idea at company scale. And phone "Focus" / "Do Not Disturb" modes are quiet hours with an urgent-bypass list.

In the interview

> **30-second answer:** "Channels sit behind one abstract class whose final `deliver` method holds the retry-with-backoff loop (Template Method); each channel only supplies its address. Each "do not send" condition is its own `SendRule` (Strategy) returning a reason string, so we can record why. The user's preferences give an ordered channel list per kind; we render the template once and try channels in order, falling back on failure. Every outcome goes through one `finish` method that stores a record and notifies listeners (Observer). For scale, a queue sits in front and workers call `send`."

Plan (35–45 min): clarify (5) → entities, User vs Preferences (5) → Template + Message (3) → channel hierarchy + retry (8) → rules (7) → service flow + single exit + listeners (7) → async/queue, idempotency, follow-ups (5).

Follow-ups with answers:

- *"Add a new channel."* → One subclass + gateway adapter.
- *"What about duplicate messages on retry?"* → Idempotency keys; at-least-once delivery (every message arrives, but some may arrive more than once) means receivers must tolerate duplicates.
- *"Why not Strategy for channels?"* → Channels share an algorithm and vary in one step; that is Template Method. (Composition-based alternative: inject a `RetryPolicy` object and a `Sender` Strategy; also valid, mention it.)

Self-check

1. Which pattern puts the retry loop in one place? → Template Method (`final deliver()` in the abstract channel).

2. Why does `SendRule.blocks` return `Optional<String>` (Java's box that holds either one value or nothing)? → So the reason for suppression is recorded.
3. Why should a suppressed notification not count against the rate limit? → The user never received it; counting it would block legitimate later messages.
4. Quiet hours 22–07: is 06:00 quiet? Is 12:00? → Yes; no.
5. Where would you put a queue to avoid blocking checkout? → Between product code and `NotificationService.send` (our `sendAsync` is the in-process version).
6. What must every exit path of `send` do? → Go through `finish()`: store the record and notify listeners.

TL;DR

- **Template Method** for channels: retry/backoff once in the parent; subclasses give the address.
- **Strategy** for send rules (quiet hours, rate limit); rules return the *reason*.
- **Observer** for outcomes; **Simple Factory** for channels; **Adapter** for providers.
- Render once, try channels in the user's order, fall back, single `finish()` exit.
- Scale with a queue in front; watch for idempotency and the rate-limit check-then-act gap.

20.3 Design an In-Memory Cache (and an LRU Cache from scratch)

Problem & clarifying questions

What it is. A cache is a small, fast store that keeps recently used values close at hand, so you don't have to fetch them again from a slow place (a database, another service, the disk). (For where caches sit in bigger systems, see System Design Fundamentals.) Everyday analogy: the few books on your desk. The library (database) has everything, but the desk (cache) holds the ones you are using. When the desk is full and you need a new book, one must go back to the shelf. **Which one goes back** is the eviction policy, and it is the rule most likely to change.

Because the question is about **one component**, vague answers are exposed quickly. The interviewer will ask: *which data structure gives O(1) get and put?* (O(1), "constant time", means the time does not grow with the number of entries.)

Question	Assumed answer
Capacity measured in?	Number of entries (not bytes).
Eviction rule?	LRU (least recently used) by default; must be swappable for FIFO or LFU without changing the cache.
Time limits?	Optional TTL (time to live) per entry. Reading an expired entry is a miss and removes it.
Stats?	Hits, misses, evictions, expirations.
Notifications?	Interested code can subscribe to removals and learn the cause.
Thread-safe?	Yes (the course version left it out; we add it).
Load from DB (database) on miss / write-back (save changes to the DB later)?	Out of scope; discussed in extensions.

Requirements

Functional

1. Holds key → value pairs, never more than a fixed number.
2. `get` returns the value or reports a miss.
3. `put` inserts a new key or overwrites the value of an existing one.
4. Inserting a new key into a full cache first evicts one entry.
5. Default rule: **LRU** (least recently used — the entry whose last access is the oldest).

6. Swapping in another rule (FIFO — first in, first out; LFU — least frequently used) needs no change to the cache class.
7. `get` and `put` are O(1).
8. An entry may carry a TTL; once it has passed, a read returns a miss and deletes the entry.
9. Counts hits, misses, evictions, expirations.
10. Registered listeners hear about every removal together with its cause.
11. Callers can invalidate a single key or empty the whole cache.
12. Every entry leaves through **one** code path, whatever the reason.

Non-functional: O(1) operations; thread-safe; testable TTL (injected clock); listeners must not slow down or deadlock the cache (a **deadlock** is when two threads each wait forever for something the other one holds).

Use cases

Actors: **Application** (reads, stores, removes keys), **Operator** (chooses the eviction rule, reads stats), **Cache** (evicts, expires, counts, notifies).

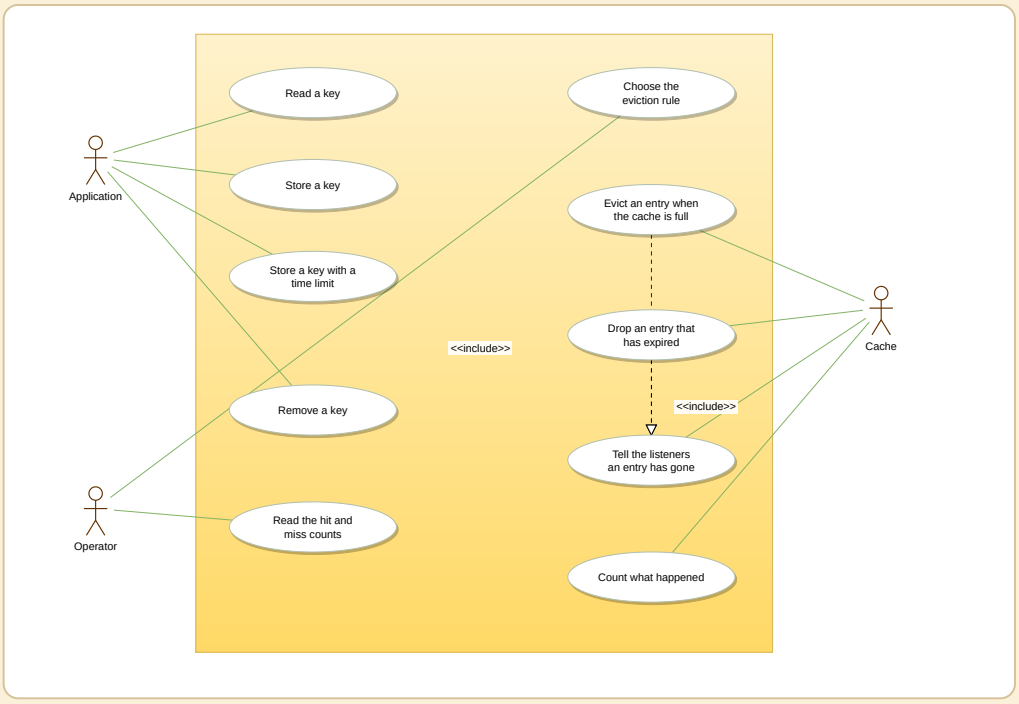


Figure — Cache use cases: read, store (with optional time limit), remove; the cache evicts, drops expired entries and tells listeners (course diagram)

Core entities

Class	Responsibility
`Node<K,V>`	One entry: key, value, `expiresAtMillis`, `frequency`, and its own `prev`/`next` links.
`DoublyLinkedList<K,V>`	Order of entries, newest at the front, with two **sentinel** (dummy) nodes so add/remove never need null checks.
`EvictionPolicy<K,V>` (interface)	**Strategy**: three events (`onInsert`, `onAccess`, `onRemove`) and one question (`victim()`). Owns the *order*.
`LruPolicy`, `FifoPolicy`, `LfuPolicy`	The three rules. LFU is O(1) using a chain of frequency buckets.
`EvictionPolicies` + `PolicyType`	**Simple Factory**: enum → policy.
`Cache<K,V>`	The map + policy + clock + stats + listeners. One lock. **One removal method.**
`Cache.Builder`	**Builder**: capacity, policy, clock with defaults, validated once.
`CacheStats`	Counters (`LongAdder`, a thread-safe counter).
`RemovalReason`	EVICTED, EXPIRED, REMOVED, CLEARED.
`EvictionListener<K,V>`	**Observer**: told key, value and reason.
`ShardedCache<K,V>`	N independent caches picked by key hash, to reduce lock contention.

Key terms — hash map, linked list, hashing, contention. A **hash map** (`HashMap`) stores key → value pairs; it turns each key into a number (its **hash**) that says which slot to look in, so finding a key takes about the same time however many entries there are. A **linked list** is a chain of nodes where each node points to the next one (a **doubly** linked list also points back to the previous one); inserting or unlinking a node you already hold is just a few pointer changes. **Lock contention** is threads queueing for the same lock; splitting data into **shards** (independent pieces, here chosen by key hash), each with its own lock, reduces it.

The core idea: two structures, each answering one question.

Question	Structure	Cost
"Where is key k?"	`HashMap<K, Node>`	O(1)
"Who should leave next?"	Doubly linked list ordered by recency	O(1) at the ends
"Move this entry to the front"	Node has its own `prev`/`next`, and the map hands us the node	O(1) — no search

A map alone cannot tell you the oldest entry. A list alone cannot find a key without walking it. Together they give O(1) for everything. A **doubly** linked list is needed because removing a node from the middle requires its predecessor, and a singly linked list would have to walk to find it.

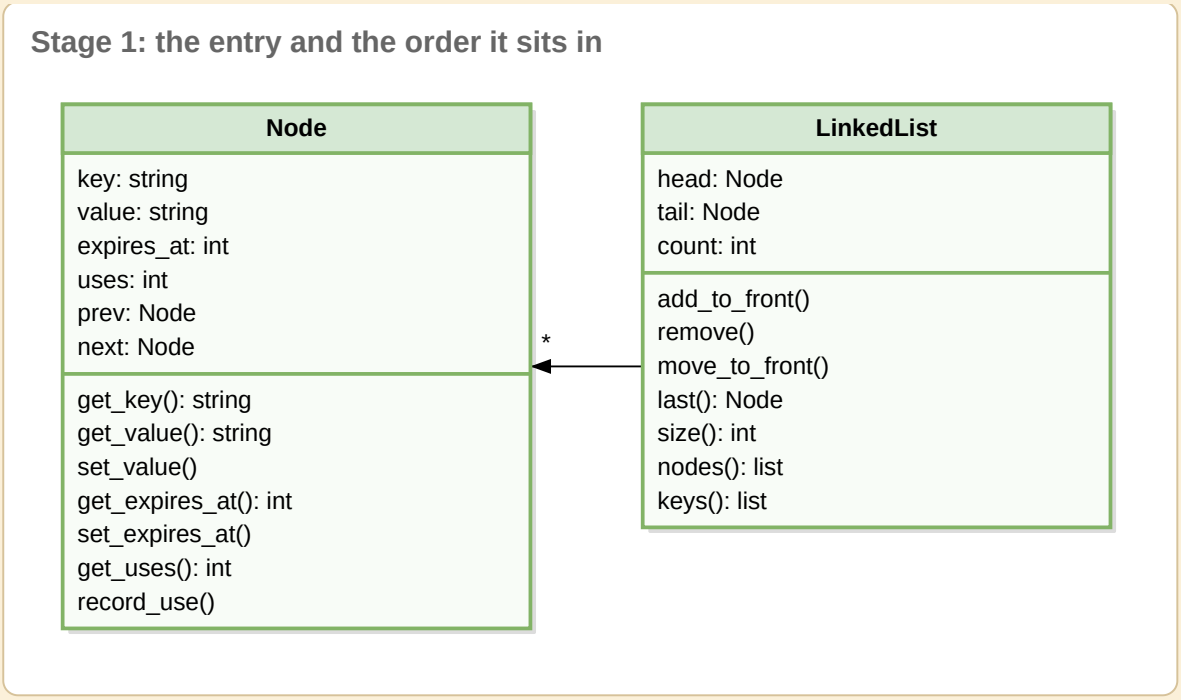


Figure — Stage 1 of the course build: the Node (key, value, expiry, use count, prev and next links) and the linked list that orders the nodes (course diagram)

Class diagram

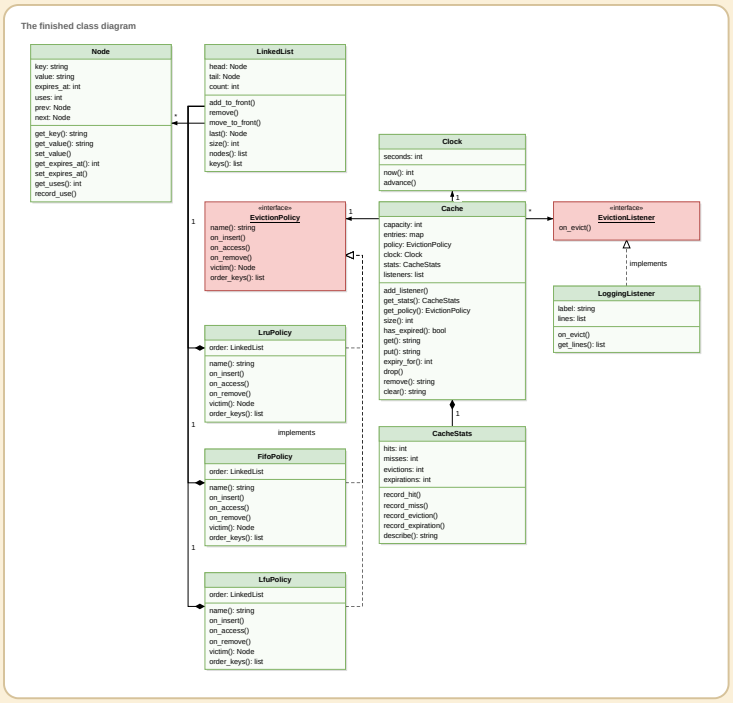
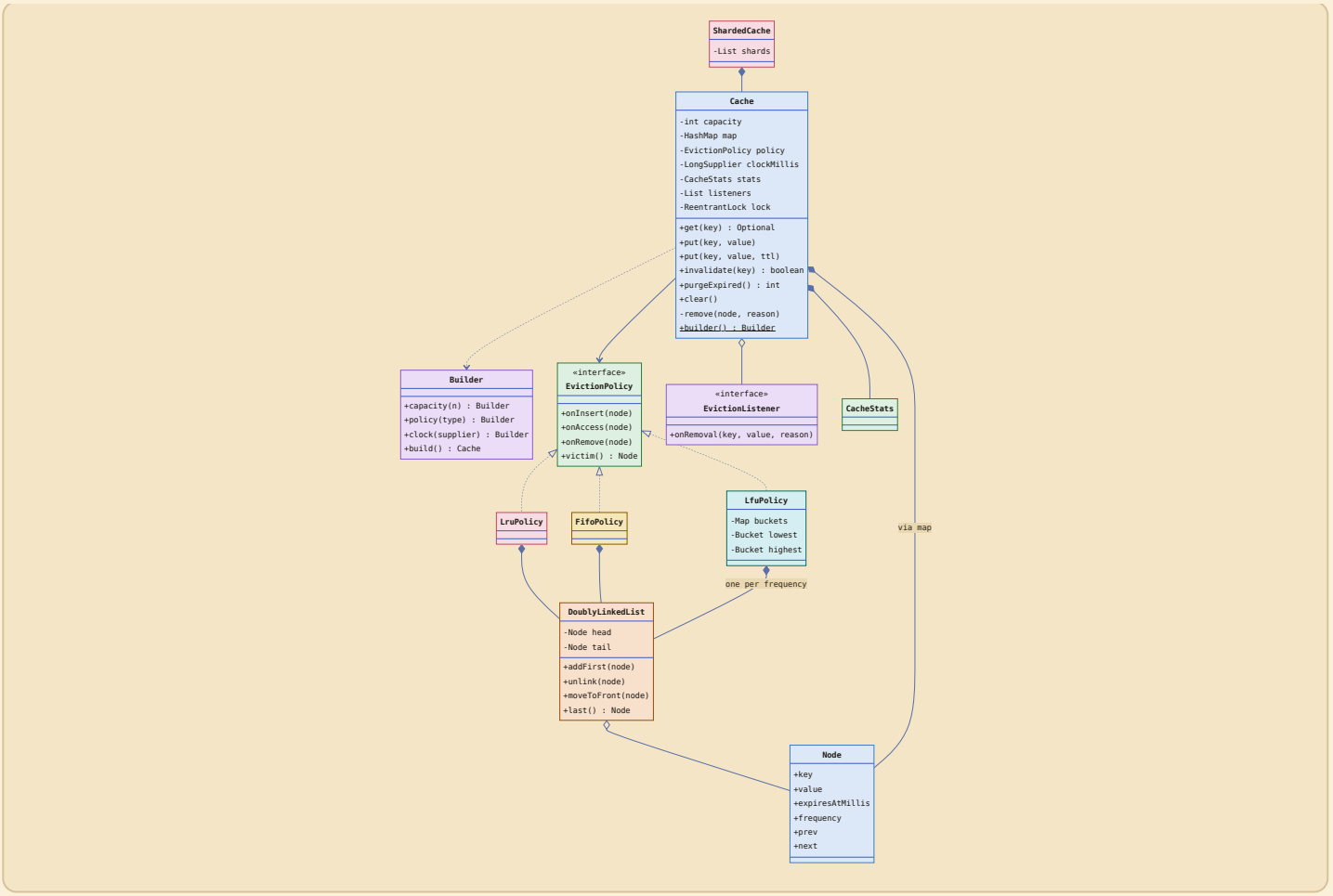


Figure — The finished cache class diagram: Cache with its map, policy, clock, stats and listeners; the three eviction policies behind one interface (course diagram)



Build order (from the course). (1) `Node` + `DoublyLinkedList`: entries can be ordered and moved to the front; (2) `EvictionPolicy` + `LruPolicy`: you can ask who should leave; (3) `FifoPolicy` + `LfuPolicy`: the same question gets three answers with no change to the cache; (4) `Cache` with clock, stats and TTL: reads and writes work; (5) `EvictionListener`: other code can react when an entry goes. Each stage leaves something runnable; reuse this order when you interview.

Who owns what. The cache owns the map (and so the nodes). The policy owns the *order* of those nodes. Neither reaches into the other's structure. That split is exactly what lets us add FIFO and LFU without touching `Cache`.

Key flows

Reading a key — three ways out, only one returns a value:

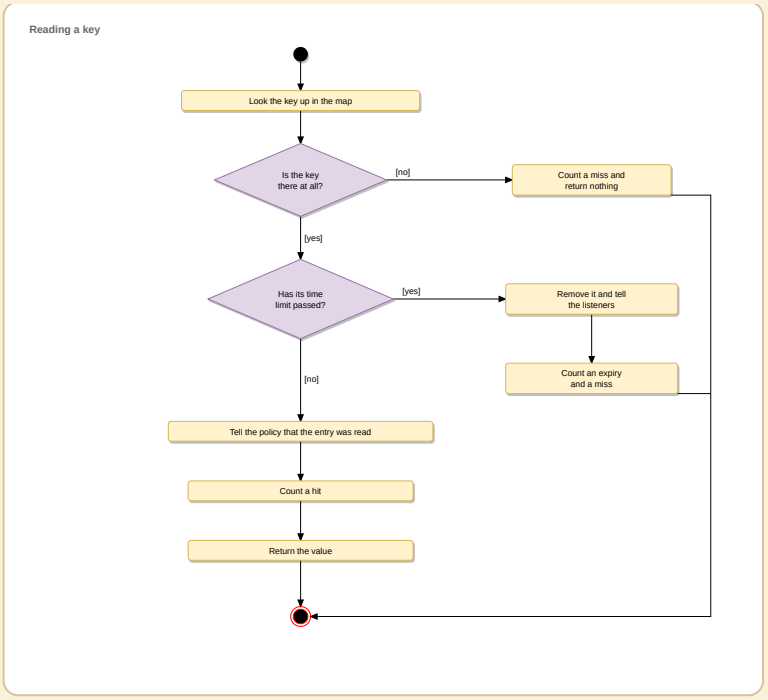


Figure — Reading a key: not present means a miss; present but expired means a miss plus an expiry removal; otherwise a hit, the policy is told, and the value is returned (course diagram)

Storing a key — eviction work happens only when the cache is already full, so a cache with free room pays nothing extra:

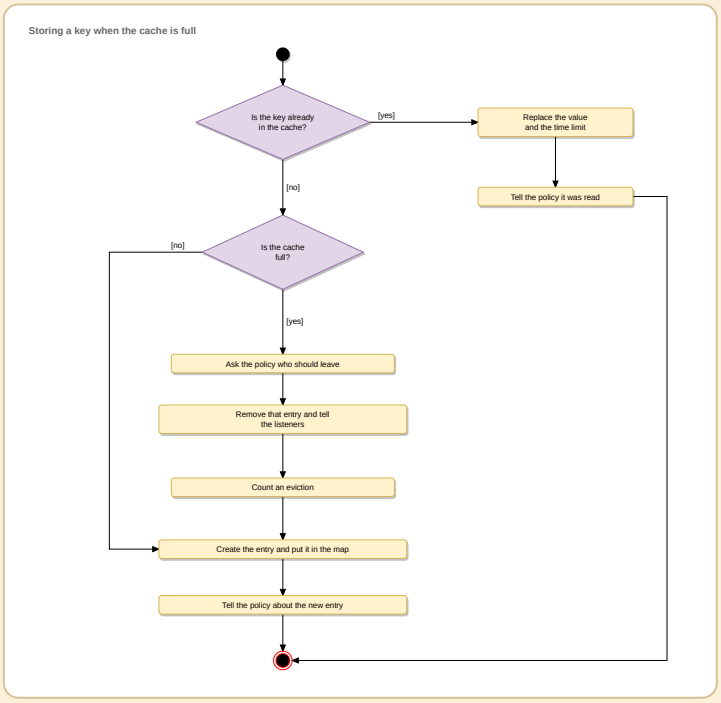
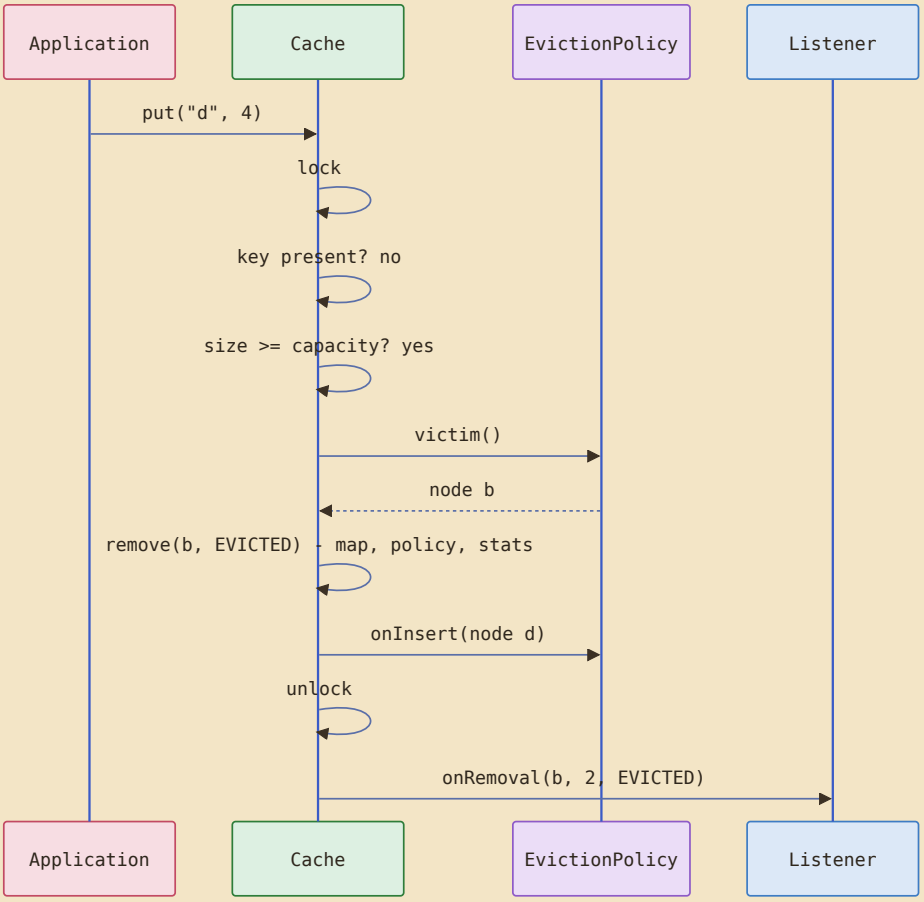


Figure — Storing a key: replace if present; otherwise, if full, ask the policy for a victim and remove it through the single removal path, then insert (course diagram)



How the three policies differ (the only difference is what they do on each event):

Event	LRU	FIFO	LFU (O(1) buckets)
`onInsert`	add to front	add to front	frequency = 1, add to bucket 1 (always first in the chain)
`onAccess`	move to front	**nothing**	move from bucket f to bucket f+1 (created right after f if missing); drop bucket f from the chain if it emptied
`onRemove`	unlink	unlink	unlink; drop the bucket if it emptied
`victim()`	back of the list	back of the list	back of the **first** bucket in the chain (least recently used among the least frequent)

The FIFO policy is LRU with one empty method, which is good evidence that the interface boundaries are well chosen. The course's LFU scanned all entries (O(n): the time grows in step with the number of entries) and named a structure grouping entries by count as the fix; our `LfuPolicy` implements that fix. The buckets are themselves kept in a **doubly linked chain in increasing frequency order**, and a new bucket f+1 is always created right after bucket f, so the chain never needs sorting and the lowest bucket is always the first one. That makes `onInsert`, `onAccess`, `onRemove` and `victim()` all O(1), even after a manual removal or an expiry empties the lowest bucket (this is the O(1) LFU scheme of Shah, Mitra and Matani, 2010). Ties inside the lowest bucket are broken by recency (the back of the bucket), so two runs of the same test always evict the same key.

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for eviction	"Who leaves" is the rule most likely to change.	A `switch (policyType)` inside `get`/`put`.
Observer for removals	Logging, metrics, deleting a temp file on eviction — none belong inside the cache. The reason is passed so one listener can handle all cases.	Callbacks (methods handed over to be called later) hard-coded per reason.
One removal method (`remove(node, reason)`)	Eviction, expiry, manual removal and clear all update map, policy, stats and listeners the same way; nothing can be forgotten on one path.	Four copies of the clean-up code.
Sentinel nodes	Add and unlink are always the same four pointer writes (a **pointer** here is a reference from one node to another); no special cases for empty list or ends.	Null checks everywhere (a classic source of bugs).
Injected clock (`LongSupplier`, a function that returns a `long`)	TTL tests run instantly by moving a fake clock forward.	`System.currentTimeMillis()` inside the cache: tests must `sleep`.
Lazy expiry + optional sweeper	Lazy = checked only when needed: expired entries are dropped when read; `purgeExpired()` lets a background task reclaim memory.	Only lazy (memory held by dead entries) or a timer per entry (expensive).
One `ReentrantLock` (Java's explicit lock object), listeners called **after unlock**	In LRU even `get` writes (moves a node), so a read-write lock (one that lets many readers in together but a writer only alone) would not help. Calling listeners outside the lock means a slow or re-entrant listener cannot block or deadlock the cache.	`synchronized` around listener calls.
Builder	Optional settings with defaults; validation once in `build()`.	Telescoping constructors (many constructors, each taking one more parameter).
Sharding (splitting data into independent pieces; `ShardedCache`)	N locks instead of one: threads touching different shards never wait for each other.	One global lock (every reader waits for every writer).

Java implementation

```java
// File: code/lld/ch20/cache/Structures.java
package lld.ch20.cache;

/** One cache entry. It carries its own links, so it can be unlinked in O(1). */
final class Node<K, V> {
    final K key;
    V value;
    long expiresAtMillis;  // 0 = never expires
    int frequency;         // used by LFU
    Node<K, V> prev, next;

    Node(K key, V value, long expiresAtMillis) {
        this.key = key;
        this.value = value;
        this.expiresAtMillis = expiresAtMillis;
    }

    boolean isExpired(long now) { return expiresAtMillis != 0 && now >= expiresAtMillis; }
}

/** Doubly linked list with two sentinel nodes: no null checks at the ends. Newest at the front. */
final class DoublyLinkedList<K, V> {
    private final Node<K, V> head = new Node<>(null, null, 0); // sentinel before the front
    private final Node<K, V> tail = new Node<>(null, null, 0); // sentinel after the back
    private int size;

    DoublyLinkedList() {
        head.next = tail;
        tail.prev = head;
    }

    void addFirst(Node<K, V> n) {
        n.prev = head;
        n.next = head.next;
        head.next.prev = n;
        head.next = n;
        size++;
    }

    void unlink(Node<K, V> n) {
        n.prev.next = n.next;
        n.next.prev = n.prev;
        n.prev = n.next = null;
        size--;
    }

    void moveToFront(Node<K, V> n) {
        unlink(n);
        addFirst(n);
    }

    /** The oldest node (at the back), or null if empty. */
    Node<K, V> last() { return size == 0 ? null : tail.prev; }

    boolean isEmpty() { return size == 0; }
}
```

```java
// File: code/lld/ch20/cache/EvictionPolicy.java
package lld.ch20.cache;

import java.util.HashMap;
import java.util.Map;

/** Strategy: owns the ORDER of entries and answers "who leaves?". The cache owns the map. */
interface EvictionPolicy<K, V> {
    void onInsert(Node<K, V> node);
    void onAccess(Node<K, V> node);
    void onRemove(Node<K, V> node);
    Node<K, V> victim();
}

/** Least recently used: a read moves the entry to the front; the back is the victim. */
final class LruPolicy<K, V> implements EvictionPolicy<K, V> {
    private final DoublyLinkedList<K, V> order = new DoublyLinkedList<>();
    public void onInsert(Node<K, V> n) { order.addFirst(n); }
    public void onAccess(Node<K, V> n) { order.moveToFront(n); }
    public void onRemove(Node<K, V> n) { order.unlink(n); }
    public Node<K, V> victim() { return order.last(); }
}

/** First in, first out: identical to LRU except that a read changes nothing. */
final class FifoPolicy<K, V> implements EvictionPolicy<K, V> {
    private final DoublyLinkedList<K, V> order = new DoublyLinkedList<>();
    public void onInsert(Node<K, V> n) { order.addFirst(n); }
    public void onAccess(Node<K, V> n) { /* insertion order only */ }
    public void onRemove(Node<K, V> n) { order.unlink(n); }
    public Node<K, V> victim() { return order.last(); }
}

/**
 * Least frequently used in O(1): one list per frequency ("bucket"), and the buckets themselves
 * chained in increasing frequency order, so the lowest non-empty bucket is always the first one.
 * Ties inside the lowest bucket go to the least recently used entry (the back of that bucket).
 */
final class LfuPolicy<K, V> implements EvictionPolicy<K, V> {
    private static final class Bucket<K, V> {
        final int frequency;
        final DoublyLinkedList<K, V> entries = new DoublyLinkedList<>();
        Bucket<K, V> lower, higher;           // neighbours in the frequency chain
        Bucket(int frequency) { this.frequency = frequency; }
    }

    private final Map<Integer, Bucket<K, V>> buckets = new HashMap<>();
    private final Bucket<K, V> lowest = new Bucket<>(0);   // sentinels of the bucket chain
    private final Bucket<K, V> highest = new Bucket<>(Integer.MAX_VALUE);

    LfuPolicy() {
        lowest.higher = highest;
        highest.lower = lowest;
    }

    public void onInsert(Node<K, V> n) {
        n.frequency = 1;
        bucketAfter(lowest, 1).entries.addFirst(n);        // frequency 1 is always the lowest
    }

    public void onAccess(Node<K, V> n) {
        Bucket<K, V> from = buckets.get(n.frequency);
        Bucket<K, V> to = bucketAfter(from, n.frequency + 1);
        from.entries.unlink(n);
        dropIfEmpty(from);
        n.frequency++;
        to.entries.addFirst(n);
    }

    public void onRemove(Node<K, V> n) {
        Bucket<K, V> b = buckets.get(n.frequency);
        b.entries.unlink(n);
        dropIfEmpty(b);                                    // the chain stays ordered: nothing to repair
    }
```

```
    public Node<K, V> victim() {
        Bucket<K, V> min = lowest.higher;                    // O(1): first bucket in the chain
        return min == highest ? null : min.entries.last();
    }

    /** The bucket for 'frequency', created right after 'prev' if it does not exist yet. */
    private Bucket<K, V> bucketAfter(Bucket<K, V> prev, int frequency) {
        Bucket<K, V> b = buckets.get(frequency);
        if (b != null) return b;
        b = new Bucket<>(frequency);
        b.lower = prev;
        b.higher = prev.higher;
        prev.higher.lower = b;
        prev.higher = b;
        buckets.put(frequency, b);
        return b;
    }

    private void dropIfEmpty(Bucket<K, V> b) {
        if (!b.entries.isEmpty()) return;
        b.lower.higher = b.higher;
        b.higher.lower = b.lower;
        buckets.remove(b.frequency);
    }
}

enum PolicyType { LRU, FIFO, LFU }

/** Factory: maps a configuration value to a policy object. */
final class EvictionPolicies {
    private EvictionPolicies() {}

    static <K, V> EvictionPolicy<K, V> create(PolicyType type) {
        return switch (type) {
            case LRU -> new LruPolicy<>();
            case FIFO -> new FifoPolicy<>();
            case LFU -> new LfuPolicy<>();
        };
    }
}
```

```java
// File: code/lld/ch20/cache/Cache.java
package lld.ch20.cache;

import java.time.Duration;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.LongAdder;
import java.util.concurrent.locks.ReentrantLock;
import java.util.function.LongSupplier;

enum RemovalReason { EVICTED, EXPIRED, REMOVED, CLEARED }

/** Observer: told every time an entry leaves, and why. */
interface EvictionListener<K, V> {
    void onRemoval(K key, V value, RemovalReason reason);
}

/** Hit / miss / eviction / expiry counters. LongAdder is cheap under contention. */
final class CacheStats {
    final LongAdder hits = new LongAdder(), misses = new LongAdder(),
            evictions = new LongAdder(), expirations = new LongAdder();

    @Override
    public String toString() {
        long h = hits.sum(), m = misses.sum();
        return "hits=" + h + " misses=" + m + " evictions=" + evictions.sum() + " expirations=" + expirations.sum()
                + String.format(" hitRate=%.0f%%", h + m == 0 ? 0.0 : 100.0 * h / (h + m));
    }
}

/**
 * Fixed-capacity cache: HashMap for "where is it?", an EvictionPolicy for "who leaves?".
 * Thread-safe with one lock (a read in LRU also writes: it moves the node).
 */
final class Cache<K, V> {
    private final int capacity;
    private final Map<K, Node<K, V>> map = new HashMap<>();
    private final EvictionPolicy<K, V> policy;
    private final LongSupplier clockMillis;
    private final CacheStats stats = new CacheStats();
    private final List<EvictionListener<K, V>> listeners = new CopyOnWriteArrayList<>();
    private final ReentrantLock lock = new ReentrantLock();

    private record Removal<K, V>(K key, V value, RemovalReason reason) {}

    private Cache(Builder<K, V> b) {
        this.capacity = b.capacity;
        this.policy = EvictionPolicies.create(b.policyType);
        this.clockMillis = b.clockMillis;
    }

    static <K, V> Builder<K, V> builder() { return new Builder<>(); }

    Optional<V> get(K key) {
        List<Removal<K, V>> removed = new ArrayList<>(1);
        lock.lock();
        try {
            Node<K, V> n = map.get(key);
            if (n == null) {
                stats.misses.increment();
                return Optional.empty();
            }
            if (n.isExpired(clockMillis.getAsLong())) { // a miss, an expiry and a removal at once
                stats.misses.increment();
                removed.add(remove(n, RemovalReason.EXPIRED));
                return Optional.empty();
            }
            stats.hits.increment();
            policy.onAccess(n);
```

```
            return Optional.of(n.value);
        } finally {
            lock.unlock();
            notifyListeners(removed); // outside the lock: a slow listener cannot block the cache
        }
    }

    void put(K key, V value) { put(key, value, null); }

    void put(K key, V value, Duration ttl) {
        List<Removal<K, V>> removed = new ArrayList<>(1);
        lock.lock();
        try {
            long expiresAt = ttl == null ? 0 : clockMillis.getAsLong() + ttl.toMillis();
            Node<K, V> existing = map.get(key);
            if (existing != null) {          // replace: counts as a use
                existing.value = value;
                existing.expiresAtMillis = expiresAt;
                policy.onAccess(existing);
                return;
            }
            if (map.size() >= capacity) removed.add(remove(policy.victim(), RemovalReason.EVICTED));
            Node<K, V> n = new Node<>(key, value, expiresAt);
            map.put(key, n);
            policy.onInsert(n);
        } finally {
            lock.unlock();
            notifyListeners(removed);
        }
    }

    boolean invalidate(K key) {
        List<Removal<K, V>> removed = new ArrayList<>(1);
        lock.lock();
        try {
            Node<K, V> n = map.get(key);
            if (n != null) removed.add(remove(n, RemovalReason.REMOVED));
            return n != null;
        } finally {
            lock.unlock();
            notifyListeners(removed);
        }
    }

    /** Called by a background sweeper (e.g. a ScheduledExecutorService) to reclaim memory early. */
    int purgeExpired() {
        List<Removal<K, V>> removed = new ArrayList<>();
        lock.lock();
        try {
            long now = clockMillis.getAsLong();
            for (Node<K, V> n : new ArrayList<>(map.values()))
                if (n.isExpired(now)) removed.add(remove(n, RemovalReason.EXPIRED));
            return removed.size();
        } finally {
            lock.unlock();
            notifyListeners(removed);
        }
    }

    void clear() {
        List<Removal<K, V>> removed = new ArrayList<>();
        lock.lock();
        try {
            for (Node<K, V> n : new ArrayList<>(map.values())) removed.add(remove(n, RemovalReason.CLEARED));
        } finally {
            lock.unlock();
            notifyListeners(removed);
        }
    }

    /** THE single exit path for every entry, whatever the reason. Caller holds the lock. */
    private Removal<K, V> remove(Node<K, V> n, RemovalReason reason) {
        map.remove(n.key);
        policy.onRemove(n);
```

```
        if (reason == RemovalReason.EVICTED) stats.evictions.increment();
        if (reason == RemovalReason.EXPIRED) stats.expirations.increment();
        return new Removal<>(n.key, n.value, reason);
    }

    private void notifyListeners(List<Removal<K, V>> removed) {
        for (Removal<K, V> r : removed)
            for (EvictionListener<K, V> l : listeners) l.onRemoval(r.key(), r.value(), r.reason());
    }

    void addListener(EvictionListener<K, V> l) { listeners.add(l); }
    CacheStats stats() { return stats; }

    int size() {
        lock.lock();
        try { return map.size(); } finally { lock.unlock(); }
    }

    /** Builder: optional settings with sensible defaults, validated once. */
    static final class Builder<K, V> {
        private int capacity = 100;
        private PolicyType policyType = PolicyType.LRU;
        private LongSupplier clockMillis = System::currentTimeMillis;

        Builder<K, V> capacity(int c) { this.capacity = c; return this; }
        Builder<K, V> policy(PolicyType p) { this.policyType = p; return this; }
        Builder<K, V> clock(LongSupplier c) { this.clockMillis = c; return this; }

        Cache<K, V> build() {
            if (capacity <= 0) throw new IllegalArgumentException("capacity must be positive");
            return new Cache<>(this);
        }
    }
}

/** Sharding: N independent caches chosen by key hash, so threads rarely share a lock. */
final class ShardedCache<K, V> {
    private final List<Cache<K, V>> shards = new ArrayList<>();

    ShardedCache(int shardCount, int capacityPerShard, PolicyType type) {
        for (int i = 0; i < shardCount; i++)
            shards.add(Cache.<K, V>builder().capacity(capacityPerShard).policy(type).build());
    }

    private Cache<K, V> shard(K key) { return shards.get(Math.floorMod(key.hashCode(), shards.size())); }
    Optional<V> get(K key) { return shard(key).get(key); }
    void put(K key, V value) { shard(key).put(key, value); }
    int size() { return shards.stream().mapToInt(Cache::size).sum(); }
}
```

```java
// File: code/lld/ch20/cache/Main.java
package lld.ch20.cache;

import java.time.Duration;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicLong;

public class Main {
    public static void main(String[] args) throws Exception {
        for (PolicyType type : PolicyType.values()) {
            System.out.println("== " + type + " (capacity 3): put a,b,c; get b,b,b,c,a,a; put d");
            Cache<String, Integer> c = Cache.<String, Integer>builder().capacity(3).policy(type).build();
            c.addListener((k, v, why) -> System.out.println("  listener: " + k + "=" + v + " left, " + why));
            c.put("a", 1);
            c.put("b", 2);
            c.put("c", 3);
            for (String k : List.of("b", "b", "b", "c", "a", "a")) c.get(k);
            c.put("d", 4);
            List<String> present = new ArrayList<>();
            for (String k : List.of("a", "b", "c", "d")) if (c.get(k).isPresent()) present.add(k);
            System.out.println("  still cached: " + present);
        }

        System.out.println("== TTL with a manual clock");
        AtomicLong now = new AtomicLong(0);
        Cache<String, String> sessions = Cache.<String, String>builder().capacity(10).clock(now::get).build();
        sessions.addListener((k, v, why) -> System.out.println("  listener: " + k + " left, " + why));
        sessions.put("session:42", "alice", Duration.ofSeconds(5));
        sessions.put("session:43", "bob", Duration.ofSeconds(5));
        sessions.put("config", "v1");
        System.out.println("  t=0s  get session:42 -> " + sessions.get("session:42").orElse("MISS"));
        now.set(6_000);
        System.out.println("  t=6s  get session:42 -> " + sessions.get("session:42").orElse("MISS"));
        System.out.println("  sweeper purged " + sessions.purgeExpired() + " more expired entry");
        System.out.println("  config (no TTL) -> " + sessions.get("config").orElse("MISS"));
        sessions.invalidate("config");
        System.out.println("  stats: " + sessions.stats());

        System.out.println("== Sharded cache under 8 threads");
        ShardedCache<Integer, Integer> sharded = new ShardedCache<>(8, 1_000, PolicyType.LRU);
        ExecutorService pool = Executors.newFixedThreadPool(8);
        for (int t = 0; t < 8; t++) {
            int seed = t;
            pool.submit(() -> {
                for (int i = 0; i < 50_000; i++) {
                    int key = (i * 31 + seed) % 20_000;
                    if (sharded.get(key).isEmpty()) sharded.put(key, key * 2);
                }
            });
        }
        pool.shutdown();
        System.out.println("  finished: " + pool.awaitTermination(30, TimeUnit.SECONDS)
                + ", size <= 8000: " + (sharded.size() <= 8_000));
    }
}
```

Walking through the code.

- `DoublyLinkedList` never touches `null` at the ends because `head` and `tail` are dummy nodes that are never removed.
- In `Cache.get`, an expired entry counts as a miss **and** an expiration, and is removed through `remove()` — all three in one place, so the counters always add up.
- Every public method collects `Removal` events while holding the lock and calls `notifyListeners` in `finally` (a block that always runs, even when an exception is thrown) **after** `unlock()`.
- `put` on an existing key replaces value and TTL and counts as an access (in LRU the key moves to the front).

- `CacheStats` uses `LongAdder`, which is faster than `AtomicLong` (a `long` that threads can update safely without a lock) when many threads increment.

Output:

```
== LRU (capacity 3): put a,b,c; get b,b,b,c,a,a; put d
  listener: b=2 left, EVICTED
  still cached: [a, c, d]
== FIFO (capacity 3): put a,b,c; get b,b,b,c,a,a; put d
  listener: a=1 left, EVICTED
  still cached: [b, c, d]
== LFU (capacity 3): put a,b,c; get b,b,b,c,a,a; put d
  listener: c=3 left, EVICTED
  still cached: [a, b, d]
== TTL with a manual clock
  t=0s  get session:42 -> alice
  listener: session:42 left, EXPIRED
  t=6s  get session:42 -> MISS
  listener: session:43 left, EXPIRED
  sweeper purged 1 more expired entry
  config (no TTL) -> v1
  listener: config left, REMOVED
  stats: hits=2 misses=1 evictions=0 expirations=2 hitRate=67%
== Sharded cache under 8 threads
  finished: true, size <= 8000: true
```

With the same access pattern (b read 3 times, then c once, then a twice), the three policies evict three **different** keys: LRU evicts b (least recently touched), FIFO evicts a (inserted first), LFU evicts c (read only once).

LRU Cache from scratch (the classic interview question)

Very often the interviewer skips the pluggable design and asks for exactly this: **"Implement an LRU cache with O(1) `get` and `put`"** (LeetCode 146; LeetCode is a popular website of coding interview problems). The answer is the same two structures, written inline:

1. A `HashMap<K, Node>` for lookup.
2. A doubly linked list with `head`/`tail` sentinels; most recent right after `head`, least recent right before `tail`.
3. `get(k)`: look up the node; if found, unlink it and add it at the front; return the value.
4. `put(k, v)`: if present, update and move to front. Otherwise, if full, remove `tail.prev` from the list **and** from the map (that is why the node stores its key); then add the new node at the front and to the map.

Java also has a shortcut: `LinkedHashMap` with `accessOrder = true` keeps entries in access order, and overriding `removeEldestEntry` evicts automatically. Mention it, but write the manual version when asked "from scratch" — that is what is being tested.

```java
// File: code/lld/ch20/lrucache/Main.java
package lld.ch20.lrucache;

import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.Map;

public class Main {

    /** The classic interview answer: HashMap + hand-written doubly linked list. get/put are O(1). */
    static final class LRUCache<K, V> {
        private final class Node {
            K key;
            V value;
            Node prev, next;
            Node(K key, V value) { this.key = key; this.value = value; }
        }

        private final int capacity;
        private final Map<K, Node> index = new HashMap<>();
        private final Node head = new Node(null, null); // sentinel: most recent side
        private final Node tail = new Node(null, null); // sentinel: least recent side

        LRUCache(int capacity) {
            if (capacity <= 0) throw new IllegalArgumentException("capacity must be positive");
            this.capacity = capacity;
            head.next = tail;
            tail.prev = head;
        }

        synchronized V get(K key) {
            Node n = index.get(key);
            if (n == null) return null;
            unlink(n);
            addFront(n);                // now the most recently used
            return n.value;
        }

        synchronized void put(K key, V value) {
            Node n = index.get(key);
            if (n != null) {            // update + mark as recently used
                n.value = value;
                unlink(n);
                addFront(n);
                return;
            }
            if (index.size() == capacity) {
                Node lru = tail.prev;   // least recently used lives just before the tail sentinel
                unlink(lru);
                index.remove(lru.key);
            }
            n = new Node(key, value);
            addFront(n);
            index.put(key, n);
        }

        private void addFront(Node n) {
            n.prev = head;
            n.next = head.next;
            head.next.prev = n;
            head.next = n;
        }

        private void unlink(Node n) {
            n.prev.next = n.next;
            n.next.prev = n.prev;
        }

        @Override
        public synchronized String toString() {
            StringBuilder sb = new StringBuilder("[");
            for (Node n = head.next; n != tail; n = n.next) sb.append(n.key).append(n.next == tail ? "" : ", ");
            return sb.append("] (most -> least recent)").toString();
```

```
        }
    }

    /** The JDK shortcut: LinkedHashMap in access order + removeEldestEntry. Not thread-safe. */
    static <K, V> Map<K, V> jdkLru(int capacity) {
        return new LinkedHashMap<>(16, 0.75f, true) {
            @Override
            protected boolean removeEldestEntry(Map.Entry<K, V> eldest) { return size() > capacity; }
        };
    }

    public static void main(String[] args) {
        System.out.println("== Hand-written LRU (LeetCode 146 sequence, capacity 2)");
        LRUCache<Integer, Integer> c = new LRUCache<>(2);
        c.put(1, 1);
        c.put(2, 2);
        System.out.println("get(1) = " + c.get(1) + "   order " + c);
        c.put(3, 3);
        System.out.println("put(3) evicts 2        order " + c);
        System.out.println("get(2) = " + c.get(2));
        c.put(4, 4);
        System.out.println("put(4) evicts 1        order " + c);
        System.out.println("get(1) = " + c.get(1) + ", get(3) = " + c.get(3) + ", get(4) = " + c.get(4));

        System.out.println("== Same sequence with LinkedHashMap(accessOrder=true)");
        Map<Integer, Integer> j = jdkLru(2);
        j.put(1, 1);
        j.put(2, 2);
        j.get(1);
        j.put(3, 3);
        j.put(4, 4);
        System.out.println("keys (least -> most recent) = " + j.keySet());
    }
}
```

Output:

```
== Hand-written LRU (LeetCode 146 sequence, capacity 2)
get(1) = 1   order [1, 2] (most -> least recent)
put(3) evicts 2        order [3, 1] (most -> least recent)
get(2) = null
put(4) evicts 1        order [4, 3] (most -> least recent)
get(1) = null, get(3) = 3, get(4) = 4
== Same sequence with LinkedHashMap(accessOrder=true)
keys (least -> most recent) = [3, 4]
```

The printed values match LeetCode 146's expected answers for this sequence (1, miss, miss, 3, 4); LeetCode's `int` version returns `-1` for a miss, while this generic version returns `null`.

Approach	get / put	Thread-safe	When to use
HashMap + own doubly linked list	O(1) / O(1)	Only with a lock (`synchronized` here)	Interviews; when you need custom behaviour
`LinkedHashMap(accessOrder=true)` + `removeEldestEntry`	O(1) / O(1)	No (wrap with a lock)	Quick, small, single-threaded caches
Caffeine / Guava cache (widely used Java caching libraries)	amortized O(1) (O(1) on average over many calls)	Yes, highly concurrent	Production Java

Concurrency & edge cases

- **Why a plain read lock is not enough:** an LRU `get` moves the node (several pointer writes). Two threads doing that at once corrupt the list. So `get` needs the exclusive lock.
- **Contention:** one lock serializes everything. `ShardedCache` splits keys over 8 caches (8 locks); the demo hammers it from 8 threads and the size never exceeds the total capacity. Trade-off: eviction is per shard, so the global LRU order is approximate.
- **Listeners:** called outside the lock; a listener that calls back into the cache cannot deadlock.

- **Cache stampede** (many threads miss the same key at once and all hit the database): use a "loading" cache where the first thread computes and the others wait for its result (`ConcurrentHashMap.compute` / `computeIfAbsent` — `computeIfAbsent` creates and stores a value only if the key is missing, as one atomic step — or a `CompletableFuture` stored per key). Chapter 13 (§13.6) builds a TTL cache whose `getOrLoad` does this with `ConcurrentHashMap.compute`.
- **TTL semantics, precisely.** An entry expires *at* its deadline (`now >= expiresAt`), so a 5-second TTL written at t=0 is already a miss at t=5.000 s. Expiry is **lazy**: a dead entry still occupies a slot until it is read, swept by `purgeExpired()`, or picked as the eviction victim (in which case it is reported as `EVICTED`, not `EXPIRED`, because it left to make room). `put(key, value)` without a TTL on an existing key **clears** the old TTL (the entry becomes permanent); pass the TTL again to keep one. A production cache would also let the sweeper run on a `ScheduledExecutorService` (Java's tool for running tasks on a timer).
- **Capacity 0 or negative:** refused by the builder.
- **Null keys/values:** not supported here; say so (Caffeine forbids nulls too).
- **Clock going backwards:** use a monotonic clock (one that only ever moves forward, unlike wall-clock time that can be adjusted; `System.nanoTime`) for TTL in production.

Extensions & follow-up questions

- **"Make it LFU."** One new policy class (done). Volunteer that the naive LFU (scan for the lowest count) is O(n) and the chained-bucket version is O(1).
- **"Size in bytes, not entries."** Add a `Weigher<K,V>` strategy; evict until `totalWeight <= maxWeight`.
- **"Load from the database on a miss."** *Read-through*: `get(key, loader)` calls the loader on a miss and stores the result. *Write-through*: writes go to cache and DB together. *Write-back (write-behind)*: writes go to cache and are flushed to DB later (faster, risk of loss). *Cache-aside*: the application does "check cache → on miss read DB → put in cache" itself.
- **"Distributed cache."** Consistent hashing (a way to assign keys to servers so that adding or removing a server moves only a small share of the keys) to spread keys over nodes (Redis Cluster, Memcached clients — Redis and Memcached are in-memory key-value stores that run as separate servers); that is high-level design (HLD: designing services, databases and networks, as opposed to the classes inside one program).
- **"Which policy is best?"** Depends on access patterns. Modern caches (Caffeine's *Window TinyLFU*) combine recency and frequency and beat pure LRU on many workloads (patterns of real requests).

Real-world example

`java.util.LinkedHashMap` with access order is the JDK's (Java Development Kit: Java's compiler plus standard library) built-in LRU. **Guava's** `CacheBuilder` and **Caffeine** expose exactly this chapter's knobs: `maximumSize` (capacity), `expireAfterWrite` / `expireAfterAccess` (TTL), `removalListener` (Observer, with a cause such as `EXPIRED`, `SIZE`, `EXPLICIT`, `REPLACED`), and `recordStats()` (hit rate: the share of reads answered from the cache). **Redis** supports eviction policies selected by configuration (`maxmemory-policy`: `allkeys-lru`, `allkeys-lfu`, `volatile-ttl`, `noeviction` and others) — a Strategy chosen by config — and approximates LRU by sampling a few keys rather than keeping a full list, to save memory. Browsers, CDNs (content delivery networks: servers spread around the world that keep copies of files close to users) and operating-system page caches (recently read disk data kept in memory) all use variants of LRU.

In the interview

> **30-second answer:** "A HashMap from key to node for O(1) lookup, and a doubly linked list with sentinels for O(1) ordering; the node carries its own links so the map hands us something we can unlink directly. The eviction rule goes behind an `EvictionPolicy` interface — LRU moves on access, FIFO doesn't, LFU uses frequency buckets for O(1). Every exit goes through one `remove` method that updates stats and notifies listeners after the lock is released. For concurrency, one lock is correct; sharding by key hash reduces contention."

Plan: sketch the map and the list together straight away and explain what each one lacks on its own → write LRU get/put → pull the rule behind an interface unprompted → TTL + stats + listeners → thread safety and sharding → follow-ups.

Follow-ups with answers:

- *"Why doubly and not singly linked?"* → Removing a node needs its predecessor; with `prev` we have it in O(1).
- *"Why does the node store the key?"* → When evicting the tail we must also delete it from the map.
- *"Make it thread-safe."* → Lock around each operation; better, shard by key; production: Caffeine.

Self-check

1. Which two structures give O(1) LRU? → HashMap (key → node) + doubly linked list (recency order).
2. What do sentinel nodes buy you? → No null checks; add/remove are always the same pointer writes.
3. How does FIFO differ from LRU in code? → `onAccess` does nothing.
4. Why call listeners after releasing the lock? → A slow or re-entrant listener cannot block or deadlock the cache.
5. Why can't a read-write lock speed up an LRU cache's `get`? → `get` modifies the list, so it is a write.
6. What is a cache stampede and one fix? → Many concurrent misses for one key hitting the DB; let one thread load while others wait on the same future.

TL;DR

- **HashMap + doubly linked list (with sentinels)** = O(1) get/put.
- **Strategy** for eviction (LRU / FIFO / O(1) LFU); **Observer** for removals; **Builder** for config.
- **One removal path** keeps stats and listeners correct; TTL is lazy + optional sweeper; clock injected.
- Thread safety: one lock (gets are writes), listeners outside the lock, **shard** to scale.
- Know `LinkedHashMap(accessOrder=true)` + `removeEldestEntry`, and Caffeine/Guava/Redis policies.

20.4 Design a Ride Sharing Service (Uber)

Problem & clarifying questions

What it is. A ride sharing service connects a rider who wants a car with a nearby driver. Everyday analogy: a taxi dispatcher with a radio. A call comes in, the dispatcher looks at the map, radios the nearest free cab; if that driver says no, the next one. At the end the meter shows the fare.

Two decisions are the whole question: **which driver is asked** (matching) and **what the trip costs** (pricing). The rest is bookkeeping. A good design lets either rule change with no edits anywhere else.

Question	Assumed answer
Vehicle types?	Economy, premium, van. Rider asks for one type.
Who can get a request?	Drivers who are online, not on a trip, with the right car type, within range.
Offer to one driver or broadcast?	One at a time, nearest first; decline → next; nobody left → cancel.
Trip states?	Requested → accepted → started → ended; canceled from requested/accepted only. Illegal actions do nothing.
Fare?	Base + per km + per minute, never below a minimum; per vehicle type.
Surge?	A multiplier when the city is busy, applied to the same fare.
Ratings?	Rider and driver rate each other once after a finished trip; accounts show the average.
Payments?	Charge the rider's payment method (cash, wallet, card) at the end.
Map?	A grid; distance measured along streets (Manhattan distance), which keeps all distances integral (whole numbers). Production systems query a routing engine (a service that works out the road route between two points) that gives back road distance plus an estimated duration, and pricing would take both from it.
Pool/shared rides, scheduled rides, live tracking, payouts?	Out of scope.

Requirements

Functional

1. Riders and drivers have a name and phone number.
2. A driver has one vehicle: plate, seats, type.
3. Drivers go online/offline; only online, free drivers get requests.
4. A rider requests a trip with pickup, drop and car type.
5. Free drivers with a matching car type are ordered by distance to the pickup, and the closest one is offered the trip first.
6. A driver accepts or declines; a decline goes to the next driver; if none are left, the request is canceled.
7. A trip moves through fixed states; an action that doesn't fit the current state does nothing.
8. Fare = base + per-km + per-minute, never below a minimum.
9. During peak demand, a surge multiplier scales that same fare.
10. Riders may cancel until the trip starts; after that, the only way out is ending it.
11. After a finished trip, rider and driver rate each other; accounts show the average.
12. Riders and drivers can see their finished trips.
13. (Added) The rider is charged through their payment method.

Non-functional: a driver can never be assigned to two trips at once (even with concurrent requests); rules swappable without editing the service; deterministic behaviour for tests (tie-breaks).

Use cases

Actors: **Rider** (request, cancel, rate), **Driver** (go online, accept/decline, start/end, rate), **System** (find nearby drivers, offer, price, surge, history).

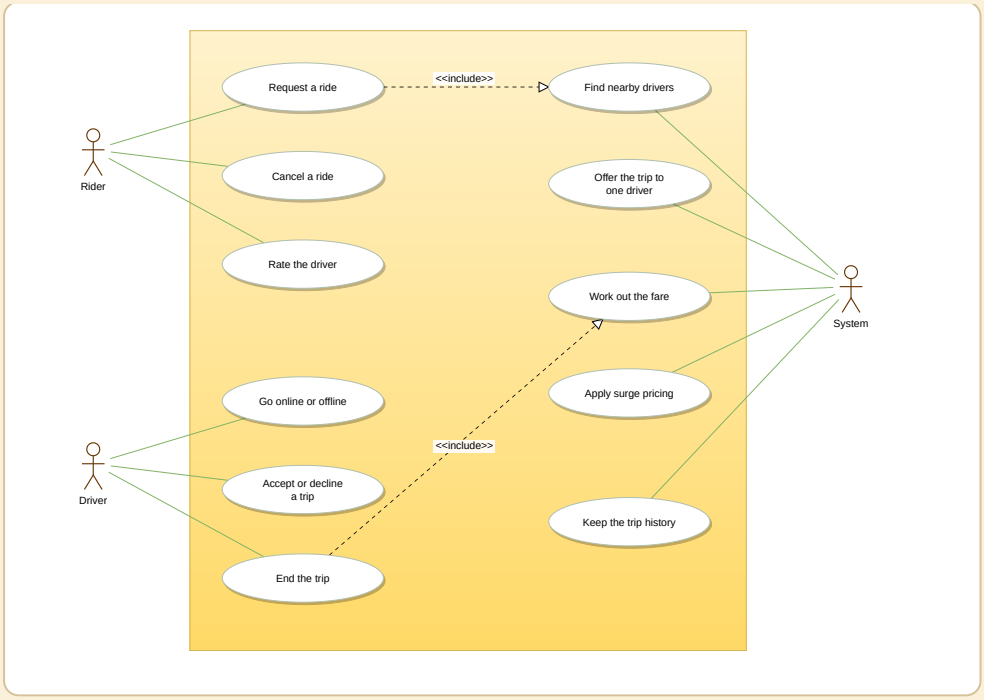


Figure — Ride sharing use cases: rider requests, cancels and rates; driver goes online, accepts or declines and ends the trip; the system matches, offers, prices and applies surge (course diagram)

Core entities

Class	Responsibility
`Location` (record)	Grid point; `distanceTo` = Manhattan distance in km.
`VehicleType` (enum), `Vehicle` (record)	Car type and details.
`Account` (abstract)	What riders and drivers share: id, name, phone, **rating as running total + count**, finished trips.
`Rider`	+ payment method.
`Driver`	+ vehicle, location, online flag, **atomic busy flag** (`tryClaim`).
`TripStatus` (enum)	The state machine: legal next states per state.
`Trip`	One journey: rider, pickup, drop, type, **ranked candidate list + cursor** (a marker for the current position in the list), status, driver, minutes, fare. Every action checks state first.
`PricingStrategy`	**Strategy**: fare for (type, km, minutes).
`StandardPricing` + `RateCard`	Base + per km + per minute, with a floor (a minimum fare), per vehicle type.
`SurgePricing`	**Decorator**: wraps any `PricingStrategy` and multiplies its answer.
`MatchingStrategy`	**Strategy**: ranks drivers. `NearestDriverMatcher`, `BestRatedMatcher`.
`PaymentMethod`	**Strategy**: `CashPayment`, `WalletPayment`.
`TripListener`	**Observer**: apps are told about trip events.
`RideSharingService`	Facade: owns accounts and trips; holds one pricing and one matching rule; knows only the order of steps.

Why the rating is a total and a count. Only the average is ever displayed, so a sum and a count suffice, and recording a new rating is O(1) whether the account has 10 trips or 10,000.

Why `Rider` and `Driver` extend `Account`. They pass the "is-a" substitution test (Liskov Substitution Principle): wherever the code reads a name or records a rating, a `Rider` or a `Driver` can be passed.

Class diagram

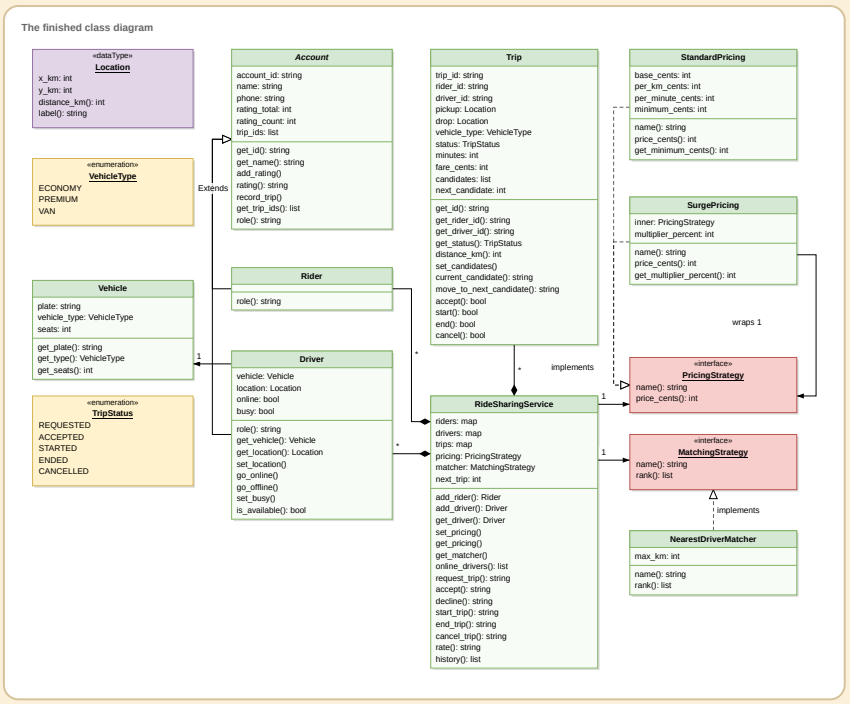
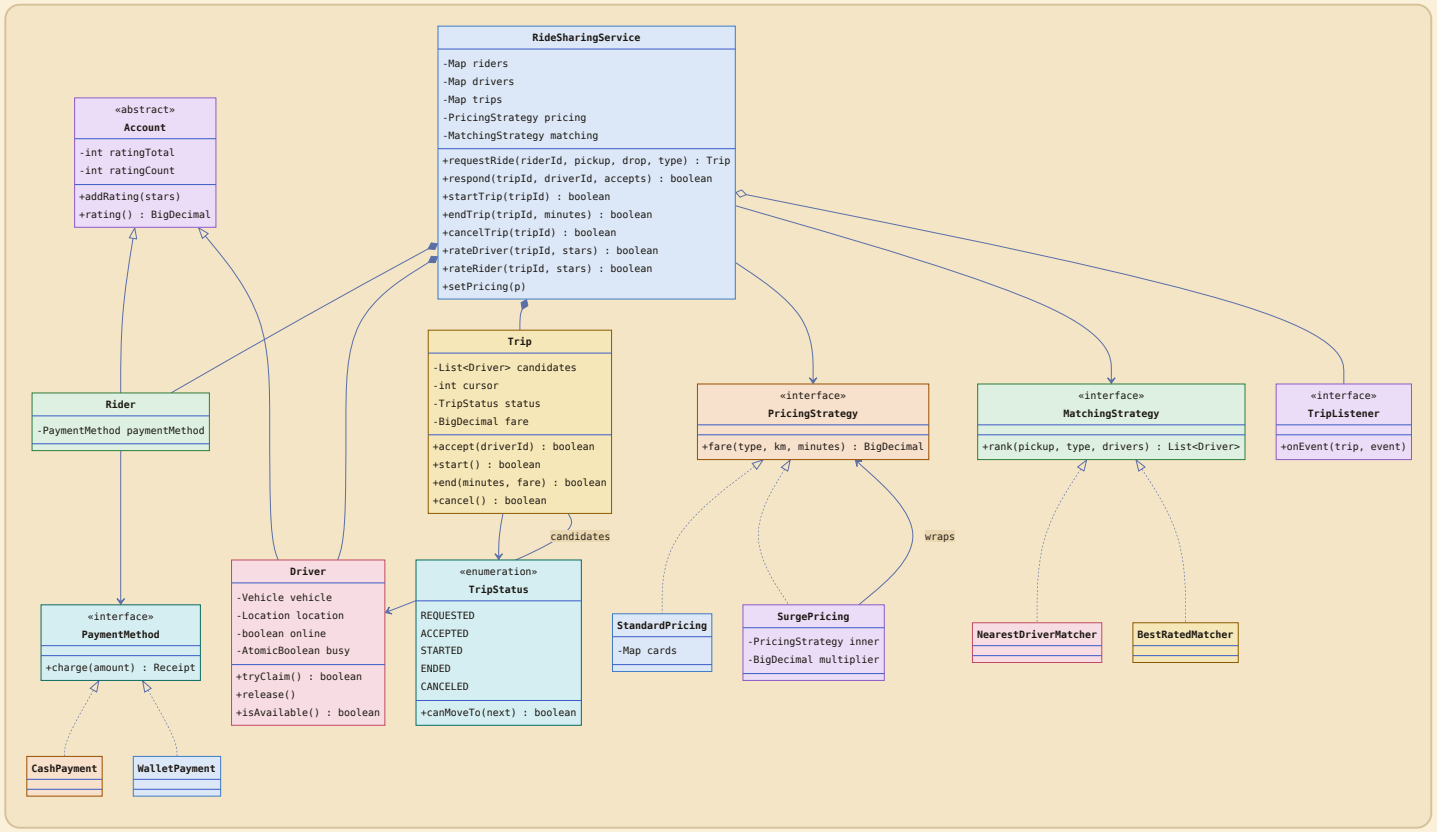


Figure — The finished ride sharing class diagram: the service owning riders, drivers and trips; PricingStrategy with StandardPricing and the SurgePricing wrapper; MatchingStrategy with the nearest-driver matcher (course diagram)

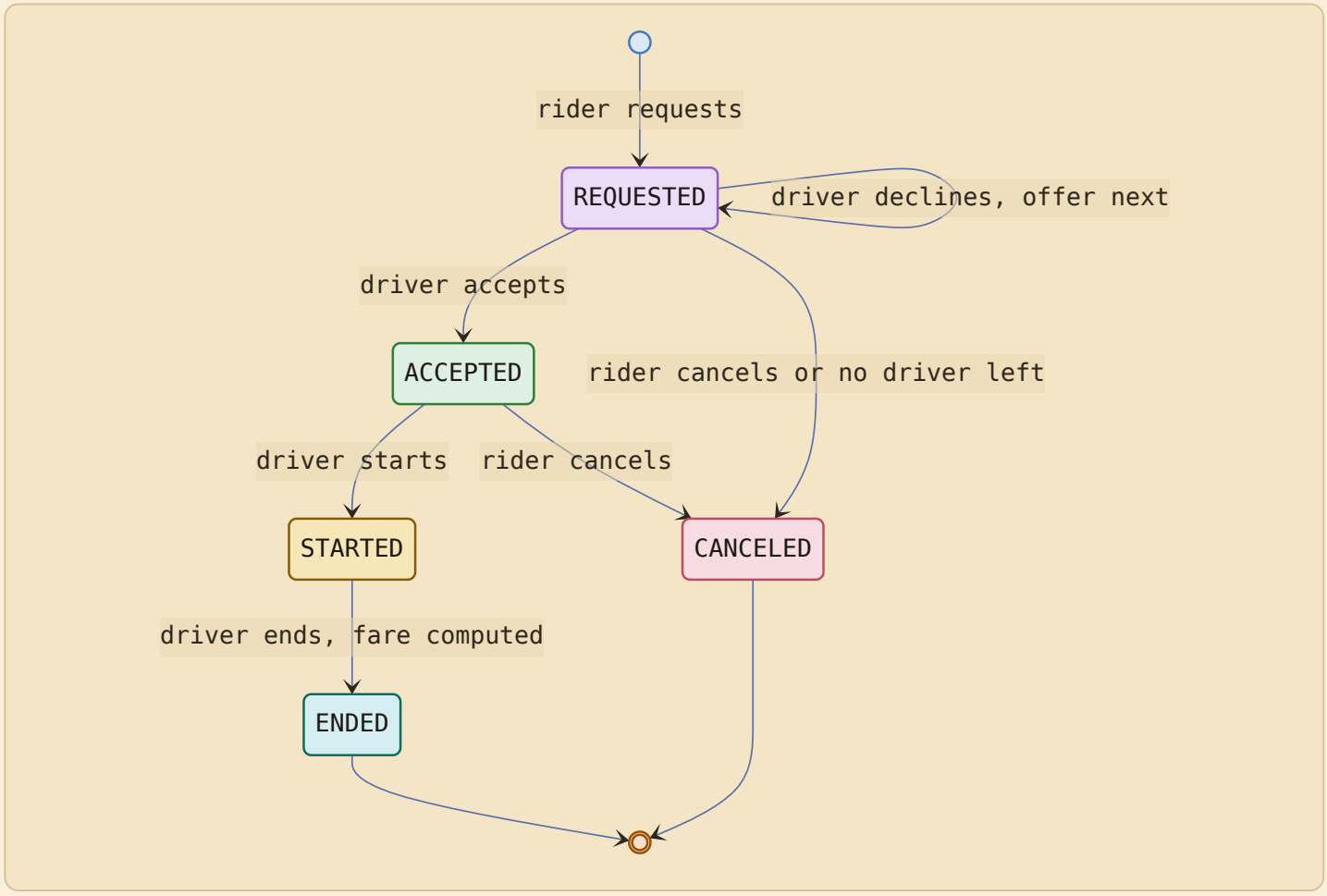
The course builds it in five stages, each leaving something runnable: (1) `Account` / `Rider` / `Driver`, `Vehicle`, `Location` (who is free and where); (2) `Trip` + `TripStatus` (the lifecycle, with illegal moves refused); (3) pricing with the surge wrapper (a finished trip has a fare); (4) matching (drivers who could take a request can be ranked); (5) `RideSharingService` (the complete flow, request through rating).

In the figure above, look for the extra arrow from `SurgePricing` back to the `PricingStrategy` interface: that "has-a of my own type" arrow is what turns it into a Decorator instead of one more Strategy.



Key flows

Trip lifecycle (state machine). A *state machine* is a fixed set of states plus the only moves allowed between them.



Finding a driver — including the case where every candidate declines:

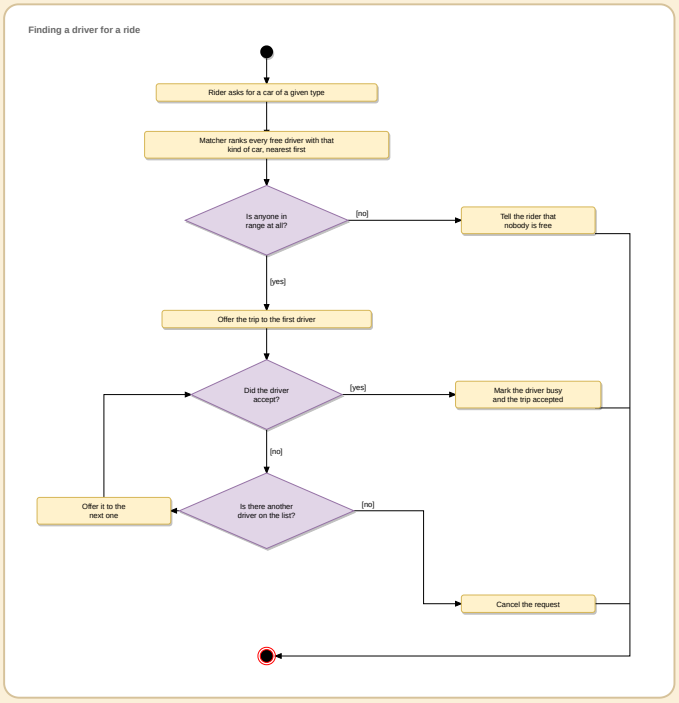


Figure — Finding a driver: rank free drivers of the right type nearest first; offer to the first; on decline move to the next; cancel when nobody is left (course diagram)

Ending and charging a trip — pricing is a single call no matter how many decorators wrap the rule:

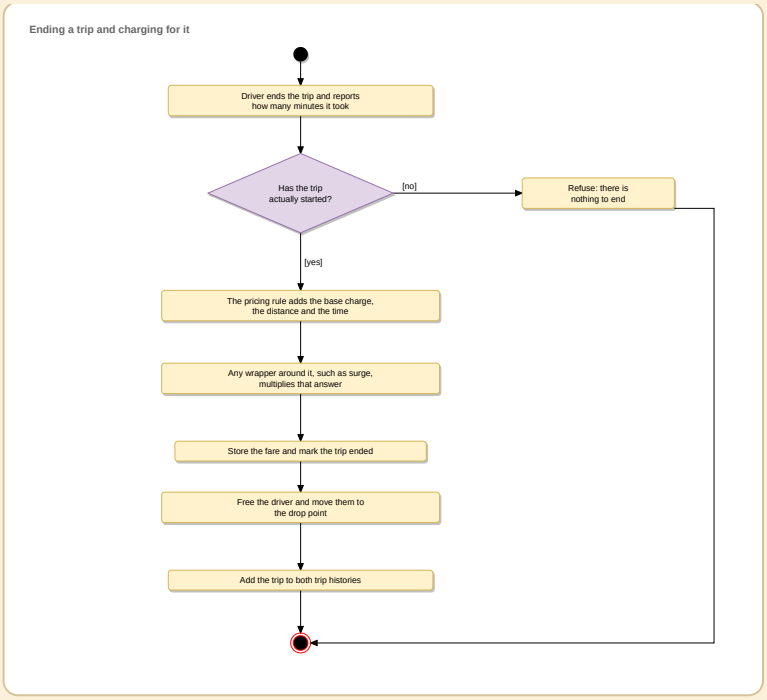
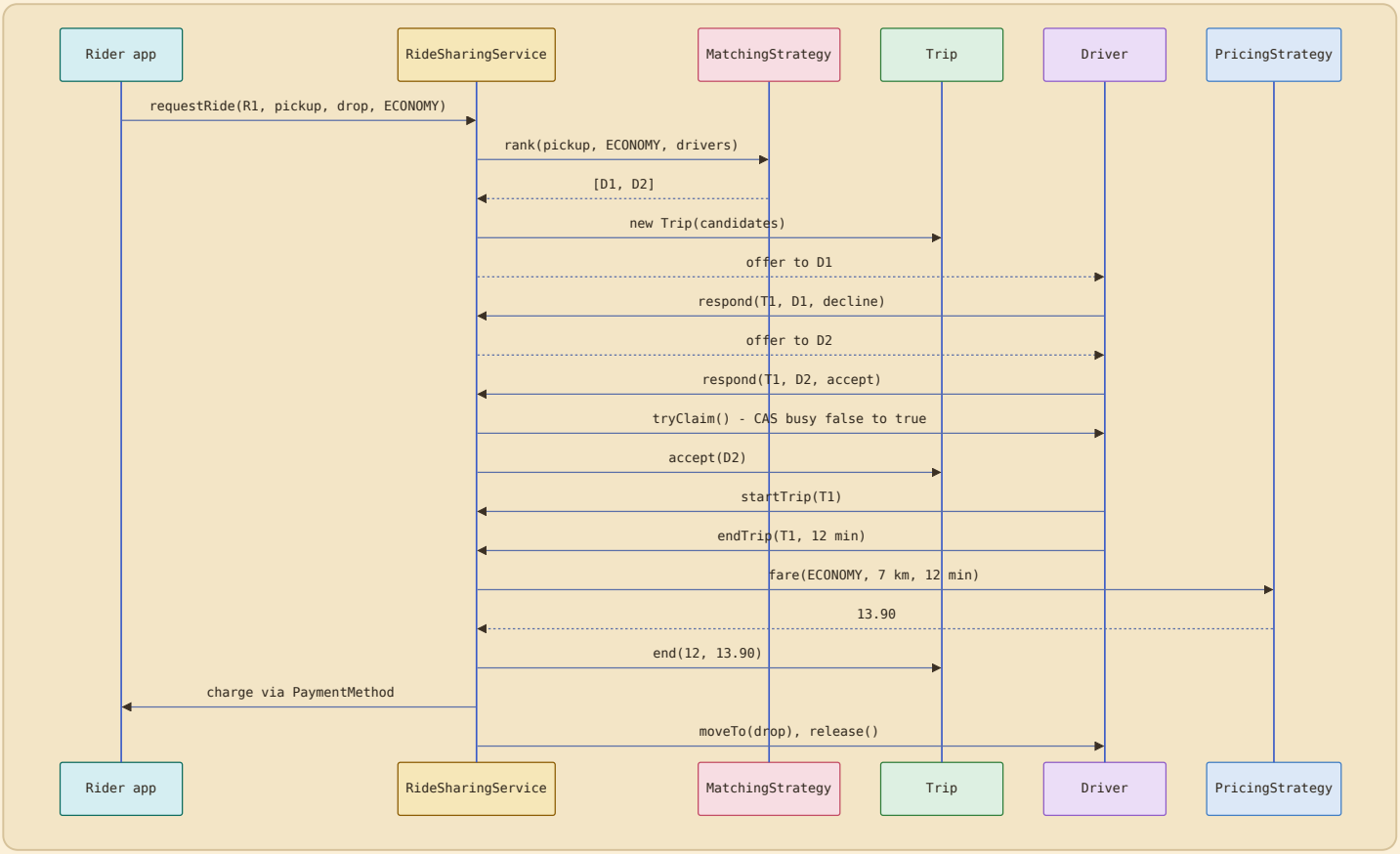


Figure — Ending a trip: refuse if not started; otherwise the pricing rule computes base plus distance plus time, any surge wrapper multiplies it, and the trip is recorded (course diagram)



Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for matching	Cities rank differently (nearest, best-rated, least-earned-today). One class each.	Sorting logic inside the service.
Matcher returns a **ranked list**, stored on the trip	A decline just moves the cursor; no new city-wide search, and the system remembers who refused.	Returning one driver: every decline re-searches from scratch.
Strategy for pricing, **Decorator** for surge	Surge wraps *any* pricing rule (even future ones). Turning it off = removing the wrapper, not editing an `if`. Decorators stack (surge + airport fee + discount).	A `surgeMultiplier` field and `if (surge)` in the pricing code.
State machine via `TripStatus.canMoveTo`	One table says what is legal; every method checks it and returns `false` on an illegal move. No exceptions for normal refusals.	Scattered `if (status == …)` checks; or the full State pattern (one class per state). The State pattern is worth it when behaviour per state is large; here a transition table is enough.
Atomic claim of a driver (`AtomicBoolean.compareAndSet`: "set to true only if it is still false", in one indivisible step)	Two trips racing for the same driver: exactly one wins, the other treats it as a decline.	Checking `isAvailable()` then setting busy: a check-then-act race (a race condition: the result depends on which thread happens to run first).
Driver **moved to the drop point** at trip end	Otherwise the next match uses where they were this morning.	Forgetting (a real bug class).
Observer for trip events	Rider/driver apps, push notifications, analytics (code that collects usage data for reports) listen; the service doesn't know them.	Service calling the notification code directly.
Deterministic tie-breaks (by driver id)	Same input → same ranking → reproducible tests.	Unordered ties: flaky tests (tests that sometimes pass and sometimes fail with no code change).
Money in `BigDecimal`, rounded `HALF_UP` (halves round up: 2.345 → 2.35) to 2 places	Exact fares. (The course keeps money as whole cents and the surge multiplier as a whole-number percentage, 150 = ×1.5, so no floating point (binary fractional numbers such as `double`) appears; `BigDecimal` is this book's equivalent.)	`double`.

Java implementation

```java
// File: code/lld/ch20/ridesharing/Model.java
package lld.ch20.ridesharing;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.concurrent.atomic.AtomicBoolean;

/** A point on a city grid. Distance is measured along streets (Manhattan distance), in km. */
record Location(int x, int y) {
    int distanceTo(Location o) { return Math.abs(x - o.x) + Math.abs(y - o.y); }
    @Override public String toString() { return "(" + x + "," + y + ")"; }
}

enum VehicleType { ECONOMY, PREMIUM, VAN }

record Vehicle(String plate, VehicleType type, int seats) {}

/** What riders and drivers share. Rating = running total + count (average is all anyone reads). */
abstract class Account {
    private final String id;
    private final String name;
    private final String phone;
    private int ratingTotal;
    private int ratingCount;
    private final List<String> finishedTrips = new ArrayList<>();

    Account(String id, String name, String phone) {
        this.id = id;
        this.name = name;
        this.phone = phone;
    }

    String id() { return id; }
    String name() { return name; }
    String phone() { return phone; }

    synchronized void addRating(int stars) {
        if (stars < 1 || stars > 5) throw new IllegalArgumentException("stars must be 1..5");
        ratingTotal += stars;
        ratingCount++;
    }

    synchronized BigDecimal rating() {
        return ratingCount == 0 ? BigDecimal.ZERO
                : BigDecimal.valueOf(ratingTotal).divide(BigDecimal.valueOf(ratingCount), 2, RoundingMode.HALF_UP);
    }

    synchronized void recordTrip(String tripId) { finishedTrips.add(tripId); }
    synchronized List<String> finishedTrips() { return Collections.unmodifiableList(new ArrayList<>(finishedTrips)); }
}

final class Rider extends Account {
    private final PaymentMethod paymentMethod;

    Rider(String id, String name, String phone, PaymentMethod paymentMethod) {
        super(id, name, phone);
        this.paymentMethod = paymentMethod;
    }

    PaymentMethod paymentMethod() { return paymentMethod; }
}

final class Driver extends Account {
    private final Vehicle vehicle;
    private volatile Location location;
    private volatile boolean online;
    private final AtomicBoolean busy = new AtomicBoolean(false);
```

```
    Driver(String id, String name, String phone, Vehicle vehicle, Location location) {
        super(id, name, phone);
        this.vehicle = vehicle;
        this.location = location;
    }

    Vehicle vehicle() { return vehicle; }
    Location location() { return location; }
    void moveTo(Location l) { this.location = l; }
    void setOnline(boolean online) { this.online = online; }
    boolean isAvailable() { return online && !busy.get(); }

    /** Atomic claim: of two trips racing for this driver, exactly one wins. */
    boolean tryClaim() { return online && busy.compareAndSet(false, true); }
    void release() { busy.set(false); }
}
```

```java
// File: code/lld/ch20/ridesharing/Trip.java
package lld.ch20.ridesharing;

import java.math.BigDecimal;
import java.util.EnumSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

/** The trip lifecycle as a table of legal moves (a compact State machine). */
enum TripStatus {
    REQUESTED, ACCEPTED, STARTED, ENDED, CANCELED;

    private static final Map<TripStatus, Set<TripStatus>> NEXT = Map.of(
            REQUESTED, EnumSet.of(ACCEPTED, CANCELED),
            ACCEPTED, EnumSet.of(STARTED, CANCELED),
            STARTED, EnumSet.of(ENDED),
            ENDED, EnumSet.noneOf(TripStatus.class),
            CANCELED, EnumSet.noneOf(TripStatus.class));

    boolean canMoveTo(TripStatus next) { return NEXT.get(this).contains(next); }
}

/** One journey. Every action checks the status first; an illegal action returns false and changes nothing. */
final class Trip {
    private final String id;
    private final String riderId;
    private final Location pickup;
    private final Location drop;
    private final VehicleType type;
    private final List<Driver> candidates; // ranked once; a decline just moves the cursor
    private int cursor;
    private TripStatus status = TripStatus.REQUESTED;
    private String driverId;
    private int minutes;
    private BigDecimal fare;
    private boolean riderRated, driverRated;

    Trip(String id, String riderId, Location pickup, Location drop, VehicleType type, List<Driver> candidates) {
        this.id = id;
        this.riderId = riderId;
        this.pickup = pickup;
        this.drop = drop;
        this.type = type;
        this.candidates = List.copyOf(candidates);
    }

    private boolean move(TripStatus next) {
        if (!status.canMoveTo(next)) return false;
        status = next;
        return true;
    }

    /** The driver currently being offered this trip, or null if the list is exhausted. */
    synchronized Driver currentCandidate() { return cursor < candidates.size() ? candidates.get(cursor) : null; }
    synchronized Driver nextCandidate() { cursor++; return currentCandidate(); }

    synchronized boolean accept(String driverId) {
        if (status != TripStatus.REQUESTED) return false;
        this.driverId = driverId;
        return move(TripStatus.ACCEPTED);
    }

    synchronized boolean start() { return move(TripStatus.STARTED); }

    synchronized boolean end(int minutes, BigDecimal fare) {
        if (!move(TripStatus.ENDED)) return false;
        this.minutes = minutes;
        this.fare = fare;
        return true;
    }

    synchronized boolean cancel() { return move(TripStatus.CANCELED); }
```

```
    synchronized boolean markRiderRated() { if (status != TripStatus.ENDED || riderRated) return false; return
riderRated = true; }
    synchronized boolean markDriverRated() { if (status != TripStatus.ENDED || driverRated) return false; return
driverRated = true; }

    String id() { return id; }
    String riderId() { return riderId; }
    Location pickup() { return pickup; }
    Location drop() { return drop; }
    VehicleType type() { return type; }
    int distanceKm() { return pickup.distanceTo(drop); }
    synchronized TripStatus status() { return status; }
    synchronized String driverId() { return driverId; }
    synchronized int minutes() { return minutes; }
    synchronized BigDecimal fare() { return fare; }
}
```

```java
// File: code/lld/ch20/ridesharing/Strategies.java
package lld.ch20.ridesharing;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.Collection;
import java.util.Comparator;
import java.util.List;
import java.util.Map;

/* ---------------- Pricing: Strategy + Decorator ---------------- */

/** What a finished trip costs. */
interface PricingStrategy {
    BigDecimal fare(VehicleType type, int distanceKm, int minutes);
}

record RateCard(BigDecimal base, BigDecimal perKm, BigDecimal perMinute, BigDecimal minimum) {}

/** Base + distance + time, with a floor. One rate card per vehicle type. */
final class StandardPricing implements PricingStrategy {
    private final Map<VehicleType, RateCard> cards;

    StandardPricing(Map<VehicleType, RateCard> cards) { this.cards = Map.copyOf(cards); }

    @Override
    public BigDecimal fare(VehicleType type, int km, int minutes) {
        RateCard c = cards.get(type);
        BigDecimal raw = c.base()
                .add(c.perKm().multiply(BigDecimal.valueOf(km)))
                .add(c.perMinute().multiply(BigDecimal.valueOf(minutes)));
        return raw.max(c.minimum()).setScale(2, RoundingMode.HALF_UP);
    }
}

/** Decorator: wraps ANY pricing rule and multiplies its answer. Remove the wrapper to switch surge off. */
final class SurgePricing implements PricingStrategy {
    private final PricingStrategy inner;
    private final BigDecimal multiplier;

    SurgePricing(PricingStrategy inner, BigDecimal multiplier) {
        this.inner = inner;
        this.multiplier = multiplier;
    }

    @Override
    public BigDecimal fare(VehicleType type, int km, int minutes) {
        return inner.fare(type, km, minutes).multiply(multiplier).setScale(2, RoundingMode.HALF_UP);
    }
}

/* ---------------- Matching: Strategy ---------------- */

/** Returns a RANKED LIST (not one driver) so a decline just moves to the next name. */
interface MatchingStrategy {
    List<Driver> rank(Location pickup, VehicleType type, Collection<Driver> drivers);
}

final class NearestDriverMatcher implements MatchingStrategy {
    private final int maxRadiusKm;

    NearestDriverMatcher(int maxRadiusKm) { this.maxRadiusKm = maxRadiusKm; }

    @Override
    public List<Driver> rank(Location pickup, VehicleType type, Collection<Driver> drivers) {
        return drivers.stream()
                .filter(Driver::isAvailable)
                .filter(d -> d.vehicle().type() == type)
                .filter(d -> d.location().distanceTo(pickup) <= maxRadiusKm)
                .sorted(Comparator.comparingInt((Driver d) -> d.location().distanceTo(pickup))
                        .thenComparing(Driver::id))            // deterministic tie-break
                .toList();
    }
```

```
}

/** A second rule to show the swap: best-rated first, nearest as tie-break. */
final class BestRatedMatcher implements MatchingStrategy {
    private final NearestDriverMatcher nearby;

    BestRatedMatcher(int maxRadiusKm) { this.nearby = new NearestDriverMatcher(maxRadiusKm); }

    @Override
    public List<Driver> rank(Location pickup, VehicleType type, Collection<Driver> drivers) {
        return nearby.rank(pickup, type, drivers).stream()
                .sorted(Comparator.comparing(Driver::rating).reversed()
                        .thenComparingInt(d -> d.location().distanceTo(pickup)))
                .toList();
    }
}

/* ---------------- Payment: Strategy ---------------- */

record Receipt(boolean paid, String detail) {}

interface PaymentMethod {
    Receipt charge(BigDecimal amount);
}

final class CashPayment implements PaymentMethod {
    public Receipt charge(BigDecimal amount) { return new Receipt(true, "collect " + amount + " in cash"); }
}

final class WalletPayment implements PaymentMethod {
    private BigDecimal balance;

    WalletPayment(BigDecimal balance) { this.balance = balance; }

    public synchronized Receipt charge(BigDecimal amount) {
        if (balance.compareTo(amount) < 0) return new Receipt(false, "wallet has only " + balance);
        balance = balance.subtract(amount);
        return new Receipt(true, "wallet debited " + amount + ", left " + balance);
    }
}
```

```java
// File: code/lld/ch20/ridesharing/RideSharingService.java
package lld.ch20.ridesharing;

import java.math.BigDecimal;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicInteger;

/** Observer: rider/driver apps (push notifications) listen to trip events. */
interface TripListener {
    void onEvent(Trip trip, String event);
}

/** The class the apps talk to. Owns accounts and trips; holds the pricing and matching rules. */
final class RideSharingService {
    private final Map<String, Rider> riders = new ConcurrentHashMap<>();
    private final Map<String, Driver> drivers = new ConcurrentHashMap<>();
    private final Map<String, Trip> trips = new ConcurrentHashMap<>();
    private final List<TripListener> listeners = new CopyOnWriteArrayList<>();
    private final AtomicInteger ids = new AtomicInteger();
    private volatile PricingStrategy pricing;
    private volatile MatchingStrategy matching;

    RideSharingService(PricingStrategy pricing, MatchingStrategy matching) {
        this.pricing = pricing;
        this.matching = matching;
    }

    void setPricing(PricingStrategy p) { this.pricing = p; }   // e.g. wrap in SurgePricing at rush hour
    void setMatching(MatchingStrategy m) { this.matching = m; }
    void addListener(TripListener l) { listeners.add(l); }
    void addRider(Rider r) { riders.put(r.id(), r); }
    void addDriver(Driver d) { drivers.put(d.id(), d); }
    Driver driver(String id) { return drivers.get(id); }
    Rider rider(String id) { return riders.get(id); }

    /** Rank candidates once, then offer to the first. No candidates -> canceled immediately. */
    Trip requestRide(String riderId, Location pickup, Location drop, VehicleType type) {
        List<Driver> ranked = matching.rank(pickup, type, drivers.values());
        Trip trip = new Trip("T" + ids.incrementAndGet(), riderId, pickup, drop, type, ranked);
        trips.put(trip.id(), trip);
        publish(trip, "requested by " + riderId + ", candidates " + ranked.stream().map(Driver::id).toList());
        offerOrCancel(trip, trip.currentCandidate());
        return trip;
    }

    /** Driver answers the offer. A lost race for the driver counts as a decline. */
    boolean respond(String tripId, String driverId, boolean accepts) {
        Trip trip = trips.get(tripId);
        Driver current = trip.currentCandidate();
        if (current == null || !current.id().equals(driverId) || trip.status() != TripStatus.REQUESTED) return false;
        if (accepts && current.tryClaim()) {
            if (trip.accept(driverId)) {
                publish(trip, "accepted by " + driverId);
                return true;
            }
            current.release(); // trip was canceled meanwhile
            return false;
        }
        publish(trip, (accepts ? "could not claim " : "declined by ") + driverId);
        offerOrCancel(trip, trip.nextCandidate());
        return false;
    }

    boolean startTrip(String tripId) {
        Trip trip = trips.get(tripId);
        boolean ok = trip.start();
        publish(trip, ok ? "started" : "start refused in state " + trip.status());
        return ok;
    }
```

```
    /** Price -> store -> charge -> free the driver -> move driver to drop point -> history. */
    boolean endTrip(String tripId, int minutes) {
        Trip trip = trips.get(tripId);
        BigDecimal fare = pricing.fare(trip.type(), trip.distanceKm(), minutes);
        if (!trip.end(minutes, fare)) {
            publish(trip, "end refused in state " + trip.status());
            return false;
        }
        Rider rider = riders.get(trip.riderId());
        Receipt receipt = rider.paymentMethod().charge(fare);
        Driver d = drivers.get(trip.driverId());
        d.moveTo(trip.drop());       // easy to forget: the car is now at the drop point
        d.release();
        rider.recordTrip(trip.id());
        d.recordTrip(trip.id());
        publish(trip, "ended: " + trip.distanceKm() + " km, " + minutes + " min, fare " + fare
                + " (" + receipt.detail() + ")");
        return true;
    }

    /** Rider may cancel before the trip starts. */
    boolean cancelTrip(String tripId) {
        Trip trip = trips.get(tripId);
        if (!trip.cancel()) {
            publish(trip, "cancel refused in state " + trip.status());
            return false;
        }
        String driverId = trip.driverId(); // read AFTER cancel: no accept can happen any more
        if (driverId != null) drivers.get(driverId).release();
        publish(trip, "canceled by rider");
        return true;
    }

    boolean rateDriver(String tripId, int stars) {
        Trip t = trips.get(tripId);
        if (!t.markDriverRated()) return false;
        drivers.get(t.driverId()).addRating(stars);
        return true;
    }

    boolean rateRider(String tripId, int stars) {
        Trip t = trips.get(tripId);
        if (!t.markRiderRated()) return false;
        riders.get(t.riderId()).addRating(stars);
        return true;
    }

    private void offerOrCancel(Trip trip, Driver next) {
        if (next != null) {
            publish(trip, "offered to " + next.id());
        } else if (trip.cancel()) {
            publish(trip, "no driver left - canceled");
        }
    }

    private void publish(Trip trip, String event) {
        for (TripListener l : listeners) l.onEvent(trip, event);
    }
}
```

```java
// File: code/lld/ch20/ridesharing/Main.java
package lld.ch20.ridesharing;

import java.math.BigDecimal;
import java.util.Map;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    public static void main(String[] args) throws Exception {
        StandardPricing standard = new StandardPricing(Map.of(
                VehicleType.ECONOMY, new RateCard(bd("2.50"), bd("1.20"), bd("0.25"), bd("5.00")),
                VehicleType.PREMIUM, new RateCard(bd("5.00"), bd("2.00"), bd("0.40"), bd("10.00")),
                VehicleType.VAN, new RateCard(bd("4.00"), bd("1.80"), bd("0.30"), bd("8.00"))));
        RideSharingService svc = new RideSharingService(standard, new NearestDriverMatcher(10));
        svc.addListener((trip, event) -> System.out.println("  [" + trip.id() + "] " + event));

        svc.addRider(new Rider("R1", "Riya", "+91-1", new WalletPayment(bd("50.00"))));
        svc.addRider(new Rider("R2", "Sam", "+91-2", new CashPayment()));
        Driver d1 = new Driver("D1", "Dev", "+91-3", new Vehicle("KA-01", VehicleType.ECONOMY, 4), new Location(0,
0));
        Driver d2 = new Driver("D2", "Ira", "+91-4", new Vehicle("KA-02", VehicleType.ECONOMY, 4), new Location(2,
1));
        Driver d3 = new Driver("D3", "Om", "+91-5", new Vehicle("KA-03", VehicleType.PREMIUM, 4), new Location(1, 1));
        Driver d4 = new Driver("D4", "Zoe", "+91-6", new Vehicle("KA-04", VehicleType.ECONOMY, 4), new Location(1,
0));
        for (Driver d : new Driver[]{d1, d2, d3, d4}) svc.addDriver(d);
        d1.setOnline(true);
        d2.setOnline(true);
        d3.setOnline(true); // d4 stays offline, so it is never offered anything

        System.out.println("== 1. Request, first driver declines, second accepts, full ride");
        Trip t1 = svc.requestRide("R1", new Location(1, 0), new Location(5, 3), VehicleType.ECONOMY);
        svc.respond(t1.id(), "D1", false);
        svc.respond(t1.id(), "D2", true);
        System.out.println("  D2 available while on trip? " + d2.isAvailable());
        svc.startTrip(t1.id());
        svc.startTrip(t1.id());                // illegal: already started
        svc.endTrip(t1.id(), 12);              // 2.50 + 7*1.20 + 12*0.25 = 13.90
        System.out.println("  D2 now at " + d2.location() + ", available? " + d2.isAvailable());
        svc.rateDriver(t1.id(), 5);
        svc.rateRider(t1.id(), 4);
        System.out.println("  rate driver twice allowed? " + svc.rateDriver(t1.id(), 1)
                + "; D2 rating " + d2.rating() + ", R1 rating " + svc.rider("R1").rating());

        System.out.println("== 2. Surge: wrap the same rule in a decorator (x1.5)");
        svc.setPricing(new SurgePricing(standard, bd("1.5")));
        Trip t2 = svc.requestRide("R2", new Location(0, 1), new Location(0, 2), VehicleType.ECONOMY);
        svc.respond(t2.id(), "D1", true);
        svc.startTrip(t2.id());
        svc.endTrip(t2.id(), 3);               // max(2.50+1.20+0.75, 5.00) = 5.00, x1.5 = 7.50
        svc.setPricing(standard);

        System.out.println("== 3. Rider cancels after accept; cancel after start is refused");
        Trip t3 = svc.requestRide("R1", new Location(1, 1), new Location(3, 3), VehicleType.PREMIUM);
        svc.respond(t3.id(), "D3", true);
        svc.cancelTrip(t3.id());
        System.out.println("  D3 released? " + d3.isAvailable());
        Trip t4 = svc.requestRide("R1", new Location(1, 1), new Location(3, 3), VehicleType.PREMIUM);
        svc.respond(t4.id(), "D3", true);
        svc.startTrip(t4.id());
        svc.cancelTrip(t4.id());
        svc.endTrip(t4.id(), 10);              // 5 + 4*2 + 10*0.4 = 17.00 -> wallet has 36.10

        System.out.println("== 4. Nobody available for a VAN");
        svc.requestRide("R2", new Location(0, 0), new Location(1, 1), VehicleType.VAN);

        System.out.println("== 5. Race: 40 riders, 10 drivers, all accept at once");
        RideSharingService race = new RideSharingService(standard, new NearestDriverMatcher(100));
        for (int i = 0; i < 10; i++) {
```

```
            Driver d = new Driver("X" + i, "x", "-", new Vehicle("P" + i, VehicleType.ECONOMY, 4), new Location(i,
0));
            d.setOnline(true);
            race.addDriver(d);
        }
        for (int i = 0; i < 40; i++) race.addRider(new Rider("Q" + i, "q", "-", new CashPayment()));
        AtomicInteger accepted = new AtomicInteger();
        CountDownLatch go = new CountDownLatch(1);
        ExecutorService pool = Executors.newFixedThreadPool(16);
        for (int i = 0; i < 40; i++) {
            String rider = "Q" + i;
            pool.submit(() -> {
                go.await();
                Trip t = race.requestRide(rider, new Location(0, 0), new Location(3, 3), VehicleType.ECONOMY);
                for (Driver c = t.currentCandidate(); c != null && t.status() == TripStatus.REQUESTED;
                     c = t.currentCandidate()) {
                    if (race.respond(t.id(), c.id(), true)) { accepted.incrementAndGet(); break; }
                }
                return null;
            });
        }
        go.countDown();
        pool.shutdown();
        pool.awaitTermination(10, TimeUnit.SECONDS);
        System.out.println("  trips accepted = " + accepted.get() + " (never more than 10 drivers)");
    }

    private static BigDecimal bd(String s) { return new BigDecimal(s); }
}
```

Walking through the code.

- `TripStatus.NEXT` is the whole lifecycle in five lines; `Trip.move` refuses any move not in the table.
- `Trip` methods are `synchronized`: rider cancel and driver accept on the same trip cannot interleave (have their steps mixed together).
- `RideSharingService.respond`: only the *current* candidate may answer. Accept first **claims the driver atomically**; if the claim fails (another trip got them), it's treated as a decline and the next candidate is offered.
- `endTrip` works in a fixed order: price, store the fare on the trip, charge the rider, relocate the driver to the drop-off point, and only then release them (so no matcher can see a free driver at the old position) — then it records history for both sides. The course lists the same steps; relocating the driver is the step most often skipped, and skipping it makes the next match use the driver's morning position.
- Ratings are allowed once per side and only for ENDED trips (`markDriverRated` / `markRiderRated`).

Output:

```
== 1. Request, first driver declines, second accepts, full ride
  [T1] requested by R1, candidates [D1, D2]
  [T1] offered to D1
  [T1] declined by D1
  [T1] offered to D2
  [T1] accepted by D2
  D2 available while on trip? false
  [T1] started
  [T1] start refused in state STARTED
  [T1] ended: 7 km, 12 min, fare 13.90 (wallet debited 13.90, left 36.10)
  D2 now at (5,3), available? true
  rate driver twice allowed? false; D2 rating 5.00, R1 rating 4.00
== 2. Surge: wrap the same rule in a decorator (x1.5)
  [T2] requested by R2, candidates [D1, D2]
  [T2] offered to D1
  [T2] accepted by D1
  [T2] started
  [T2] ended: 1 km, 3 min, fare 7.50 (collect 7.50 in cash)
== 3. Rider cancels after accept; cancel after start is refused
  [T3] requested by R1, candidates [D3]
  [T3] offered to D3
  [T3] accepted by D3
  [T3] canceled by rider
  D3 released? true
  [T4] requested by R1, candidates [D3]
  [T4] offered to D3
  [T4] accepted by D3
  [T4] started
  [T4] cancel refused in state STARTED
  [T4] ended: 4 km, 10 min, fare 17.00 (wallet debited 17.00, left 19.10)
== 4. Nobody available for a VAN
  [T5] requested by R2, candidates []
  [T5] no driver left - canceled
== 5. Race: 40 riders, 10 drivers, all accept at once
  trips accepted = 10 (never more than 10 drivers)
```

Check the fares: trip 1 is 7 km and 12 min → 2.50 + 7 × 1.20 + 12 × 0.25 = **13.90**. Trip 2 is 1 km, 3 min → 4.45, below the 5.00 minimum → 5.00, then the surge decorator × 1.5 → **7.50**. The race test shows 40 concurrent requests against 10 drivers produce exactly 10 accepted trips — no driver is ever double-booked.

Machine-coding variant: carpool / "offer and select a ride"

A different ride-sharing problem is popular in machine-coding rounds (it appears in the kumaransg/LLD collection — a public GitHub repository of low-level design (LLD: designing the classes inside one program) problems — as a Flipkart-style round). Here there is **no dispatcher**: users *offer* spare seats in their own vehicle on a route, and other users *select* a ride using a **selection strategy**.

Requirements (from the problem statement, in brief):

- `add_user`, `add_vehicle` (a user may own several vehicles).
- `offer_ride(user, origin, seats, vehicle, destination)` — **fails if that vehicle already has an active offered ride**.
- `select_ride(user, origin, destination, seats, strategy)` — seats is 1 or 2; strategy is **Most Vacant** (most free seats) or **Preferred Vehicle** (e.g. "Activa", a scooter model).
- `end_ride(ride)` and `print_ride_stats()` — rides *taken* and *offered* per user, counting only **ended** rides (an offered ride counts even if nobody took it).

Design: `Ride` with an atomic `book(user, seats)`; `RideSelectionStrategy` (Strategy) with `MostVacant` and `PreferredVehicle`; one `RideService` that filters active rides by route and seats, excludes the user's own offers, and lets the strategy choose. The demo replays the problem's sample test case and prints the expected answers (2(c), 2(b), none, 2(d), none; then the stats).

```java
// File: code/lld/ch20/carpool/Main.java
package lld.ch20.carpool;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;

/**
 * Machine-coding variant (Flipkart-style "ride sharing / carpool"): users OFFER seats in their own
 * vehicle on a route; other users SELECT a ride with a selection strategy.
 */
public class Main {

    record User(String name, String gender, int age) {}

    record Vehicle(String owner, String model, String plate) {}

    enum RideStatus { ACTIVE, ENDED }

    static final class Ride {
        final String id;
        final User offeredBy;
        final Vehicle vehicle;
        final String origin, destination;
        private int availableSeats;
        private RideStatus status = RideStatus.ACTIVE;
        private final List<User> riders = new ArrayList<>();

        Ride(String id, User offeredBy, Vehicle vehicle, String origin, String destination, int seats) {
            this.id = id;
            this.offeredBy = offeredBy;
            this.vehicle = vehicle;
            this.origin = origin;
            this.destination = destination;
            this.availableSeats = seats;
        }

        synchronized int availableSeats() { return availableSeats; }

        /** Atomic check-and-book, so two users cannot take the last seat. */
        synchronized boolean book(User u, int seats) {
            if (status != RideStatus.ACTIVE || availableSeats < seats) return false;
            availableSeats -= seats;
            riders.add(u);
            return true;
        }

        @Override
        public String toString() {
            return id + " " + offeredBy.name() + " " + vehicle.model() + " " + origin + "->" + destination
                    + " seats=" + availableSeats;
        }
    }

    /** Strategy: pick one of the matching rides. */
    interface RideSelectionStrategy {
        Optional<Ride> select(List<Ride> candidates);
    }

    static final class MostVacant implements RideSelectionStrategy {
        public Optional<Ride> select(List<Ride> c) { return
c.stream().max(Comparator.comparingInt(Ride::availableSeats)); }
    }

    record PreferredVehicle(String model) implements RideSelectionStrategy {
        public Optional<Ride> select(List<Ride> c) {
            return c.stream().filter(r -> r.vehicle.model().equalsIgnoreCase(model)).findFirst();
        }
    }
```

```
    static final class RideService {
        private final Map<String, User> users = new LinkedHashMap<>();
        private final Map<String, Vehicle> vehicles = new LinkedHashMap<>();   // plate -> vehicle
        private final Map<String, Ride> rides = new LinkedHashMap<>();
        private final Map<String, int[]> stats = new LinkedHashMap<>();        // name -> {taken, offered}
        private int nextId = 0;

        void addUser(String name, String gender, int age) {
            users.put(name, new User(name, gender, age));
            stats.put(name, new int[2]);
        }

        void addVehicle(String owner, String model, String plate) { vehicles.put(plate, new Vehicle(owner, model,
plate)); }

        /** Fails if this vehicle already has an ACTIVE offered ride. */
        synchronized Optional<Ride> offerRide(String name, String origin, int seats, String plate, String destination)
{
            Vehicle v = vehicles.get(plate);
            if (v == null || !v.owner().equals(name)) return Optional.empty();
            boolean busy = rides.values().stream()
                    .anyMatch(r -> r.vehicle.plate().equals(plate) && r.status == RideStatus.ACTIVE);
            if (busy) return Optional.empty();
            Ride r = new Ride("2" + (char) ('a' + nextId++), users.get(name), v, origin, destination, seats);
            rides.put(r.id, r);
            return Optional.of(r);
        }

        synchronized Optional<Ride> selectRide(String name, String origin, String destination, int seats,
                                               RideSelectionStrategy strategy) {
            if (seats < 1 || seats > 2) throw new IllegalArgumentException("a user may request 1 or 2 seats");
            List<Ride> matching = rides.values().stream()
                    .filter(r -> r.status == RideStatus.ACTIVE)
                    .filter(r -> r.origin.equals(origin) && r.destination.equals(destination))
                    .filter(r -> !r.offeredBy.name().equals(name))
                    .filter(r -> r.availableSeats() >= seats)
                    .toList();
            Optional<Ride> chosen = strategy.select(matching);
            chosen.ifPresent(r -> r.book(users.get(name), seats));
            return chosen;
        }

        synchronized void endRide(String rideId) {
            Ride r = rides.get(rideId);
            if (r == null || r.status == RideStatus.ENDED) return;
            r.status = RideStatus.ENDED;
            stats.get(r.offeredBy.name())[1]++;
            for (User u : r.riders) stats.get(u.name())[0]++;
        }

        void printRideStats() {
            stats.forEach((n, s) -> System.out.println("  " + n + ": " + s[0] + " Taken, " + s[1] + " Offered"));
        }
    }

    public static void main(String[] args) {
        RideService s = new RideService();
        s.addUser("Rohan", "M", 36);    s.addVehicle("Rohan", "Swift", "KA-01-12345");
        s.addUser("Shashank", "M", 29); s.addVehicle("Shashank", "Baleno", "TS-05-62395");
        s.addUser("Nandini", "F", 29);
        s.addUser("Shipra", "F", 27);   s.addVehicle("Shipra", "Polo", "KA-05-41491");
                                        s.addVehicle("Shipra", "Activa", "KA-12-12332");
        s.addUser("Gaurav", "M", 29);
        s.addUser("Rahul", "M", 35);    s.addVehicle("Rahul", "XUV", "KA-05-1234");

        System.out.println("== offer_ride");
        print(s.offerRide("Rohan", "Hyderabad", 1, "KA-01-12345", "Bangalore"));
        print(s.offerRide("Shipra", "Bangalore", 1, "KA-12-12332", "Mysore"));
        print(s.offerRide("Shipra", "Bangalore", 2, "KA-05-41491", "Mysore"));
        print(s.offerRide("Shashank", "Hyderabad", 2, "TS-05-62395", "Bangalore"));
        print(s.offerRide("Rahul", "Hyderabad", 5, "KA-05-1234", "Bangalore"));
        print(s.offerRide("Rohan", "Bangalore", 1, "KA-01-12345", "Pune")); // same vehicle still active

        System.out.println("== select_ride");
```

```
        print(s.selectRide("Nandini", "Bangalore", "Mysore", 1, new MostVacant()));
        print(s.selectRide("Gaurav", "Bangalore", "Mysore", 1, new PreferredVehicle("Activa")));
        print(s.selectRide("Shashank", "Mumbai", "Bangalore", 1, new MostVacant()));
        print(s.selectRide("Rohan", "Hyderabad", "Bangalore", 1, new PreferredVehicle("Baleno")));
        print(s.selectRide("Shashank", "Hyderabad", "Bangalore", 1, new PreferredVehicle("Polo")));

        System.out.println("== end_ride 2a..2d, then print_ride_stats");
        for (String id : List.of("2a", "2b", "2c", "2d")) s.endRide(id);
        s.printRideStats();
    }

    private static void print(Optional<Ride> r) {
        System.out.println("  " + r.map(Ride::toString).orElse("No rides found / request failed"));
    }
}
```

Output:

```
== offer_ride
  2a Rohan Swift Hyderabad->Bangalore seats=1
  2b Shipra Activa Bangalore->Mysore seats=1
  2c Shipra Polo Bangalore->Mysore seats=2
  2d Shashank Baleno Hyderabad->Bangalore seats=2
  2e Rahul XUV Hyderabad->Bangalore seats=5
  No rides found / request failed
== select_ride
  2c Shipra Polo Bangalore->Mysore seats=1
  2b Shipra Activa Bangalore->Mysore seats=0
  No rides found / request failed
  2d Shashank Baleno Hyderabad->Bangalore seats=1
  No rides found / request failed
== end_ride 2a..2d, then print_ride_stats
  Rohan: 1 Taken, 1 Offered
  Shashank: 0 Taken, 1 Offered
  Nandini: 1 Taken, 0 Offered
  Shipra: 0 Taken, 2 Offered
  Gaurav: 1 Taken, 0 Offered
  Rahul: 0 Taken, 0 Offered
```

	Uber-style (20.4 main design)	Carpool variant
Who supplies the ride	Professional drivers, online/offline	Any user, per offered ride
Who chooses	The system (matching strategy) offers to drivers	The rider picks with a selection strategy
Key pattern	Strategy + Decorator + state machine	Strategy (selection)
Key invariant	A driver is on at most one trip	A vehicle has at most one active offer; seats never go negative

Concurrency & edge cases

- **Double booking a driver:** prevented by `Driver.tryClaim()` (compare-and-set). Proven by the 40-riders/10-drivers race in the output.
- **Cancel vs accept race:** both go through `synchronized` `Trip` methods; if the rider cancels after the claim but before `accept`, `accept` fails and the driver is released.
- **Stale candidate list:** the list is computed once. A driver may go offline or be claimed later; `tryClaim` fails and we move on. A nearer driver who becomes free later is not noticed (real systems re-match periodically).
- **Offer timeouts:** a driver who never answers should be treated as a decline after N seconds (a `ScheduledExecutorService` task per offer).
- **No drivers at all:** trip canceled immediately (see trip T5).
- **Illegal transitions** (start twice, cancel after start, end before start): return `false`; state unchanged.

- **Wallet with insufficient balance:** `Receipt(paid=false)`; a real system would retry another method or record a debt on the rider's account.
- **Rating abuse:** one rating per side per trip; stars validated 1–5.

Extensions & follow-up questions

- **"Rank by rating instead."** Swap `MatchingStrategy` (`BestRatedMatcher` is included) — one line at startup.
- **"Airport fee, promo code, tolls."** More decorators around the pricing rule: `new PromoDiscount(new AirportFee(new SurgePricing(standard, 1.5)))`.
- **"How is surge computed?"** From demand (open requests) vs supply (free drivers) per area and time window; the result feeds the multiplier. The pricing design doesn't change.
- **"Find nearby drivers fast in a real city."** A geospatial index (a lookup structure for finding things by location): split the map into cells (geohash, Google S2 or Uber's H3 hexagons — three standard ways of dividing the globe into numbered cells) and keep a map cell → drivers; search the pickup cell and its neighbours instead of all drivers.
- **"Shared rides (pool)."** Honest answer: this design assumes one rider per trip. The change starts at `Trip` (many riders, many stops) and a matcher that considers detours.
- **"Live tracking."** Drivers send location updates every few seconds; store the latest per driver and publish to the rider (pub-sub, see 20.6).
- **"Scheduled rides."** A request with a future time; a scheduler (a component that runs jobs at chosen times) starts matching shortly before.

Real-world example

Uber really does show riders a surge ("dynamic pricing") multiplier when demand exceeds supply in an area, and it sends a request to a nearby driver who has a short time to accept before it goes to someone else — the ranked-list-with-cursor idea. Uber open-sourced (published free for anyone to use) **H3**, a hierarchical (cells nest inside bigger cells) hexagonal grid that its engineering blog describes as built for optimizing ride pricing and dispatch (for example, bucketing supply and demand per hexagon); the same cell idea answers "which drivers are near this point" without scanning the whole city. Ola, Lyft and Grab follow the same request → match → accept → trip → rate lifecycle.

In the interview

> **30-second answer:** "Two rules behind interfaces: `MatchingStrategy` returns a ranked list of free drivers with the right car, and `PricingStrategy` computes base + distance + time with a floor. Surge is a Decorator that wraps any pricing rule and multiplies it. The trip is a small state machine; every action checks the transition table and returns false if illegal. A decline moves a cursor along the stored ranking. Drivers are claimed with a compare-and-set so one driver can never get two trips."

Tip: name the two rules (matching and pricing) in your first minutes and say both go behind interfaces; that one sentence pre-answers most follow-ups. Then spend real time on surge: *wrapping* the pricing rule instead of adding a flag to it is what separates strong answers from weak ones.

Plan (40 min): clarify (5) → accounts, vehicle, location (5) → trip + state machine (8) → pricing + surge decorator (7) → matching list + decline flow (7) → concurrency (claim) (4) → extensions (4).

Follow-ups with answers:

- *"What happens when a driver declines?"* → Next driver in the stored ranked list; none left → cancel.
- *"Strategy vs Decorator here?"* → Strategy replaces the rule; Decorator wraps it and remains in the call path (it holds an `inner` of the same interface).
- *"Two riders get the same driver?"* → Impossible: atomic claim; the loser moves to its next candidate.

Self-check

1. Why does the matcher return a list instead of one driver? → So a decline just moves to the next candidate without re-searching and remembers who refused.
2. What makes `SurgePricing` a Decorator? → It is a `PricingStrategy` that also wraps a second `PricingStrategy` and adjusts that one's result.
3. Can a rider cancel a STARTED trip? → No; `STARTED` can only move to `ENDED`.

4. Why update the driver's location to the drop-off when the trip ends? → Later matches must use their real position.
5. Which Java class makes the driver claim atomic? → `AtomicBoolean` (`compareAndSet(false, true)`).
6. In the carpool variant, when does `offer_ride` fail? → When the same vehicle already has an active offered ride.

TL;DR

- **Strategy** ×3 (matching, pricing, payment) + **Decorator** (surge) + **state machine** (trip).
- Matcher returns a **ranked list**; the trip keeps a cursor; decline = next.
- **Atomic claim** of drivers prevents double booking; relocate the driver to the drop-off at trip end.
- Ratings = running total + count; `BigDecimal` fares with a minimum.
- Carpool variant: users offer seats; riders select with Most Vacant / Preferred Vehicle strategies.

20.5 Design a Logging Framework

Problem & clarifying questions

What it is. A logging framework lets code write messages like "payment 991 authorised" with a **level** (how serious it is) and sends them to one or more **destinations** (console, file, a log server), in a chosen **format**. Everyday analogy: a company's internal mail room. Every department drops notes in; each note has an urgency stamp; the mail room decides, by department and urgency, which notes get copied to which inboxes and in what envelope.

Think Log4j 2, Logback or `java.util.logging` (the most widely used Java logging libraries): this question asks you to design a small version of them.

Question	Assumed answer
Levels?	TRACE < DEBUG < INFO < WARN < ERROR < FATAL (and OFF).
Per-module configuration?	Yes: loggers are named `app.payment`, `app.orders`… and form a **hierarchy** by dots. A child inherits its level from the nearest ancestor that sets one.
Destinations?	Console, file, in-memory (tests); pluggable. Each can have its own minimum level.
Formats?	Plain pattern and JSON (a text format of nested key-value pairs); pluggable.
Propagation?	An event goes to the logger's own destinations and then **up** to its ancestors' destinations, unless that logger turns propagation (additivity) off.
Async?	Yes, optional: the application thread must not wait for disk I/O (input/output: reading or writing the disk).
Thread-safe?	Yes: many threads log at once.
Config files, rolling files (a log file is closed and a new one started at a size or date limit), remote shipping (sending logs to another server)?	Out of scope (extensions).

Requirements

Functional

1. Log a message at a level; messages below the logger's effective level are dropped cheaply.
2. Loggers are obtained by name; the same name returns the same logger.
3. Loggers form a dot-separated hierarchy under a root; level is inherited when not set.
4. A logger has zero or more appenders (destinations). Each appender has a minimum level and a formatter.

5. Events propagate up to parent appenders unless additivity is off.
6. Formatters are pluggable (pattern, JSON).
7. Asynchronous appending is available without changing the appenders.
8. Exceptions can be attached to an event.
9. Expensive messages can be built lazily (only if the level is enabled).

Non-functional: very cheap when disabled; thread-safe; logging must never crash the application; testable (inject a clock).

Use cases

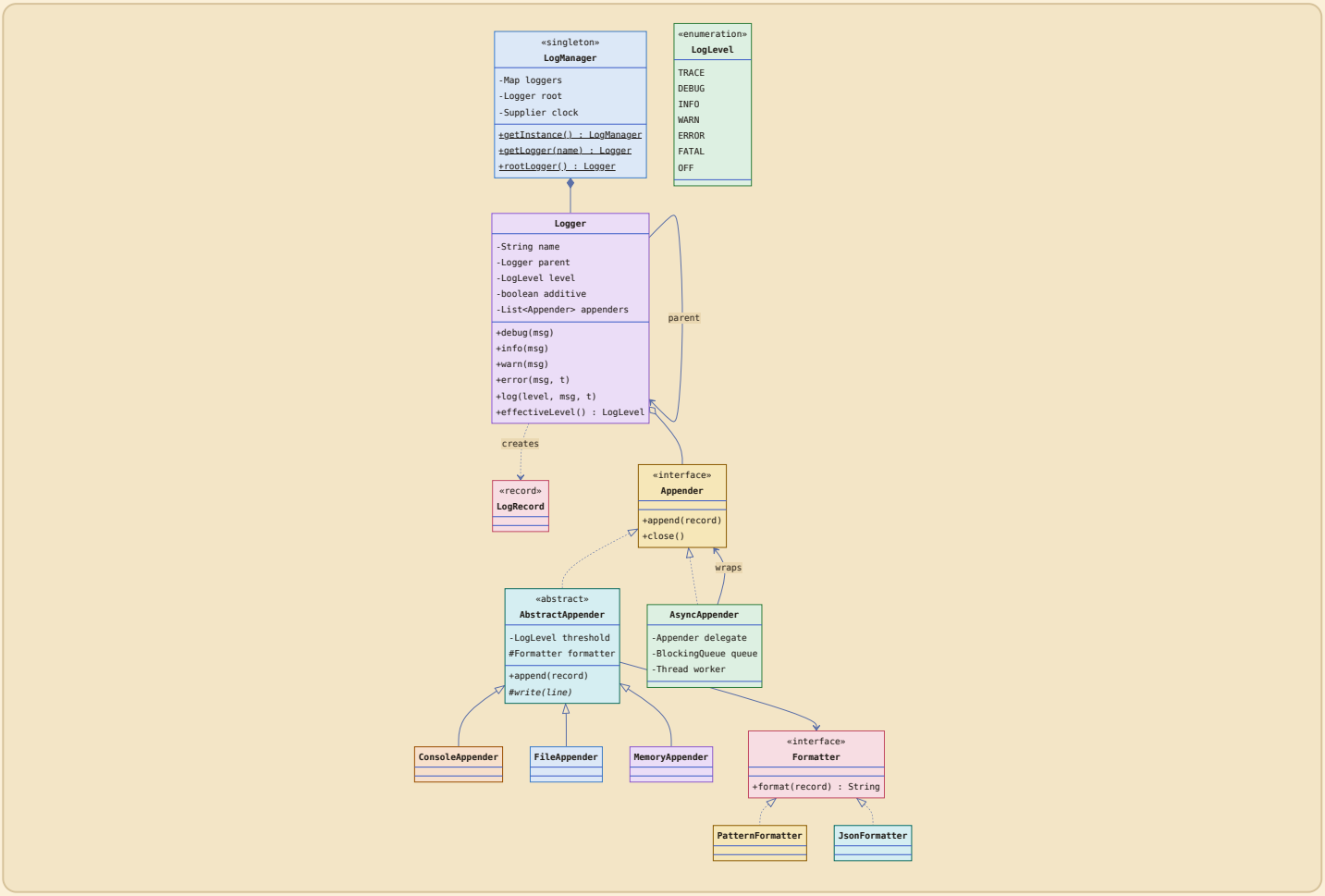
Actors: **Developer code** (logs messages), **Operator** (configures levels, appenders, formats at startup or runtime — while the program is running), **Framework** (filters, formats, routes, writes).

Main use cases: *Log a message*, *Get a logger by name*, *Set a level for a subtree*, *Add a destination*, *Choose a format*, *Switch to async writing*, *Stop propagation for a subtree*.

Core entities

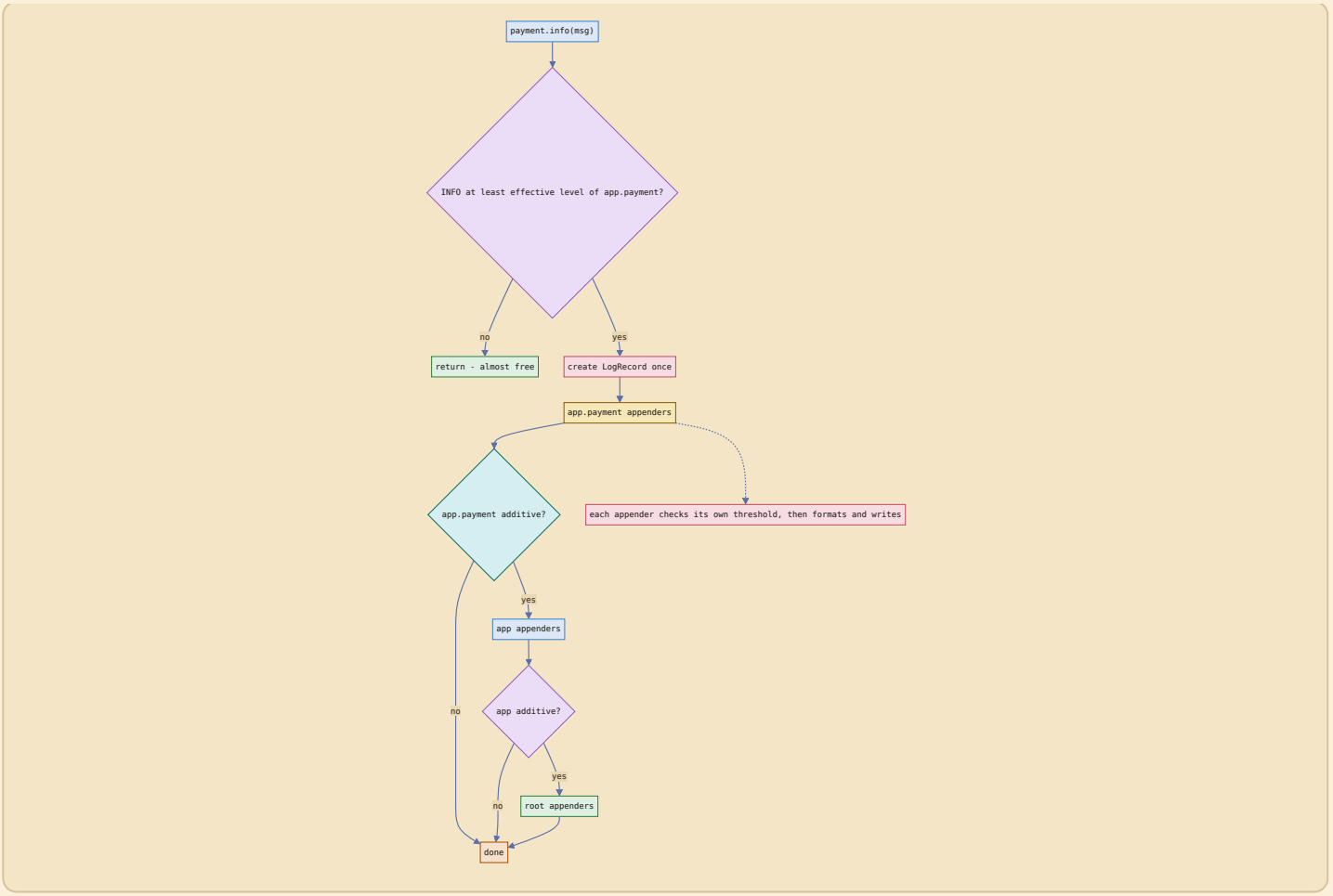
Class	Responsibility
`LogLevel` (enum)	Ordered severities; `isAtLeast(threshold)`.
`LogRecord` (record)	One immutable (cannot change after creation) event: time, level, logger name, thread, message, error.
`Logger`	Named node in the hierarchy: parent, optional level, appenders, additivity; `log()` walks the chain upward.
`LogManager`	**Singleton** registry; `getLogger(name)` creates parents first and returns one instance per name; holds the clock.
`Appender` (interface)	A destination (sink).
`AbstractAppender`	**Template Method**: `append()` checks the appender's threshold and formats; subclasses only `write(line)`.
`ConsoleAppender`, `FileAppender`, `MemoryAppender`	Concrete sinks.
`AsyncAppender`	**Decorator**: wraps any appender with a bounded queue + one writer thread.
`Formatter` (interface)	**Strategy**: record → text. `PatternFormatter`, `JsonFormatter`.

Class diagram



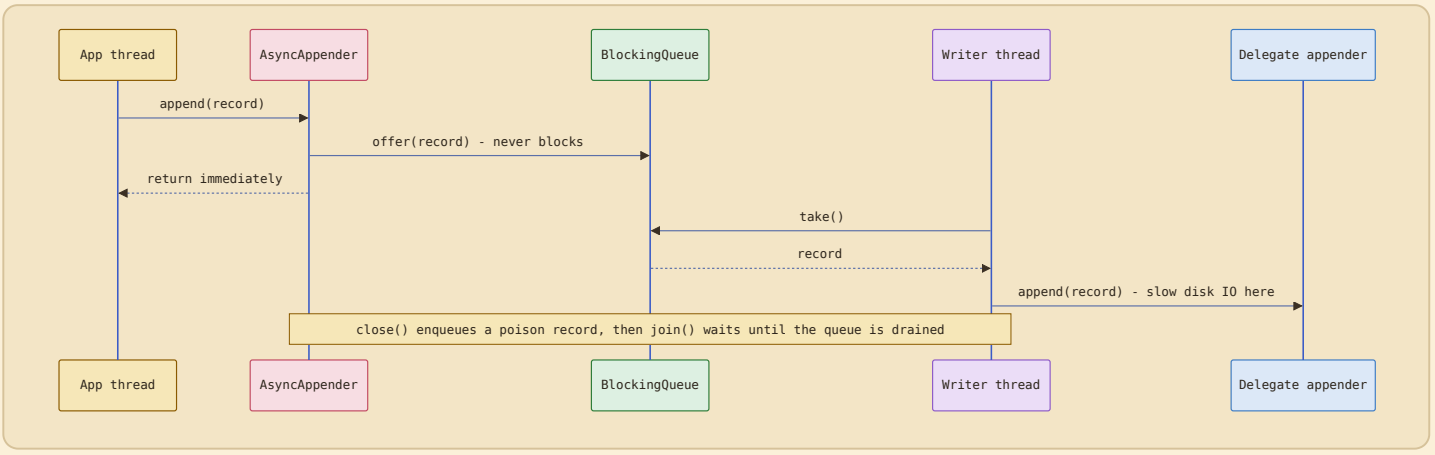
Key flows

Logging one event (Chain of Responsibility up the hierarchy). *Chain of Responsibility* = a request is passed along a chain of handlers; each may handle it and/or pass it on. Here every logger in the chain hands the record to its own appenders and passes it to its parent, unless additivity is off.



Note the Log4j/Logback rule we copy: the **level check happens once, at the logger where the call was made**. Parents' *levels* are not re-checked during propagation; only each **appender's own threshold** filters. That is why, in the demo, a DEBUG event from `app.payment` (set to DEBUG) still reaches the root console appender even though root is at INFO.

Async appending.



Design decisions & patterns used

Pattern / decision	Where	Why	Alternative rejected
Chain of Responsibility	`Logger.log` walks `this → parent → … → root`, stopping when `additive == false`	Configure destinations once near the root; subtrees can add or stop.	Every logger listing all its appenders explicitly.
Singleton	`LogManager` (initialization-on-demand holder)	One registry per process, so `getLogger("x")` always returns the same object. Holder idiom (a standard coding recipe) = lazy + thread-safe with no locking (the JVM — Java Virtual Machine, the program that runs Java code — initializes the nested class once).	Public static mutable (changeable) map; double-checked locking (check, lock, check again; easy to get wrong).
Static factory method	`LogManager.getLogger(name)`	Hides creation, caches instances, creates parents first.	`new Logger(...)` by users (breaks the hierarchy).
Strategy	`Formatter`	Format is independent of destination: any sink × any format.	`JsonFileAppender`, `PatternFileAppender`, … (class explosion).
Template Method	`AbstractAppender.append` (final) → `write`	Threshold + formatting done once; sinks only write text.	Each sink re-implementing filtering.
Decorator	`AsyncAppender` wraps any `Appender`	Async becomes an add-on, not a rewrite of every sink.	`AsyncFileAppender`, `AsyncConsoleAppender`, …
Level check first; **lazy** `Supplier<String>` (a function that builds the string only when called)	`isEnabled`, `debug(Supplier)`	Disabled logging costs one comparison; expensive strings are never built.	Always building the message (a real performance bug).
Appender failures caught	`Logger.log`	Logging must never crash the app.	Letting exceptions escape.
Parent resolved **outside** `computeIfAbsent`	`LogManager.loggerFor`	A `ConcurrentHashMap` mapping function must not modify the same map (the JDK may throw `IllegalStateException: Recursive update`).	Recursive `computeIfAbsent`.

A note on another common "CoR logger" answer. Many LLD write-ups model Chain of Responsibility as a chain of *level handlers* (`ErrorHandler → WarnHandler → InfoHandler → DebugHandler`), each writing messages of "its" level and passing others along. It works for a toy, but real frameworks put the chain on the **logger hierarchy** (as here) because that is what operators actually configure. If an interviewer expects the level-handler version, explain both and why the hierarchy is the one Log4j/Logback use.

Java implementation

```java
// File: code/lld/ch20/logging/Core.java
package lld.ch20.logging;

import java.time.Instant;
import java.time.ZoneOffset;
import java.time.format.DateTimeFormatter;

/** Severity, lowest to highest. A logger set to WARN drops TRACE, DEBUG and INFO. */
enum LogLevel {
    TRACE, DEBUG, INFO, WARN, ERROR, FATAL, OFF;

    boolean isAtLeast(LogLevel threshold) { return this.ordinal() >= threshold.ordinal(); }
}

/** One immutable log event. Created once, passed along the chain, formatted by each appender. */
record LogRecord(Instant time, LogLevel level, String loggerName, String thread, String message, Throwable error) {}

/** Strategy: how a record becomes text. */
interface Formatter {
    String format(LogRecord r);
}

final class PatternFormatter implements Formatter {
    private static final DateTimeFormatter TIME =
            DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss.SSS").withZone(ZoneOffset.UTC);

    @Override
    public String format(LogRecord r) {
        String line = String.format("%s %-5s [%s] %s - %s", TIME.format(r.time()), r.level(), r.thread(),
                r.loggerName(), r.message());
        return r.error() == null ? line : line + " | " + r.error();
    }
}

final class JsonFormatter implements Formatter {
    @Override
    public String format(LogRecord r) {
        return "{\"ts\":\"" + r.time() + "\",\"level\":\"" + r.level() + "\",\"logger\":\"" + r.loggerName()
                + "\",\"msg\":\"" + r.message().replace("\"", "\\\"") + "\"}";
    }
}
```

```java
// File: code/lld/ch20/logging/Appenders.java
package lld.ch20.logging;

import java.io.BufferedWriter;
import java.io.IOException;
import java.io.PrintStream;
import java.io.UncheckedIOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.atomic.AtomicLong;

/** A sink (destination). Each has its own threshold and formatter. */
interface Appender extends AutoCloseable {
    void append(LogRecord r);

    @Override
    default void close() {}
}

/** Shared threshold + formatter handling, so concrete appenders only write text. */
abstract class AbstractAppender implements Appender {
    private final LogLevel threshold;
    protected final Formatter formatter;

    AbstractAppender(LogLevel threshold, Formatter formatter) {
        this.threshold = threshold;
        this.formatter = formatter;
    }

    @Override
    public final void append(LogRecord r) {
        if (r.level().isAtLeast(threshold)) write(formatter.format(r));
    }

    protected abstract void write(String line);
}

final class ConsoleAppender extends AbstractAppender {
    private final PrintStream out;

    ConsoleAppender(LogLevel threshold, Formatter f, PrintStream out) {
        super(threshold, f);
        this.out = out;
    }

    @Override
    protected synchronized void write(String line) { out.println(line); }
}

final class FileAppender extends AbstractAppender {
    private final BufferedWriter writer;

    FileAppender(LogLevel threshold, Formatter f, Path file) throws IOException {
        super(threshold, f);
        this.writer = Files.newBufferedWriter(file, StandardCharsets.UTF_8,
                StandardOpenOption.CREATE, StandardOpenOption.APPEND);
    }

    @Override
    protected synchronized void write(String line) {
        try {
            writer.write(line);
            writer.newLine();
        } catch (IOException e) {
            throw new UncheckedIOException(e);
        }
    }
```

```
    @Override
    public synchronized void close() {
        try {
            writer.close();
        } catch (IOException e) {
            throw new UncheckedIOException(e);
        }
    }
}

/** Keeps lines in memory - handy for tests. */
final class MemoryAppender extends AbstractAppender {
    private final List<String> lines = new ArrayList<>();

    MemoryAppender(LogLevel threshold, Formatter f) { super(threshold, f); }

    @Override
    protected synchronized void write(String line) { lines.add(line); }

    synchronized List<String> lines() { return List.copyOf(lines); }
}

/**
 * Decorator: makes ANY appender asynchronous. Callers only enqueue; one background thread writes.
 * Bounded queue: when full we drop and count (a common production choice), never block the app.
 */
final class AsyncAppender implements Appender {
    private static final LogRecord POISON = new LogRecord(null, LogLevel.OFF, "", "", "", null);
    private final Appender delegate;
    private final BlockingQueue<LogRecord> queue;
    private final Thread worker;
    private final AtomicLong dropped = new AtomicLong();
    private volatile boolean closed;

    AsyncAppender(Appender delegate, int capacity) {
        this.delegate = delegate;
        this.queue = new ArrayBlockingQueue<>(capacity);
        this.worker = Thread.ofPlatform().daemon().name("async-log-writer").start(this::drain);
    }

    @Override
    public void append(LogRecord r) {
        if (closed || !queue.offer(r)) dropped.incrementAndGet(); // full or closed: count it, never block
    }

    private void drain() {
        try {
            for (LogRecord r = queue.take(); r != POISON; r = queue.take()) {
                try {
                    delegate.append(r);
                } catch (RuntimeException e) {  // one bad write must not kill the writer thread
                    System.err.println("async appender: delegate failed: " + e);
                }
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    long dropped() { return dropped.get(); }

    /** Flush everything already queued, then stop. Records appended after this point are counted as dropped. */
    @Override
    public void close() {
        if (closed) return;
        closed = true;
        try {
            queue.put(POISON);
            worker.join();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
        delegate.close();
```

```
    }
}
```

```java
// File: code/lld/ch20/logging/Logger.java
package lld.ch20.logging;

import java.time.Instant;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.function.Supplier;

/**
 * A named logger in a dot-separated hierarchy ("app.payment" -> "app" -> root).
 * An event is handed to this logger's appenders, then passed UP to the parent's appenders
 * (Chain of Responsibility) until a logger with additivity=false stops it.
 */
final class Logger {
    private final String name;
    private final Logger parent;
    private final LogManager manager;
    private volatile LogLevel level;                    // null = inherit from the nearest ancestor
    private volatile boolean additive = true;
    private final List<Appender> appenders = new CopyOnWriteArrayList<>();

    Logger(String name, Logger parent, LogManager manager) {
        this.name = name;
        this.parent = parent;
        this.manager = manager;
    }

    String name() { return name; }
    void setLevel(LogLevel level) { this.level = level; }
    void setAdditive(boolean additive) { this.additive = additive; }
    void addAppender(Appender a) { appenders.add(a); }

    LogLevel effectiveLevel() {
        for (Logger l = this; l != null; l = l.parent) if (l.level != null) return l.level;
        return LogLevel.INFO;
    }

    boolean isEnabled(LogLevel lvl) { return lvl != LogLevel.OFF && lvl.isAtLeast(effectiveLevel()); }

    void trace(String msg) { log(LogLevel.TRACE, msg, null); }
    void debug(String msg) { log(LogLevel.DEBUG, msg, null); }
    void info(String msg) { log(LogLevel.INFO, msg, null); }
    void warn(String msg) { log(LogLevel.WARN, msg, null); }
    void error(String msg, Throwable t) { log(LogLevel.ERROR, msg, t); }

    /** Lazy message: the (possibly expensive) string is only built if the level is enabled. */
    void debug(Supplier<String> msg) { if (isEnabled(LogLevel.DEBUG)) log(LogLevel.DEBUG, msg.get(), null); }

    void log(LogLevel lvl, String msg, Throwable t) {
        if (!isEnabled(lvl)) return;                    // cheap check first
        LogRecord r = new LogRecord(manager.now(), lvl, name, Thread.currentThread().getName(), msg, t);
        for (Logger l = this; l != null; l = l.parent) { // the chain
            for (Appender a : l.appenders) {
                try {
                    a.append(r);
                } catch (RuntimeException e) {
                    System.err.println("appender failed: " + e); // logging must never crash the app
                }
            }
            if (!l.additive) break;
        }
    }
}

/** Singleton registry of loggers (initialization-on-demand holder idiom: lazy and thread-safe). */
final class LogManager {
    private final Map<String, Logger> loggers = new ConcurrentHashMap<>();
    private final Logger root = new Logger("root", null, this);
    private volatile Supplier<Instant> clock = Instant::now;

    private LogManager() { root.setLevel(LogLevel.INFO); }
```

```
    private static final class Holder { static final LogManager INSTANCE = new LogManager(); }

    static LogManager getInstance() { return Holder.INSTANCE; }

    /** Factory method: same name -> same Logger instance; parents are created first. */
    static Logger getLogger(String name) { return getInstance().loggerFor(name); }

    static Logger rootLogger() { return getInstance().root; }

    private Logger loggerFor(String name) {
        if (name == null || name.isEmpty() || name.equals("root")) return root;
        Logger existing = loggers.get(name);
        if (existing != null) return existing;
        int dot = name.lastIndexOf('.');
        Logger parent = dot < 0 ? root : loggerFor(name.substring(0, dot)); // resolved OUTSIDE computeIfAbsent
        return loggers.computeIfAbsent(name, n -> new Logger(n, parent, this));
    }

    Instant now() { return clock.get(); }
    void setClock(Supplier<Instant> c) { this.clock = c; }
}
```

```java
// File: code/lld/ch20/logging/Main.java
package lld.ch20.logging;

import java.nio.file.Files;
import java.nio.file.Path;
import java.time.Instant;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    public static void main(String[] args) throws Exception {
        LogManager.getInstance().setClock(() -> Instant.parse("2026-09-28T10:15:30.123Z")); // fixed for a stable demo

        Logger root = LogManager.rootLogger();
        root.setLevel(LogLevel.INFO);
        root.addAppender(new ConsoleAppender(LogLevel.TRACE, new PatternFormatter(), System.out));

        Logger payment = LogManager.getLogger("app.payment");
        Logger app = LogManager.getLogger("app");
        System.out.println("== 1. Same name -> same instance: " + (payment == LogManager.getLogger("app.payment")));

        System.out.println("== 2. Levels: 'app.payment' inherits INFO from root");
        payment.debug("hidden: DEBUG < INFO");
        payment.info("payment 991 authorised");
        payment.warn("retrying gateway call");

        System.out.println("== 3. Raise one subtree to DEBUG; extra JSON sink only for payments");
        MemoryAppender audit = new MemoryAppender(LogLevel.DEBUG, new JsonFormatter());
        payment.setLevel(LogLevel.DEBUG);
        payment.addAppender(audit);
        payment.debug(() -> "card token=" + "tok_" + 42 + " (built lazily)");
        LogManager.getLogger("app.orders").debug("still hidden: app.orders inherits INFO");
        System.out.println("  audit sink got: " + audit.lines());

        System.out.println("== 4. Additivity off: 'app.payment' events stop at its own appenders");
        payment.setAdditive(false);
        payment.error("refund failed", new IllegalStateException("gateway 503"));
        System.out.println("  (nothing printed on console above) audit sink now has " + audit.lines().size() + "
lines");
        payment.setAdditive(true);

        System.out.println("== 5. File sink with its own threshold (WARN+)");
        Path file = Files.createTempFile("app", ".log");
        FileAppender fileSink = new FileAppender(LogLevel.WARN, new PatternFormatter(), file);
        app.addAppender(fileSink);
        app.info("info goes to console only");
        app.warn("disk 91% full");
        fileSink.close();
        System.out.println("  file contains: " + Files.readAllLines(file));
        Files.delete(file);

        System.out.println("== 6. Async appender: 4 threads x 1000 events, then close() flushes");
        CountingAppender counter = new CountingAppender();
        AsyncAppender async = new AsyncAppender(counter, 10_000);
        Logger worker = LogManager.getLogger("worker");
        worker.setAdditive(false);
        worker.addAppender(async);
        List<Thread> threads = new ArrayList<>();
        for (int t = 0; t < 4; t++) {
            int id = t;
            threads.add(Thread.ofPlatform().start(() -> {
                for (int i = 0; i < 1000; i++) worker.info("job " + id + "-" + i);
            }));
        }
        for (Thread t : threads) t.join();
        async.close();
        System.out.println("  written=" + counter.count.get() + " dropped=" + async.dropped()
                + " total=" + (counter.count.get() + async.dropped()));
    }

    static final class CountingAppender implements Appender {
        final AtomicInteger count = new AtomicInteger();
```

```
        public void append(LogRecord r) { count.incrementAndGet(); }
    }
}
```

Walking through the code.

- `Logger.effectiveLevel` walks up until a logger has a level; root always has one.
- `Logger.log` creates the `LogRecord` **once** and hands the same immutable object to every appender in the chain (safe to share across threads because it is a record with final fields).
- `AbstractAppender.append` is `final`; `ConsoleAppender` and `FileAppender` synchronize `write` so lines from different threads never interleave mid-line.
- `AsyncAppender` uses a bounded `ArrayBlockingQueue` (a fixed-size, thread-safe queue) and `offer` (non-blocking: returns at once instead of waiting): when full, it drops and counts instead of stalling the application. `close()` puts a "poison pill" record (a special marker that means "stop after this") so the writer drains everything before stopping. The no-loss promise is precise: every record appended **before** `close()` is either written or counted in `dropped` (queue full); a record appended after `close()` is counted as dropped too, so `written + dropped` always equals the number of `append` calls. A delegate that throws is caught inside the writer loop, so one bad write cannot kill the writer thread (which would otherwise leave `close()` waiting forever on a full queue).
- `LogManager` uses the holder idiom: `Holder.INSTANCE` is created the first time `getInstance()` runs, and the JVM guarantees that happens once.

Output:

```
== 1. Same name -> same instance: true
== 2. Levels: 'app.payment' inherits INFO from root
2026-09-28 10:15:30.123 INFO  [main] app.payment - payment 991 authorised
2026-09-28 10:15:30.123 WARN  [main] app.payment - retrying gateway call
== 3. Raise one subtree to DEBUG; extra JSON sink only for payments
2026-09-28 10:15:30.123 DEBUG [main] app.payment - card token=tok_42 (built lazily)
  audit sink got: [{"ts":"2026-09-28T10:15:30.123Z","level":"DEBUG","logger":"app.payment","msg":"card token=tok_42
(built lazily)"}]
== 4. Additivity off: 'app.payment' events stop at its own appenders
  (nothing printed on console above) audit sink now has 2 lines
== 5. File sink with its own threshold (WARN+)
2026-09-28 10:15:30.123 INFO  [main] app - info goes to console only
2026-09-28 10:15:30.123 WARN  [main] app - disk 91% full
  file contains: [2026-09-28 10:15:30.123 WARN  [main] app - disk 91% full]
== 6. Async appender: 4 threads x 1000 events, then close() flushes
  written=4000 dropped=0 total=4000
```

Step 4: with additivity off, the ERROR went only to the JSON audit sink (2 lines now) and not to the console. Step 6: 4 threads × 1000 events all arrived (4000 written, 0 dropped), and `close()` flushed the queue.

Concurrency & edge cases

- **Many threads logging:** appenders list is `CopyOnWriteArrayList` (reads without locking, rare writes); level and additivity are `volatile` (a Java keyword that makes every thread see the latest written value) so a runtime change is seen by all threads.
- **Interleaved output:** each sink's `write` is synchronized.
- **Async queue full:** drop-and-count (never block). Alternatives: block (back-pressure — slowing the producer down when the consumer cannot keep up — can stall the app) or drop only low levels. Logback's `AsyncAppender`, by default, starts discarding TRACE/DEBUG/INFO events when the queue is 80% full.
- **Shutdown:** without `close()`, queued events are lost; register a JVM shutdown hook (code the JVM runs just before the program exits) that closes appenders.
- **Recursive logging** (an appender that logs while appending) can loop forever; guard with a thread-local (each thread gets its own separate copy) "already inside" flag.
- **Security:** never let log messages trigger lookups or code; Log4Shell (CVE-2021-44228, December 2021; a CVE is a public id for a known security flaw) came from Log4j 2 interpreting `${jndi:...}` (a lookup that could load code from a remote server) inside logged strings. Treat messages as plain data.
- **Sensitive data:** mask card numbers, tokens and passwords before formatting.

Extensions & follow-up questions

- **"Rolling files."** A `RollingFileAppender` with a `RollingPolicy` (size- or time-based) Strategy: when triggered, close, rename to `app-2026-09-28.log`, open a new file.
- **"Config file."** Parse XML/YAML/properties (common configuration-file text formats) into calls on `LogManager` (set levels, attach appenders). Watching the file lets levels change at runtime.
- **"Filters."** A `Filter` chain per appender (e.g. only `app.payment`, only messages containing "refund") — another Chain of Responsibility.
- **"MDC / request id."** A thread-local map (Mapped Diagnostic Context) copied into each `LogRecord`, so every line of one request carries the same id.
- **"Ship logs to a server."** A network appender behind `AsyncAppender`, batching records.
- **"Structured logging."** Key-value fields in `LogRecord` rendered by `JsonFormatter` for tools like Elasticsearch/Kibana (a search engine and its dashboard, often used to store and browse logs).

Real-world example

SLF4J (Simple Logging Facade for Java) is a facade (the API your code calls); **Logback** and **Log4j 2** are implementations with exactly this model: dot-named loggers in a hierarchy, inherited levels, appenders with layouts/encoders, and *additivity*. Log4j 2's async loggers use the LMAX Disruptor (a lock-free ring buffer: a fixed-size circular array that threads share without taking locks) to hand events to a background thread. `java.util.logging` has the same shape with different names: `Logger`, `Handler` (appender), `Formatter`, `Level`, and `setUseParentHandlers(false)` (additivity off).

In the interview

> **30-second answer:** "Loggers are named and form a dot hierarchy under a root managed by a singleton `LogManager`. A log call first checks the effective level (inherited), builds one immutable record, and passes it up the chain of loggers to their appenders — Chain of Responsibility — until additivity stops it. Each appender has its own threshold and a pluggable formatter (Strategy). `AsyncAppender` is a Decorator with a bounded queue and a writer thread, so the app never waits for disk."

Plan (35 min): clarify (5) → levels + record (4) → logger hierarchy + effective level (6) → appenders + formatters (6) → propagation/additivity (5) → singleton + thread safety (4) → async (5).

Follow-ups with answers:

- *"Why is a disabled debug call cheap?"* → One level comparison before any string is built; use `Supplier` for expensive messages.
- *"How do you make it thread-safe?"* → Immutable records, `CopyOnWriteArrayList` appenders, synchronized sinks, `ConcurrentHashMap` registry, volatile config.
- *"What happens to queued logs on crash?"* → They're lost unless flushed; close on shutdown, or accept the trade-off for speed.

Self-check

1. `app.payment` has no level; `app` is WARN; root is INFO. What is `app.payment`'s effective level? → WARN (nearest ancestor that sets one).
2. What does additivity=false do? → Stops the event from reaching ancestor loggers' appenders.
3. Which pattern lets any appender become async? → Decorator (`AsyncAppender`).
4. Why is `Formatter` separate from `Appender`? → So formats and destinations combine freely (Strategy) without a class per combination.
5. Why resolve the parent before `computeIfAbsent`? → Modifying the same `ConcurrentHashMap` inside its mapping function is not allowed.
6. How does `close()` guarantee nothing queued is lost? → It enqueues a poison pill and joins the writer thread, which drains every record queued before the pill. (Records refused because the queue was full, or appended after `close()`, are not lost silently: they are counted as dropped.)

TL;DR

- Named **logger hierarchy**; effective level inherited; one cheap level check up front.

- **Chain of Responsibility** up to the root, stopped by additivity; **Singleton** `LogManager`.
- **Strategy** formatters, **Template Method** appender base, **Decorator** async appender.
- Immutable `LogRecord`, thread-safe sinks, never crash the app, never block on a full queue.

20.6 Design a Pub-Sub System

Problem & clarifying questions

What it is. Publish-subscribe (pub-sub) is a way for parts of a system to talk **without knowing each other**. A *publisher* sends a message to a named *topic* ("orders"); every *subscriber* of that topic receives it. Everyday analogy: a newspaper. The newspaper prints once; every subscriber gets a copy; the newspaper doesn't know or care who reads it, and new readers can sign up any time.

It is the **Observer pattern** turned into infrastructure: in Observer the subject calls its observers directly; in pub-sub a **broker** in the middle decouples them further (makes each side depend less on the other: by topic, by time, and often by machine). We design a simplified, Kafka-like, in-process (living inside one running program) broker.

Question	Assumed answer
Delivery to how many?	Every subscription gets every message (fan-out).
Order?	Messages are delivered in publish order per subscription.
Guarantee?	**At-least-once**: a message is only acknowledged after the subscriber processes it; failures are retried; duplicates are possible.
Slow subscriber?	Must not slow down the others or the publisher.
Keep messages after delivery?	Yes: an append-only log with **offsets**, so late subscribers can start from the beginning and anyone can replay.
Poison message (always fails)?	After N attempts, send to a dead-letter topic (a side topic that holds messages that could not be processed) and move on.
Push or pull?	Push to a callback, driven by one dispatcher (the loop that hands out messages) per subscription.
Partitions, consumer groups sharing work, persistence to disk, retention?	Out of scope (extensions; each is explained there).

Requirements

Functional

1. Create topics; publish a message (key, payload) to a topic; the broker assigns an **offset** (its position in the topic's log).
2. Subscribe to a topic with a subscription id and a callback, starting from EARLIEST (offset 0) or LATEST (only new messages).
3. Each subscription receives every message in order.
4. A message is acknowledged (offset committed) only after the callback returns normally.
5. Failures are retried with backoff; after `maxAttempts` the message goes to a dead-letter topic.
6. A subscription can **seek** to an offset to replay.
7. Unsubscribe; shut down cleanly.

Non-functional: thread-safe publish from many threads; subscribers isolated from each other; no busy-waiting (a thread spinning in a loop checking "anything yet?" and burning CPU (processor) time; here threads sleep until data arrives); ordering preserved.

Use cases

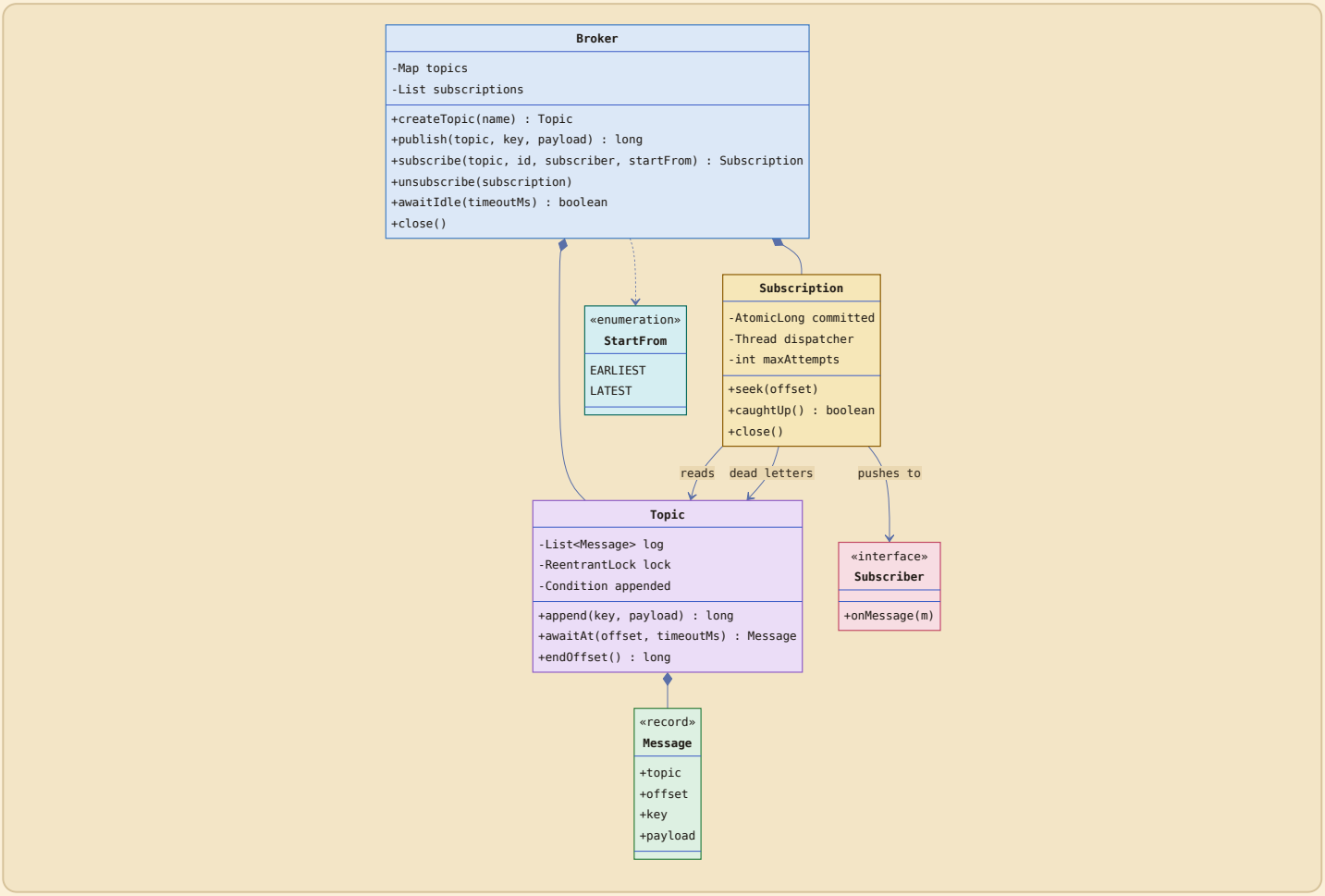
Actors: **Publisher** (publishes), **Subscriber** (subscribes, processes, fails sometimes), **Operator** (creates topics, replays, inspects dead letters), **Broker** (stores, dispatches, retries).

Core entities

Class	Responsibility
`Message` (record)	topic, offset, key, payload. Immutable.
`Topic`	Append-only log (`List<Message>`), guarded by a `ReentrantLock` with a `Condition` (an object a thread can wait on until another thread signals it) "appended" to wake waiting dispatchers.
`Subscriber` (interface)	Callback `onMessage(Message)`; throwing = negative acknowledgement ("I could not process this").
`Subscription`	One subscriber on one topic: committed offset (`AtomicLong`), its **own dispatcher thread**, retry and dead-letter logic, `seek`.
`StartFrom` (enum)	EARLIEST / LATEST.
`Broker`	Topic registry, `publish`, `subscribe`, `unsubscribe`, dead-letter topic, `awaitIdle` (tests), `close`.

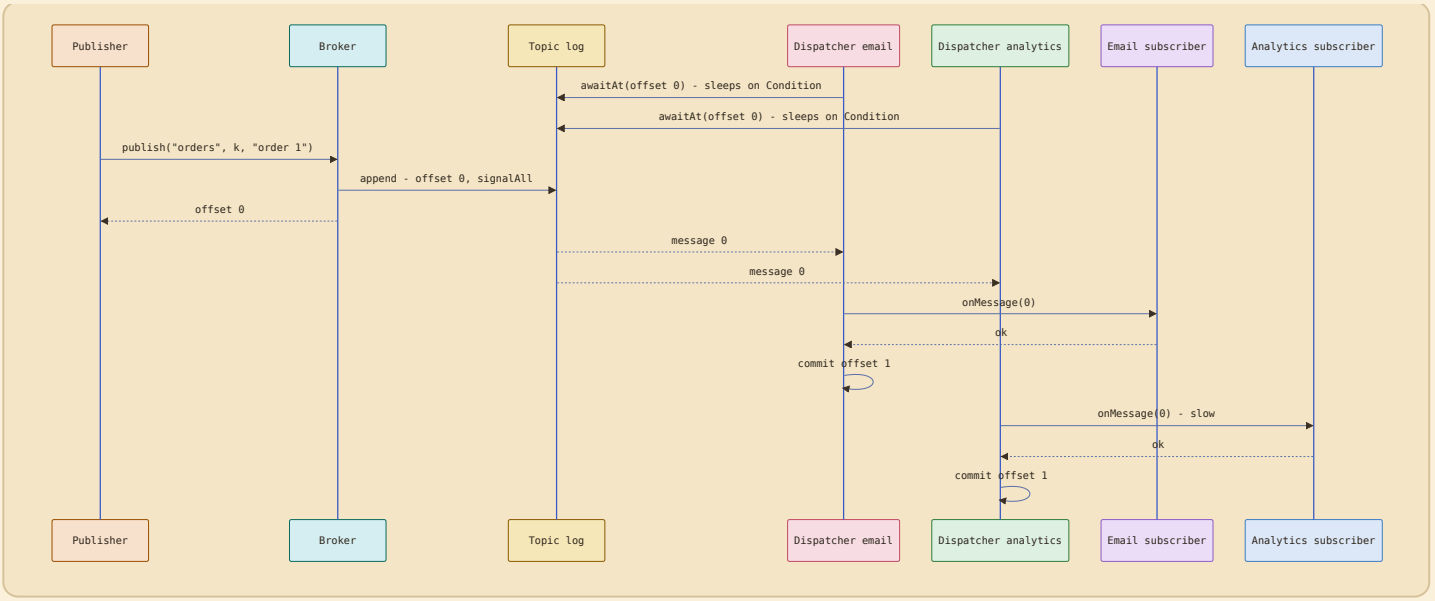
The key idea: messages are not removed when consumed. Each subscription only remembers *how far it has read* (its offset). That gives, almost for free: late subscribers from the beginning, replay (`seek`), and independent progress per subscriber. This is Kafka's model. (Classic queues such as RabbitMQ, a popular open-source message broker, instead delete a message once it is acknowledged.)

Class diagram

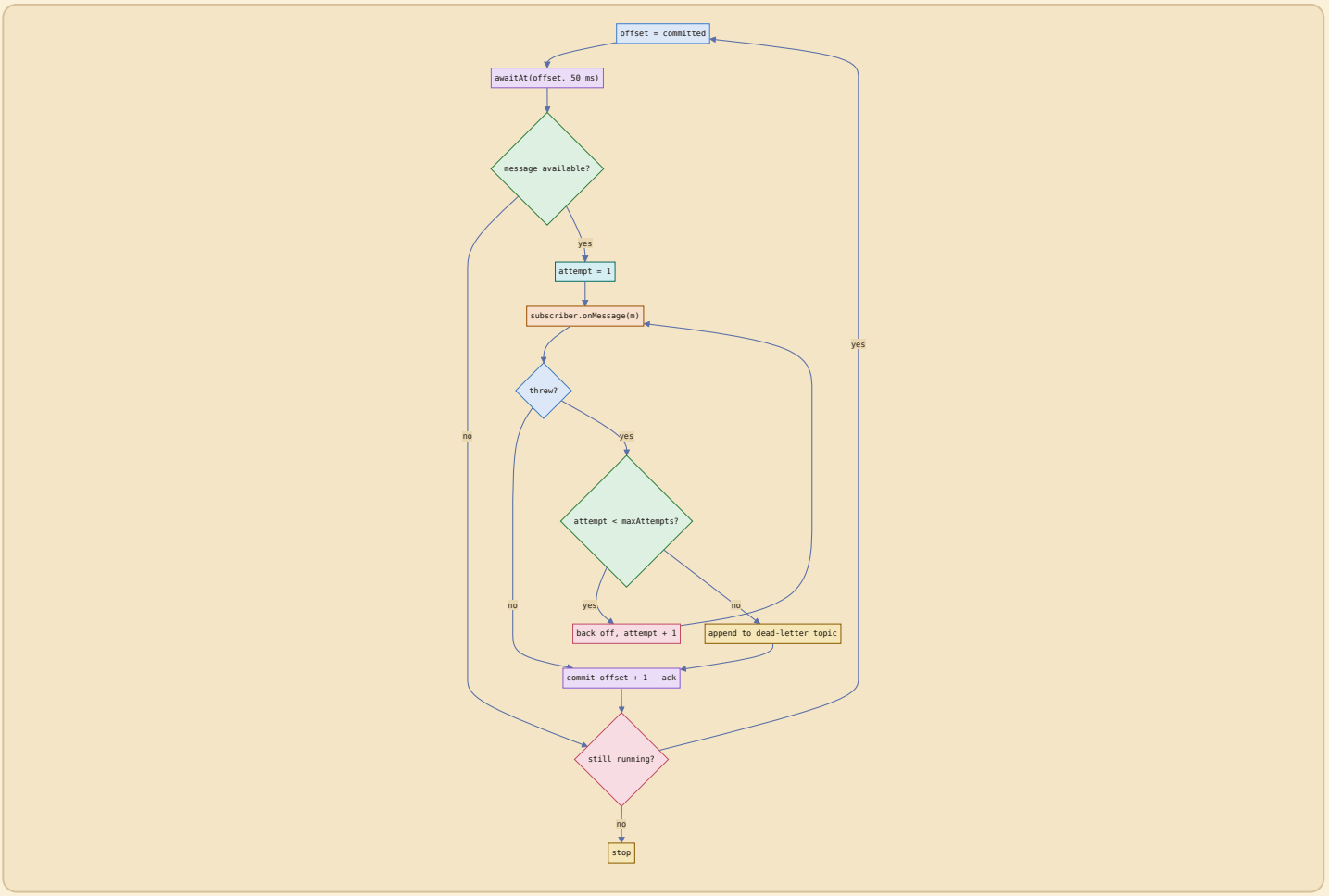


Key flows

Publish and fan-out.



One subscription's dispatcher loop (at-least-once with dead letters).



Design decisions & patterns used

Decision	Why	Alternative rejected
Publish-Subscribe (Observer via a broker)	Publishers and subscribers never reference each other; add a subscriber with zero publisher changes.	Publisher calling each consumer directly (tight coupling).
Append-only log + offsets	Replay, late joiners and independent progress. Ordering is the log order.	Deleting on delivery: no replay; per-subscriber copies: wasteful.
One dispatcher thread per subscription (virtual threads: Java 21's very cheap threads managed by the JVM rather than the operating system)	A slow subscriber only delays itself (see "analytics" in the output). Order within a subscription is kept because only one thread delivers for it. Virtual threads make one-thread-per-subscription cheap.	One shared thread delivering to all subscribers in turn (the slowest blocks everyone); a thread pool (a fixed set of reusable threads) per message (loses ordering).
`Lock` + `Condition` wait, with timeout	No busy-waiting; the timeout lets the loop notice `close()` and `seek()`. `while` loop guards against spurious wake-ups (a waiting thread may wake up although nobody signalled it).	`Thread.sleep` polling (repeatedly checking; wastes CPU and adds latency — the delay before something is handled).
Ack after processing (commit after `onMessage`)	At-least-once: a crash mid-processing re-delivers.	Commit before processing = at-most-once (each message is delivered once or not at all; can lose messages).
Dead-letter topic	One poison message cannot block the subscription forever; operators can inspect and re-publish.	Infinite retries (the subscription is stuck).
`compareAndSet` when committing	If `seek()` ran during delivery, the seek wins instead of being overwritten.	Plain `set`.

Delivery guarantees in one table:

Guarantee	How	Risk
At-most-once	Commit offset, then process	A crash loses the message
At-least-once (ours)	Process, then commit	A crash or retry duplicates the message → make subscribers **idempotent**
Exactly-once	At-least-once + deduplication (spotting and dropping repeats) / transactions that commit output and offset together	Complex; Kafka offers it within its own transactions

Java implementation

```java
// File: code/lld/ch20/pubsub/Topic.java
package lld.ch20.pubsub;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

/** One published message. The offset is its position in the topic's log. */
record Message(String topic, long offset, String key, String payload) {}

/**
 * An append-only log (like one Kafka partition). Messages are never removed by consumption:
 * each subscription just remembers how far it has read (its offset).
 */
final class Topic {
    private final String name;
    private final List<Message> log = new ArrayList<>();
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition appended = lock.newCondition();

    Topic(String name) { this.name = name; }

    String name() { return name; }

    long append(String key, String payload) {
        lock.lock();
        try {
            long offset = log.size();
            log.add(new Message(name, offset, key, payload));
            appended.signalAll();                  // wake every waiting dispatcher
            return offset;
        } finally {
            lock.unlock();
        }
    }

    /** Returns the message at 'offset', waiting up to the timeout for it to be published. */
    Message awaitAt(long offset, long timeoutMs) throws InterruptedException {
        lock.lock();
        try {
            long nanos = TimeUnit.MILLISECONDS.toNanos(timeoutMs);
            while (offset >= log.size()) {         // loop: guards against spurious wake-ups
                if (nanos <= 0) return null;
                nanos = appended.awaitNanos(nanos);
            }
            return log.get((int) offset);
        } finally {
            lock.unlock();
        }
    }

    long endOffset() {
        lock.lock();
        try {
            return log.size();
        } finally {
            lock.unlock();
        }
    }
}
```

```java
// File: code/lld/ch20/pubsub/Broker.java
package lld.ch20.pubsub;

import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicLong;

/** Observer (push style): the broker calls this for every message, in order. Throw = negative ack. */
interface Subscriber {
    void onMessage(Message m) throws Exception;
}

enum StartFrom { EARLIEST, LATEST }

/**
 * One subscriber's view of one topic: its committed offset and its own dispatcher thread.
 * Delivery is at-least-once and in order. A message is acknowledged (offset committed)
 * only after onMessage returns normally. After maxAttempts failures it goes to the dead-letter topic.
 */
final class Subscription {
    private final String id;
    private final Topic topic;
    private final Subscriber subscriber;
    private final int maxAttempts;
    private final Topic deadLetters;
    private final AtomicLong committed;        // next offset to deliver
    private volatile boolean running = true;
    private final Thread dispatcher;

    Subscription(String id, Topic topic, Subscriber subscriber, long startOffset, int maxAttempts, Topic deadLetters)
{
        this.id = id;
        this.topic = topic;
        this.subscriber = subscriber;
        this.maxAttempts = maxAttempts;
        this.deadLetters = deadLetters;
        this.committed = new AtomicLong(startOffset);
        this.dispatcher = Thread.ofVirtual().name("dispatch-" + id).start(this::loop);
    }

    private void loop() {
        try {
            while (running) {
                long offset = committed.get();
                Message m = topic.awaitAt(offset, 50);
                if (m == null) continue;              // timed out: re-check 'running' and any seek()
                deliver(m);
                committed.compareAndSet(offset, offset + 1); // ack; a concurrent seek() wins
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    private void deliver(Message m) throws InterruptedException {
        for (int attempt = 1; attempt <= maxAttempts; attempt++) {
            try {
                subscriber.onMessage(m);
                return;
            } catch (Exception e) {
                if (attempt == maxAttempts) {
                    deadLetters.append(id, m.topic() + "@" + m.offset() + " " + m.payload() + " -> " +
e.getMessage());
                    return;                           // skip it so one poison message cannot block the rest
                }
                Thread.sleep(5L * attempt);           // simple backoff
            }
        }
    }

    String id() { return id; }
```

```
    long committedOffset() { return committed.get(); }
    boolean caughtUp() { return committed.get() >= topic.endOffset(); }

    /** Replay (or skip) by moving the offset - possible because the log keeps every message. */
    void seek(long offset) { committed.set(offset); }

    void close() throws InterruptedException {
        running = false;
        dispatcher.join();
    }
}

/** Topic registry + publish + subscribe. Publishers never know who the subscribers are. */
final class Broker implements AutoCloseable {
    static final String DLQ = "__dead_letters";
    private final Map<String, Topic> topics = new ConcurrentHashMap<>();
    private final List<Subscription> subscriptions = new CopyOnWriteArrayList<>();

    Broker() { createTopic(DLQ); }

    Topic createTopic(String name) { return topics.computeIfAbsent(name, Topic::new); }

    long publish(String topic, String key, String payload) {
        Topic t = topics.get(topic);
        if (t == null) throw new IllegalArgumentException("no topic " + topic);
        return t.append(key, payload);
    }

    Subscription subscribe(String topic, String subscriptionId, Subscriber s, StartFrom from) {
        Topic t = topics.get(topic);
        if (t == null) throw new IllegalArgumentException("no topic " + topic);
        long start = from == StartFrom.EARLIEST ? 0 : t.endOffset();
        Subscription sub = new Subscription(subscriptionId, t, s, start, 3, topics.get(DLQ));
        subscriptions.add(sub);
        return sub;
    }

    void unsubscribe(Subscription s) throws InterruptedException {
        subscriptions.remove(s);
        s.close();
    }

    /** Test/shutdown helper: wait until every subscription has acknowledged everything. */
    boolean awaitIdle(long timeoutMs) throws InterruptedException {
        long deadline = System.currentTimeMillis() + timeoutMs;
        while (System.currentTimeMillis() < deadline) {
            if (subscriptions.stream().allMatch(Subscription::caughtUp)) return true;
            Thread.sleep(5);
        }
        return false;
    }

    Topic topic(String name) { return topics.get(name); }

    /** Stops every dispatcher. Declared without InterruptedException so try-with-resources stays simple. */
    @Override
    public void close() {
        try {
            for (Subscription s : subscriptions) s.close();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();      // keep the interrupt visible to the caller
        }
    }
}
```

```java
// File: code/lld/ch20/pubsub/Main.java
package lld.ch20.pubsub;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;

public class Main {

    /** Records what it received; can be told to fail some messages. */
    static final class RecordingSubscriber implements Subscriber {
        final List<String> received = Collections.synchronizedList(new ArrayList<>());
        final Set<Long> failOnceAt = ConcurrentHashMap.newKeySet();
        final String alwaysFailPayload;
        final long delayMs;

        RecordingSubscriber(long delayMs, String alwaysFailPayload) {
            this.delayMs = delayMs;
            this.alwaysFailPayload = alwaysFailPayload;
        }

        @Override
        public void onMessage(Message m) throws Exception {
            if (delayMs > 0) Thread.sleep(delayMs);
            if (m.payload().equals(alwaysFailPayload)) throw new Exception("cannot parse");
            if (failOnceAt.remove(m.offset())) throw new Exception("transient error");
            received.add(m.offset() + ":" + m.payload());
        }
    }

    public static void main(String[] args) throws Exception {
        try (Broker broker = new Broker()) {
            broker.createTopic("orders");

            RecordingSubscriber email = new RecordingSubscriber(0, null);
            RecordingSubscriber analytics = new RecordingSubscriber(20, null);       // slow consumer
            RecordingSubscriber inventory = new RecordingSubscriber(0, "BAD-JSON"); // poison message
            inventory.failOnceAt.add(1L);                                           // transient failure

            broker.subscribe("orders", "email", email, StartFrom.EARLIEST);
            broker.subscribe("orders", "analytics", analytics, StartFrom.EARLIEST);
            broker.subscribe("orders", "inventory", inventory, StartFrom.EARLIEST);

            System.out.println("== 1. Fan-out: every subscriber gets every message, in order");
            for (String p : List.of("order#1", "order#2", "BAD-JSON", "order#3"))
                broker.publish("orders", "k", p);
            System.out.println("  all acknowledged: " + broker.awaitIdle(5_000));
            System.out.println("  email     " + email.received);
            System.out.println("  analytics " + analytics.received + "  (slow, but did not slow the others)");
            System.out.println("  inventory " + inventory.received + "  (offset 1 retried, offset 2 dead-lettered)");
            Message dead = broker.topic(Broker.DLQ).awaitAt(0, 1_000);
            System.out.println("  dead letter: " + dead.payload());

            System.out.println("== 2. A late subscriber chooses where to start");
            RecordingSubscriber auditFromStart = new RecordingSubscriber(0, null);
            RecordingSubscriber onlyNew = new RecordingSubscriber(0, null);
            broker.subscribe("orders", "audit", auditFromStart, StartFrom.EARLIEST);
            Subscription latest = broker.subscribe("orders", "live", onlyNew, StartFrom.LATEST);
            broker.publish("orders", "k", "order#4");
            broker.awaitIdle(5_000);
            System.out.println("  audit (EARLIEST) " + auditFromStart.received);
            System.out.println("  live  (LATEST)   " + onlyNew.received);

            System.out.println("== 3. Replay: seek the 'live' subscription back to offset 3");
            latest.seek(3);
            broker.awaitIdle(5_000);
            System.out.println("  live after replay " + onlyNew.received + " (order#4 seen twice: at-least-once)");

            System.out.println("== 4. 4 publisher threads x 250 messages on a new topic");
            broker.createTopic("clicks");
            RecordingSubscriber clicks = new RecordingSubscriber(0, null);
```

```
            broker.subscribe("clicks", "counter", clicks, StartFrom.EARLIEST);
            List<Thread> pubs = new ArrayList<>();
            for (int t = 0; t < 4; t++) {
                int id = t;
                pubs.add(Thread.ofPlatform().start(() -> {
                    for (int i = 0; i < 250; i++) broker.publish("clicks", "user" + id, "c" + id + "-" + i);
                }));
            }
            for (Thread t : pubs) t.join();
            broker.awaitIdle(5_000);
            boolean inOrder = true;
            for (int i = 0; i < clicks.received.size(); i++)
                inOrder &= clicks.received.get(i).startsWith(i + ":");
            System.out.println("  received " + clicks.received.size() + " messages, offsets strictly in order: " +
inOrder);
        }
    }
}
```

Walking through the code.

- `Topic.append` adds to the list and calls `signalAll()` under the lock, waking every dispatcher waiting for new data.
- `Topic.awaitAt` waits on the `Condition` in a `while` loop with `awaitNanos` (correct handling of spurious wake-ups and timeouts), then returns the message at that offset.
- `Subscription.loop` delivers one message at a time and only then commits `offset + 1` with `compareAndSet`, so a concurrent `seek()` wins.
- `deliver` retries with a small backoff and, after `maxAttempts`, writes a description to the `__dead_letters` topic and moves on.
- `Broker.awaitIdle` is a test helper: it waits until every subscription's committed offset has reached its topic's end.

Output:

```
== 1. Fan-out: every subscriber gets every message, in order
  all acknowledged: true
  email     [0:order#1, 1:order#2, 2:BAD-JSON, 3:order#3]
  analytics [0:order#1, 1:order#2, 2:BAD-JSON, 3:order#3]  (slow, but did not slow the others)
  inventory [0:order#1, 1:order#2, 3:order#3]  (offset 1 retried, offset 2 dead-lettered)
  dead letter: orders@2 BAD-JSON -> cannot parse
== 2. A late subscriber chooses where to start
  audit (EARLIEST) [0:order#1, 1:order#2, 2:BAD-JSON, 3:order#3, 4:order#4]
  live  (LATEST)   [4:order#4]
== 3. Replay: seek the 'live' subscription back to offset 3
  live after replay [4:order#4, 3:order#3, 4:order#4] (order#4 seen twice: at-least-once)
== 4. 4 publisher threads x 250 messages on a new topic
  received 1000 messages, offsets strictly in order: true
```

Points to notice: the slow "analytics" subscriber still got everything, and the other subscribers were not delayed; "inventory" retried offset 1 (a transient — temporary — failure) and dead-lettered offset 2 (the poison message) while still receiving offset 3; after `seek(3)`, "live" saw order#4 twice — that is what at-least-once means. The 1000 messages from 4 concurrent publishers got unique offsets and arrived strictly in order.

Concurrency & edge cases

- **Concurrent publishers:** `append` is under a lock, so offsets are unique and gap-free.
- **Readers vs writers:** dispatchers only read under the same lock; messages are immutable records, safe to share.
- **Unbounded memory:** the log grows forever; real brokers apply **retention** (delete segments — the chunks the log is stored in — older than N days or beyond N bytes).
- **Slow subscriber falling far behind:** it is fine here (it just reads older offsets); with retention it may fall off the end — detect "offset out of range" and reset.
- **Subscriber that blocks forever:** add a processing timeout. It stalls only its own subscription, because every subscription has its own dispatcher thread. (One Java 21 caveat: a virtual thread that blocks while inside a `synchronized` block *pins* its carrier platform thread (the real operating-system thread it

runs on); many such subscribers could exhaust the carriers. Prefer `ReentrantLock` in subscriber code, or use platform threads.)

- **Duplicates:** at-least-once means subscribers must be idempotent (e.g. remember processed order ids).
- **Shutdown:** `close()` flips `running` and joins each dispatcher (waits for its thread to finish); the 50 ms wait timeout bounds how long that takes.

Extensions & follow-up questions

- **"Consumer groups (share the work)."** Split a topic into **partitions** (several logs); messages with the same key go to the same partition (`hash(key) % n`), keeping per-key order. A **consumer group** is a set of consumers that split one topic's work between them; a group's members each own some partitions; the committed offset is per (group, partition). That is how Kafka scales consumers.
- **"Persist to disk."** Append to segment files; keep an index (a lookup table) from offset to file position; store committed offsets in a special topic (Kafka uses `__consumer_offsets`).
- **"Pull instead of push."** Expose `poll(subscriptionId, maxMessages)` returning messages from the committed offset and a `commit(offset)` call; the client controls its pace (back-pressure for free).
- **"Message filtering."** Subscribe with a predicate (a yes/no test, e.g. only `region = EU`); dispatcher skips non-matching messages but still commits them.
- **"Wildcard topics."** Subscribe to `orders.*` (`*` is a wildcard that matches any name part); the broker matches topic names on creation.
- **"Delayed messages / retry topics."** Re-publish failed messages to `orders.retry.5s` consumed after a delay, instead of blocking the dispatcher with sleeps.

Real-world example

Apache Kafka is exactly this model at scale: topics split into partitions, each an append-only log; consumers track offsets; consumer groups share partitions; ordering is per partition; messages are retained by time/size rather than deleted on read, so services can replay history. **Google Cloud Pub/Sub** and **AWS SNS + SQS** (Amazon's notification and queue services) offer managed fan-out with acknowledgements and **dead-letter** topics/queues. In Java, Spring Kafka's (Spring is the most popular Java application framework) `DeadLetterPublishingRecoverer` publishes a record that keeps failing to a dead-letter topic after the retries are used up — the same idea as our `__dead_letters`. Inside one JVM, Guava's `EventBus` and Spring's `ApplicationEventPublisher` are simple in-process pub-sub.

In the interview

> **30-second answer:** "Each topic is an append-only log; publishing appends and returns an offset. Each subscription keeps a committed offset and has its own dispatcher thread that waits on a condition for new data, calls the subscriber, and commits only after success — at-least-once and in order. Failures retry with backoff, then go to a dead-letter topic. Because messages aren't deleted on read, late subscribers can start at the beginning and anyone can replay by seeking. To scale, split topics into partitions by key and let consumer groups share partitions."

Plan (35–40 min): clarify guarantees (5) → topic log + offsets (6) → subscription + dispatcher + ack (8) → retries + DLQ (dead-letter queue) (5) → thread safety and ordering (6) → partitions / consumer groups / persistence (5).

Follow-ups with answers:

- *"How do you keep a slow consumer from slowing others?"* → One dispatcher per subscription; the log is shared, progress is not.
- *"Exactly-once?"* → Hard; at-least-once + idempotent consumers is the practical answer.
- *"How is order guaranteed?"* → One writer (the lock) assigns offsets; one dispatcher per subscription delivers in offset order. With partitions, order is per partition (per key).

Self-check

1. What is an offset? → A message's position in a topic's log; subscribers remember the next one to read.
2. Commit before or after processing for at-least-once? → After.
3. Why a dead-letter topic? → So one poison message cannot block a subscription forever.

4. Why does replay work here but not in a delete-on-ack queue? → Messages stay in the log; seeking just moves the offset.
5. Why `while` around `Condition.await`? → Spurious wake-ups; re-check the condition every time.
6. How does Kafka let several consumers share one topic's work while keeping order per key? → Partitions by key hash; each partition consumed by one group member.

TL;DR

- **Pub-sub = Observer through a broker**; publishers and subscribers are decoupled.
- **Append-only log + per-subscription offsets** → fan-out, late joiners, replay.
- **One dispatcher per subscription**: isolation + order; **ack after processing** = at-least-once.
- Retries with backoff, then **dead-letter**; subscribers must be **idempotent**.
- Scale: partitions by key, consumer groups, retention, persisted offsets (Kafka).

20.7 Design a Rate Limiter

Problem & clarifying questions

What it is. A rate limiter decides whether a request is allowed right now, so one client can't overwhelm a service (by accident or on purpose). Everyday analogy: a turnstile that lets through at most 10 people per minute per ticket type. Over the limit, the answer is "come back in 6 seconds" — in HTTP, status **429 Too Many Requests** with a `Retry-After` header (a header is a named line of metadata sent with an HTTP response).

Chapter 13 (§13.11) already built the concurrency core: a thread-safe token bucket (a bucket of tokens refilled at a steady rate; each request spends one, and an empty bucket means "wait") with `synchronized` and a lock-free CAS version (CAS = compare-and-set: update a value only if it still holds what you last read, as one hardware-level step, so no lock is needed), and compared the algorithms. This section is about the **object-oriented design around it**: pluggable algorithms, rules per API and per user tier (a plan level such as free or paid), a factory, combined limits, and a clean decision object.

Question	Assumed answer
Limit by what?	Per (user, API). Could also be IP (the caller's network address) or API key (a secret string that identifies the calling app) — same design, different key.
Different limits?	Yes: per API and per tier (FREE vs PREMIUM); a default for everything else.
Algorithms?	Token bucket, fixed window, sliding window log, sliding window counter (all compared in the table under Key flows) — swappable per rule.
Several limits at once?	Yes, e.g. "2 per 10 s **and** 3 per minute" on `/login`.
Response?	Allowed / denied + remaining + retry-after.
Single process or distributed (spread over several machines)?	Single process here; distributed in extensions.

Requirements

Functional

1. `check(userId, api)` → `Decision(allowed, remaining, retryAfter)`.
2. Rules configured per API and tier; fallback default rule.
3. Each (user, api) gets its **own** limiter instance, created on first use.
4. Algorithms pluggable per rule.
5. Several rules can apply together (all must allow).

Non-functional: O(1) (or close) per check; thread-safe (exactly N allowed under any concurrency); injectable clock for tests; memory bounded (discussed).

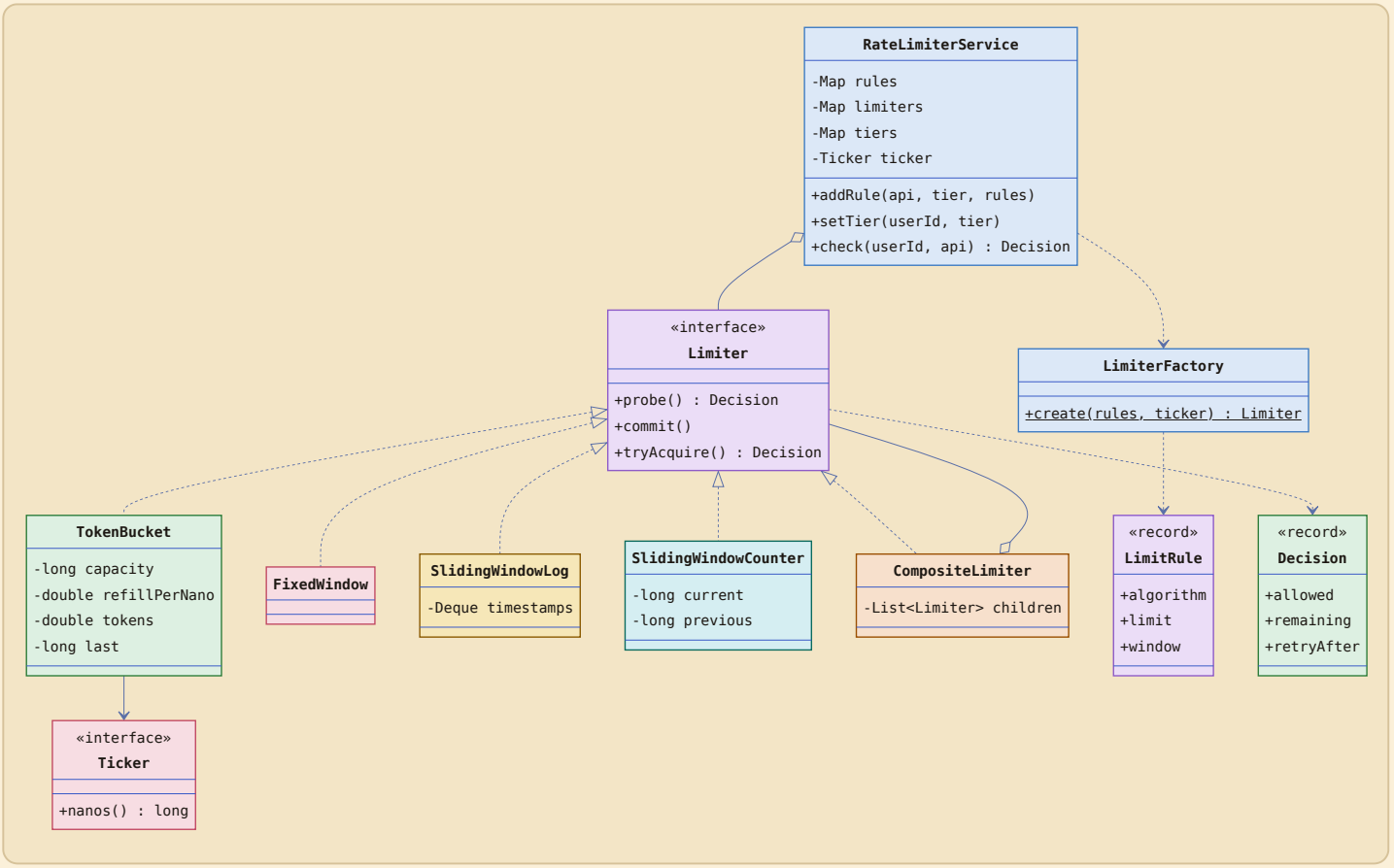
Use cases

Actors: **Client** (sends requests), **API gateway / filter** (the front door that every request passes through; calls `check` before the handler — the code that does the real work), **Operator** (configures rules and tiers).

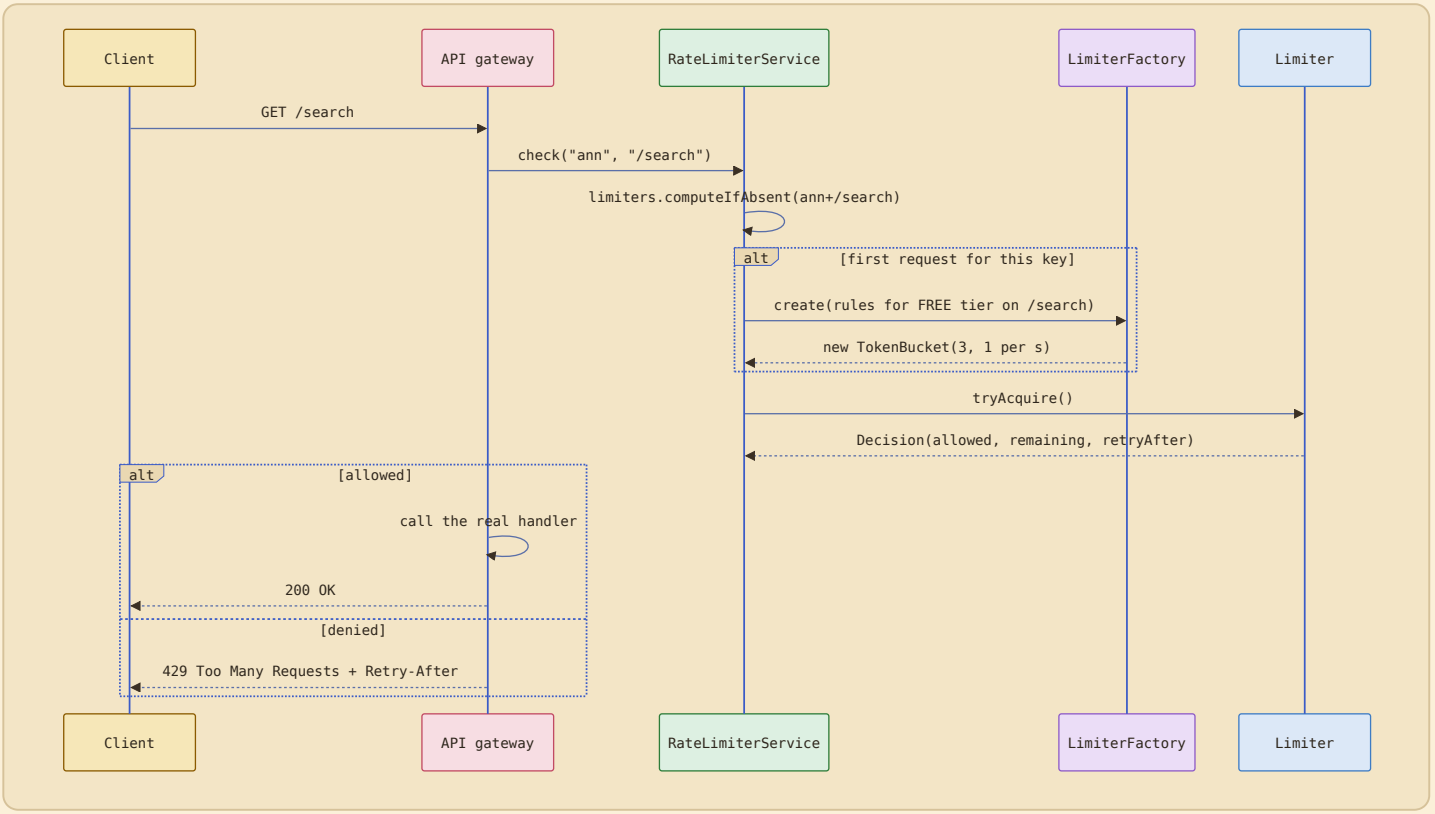
Core entities

Class	Responsibility
`Limiter` (interface)	**Strategy**: `probe()` (would one more request pass? consumes nothing), `commit()` (record it), and `tryAcquire()` = both in one atomic step → `Decision`. One instance per key.
`TokenBucket`, `FixedWindow`, `SlidingWindowLog`, `SlidingWindowCounter`	The algorithms (each `synchronized`).
`CompositeLimiter`	**Composite**: allowed only if every child allows; probes all children, then commits to all, under one lock, so a denied request consumes nothing.
`Decision` (record)	allowed, remaining, retryAfter.
`LimitRule` (record)	algorithm, limit, window.
`Algorithm`, `Tier` (enums)	Configuration values.
`LimiterFactory`	**Simple Factory**: rule(s) → `Limiter` (single or composite).
`RateLimiterService`	**Facade**: resolves the user's tier and the API's rules, keeps one limiter per (user, api) in a `ConcurrentHashMap`.
`Ticker`	Injected time source (nanoseconds).

Class diagram



Key flows



The four algorithms side by side (limit N per window W):

Algorithm	State per key	Bursts	Accuracy	Notes
Token bucket	tokens, last refill time	Up to capacity	Good	Most common for APIs; smooth refill
Fixed window counter	window start, count	**Up to 2N across a boundary**	Poor at edges	Simplest
Sliding window log	one timestamp per allowed request	None beyond N	Exact	Memory O(N) per key
Sliding window counter	2 counters	Approximate	Good	Estimate = previous × overlap + current
(Leaky bucket)	a queue	Smooths output to a constant rate	—	Traffic shaping (evening out the flow); not implemented here

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy (`Limiter`)	Algorithms differ in accuracy/memory; pick per rule. New algorithm = new class.	`switch` on algorithm inside `check`.
Simple Factory (`LimiterFactory`)	One place turns config into objects; service never names concrete classes.	`new TokenBucket(...)` in the service.
Composite (`CompositeLimiter`)	"10/s and 1000/day" is just a limiter made of limiters; callers don't know. **Check all, then commit all**, so a request denied by one child never uses up a slot in another.	Special-casing multiple rules in the service; or calling each child's `tryAcquire` in turn (a later denial would leave earlier children charged for a request that never ran).
Facade (`RateLimiterService`)	Gateway code asks one question.	Gateway juggling tiers, rules and limiters.
One limiter per key via `computeIfAbsent`	Exactly one limiter per (user, api), even when the first two requests race.	`get` then `put` (two limiters could be created and one's counts lost).
`Decision` with `remaining` and `retryAfter`	Maps directly to HTTP headers (`X-RateLimit-Remaining`, `Retry-After`).	`boolean`.
Injected `Ticker`	Tests move time instantly (the demo never sleeps).	`System.nanoTime()` inside algorithms.
Each algorithm `synchronized`; `tryAcquire` = `probe` + `commit` under the limiter's lock	Refill + check + take must be one atomic step (see §13.11 for the lock-free version).	Unsynchronized fields (two threads take the last token).

Java implementation

```java
// File: code/lld/ch20/ratelimiter/Limiters.java
package lld.ch20.ratelimiter;

import java.time.Duration;
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.List;

/** Where time comes from (nanoseconds). Injected so tests do not sleep. */
interface Ticker { long nanos(); }

/** The answer to one request: allowed or not, what is left, and when to retry (HTTP 429 Retry-After). */
record Decision(boolean allowed, long remaining, Duration retryAfter) {
    static Decision allow(long remaining) { return new Decision(true, remaining, Duration.ZERO); }
    static Decision deny(Duration retryAfter) { return new Decision(false, 0, retryAfter); }
}

/**
 * Strategy: one algorithm, one instance per key (user, API key, IP...). Must be thread-safe.
 * Two-phase so that limits can be combined without side effects:
 * probe() answers "would one more request be allowed now?" WITHOUT using up anything,
 * commit() records the request. Both must be called while holding this object's lock.
 */
interface Limiter {
    Decision probe();

    void commit();

    /** Check and take in ONE atomic step (probe + commit under this limiter's lock). */
    default Decision tryAcquire() {
        synchronized (this) {
            Decision d = probe();
            if (d.allowed()) commit();
            return d;
        }
    }
}

/** Token bucket: bursts up to 'capacity', refills at 'perSecond'. Lazy refill, no background thread. */
final class TokenBucket implements Limiter {
    private final long capacity;
    private final double refillPerNano;
    private final Ticker ticker;
    private double tokens;
    private long last;

    TokenBucket(long capacity, double perSecond, Ticker ticker) {
        this.capacity = capacity;
        this.refillPerNano = perSecond / 1e9;
        this.ticker = ticker;
        this.tokens = capacity;
        this.last = ticker.nanos();
    }

    @Override
    public synchronized Decision probe() {
        long now = ticker.nanos();
        tokens = Math.min(capacity, tokens + (now - last) * refillPerNano); // refilling is safe to do early
        last = now;
        if (tokens >= 1) return Decision.allow((long) (tokens - 1));
        long waitNanos = refillPerNano == 0 ? Long.MAX_VALUE : (long) Math.ceil((1 - tokens) / refillPerNano);
        return Decision.deny(Duration.ofNanos(waitNanos));
    }

    @Override
    public synchronized void commit() { tokens -= 1; }
}

/** Fixed window counter: simplest; allows up to 2x the limit across a window boundary. */
final class FixedWindow implements Limiter {
    private final long limit;
```

```
    private final long windowNanos;
    private final Ticker ticker;
    private long windowStart;
    private long count;

    FixedWindow(long limit, Duration window, Ticker ticker) {
        this.limit = limit;
        this.windowNanos = window.toNanos();
        this.ticker = ticker;
    }

    @Override
    public synchronized Decision probe() {
        long now = ticker.nanos();
        long currentWindow = now - Math.floorMod(now, windowNanos);
        if (currentWindow != windowStart) {
            windowStart = currentWindow;
            count = 0;
        }
        if (count < limit) return Decision.allow(limit - count - 1);
        return Decision.deny(Duration.ofNanos(windowStart + windowNanos - now));
    }

    @Override
    public synchronized void commit() { count++; }
}

/** Sliding window log: exact, but stores one timestamp per allowed request. */
final class SlidingWindowLog implements Limiter {
    private final long limit;
    private final long windowNanos;
    private final Ticker ticker;
    private final Deque<Long> log = new ArrayDeque<>();
    private long probedAt;                       // commit() records the time the probe saw

    SlidingWindowLog(long limit, Duration window, Ticker ticker) {
        this.limit = limit;
        this.windowNanos = window.toNanos();
        this.ticker = ticker;
    }

    @Override
    public synchronized Decision probe() {
        long now = ticker.nanos();
        probedAt = now;
        while (!log.isEmpty() && log.peekFirst() <= now - windowNanos) log.pollFirst();
        if (log.size() < limit) return Decision.allow(limit - log.size() - 1);
        return Decision.deny(Duration.ofNanos(log.peekFirst() + windowNanos - now));
    }

    @Override
    public synchronized void commit() { log.addLast(probedAt); }
}

/** Sliding window counter: two counters, weights the previous window by how much of it still overlaps. */
final class SlidingWindowCounter implements Limiter {
    private final long limit;
    private final long windowNanos;
    private final Ticker ticker;
    private long currentStart;
    private long current, previous;

    SlidingWindowCounter(long limit, Duration window, Ticker ticker) {
        this.limit = limit;
        this.windowNanos = window.toNanos();
        this.ticker = ticker;
    }

    @Override
    public synchronized Decision probe() {
        long now = ticker.nanos();
        long start = now - Math.floorMod(now, windowNanos);
        if (start != currentStart) {
            previous = (start - currentStart == windowNanos) ? current : 0; // older than one window: forget it
```

```
            current = 0;
            currentStart = start;
        }
        double overlap = 1.0 - (double) (now - currentStart) / windowNanos;
        double estimate = previous * overlap + current;
        if (estimate + 1 <= limit) return Decision.allow((long) (limit - estimate - 1));
        return Decision.deny(Duration.ofNanos(currentStart + windowNanos - now));
    }

    @Override
    public synchronized void commit() { current++; }
}

/**
 * Composite: allowed only if EVERY child allows (e.g. 10/second AND 1000/day).
 * Check-all-then-commit under one lock: a request that ANY child denies uses up NOTHING,
 * so a denied request can never eat a slot in an earlier child.
 * The children are created for this composite only (by LimiterFactory) and never shared,
 * so holding the composite's lock is enough to make probe-all + commit-all atomic.
 */
final class CompositeLimiter implements Limiter {
    private final List<Limiter> children;

    CompositeLimiter(List<Limiter> children) { this.children = List.copyOf(children); }

    @Override
    public synchronized Decision probe() {
        long remaining = Long.MAX_VALUE;
        for (Limiter l : children) {
            Decision d;
            synchronized (l) { d = l.probe(); }
            if (!d.allowed()) return d;           // first denial wins; nothing has been consumed
            remaining = Math.min(remaining, d.remaining());
        }
        return Decision.allow(remaining);
    }

    @Override
    public synchronized void commit() {
        for (Limiter l : children) synchronized (l) { l.commit(); }
    }
}
```

```java
// File: code/lld/ch20/ratelimiter/RateLimiterService.java
package lld.ch20.ratelimiter;

import java.time.Duration;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

enum Tier { FREE, PREMIUM }

enum Algorithm { TOKEN_BUCKET, FIXED_WINDOW, SLIDING_LOG, SLIDING_COUNTER }

/** Configuration: which algorithm and how many requests per window (token bucket: burst = limit). */
record LimitRule(Algorithm algorithm, long limit, Duration window) {}

/** Factory: turns a rule into a fresh Limiter object. The only place that knows the concrete classes. */
final class LimiterFactory {
    private LimiterFactory() {}

    static Limiter create(List<LimitRule> rules, Ticker ticker) {
        if (rules.size() == 1) return create(rules.get(0), ticker);
        return new CompositeLimiter(rules.stream().map(r -> create(r, ticker)).toList());
    }

    static Limiter create(LimitRule r, Ticker ticker) {
        return switch (r.algorithm()) {
            case TOKEN_BUCKET -> new TokenBucket(r.limit(), r.limit() / (r.window().toNanos() / 1e9), ticker);
            case FIXED_WINDOW -> new FixedWindow(r.limit(), r.window(), ticker);
            case SLIDING_LOG -> new SlidingWindowLog(r.limit(), r.window(), ticker);
            case SLIDING_COUNTER -> new SlidingWindowCounter(r.limit(), r.window(), ticker);
        };
    }
}

/** Facade used by an API gateway / servlet filter: "may user U call API A right now?" */
final class RateLimiterService {
    private record Key(String userId, String api) {}

    private final Map<String, Map<Tier, List<LimitRule>>> rules = new ConcurrentHashMap<>();
    private final List<LimitRule> defaultRules;
    private final Map<Key, Limiter> limiters = new ConcurrentHashMap<>();
    private final Map<String, Tier> tiers = new ConcurrentHashMap<>();
    private final Ticker ticker;

    RateLimiterService(List<LimitRule> defaultRules, Ticker ticker) {
        this.defaultRules = List.copyOf(defaultRules);
        this.ticker = ticker;
    }

    void addRule(String api, Tier tier, LimitRule... r) {
        rules.computeIfAbsent(api, a -> new ConcurrentHashMap<>()).put(tier, List.of(r));
    }

    void setTier(String userId, Tier tier) { tiers.put(userId, tier); }

    Decision check(String userId, String api) {
        // computeIfAbsent creates exactly ONE limiter per (user, api), even under a race
        Limiter limiter = limiters.computeIfAbsent(new Key(userId, api), k -> {
            Tier tier = tiers.getOrDefault(userId, Tier.FREE);
            List<LimitRule> r = rules.getOrDefault(api, Map.of()).getOrDefault(tier, defaultRules);
            return LimiterFactory.create(r, ticker);
        });
        return limiter.tryAcquire();
    }
}
```

```java
// File: code/lld/ch20/ratelimiter/Main.java
package lld.ch20.ratelimiter;

import java.time.Duration;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicLong;

public class Main {
    static final AtomicLong NOW = new AtomicLong();          // manual clock in nanoseconds
    static final Ticker TICKER = NOW::get;

    static void at(double seconds) { NOW.set((long) (seconds * 1e9)); }

    public static void main(String[] args) throws Exception {
        RateLimiterService svc = new RateLimiterService(
                List.of(new LimitRule(Algorithm.FIXED_WINDOW, 100, Duration.ofMinutes(1))), TICKER);
        svc.addRule("/search", Tier.FREE, new LimitRule(Algorithm.TOKEN_BUCKET, 3, Duration.ofSeconds(3))); // 1/s,
burst 3
        svc.addRule("/search", Tier.PREMIUM, new LimitRule(Algorithm.TOKEN_BUCKET, 10, Duration.ofSeconds(1)));
        svc.addRule("/login", Tier.FREE,
                new LimitRule(Algorithm.SLIDING_LOG, 2, Duration.ofSeconds(10)),
                new LimitRule(Algorithm.SLIDING_LOG, 3, Duration.ofMinutes(1)));    // composite
        svc.setTier("pat", Tier.PREMIUM);

        System.out.println("== 1. Per-user, per-API token bucket (FREE: burst 3, 1 token/s)");
        at(0);
        for (int i = 1; i <= 5; i++) System.out.println("  t=0.0 ann /search #" + i + " -> " + show(svc.check("ann",
"/search")));
        at(1.0);
        System.out.println("  t=1.0 ann /search    -> " + show(svc.check("ann", "/search")));
        System.out.println("  t=1.0 bob /search    -> " + show(svc.check("bob", "/search")) + "  (own bucket)");
        int premium = 0;
        for (int i = 0; i < 12; i++) if (svc.check("pat", "/search").allowed()) premium++;
        System.out.println("  t=1.0 pat (PREMIUM) 12 calls -> " + premium + " allowed");
        System.out.println("  t=1.0 ann /orders (default rule) -> " + show(svc.check("ann", "/orders")));

        System.out.println("== 2. Composite rule on /login: 2 per 10 s AND 3 per minute");
        double[] times = {0, 1, 2, 11, 12, 61};
        at(100);
        for (double t : times) {
            at(100 + t);
            System.out.println("  t=+" + (int) t + "s ann /login -> " + show(svc.check("ann", "/login")));
        }

        System.out.println("== 3. A denied request uses up nothing: 3 per minute AND 2 per 10 s (minute rule first)");
        Limiter login = LimiterFactory.create(List.of(
                new LimitRule(Algorithm.SLIDING_LOG, 3, Duration.ofMinutes(1)),
                new LimitRule(Algorithm.SLIDING_LOG, 2, Duration.ofSeconds(10))), TICKER);
        for (double t : new double[]{0, 1, 2, 11}) {
            at(200 + t);
            System.out.println("  t=+" + (int) t + "s -> " + show(login.tryAcquire()));
        }

        System.out.println("== 4. Boundary burst: limit 5 per 10 s, 5 calls at t=9.9 and 5 at t=10.1");
        for (Algorithm a : List.of(Algorithm.FIXED_WINDOW, Algorithm.SLIDING_LOG, Algorithm.SLIDING_COUNTER)) {
            Limiter l = LimiterFactory.create(new LimitRule(a, 5, Duration.ofSeconds(10)), TICKER);
            int allowed = 0;
            at(1009.9);
            for (int i = 0; i < 5; i++) if (l.tryAcquire().allowed()) allowed++;
            at(1010.1);
            for (int i = 0; i < 5; i++) if (l.tryAcquire().allowed()) allowed++;
            System.out.printf("  %-16s allowed %d of 10 within 0.2 s%n", a, allowed);
        }

        System.out.println("== 5. 64 threads x 10 calls on one bucket (capacity 50, no refill)");
        Limiter bucket = new TokenBucket(50, 0, TICKER);
        AtomicInteger ok = new AtomicInteger();
```

```
        CountDownLatch start = new CountDownLatch(1);
        ExecutorService pool = Executors.newFixedThreadPool(64);
        List<java.util.concurrent.Future<?>> fs = new ArrayList<>();
        for (int t = 0; t < 64; t++)
            fs.add(pool.submit(() -> {
                start.await();
                for (int i = 0; i < 10; i++) if (bucket.tryAcquire().allowed()) ok.incrementAndGet();
                return null;
            }));
        start.countDown();
        for (var f : fs) f.get();
        pool.shutdown();
        pool.awaitTermination(5, TimeUnit.SECONDS);
        System.out.println("  allowed = " + ok.get() + " (exactly 50, never more)");
    }

    private static String show(Decision d) {
        return d.allowed() ? "200 OK (remaining " + d.remaining() + ")"
                : "429 Too Many Requests (retry after " + d.retryAfter().toMillis() + " ms)";
    }
}
```

Walking through the code.

- `TokenBucket` refills **lazily**: on each call it adds `elapsed × rate` tokens (capped at capacity). No background thread. When empty, `retryAfter` is the time until one whole token exists.
- `FixedWindow` computes the current window start with `now − (now mod W)` and resets the count when the window changes.
- `SlidingWindowLog` drops timestamps older than `now − W` and allows if fewer than N remain.
- `SlidingWindowCounter` estimates the count in the sliding window as `previous × (portion of previous window still inside) + current`.
- `Limiter` is split into `probe()` (answer only) and `commit()` (record); the default `tryAcquire()` runs both inside `synchronized (this)`, and every algorithm's own methods are `synchronized` on the same object, so check-and-take stays one atomic step.
- `CompositeLimiter` probes **every** child first and returns the first denial without committing anything; only if all allow does it commit to all of them. Its children are created for it alone by the factory and never shared, so the composite's lock makes the whole probe-then-commit atomic. `remaining` is the smallest of the children's.
- `RateLimiterService.check` builds the limiter inside `computeIfAbsent`, so the tier and rule are resolved once per key.

Output:

```
== 1. Per-user, per-API token bucket (FREE: burst 3, 1 token/s)
  t=0.0 ann /search #1 -> 200 OK (remaining 2)
  t=0.0 ann /search #2 -> 200 OK (remaining 1)
  t=0.0 ann /search #3 -> 200 OK (remaining 0)
  t=0.0 ann /search #4 -> 429 Too Many Requests (retry after 1000 ms)
  t=0.0 ann /search #5 -> 429 Too Many Requests (retry after 1000 ms)
  t=1.0 ann /search    -> 200 OK (remaining 0)
  t=1.0 bob /search    -> 200 OK (remaining 2)  (own bucket)
  t=1.0 pat (PREMIUM) 12 calls -> 10 allowed
  t=1.0 ann /orders (default rule) -> 200 OK (remaining 99)
== 2. Composite rule on /login: 2 per 10 s AND 3 per minute
  t=+0s ann /login -> 200 OK (remaining 1)
  t=+1s ann /login -> 200 OK (remaining 0)
  t=+2s ann /login -> 429 Too Many Requests (retry after 8000 ms)
  t=+11s ann /login -> 200 OK (remaining 0)
  t=+12s ann /login -> 429 Too Many Requests (retry after 48000 ms)
  t=+61s ann /login -> 200 OK (remaining 1)
== 3. A denied request uses up nothing: 3 per minute AND 2 per 10 s (minute rule first)
  t=+0s -> 200 OK (remaining 1)
  t=+1s -> 200 OK (remaining 0)
  t=+2s -> 429 Too Many Requests (retry after 8000 ms)
  t=+11s -> 200 OK (remaining 0)
== 4. Boundary burst: limit 5 per 10 s, 5 calls at t=9.9 and 5 at t=10.1
  FIXED_WINDOW     allowed 10 of 10 within 0.2 s
  SLIDING_LOG      allowed 5 of 10 within 0.2 s
  SLIDING_COUNTER  allowed 5 of 10 within 0.2 s
== 5. 64 threads x 10 calls on one bucket (capacity 50, no refill)
  allowed = 50 (exactly 50, never more)
```

Step 1: ann's burst of 3 is allowed, then 429 with "retry after 1000 ms"; a second later exactly one more is allowed; bob and the PREMIUM user have their own buckets. Step 2: the /login composite denies at +2s (10-second rule) and at +12s (1-minute rule, retry when the +0s call leaves the minute window). Step 3: the denied +2s call consumed nothing, so the +11s call passes both rules. Step 4 is the classic interview point: **fixed window lets 10 requests through in 0.2 seconds** across the boundary, while both sliding algorithms hold the line at 5. Step 5 proves thread safety: 640 concurrent attempts, exactly 50 allowed.

Concurrency & edge cases

- **Atomicity:** each limiter's `tryAcquire` is `synchronized`; the service map is a `ConcurrentHashMap`. Contention is per key, so different users never block each other.
- **Composite and denied requests.** The naive composite calls each child's `tryAcquire()` in turn. If child 1 allows (and consumes) and child 2 then denies, the request never runs but child 1 has been charged for it. Step 3 of the demo is built to expose that: with "3 per minute" first, the denied call at +2s would have used up the third per-minute slot, and the legitimate call at +11s would be refused for about 49 s. Our composite probes all children and commits only if all allow, so +11s is correctly allowed. (An alternative is *acquire, and refund on denial*, which needs a `release()` in every algorithm and is easy to get wrong when a window rolls over in between.)
- **Memory growth:** one limiter per (user, api) forever. Evict idle limiters (e.g. store them in a cache with expire-after-access, like 20.3). Set the idle timeout to at least the longest window (or the full-refill time of a bucket): after that much idle time the old limiter would allow a full burst anyway, so replacing it with a fresh one changes nothing.
- **Clock:** use a monotonic clock (`System.nanoTime`) — wall-clock time (the calendar time a person reads) can jump backwards.
- **Rule changes at runtime:** existing limiters keep old settings; clear the map (or version the rules) when config changes.

Extensions & follow-up questions

- **"Distributed (many gateway servers)."** Keep counters in a shared store: Redis with an atomic Lua script (Redis can run small Lua programs as one uninterrupted step) (or `INCR` + `EXPIRE` for fixed windows: Redis commands for "add one" and "delete after N seconds"). Trade-off: a network hop (one extra round trip to another machine) per request; mitigate with local pre-allocation of tokens. Or use sticky routing (the load balancer — the component that spreads requests over servers — always sends one user to the same server) so each user always hits the same node.

- **"Leaky bucket."** A bounded queue drained at a constant rate by a worker: smooths bursts into a steady output (good for protecting a fragile downstream — a service that this one calls).
- **"Different cost per request."** `tryAcquire(permits)`: an expensive export costs 10 tokens.
- **"Soft limits / warnings."** Return `remaining` and warn the client at 80%.
- **"Rate limit by IP when not logged in."** The key is just a string; the rule resolver decides which key to use.
- **"Where does the limiter run?"** In an API gateway or a servlet filter / Spring `HandlerInterceptor` (Java web hooks that run before every request) before the controller (the class that handles a given URL).

Real-world example

Stripe has written publicly about using several limiters together ("Scaling your API with rate limiters": a request-rate limiter based on the token bucket, a concurrent-requests limiter, and load shedders — components that deliberately drop low-priority work when overloaded — implemented with Redis) — several independent checks on one request, the composite idea. **GitHub's API** returns `X-RateLimit-Limit`, `X-RateLimit-Remaining` and `X-RateLimit-Reset` headers, which is our `Decision`. HTTP 429 is defined in RFC 6585 (RFCs are the official documents that define internet standards). **NGINX** (a widely used web server and reverse proxy) `limit_req` uses the leaky-bucket method. In Java, **Guava's** `RateLimiter`, **Resilience4j's** `RateLimiter` and **Bucket4j** (token bucket) are common libraries. Cloudflare has described using the sliding-window-counter approximation (weighting the previous window) to rate limit at the edge (its servers closest to users) cheaply.

In the interview

> **30-second answer:** "A `Limiter` interface with interchangeable algorithms — token bucket by default, sliding window when exactness matters. A factory builds limiters from rules; rules are chosen by API and user tier; a composite limiter combines rules like '10 per second and 1000 per day'. The service keeps one limiter per (user, api) in a `ConcurrentHashMap` via `computeIfAbsent`, and returns a decision with remaining and retry-after for the 429 response. Each limiter makes refill-check-take atomic; for multiple servers, move the state into Redis with an atomic script."

Plan (30–40 min): clarify key/limits (4) → algorithms table and choice (8) → OO design: interface, factory, service, composite (10) → thread safety (5, point to §13.11) → distributed + memory (5).

Follow-ups with answers:

- *"Why is fixed window risky?"* → Up to 2N requests across a window boundary (shown in the output).
- *"Token bucket vs sliding log?"* → Bucket allows bursts, O(1) memory; log is exact but stores every timestamp.
- *"How do you avoid creating two limiters for the same user?"* → `ConcurrentHashMap.computeIfAbsent`.

Self-check

1. Which algorithm allows a burst up to its capacity and then a steady rate? → Token bucket.
2. How many requests can a fixed window of limit 5 let through in 0.2 s across a boundary? → Up to 10.
3. Which pattern lets "2 per 10 s AND 3 per minute" look like one limiter? → Composite.
4. What HTTP status and header go with a denial? → 429 and `Retry-After`.
5. Why must refill + check + take be atomic? → Otherwise two threads can both see the last token and both take it.
6. How do you keep memory bounded with millions of users? → Evict limiters idle for at least the longest window (expire-after-access cache); by then a fresh limiter behaves exactly like the old one.
7. Why does `CompositeLimiter` probe every child before committing to any? → So a request denied by one rule does not use up a slot in another rule.

TL;DR

> - **Strategy** for algorithms, **Simple Factory** from rules, **Composite** for combined limits (probe all, then commit all), **Facade** service.
> - One limiter per (user, api) via `computeIfAbsent`; `Decision` → 200 (HTTP "OK") or 429 + Retry-After.

- Token bucket (bursty, O(1)), fixed window (2× edge burst), sliding log (exact, O(N)), sliding counter (approximate, O(1)).
- Atomic refill-check-take (§13.11); Redis + Lua for distributed limits.

20.8 Chapter summary

Component	Anchor pattern(s)	The one sentence to remember
Splitwise	Strategy	Split rules return shares; the ledger stores one signed number per pair; simplify with net positions + greedy (≤ n − 1 payments).
Ledger Co. (full case in Ch 21 §21.6)	—	A loan ledger, not a bill-splitting ledger: same "one authoritative record + `BigDecimal`" idea, different rules.
Notification service	Template Method + Strategy + Observer	Retry once in the parent, one class per reason not to send, one exit that tells the listeners.
In-memory cache	Strategy + Observer	HashMap + doubly linked list; eviction policy owns the order; one removal path.
LRU cache	— (data structure)	Map to node, move to front on use, evict before tail; or `LinkedHashMap(accessOrder=true)`.
Ride sharing	Strategy + Decorator + state machine	Ranked list for declines, surge wraps pricing, atomic driver claim.
Carpool variant	Strategy	Offer seats per vehicle; select with Most Vacant / Preferred Vehicle.
Logging framework	Chain of Responsibility + Singleton + Decorator	Hierarchy with inherited levels; propagate up until additivity stops; async is a wrapper.
Pub-sub	Observer via broker	Append-only log + per-subscription offsets; ack after processing; dead letters.
Rate limiter	Strategy + Factory + Composite	One limiter per key; algorithms swappable; 429 with Retry-After.

The common thread. In every design, the part most likely to change sits behind an interface, and the "service" class only orders the steps. When the interviewer says "now add X", the best answer is a new class and a one-line change where objects are wired together.

20.9 Chapter self-check

1. Which two patterns appear in more than half of this chapter's designs? → Strategy and Observer.
2. Strategy vs Decorator in one line each? → Strategy *replaces* a rule; Decorator *wraps* one and adds to its result.
3. Strategy vs Template Method? → Strategy swaps the whole algorithm (composition); Template Method fixes the algorithm and lets subclasses fill steps (inheritance).
4. Which designs used a "single exit" method, and why? → Notification (`finish`) and cache (`remove`): so records, stats and listeners can never be skipped on one path.
5. Where did we call listeners outside a lock, and why? → The cache: a slow or re-entrant listener must not block or deadlock it.
6. What does at-least-once delivery require from consumers? → Idempotency.
7. Name three places this chapter used compare-and-set or `computeIfAbsent` for correctness. → Driver claim (`AtomicBoolean`), subscription commit (`AtomicLong.compareAndSet`), limiter creation (`computeIfAbsent`).

20.10 Chapter TL;DR

- Components are small, so interviews probe the **rule most likely to change** — put it behind an interface.

- **Strategy** everywhere (splits, eviction, matching, pricing, send rules, formatters, rate-limit algorithms); **Observer** for "tell someone when it happens"; **Decorator** to add behaviour without editing (surge, async); **Template Method** for a shared algorithm with one varying step; **Chain of Responsibility** for the logger hierarchy; **Composite** for combined limits.
- Money in `BigDecimal`; time from an injected clock; thread safety with the smallest correct tool (one lock, CAS, `computeIfAbsent`, per-key locks, sharding).
- Always volunteer the honest limits: greedy is not optimal, LRU needs a write lock, at-least-once duplicates, fixed windows burst at edges.

Chapter 21 — Case Studies: Machine-Coding Round Problems

> **Goal.** Chapters 14–20 taught you to *design* a system on a whiteboard. This chapter trains the harder cousin of that interview: the **machine-coding round**, where you get a written problem statement, 90–120 minutes, and must hand over **running code** that the interviewer will feed their own inputs into. You will learn how the round is scored, a project skeleton you can type from memory, a minute-by-minute time plan, and then eight real problems — each solved end to end with a working, demo-able (ready to run in front of the interviewer) Java program.

How each problem is laid out: *Problem & clarifying questions → Requirements → Use cases → Core entities → Class diagram → Key flows → Design decisions & patterns → Java implementation (+ real output) → Concurrency & edge cases → Extensions → In the interview → Self-check → TL;DR.*

All code is Java 21, standard library only (only the classes that ship with Java, no extra downloads), one package (Java's folder-like grouping of related classes) per problem under `code/lld/ch21/<name>/`. Run from `chapters/21-case-machine-coding/code`:

```
javac --release 21 -d /tmp/lld_21 $(find . -name '*.java')
java -cp /tmp/lld_21 lld.ch21.taskmgmt.Main       # or wallet, calendar, codingblox, propertyhunt, ledger, inventory, filesearch
```

Every demo uses a fixed clock, a seeded random generator (a random-number generator started from a fixed "seed" number, so it produces the same sequence on every run), or sorted output, so what you see below is exactly what you get on every run.

Problem map

#	Problem	Where it is asked (source)	The one idea that makes it work	Patterns
21.1	Task Management System	awesome-low-level-design	Status *state machine* + composable filters	State (enum), Specification, Strategy, Observer
21.2	Digital Wallet Service	awesome-low-level-design	`BigDecimal` money, idempotency keys, lock ordering	Strategy (offers), Facade
21.3	Event Calendar	Flipkart (kumaransg/LLD)	Half-open intervals + a boundary *sweep* to find free slots	Strategy (representative choice), Repository
21.4	Coding Blox (LeetCode-like platform)	Flipkart (kumaransg/LLD)	Command dispatcher + level-based scoring rule	Command, Strategy, Repository
21.5	PropertyHunt	ClearTrip (kumaransg/LLD)	Optional search criteria → list of predicates; unit normalisation	Builder, Specification, Strategy
21.6	Ledger Co (loans & EMIs)	Navi (kumaransg/LLD, Geektrust)	Derive balances from facts instead of storing them	Strategy (interest), Command
21.7	Inventory Management System	Common (Amazon/Flipkart-style)	Append-only movement ledger + per-SKU lock	Strategy (allocation), Observer (reorder)
21.8	Unix File Search (`find`)	Common OOD/machine-coding question	Filters as Specifications combined by Composite AND/OR/NOT; Builder for criteria; a parser for `find` syntax	Specification, Composite (×2), Builder, Interpreter-style parser

(Food ordering "FoodKart", another kumaransg/LLD problem, is solved in Chapter 19.)

> **Key terms — the sources and patterns in the map.** *awesome-low-level-design* and *kumaransg/LLD* are public GitHub repositories that collect low-level design (LLD) problems; Geektrust is a hiring platform that sets such coding challenges. The patterns, one line each: a **state machine** is a fixed set of states plus the moves allowed between them (here kept in an enum, a Java type with a fixed list of

named values); **Specification** wraps one yes/no rule in an object so rules can be combined with AND / OR / NOT; **Strategy** puts each interchangeable rule in its own class behind one interface (a list of methods with no code, which several classes implement in their own way); **Observer** lets objects sign up to be told when something happens; **Facade** is one simple front class over several others; **Repository** hides how objects are stored behind save / find methods; **Command** turns each action into an object or handler that can be looked up by name; **Builder** assembles an object with many optional settings step by step; **Composite** lets a group of objects be treated like a single one. A few other words from the table: **idempotency keys** are explained in 21.2; **lock ordering** means always taking several locks in the same fixed order, so two threads can never block each other forever (a **thread** is one independent line of execution inside a running program; a **lock** is an object only one thread can hold at a time, while others wait); a **half-open interval** `[start, end)` includes its start but not its end; a **sweep** walks through sorted time points one by one; **unit normalisation** converts every input to one unit (e.g. square feet) as soon as it is read; an **SKU** (stock keeping unit) is the id of one sellable item; **OOD** is object-oriented design; an **Interpreter-style parser** turns text written in a small language into a tree of objects.

21.0 How the machine-coding round works

What it is

A **machine-coding round** (also called an *LLD coding round*) is an object-oriented design (designing a program as classes of objects that hold data and behaviour together) interview in which **working code is the answer**. The same round is called *object-oriented design* or *low-level design* at other companies; machine coding simply adds the rule that the code must compile and run at the end. You get a one- or two-page problem statement (usually a PDF), a laptop with your own IDE (integrated development environment: the code editor plus compiler and debugger, such as IntelliJ IDEA), and 90–120 minutes. Internet use is limited to looking up syntax. There is **no database, no UI (user interface: screens) and no web framework (a library for building websites and web APIs — an API is the set of calls a program or service offers to other programs)**: all data lives in memory (maps and lists), and the program is driven by a `main` method, a list of commands, a file, or unit tests (small automated checks that call one piece of code and compare the result with what is expected).

Do not confuse it with the **system design** interview. That round is about keeping a product alive under millions of users, so the conversation is about servers, databases, caches and message queues (see System Design Fundamentals if those words are new). This round never leaves a single program: its vocabulary is **classes** (a *class* is a named code type that bundles some data with the methods that act on that data), their fields and methods, and how the classes relate. Same product, very different zoom level — and practising for one leaves you unready for the other.

Everyday analogy: a whiteboard design interview is an architect *describing* a house. A machine-coding round is being asked to *build a small shed* while the inspector watches — then the inspector leans on every wall.

The format is especially common at Indian product companies such as Flipkart, Swiggy, PhonePe, Navi, Razorpay, CRED, Meesho and Cleartrip; several of this chapter's problems are statements candidates have shared from those rounds.

How it is judged

Interviewers use a rubric (a scoring checklist). The exact wording differs by company, but it always covers the same things. Read the "Expectations" section of any statement in this chapter (Coding Blox, PropertyHunt and Event Calendar all have one) and you will see these repeated almost word for word.

#	Criterion	What the interviewer checks	How to score well
1	**Working, demo-able code**	Does it compile? Does the driver (the small piece of code that feeds input into the program and prints the results) run the sample input and print the expected output? Does it survive *their* inputs?	Get a thin end-to-end path working in the first hour. Never finish with code that does not compile.
2	**Functional completeness**	Every *mandatory* requirement done; bonus only after that.	Implement requirements in the statement's order; tick them off.
3	**Clean models**	Entities have one job, fields are private, invariants (rules that must always be true, such as "balance is never negative") live inside the entity (for example a task refuses an illegal status change).	Use enums for fixed sets, records (Java's compact, unchangeable data classes) for values, classes for things that change.
4	**Service interfaces / separation of concerns** (each class handles one kind of work)	Parsing, business logic (the product's actual rules) and storage are in different classes. No `System.out` inside services.	Driver → Service → Repository layers (skeleton below).
5	**Extensibility**	"Add a new offer / sort order / scoring rule" should mean *adding a class*, not editing `if` chains.	Strategy and Specification for things the statement says "can be X or Y".
6	**Edge cases & failure handling**	Unknown IDs, duplicates, invalid input, wrong user, double-run. The program prints a clear error and **keeps going**.	Custom exceptions (your own error classes; an exception is how Java signals an error) with good messages; the driver catches them per command.
7	**Readability**	Intuitive names, small methods, no copy-paste.	Name classes after nouns in the statement; name methods after verbs in it.
8	**Concurrency (when asked)**	Wallets, inventory, booking: can two threads corrupt state?	Say where the lock is and why; demo it with a thread pool (a fixed set of reusable threads) if time allows.
9	**Tests (bonus)**	Some companies give extra credit for unit tests.	A tiny `assert`-style self-check in `main` is enough if there is no JUnit (Java's standard unit-testing library).

The five-step method, and how the clock changes it

The whiteboard version of this round (Chapter 1) always follows five steps in a fixed order:

1. **Requirements** — turn the one-line prompt into a short written list of what the system must do.
2. **Actors and use cases** — name *who* uses the system (actors) and *what* each of them does with it (use cases).
3. **Classes** — derive classes from the requirements and give each one responsibility.
4. **Relationships** — decide how each pair of classes is connected and draw it as a class diagram.
5. **Code** — implement the key classes, plus the two or three methods that make the main flows work.

Getting the order right beats going fast: each step is the input of the next, so if you skip requirements, you end up inventing classes nobody asked for. Every problem in this chapter is laid out in that same order.

A whiteboard round usually lasts 45 minutes to an hour. A budget for the 45-minute version:

Step	Minutes	Finished artefact
Requirements	5–8	8–12 written requirements
Actors & use cases	3–5	2–4 actors, 6–10 use cases
Classes	8–10	Class list, one responsibility each
Relationships	8–10	Class diagram with relationships and multiplicity (how many objects sit on each side, e.g. one-to-many)
Code	12–15	2–3 methods for the main flows

Two things in that table catch people off guard. The first two steps — which name no class at all — may take as much as a quarter of the time. And coding gets the biggest single slot yet still covers only two or three methods: nobody finishes the whole product, and you are graded on the slice you chose to build well, not on how much ground you covered. **Machine coding keeps the five steps but flips the proportions:** steps 1–4 shrink to about 20 minutes (see the time plan below) and code fills the rest, because now every mandatory feature must actually run.

How big is a complete answer? For a parking lot, roughly twelve requirements lead to about twelve classes joined by some fifteen relationships. Only three of them carry the main flows — issuing a ticket, holding a vehicle, pricing the stay. On a whiteboard you write those three properly and describe each of the other nine in one sentence. The costliest mistake is detailing all twelve equally: it looks thorough but leaves no time to demonstrate one complete flow. In machine coding the same instinct applies — make one thin path run end to end first, then widen.

Beyond "does it run", the interviewer is watching five things:

- **Scope** — did you pin down a bounded list of what you will build? A well-bounded design outranks a sprawling, open-ended one.
- **One job per class** — can you describe every class in a single sentence? A class that needs two sentences is usually two classes.
- **Relationships** — does a class *hold* another, *own* it, or *extend* it? These are different designs, and picking the wrong one is spotted immediately.
- **Code and model agree** — if the code contains a method your model has no place for, either the model or the code needs fixing.
- **A trade-off** — the option you took *and* the one you turned down, both said out loud.

Notice what is missing: nobody grades your UML notation (UML: the standard box-and-arrow notation for class diagrams) or counts how many diagram types you drew. The model is what gets marked; a diagram is just one way of displaying it.

The single most common failure is **running out of time with nothing that runs**. The second is a "god class" (a class that does everything): one `Manager` with every map and every method. Both are avoidable with the skeleton and time plan below.

A reusable project skeleton

Type this structure from memory in the first five minutes. It is the same for every problem, which is exactly why it saves time.

```
src/
  model/          entities & value objects      (Task, Wallet, Loan, Property ...)   - enums, records, small classes
  repository/     storage contracts + in-memory (TaskRepository, InMemoryTaskRepository)
  service/        business rules, one per area  (TaskService, WalletService)          - no printing, no parsing
  strategy/       pluggable rules               (Offer, AllocationStrategy, SortBy)
  exception/      domain exceptions             (NotFoundException, InsufficientFundsException)
  driver/         Main + CommandProcessor       (parses input, calls services, prints output)
```

Layer	Knows about	Must NOT know about	Why
model	nothing but itself (and other models)	services, storage, console	Entities guard their own invariants (e.g. `Task.changeStatus` rejects `TODO → DONE`).
repository	model	services, console	Swapping `HashMap` for a DB (database) later touches only this layer (Dependency Inversion — services depend on interfaces, not on concrete classes; Chapter 4).
service	model, repository *interfaces*, strategy *interfaces*	concrete repositories (injected — handed in from outside rather than created inside), console	This is the part the interviewer reads most carefully.
strategy	model	services	Each "X or Y" rule in the statement becomes one interface + small classes (Open/Closed).
exception	—	—	One class per failure *kind*, with a message a user could read.
driver	services	repositories' internals	Parsing and printing are replaceable; business logic is not.

In this book each problem lives in **one Java package** (for example `lld.ch21.wallet`) so the full listing stays short and readable. In the real round, create the sub-packages above — it costs nothing and the interviewer notices. Three reusable building blocks appear again and again:

1. **A generic in-memory repository** — a `ConcurrentHashMap<K, V>` (a map that many threads can use at once; `K, V` are type parameters, so the same class works for any key and value type — that is what *generic* means) behind an interface with `save / findById / findAll / delete` (see `Repository<K,V>` in Coding Blox, 21.4).
2. **An ID generator** — an `AtomicInteger` (an integer threads can increment safely without a lock) per entity type (`"T" + seq.incrementAndGet()`), thread-safe and predictable, which matches statements that say "auto-increment id starting with 1".
3. **A command processor** — a `Map<String, Consumer<String[]>>` from command name to handler (`Consumer` is a function that takes an argument and returns nothing; a *handler* is the code that deals with one command), fed by a built-in `List<String>` of sample commands (Coding Blox, PropertyHunt, Ledger). Adding a command = adding one map entry (Command pattern, Chapter 10).

The 90-minute time plan

Minutes	Activity	Output you should have
0–10	Read the statement twice. Ask clarifying questions. Mark mandatory vs bonus. Write the sample input as your driver's input list.	A numbered requirement list, a list of questions answered.
10–20	Nouns → entities, verbs → service methods. Pick the 1–2 patterns the statement is begging for ("can be sorted by X or Y" → Strategy).	Skeleton packages created, enums and records typed.
20–60	Implement mandatory requirements **in order**, running the driver after each one.	Sample input produces the sample output.
60–75	Edge cases and validation: unknown ids, duplicates, wrong owner, invalid numbers. Custom exceptions.	Driver prints errors and continues.
75–85	Bonus features (only if all mandatory ones work). Thread-safety where money or stock is involved.	Bonus commands in the driver.
85–90	Clean up names, delete dead code (code that is never run), run everything one last time. Prepare a 2-minute walkthrough.	Code that compiles, a demo, a story.

A good rule: **run the program at least every 15 minutes**. A compile error found at minute 88 is a disaster; the same error found at minute 30 is a 20-second fix.

What to say in the first two minutes

1. "I'll keep everything in memory with a repository interface, so a DB can be added later without touching services."
2. "I'll drive it from `main` with the sample commands from the statement, and print errors without crashing."
3. "The mandatory list is A, B, C; the bonus is D, E. I'll do A–C first."
4. "Where does it need to be thread-safe?" (Ask. For wallets and inventory the answer is always yes.)

Common mistakes in this round

Mistake	Why it hurts	Do this instead
Starting with a perfect class diagram for 20 minutes	No time left for code	Enums/records first; the diagram lives in your head and in names
One `Manager` class with 15 maps	Fails "separation of concerns"	One service per area + repositories
`System.out.println` inside the service	Service cannot be reused or tested	Services return data; the driver prints
`double` for money	`0.1 + 0.2 != 0.3` — interviewers test this	`BigDecimal` (Java's exact decimal number type) with a fixed scale (number of digits after the decimal point; 21.2) or whole units (`long` paise; a paisa is 1/100 of a rupee)
Crashing on the first bad command	"Fails gracefully" is an explicit criterion	Catch domain exceptions (errors that describe a broken business rule) in the driver loop, print, continue
Implementing bonus before mandatory	Partial credit on the mandatory part	Mandatory in order; bonus last
Writing every class in full detail	Nothing works end to end when time runs out	Make the core flow run first; stub (write an empty placeholder for) or name the rest
`Random` without a seed	Output changes every run; hard to demo	Seed it, or put randomness behind a Strategy (21.4)
Hard-coding "today" with `LocalDate.now()`	Demo changes daily	Inject a `java.time.Clock` (Java's replaceable source of the current time; 21.1, 21.2)

Real-world examples

The skeleton above is the standard layering of real Java back ends (the server-side part of an application): in a Spring Boot (the most popular Java framework for building services) service, controllers (classes that receive web requests) play the "driver" role, `@Service` classes hold business rules, and `@Repository` interfaces (Spring Data) hide storage. (The `@Something` markers are annotations: labels a framework reads to know what a class is for.) The in-memory repository you write in the round is exactly the kind of fake that teams use in unit tests for those services.

In the interview

30-second answer — "How do you approach a machine-coding problem?" "I read the statement twice, list mandatory and bonus requirements and ask about anything ambiguous. I set up model / repository / service / strategy / exception / driver packages, put the sample input into the driver, and implement requirements one by one, running after each. Services return data and throw domain exceptions; the driver prints and keeps going. I finish with edge cases, thread-safety where state is shared, and bonus features if time allows."

Follow-ups

- *"Why an interface for an in-memory repository?"* — So services depend on an abstraction (an interface rather than a concrete class); a JDBC (Java's standard API for SQL databases) or Redis (a fast in-

memory key-value store) implementation can replace it with no change to business code, and tests can use the in-memory one.

- *"How do you make random behaviour testable?"* — Put it behind an interface (a Strategy) and inject a seeded or scripted implementation for demos and tests.
- *"How much do you test?"* — At minimum, the driver replays the statement's samples and prints results I can compare with the expected output. With time, a few JUnit tests on the service layer.

Self-check

- **Q:** Name the six packages of the skeleton. → **A:** model, repository, service, strategy, exception, driver.
- **Q:** Where should `System.out.println` live? → **A:** Only in the driver (command processor / `Main`).
- **Q:** What do you do at minute 60 if the bonus is tempting but one mandatory requirement is missing? → **A:** Finish the mandatory one; bonus only after all mandatory ones run.
- **Q:** Why inject a `Clock`? → **A:** So time-based logic (due dates, reminders) gives the same output on every run and can be tested.

TL;DR

Working code first, clean layering second, patterns where the statement asks for variation, graceful errors everywhere. Skeleton: **model → repository → service → strategy → exception → driver**. Run often.

21.1 Design a Task Management System

Problem & clarifying questions

Build the core of a tool like Jira, Asana or Trello (popular team task-tracking apps): users create tasks, assign them to people, move them through statuses, comment on them, and search/filter them; the system reminds people about tasks that are due. The awesome-low-level-design version asks for create / update / delete, title + description + due date + priority + status, assignment, reminders, search and filtering (by priority, due date, assignee), marking tasks complete, task history, and safe concurrent access (several users acting at the same moment without corrupting data).

Questions to ask (with the answers this solution assumes):

Question	Assumed answer
Which statuses exist, and can any status go to any other?	TODO, IN_PROGRESS, BLOCKED, DONE, CANCELLED. Only listed transitions are legal (e.g. TODO cannot jump straight to DONE; CANCELLED is final; DONE can be re-opened).
One assignee or many?	One assignee (nullable: the field may be `null`, Java's "no value"). A list is an easy extension.
What does "reminder" mean in a console app?	A method a scheduler would call: "notify owners of open tasks due within N days or overdue".
Should due dates in the past be allowed?	No, not on creation.
How are notifications delivered?	Through a listener interface; the demo prints them.
"View task history" — history of what?	Two readings: each task's audit log (a permanent record of who changed what, when) — built here — and a user's list of finished tasks, which is just a search: `assignedTo(u).and(withStatus(DONE))`.
"Set reminders" — per task, or a sweep?	A sweep over due dates is built here; a per-task reminder time is an extension (below).

Requirements

Functional

1. Register users.

2. Create a task (title, description, priority, due date); update title / priority / due date; delete a task.
3. Assign / reassign a task to a user.
4. Change status following the allowed transitions; mark done.
5. Comment on a task.
6. Search with any combination of filters (assignee, status, minimum priority, due before, open only, title contains) and a sort order (due date, priority, id).
7. Reminders for open tasks due soon or overdue.
8. Per-task activity history.

Non-functional

- Thread-safe: two people editing the same task must not corrupt it.
- Extensible: new filters, sort orders and notification channels without editing existing code.
- Deterministic demo (injected `Clock`).

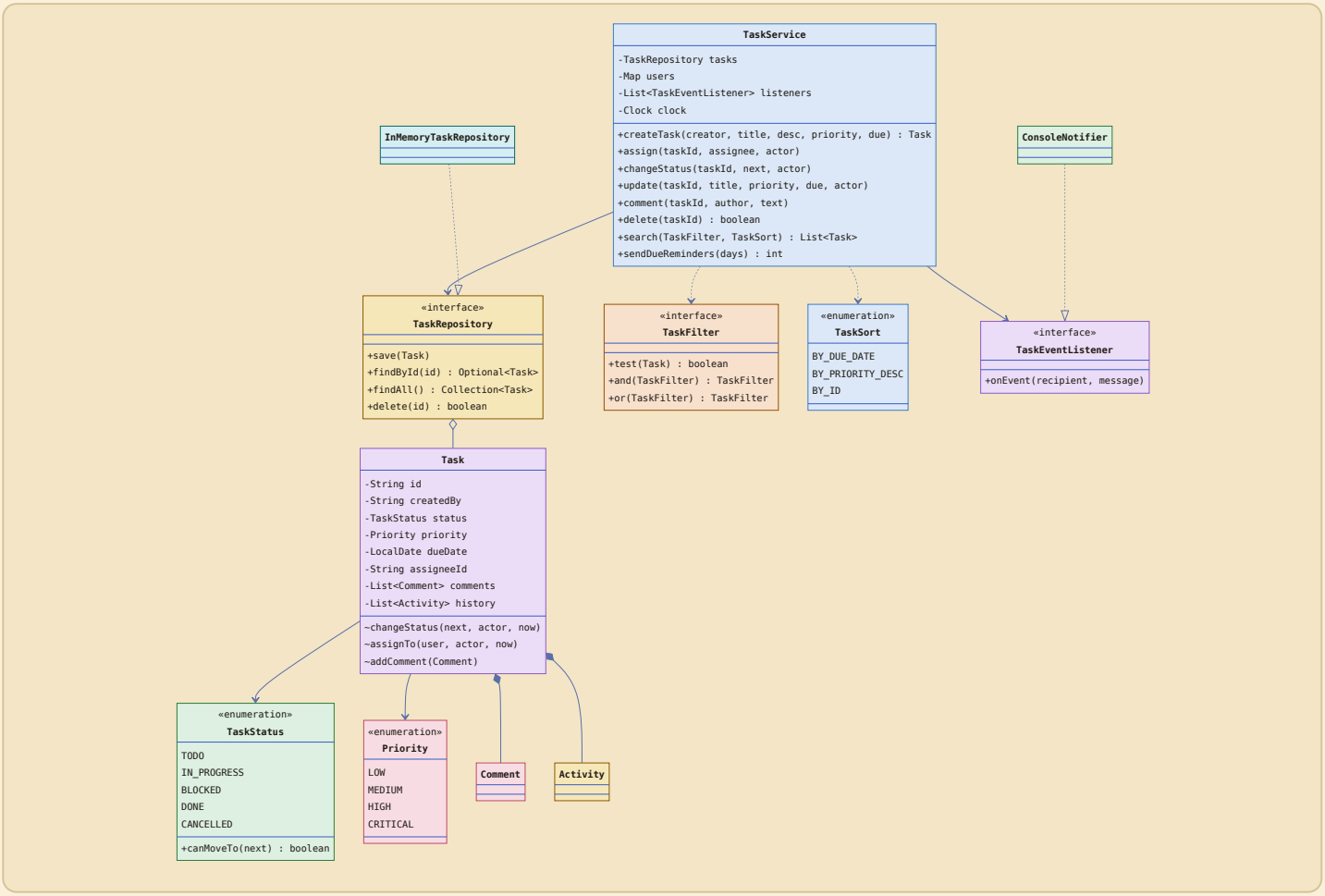
Use cases

Actor	Use cases
Task creator (any user)	create task, update, delete, assign, comment, search
Assignee	change status, comment, see own tasks
Scheduler (system actor)	send due-date reminders
Notification channel (external)	receives events (assigned, status changed, commented, reminder)

Core entities

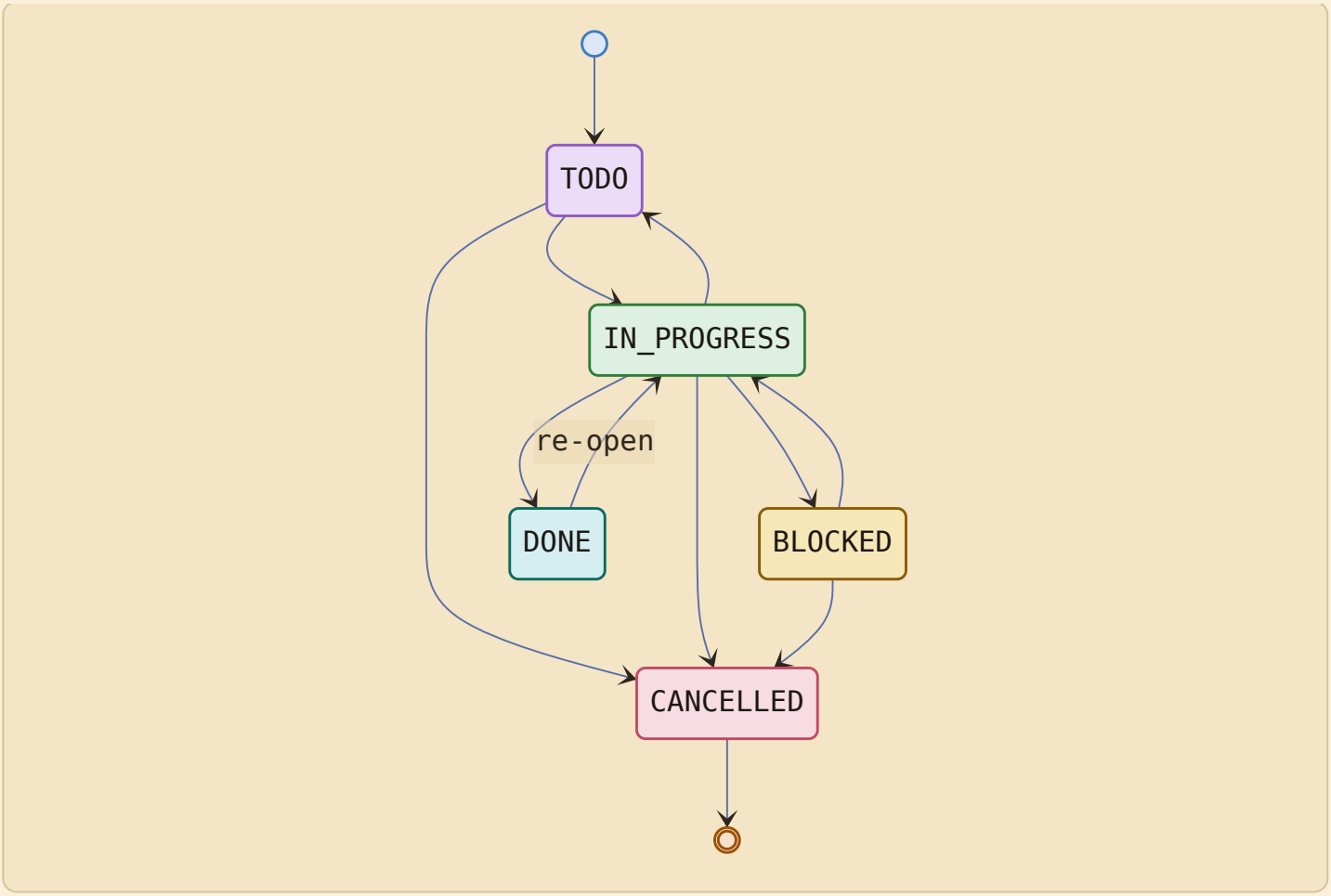
Class	Kind	Responsibility
`Task`	class	Holds task data; enforces legal status changes; records its own history. All mutators (methods that change state) `synchronized` (only one thread at a time may run them on the same object).
`TaskStatus`	enum	Lists statuses and, for each, which statuses it may move to (a tiny state machine).
`Priority`	enum	LOW < MEDIUM < HIGH < CRITICAL (enum order is used for comparison).
`User`, `Comment`, `Activity`	records	Immutable (cannot change after creation) values.
`TaskRepository` / `InMemoryTaskRepository`	interface / class	Storage contract and a `ConcurrentHashMap` implementation.
`TaskFilter`	functional interface (an interface with one method, so a short lambda function can implement it)	Composable predicate (a yes/no test; Specification pattern) with factory methods (static methods that create objects) and `and` / `or`.
`TaskSort`	enum	Named comparators (objects that decide which of two items sorts first; Strategy). `BY_ID` compares the number inside the id, so `T2` sorts before `T10`.
`TaskEventListener` / `ConsoleNotifier`	interface / class	Observer for notifications.
`TaskService`	class	Facade with every use case; validates input, uses the clock, fires events.
`NotFoundException`, `InvalidTransitionException`	exceptions	Clear failure messages.

Class diagram

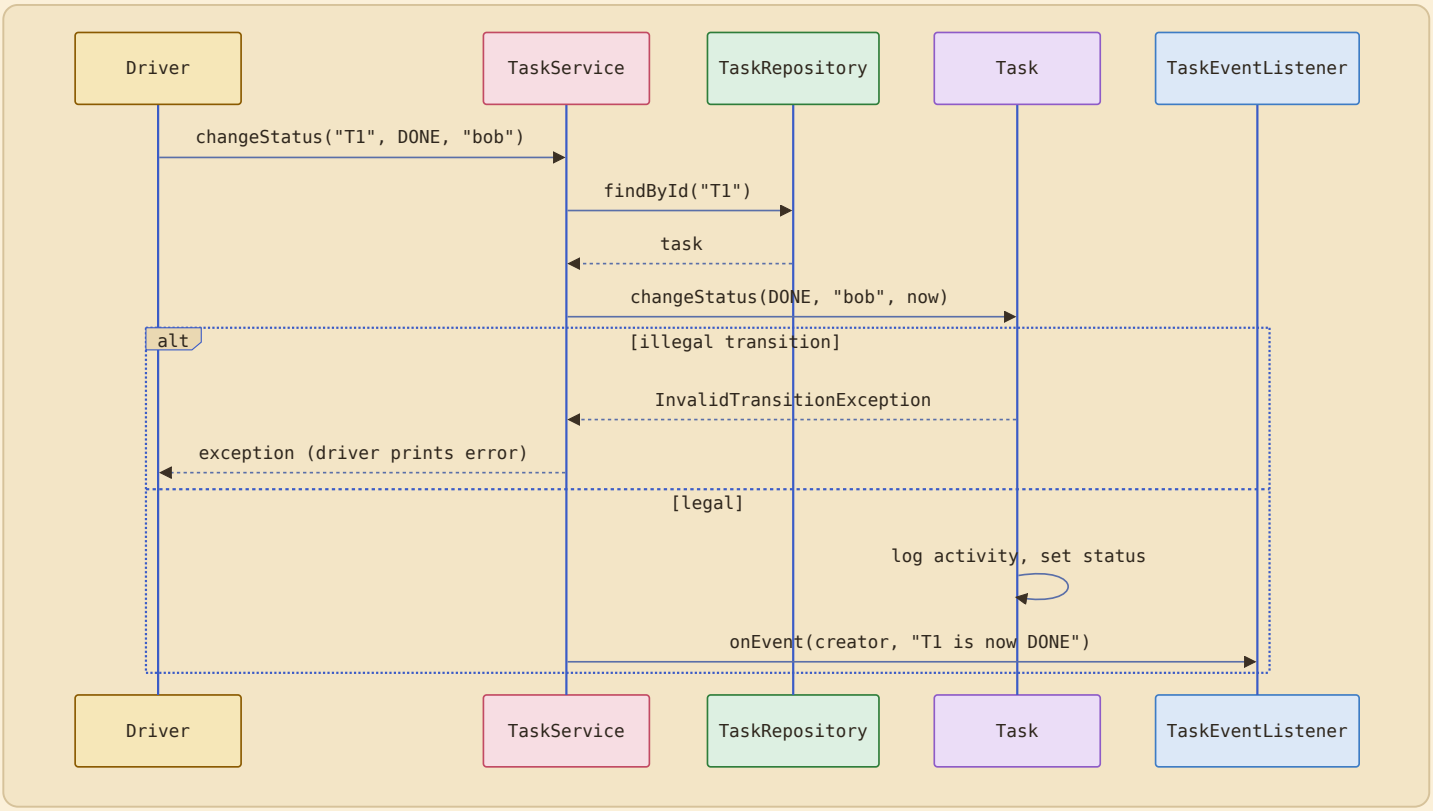


Key flows

Task status life cycle — the rules live in `TaskStatus.allowedNext()`:



Change status and notify:



Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Transitions stored in the `TaskStatus` enum	State (enum-based)	All rules in one place; `Task` just asks `canMoveTo`.	Full State pattern (one class per state, each holding that state's behaviour) with one class per status — overkill when states carry no behaviour besides transitions.
`TaskFilter extends Predicate<Task>` with `and/or`	Specification	Any combination of filters without a method per combination; a new filter is one factory method.	`search(String assignee, Status s, Priority p, ...)` with many nullable parameters — grows forever.
`TaskSort` enum of comparators	Strategy	Sort order is data the caller picks.	`switch` on a sort string inside the service.
`TaskEventListener` list	Observer	Email/Slack/push can be added without touching the service.	Calling `System.out` from the service.
`TaskService` as the only entry point	Facade	Driver sees one simple API.	Driver talking to repositories directly.
`Clock` injected	Dependency injection (a class receives the objects it needs from outside instead of creating them)	Deterministic reminders and tests.	`LocalDate.now()` scattered in code.
`synchronized` on `Task` mutators	Monitor per entity (each object's own lock guards its methods)	Fine-grained (many small locks instead of one big one): two different tasks never block each other.	One global lock on the service — simpler but serialises everything (forces all work to run one at a time).

Java implementation

```java
// File: lld/ch21/taskmgmt/Priority.java
package lld.ch21.taskmgmt;

/** Ordered from least to most urgent, so natural enum order can be used for sorting. */
public enum Priority { LOW, MEDIUM, HIGH, CRITICAL }
```

```java
// File: lld/ch21/taskmgmt/TaskStatus.java
package lld.ch21.taskmgmt;

import java.util.EnumSet;
import java.util.Set;

/** Task lifecycle. Each status knows which statuses it may move to (a tiny state machine). */
public enum TaskStatus {
    TODO, IN_PROGRESS, BLOCKED, DONE, CANCELLED;

    public Set<TaskStatus> allowedNext() {
        return switch (this) {
            case TODO -> EnumSet.of(IN_PROGRESS, CANCELLED);
            case IN_PROGRESS -> EnumSet.of(BLOCKED, DONE, TODO, CANCELLED);
            case BLOCKED -> EnumSet.of(IN_PROGRESS, CANCELLED);
            case DONE -> EnumSet.of(IN_PROGRESS);          // re-open
            case CANCELLED -> EnumSet.noneOf(TaskStatus.class);
        };
    }

    public boolean canMoveTo(TaskStatus next) { return allowedNext().contains(next); }

    public boolean isOpen() { return this != DONE && this != CANCELLED; }
}
```

```java
// File: lld/ch21/taskmgmt/User.java
package lld.ch21.taskmgmt;

public record User(String id, String name) { }
```

```java
// File: lld/ch21/taskmgmt/Comment.java
package lld.ch21.taskmgmt;

import java.time.LocalDateTime;

public record Comment(String authorId, String text, LocalDateTime at) { }
```

```java
// File: lld/ch21/taskmgmt/Activity.java
package lld.ch21.taskmgmt;

import java.time.LocalDateTime;

/** One line of a task's audit history ("who did what, when"). */
public record Activity(LocalDateTime at, String actorId, String description) {
    @Override public String toString() { return at.toLocalTime() + " " + actorId + ": " + description; }
}
```

```java
// File: lld/ch21/taskmgmt/Task.java
package lld.ch21.taskmgmt;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

/**
 * The central entity. All mutators are synchronized so two threads editing
 * the same task cannot interleave (e.g. a status change racing a reassignment).
 */
public class Task {
    private final String id;
    private final String createdBy;
    private String title;
    private String description;
    private Priority priority;
    private LocalDate dueDate;
    private TaskStatus status = TaskStatus.TODO;
    private String assigneeId;                // null = unassigned
    private final List<Comment> comments = new ArrayList<>();
    private final List<Activity> history = new ArrayList<>();

    Task(String id, String createdBy, String title, String description, Priority priority, LocalDate dueDate) {
        this.id = id;
        this.createdBy = createdBy;
        this.title = title;
        this.description = description;
        this.priority = priority;
        this.dueDate = dueDate;
    }

    public String id() { return id; }
    public String createdBy() { return createdBy; }
    public synchronized String title() { return title; }
    public synchronized String description() { return description; }
    public synchronized Priority priority() { return priority; }
    public synchronized LocalDate dueDate() { return dueDate; }
    public synchronized TaskStatus status() { return status; }
    public synchronized String assigneeId() { return assigneeId; }
    public synchronized List<Comment> comments() { return List.copyOf(comments); }
    public synchronized List<Activity> history() { return List.copyOf(history); }

    synchronized void changeStatus(TaskStatus next, String actor, LocalDateTime now) {
        if (!status.canMoveTo(next)) throw new InvalidTransitionException(id, status, next);
        log(now, actor, "status " + status + " -> " + next);
        status = next;
    }

    synchronized void assignTo(String userId, String actor, LocalDateTime now) {
        if (!status.isOpen()) throw new IllegalStateException("Cannot assign a " + status + " task " + id);
        log(now, actor, "assigned to " + userId);
        assigneeId = userId;
    }

    synchronized void update(String newTitle, Priority newPriority, LocalDate newDue, String actor, LocalDateTime now)
{
        if (newTitle != null) { title = newTitle; }
        if (newPriority != null) { priority = newPriority; }
        if (newDue != null) { dueDate = newDue; }
        log(now, actor, "updated title/priority/due -> " + title + "/" + priority + "/" + dueDate);
    }

    synchronized void addComment(Comment c) {
        comments.add(c);
        log(c.at(), c.authorId(), "commented \"" + c.text() + "\"");
    }

    private void log(LocalDateTime at, String actor, String what) { history.add(new Activity(at, actor, what)); }

    @Override public synchronized String toString() {
        return "%s %-22s %-8s %-11s due=%s assignee=%s".formatted(id, title, priority, status, dueDate,
```

```
                assigneeId == null ? "-" : assigneeId);
    }
}
```

```
// File: lld/ch21/taskmgmt/InvalidTransitionException.java
package lld.ch21.taskmgmt;

public class InvalidTransitionException extends RuntimeException {
    public InvalidTransitionException(String taskId, TaskStatus from, TaskStatus to) {
        super("Task " + taskId + " cannot move from " + from + " to " + to);
    }
}
```

```
// File: lld/ch21/taskmgmt/NotFoundException.java
package lld.ch21.taskmgmt;

public class NotFoundException extends RuntimeException {
    public NotFoundException(String what, String id) { super(what + " not found: " + id); }
}
```

```
// File: lld/ch21/taskmgmt/TaskRepository.java
package lld.ch21.taskmgmt;

import java.util.Collection;
import java.util.Optional;

/** Storage contract. The service depends on this, not on a HashMap (Dependency Inversion). */
public interface TaskRepository {
    void save(Task task);
    Optional<Task> findById(String id);
    Collection<Task> findAll();
    boolean delete(String id);
}
```

```
// File: lld/ch21/taskmgmt/InMemoryTaskRepository.java
package lld.ch21.taskmgmt;

import java.util.Collection;
import java.util.List;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;

public class InMemoryTaskRepository implements TaskRepository {
    private final ConcurrentHashMap<String, Task> tasks = new ConcurrentHashMap<>();

    @Override public void save(Task task) { tasks.put(task.id(), task); }
    @Override public Optional<Task> findById(String id) { return Optional.ofNullable(tasks.get(id)); }
    @Override public Collection<Task> findAll() { return List.copyOf(tasks.values()); }
    @Override public boolean delete(String id) { return tasks.remove(id) != null; }
}
```

```java
// File: lld/ch21/taskmgmt/TaskFilter.java
package lld.ch21.taskmgmt;

import java.time.LocalDate;
import java.util.function.Predicate;

/**
 * Specification pattern: small, named predicates that combine with and()/or().
 * Adding a new filter = adding one factory method; the search code never changes (Open/Closed).
 */
@FunctionalInterface
public interface TaskFilter extends Predicate<Task> {

    static TaskFilter all() { return t -> true; }
    static TaskFilter assignedTo(String userId) { return t -> userId.equals(t.assigneeId()); }
    static TaskFilter withStatus(TaskStatus s) { return t -> t.status() == s; }
    static TaskFilter atLeast(Priority p) { return t -> t.priority().compareTo(p) >= 0; }
    static TaskFilter dueOnOrBefore(LocalDate d) { return t -> t.dueDate() != null && !t.dueDate().isAfter(d); }
    static TaskFilter open() { return t -> t.status().isOpen(); }
    static TaskFilter titleContains(String word) {
        return t -> t.title().toLowerCase().contains(word.toLowerCase());
    }

    default TaskFilter and(TaskFilter other) { return t -> test(t) && other.test(t); }
    default TaskFilter or(TaskFilter other) { return t -> test(t) || other.test(t); }
}
```

```java
// File: lld/ch21/taskmgmt/TaskSort.java
package lld.ch21.taskmgmt;

import java.time.LocalDate;
import java.util.Comparator;

/** Sorting strategies; each constant wraps a Comparator (Strategy pattern). */
public enum TaskSort {
    BY_DUE_DATE(Comparator.comparing(Task::dueDate, Comparator.nullsLast(Comparator.<LocalDate>naturalOrder()))),
    BY_PRIORITY_DESC(Comparator.comparing(Task::priority).reversed()),
    BY_ID(Comparator.comparingInt((Task t) -> Integer.parseInt(t.id().substring(1))));   // numeric: T2 before T10

    private final Comparator<Task> comparator;
    TaskSort(Comparator<Task> c) { this.comparator = c.thenComparing(Task::id); }
    public Comparator<Task> comparator() { return comparator; }
}
```

```java
// File: lld/ch21/taskmgmt/TaskEventListener.java
package lld.ch21.taskmgmt;

/** Observer: anything that wants to hear about task events (email, Slack, push...). */
public interface TaskEventListener {
    void onEvent(String recipientId, String message);
}
```

```java
// File: lld/ch21/taskmgmt/ConsoleNotifier.java
package lld.ch21.taskmgmt;

public class ConsoleNotifier implements TaskEventListener {
    @Override public void onEvent(String recipientId, String message) {
        System.out.println("  [notify " + recipientId + "] " + message);
    }
}
```

```java
// File: lld/ch21/taskmgmt/TaskService.java
package lld.ch21.taskmgmt;

import java.time.Clock;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicInteger;

/** Facade for every use case. Holds no storage details and no printing. */
public class TaskService {
    private final TaskRepository tasks;
    private final Map<String, User> users = new ConcurrentHashMap<>();
    private final List<TaskEventListener> listeners = new CopyOnWriteArrayList<>();
    private final AtomicInteger seq = new AtomicInteger();
    private final Clock clock;

    public TaskService(TaskRepository tasks, Clock clock) {
        this.tasks = tasks;
        this.clock = clock;
    }

    public void addListener(TaskEventListener l) { listeners.add(l); }

    public User addUser(String id, String name) {
        User u = new User(id, name);
        if (users.putIfAbsent(id, u) != null) throw new IllegalArgumentException("User exists: " + id);
        return u;
    }

    public Task createTask(String creatorId, String title, String description, Priority priority, LocalDate due) {
        requireUser(creatorId);
        if (title == null || title.isBlank()) throw new IllegalArgumentException("Title is required");
        if (due != null && due.isBefore(today())) throw new IllegalArgumentException("Due date is in the past: " +
due);
        Task t = new Task("T" + seq.incrementAndGet(), creatorId, title, description, priority, due);
        tasks.save(t);
        return t;
    }

    public void assign(String taskId, String assigneeId, String actorId) {
        requireUser(actorId);
        requireUser(assigneeId);
        Task t = get(taskId);
        t.assignTo(assigneeId, actorId, now());
        if (!assigneeId.equals(actorId)) fire(assigneeId, actorId + " assigned you " + t.id() + " '" + t.title() +
"'");
    }

    public void changeStatus(String taskId, TaskStatus next, String actorId) {
        requireUser(actorId);
        Task t = get(taskId);
        t.changeStatus(next, actorId, now());
        if (!t.createdBy().equals(actorId)) fire(t.createdBy(), t.id() + " is now " + next);
    }

    public void update(String taskId, String title, Priority priority, LocalDate due, String actorId) {
        requireUser(actorId);
        if (title != null && title.isBlank()) throw new IllegalArgumentException("Title cannot be blank");
        if (due != null && due.isBefore(today())) throw new IllegalArgumentException("Due date is in the past: " +
due);
        get(taskId).update(title, priority, due, actorId, now());
    }

    public void comment(String taskId, String authorId, String text) {
        requireUser(authorId);
        Task t = get(taskId);
        t.addComment(new Comment(authorId, text, now()));
        String other = authorId.equals(t.assigneeId()) ? t.createdBy() : t.assigneeId();
        if (other != null && !other.equals(authorId)) fire(other, authorId + " commented on " + t.id());
```

```
    }

    public boolean delete(String taskId) { return tasks.delete(taskId); }

    public List<Task> search(TaskFilter filter, TaskSort sort) {
        return tasks.findAll().stream().filter(filter).sorted(sort.comparator()).toList();
    }

    /** Reminder sweep: open tasks due within `days` days (or overdue). A scheduler would call this. */
    public int sendDueReminders(int days) {
        LocalDate limit = today().plusDays(days);
        List<Task> due = search(TaskFilter.open().and(TaskFilter.dueOnOrBefore(limit)), TaskSort.BY_DUE_DATE);
        for (Task t : due) {
            String who = t.assigneeId() != null ? t.assigneeId() : t.createdBy();
            String when = t.dueDate().isBefore(today()) ? "OVERDUE since " + t.dueDate() : "due " + t.dueDate();
            fire(who, "Reminder: " + t.id() + " '" + t.title() + "' is " + when);
        }
        return due.size();
    }

    public Task get(String taskId) {
        return tasks.findById(taskId).orElseThrow(() -> new NotFoundException("Task", taskId));
    }

    private void requireUser(String id) { if (!users.containsKey(id)) throw new NotFoundException("User", id); }
    private void fire(String to, String msg) { listeners.forEach(l -> l.onEvent(to, msg)); }
    private LocalDate today() { return LocalDate.now(clock); }
    private LocalDateTime now() { return LocalDateTime.now(clock); }
}
```

```java
// File: lld/ch21/taskmgmt/Main.java
package lld.ch21.taskmgmt;

import java.time.Clock;
import java.time.Instant;
import java.time.LocalDate;
import java.time.ZoneOffset;

public class Main {
    public static void main(String[] args) {
        // Fixed clock => the demo prints the same thing on every run.
        Clock clock = Clock.fixed(Instant.parse("2026-09-28T09:00:00Z"), ZoneOffset.UTC);
        LocalDate today = LocalDate.now(clock);
        TaskService svc = new TaskService(new InMemoryTaskRepository(), clock);
        svc.addListener(new ConsoleNotifier());

        svc.addUser("alice", "Alice");
        svc.addUser("bob", "Bob");
        svc.addUser("carol", "Carol");

        System.out.println("== create & assign");
        Task t1 = svc.createTask("alice", "Fix login bug", "NPE on empty password", Priority.CRITICAL,
today.plusDays(1));
        Task t2 = svc.createTask("alice", "Write release notes", "", Priority.LOW, today.plusDays(7));
        Task t3 = svc.createTask("bob", "Upgrade to Java 21", "", Priority.HIGH, today.plusDays(3));
        Task t4 = svc.createTask("carol", "Review PR #42", "", Priority.MEDIUM, today);
        svc.assign(t1.id(), "bob", "alice");
        svc.assign(t2.id(), "carol", "alice");
        svc.assign(t3.id(), "bob", "bob");

        System.out.println("== work on tasks");
        svc.changeStatus(t1.id(), TaskStatus.IN_PROGRESS, "bob");
        svc.comment(t1.id(), "bob", "Root cause found in PasswordValidator");
        svc.changeStatus(t1.id(), TaskStatus.DONE, "bob");
        svc.update(t3.id(), null, Priority.CRITICAL, null, "bob");

        System.out.println("== search: bob's open tasks, by priority");
        svc.search(TaskFilter.assignedTo("bob").and(TaskFilter.open()), TaskSort.BY_PRIORITY_DESC)
           .forEach(t -> System.out.println("  " + t));
        System.out.println("== search: priority >= MEDIUM due within 3 days, by due date");
        svc.search(TaskFilter.atLeast(Priority.MEDIUM).and(TaskFilter.dueOnOrBefore(today.plusDays(3))),
                   TaskSort.BY_DUE_DATE).forEach(t -> System.out.println("  " + t));

        System.out.println("== reminders (due within 3 days)");
        System.out.println("  reminders sent: " + svc.sendDueReminders(3));

        System.out.println("== history of " + t1.id());
        svc.get(t1.id()).history().forEach(a -> System.out.println("  " + a));

        System.out.println("== edge cases");
        attempt(() -> svc.changeStatus(t2.id(), TaskStatus.DONE, "carol"));   // TODO -> DONE skips IN_PROGRESS
        attempt(() -> svc.assign("T99", "bob", "alice"));
        attempt(() -> svc.assign(t2.id(), "dave", "alice"));
        attempt(() -> svc.createTask("alice", "Past task", "", Priority.LOW, today.minusDays(1)));
        svc.changeStatus(t4.id(), TaskStatus.CANCELLED, "carol");
        attempt(() -> svc.assign(t4.id(), "bob", "carol"));
        System.out.println("  deleted T2? " + svc.delete(t2.id()) + ", remaining=" + svc.search(TaskFilter.all(),
TaskSort.BY_ID).size());
    }

    private static void attempt(Runnable r) {
        try { r.run(); System.out.println("  ok"); }
        catch (RuntimeException e) { System.out.println("  error: " + e.getMessage()); }
    }
}
```

Output:

```
== create & assign
  [notify bob] alice assigned you T1 'Fix login bug'
  [notify carol] alice assigned you T2 'Write release notes'
== work on tasks
  [notify alice] T1 is now IN_PROGRESS
  [notify alice] bob commented on T1
  [notify alice] T1 is now DONE
== search: bob's open tasks, by priority
  T3 Upgrade to Java 21     CRITICAL TODO        due=2026-10-01 assignee=bob
== search: priority >= MEDIUM due within 3 days, by due date
  T4 Review PR #42          MEDIUM   TODO        due=2026-09-28 assignee=-
  T1 Fix login bug          CRITICAL DONE        due=2026-09-29 assignee=bob
  T3 Upgrade to Java 21     CRITICAL TODO        due=2026-10-01 assignee=bob
== reminders (due within 3 days)
  [notify carol] Reminder: T4 'Review PR #42' is due 2026-09-28
  [notify bob] Reminder: T3 'Upgrade to Java 21' is due 2026-10-01
  reminders sent: 2
== history of T1
  09:00 alice: assigned to bob
  09:00 bob: status TODO -> IN_PROGRESS
  09:00 bob: commented "Root cause found in PasswordValidator"
  09:00 bob: status IN_PROGRESS -> DONE
== edge cases
  error: Task T2 cannot move from TODO to DONE
  error: Task not found: T99
  error: User not found: dave
  error: Due date is in the past: 2026-09-27
  error: Cannot assign a CANCELLED task T4
  deleted T2? true, remaining=3
```

What to notice in the output: self-assignment (`bob` assigning T3 to himself) sends no notification; `T1` shows up in the priority/due search even though it is DONE (that search did not ask for `open()`), but it is **not** in the reminders, which only consider open tasks; the illegal `TODO → DONE` jump is rejected with a clear message; and the history of T1 is a complete audit trail.

Concurrency & edge cases

- **Two users update the same task at once.** Each mutator of `Task` is `synchronized`, so a status change and a reassignment cannot interleave (have their steps mixed together); getters (methods that only read a field) are synchronized too so readers see a consistent value.
- **Storage.** `ConcurrentHashMap` for tasks and users; `findAll()` returns a copy so a search is never disturbed by a concurrent insert.
- **Listeners** (objects that asked to be told about events) are in a `CopyOnWriteArrayList` (a list that copies itself on each write, so reading needs no lock) — registering a listener while events fire is safe.
- **IDs** from an `AtomicInteger` — no duplicates under concurrency.
- **Edge cases handled:** unknown task / user / actor, blank title, past due date (on create and on update), illegal transition, assigning a closed (DONE/CANCELLED) task, self-assignment (no noise), comment notifications go to "the other party".
- **Not handled (say so):** lost updates between two *editors of the text fields* (last writer wins). Fix: optimistic locking with a `version` field — reject an update whose version is stale (out of date). Optimistic locking takes no lock up front; it only checks, at save time, that nobody else changed the record in the meantime.

Extensions & follow-up questions

- **Sub-tasks / dependencies?** Add `parentId` and a `Set<String> blockedBy`; forbid DONE while a blocker is open (a check inside `changeStatus`).
- **Multiple assignees or watchers?** Replace `assigneeId` with `Set<String>`; notify all watchers through the same listener.
- **Recurring tasks?** A `RecurrenceRule` (daily/weekly) that creates the next task when one is marked DONE.
- **Custom workflows per project (like Jira)?** Move the transition table from the enum into a `Workflow` object (a `Map<Status, Set<Status>>`) injected per project.
- **A reminder time chosen by the user?** Store `remindAt` on the task; the scheduler's sweep fires every reminder whose time has passed and clears it.

- **Real reminders?** A `ScheduledExecutorService` (Java's tool for running tasks on a timer) calling `sendDueReminders(1)` every hour; store "already reminded" per task to avoid spam.
- **Search at scale?** Secondary indexes (extra lookup maps keyed by a field other than the id: `Map<String, Set<Task>>` by assignee, `TreeMap<LocalDate, Set<Task>>` by due date; a `TreeMap` keeps its keys sorted) or a search engine such as Elasticsearch (a popular open-source search server).

Real-world example

In Jira, each project has a *workflow*: a set of statuses and the transitions allowed between them, and a transition can fire notifications to watchers. Search is done with JQL, a query language that combines conditions like `assignee = currentUser() AND status != Done ORDER BY priority DESC` (DESC = descending) — the same "compose filters, then sort" idea as `TaskFilter` + `TaskSort`, just with a text syntax on top.

In the interview

Present in this order (30–45 min version): (1) statuses and transitions on the whiteboard — it shows you think about invariants; (2) `Task` and `TaskService` API; (3) filters as composable predicates — the pattern interviewers love here; (4) the observer for notifications; (5) run the demo. If asked "why not the full State pattern?", answer: statuses have no behaviour except which transitions are legal, so an enum with a transition table is simpler and still closed to illegal moves.

Self-check

- **Q:** Where is the rule "TODO cannot go to DONE" coded? → **A:** In `TaskStatus.allowedNext()`; `Task.changeStatus` enforces it.
- **Q:** How do you add a "created by user X" filter? → **A:** Add `static TaskFilter createdBy(String u) { return t -> u.equals(t.createdBy()); }` — nothing else changes.
- **Q:** Why does the service take a `Clock`? → **A:** So "today" is controllable in demos and tests.
- **Q:** Why is `Task` synchronized per instance instead of a lock in the service? → **A:** Finer-grained: edits to different tasks never contend.

TL;DR

Task = entity that guards its own state machine and history. Service = facade. Filters = Specification, sorts = Strategy, notifications = Observer. Thread-safety per task.

21.2 Design a Digital Wallet Service

Problem & clarifying questions

Build the core of a wallet (an app account that stores money for payments) like Paytm, PhonePe or PayPal balance: users hold wallets, add money, transfer to other users and see a statement. The awesome-low-level-design statement also lists account and personal-info management, adding and removing payment methods (cards, bank accounts), transfers to *external* accounts, multiple currencies with conversion, security, **concurrent transactions with data consistency**, and scale. Promotional **cashback** (money given back to the customer after a payment, as a reward) is not in that statement; it is added here because wallet machine-coding variants often ask for it, and it is a clean place to show the Strategy pattern and double-entry thinking (the accounting rule that every amount added somewhere is taken from somewhere else, so the books always balance). In a machine-coding round the interviewer focuses on three things: **money arithmetic**, **no double-spending under concurrency** (spending the same money twice because two requests ran at the same time), and **no double-charging on retries** (idempotency).

> **Key term — idempotency:** an operation is **idempotent** if running it twice has the same effect as running it once. Networks fail in the middle of requests, so clients retry; if "transfer 100" is not idempotent, a retry moves another 100. The usual fix is an **idempotency key**: the client sends a unique id with the request, and the server remembers the result for each id and returns it again for a repeat instead of re-running the work. More in System Design Fundamentals.

Question	Assumed answer
Money type?	`BigDecimal` with scale 2 (paise/cents). Inputs with more than 2 decimals are rejected, not silently rounded.
Can a balance go negative?	No.
What if a client retries a request (network timeout — it gave up waiting for the reply)?	Each request carries an **idempotency key**; a repeated key returns the original result and moves no money.
Offers?	Cashback strategies: "10% on first transfer, max 50" and "flat 25 on transfers ≥ 1000". Offers do not stack — the best one wins.
Where does cashback money come from?	From a **promotions wallet** (`PROMO`) that marketing tops up. Cashback is an ordinary transfer PROMO → sender, so money is never created from nothing. If the budget is used up, no cashback is paid and the transfer still stands.
Top-up source?	An external bank/card; modelled as a credit (money added to the wallet) with a reference.
Multi-currency?	Out of scope for the 90-minute version; discussed in extensions.

Requirements

Functional

1. Create a wallet for a user.
2. Top up a wallet.
3. Transfer between two wallets (atomic: both sides or neither).
4. Transaction history (statement) per wallet, with running balance.
5. Apply the best eligible cashback offer after a transfer, paid out of the funded promotions wallet.
6. Idempotent top-up and transfer.

Non-functional

- **Correctness under concurrency:** no lost updates (one thread's change silently overwritten by another's), no negative balances, no deadlocks (two threads each waiting forever for a lock the other holds), total money conserved (only top-ups add money; transfers and cashback only move it).
- Exact decimal arithmetic.
- New offers added without touching the transfer code.

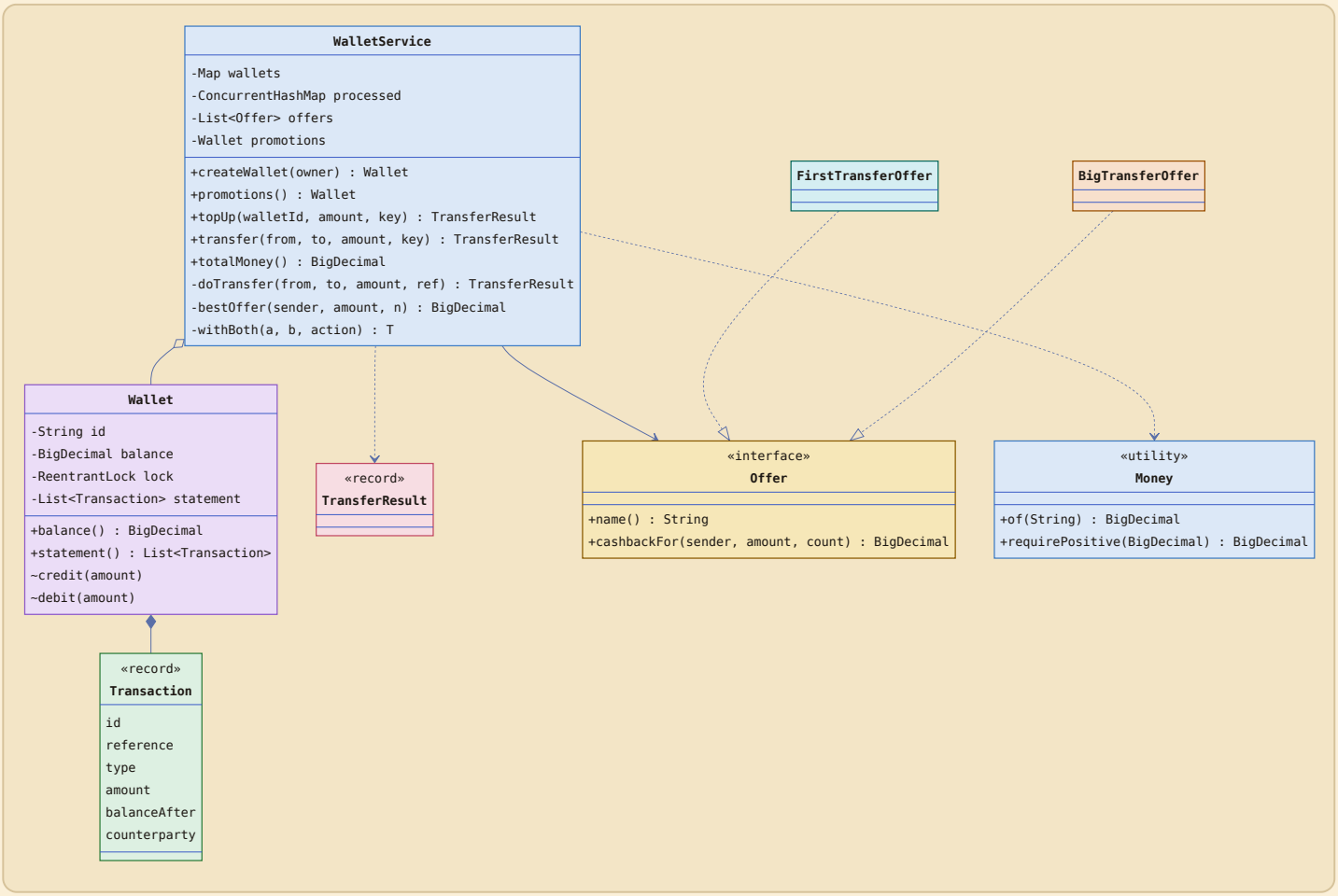
Use cases

Actor	Use cases
User	create wallet, top up, transfer, view statement
Bank / card gateway (external)	source of top-ups
Marketing (configures)	defines offers, funds the promotions wallet
Client app (retrying)	re-sends the same request with the same idempotency key

Core entities

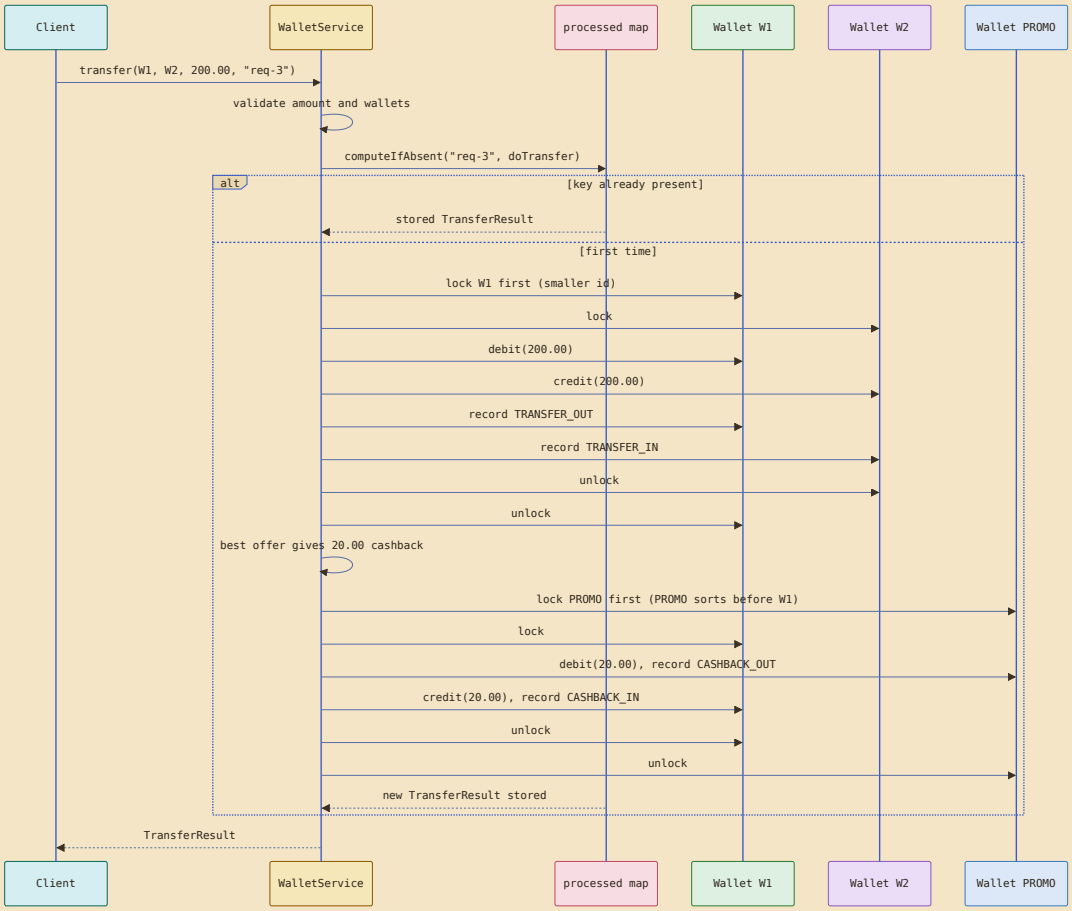
Class	Kind	Responsibility
`Wallet`	class	Balance + own statement + its `ReentrantLock` (Java's explicit lock object). `debit` refuses to go below zero. The service owns one special wallet, `PROMO`, that funds cashback.
`Transaction`	record	Immutable ledger line: type, amount, balance after, counterparty (the other wallet involved), reference.
`TxnType`	enum	TOP_UP, TRANSFER_OUT, TRANSFER_IN, CASHBACK_OUT (on PROMO), CASHBACK_IN (on the user).
`TransferResult`	record	What the caller gets (stored for idempotent replays).
`Money`	utility	Normalises and validates `BigDecimal` amounts.
`Offer` + `FirstTransferOffer`, `BigTransferOffer`	interface + strategies	Compute cashback for a completed transfer.
`WalletService`	class	Facade: creates wallets, orchestrates (calls in the right order) locking, idempotency, offers.
`InsufficientFundsException`, `WalletNotFoundException`	exceptions	Failure reasons.

Class diagram



Key flows

Transfer with idempotency and lock ordering:


Client
WalletService
processed map
Wallet W1
Wallet W2
Wallet PROMO
transfer(W1, W2, 200.00, "req-3")
validate amount and wallets
computeIfAbsent("req-3", doTransfer)
alt
[key already present]
stored TransferResult
[first time]
lock W1 first (smaller id)
lock
debit(200.00)
credit(200.00)
record TRANSFER_OUT
record TRANSFER_IN
unlock
unlock
best offer gives 20.00 cashback
lock PROMO first (PROMO sorts before W1)
lock
debit(20.00), record CASHBACK_OUT
credit(20.00), record CASHBACK_IN
unlock
unlock
new TransferResult stored
TransferResult
Client
WalletService
processed map
Wallet W1
Wallet W2
Wallet PROMO

Design decisions & patterns used

Decision	Why	Alternative rejected
`BigDecimal`, scale 2, reject > 2 decimals	`double` cannot represent 0.1 exactly; money must be exact. Rejecting `1.005` avoids silent rounding surprises.	`double` (wrong), `long` paise (also correct, and a fine choice — say so).
Lock per wallet, acquired in id order	Two opposite transfers (A→B and B→A) would deadlock if each locked its own source first. A global order (smaller id first) makes a cycle impossible.	One global lock — correct but every transfer in the system serialises.
Debit before credit, both under both locks	`debit` throws before any state changes, so a failed transfer leaves both wallets untouched (atomicity).	Credit then debit — would need a compensating action (a new step that undoes an earlier one) on failure.
Idempotency via `ConcurrentHashMap.computeIfAbsent` (creates and stores a value only if the key is missing, as one atomic step)	The check "seen this key?" and the execution are one atomic step per key, so two concurrent retries cannot both run. If the transfer throws, nothing is stored and the client can retry.	`if (!map.containsKey(k)) { run; put }` — check-then-act race (another thread can act between the check and the action).
Cashback paid from a funded `PROMO` wallet	Double-entry: every credit has a matching debit, so the sum of all wallets changes only on top-ups — which is exactly what the demo checks. A campaign cannot overspend its budget.	Crediting the user out of thin air — the books no longer balance and nobody can say what a campaign cost.
Cashback as a second step, after the transfer locks are released	Only two wallets are ever locked at once, and ordinary transfers never wait on the busy `PROMO` lock.	Locking sender, receiver and PROMO together for every transfer — correct, but every transfer in the system would queue on PROMO.
`Offer` interface, best one wins	Strategy: new promotions are new classes; the "no stacking" policy lives in one method.	`if/else` on promotion codes inside `transfer`.
Statement stored in the wallet	Reading a wallet's history needs no global scan.	One global transaction list filtered on read — fine for small data, slow at scale.

`computeIfAbsent` detail worth knowing: the JDK (Java Development Kit: Java's compiler and standard library) documents that the whole call is atomic, that the mapping function runs only when the key is absent, that an exception from the function is rethrown with no mapping recorded, and that other threads updating the *same map* may block while it runs — so the function should be short and must not update the map itself. Our function only touches wallets, never `processed`, so it is safe. At higher scale, store a `CompletableFuture` (a handle to a result that will be ready later) with `putIfAbsent` and run the transfer outside the map (extension below).

Java implementation

```
// File: lld/ch21/wallet/Money.java
package lld.ch21.wallet;

import java.math.BigDecimal;
import java.math.RoundingMode;

/** Helpers that keep every amount at 2 decimal places. Never use double for money. */
public final class Money {
    public static final int SCALE = 2;
    private Money() { }

    public static BigDecimal of(String amount) { return normalize(new BigDecimal(amount)); }

    public static BigDecimal normalize(BigDecimal amount) { return amount.setScale(SCALE, RoundingMode.HALF_EVEN); }

    public static BigDecimal requirePositive(BigDecimal amount) {
        if (amount == null || amount.signum() <= 0) throw new IllegalArgumentException("Amount must be positive: " +
amount);
        if (amount.stripTrailingZeros().scale() > SCALE)
            throw new IllegalArgumentException("At most " + SCALE + " decimal places: " + amount);
        return normalize(amount);
    }
}
```

```
// File: lld/ch21/wallet/TxnType.java
package lld.ch21.wallet;

/** CASHBACK_OUT is written on the promotions wallet, CASHBACK_IN on the user who receives it. */
public enum TxnType { TOP_UP, TRANSFER_OUT, TRANSFER_IN, CASHBACK_OUT, CASHBACK_IN }
```

```
// File: lld/ch21/wallet/Transaction.java
package lld.ch21.wallet;

import java.math.BigDecimal;
import java.time.Instant;

/** Immutable ledger entry. Each transfer writes two entries (out + in) sharing one reference. */
public record Transaction(String id, String reference, String walletId, TxnType type, BigDecimal amount,
                          BigDecimal balanceAfter, String counterparty, Instant at) {
    @Override public String toString() {
        return "%-5s %-12s %8s  bal=%8s  ref=%s%s".formatted(id, type, amount, balanceAfter, reference,
                counterparty == null ? "" : " with " + counterparty);
    }
}
```

```
// File: lld/ch21/wallet/TransferResult.java
package lld.ch21.wallet;

import java.math.BigDecimal;

/** What the caller gets back. A replayed (duplicate) request returns the stored result. */
public record TransferResult(String reference, String from, String to, BigDecimal amount, BigDecimal cashback) { }
```

```java
// File: lld/ch21/wallet/Wallet.java
package lld.ch21.wallet;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.locks.ReentrantLock;

/**
 * Holds a balance and its own statement. The balance is only changed while `lock` is held;
 * the service decides lock order when two wallets are involved.
 */
public class Wallet {
    private final String id;
    private final String ownerName;
    private final ReentrantLock lock = new ReentrantLock();
    private BigDecimal balance = Money.normalize(BigDecimal.ZERO);
    private final List<Transaction> statement = new ArrayList<>();

    Wallet(String id, String ownerName) { this.id = id; this.ownerName = ownerName; }

    public String id() { return id; }
    public String ownerName() { return ownerName; }
    ReentrantLock lock() { return lock; }

    public BigDecimal balance() {
        lock.lock();
        try { return balance; } finally { lock.unlock(); }
    }

    public List<Transaction> statement() {
        lock.lock();
        try { return List.copyOf(statement); } finally { lock.unlock(); }
    }

    // --- called only with lock held ---
    void credit(BigDecimal amt) { balance = balance.add(amt); }

    void debit(BigDecimal amt) {
        if (balance.compareTo(amt) < 0) throw new InsufficientFundsException(id, balance, amt);
        balance = balance.subtract(amt);
    }

    BigDecimal rawBalance() { return balance; }
    void record(Transaction t) { statement.add(t); }
}
```

```java
// File: lld/ch21/wallet/InsufficientFundsException.java
package lld.ch21.wallet;

import java.math.BigDecimal;

public class InsufficientFundsException extends RuntimeException {
    public InsufficientFundsException(String walletId, BigDecimal balance, BigDecimal needed) {
        super("Insufficient funds in " + walletId + ": balance " + balance + ", needed " + needed);
    }
}
```

```java
// File: lld/ch21/wallet/WalletNotFoundException.java
package lld.ch21.wallet;

public class WalletNotFoundException extends RuntimeException {
    public WalletNotFoundException(String id) { super("Wallet not found: " + id); }
}
```

```java
// File: lld/ch21/wallet/Offer.java
package lld.ch21.wallet;

import java.math.BigDecimal;

/** Strategy: an offer looks at a completed transfer and returns the cashback (zero if not eligible). */
public interface Offer {
    String name();
    BigDecimal cashbackFor(Wallet sender, BigDecimal amount, int senderTransferCount);
}
```

```java
// File: lld/ch21/wallet/FirstTransferOffer.java
package lld.ch21.wallet;

import java.math.BigDecimal;
import java.math.RoundingMode;

/** 10% cashback on a user's first transfer, capped. */
public class FirstTransferOffer implements Offer {
    private final BigDecimal percent;
    private final BigDecimal cap;

    public FirstTransferOffer(String percent, String cap) {
        this.percent = new BigDecimal(percent);
        this.cap = Money.of(cap);
    }

    @Override public String name() { return "FIRST_TRANSFER"; }

    @Override public BigDecimal cashbackFor(Wallet sender, BigDecimal amount, int count) {
        if (count != 1) return BigDecimal.ZERO;
        BigDecimal raw = amount.multiply(percent).divide(BigDecimal.valueOf(100), Money.SCALE,
RoundingMode.HALF_EVEN);
        return raw.min(cap);
    }
}
```

```java
// File: lld/ch21/wallet/BigTransferOffer.java
package lld.ch21.wallet;

import java.math.BigDecimal;

/** Flat cashback for any transfer at or above a threshold. */
public class BigTransferOffer implements Offer {
    private final BigDecimal threshold;
    private final BigDecimal flat;

    public BigTransferOffer(String threshold, String flat) {
        this.threshold = Money.of(threshold);
        this.flat = Money.of(flat);
    }

    @Override public String name() { return "BIG_TRANSFER"; }

    @Override public BigDecimal cashbackFor(Wallet sender, BigDecimal amount, int count) {
        return amount.compareTo(threshold) >= 0 ? flat : BigDecimal.ZERO;
    }
}
```

```java
// File: lld/ch21/wallet/WalletService.java
package lld.ch21.wallet;

import java.math.BigDecimal;
import java.time.Clock;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.function.Supplier;

public class WalletService {
    public static final String PROMOTIONS_ID = "PROMO";
    private final Map<String, Wallet> wallets = new ConcurrentHashMap<>();
    /** Idempotency store: request key -> result of the first successful execution. */
    private final ConcurrentHashMap<String, TransferResult> processed = new ConcurrentHashMap<>();
    private final Map<String, AtomicInteger> transferCounts = new ConcurrentHashMap<>();
    private final List<Offer> offers;
    private final AtomicInteger txnSeq = new AtomicInteger();
    private final AtomicInteger walletSeq = new AtomicInteger();
    private final Clock clock;
    /** Marketing's budget. Cashback is a real transfer out of this wallet, so no money is created. */
    private final Wallet promotions = new Wallet(PROMOTIONS_ID, "Promotions");

    public WalletService(List<Offer> offers, Clock clock) {
        this.offers = List.copyOf(offers);
        this.clock = clock;
        wallets.put(promotions.id(), promotions);
    }

    public Wallet promotions() { return promotions; }

    public Wallet createWallet(String ownerName) {
        Wallet w = new Wallet("W" + walletSeq.incrementAndGet(), ownerName);
        wallets.put(w.id(), w);
        return w;
    }

    public TransferResult topUp(String walletId, BigDecimal amount, String idempotencyKey) {
        BigDecimal amt = Money.requirePositive(amount);
        Wallet w = get(walletId);
        return processed.computeIfAbsent(idempotencyKey, key -> {
            w.lock().lock();
            try {
                w.credit(amt);
                w.record(txn(key, w, TxnType.TOP_UP, amt, null));
            } finally { w.lock().unlock(); }
            return new TransferResult(key, "BANK", walletId, amt, BigDecimal.ZERO);
        });
    }

    /**
     * Moves money atomically. computeIfAbsent makes the key check-and-run atomic, so a retried
     * request never debits twice. If the lambda throws, nothing is stored and the client may retry.
     */
    public TransferResult transfer(String fromId, String toId, BigDecimal amount, String idempotencyKey) {
        if (fromId.equals(toId)) throw new IllegalArgumentException("Cannot transfer to the same wallet");
        BigDecimal amt = Money.requirePositive(amount);
        Wallet from = get(fromId), to = get(toId);
        return processed.computeIfAbsent(idempotencyKey, key -> doTransfer(from, to, amt, key));
    }

    private TransferResult doTransfer(Wallet from, Wallet to, BigDecimal amt, String ref) {
        // Step 1: the transfer itself, under both wallet locks.
        int n = withBoth(from, to, () -> {
            from.debit(amt);                        // throws before any change if funds are short
            to.credit(amt);
            from.record(txn(ref, from, TxnType.TRANSFER_OUT, amt, to.id()));
            to.record(txn(ref, to, TxnType.TRANSFER_IN, amt, from.id()));
            return transferCounts.computeIfAbsent(from.id(), k -> new AtomicInteger()).incrementAndGet();
        });
        // Step 2: cashback is a second, ordinary transfer PROMO -> sender (same lock-ordering rule).
```

```
        BigDecimal offered = bestOffer(from, amt, n);
        BigDecimal cashback = offered.signum() == 0 || from == promotions ? BigDecimal.ZERO
                : withBoth(promotions, from, () -> {
                    if (promotions.rawBalance().compareTo(offered) < 0) return BigDecimal.ZERO;   // budget used up
                    promotions.debit(offered);
                    from.credit(offered);
                    promotions.record(txn(ref, promotions, TxnType.CASHBACK_OUT, offered, from.id()));
                    from.record(txn(ref, from, TxnType.CASHBACK_IN, offered, promotions.id()));
                    return offered;
                });
        return new TransferResult(ref, from.id(), to.id(), amt, cashback);
    }

    /** Locks two wallets, always the smaller id first, so two opposite operations can never deadlock. */
    private static <T> T withBoth(Wallet a, Wallet b, Supplier<T> action) {
        Wallet first = a.id().compareTo(b.id()) < 0 ? a : b;
        Wallet second = first == a ? b : a;
        first.lock().lock();
        try {
            second.lock().lock();
            try { return action.get(); } finally { second.lock().unlock(); }
        } finally { first.lock().unlock(); }
    }

    /** Offers do not stack: the single most generous one wins. */
    private BigDecimal bestOffer(Wallet sender, BigDecimal amt, int n) {
        return offers.stream().map(o -> o.cashbackFor(sender, amt, n))
                .max(Comparator.naturalOrder()).orElse(BigDecimal.ZERO);
    }

    public Wallet get(String id) {
        Wallet w = wallets.get(id);
        if (w == null) throw new WalletNotFoundException(id);
        return w;
    }

    /** Sum of every wallet, promotions included: only top-ups change it (cashback just moves money). */
    public BigDecimal totalMoney() {
        return wallets.values().stream().map(Wallet::balance).reduce(BigDecimal.ZERO, BigDecimal::add);
    }

    private Transaction txn(String ref, Wallet w, TxnType type, BigDecimal amt, String counterparty) {
        return new Transaction("TX" + txnSeq.incrementAndGet(), ref, w.id(), type, amt, w.rawBalance(),
                counterparty, clock.instant());
    }
}
```

```java
// File: lld/ch21/wallet/Main.java
package lld.ch21.wallet;

import java.math.BigDecimal;
import java.time.Clock;
import java.time.Instant;
import java.time.ZoneOffset;
import java.util.List;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class Main {
    public static void main(String[] args) throws InterruptedException {
        Clock clock = Clock.fixed(Instant.parse("2026-09-28T10:00:00Z"), ZoneOffset.UTC);
        WalletService svc = new WalletService(
                List.of(new FirstTransferOffer("10", "50.00"), new BigTransferOffer("1000.00", "25.00")), clock);

        svc.topUp(svc.promotions().id(), Money.of("100.00"), "budget-1");   // marketing funds the campaign
        Wallet alice = svc.createWallet("Alice");
        Wallet bob = svc.createWallet("Bob");

        System.out.println("== top-ups");
        svc.topUp(alice.id(), Money.of("2000.00"), "req-1");
        svc.topUp(bob.id(), Money.of("300.50"), "req-2");
        svc.topUp(alice.id(), Money.of("2000.00"), "req-1");       // client retry: same key, no double credit
        System.out.println("  Alice=" + alice.balance() + "  Bob=" + bob.balance());

        System.out.println("== transfers");
        TransferResult r1 = svc.transfer(alice.id(), bob.id(), Money.of("200.00"), "req-3");
        System.out.println("  " + r1);
        TransferResult r1again = svc.transfer(alice.id(), bob.id(), Money.of("200.00"), "req-3");
        System.out.println("  retry returns same result? " + (r1 == r1again));
        System.out.println("  " + svc.transfer(alice.id(), bob.id(), Money.of("1200.00"), "req-4"));
        System.out.println("  Alice=" + alice.balance() + "  Bob=" + bob.balance() + "  PROMO=" +
svc.promotions().balance());

        System.out.println("== edge cases");
        attempt(() -> svc.transfer(bob.id(), alice.id(), Money.of("99999.00"), "req-5"));
        attempt(() -> svc.transfer(bob.id(), alice.id(), new BigDecimal("-5"), "req-6"));
        attempt(() -> svc.transfer(bob.id(), alice.id(), new BigDecimal("1.005"), "req-7"));
        attempt(() -> svc.transfer(bob.id(), bob.id(), Money.of("1.00"), "req-8"));
        attempt(() -> svc.transfer(bob.id(), "W99", Money.of("1.00"), "req-9"));
        System.out.println("  failed req-5 can be retried later: " + svc.transfer(bob.id(), alice.id(),
Money.of("10.00"), "req-5").amount());

        System.out.println("== statement for Alice");
        alice.statement().forEach(t -> System.out.println("  " + t));
        System.out.println("== statement for PROMO");
        svc.promotions().statement().forEach(t -> System.out.println("  " + t));

        System.out.println("== concurrency: 1000 opposite transfers on 8 threads");
        BigDecimal before = svc.totalMoney();
        ExecutorService pool = Executors.newFixedThreadPool(8);
        for (int i = 0; i < 1000; i++) {
            final int n = i;
            pool.submit(() -> {
                try {
                    if (n % 2 == 0) svc.transfer(alice.id(), bob.id(), Money.of("1.00"), "c-" + n);
                    else svc.transfer(bob.id(), alice.id(), Money.of("1.00"), "c-" + n);
                } catch (InsufficientFundsException ignored) { }
            });
        }
        pool.shutdown();
        pool.awaitTermination(10, TimeUnit.SECONDS);
        System.out.println("  total before=" + before + " after=" + svc.totalMoney()
                + " conserved=" + (before.compareTo(svc.totalMoney()) == 0));
        System.out.println("  Alice=" + alice.balance() + "  Bob=" + bob.balance() + "  PROMO=" +
svc.promotions().balance());
    }

    private static void attempt(Runnable r) {
        try { r.run(); System.out.println("  ok"); }
```

```
        catch (RuntimeException e) { System.out.println("  error: " + e.getMessage()); }
    }
}
```

Output:

```
== top-ups
  Alice=2000.00  Bob=300.50
== transfers
  TransferResult[reference=req-3, from=W1, to=W2, amount=200.00, cashback=20.00]
  retry returns same result? true
  TransferResult[reference=req-4, from=W1, to=W2, amount=1200.00, cashback=25.00]
  Alice=645.00  Bob=1700.50  PROMO=55.00
== edge cases
  error: Insufficient funds in W2: balance 1700.50, needed 99999.00
  error: Amount must be positive: -5
  error: At most 2 decimal places: 1.005
  error: Cannot transfer to the same wallet
  error: Wallet not found: W99
  failed req-5 can be retried later: 10.00
== statement for Alice
  TX2   TOP_UP        2000.00  bal= 2000.00  ref=req-1
  TX4   TRANSFER_OUT   200.00  bal= 1800.00  ref=req-3 with W2
  TX7   CASHBACK_IN     20.00  bal= 1820.00  ref=req-3 with PROMO
  TX8   TRANSFER_OUT  1200.00  bal=  620.00  ref=req-4 with W2
  TX11  CASHBACK_IN     25.00  bal=  645.00  ref=req-4 with PROMO
  TX13  TRANSFER_IN     10.00  bal=  655.00  ref=req-5 with W2
== statement for PROMO
  TX1   TOP_UP         100.00  bal=  100.00  ref=budget-1
  TX6   CASHBACK_OUT    20.00  bal=   80.00  ref=req-3 with W1
  TX10  CASHBACK_OUT    25.00  bal=   55.00  ref=req-4 with W1
  TX14  CASHBACK_OUT     1.00  bal=   54.00  ref=req-5 with W2
== concurrency: 1000 opposite transfers on 8 threads
  total before=2400.50 after=2400.50 conserved=true
  Alice=655.00  Bob=1691.50  PROMO=54.00
```

Reading the output: marketing first funds `PROMO` with 100.00; the retried top-up `req-1` did not add another 2000; the retried transfer `req-3` returned the *same object* and moved no money; Alice's first transfer earned 10% (20.00), her 1200.00 transfer earned the flat 25.00 (the first-transfer offer no longer applies); the failed `req-5` stored nothing, so re-using that key later succeeded (and, being Bob's first successful transfer, earned him 1.00). Every cashback appears twice — `CASHBACK_OUT` on PROMO (100.00 → 80.00 → 55.00 → 54.00) and `CASHBACK_IN` on the receiver. Finally 1000 concurrent opposite transfers kept the total (PROMO included) exactly constant — no lost updates, no deadlock. (Each side starts with more than 500.00, so no transfer in that race can fail, and the final balances are the same on every run.)

Concurrency & edge cases

- **Race on one wallet:** every read-modify-write (read a value, compute a new one, write it back — three steps another thread can slip between) of `balance` happens with the wallet's lock held.
- **Deadlock:** avoided by a global lock order (lexicographic — dictionary order — wallet id; `"PROMO"` sorts before `"W1"`). Every two-lock path goes through one helper, `withBoth`, so the order cannot be forgotten. The program never holds more than two wallet locks at a time.
- **Double submit:** idempotency key → at most one execution per key.
- **Same key, different body** (client bug): today the stored result is returned. Production systems also store a hash (a short fingerprint computed from the data) of the request and reject a mismatch with an error.
- **Validation:** non-positive amounts, more than 2 decimals, same source and target, unknown wallet, insufficient funds.
- **Cashback money source:** a transfer from the `PROMO` wallet, so the books balance (double-entry: every credit has a matching debit). The cashback step runs after the transfer's locks are released, so for a moment the transfer is visible without its cashback — fine for a reward. If the budget cannot cover the offer, the result reports a cashback of zero and the transfer is kept.

Extensions & follow-up questions

- **Multi-currency?** Each wallet has a `Currency`; transfers across currencies go through a `FxRateProvider` (FX = foreign exchange) interface; store both amounts and the rate used in the transaction. Use `java.util.Currency` for codes and decimal places.
- **Payment methods (card, bank, UPI — India's instant bank-to-bank payment system)?** A `PaymentMethod` interface with `charge(amount)`; top-up calls it first, and only credits the wallet on success (or uses a pending → confirmed state if the gateway is asynchronous). Users add and remove methods on their account.
- **Transfers to external accounts (withdraw to bank)?** Debit the wallet into a `PENDING_PAYOUT` state, call the bank through a `PayoutGateway` interface, and either confirm, or post a compensating credit if the bank rejects it.
- **Reversal / refund?** Never edit a past transaction; post a new compensating transaction referencing the original.
- **Distributed system?** Wallet balances in a database with row locks (`SELECT ... FOR UPDATE` locks the rows it reads until the transaction ends) or optimistic versioning; idempotency keys in a table with a unique constraint (a database rule that refuses a second row with the same value); ledgers as double-entry journals.
- **Long transfers blocking the idempotency map?** Use `putIfAbsent(key, new CompletableFuture<>())`: the winner runs the transfer outside the map and completes the future; duplicates just wait on `future.join()` (blocks until the result is ready). On failure, remove the key so a retry can run.
- **Limits (daily cap, KYC tier — KYC, "know your customer", is the identity check that sets how much a user may move)?** A chain of validators (Chain of Responsibility) run before `doTransfer`.
- **Why not `AtomicReference<BigDecimal>` + CAS (compare-and-set: replace a value only if it still equals what you last read, in one indivisible step)?** Works for a single wallet, but a transfer touches two balances that must change together; CAS on two variables is not atomic, so locks are simpler.

Real-world example

Stripe's (a large online-payments company) API accepts an `Idempotency-Key` header (a named piece of metadata sent along with an HTTP request): if a request times out and the client retries with the same key, Stripe returns the saved result of the first attempt instead of charging twice. Wallets such as Paytm and PhonePe run cashback campaigns with rules like "X% up to a cap on your first transaction" — exactly the shape of `FirstTransferOffer`.

In the interview

Lead with the three risks: exact money, concurrency, retries (and, if offers come up, "cashback must come out of a budget wallet"). Show `Money`, then `doTransfer` with the lock ordering (draw the A→B / B→A deadlock and how ordering breaks the cycle), then idempotency. Run the concurrent demo and point at `conserved=true`. Offers are a 2-minute Strategy mention. If time is short, cut multi-currency and payment methods — say you would add them behind interfaces.

Self-check

- **Q:** Why is `new BigDecimal("0.1")` preferred over `new BigDecimal(0.1)`? → **A:** The double constructor captures the binary approximation of 0.1 (`double` stores numbers in base 2, where 0.1 has no exact form) (a long, inexact decimal); the String constructor is exact.
- **Q:** Two threads transfer W1→W2 and W2→W1 at once. Why no deadlock? → **A:** Both lock W1 first (smaller id), so one waits for the other instead of each holding what the other needs.
- **Q:** A transfer fails for insufficient funds. Can the same idempotency key be reused? → **A:** Yes — `computeIfAbsent` stores nothing when the function throws.
- **Q:** Where would a "max 3 transfers per minute" rule go? → **A:** A validator run before `doTransfer`, not inside the wallet.

TL;DR

`BigDecimal` money, lock both wallets in a global order, debit before credit, idempotency key → run at most once, offers as Strategies paid out of a funded promotions wallet. Prove it with a concurrent demo where the total is conserved.

21.3 Design an Event Calendar (Flipkart)

Problem & clarifying questions

This is a Flipkart machine-coding statement (kumaransg/LLD, `Event_calendar_flipkart`). In short: onboard users with **working hours**; group users into **teams** (a user belongs to at most one team); create **events** on one day with a start and end time, whose participants are users, teams, or both. A user can be in **only one event at a time**. For team participants the event creator says how many **representatives (N)** each team must send; the system blocks the event against N *available* members of each team, and if any team cannot supply N the event is not created. Required features: create user, create team, create event, **get a user's events in a time range** (including events they were blocked for as a team representative), and **suggest free slots** for a list of users/teams on a day. Bonus: **update event time** (report failure if impossible) and **recurring events** (time, start date, end date, frequency). The statement says: no DB, no UI, no CLI (command-line interface: a program you type commands into) needed — a `main` or tests is enough — and "do not make assumptions, ask".

Clarifying questions (and the answers this solution uses):

Question	Answer used
Is an event that ends at 15:00 in conflict with one that starts at 15:00?	No. Slots are **half-open** `[start, end)` (the start minute belongs to the slot, the end minute does not).
Must an event lie inside every attendee's working hours?	Yes (the statement's Event5 fails because F starts at 11:00).
Which team members become representatives?	The first N free members in team order (a pluggable strategy).
If a user is invited directly *and* is in an invited team, do they count as a representative?	No — a direct attendee is not picked again as a representative.
Free slots for a team with N representatives: what counts as "free"?	A moment is free for the team if at least N members are working and not busy then.
Recurring event: if one occurrence clashes?	Reject the whole series (all-or-nothing).

A note on the statement's sample. For "free slots for user A and team T1 with 1 representative today" the sample answer says *10AM to 3PM and 4PM to 7PM*. But T1 = {C, E} and neither works before 11:00 (C starts 11:30, E 11:00), so by the statement's own rules the first slot should start at **11:00**. Our program prints `11:00-15:00, 16:00-19:00`. Pointing out such an inconsistency politely ("I think the sample means 11AM; shall I go with the rules?") is exactly what "do not make assumptions, ask" is testing. (A smaller slip in the same sample: its step 1 says "Onboard 5 Users" but then lists six, A to F. The demo creates all six.)

Requirements

Functional

1. Create user with working hours.
2. Create team of existing users (one team per user).
3. Create event for users and/or teams with N representatives; reject if any attendee is outside working hours, already busy, or a team lacks N free members.
4. Events of a user between two date-times (direct + as representative).
5. Free slots for users/teams on a day.
6. Bonus: update event time (atomic: old booking kept on failure).
7. Bonus: recurring events (daily/weekly between two dates).

Non-functional

- The check "is everyone free?" and the booking must be **atomic**, or two concurrent creations could book the same person twice.
- Representative choice should be replaceable (fairness, least-busy, random).

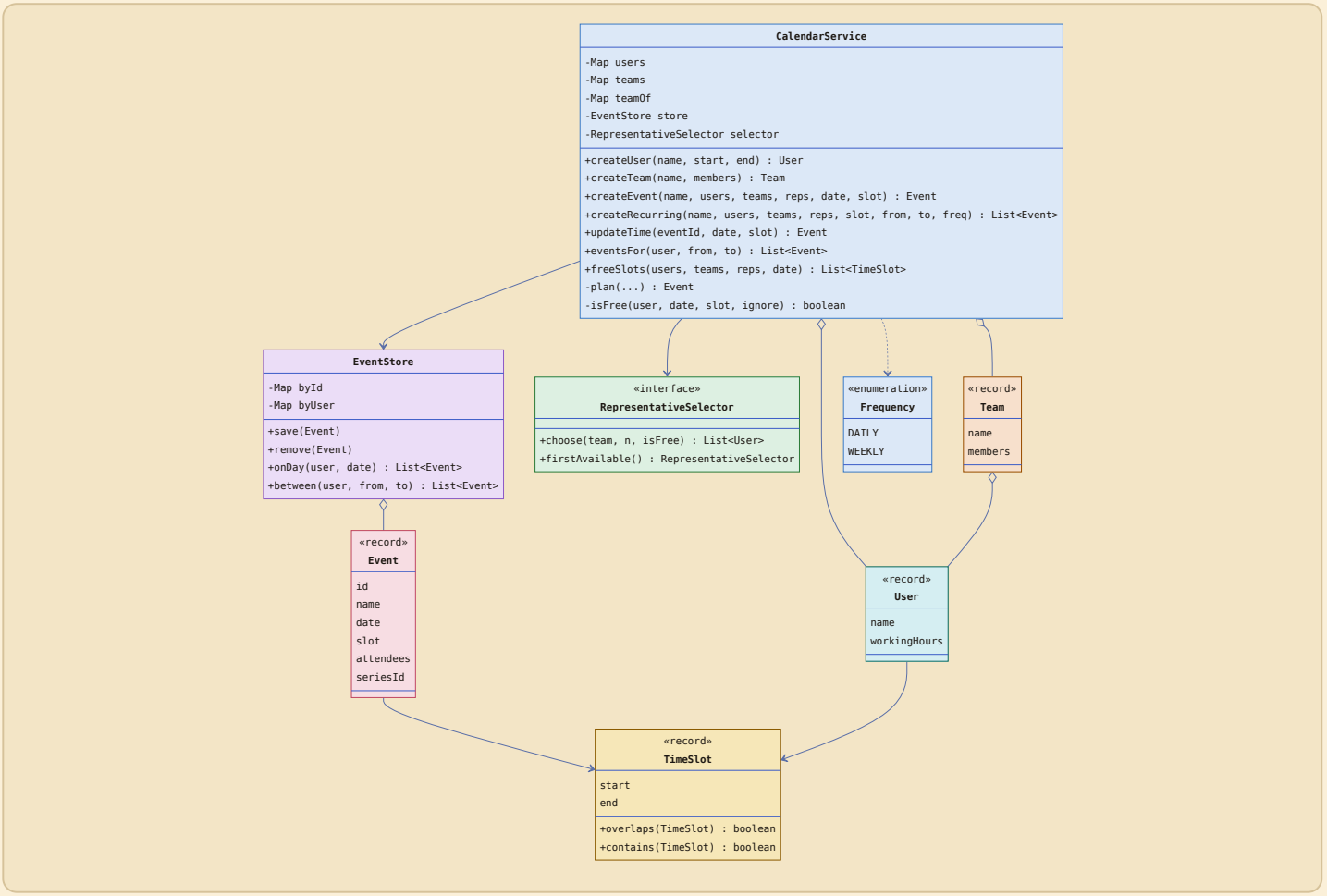
Use cases

Actor	Use cases
Admin	onboard user, create team
Organiser	create event, create recurring event, move event, find free slots
Attendee	view own events in a range

Core entities

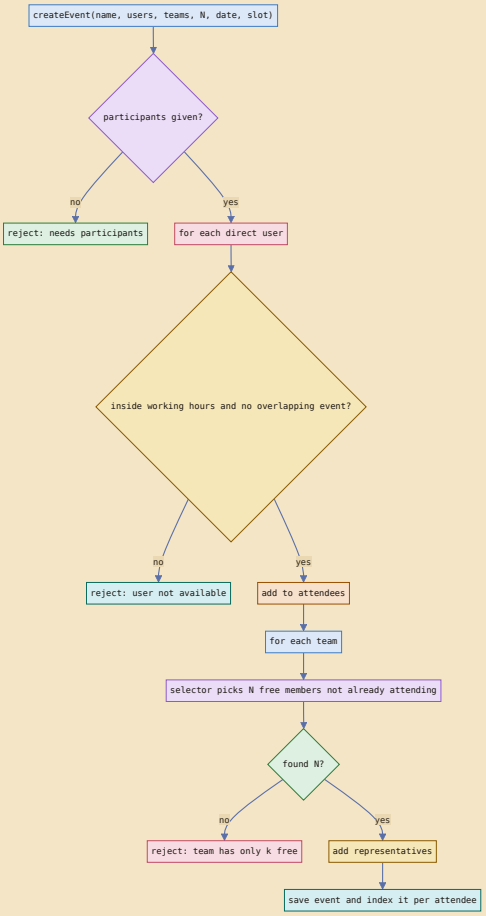
Class	Kind	Responsibility
`TimeSlot`	record	Half-open `[start, end)` on one day; `overlaps`, `contains`; validates start < end.
`User`	record	Name + working hours.
`Team`	record	Name + ordered members.
`Event`	record	One concrete occurrence: date, slot, what was requested (users, teams, N) and who is actually blocked (`attendees`), optional `seriesId`.
`Frequency`	enum	DAILY / WEEKLY — knows the next date.
`EventStore`	class	Repository + index `user → date → events` for fast conflict checks and range queries ("everything between X and Y"; `TreeMap.subMap` returns just that slice of a sorted map).
`RepresentativeSelector`	interface	Strategy: pick N free members of a team.
`CalendarService`	class	All use cases; planning (validation + representative choice) separated from saving.
`CalendarException`	exception	Readable reason for every rejection.

Class diagram

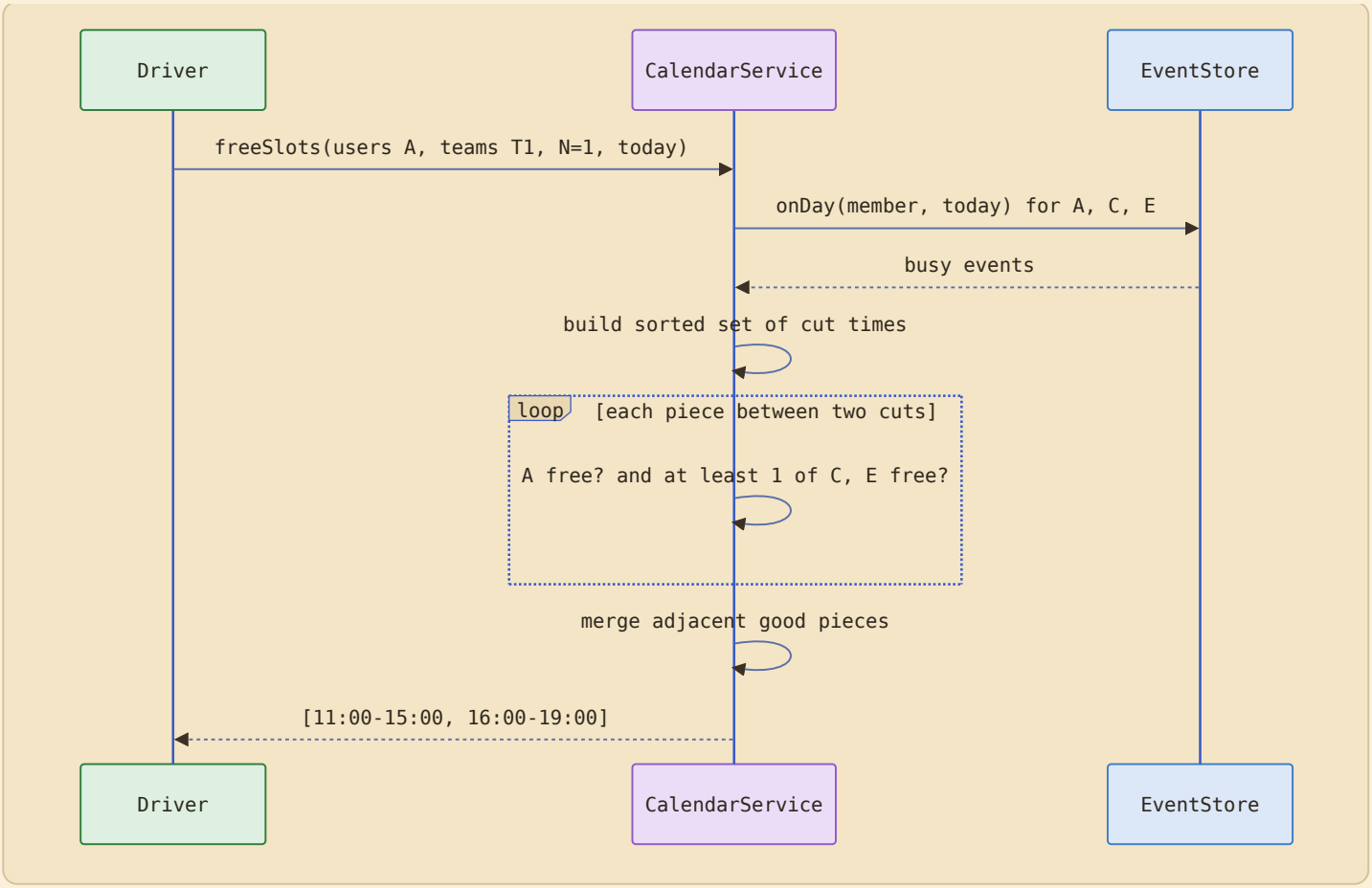


Key flows

Create an event (plan, then save):



Free-slot sweep. Collect every "interesting" time on that day: each involved person's working-hour start and end, and the start and end of each of their events. Sort them. Between two consecutive points nothing changes, so test each small piece once: all direct users free, and each team has at least N free members. Merge neighbouring good pieces. For the statement's example (A + T1, N = 1, today) the cut points are 10:00, 11:00, 11:30, 15:00, 16:00, 18:30, 19:00, 19:30, and the good pieces merge into 11:00–15:00 and 16:00–19:00.



Design decisions & patterns used

Decision	Why	Alternative rejected
Half-open intervals	Back-to-back meetings do not conflict; `overlaps` is one line: `a.start < b.end && b.start < a.end`.	Closed intervals — 14:00–15:00 would clash with 15:00–16:00.
`plan()` separate from `save()`	Validation + representative choice produce an `Event` without side effects (no change to any stored state). `createEvent`, `createRecurring` (plan all, then save all) and `updateTime` (plan while ignoring the old event, then swap) all reuse it — all-or-nothing for free.	Saving as you go and rolling back (undoing the saved steps) on failure.
`RepresentativeSelector` strategy	"Which N members?" is a policy; fairness or least-busy can be plugged in.	Hard-coded loop in the service.
Index `user → TreeMap<date, events>`	Conflict checks read only one user's one day; range queries use `subMap`.	Scanning all events each time.
Sweep over boundaries for free slots	Exact, simple, no minute-by-minute loops; works for any N.	Checking every 15 or 30 minutes — misses odd times like 11:30 and is slower.
Service methods `synchronized`	Check-then-book must be atomic; the store's lists are not thread-safe on their own.	Per-user locks — faster, but an event locks many users, so you need lock ordering; mention it as a scale-up.

Java implementation

```java
// File: lld/ch21/calendar/TimeSlot.java
package lld.ch21.calendar;

import java.time.LocalTime;

/** Half-open interval [start, end) inside one day. 14:00-15:00 and 15:00-16:00 do not overlap. */
public record TimeSlot(LocalTime start, LocalTime end) {
    public TimeSlot {
        if (!start.isBefore(end)) throw new IllegalArgumentException("start must be before end: " + start + "-" +
end);
    }
    public boolean overlaps(TimeSlot o) { return start.isBefore(o.end) && o.start.isBefore(end); }
    public boolean contains(TimeSlot o) { return !o.start.isBefore(start) && !o.end.isAfter(end); }
    @Override public String toString() { return start + "-" + end; }
}
```

```java
// File: lld/ch21/calendar/User.java
package lld.ch21.calendar;

public record User(String name, TimeSlot workingHours) { }
```

```java
// File: lld/ch21/calendar/Team.java
package lld.ch21.calendar;

import java.util.List;

/** Members are kept in onboarding order; representative selection walks this order. */
public record Team(String name, List<User> members) {
    public Team { members = List.copyOf(members); }
}
```

```java
// File: lld/ch21/calendar/Event.java
package lld.ch21.calendar;

import java.time.LocalDate;
import java.util.List;
import java.util.Set;

/**
 * One concrete occurrence. `attendees` is the resolved list of users that the event blocks
 * (direct users + the representatives chosen from each team).
 */
public record Event(String id, String name, LocalDate date, TimeSlot slot, Set<String> directUsers,
                    Set<String> teams, int representativesPerTeam, List<String> attendees, String seriesId) {
    public Event { attendees = List.copyOf(attendees); }
    public boolean blocks(String user) { return attendees.contains(user); }
    @Override public String toString() { return name + " " + date + " " + slot + " " + attendees; }
}
```

```java
// File: lld/ch21/calendar/Frequency.java
package lld.ch21.calendar;

import java.time.LocalDate;

public enum Frequency {
    DAILY { LocalDate next(LocalDate d) { return d.plusDays(1); } },
    WEEKLY { LocalDate next(LocalDate d) { return d.plusWeeks(1); } };

    abstract LocalDate next(LocalDate d);
}
```

```java
// File: lld/ch21/calendar/CalendarException.java
package lld.ch21.calendar;

public class CalendarException extends RuntimeException {
    public CalendarException(String message) { super(message); }
}
```

```java
// File: lld/ch21/calendar/EventStore.java
package lld.ch21.calendar;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Collection;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.TreeMap;
import java.util.concurrent.ConcurrentHashMap;

/** Repository: events by id and an index user -> date -> events. Not thread-safe on its own: CalendarService guards it. */
public class EventStore {
    private final Map<String, Event> byId = new ConcurrentHashMap<>();
    private final Map<String, TreeMap<LocalDate, List<Event>>> byUser = new ConcurrentHashMap<>();

    public void save(Event e) {
        byId.put(e.id(), e);
        for (String u : e.attendees())
            byUser.computeIfAbsent(u, k -> new TreeMap<>()).computeIfAbsent(e.date(), d -> new ArrayList<>()).add(e);
    }

    public void remove(Event e) {
        byId.remove(e.id());
        for (String u : e.attendees()) {
            List<Event> day = byUser.getOrDefault(u, new TreeMap<>()).get(e.date());
            if (day != null) day.removeIf(x -> x.id().equals(e.id()));
        }
    }

    public Optional<Event> find(String id) { return Optional.ofNullable(byId.get(id)); }

    public List<Event> onDay(String user, LocalDate date) {
        return List.copyOf(byUser.getOrDefault(user, new TreeMap<>()).getOrDefault(date, List.of()));
    }

    public List<Event> between(String user, LocalDate from, LocalDate to) {
        List<Event> out = new ArrayList<>();
        byUser.getOrDefault(user, new TreeMap<>()).subMap(from, true, to, true).values().forEach(out::addAll);
        return out;
    }

    public Collection<Event> all() { return List.copyOf(byId.values()); }
}
```

```java
// File: lld/ch21/calendar/RepresentativeSelector.java
package lld.ch21.calendar;

import java.util.List;
import java.util.function.Predicate;

/** Strategy: which N free members represent a team. */
public interface RepresentativeSelector {
    List<User> choose(Team team, int n, Predicate<User> isFree);

    /** Default: first N free members in team order (deterministic, easy to explain). */
    static RepresentativeSelector firstAvailable() {
        return (team, n, isFree) -> team.members().stream().filter(isFree).limit(n).toList();
    }
}
```

```java
// File: lld/ch21/calendar/CalendarService.java
package lld.ch21.calendar;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.util.ArrayList;
import java.util.Collection;
import java.util.Comparator;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.TreeSet;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;

public class CalendarService {
    private final Map<String, User> users = new ConcurrentHashMap<>();
    private final Map<String, Team> teams = new ConcurrentHashMap<>();
    private final Map<String, String> teamOf = new ConcurrentHashMap<>();   // user -> team
    private final EventStore store = new EventStore();
    private final RepresentativeSelector selector;
    private final AtomicInteger seq = new AtomicInteger();

    public CalendarService(RepresentativeSelector selector) { this.selector = selector; }

    public User createUser(String name, LocalTime workStart, LocalTime workEnd) {
        User u = new User(name, new TimeSlot(workStart, workEnd));
        if (users.putIfAbsent(name, u) != null) throw new CalendarException("User already exists: " + name);
        return u;
    }

    public synchronized Team createTeam(String name, String... memberNames) {
        if (teams.containsKey(name)) throw new CalendarException("Team already exists: " + name);
        List<User> members = new ArrayList<>();
        for (String m : memberNames) {
            User u = user(m);
            if (teamOf.containsKey(m)) throw new CalendarException(m + " already belongs to team " + teamOf.get(m));
            members.add(u);
        }
        members.forEach(u -> teamOf.put(u.name(), name));
        Team t = new Team(name, members);
        teams.put(name, t);
        return t;
    }

    /** synchronized: the check-then-book sequence must be atomic or two events could grab the same person. */
    public synchronized Event createEvent(String name, Set<String> userNames, Set<String> teamNames,
                                           int reps, LocalDate date, TimeSlot slot) {
        Event e = plan(name, userNames, teamNames, reps, date, slot, null, null);
        store.save(e);
        return e;
    }

    /** Bonus: recurring event. All occurrences are validated first; nothing is saved unless all fit. */
    public synchronized List<Event> createRecurring(String name, Set<String> userNames, Set<String> teamNames, int
reps,
                                                     TimeSlot slot, LocalDate from, LocalDate to, Frequency freq) {
        String series = "S" + seq.incrementAndGet();
        List<Event> planned = new ArrayList<>();
        for (LocalDate d = from; !d.isAfter(to); d = freq.next(d)) {
            planned.add(plan(name + "@" + d, userNames, teamNames, reps, d, slot, series, null));
        }
        planned.forEach(store::save);
        return planned;
    }

    /** Bonus: move an event. We try the new time ignoring the event itself; on failure nothing changes. */
    public synchronized Event updateTime(String eventId, LocalDate date, TimeSlot slot) {
        Event old = store.find(eventId).orElseThrow(() -> new CalendarException("No such event " + eventId));
        Event moved = plan(old.name(), old.directUsers(), old.teams(), old.representativesPerTeam(),
                date, slot, old.seriesId(), old);          // keeps old.id(); throws if the new time does not fit
```

```
        store.remove(old);
        store.save(moved);
        return moved;
    }

    /** Events a user is blocked for between two date-times (includes team-representative bookings). */
    public synchronized List<Event> eventsFor(String userName, LocalDateTime from, LocalDateTime to) {
        user(userName);
        return store.between(userName, from.toLocalDate(), to.toLocalDate()).stream()
                .filter(e -> LocalDateTime.of(e.date(), e.slot().end()).isAfter(from)
                          && LocalDateTime.of(e.date(), e.slot().start()).isBefore(to))
                .sorted(Comparator.comparing(Event::date).thenComparing(e -> e.slot().start()))
                .toList();
    }

    /**
     * Free slots on a day where every listed user is free AND each listed team has at least `reps`
     * free members. Sweep: cut the day at every boundary (working-hour edges, event edges),
     * test each small piece, then merge adjacent good pieces.
     */
    public synchronized List<TimeSlot> freeSlots(Set<String> userNames, Set<String> teamNames, int reps, LocalDate
date) {
        List<User> direct = userNames.stream().map(this::user).toList();
        List<Team> tms = teamNames.stream().map(this::team).toList();
        List<User> everyone = new ArrayList<>(direct);
        tms.forEach(t -> everyone.addAll(t.members()));

        TreeSet<LocalTime> cuts = new TreeSet<>();
        for (User u : everyone) {
            cuts.add(u.workingHours().start());
            cuts.add(u.workingHours().end());
            for (Event e : store.onDay(u.name(), date)) { cuts.add(e.slot().start()); cuts.add(e.slot().end()); }
        }
        List<TimeSlot> result = new ArrayList<>();
        LocalTime prev = null;
        for (LocalTime cut : cuts) {
            if (prev != null) {
                TimeSlot piece = new TimeSlot(prev, cut);
                boolean ok = direct.stream().allMatch(u -> isFree(u, date, piece, null))
                        && tms.stream().allMatch(t -> t.members().stream()
                                .filter(m -> isFree(m, date, piece, null)).count() >= reps);
                if (ok) {
                    int last = result.size() - 1;
                    if (last >= 0 && result.get(last).end().equals(piece.start()))
                        result.set(last, new TimeSlot(result.get(last).start(), piece.end()));
                    else result.add(piece);
                }
            }
            prev = cut;
        }
        return result;
    }

    // ------------------------------------------------------------------ helpers

    private Event plan(String name, Set<String> userNames, Set<String> teamNames, int reps,
                       LocalDate date, TimeSlot slot, String series, Event ignore) {
        if (userNames.isEmpty() && teamNames.isEmpty()) throw new CalendarException("Event needs participants");
        if (!teamNames.isEmpty() && reps < 1) throw new CalendarException("Representatives must be >= 1");
        Set<String> attendees = new LinkedHashSet<>();
        for (String n : new TreeSet<>(userNames)) {          // sorted => deterministic order
            User u = user(n);
            if (!isFree(u, date, slot, ignore))
                throw new CalendarException(name + ": " + n + " is not available " + date + " " + slot);
            attendees.add(n);
        }
        for (String tn : new TreeSet<>(teamNames)) {
            Team t = team(tn);
            List<User> chosen = selector.choose(t, reps,
                    m -> !attendees.contains(m.name()) && isFree(m, date, slot, ignore));
            if (chosen.size() < reps)
                throw new CalendarException(name + ": team " + tn + " has only " + chosen.size()
                        + " free member(s), needs " + reps);
            chosen.forEach(m -> attendees.add(m.name()));
```

```
        }
        String id = ignore != null ? ignore.id() : "E" + seq.incrementAndGet();
        return new Event(id, name, date, slot, Set.copyOf(userNames), Set.copyOf(teamNames), reps,
                List.copyOf(attendees), series);
    }

    private boolean isFree(User u, LocalDate date, TimeSlot slot, Event ignore) {
        if (!u.workingHours().contains(slot)) return false;
        return store.onDay(u.name(), date).stream()
                .filter(e -> ignore == null || !e.id().equals(ignore.id()))
                .noneMatch(e -> e.slot().overlaps(slot));
    }

    private User user(String n) {
        User u = users.get(n);
        if (u == null) throw new CalendarException("Unknown user " + n);
        return u;
    }

    private Team team(String n) {
        Team t = teams.get(n);
        if (t == null) throw new CalendarException("Unknown team " + n);
        return t;
    }

    public synchronized Collection<Event> allEvents() { return store.all(); }
}
```

```java
// File: lld/ch21/calendar/Main.java
package lld.ch21.calendar;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.util.List;
import java.util.Set;
import java.util.function.Supplier;

/** Replays the sample test cases from the problem statement, then the bonus features. */
public class Main {
    public static void main(String[] args) {
        CalendarService cal = new CalendarService(RepresentativeSelector.firstAvailable());
        LocalDate today = LocalDate.of(2026, 9, 28), tomorrow = today.plusDays(1);

        System.out.println("== 1. onboard users");
        cal.createUser("A", t("10:00"), t("19:00"));
        cal.createUser("B", t("09:30"), t("17:30"));
        cal.createUser("C", t("11:30"), t("18:30"));
        cal.createUser("D", t("10:00"), t("18:00"));
        cal.createUser("E", t("11:00"), t("19:30"));
        cal.createUser("F", t("11:00"), t("18:30"));
        System.out.println("  6 users created");

        System.out.println("== 2. teams");
        cal.createTeam("T1", "C", "E");
        cal.createTeam("T2", "B", "D", "F");
        System.out.println("  T1=[C,E] T2=[B,D,F]");
        attempt(() -> cal.createTeam("T3", "A", "C"));

        System.out.println("== 3. events");
        attempt(() -> cal.createEvent("Event1", Set.of("A"), Set.of("T1"), 2, tomorrow, slot("14:00", "15:00")));
        attempt(() -> cal.createEvent("Event2", Set.of("C"), Set.of(), 0, tomorrow, slot("14:00", "15:00")));
        attempt(() -> cal.createEvent("Event3", Set.of(), Set.of("T1", "T2"), 2, today, slot("15:00", "16:00")));
        attempt(() -> cal.createEvent("Event4", Set.of("A"), Set.of("T2"), 1, today, slot("15:00", "16:00")));
        attempt(() -> cal.createEvent("Event5", Set.of("A", "F"), Set.of(), 0, today, slot("10:00", "11:00")));

        System.out.println("== 4. events in a time range (today 10:00 .. tomorrow 17:00)");
        LocalDateTime from = LocalDateTime.of(today, t("10:00")), to = LocalDateTime.of(tomorrow, t("17:00"));
        for (String u : List.of("A", "B", "F"))
            System.out.println("  " + u + ": " + cal.eventsFor(u, from, to).stream().map(Event::name).toList());

        System.out.println("== 5. free slots today for A + T1 (1 representative)");
        System.out.println("  " + cal.freeSlots(Set.of("A"), Set.of("T1"), 1, today));
        System.out.println("  free slots today for B + D: " + cal.freeSlots(Set.of("B", "D"), Set.of(), 0, today));

        System.out.println("== bonus: update event time");
        Event e1 = cal.allEvents().stream().filter(e -> e.name().equals("Event1")).findFirst().orElseThrow();
        attempt(() -> cal.updateTime(e1.id(), tomorrow, slot("18:30", "19:00")));   // needs 2 of T1, but C stops at
18:30
        attempt(() -> cal.updateTime(e1.id(), tomorrow, slot("16:00", "17:00")));

        System.out.println("== bonus: recurring daily stand-up (A + D), 3 days");
        attempt(() -> cal.createRecurring("Standup", Set.of("A", "D"), Set.of(), 0, slot("10:00", "10:15"),
                today, today.plusDays(2), Frequency.DAILY));
        attempt(() -> cal.createRecurring("Sync", Set.of("A"), Set.of(), 0, slot("15:30", "16:30"),
                today, today.plusDays(2), Frequency.DAILY));   // clashes with Event4 today -> whole series rejected
        System.out.println("  A today..+2: " + cal.eventsFor("A", LocalDateTime.of(today, t("00:00")),
                LocalDateTime.of(today.plusDays(2), t("23:59"))).stream().map(Event::name).toList());
    }

    static LocalTime t(String s) { return LocalTime.parse(s); }
    static TimeSlot slot(String a, String b) { return new TimeSlot(t(a), t(b)); }

    static void attempt(Supplier<?> s) {
        try { System.out.println("  ok: " + s.get()); }
        catch (RuntimeException e) { System.out.println("  FAILED: " + e.getMessage()); }
    }
}
```

Output:

```
== 1. onboard users
  6 users created
== 2. teams
  T1=[C,E] T2=[B,D,F]
  FAILED: C already belongs to team T1
== 3. events
  ok: Event1 2026-09-29 14:00-15:00 [A, C, E]
  FAILED: Event2: C is not available 2026-09-29 14:00-15:00
  ok: Event3 2026-09-28 15:00-16:00 [C, E, B, D]
  ok: Event4 2026-09-28 15:00-16:00 [A, F]
  FAILED: Event5: F is not available 2026-09-28 10:00-11:00
== 4. events in a time range (today 10:00 .. tomorrow 17:00)
  A: [Event4, Event1]
  B: [Event3]
  F: [Event4]
== 5. free slots today for A + T1 (1 representative)
  [11:00-15:00, 16:00-19:00]
  free slots today for B + D: [10:00-15:00, 16:00-17:30]
== bonus: update event time
  FAILED: Event1: team T1 has only 1 free member(s), needs 2
  ok: Event1 2026-09-29 16:00-17:00 [A, C, E]
== bonus: recurring daily stand-up (A + D), 3 days
  ok: [Standup@2026-09-28 2026-09-28 10:00-10:15 [A, D], Standup@2026-09-29 2026-09-29 10:00-10:15 [A, D],
Standup@2026-09-30 2026-09-30 10:00-10:15 [A, D]]
  FAILED: Sync@2026-09-28: A is not available 2026-09-28 15:30-16:30
  A today..+2: [Standup@2026-09-28, Event4, Standup@2026-09-29, Event1, Standup@2026-09-30]
```

Checking against the statement: Event2 fails (C is in Event1), Event5 fails (F starts at 11:00), user A sees Event4 and Event1, and user B sees Event3 — because T2's first two free members (B, D) represented it in Event3, which left F as the only free T2 member for Event4. The update to 18:30–19:00 is refused because Event1 needs both T1 members and C stops at 18:30; the old booking stays. The "Sync" series is rejected as a whole because its first day clashes with Event4.

Concurrency & edge cases

- **Double booking under concurrency:** all mutating and reading methods are `synchronized` on the service, so the "is free?" check and the save are one atomic step.
- **Edge cases handled:** start ≥ end, unknown user/team, user already in a team, empty participant list, N < 1 with teams, direct attendee also in an invited team (not double-counted), event outside working hours, recurring series with any clash (nothing saved), moving an event onto a slot where it would conflict with *itself* (we ignore the event being moved).
- **Time zones:** out of scope; everything is local time on one day. Real calendars store instants (exact moments, in UTC — the world reference time zone) plus the user's zone.
- **Multi-day events:** the statement fixes start and end on the same day; extension would split an event into per-day `TimeSlot`s.

Extensions & follow-up questions

- **Invites with accept/decline?** Add `InviteStatus {PENDING, ACCEPTED, DECLINED}` per attendee; a declined attendee no longer blocks the slot.
- **Fair representative choice?** A `LeastBusySelector` that sorts free members by number of events this week.
- **Recurring events at scale?** Store one *rule* (like the iCalendar RRULE — the standard text format for "repeats every Tuesday until June") and expand occurrences lazily (only when asked) for the queried range instead of materialising (creating and storing) every occurrence.
- **Rooms?** Treat a room like a user with "working hours" = building hours; the same availability logic applies.
- **Faster free-slot search for big teams?** For each team, a sweep counting free members per piece is O(members × events); with many members, use a count array or segment tree (a tree that answers "how many in this range?" quickly) over minutes.

Real-world example

Google Calendar's "Find a time" and Outlook's Scheduling Assistant lay attendees' busy blocks side by side and highlight times when everyone is free — the same intersection this sweep computes. Both products store

recurring meetings as a rule plus exceptions (following the iCalendar RRULE standard) instead of thousands of copies.

In the interview

Start by drawing a timeline with two users' working hours and events, and show that free time is "working time minus busy time, intersected". Explain half-open intervals in one sentence. Code `TimeSlot`, `plan()` and `createEvent` first — that satisfies the first three requirements. Then `eventsFor`, then the sweep. Mention the sample inconsistency. Bonus features fall out of `plan()` almost for free, which is the design win to point at.

Self-check

- **Q:** Why does `plan()` take an `ignore` event? → **A:** When moving an event, its current booking must not count as a conflict with its new time.
- **Q:** Why is the recurring series all-or-nothing easy here? → **A:** We plan every occurrence first (no side effects) and save only if all plans succeed.
- **Q:** What is the overlap test for half-open intervals? → **A:** `a.start < b.end && b.start < a.end`.
- **Q:** Why a `TreeMap` per user? → **A:** Dates are ordered, so a range query is `subMap(from, to)` instead of a full scan.

TL;DR

Half-open `TimeSlot`s, per-user day index, a side-effect-free `plan()` reused by create/recurring/update, a pluggable representative selector, and a boundary sweep for free slots. Ask about the sample that contradicts the rules.

21.4 Design a LeetCode-like Coding Platform — "Coding Blox" (Flipkart)

Problem & clarifying questions

A Flipkart SDE-2 (Software Development Engineer, level 2: a mid-level role) machine-coding statement (kumaransg/LLD, `leetcode-lld-flipkart-coding-blox`). Coding Blox lets users sign up, create contests, and take part in contests hosted by others. Questions and contests each have a level — LOW, MEDIUM or HIGH — and a contest of a level uses **all questions of that level** (the question list is independent of the contest). The platform is driven by commands:

Command	Meaning
`CreateUser <name>`	Register a user with a default score of **1500**.
`CreateQuestion <level> <score>`	Add a question; ids auto-increment from 1.
`ListQuestion [level]`	All questions, or only that level.
`CreateContest <name> <level> <creator>`	Creator automatically attends; contest ids auto-increment from 1.
`ListContest [level]`	All contests, or only that level.
`AttendContest <contestId> <user>`	Register for a contest.
`RunContest <contestId> <creator>`	Only the creator may run it. Simulate which questions each participant solved (random), sum their scores (`contestPoints`), then update ratings: LOW → `score + (points − 50)`, MEDIUM → `score + (points − 30)`, HIGH → `score + points`.
`LeaderBoard <asc/desc>`	Users with scores, sorted.
Bonus `ContestHistory <contestId>`	Per-contest leaderboard: user, points, solved question ids.
Bonus `WithdrawContest <contestId> <user>`	Leave before the contest runs; the creator cannot withdraw.

Expectations in the statement: create the sample data yourself, demo through a driver or tests, modular OO (object-oriented) design with interfaces, extensible, graceful edge-case handling, DRY ("don't repeat yourself": each piece of logic written once).

Question	Answer used
How many questions does a user "solve" in a run?	Random 1..all questions of the contest level. The randomness sits behind a `SolveSimulator` interface and is **seeded** for a reproducible demo.
Can a contest run twice?	No — second run is an error, so ratings are never applied twice.
Can someone attend after the run?	No.
Leaderboard tie-break?	By name.
Can a contest be created if no question of that level exists?	No (it would be empty).

Requirements

Functional: all commands above, including both bonus commands. **Non-functional:** commands from a built-in input list; errors printed, processing continues; scoring rule and solving logic replaceable; rating update safe if two contests finish at once.

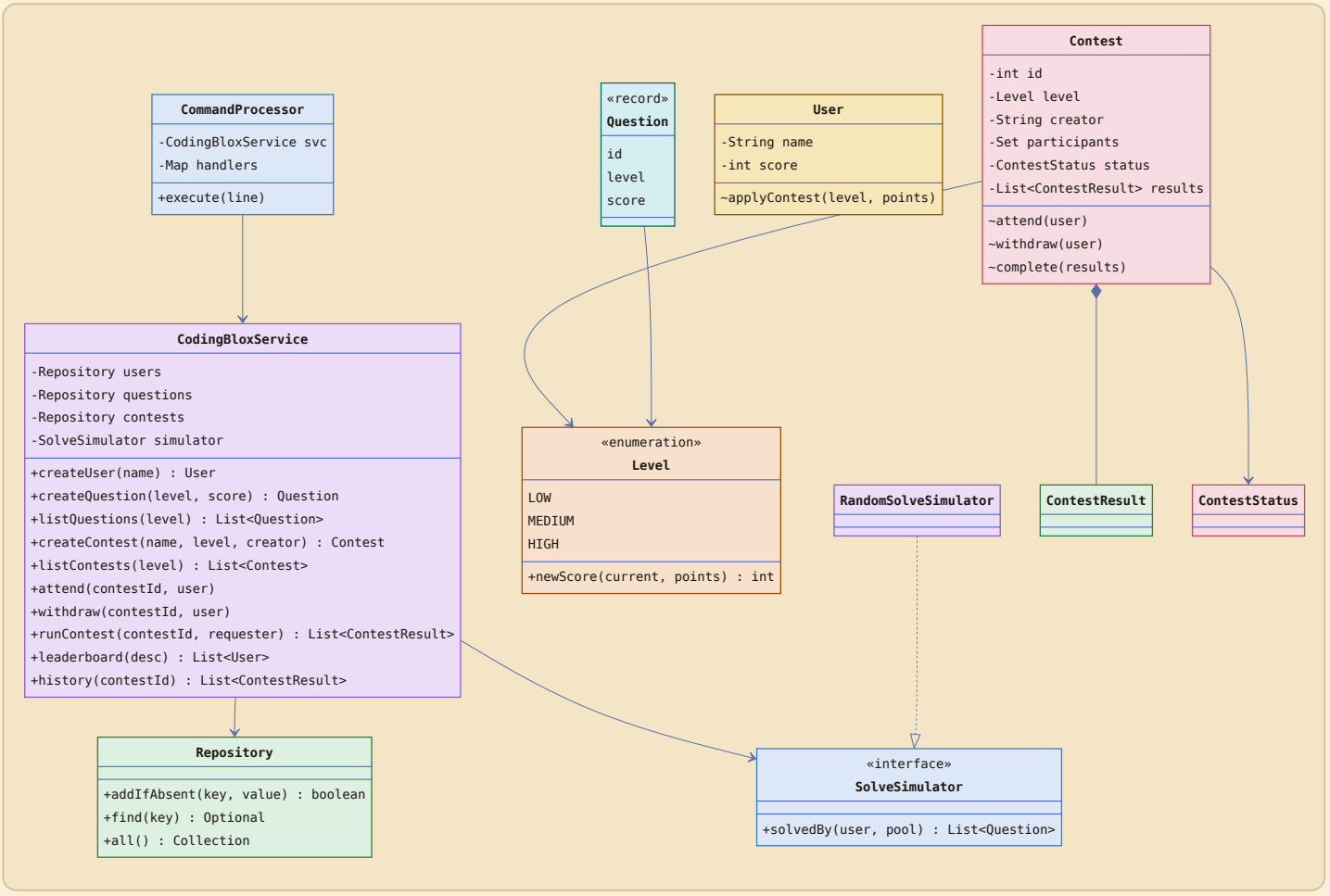
Use cases

Actor	Use cases
User	sign up, attend/withdraw, view leaderboard, view contest history
Contest creator (a user)	create contest, run contest
Admin / problem setter	create and list questions
Judge (system actor, simulated)	decides which questions each participant solved

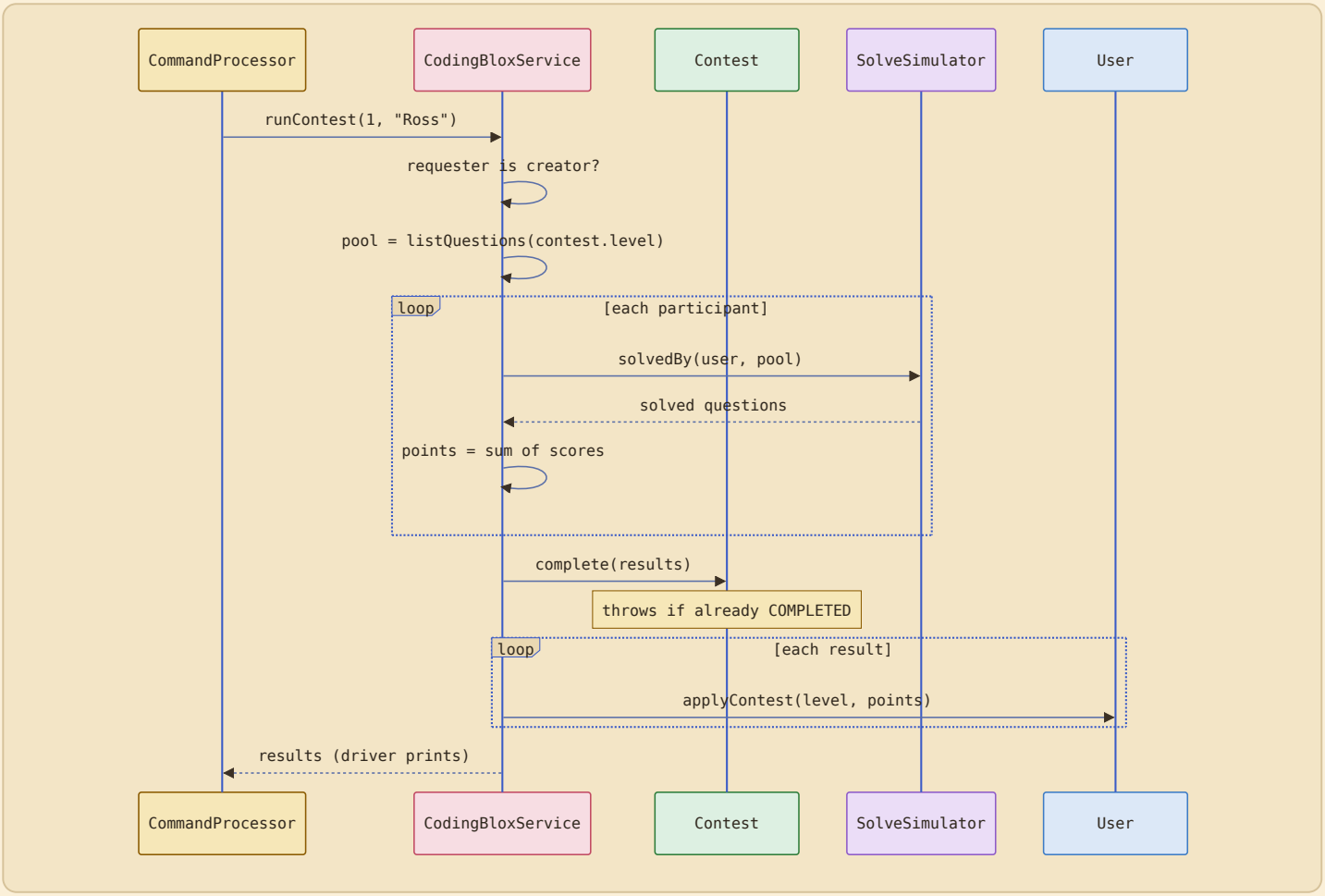
Core entities

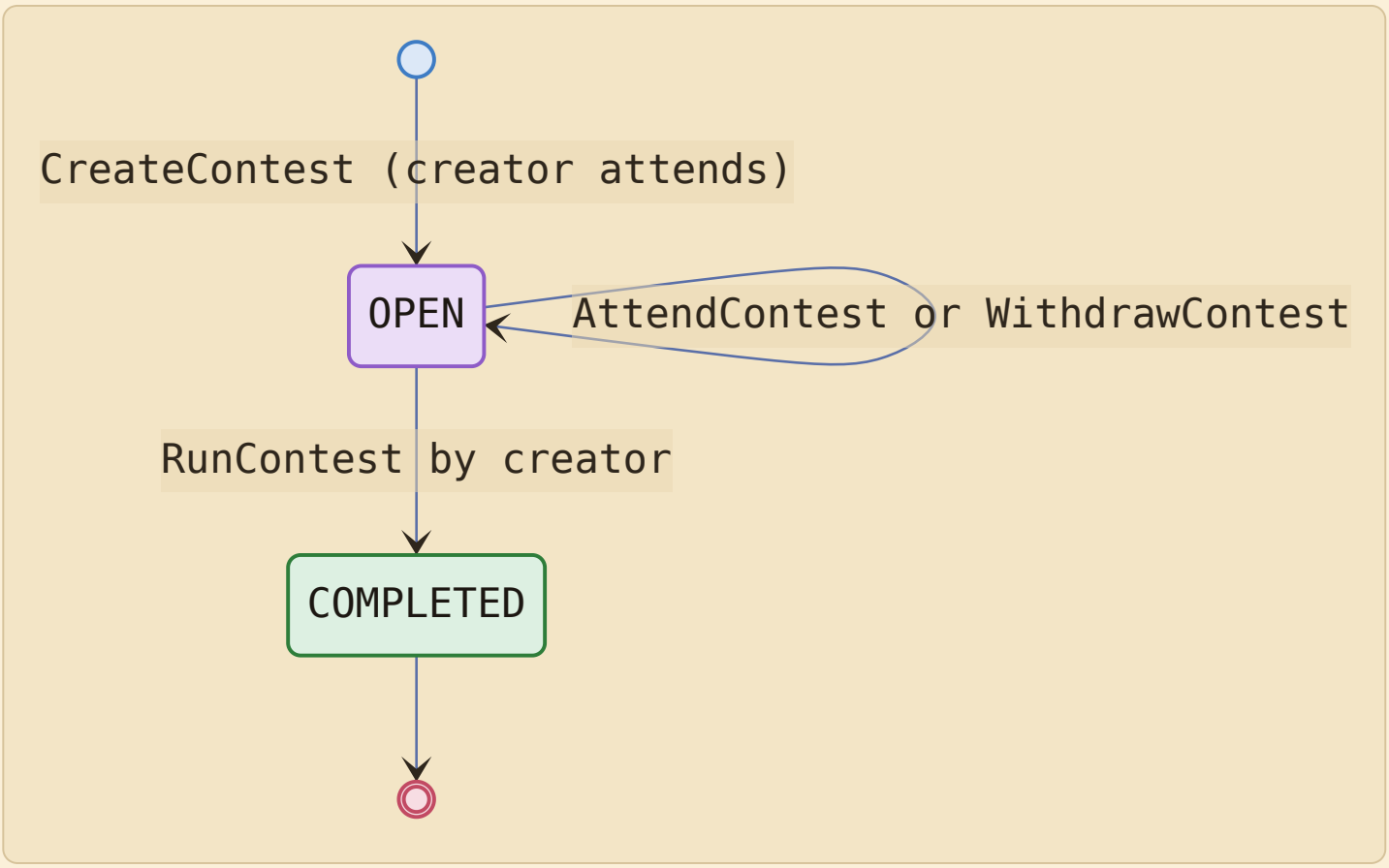
Class	Kind	Responsibility
`Level`	enum	LOW/MEDIUM/HIGH, each with its penalty; `newScore(current, points)` implements the rating rule.
`User`	class	Name + score; `applyContest` updates the score atomically.
`Question`	record	id, level, score.
`Contest`	class	id, name, level, creator, participants, status, results; guards "open only" rules.
`ContestStatus`	enum	OPEN, COMPLETED.
`ContestResult`	record	user, points, solved ids.
`SolveSimulator` / `RandomSolveSimulator`	interface / class	Strategy for "who solved what".
`Repository<K,V>`	generic class	In-memory storage reused for users, questions and contests.
`CodingBloxService`	class	Business rules for every command.
`CommandProcessor`	class	Parses (splits and reads) a line, dispatches to (calls) a handler from a map, prints output/errors.
`BloxException`	exception	Readable failure.

Class diagram



Key flows


CommandProcessor
CodingBloxService
Contest
SolveSimulator
User
runContest(1, "Ross")
requester is creator?
pool = listQuestions(contest.level)
loop
[each participant]
solvedBy(user, pool)
solved questions
points = sum of scores
complete(results)
throws if already COMPLETED
loop
[each result]
applyContest(level, points)
results (driver prints)
CommandProcessor
CodingBloxService
Contest
SolveSimulator
User


CreateContest (creator attends)
OPEN
AttendContest or WithdrawContest
RunContest by creator
COMPLETED

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Command name → handler map	Command	Adding a command is one `put`; no giant `switch`. Parsing and printing stay out of the service.	Parsing inside the service.
Scoring rule inside `Level` enum	Strategy (each enum constant carries its own data)	The rule depends only on the level; a new level brings its own rule.	`if (level == LOW) ... else if ...` in `runContest`.
`SolveSimulator` interface	Strategy	Randomness is replaceable: seeded for demo, scripted for tests, a real judge later.	`new Random()` inside the service — untestable.
`Contest.complete` checks status	Entity guards invariant	"Run once" enforced where the state lives, so scores are never applied twice.	Checking in the driver.
Generic `Repository<K,V>`	Repository	Three stores, one class (DRY — the statement asks for it).	Three copy-pasted maps with lookup code.
Results computed, then `complete`, then ratings	Ordering for safety	If `complete` throws (already run), no rating has changed yet.	Updating ratings inside the loop.

Java implementation

```java
// File: lld/ch21/codingblox/Level.java
package lld.ch21.codingblox;

/** Difficulty of a question or a contest. Each contest level carries its own scoring rule. */
public enum Level {
    LOW(50), MEDIUM(30), HIGH(0);

    private final int penalty;
    Level(int penalty) { this.penalty = penalty; }

    /** newScore = currentScore + (contestPoints - penalty), exactly as the statement defines. */
    public int newScore(int current, int contestPoints) { return current + contestPoints - penalty; }

    public static Level parse(String s) {
        try { return valueOf(s.toUpperCase()); }
        catch (IllegalArgumentException e) { throw new BloxException("Unknown level: " + s); }
    }
}
```

```java
// File: lld/ch21/codingblox/User.java
package lld.ch21.codingblox;

public class User {
    public static final int DEFAULT_SCORE = 1500;
    private final String name;
    private int score = DEFAULT_SCORE;

    public User(String name) { this.name = name; }
    public String name() { return name; }
    public synchronized int score() { return score; }
    /** Read-modify-write under one lock, so two contests finishing together cannot lose an update. */
    synchronized void applyContest(Level level, int points) { score = level.newScore(score, points); }
}
```

```java
// File: lld/ch21/codingblox/Question.java
package lld.ch21.codingblox;

public record Question(int id, Level level, int score) {
    @Override public String toString() { return "Q" + id + "(" + level + "," + score + ")"; }
}
```

```java
// File: lld/ch21/codingblox/Contest.java
package lld.ch21.codingblox;

import java.util.ArrayList;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Set;

public class Contest {
    private final int id;
    private final String name;
    private final Level level;
    private final String creator;
    private final Set<String> participants = new LinkedHashSet<>();
    private final List<ContestResult> results = new ArrayList<>();
    private ContestStatus status = ContestStatus.OPEN;

    Contest(int id, String name, Level level, String creator) {
        this.id = id; this.name = name; this.level = level; this.creator = creator;
        participants.add(creator);                      // the creator always attends
    }

    public int id() { return id; }
    public String name() { return name; }
    public Level level() { return level; }
    public String creator() { return creator; }
    public synchronized ContestStatus status() { return status; }
    public synchronized List<String> participants() { return List.copyOf(participants); }
    public synchronized List<ContestResult> results() { return List.copyOf(results); }

    synchronized void attend(String user) {
        requireOpen();
        if (!participants.add(user)) throw new BloxException(user + " already registered for contest " + id);
    }

    synchronized void withdraw(String user) {
        requireOpen();
        if (user.equals(creator)) throw new BloxException("Creator cannot withdraw from contest " + id);
        if (!participants.remove(user)) throw new BloxException(user + " is not in contest " + id);
    }

    synchronized void complete(List<ContestResult> r) {
        requireOpen();
        results.addAll(r);
        status = ContestStatus.COMPLETED;
    }

    private void requireOpen() {
        if (status != ContestStatus.OPEN) throw new BloxException("Contest " + id + " has already run");
    }

    @Override public String toString() { return id + " " + name + " " + level + " by " + creator + " [" + status() +
"]"; }
}
```

```java
// File: lld/ch21/codingblox/ContestStatus.java
package lld.ch21.codingblox;

public enum ContestStatus { OPEN, COMPLETED }
```

```java
// File: lld/ch21/codingblox/ContestResult.java
package lld.ch21.codingblox;

import java.util.List;

public record ContestResult(String user, int points, List<Integer> solvedIds) { }
```

```java
// File: lld/ch21/codingblox/BloxException.java
package lld.ch21.codingblox;

public class BloxException extends RuntimeException {
    public BloxException(String message) { super(message); }
}
```

```java
// File: lld/ch21/codingblox/Repository.java
package lld.ch21.codingblox;

import java.util.Collection;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;

/** Tiny generic in-memory repository reused for users, questions and contests. */
public class Repository<K, V> {
    private final Map<K, V> items = new ConcurrentHashMap<>();
    public boolean addIfAbsent(K key, V value) { return items.putIfAbsent(key, value) == null; }
    public Optional<V> find(K key) { return Optional.ofNullable(items.get(key)); }
    public Collection<V> all() { return List.copyOf(items.values()); }
}
```

```java
// File: lld/ch21/codingblox/SolveSimulator.java
package lld.ch21.codingblox;

import java.util.List;

/** Strategy: decides which questions a user solved. Real system = judge results; demo = seeded random. */
public interface SolveSimulator {
    List<Question> solvedBy(String user, List<Question> pool);
}
```

```java
// File: lld/ch21/codingblox/RandomSolveSimulator.java
package lld.ch21.codingblox;

import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;
import java.util.Random;

public class RandomSolveSimulator implements SolveSimulator {
    private final Random random;
    public RandomSolveSimulator(long seed) { this.random = new Random(seed); }

    @Override public List<Question> solvedBy(String user, List<Question> pool) {
        if (pool.isEmpty()) return List.of();
        List<Question> copy = new ArrayList<>(pool);
        Collections.shuffle(copy, random);
        int k = 1 + random.nextInt(copy.size());          // solves between 1 and all questions
        List<Question> picked = new ArrayList<>(copy.subList(0, k));
        picked.sort(Comparator.comparingInt(Question::id));
        return picked;
    }
}
```

```java
// File: lld/ch21/codingblox/CodingBloxService.java
package lld.ch21.codingblox;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;

public class CodingBloxService {
    private final Repository<String, User> users = new Repository<>();
    private final Repository<Integer, Question> questions = new Repository<>();
    private final Repository<Integer, Contest> contests = new Repository<>();
    private final AtomicInteger questionSeq = new AtomicInteger();
    private final AtomicInteger contestSeq = new AtomicInteger();
    private final SolveSimulator simulator;

    public CodingBloxService(SolveSimulator simulator) { this.simulator = simulator; }

    public User createUser(String name) {
        User u = new User(name);
        if (!users.addIfAbsent(name, u)) throw new BloxException("User already exists: " + name);
        return u;
    }

    public Question createQuestion(Level level, int score) {
        if (score <= 0) throw new BloxException("Score must be positive");
        Question q = new Question(questionSeq.incrementAndGet(), level, score);
        questions.addIfAbsent(q.id(), q);
        return q;
    }

    public List<Question> listQuestions(Level levelOrNull) {
        return questions.all().stream().filter(q -> levelOrNull == null || q.level() == levelOrNull)
                .sorted(Comparator.comparingInt(Question::id)).toList();
    }

    public Contest createContest(String name, Level level, String creator) {
        user(creator);
        if (listQuestions(level).isEmpty()) throw new BloxException("No " + level + " questions to build a contest");
        Contest c = new Contest(contestSeq.incrementAndGet(), name, level, creator);
        contests.addIfAbsent(c.id(), c);
        return c;
    }

    public List<Contest> listContests(Level levelOrNull) {
        return contests.all().stream().filter(c -> levelOrNull == null || c.level() == levelOrNull)
                .sorted(Comparator.comparingInt(Contest::id)).toList();
    }

    public void attend(int contestId, String userName) { user(userName); contest(contestId).attend(userName); }

    public void withdraw(int contestId, String userName) { user(userName); contest(contestId).withdraw(userName); }

    /** Only the creator may run. Every participant gets random solved questions, then scores update. */
    public List<ContestResult> runContest(int contestId, String requester) {
        Contest c = contest(contestId);
        if (!c.creator().equals(requester)) throw new BloxException(requester + " did not create contest " +
contestId);
        List<Question> pool = listQuestions(c.level());
        List<ContestResult> results = new ArrayList<>();
        for (String name : c.participants()) {
            List<Question> solved = simulator.solvedBy(name, pool);
            int points = solved.stream().mapToInt(Question::score).sum();
            results.add(new ContestResult(name, points, solved.stream().map(Question::id).toList()));
        }
        c.complete(results);                                // throws if already run: scores never double-apply
        for (ContestResult r : results) user(r.user()).applyContest(c.level(), r.points());
        return results;
    }

    public List<User> leaderboard(boolean descending) {
        Comparator<User> byScore = Comparator.comparingInt(User::score);
        if (descending) byScore = byScore.reversed();
```

```
        return users.all().stream().sorted(byScore.thenComparing(User::name)).toList();
    }

    public List<ContestResult> history(int contestId) {
        Contest c = contest(contestId);
        if (c.status() != ContestStatus.COMPLETED) throw new BloxException("Contest " + contestId + " has not run
yet");
        return c.results().stream().sorted(Comparator.comparingInt(ContestResult::points).reversed()
                .thenComparing(ContestResult::user)).toList();
    }

    private User user(String n) { return users.find(n).orElseThrow(() -> new BloxException("No such user: " + n)); }
    private Contest contest(int id) { return contests.find(id).orElseThrow(() -> new BloxException("No such contest: "
+ id)); }
}
```

```java
// File: lld/ch21/codingblox/CommandProcessor.java
package lld.ch21.codingblox;

import java.util.Arrays;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.function.Consumer;

/**
 * Driver layer. Parses one text line and dispatches to a handler (Command pattern: name -> action).
 * All printing lives here; the service returns data only.
 */
public class CommandProcessor {
    private final CodingBloxService svc;
    private final Map<String, Consumer<String[]>> handlers = new LinkedHashMap<>();

    public CommandProcessor(CodingBloxService svc) {
        this.svc = svc;
        handlers.put("createuser", a -> out("User " + svc.createUser(arg(a, 1)).name() + " created with score 1500"));
        handlers.put("createquestion", a -> out("Question created: " + svc.createQuestion(Level.parse(arg(a, 1)),
Integer.parseInt(arg(a, 2)))));
        handlers.put("listquestion", a -> svc.listQuestions(a.length > 1 ? Level.parse(a[1]) : null).forEach(q ->
out("  " + q)));
        handlers.put("createcontest", a -> out("Contest created: " + svc.createContest(arg(a, 1), Level.parse(arg(a,
2)), arg(a, 3))));
        handlers.put("listcontest", a -> svc.listContests(a.length > 1 ? Level.parse(a[1]) : null).forEach(c -> out("
" + c)));
        handlers.put("attendcontest", a -> { svc.attend(Integer.parseInt(arg(a, 1)), arg(a, 2)); out(a[2] + "
registered for contest " + a[1]); });
        handlers.put("withdrawcontest", a -> { svc.withdraw(Integer.parseInt(arg(a, 1)), arg(a, 2)); out(a[2] + "
withdrew from contest " + a[1]); });
        handlers.put("runcontest", a -> svc.runContest(Integer.parseInt(arg(a, 1)), arg(a, 2))
                .forEach(r -> out("  " + r.user() + " solved " + r.solvedIds() + " -> " + r.points() + " pts")));
        handlers.put("leaderboard", a -> svc.leaderboard(!a[a.length - 1].equalsIgnoreCase("asc"))
                .forEach(u -> out("  " + u.name() + " : " + u.score())));
        handlers.put("contesthistory", a -> svc.history(Integer.parseInt(arg(a, 1)))
                .forEach(r -> out("  " + r.user() + " : " + r.points() + " " + r.solvedIds())));
    }

    public void execute(String line) {
        System.out.println("> " + line);
        String[] a = Arrays.stream(line.trim().split("\\s+")).map(s -> s.replace("\"", "")).toArray(String[]::new);
        Consumer<String[]> h = handlers.get(a[0].toLowerCase());
        try {
            if (h == null) throw new BloxException("Unknown command: " + a[0]);
            h.accept(a);
        } catch (BloxException | NumberFormatException e) {
            out("ERROR: " + e.getMessage());
        }
    }

    private static String arg(String[] a, int i) {
        if (i >= a.length) throw new BloxException("Missing argument #" + i + " for " + a[0]);
        return a[i];
    }

    private static void out(String s) { System.out.println(s); }
}
```

```java
// File: lld/ch21/codingblox/Main.java
package lld.ch21.codingblox;

import java.util.List;

public class Main {
    static final List<String> SAMPLE_INPUT = List.of(
            "CreateUser Ross", "CreateUser Monica", "CreateUser Joey", "CreateUser Chandler",
            "CreateQuestion \"LOW\" 10", "CreateQuestion LOW 20", "CreateQuestion LOW 30",
            "CreateQuestion MEDIUM 40", "CreateQuestion MEDIUM 50", "CreateQuestion HIGH 100",
            "ListQuestion LOW",
            "CreateContest \"diwali_contest\" LOW \"Ross\"",
            "CreateContest holi_contest MEDIUM Monica",
            "ListContest",
            "AttendContest 1 Monica", "AttendContest 1 Joey", "AttendContest 1 Chandler",
            "AttendContest 1 Joey",                  // duplicate
            "WithdrawContest 1 Chandler",
            "WithdrawContest 1 Ross",                // creator cannot withdraw
            "RunContest 1 Monica",                   // not the creator
            "RunContest 1 Ross",
            "RunContest 1 Ross",                     // already run
            "AttendContest 1 Chandler",              // too late
            "ContestHistory 1",
            "AttendContest 2 Chandler",
            "RunContest 2 Monica",
            "LeaderBoard score desc",
            "LeaderBoard asc",
            "CreateContest x HIGH Rachel",           // unknown user
            "Dance 1");                              // unknown command

    public static void main(String[] args) {
        CommandProcessor cli = new CommandProcessor(new CodingBloxService(new RandomSolveSimulator(42)));
        SAMPLE_INPUT.forEach(cli::execute);
    }
}
```

Output:

```
> CreateUser Ross
User Ross created with score 1500
> CreateUser Monica
User Monica created with score 1500
> CreateUser Joey
User Joey created with score 1500
> CreateUser Chandler
User Chandler created with score 1500
> CreateQuestion "LOW" 10
Question created: Q1(LOW,10)
> CreateQuestion LOW 20
Question created: Q2(LOW,20)
> CreateQuestion LOW 30
Question created: Q3(LOW,30)
> CreateQuestion MEDIUM 40
Question created: Q4(MEDIUM,40)
> CreateQuestion MEDIUM 50
Question created: Q5(MEDIUM,50)
> CreateQuestion HIGH 100
Question created: Q6(HIGH,100)
> ListQuestion LOW
  Q1(LOW,10)
  Q2(LOW,20)
  Q3(LOW,30)
> CreateContest "diwali_contest" LOW "Ross"
Contest created: 1 diwali_contest LOW by Ross [OPEN]
> CreateContest holi_contest MEDIUM Monica
Contest created: 2 holi_contest MEDIUM by Monica [OPEN]
> ListContest
  1 diwali_contest LOW by Ross [OPEN]
  2 holi_contest MEDIUM by Monica [OPEN]
> AttendContest 1 Monica
Monica registered for contest 1
> AttendContest 1 Joey
Joey registered for contest 1
> AttendContest 1 Chandler
Chandler registered for contest 1
> AttendContest 1 Joey
ERROR: Joey already registered for contest 1
> WithdrawContest 1 Chandler
Chandler withdrew from contest 1
> WithdrawContest 1 Ross
ERROR: Creator cannot withdraw from contest 1
> RunContest 1 Monica
ERROR: Monica did not create contest 1
> RunContest 1 Ross
  Ross solved [2] -> 20 pts
  Monica solved [1, 2] -> 30 pts
  Joey solved [1, 2] -> 30 pts
> RunContest 1 Ross
ERROR: Contest 1 has already run
> AttendContest 1 Chandler
ERROR: Contest 1 has already run
> ContestHistory 1
  Joey : 30 [1, 2]
  Monica : 30 [1, 2]
  Ross : 20 [2]
> AttendContest 2 Chandler
Chandler registered for contest 2
> RunContest 2 Monica
  Monica solved [4, 5] -> 90 pts
  Chandler solved [4] -> 40 pts
> LeaderBoard score desc
  Monica : 1540
  Chandler : 1510
  Joey : 1480
  Ross : 1470
> LeaderBoard asc
  Ross : 1470
  Joey : 1480
  Chandler : 1510
  Monica : 1540
> CreateContest x HIGH Rachel
```

```
ERROR: No such user: Rachel
> Dance 1
ERROR: Unknown command: Dance
```

Checking the arithmetic: contest 1 is LOW, so each participant gets `points − 50`: Ross 1500 + (20 − 50) = 1470, Joey and Monica 1500 + (30 − 50) = 1480. Contest 2 is MEDIUM (`points − 30`): Monica 1480 + (90 − 30) = 1540, Chandler 1500 + (40 − 30) = 1510. The leaderboard matches. (Which questions are "solved" comes from the seeded simulator — change the seed and you get different but equally valid results.)

Concurrency & edge cases

- **Concurrent commands:** repositories are `ConcurrentHashMap`s; `Contest` methods are `synchronized`; `User.applyContest` does its read-modify-write under the user's lock, so two contests finishing together cannot lose a rating update.
- **Double run:** `Contest.complete` throws if already completed; ratings are updated *after* it succeeds.
- **Known gap (say it):** `runContest` copies the participant list, simulates, then calls `complete`. A user who attends in that short window is registered but gets no result. Fix: add a `RUNNING` status and have `Contest` switch to it and hand back the participant list in one `synchronized` step, so no one can attend or withdraw once the run has started.
- **Edge cases handled:** duplicate user, unknown user/contest/level, non-positive question score, contest with no questions of its level, duplicate attend, attend/withdraw after the run, creator withdraw, non-creator run, missing arguments, non-numeric ids, unknown command.

Extensions & follow-up questions

- **Pick a fixed number of questions per contest?** A `QuestionSelectionStrategy` (random k of level, or by tags) called at contest creation; store the chosen ids in the contest.
- **Real submissions?** Replace `SolveSimulator` with a judge that returns verdicts per submission; add `Submission(user, question, verdict, time)` and score by time/penalty like ICPC (the International Collegiate Programming Contest).
- **Elo-style ratings?** (Elo is the chess rating system: beating a stronger player earns more points.) A `RatingStrategy` interface taking all participants' results (ratings depend on opponents), selected per contest type.
- **Scheduled contests?** Add `startTime/endTime`, a `SCHEDULED → RUNNING → COMPLETED` state machine, and a scheduler that triggers `run`.
- **Leaderboard at scale?** Keep a `TreeSet` (a set that keeps its items sorted) sorted by (score, name) or a Redis sorted set; update on each rating change instead of sorting on every query.

Real-world example

LeetCode's weekly contests publish a live ranking during the contest, and afterwards each participant's contest rating is recalculated from their rank relative to others — a more elaborate version of the per-level rule in this statement. HackerRank and Codeforces follow the same shape: contests reference a pool of problems, participants register, and ratings change only after the contest is finalised.

In the interview

Paste the sample commands into the driver list first. Build `CreateUser`, `CreateQuestion`, `CreateContest`, `AttendContest`, `RunContest`, `LeaderBoard` in that order, running after each. Point out the three design moves: command map, scoring rule on the enum, and randomness behind an interface (with a seed). Then bonus commands — both are tiny because `Contest` already stores results and guards its status.

Self-check

- **Q:** Why must `RunContest` update ratings only after marking the contest complete? → **A:** If the contest had already run, `complete` throws before any rating changes — no double application.
- **Q:** How would you add a `VERY_HIGH` level with bonus +20? → **A:** Add an enum constant with its own rule (e.g. a negative penalty); nothing else changes.
- **Q:** How do you make `RunContest` deterministic in tests? → **A:** Inject a scripted `SolveSimulator` (or a seeded random one).
- **Q:** Where is "creator always attends" enforced? → **A:** In the `Contest` constructor.

TL;DR

Command map in the driver, rules in the service, invariants in `Contest`, scoring on the `Level` enum, randomness behind a seeded `SolveSimulator`. Complete first, then update ratings.

21.5 Design a Property Listing Site — "PropertyHunt" (ClearTrip)

Problem & clarifying questions

A ClearTrip machine-coding statement (kumaransg/LLD, `PropertyHunt`). The folder holds two versions: a text `readme.md` and a longer version as screenshots in `Question Image/`; the summary below follows the longer one, which adds ViewSold, the range rules and the "shortlist *other users'* property" wording. Build a console application (a text-only program run in a terminal) for a property platform where users can: **register**; **list a property** they want to sell or rent; **view their own listings**; **shortlist** other users' properties; **view shortlisted** properties (sold ones still appear, with status SOLD); **search** available properties with any combination of **location** (case-insensitive — capital and small letters treated the same — exact match), **price range**, **listing type** (sell/rent), **size range**, **number of rooms** (1BHK/2BHK/3BHK — bedroom, hall, kitchen: 1BHK is one bedroom), and **sort by** price or size (ascending); **mark sold** (only the owner); and **view sold** properties. Ranges: `X-Y` inclusive; a single `X` means "at least X" for size and "at most X" for price. **Bonus:** sizes in sq ft / sq m / sq yd (buyer and seller may use different units); **nearby locations** on a listing, with nearby matches shown last; prices like `40000`, `40k`, `60L`, `2Cr` (k = thousand; L = lakh = 100,000; Cr = crore = 10,000,000 — the Indian number units); comma-separated values for location and rooms in search. The statement asks for in-memory storage, graceful errors ("Property does not exist"), and a clean, extensible OO design.

Question	Answer used
Input format?	One command per line, fields separated by `
Can I shortlist my own property?	No — the screenshot version says users shortlist *others'* properties. (The text readme's sample has Jack shortlisting his own listing; that is exactly the kind of conflict to ask about.)
Can I shortlist a sold property?	No; but a property sold *after* I shortlisted it stays in my list with status SOLD.
Units internally?	Price in whole rupees (`long`), size in square feet (`double`).
Sort with location?	Exact location matches first, nearby matches after, then the chosen sort.
Mark sold — record buyer and price?	The screenshot sample writes `marksold - 2, xyz, 50000` (property, buyer, price). We store only the status; keeping buyer and sale price is a small extension (a `Sale` record).

The sample search (`bellandur, 30000-50000, rent, ...`) returns a *sarjapur* property. In the screenshot version that is consistent: the sarjapur listing declares `nearby bellandur|kadubeesanahalli|marathahalli`, so it is a nearby match. The text readme's sample drops the `nearby` field, and there the result only makes sense if you know about the bonus. Our demo lists sarjapur with the same three nearby localities, and the search returns it.

Requirements

Functional: RegisterUser, ListProperty, Search, Shortlist, ViewShortlisted, ViewListed, MarkSold, ViewSold — plus all four bonus items. **Non-functional:** search filters combine freely; new filters or units without rewriting search; clear errors; safe under concurrent use.

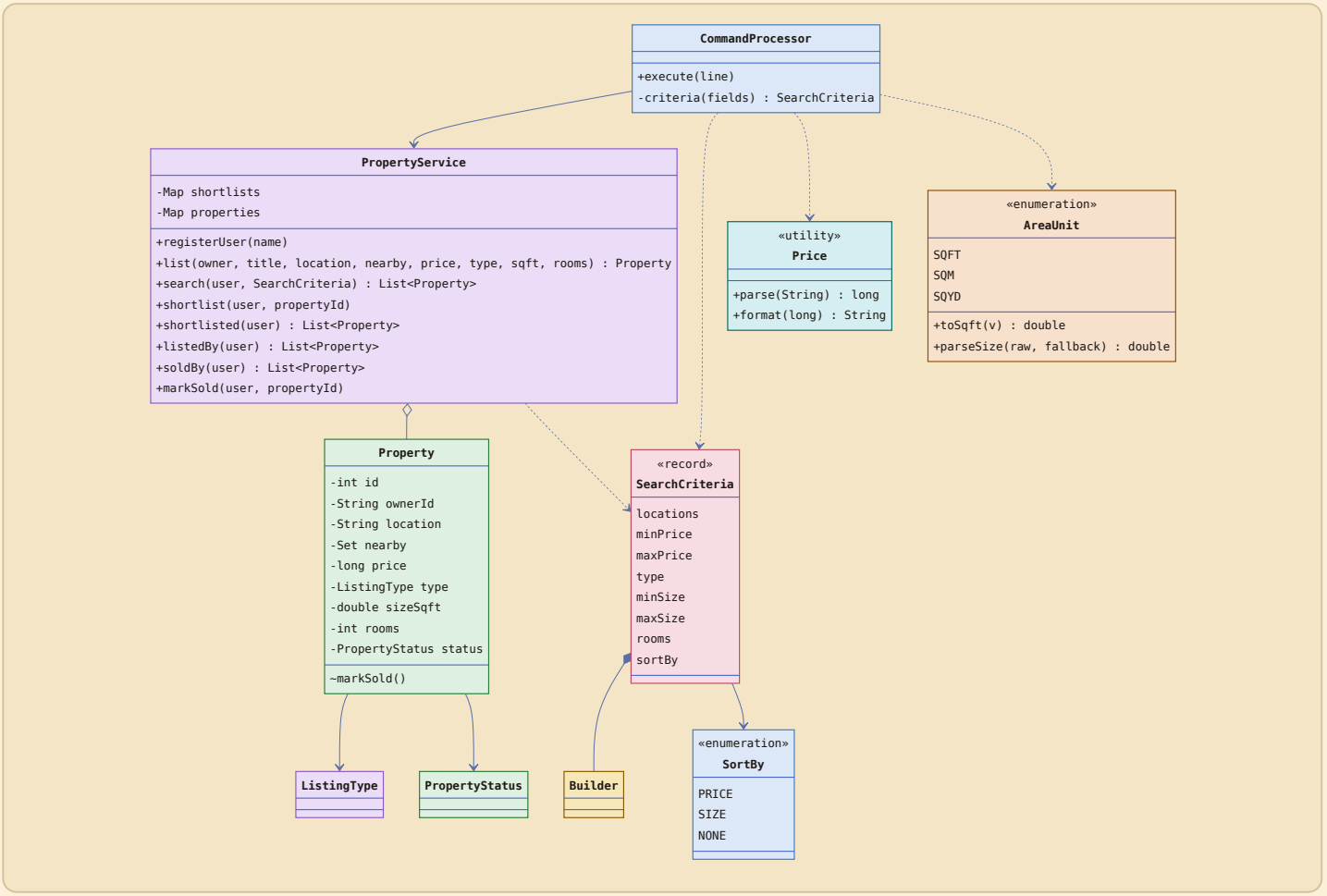
Use cases

Actor	Use cases
Seller / landlord	list property, view listed, mark sold, view sold
Buyer / tenant	search, shortlist, view shortlisted
(Any user can be both — roles are per property, not per account: the owner of a property is its seller.)	

Core entities

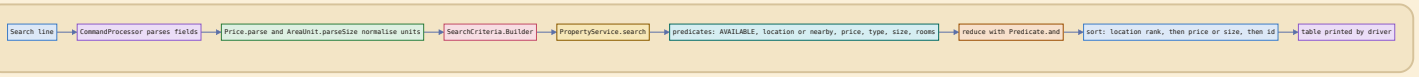
Class	Kind	Responsibility
`Property`	class	Listing data, owner, nearby locations, status; `markSold` guards double sale.
`ListingType`, `PropertyStatus`, `SortBy`	enums	SELL/RENT, AVAILABLE/SOLD, sort comparators.
`Price`	utility	Parse `40k/60L/2Cr/1.5Cr` → rupees; format back.
`AreaUnit`	enum	SQFT/SQM/SQYD with conversion to square feet; parses "1500 sq ft", "70 sqm".
`SearchCriteria` (+ `Builder`)	record	All-optional search parameters.
`PropertyService`	class	Register, list, search, shortlist, mark sold, views; owner-vs-buyer rules.
`CommandProcessor`	class	Parses `
`HuntException`	exception	Readable errors.

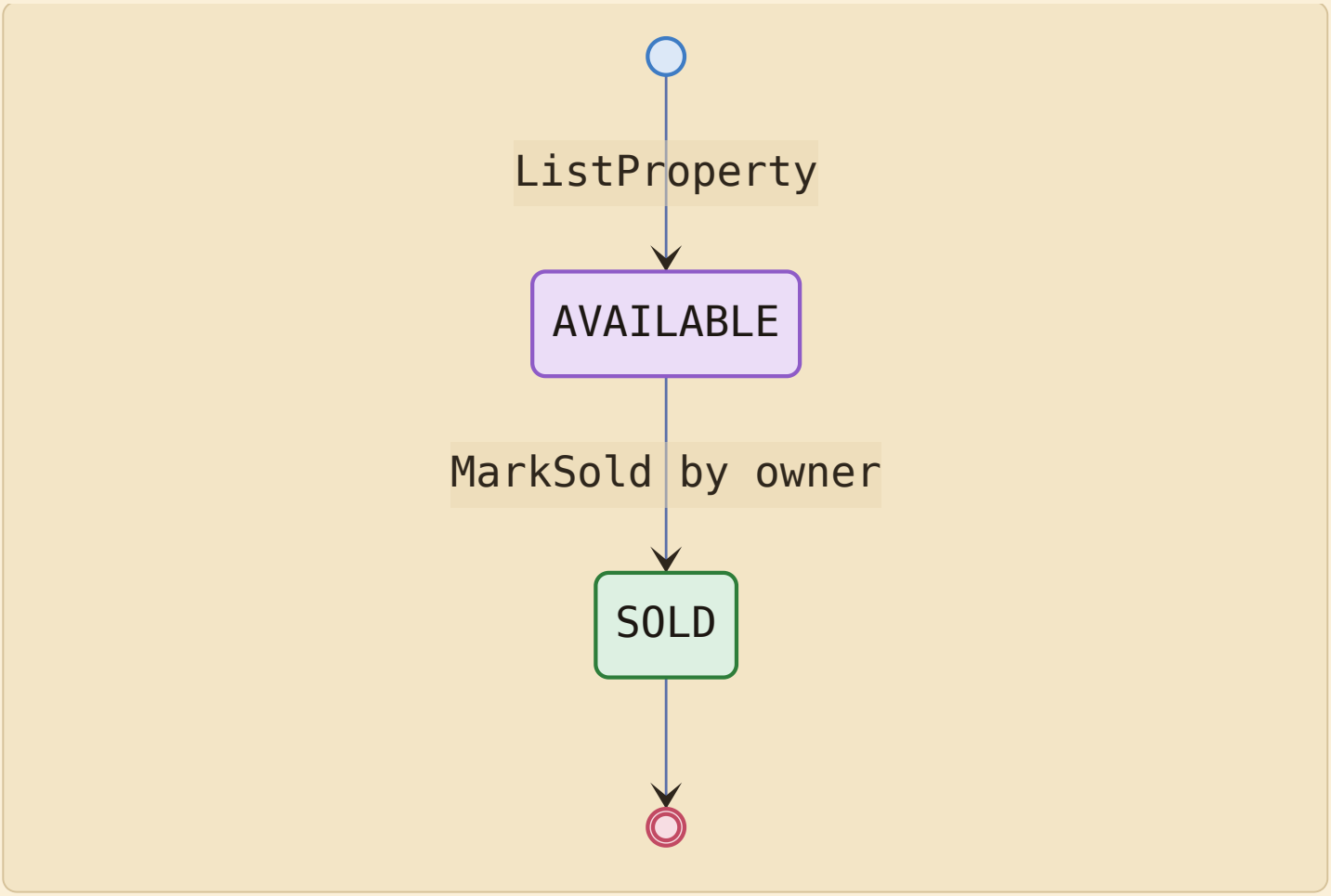
Class diagram



Key flows

Search = build criteria → list of predicates → one combined predicate → sort:





Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
All-optional `SearchCriteria` with a Builder	Builder	Eight optional fields; a constructor call with eight nulls is unreadable.	Telescoping constructors (a chain of constructors, each taking one more parameter) / overloads (several methods with the same name but different parameters).
Each criterion becomes a `Predicate<Property>`, reduced with `and`	Specification	Any combination works; a new filter is one line; no nested `if` pyramid.	One `if` per combination.
`SortBy` enum of comparators	Strategy	Adding "sort by rooms" is one constant.	`switch` inside search.
Normalise at the boundary (rupees, sq ft)	—	Business code compares plain numbers; units only matter while parsing.	Storing strings like "90L" and comparing them.
Location rank 0/1/2	—	Exact matches first, nearby second, others filtered out — one small function serves filtering *and* ordering.	Two separate searches merged.
Shortlist stored per user as a set of ids	—	View shortlisted shows current status (SOLD appears automatically).	Copying property objects into the shortlist — they would go stale.

Java implementation

```java
// File: lld/ch21/propertyhunt/Price.java
package lld.ch21.propertyhunt;

import java.math.BigDecimal;
import java.util.Locale;

/** Parses and prints Indian-style prices: 40000, 40k, 60L, 2Cr, 1.5Cr. Stored as whole rupees. */
public final class Price {
    private Price() { }

    public static long parse(String raw) {
        String s = raw.trim().toLowerCase(Locale.ROOT).replace(",", "").replace(" ", "");
        long mult = 1;
        if (s.endsWith("cr")) { mult = 10_000_000L; s = s.substring(0, s.length() - 2); }
        else if (s.endsWith("l")) { mult = 100_000L; s = s.substring(0, s.length() - 1); }
        else if (s.endsWith("k")) { mult = 1_000L; s = s.substring(0, s.length() - 1); }
        try {
            return new BigDecimal(s).multiply(BigDecimal.valueOf(mult)).longValueExact();
        } catch (ArithmeticException | NumberFormatException e) {
            throw new HuntException("Bad price: " + raw);
        }
    }

    public static String format(long rupees) {
        if (rupees >= 10_000_000L) return shift(rupees, 7) + "Cr";
        if (rupees >= 100_000L) return shift(rupees, 5) + "L";
        if (rupees >= 1_000L) return shift(rupees, 3) + "k";
        return Long.toString(rupees);
    }

    /** Exact decimal shift, e.g. shift(25000000, 7) = "2.5" - no double rounding. */
    private static String shift(long rupees, int places) {
        return BigDecimal.valueOf(rupees).movePointLeft(places).stripTrailingZeros().toPlainString();
    }
}
```

```java
// File: lld/ch21/propertyhunt/AreaUnit.java
package lld.ch21.propertyhunt;

import java.util.Locale;

/** Every size is converted to square feet internally so buyer and seller units can differ. */
public enum AreaUnit {
    SQFT(1.0), SQM(10.7639), SQYD(9.0);

    private final double sqftPerUnit;
    AreaUnit(double f) { this.sqftPerUnit = f; }

    public double toSqft(double v) { return v * sqftPerUnit; }

    /** Accepts "sqft", "sq ft", "sqm", "sq meter", "sqyd", "sq yard" ... (spaces ignored). */
    public static AreaUnit parse(String raw) {
        String s = raw.toLowerCase(Locale.ROOT).replace(" ", "").replace(".", "");
        if (s.isEmpty() || s.startsWith("sqf") || s.equals("ft")) return SQFT;
        if (s.startsWith("sqm")) return SQM;
        if (s.startsWith("sqy")) return SQYD;
        throw new HuntException("Unknown size unit: " + raw);
    }

    /** "1800 sqft" -> 1800.0 ; "200 sqyd" -> 1800.0 ; a bare number uses `fallback`. */
    public static double parseSize(String raw, AreaUnit fallback) {
        String s = raw.trim();
        int i = 0;
        while (i < s.length() && (Character.isDigit(s.charAt(i)) || s.charAt(i) == '.')) i++;
        if (i == 0) throw new HuntException("Bad size: " + raw);
        double value = Double.parseDouble(s.substring(0, i));
        String unit = s.substring(i).trim();
        return (unit.isEmpty() ? fallback : parse(unit)).toSqft(value);
    }
}
```

```java
// File: lld/ch21/propertyhunt/ListingType.java
package lld.ch21.propertyhunt;

public enum ListingType {
    SELL, RENT;

    public static ListingType parse(String s) {
        return switch (s.trim().toLowerCase()) {
            case "sell", "sale" -> SELL;
            case "rent" -> RENT;
            default -> throw new HuntException("Listing type must be sell or rent: " + s);
        };
    }
}
```

```java
// File: lld/ch21/propertyhunt/PropertyStatus.java
package lld.ch21.propertyhunt;

public enum PropertyStatus { AVAILABLE, SOLD }
```

```java
// File: lld/ch21/propertyhunt/Property.java
package lld.ch21.propertyhunt;

import java.util.Set;

public class Property {
    private final int id;
    private final String ownerId;
    private final String title;
    private final String location;          // stored lower-case for case-insensitive match
    private final Set<String> nearby;       // bonus: nearby locations, lower-case
    private final long price;
    private final ListingType type;
    private final double sizeSqft;
    private final int rooms;
    private PropertyStatus status = PropertyStatus.AVAILABLE;

    Property(int id, String ownerId, String title, String location, Set<String> nearby, long price,
             ListingType type, double sizeSqft, int rooms) {
        this.id = id; this.ownerId = ownerId; this.title = title; this.location = location;
        this.nearby = Set.copyOf(nearby); this.price = price; this.type = type;
        this.sizeSqft = sizeSqft; this.rooms = rooms;
    }

    public int id() { return id; }
    public String ownerId() { return ownerId; }
    public String title() { return title; }
    public String location() { return location; }
    public Set<String> nearby() { return nearby; }
    public long price() { return price; }
    public ListingType type() { return type; }
    public double sizeSqft() { return sizeSqft; }
    public int rooms() { return rooms; }
    public synchronized PropertyStatus status() { return status; }

    synchronized void markSold() {
        if (status == PropertyStatus.SOLD) throw new HuntException("Property " + id + " is already sold");
        status = PropertyStatus.SOLD;
    }
}
```

```java
// File: lld/ch21/propertyhunt/SortBy.java
package lld.ch21.propertyhunt;

import java.util.Comparator;

public enum SortBy {
    PRICE(Comparator.comparingLong(Property::price)),
    SIZE(Comparator.comparingDouble(Property::sizeSqft)),
    NONE(Comparator.comparingInt(Property::id));

    final Comparator<Property> comparator;
    SortBy(Comparator<Property> c) { this.comparator = c.thenComparingInt(Property::id); }

    public static SortBy parse(String s) { return s == null ? NONE : valueOf(s.trim().toUpperCase()); }
}
```

```java
// File: lld/ch21/propertyhunt/SearchCriteria.java
package lld.ch21.propertyhunt;

import java.util.Set;

/** Every field is optional (null = "don't care"). Built with the nested Builder. */
public record SearchCriteria(Set<String> locations, Long minPrice, Long maxPrice, ListingType type,
                             Double minSize, Double maxSize, Set<Integer> rooms, SortBy sortBy) {

    public static Builder builder() { return new Builder(); }

    public static final class Builder {
        private Set<String> locations; private Long minPrice, maxPrice; private ListingType type;
        private Double minSize, maxSize; private Set<Integer> rooms; private SortBy sortBy = SortBy.NONE;

        public Builder locations(Set<String> l) { locations = l; return this; }
        public Builder price(Long min, Long max) { minPrice = min; maxPrice = max; return this; }
        public Builder type(ListingType t) { type = t; return this; }
        public Builder size(Double min, Double max) { minSize = min; maxSize = max; return this; }
        public Builder rooms(Set<Integer> r) { rooms = r; return this; }
        public Builder sortBy(SortBy s) { sortBy = s; return this; }
        public SearchCriteria build() {
            return new SearchCriteria(locations, minPrice, maxPrice, type, minSize, maxSize, rooms, sortBy);
        }
    }
}
```

```java
// File: lld/ch21/propertyhunt/HuntException.java
package lld.ch21.propertyhunt;

public class HuntException extends RuntimeException {
    public HuntException(String message) { super(message); }
}
```

```java
// File: lld/ch21/propertyhunt/PropertyService.java
package lld.ch21.propertyhunt;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.function.Predicate;

public class PropertyService {
    private final Map<String, Set<Integer>> shortlists = new ConcurrentHashMap<>();  // user -> ids (the user
registry)
    private final Map<Integer, Property> properties = new ConcurrentHashMap<>();
    private final AtomicInteger seq = new AtomicInteger();

    public void registerUser(String name) {
        if (shortlists.putIfAbsent(name, ConcurrentHashMap.newKeySet()) != null)
            throw new HuntException("User already registered: " + name);
    }

    public Property list(String owner, String title, String location, Set<String> nearby, long price,
                         ListingType type, double sizeSqft, int rooms) {
        requireUser(owner);
        if (price <= 0 || sizeSqft <= 0 || rooms <= 0) throw new HuntException("Price, size and rooms must be
positive");
        Property p = new Property(seq.incrementAndGet(), owner, title, location.trim().toLowerCase(),
                lower(nearby), price, type, sizeSqft, rooms);
        properties.put(p.id(), p);
        return p;
    }

    /** Only AVAILABLE properties. Exact location matches first, nearby matches last, then the chosen sort. */
    public List<Property> search(String user, SearchCriteria c) {
        requireUser(user);
        Set<String> locs = c.locations() == null ? null : lower(c.locations());
        List<Predicate<Property>> filters = new ArrayList<>();
        filters.add(p -> p.status() == PropertyStatus.AVAILABLE);
        if (locs != null) filters.add(p -> locationRank(p, locs) < 2);
        if (c.minPrice() != null) filters.add(p -> p.price() >= c.minPrice());
        if (c.maxPrice() != null) filters.add(p -> p.price() <= c.maxPrice());
        if (c.type() != null) filters.add(p -> p.type() == c.type());
        if (c.minSize() != null) filters.add(p -> p.sizeSqft() >= c.minSize());
        if (c.maxSize() != null) filters.add(p -> p.sizeSqft() <= c.maxSize());
        if (c.rooms() != null) filters.add(p -> c.rooms().contains(p.rooms()));
        Predicate<Property> all = filters.stream().reduce(x -> true, Predicate::and);

        Comparator<Property> order = locs == null ? c.sortBy().comparator
                : Comparator.<Property>comparingInt(p -> locationRank(p, locs)).thenComparing(c.sortBy().comparator);
        return properties.values().stream().filter(all).sorted(order).toList();
    }

    public void shortlist(String user, int propertyId) {
        requireUser(user);
        Property p = get(propertyId);
        if (p.ownerId().equals(user)) throw new HuntException("You cannot shortlist your own property");
        if (p.status() == PropertyStatus.SOLD) throw new HuntException("Property " + propertyId + " is already sold");
        if (!shortlists.get(user).add(propertyId)) throw new HuntException("Property " + propertyId + " already
shortlisted");
    }

    /** Includes sold ones; the caller prints status so the buyer sees "SOLD". */
    public List<Property> shortlisted(String user) {
        requireUser(user);
        return
shortlists.get(user).stream().map(properties::get).sorted(Comparator.comparingInt(Property::id)).toList();
    }

    public List<Property> listedBy(String user) {
        requireUser(user);
```

```
        return properties.values().stream().filter(p -> p.ownerId().equals(user))
                .sorted(Comparator.comparingInt(Property::id)).toList();
    }

    public List<Property> soldBy(String user) {
        return listedBy(user).stream().filter(p -> p.status() == PropertyStatus.SOLD).toList();
    }

    public void markSold(String user, int propertyId) {
        requireUser(user);
        Property p = get(propertyId);
        if (!p.ownerId().equals(user)) throw new HuntException("Only the owner can mark property " + propertyId + "
sold");
        p.markSold();
    }

    /** 0 = exact location match, 1 = nearby match, 2 = no match. */
    private static int locationRank(Property p, Set<String> locs) {
        if (locs.contains(p.location())) return 0;
        return p.nearby().stream().anyMatch(locs::contains) ? 1 : 2;
    }

    private Property get(int id) {
        Property p = properties.get(id);
        if (p == null) throw new HuntException("Property does not exist: " + id);
        return p;
    }

    private void requireUser(String u) { if (!shortlists.containsKey(u)) throw new HuntException("User not registered:
" + u); }

    private static Set<String> lower(Set<String> s) {
        Set<String> out = new LinkedHashSet<>();
        s.forEach(x -> out.add(x.trim().toLowerCase()));
        return out;
    }
}
```

```java
// File: lld/ch21/propertyhunt/CommandProcessor.java
package lld.ch21.propertyhunt;

import java.util.Arrays;
import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;

/**
 * Parses '|'-separated commands. Inside a field, ',' separates multiple values (bonus requirement).
 * "null" or an empty field means "not given".
 */
public class CommandProcessor {
    private final PropertyService svc;
    public CommandProcessor(PropertyService svc) { this.svc = svc; }

    public void execute(String line) {
        System.out.println("-> " + line);
        String[] f = Arrays.stream(line.split("\\|", -1)).map(String::trim).toArray(String[]::new);
        try {
            switch (f[0].toLowerCase()) {
                case "registeruser" -> { svc.registerUser(f[1]); out("User registered successfully."); }
                case "listproperty" -> {
                    Set<String> nearby = f.length > 8 && given(f[8]) ? csv(f[8]) : Set.of();
                    Property p = svc.list(f[1], f[2], f[3], nearby, Price.parse(f[4]), ListingType.parse(f[5]),
                            AreaUnit.parseSize(f[6], AreaUnit.SQFT), rooms(f[7]));
                    out("Property listed successfully. id=" + p.id());
                }
                case "search" -> printTable(svc.search(f[1], criteria(f)), false);
                case "shortlist" -> { svc.shortlist(f[1], Integer.parseInt(f[2])); out("Shortlisted property"); }
                case "viewshortlisted" -> printTable(svc.shortlisted(f[1]), true);
                case "viewlisted" -> printTable(svc.listedBy(f[1]), true);
                case "viewsold" -> printTable(svc.soldBy(f[1]), true);
                case "marksold" -> { svc.markSold(f[1], Integer.parseInt(f[2])); out("Property marked SOLD"); }
                default -> out("ERROR: unknown command " + f[0]);
            }
        } catch (HuntException | IllegalArgumentException | ArrayIndexOutOfBoundsException e) {
            out("ERROR: " + e.getMessage());
        }
    }

    /** Search|user|locations|priceRange|type|sizeRange|rooms|sortBy */
    private SearchCriteria criteria(String[] f) {
        SearchCriteria.Builder b = SearchCriteria.builder();
        if (given(f[2])) b.locations(csv(f[2]));
        if (given(f[3])) {
            String[] r = f[3].split("-");                       // "X-Y" inclusive, "X" = at most X
            if (r.length == 2) b.price(Price.parse(r[0]), Price.parse(r[1]));
            else b.price(null, Price.parse(r[0]));
        }
        if (given(f[4])) b.type(ListingType.parse(f[4]));
        if (given(f[5])) {
            String[] r = f[5].split("-");                       // "X-Y" inclusive, "X" = at least X
            if (r.length == 2) {
                AreaUnit unit = unitOf(r[1]);                   // "1000-2000 sqft": unit written once
                b.size(AreaUnit.parseSize(r[0], unit), AreaUnit.parseSize(r[1], unit));
            } else b.size(AreaUnit.parseSize(r[0], AreaUnit.SQFT), null);
        }
        if (given(f[6])) b.rooms(csv(f[6]).stream().map(CommandProcessor::rooms).collect(Collectors.toSet()));
        if (f.length > 7 && given(f[7])) b.sortBy(SortBy.parse(f[7]));
        return b.build();
    }

    private static AreaUnit unitOf(String s) {
        String unit = s.replaceAll("[0-9.\\s]", "");
        return unit.isEmpty() ? AreaUnit.SQFT : AreaUnit.parse(unit);
    }

    private static int rooms(String s) { return Integer.parseInt(s.toUpperCase().replace("BHK", "").trim()); }
    private static boolean given(String s) { return !s.isEmpty() && !s.equalsIgnoreCase("null"); }
    private static Set<String> csv(String s) {
        return Arrays.stream(s.split(",")).map(String::trim).filter(x -> !x.isEmpty()).collect(Collectors.toSet());
    }
}
```

```
    private static void printTable(List<Property> ps, boolean withStatus) {
        if (ps.isEmpty()) { out("<- (no properties)"); return; }
        out("<- %-3s %-20s %-10s %-6s %-9s %-5s %-5s%s".formatted("Id", "Title", "Location", "Price", "Size",
                "Rooms", "For", withStatus ? " Status" : ""));
        for (Property p : ps)
            out("<- %-3d %-20s %-10s %-6s %-9s %-5s %-5s%s".formatted(p.id(), p.title(), p.location(),
                    Price.format(p.price()), Math.round(p.sizeSqft()) + "sqft", p.rooms() + "BHK",
                    p.type() == ListingType.SELL ? "Sale" : "Rent", withStatus ? " " + p.status() : ""));
    }

    private static void out(String s) { System.out.println(s); }
}
```

```
// File: lld/ch21/propertyhunt/Main.java
package lld.ch21.propertyhunt;

import java.util.List;

public class Main {
    static final List<String> SAMPLE_INPUT = List.of(
            "RegisterUser|Jack",
            "RegisterUser|Jill",
            "RegisterUser|Jack",
            "ListProperty|Jack|A 3BHK for sale|bellandur|90L|sell|1800 sqft|3",
            "ListProperty|Jack|A 2BHK for rent|sarjapur|40k|rent|1500 sq
ft|2|bellandur,kadubeesanahalli,marathahalli",
            "ListProperty|Jill|Cosy 1BHK|HSR|25k|rent|70 sqm|1BHK",
            "ListProperty|Jill|Villa|whitefield|2.5Cr|sell|400 sqyd|4",
            "Search|Jack|null|null|null|null|null|null",
            "Search|Jack|bellandur|30000-50000|rent|150sqft|2|price",
            "Search|Jack|bellandur|null|null|null|null|price",
            "Search|Jill|null|1Cr|null|1000-2000 sqft|2,3|size",
            "Search|Jill|hsr,sarjapur|null|rent|null|1BHK,2BHK|price",
            "Shortlist|Jill|1",
            "Shortlist|Jill|2",
            "Shortlist|Jill|9",
            "Shortlist|Jack|1",
            "MarkSold|Jill|1",
            "MarkSold|Jack|1",
            "ViewShortlisted|Jill",
            "ViewListed|Jack",
            "ViewSold|Jack",
            "Search|Jill|null|null|sell|null|null|price",
            "ListProperty|Jill|Bad|x|abc|sell|100 sqft|1",
            "Search|Bob|null|null|null|null|null|null");

    public static void main(String[] args) {
        CommandProcessor cli = new CommandProcessor(new PropertyService());
        SAMPLE_INPUT.forEach(cli::execute);
    }
}
```

Output:

```
-> RegisterUser|Jack
User registered successfully.
-> RegisterUser|Jill
User registered successfully.
-> RegisterUser|Jack
ERROR: User already registered: Jack
-> ListProperty|Jack|A 3BHK for sale|bellandur|90L|sell|1800 sqft|3
Property listed successfully. id=1
-> ListProperty|Jack|A 2BHK for rent|sarjapur|40k|rent|1500 sq ft|2|bellandur,kadubeesanahalli,marathahalli
Property listed successfully. id=2
-> ListProperty|Jill|Cosy 1BHK|HSR|25k|rent|70 sqm|1BHK
Property listed successfully. id=3
-> ListProperty|Jill|Villa|whitefield|2.5Cr|sell|400 sqyd|4
Property listed successfully. id=4
-> Search|Jack|null|null|null|null|null|null
<- Id  Title               Location   Price  Size      Rooms For
<- 1   A 3BHK for sale     bellandur  90L    1800sqft  3BHK  Sale
<- 2   A 2BHK for rent     sarjapur   40k    1500sqft  2BHK  Rent
<- 3   Cosy 1BHK           hsr        25k    753sqft   1BHK  Rent
<- 4   Villa               whitefield 2.5Cr  3600sqft  4BHK  Sale
-> Search|Jack|bellandur|30000-50000|rent|150sqft|2|price
<- Id  Title               Location   Price  Size      Rooms For
<- 2   A 2BHK for rent     sarjapur   40k    1500sqft  2BHK  Rent
-> Search|Jack|bellandur|null|null|null|null|price
<- Id  Title               Location   Price  Size      Rooms For
<- 1   A 3BHK for sale     bellandur  90L    1800sqft  3BHK  Sale
<- 2   A 2BHK for rent     sarjapur   40k    1500sqft  2BHK  Rent
-> Search|Jill|null|1Cr|null|1000-2000 sqft|2,3|size
<- Id  Title               Location   Price  Size      Rooms For
<- 2   A 2BHK for rent     sarjapur   40k    1500sqft  2BHK  Rent
<- 1   A 3BHK for sale     bellandur  90L    1800sqft  3BHK  Sale
-> Search|Jill|hsr,sarjapur|null|rent|null|1BHK,2BHK|price
<- Id  Title               Location   Price  Size      Rooms For
<- 3   Cosy 1BHK           hsr        25k    753sqft   1BHK  Rent
<- 2   A 2BHK for rent     sarjapur   40k    1500sqft  2BHK  Rent
-> Shortlist|Jill|1
Shortlisted property
-> Shortlist|Jill|2
Shortlisted property
-> Shortlist|Jill|9
ERROR: Property does not exist: 9
-> Shortlist|Jack|1
ERROR: You cannot shortlist your own property
-> MarkSold|Jill|1
ERROR: Only the owner can mark property 1 sold
-> MarkSold|Jack|1
Property marked SOLD
-> ViewShortlisted|Jill
<- Id  Title               Location   Price  Size      Rooms For   Status
<- 1   A 3BHK for sale     bellandur  90L    1800sqft  3BHK  Sale  SOLD
<- 2   A 2BHK for rent     sarjapur   40k    1500sqft  2BHK  Rent  AVAILABLE
-> ViewListed|Jack
<- Id  Title               Location   Price  Size      Rooms For   Status
<- 1   A 3BHK for sale     bellandur  90L    1800sqft  3BHK  Sale  SOLD
<- 2   A 2BHK for rent     sarjapur   40k    1500sqft  2BHK  Rent  AVAILABLE
-> ViewSold|Jack
<- Id  Title               Location   Price  Size      Rooms For   Status
<- 1   A 3BHK for sale     bellandur  90L    1800sqft  3BHK  Sale  SOLD
-> Search|Jill|null|null|sell|null|null|price
<- Id  Title               Location   Price  Size      Rooms For
<- 4   Villa               whitefield 2.5Cr  3600sqft  4BHK  Sale
-> ListProperty|Jill|Bad|x|abc|sell|100 sqft|1
ERROR: Bad price: abc
-> Search|Bob|null|null|null|null|null|null
ERROR: User not registered: Bob
```

Things to notice: 70 sq m was stored as 753 sq ft and 400 sq yd as 3600 sq ft; the `bellandur` search returns the bellandur listing first and the sarjapur listing (nearby) last; after `MarkSold`, property 1 disappears from searches but still shows as SOLD in Jill's shortlist; bad input produces an error line and the program continues.

Concurrency & edge cases

- **Storage:** `ConcurrentHashMap`s; each user's shortlist is a concurrent set; `Property.markSold` is `synchronized` so two "mark sold" calls cannot both succeed.
- **A search racing a sale** may still show a property that is being sold at that instant — acceptable for listings (eventual consistency: different readers may briefly see different values, but they all catch up soon); a purchase flow would re-check status under a lock.
- **Edge cases handled:** duplicate registration, unregistered user, unknown property, own-property shortlist, duplicate shortlist, shortlisting a sold property, non-owner mark sold, double sale, bad price/size/unit/type, non-positive values, missing fields.

Extensions & follow-up questions

- **Descending sort / multiple sort keys?** `SortBy` holds a comparator; add a direction flag and `.reversed()`, or accept a list of keys and chain `thenComparing`.
- **Fuzzy location ("Bellandur Road")?** Normalise locations to ids through a `LocationResolver` interface (gazetteer / geocoding — a directory of place names / turning an address into map coordinates).
- **"Within 5 km"?** Store latitude/longitude and filter by haversine distance (the standard formula for the distance between two points on a sphere); at scale use a geo-index (a lookup structure for finding things by location: geohash, R-tree) or a search engine with geo queries.
- **Fast search over millions?** Secondary indexes (by location, type, rooms) or Elasticsearch/OpenSearch; price and size as numeric range fields.
- **Notifications on price drop for shortlisted items?** Observer on `Property` price changes → notify users who shortlisted it.

Real-world example

Property portals like 99acres, MagicBricks and NoBroker let you filter by locality, budget, BHK count, buy/rent and area, and often suggest nearby localities when results are few; Indian listings commonly quote prices in lakhs and crores — exactly why the statement asks you to parse `60L` and `2Cr`.

In the interview

Start with the model and units (show `Price.parse` handling `90L`, and `AreaUnit`). Then implement register → list → search with *one* filter → add filters as predicates → sort. Shortlist, mark sold and the views are small after that. Call out the Builder and predicate list as your extensibility story, and the "normalise at the boundary" principle.

Self-check

- **Q:** How does a buyer searching in sq m match a seller who listed in sq yd? → **A:** Both are converted to sq ft at parse time; search compares sq ft.
- **Q:** Why store shortlists as ids rather than objects? → **A:** Status is read live from the property, so SOLD shows without extra updates.
- **Q:** What does a single number mean in the price field? → **A:** "At most X" (for size it means "at least X").
- **Q:** How do you add a "furnished" filter? → **A:** A field on `Property`, an optional field in `SearchCriteria`, and one predicate line in `search`.

TL;DR

Normalise price and size at the edges, turn every optional criterion into a predicate and AND them together, sort by (location rank, chosen key, id), and keep shortlists as ids so status stays live.

21.6 Design a Loan Ledger — "Ledger Co" (Navi)

Problem & clarifying questions

This problem (kumaransg/LLD, `ledger_company_navi`) is the Geektrust "Ledger Co" backend (server-side logic, no screens) challenge; the repository files it under Navi, which has used it as a hiring exercise. Banks lend money to borrowers; borrowers repay in **EMIs** (Equated Monthly Instalments — the same amount every

month) and may also pay a **lump sum** (a one-off extra payment) at any point. The program reads commands and answers balance questions:

Command	Meaning
`LOAN BANK BORROWER P N R`	Borrower takes principal `P` (the amount borrowed) for `N` years at `R`% simple interest (interest charged only on the original principal, never on interest) per year.
`PAYMENT BANK BORROWER LUMP_SUM EMI_NO`	A lump sum paid right after EMI number `EMI_NO`.
`BALANCE BANK BORROWER EMI_NO`	Print `BANK BORROWER AMOUNT_PAID EMIS_LEFT` as of (after) EMI number `EMI_NO`.

Rules from the statement: interest `I = P × N × R / 100` (R is a percentage); total repayable `A = P + I`; monthly EMI = `A / (N × 12)` **rounded up** to a whole number (86.67 → 87, and even 100.10 → 101); amount paid as of EMI *k* = `k × EMI` + every lump sum whose EMI number is *k* or less (a lump sum "after EMI 10" *is* included in `BALANCE ... 10`); EMIs left = remaining amount ÷ EMI, **rounded up** (23.1 → 24); and if the last EMI is more than what is still owed, it shrinks to exactly that amount. A lump sum therefore reduces the *number* of EMIs, never the EMI amount. Only `BALANCE` produces output, in the form `BANK_NAME BORROWER_NAME AMOUNT_PAID NO_OF_EMIS_LEFT`.

Question	Answer used
Can amount paid exceed `A`?	No — capped at `A` (then EMIs left = 0). This is also the "last EMI shrinks" rule: the final payment is only what is left.
Is interest a whole number?	The statement defines rounding only for the EMI and the EMI count. We *assume* interest is also rounded up to a whole unit (say so to the interviewer); every official sample has whole-number interest, so both samples come out the same either way.
One loan per bank-borrower pair?	Yes — the pair identifies the loan; a second `LOAN` for the same pair is an error.
`EMI_NO` beyond the tenure (the loan's total length)?	Error. `EMI_NO = 0` is valid (before any EMI).

Requirements

Functional: LOAN, PAYMENT, BALANCE exactly as above; reproduce the official sample outputs. **Non-functional:** input from a list/file; one bad line must not stop the rest; interest rule replaceable; exact arithmetic (`BigDecimal`).

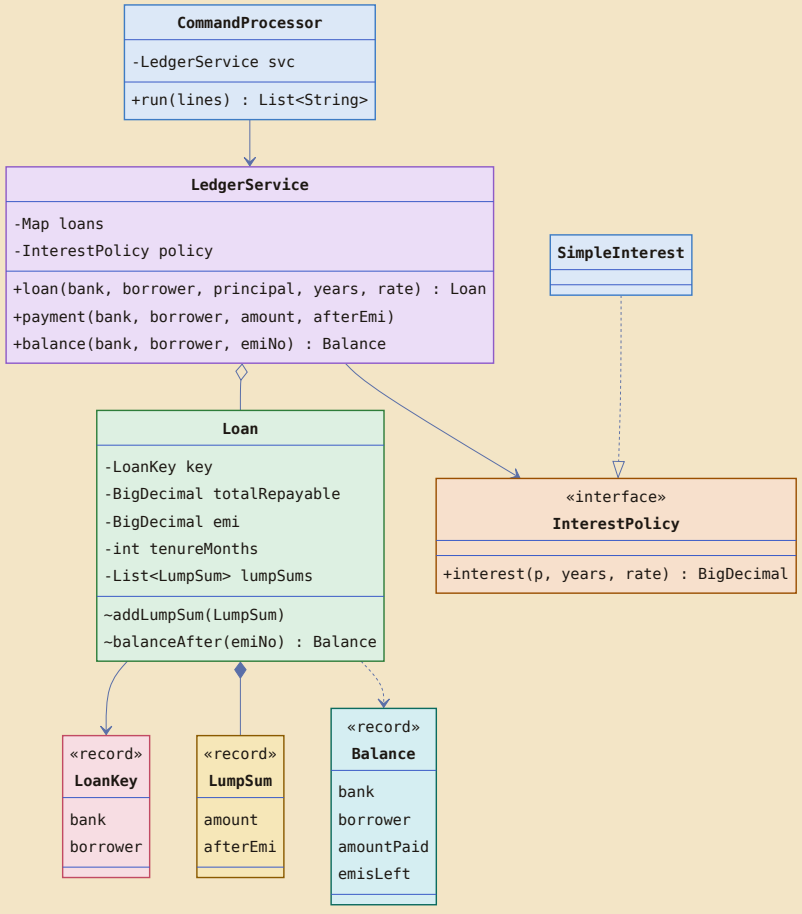
Use cases

Actor	Use cases
Bank	records a loan
Borrower	pays lump sums
Anyone (support agent, borrower app)	asks for the balance after EMI *k*

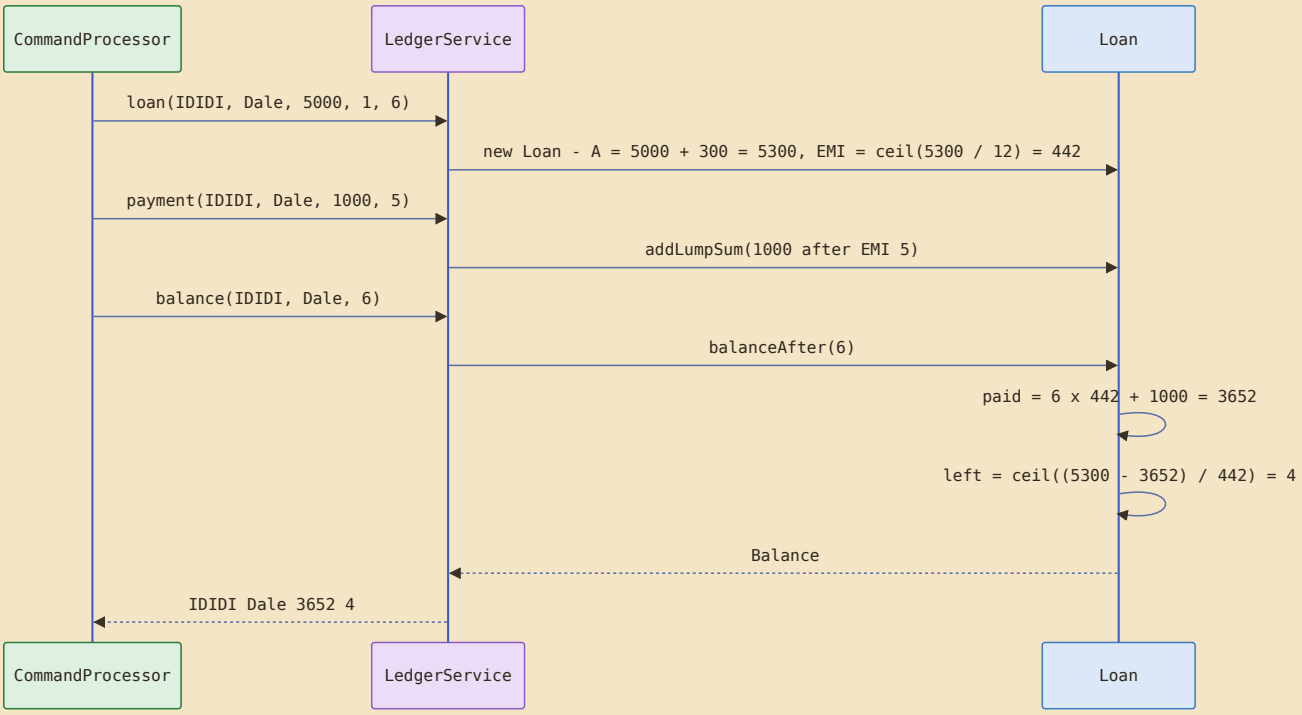
Core entities

Class	Kind	Responsibility
`LoanKey`	record	(bank, borrower) — the loan's identity; a perfect `Map` key because records implement `equals/hashCode` (the two methods a map uses to decide whether two keys are the same).
`Loan`	class	Computes `A` and EMI once; stores lump sums; derives a `Balance` for any EMI number.
`LumpSum`	record	amount + "after EMI number".
`Balance`	record	Output line: bank, borrower, amount paid, EMIs left.
`InterestPolicy` / `SimpleInterest`	interface / class	Strategy for total interest.
`LedgerService`	class	Store of loans; the three operations.
`CommandProcessor`	class	Parses lines, validates argument counts and numbers, collects output.
`LedgerException`	exception	Readable failure.

Class diagram



Key flows



Worked example (Sample 2, Shelly): P = 15000, N = 2, R = 9 → I = 2700, A = 17700, EMI = ⌈17700 / 24⌉ = ⌈737.5⌉ = 738. After 12 EMIs plus a 7000 lump sum: paid = 12 × 738 + 7000 = 15856; remaining = 1844; EMIs left = ⌈1844 / 738⌉ = ⌈2.499⌉ = 3. Output `UON Shelly 15856 3`.

Design decisions & patterns used

Decision	Why	Alternative rejected
Derive balance from facts (A, EMI, lump sums) on every query	Queries can ask about *any* EMI number, even one before a later payment. Storing a running balance would need replays.	Mutable `remaining` field updated per command — breaks when a BALANCE asks about an earlier EMI.
`InterestPolicy` Strategy	Simple interest today; reducing-balance or compound tomorrow without touching `Loan`.	Formula hard-coded in `Loan`.
`LoanKey` record as map key	Value semantics (two keys with the same fields count as equal) for free (`equals/hashCode`).	Concatenated string `"IDIDI#Dale"` — fragile.
`BigDecimal` + explicit `RoundingMode.CEILING`	The statement's rounding is "always up"; ceiling on integers is exact.	`Math.ceil` on doubles — fine for small numbers but floating error (tiny inaccuracies of binary floating-point numbers) can bite (e.g. a value like 4.000000001 ceils to 5).
Driver returns `List<String>`	Easy to compare with the expected output in a test.	Printing inside the service.

Java implementation

```java
// File: lld/ch21/ledger/LoanKey.java
package lld.ch21.ledger;

/** A borrower can hold one loan per bank; (bank, borrower) identifies it. */
public record LoanKey(String bank, String borrower) { }
```

```java
// File: lld/ch21/ledger/LumpSum.java
package lld.ch21.ledger;

import java.math.BigDecimal;

/** Extra payment made right after EMI number `afterEmi`. */
public record LumpSum(BigDecimal amount, int afterEmi) { }
```

```java
// File: lld/ch21/ledger/Balance.java
package lld.ch21.ledger;

import java.math.BigDecimal;

public record Balance(String bank, String borrower, BigDecimal amountPaid, int emisLeft) {
    @Override public String toString() { return bank + " " + borrower + " " + amountPaid.toPlainString() + " " +
emisLeft; }
}
```

```java
// File: lld/ch21/ledger/InterestPolicy.java
package lld.ch21.ledger;

import java.math.BigDecimal;

/** Strategy for total interest. The problem uses simple interest; a bank could plug in another rule. */
public interface InterestPolicy {
    BigDecimal interest(BigDecimal principal, BigDecimal years, BigDecimal ratePercent);
}
```

```java
// File: lld/ch21/ledger/SimpleInterest.java
package lld.ch21.ledger;

import java.math.BigDecimal;
import java.math.RoundingMode;

/** I = P * N * R / 100, rounded UP to a whole unit (the statement rounds amounts up). */
public class SimpleInterest implements InterestPolicy {
    @Override public BigDecimal interest(BigDecimal p, BigDecimal n, BigDecimal r) {
        return p.multiply(n).multiply(r).divide(BigDecimal.valueOf(100), 0, RoundingMode.CEILING);
    }
}
```

```java
// File: lld/ch21/ledger/Loan.java
package lld.ch21.ledger;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.List;

/**
 * Loan with fixed EMI. Amount repaid A = P + I. EMI = ceil(A / (N*12)).
 * Lump sums shorten the loan (fewer EMIs) but never change the EMI amount.
 */
public class Loan {
    private final LoanKey key;
    private final BigDecimal totalRepayable;
    private final BigDecimal emi;
    private final int tenureMonths;
    private final List<LumpSum> lumpSums = new ArrayList<>();

    Loan(LoanKey key, BigDecimal principal, int years, BigDecimal rate, InterestPolicy policy) {
        if (principal.signum() <= 0 || years <= 0 || rate.signum() < 0)
            throw new LedgerException("Invalid loan terms for " + key);
        this.key = key;
        this.tenureMonths = years * 12;
        this.totalRepayable = principal.add(policy.interest(principal, BigDecimal.valueOf(years), rate));
        this.emi = totalRepayable.divide(BigDecimal.valueOf(tenureMonths), 0, RoundingMode.CEILING);
    }

    public BigDecimal emi() { return emi; }
    public BigDecimal totalRepayable() { return totalRepayable; }

    synchronized void addLumpSum(LumpSum l) {
        if (l.amount().signum() <= 0) throw new LedgerException("Lump sum must be positive");
        if (l.afterEmi() < 0 || l.afterEmi() > tenureMonths) throw new LedgerException("EMI number out of range: " +
l.afterEmi());
        lumpSums.add(l);
    }

    /** Amount paid after `emiNo` EMIs (lump sums made at or before that EMI count), capped at A. */
    synchronized Balance balanceAfter(int emiNo) {
        if (emiNo < 0 || emiNo > tenureMonths) throw new LedgerException("EMI number out of range: " + emiNo);
        BigDecimal lumps = lumpSums.stream().filter(l -> l.afterEmi() <= emiNo)
                .map(LumpSum::amount).reduce(BigDecimal.ZERO, BigDecimal::add);
        BigDecimal paid = emi.multiply(BigDecimal.valueOf(emiNo)).add(lumps).min(totalRepayable);
        BigDecimal remaining = totalRepayable.subtract(paid);
        int left = remaining.divide(emi, 0, RoundingMode.CEILING).intValueExact();
        return new Balance(key.bank(), key.borrower(), paid, left);
    }
}
```

```java
// File: lld/ch21/ledger/LedgerException.java
package lld.ch21.ledger;

public class LedgerException extends RuntimeException {
    public LedgerException(String message) { super(message); }
}
```

```java
// File: lld/ch21/ledger/LedgerService.java
package lld.ch21.ledger;

import java.math.BigDecimal;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

public class LedgerService {
    private final Map<LoanKey, Loan> loans = new ConcurrentHashMap<>();
    private final InterestPolicy policy;

    public LedgerService(InterestPolicy policy) { this.policy = policy; }

    public Loan loan(String bank, String borrower, BigDecimal principal, int years, BigDecimal rate) {
        LoanKey key = new LoanKey(bank, borrower);
        Loan loan = new Loan(key, principal, years, rate, policy);
        if (loans.putIfAbsent(key, loan) != null) throw new LedgerException("Loan already exists for " + bank + " " +
borrower);
        return loan;
    }

    public void payment(String bank, String borrower, BigDecimal amount, int afterEmi) {
        find(bank, borrower).addLumpSum(new LumpSum(amount, afterEmi));
    }

    public Balance balance(String bank, String borrower, int emiNo) { return find(bank, borrower).balanceAfter(emiNo);
}

    private Loan find(String bank, String borrower) {
        Loan l = loans.get(new LoanKey(bank, borrower));
        if (l == null) throw new LedgerException("No loan for " + bank + " " + borrower);
        return l;
    }
}
```

```java
// File: lld/ch21/ledger/CommandProcessor.java
package lld.ch21.ledger;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;

/**
 * LOAN BANK BORROWER PRINCIPAL YEARS RATE
 * PAYMENT BANK BORROWER LUMP_SUM EMI_NO
 * BALANCE BANK BORROWER EMI_NO      -> prints "BANK BORROWER AMOUNT_PAID EMIS_LEFT"
 * Only BALANCE produces output (as in the original statement); errors are printed with "ERROR".
 */
public class CommandProcessor {
    private final LedgerService svc;
    public CommandProcessor(LedgerService svc) { this.svc = svc; }

    public List<String> run(List<String> lines) {
        List<String> output = new ArrayList<>();
        for (String line : lines) {
            if (line.isBlank()) continue;
            String[] t = line.trim().split("\\s+");
            try {
                switch (t[0]) {
                    case "LOAN" -> { need(t, 6); svc.loan(t[1], t[2], num(t[3]), Integer.parseInt(t[4]), num(t[5])); }
                    case "PAYMENT" -> { need(t, 5); svc.payment(t[1], t[2], num(t[3]), Integer.parseInt(t[4])); }
                    case "BALANCE" -> { need(t, 4); output.add(svc.balance(t[1], t[2],
Integer.parseInt(t[3])).toString()); }
                    default -> throw new LedgerException("Unknown command " + t[0]);
                }
            } catch (LedgerException | NumberFormatException e) {
                output.add("ERROR " + e.getMessage() + "  [" + line + "]");
            }
        }
        return output;
    }

    private static BigDecimal num(String s) {
        try { return new BigDecimal(s); } catch (NumberFormatException e) { throw new LedgerException("Not a number: "
+ s); }
    }

    private static void need(String[] t, int n) {
        if (t.length != n) throw new LedgerException(t[0] + " expects " + (n - 1) + " arguments");
    }
}
```

```java
// File: lld/ch21/ledger/Main.java
package lld.ch21.ledger;

import java.util.List;

public class Main {
    static final List<String> SAMPLE_1 = List.of(
            "LOAN IDIDI Dale 10000 5 4",
            "LOAN MBI Harry 2000 2 2",
            "BALANCE IDIDI Dale 5",
            "BALANCE IDIDI Dale 40",
            "BALANCE MBI Harry 12",
            "BALANCE MBI Harry 0");

    static final List<String> SAMPLE_2 = List.of(
            "LOAN IDIDI Dale 5000 1 6",
            "LOAN MBI Harry 10000 3 7",
            "LOAN UON Shelly 15000 2 9",
            "PAYMENT IDIDI Dale 1000 5",
            "PAYMENT MBI Harry 5000 10",
            "PAYMENT UON Shelly 7000 12",
            "BALANCE IDIDI Dale 3",
            "BALANCE IDIDI Dale 6",
            "BALANCE UON Shelly 12",
            "BALANCE MBI Harry 12");

    static final List<String> EDGE_CASES = List.of(
            "LOAN SBI Asha 1000 1 12",
            "PAYMENT SBI Asha 2000 1",           // lump sum bigger than the rest of the loan
            "BALANCE SBI Asha 2",
            "LOAN SBI Asha 5 1 1",               // duplicate loan
            "BALANCE HDFC Nobody 1",             // unknown loan
            "BALANCE SBI Asha 13",               // beyond tenure
            "PAYMENT SBI Asha ten 1",            // bad number
            "REFUND SBI Asha 1");                // unknown command

    public static void main(String[] args) {
        run("Sample 1", SAMPLE_1);
        run("Sample 2", SAMPLE_2);
        run("Edge cases", EDGE_CASES);
    }

    private static void run(String title, List<String> input) {
        System.out.println("== " + title);
        new CommandProcessor(new LedgerService(new SimpleInterest())).run(input).forEach(System.out::println);
    }
}
```

Output:

```
== Sample 1
IDIDI Dale 1000 55
IDIDI Dale 8000 20
MBI Harry 1044 12
MBI Harry 0 24
== Sample 2
IDIDI Dale 1326 9
IDIDI Dale 3652 4
UON Shelly 15856 3
MBI Harry 9044 10
== Edge cases
SBI Asha 1120 0
ERROR Loan already exists for SBI Asha  [LOAN SBI Asha 5 1 1]
ERROR No loan for HDFC Nobody  [BALANCE HDFC Nobody 1]
ERROR EMI number out of range: 13  [BALANCE SBI Asha 13]
ERROR Not a number: ten  [PAYMENT SBI Asha ten 1]
ERROR Unknown command REFUND  [REFUND SBI Asha 1]
```

The first two blocks are exactly the expected outputs of the two official Geektrust samples. In the edge-case block, Asha's 2000 lump sum after EMI 1 overpays a 1120 loan, so the amount paid is capped at 1120 and 0 EMIs remain.

Concurrency & edge cases

- **Concurrency:** loans live in a `ConcurrentHashMap` (`putIfAbsent` makes "one loan per pair" atomic); `Loan`'s lump-sum list is guarded by `synchronized` methods, so a `PAYMENT` and a `BALANCE` on the same loan do not interfere.
- **Edge cases handled:** duplicate loan, unknown loan, EMI number < 0 or > tenure, non-positive principal/years, negative rate, non-positive lump sum, overpayment (capped), wrong argument count, non-numeric values, unknown command, blank lines.

Extensions & follow-up questions

- **Reducing-balance (amortised) EMI like real banks?** (Reducing-balance: each month's interest is charged only on what is still owed; amortised: spread into equal payments that cover interest and principal.) New `InterestPolicy`/`EmiCalculator`: `EMI = P·r·(1+r)^n / ((1+r)^n − 1)` with monthly rate `r`; balance after k months uses the amortisation schedule. The `Loan` API stays the same.
- **Lump sum that should reduce the EMI instead of the tenure?** Make that a `PrepaymentPolicy` (REDUCE_TENURE / REDUCE_EMI) chosen per loan.
- **Late payments and penalties?** Record actual payment events with dates instead of assuming EMIs are paid on time; derive "EMIs paid" from them.
- **Read from a file?** `Files.readAllLines(Path.of(args[0]))` fed into the same `run` method — the driver's job, no core change.

Real-world example

Indian lenders such as Navi, banks and NBFCs (non-banking financial companies: lenders that are not licensed banks) show borrowers an EMI schedule and let them "part-prepay" a loan; most offer a choice between reducing the tenure (fewer EMIs, the Ledger Co rule) or reducing the EMI amount. Real loans use reducing-balance interest rather than the flat simple interest in this puzzle — a good point to raise when asked about extensions.

In the interview

Show the three formulas first and compute one sample by hand — it proves you understood the rounding. Then code `Loan.balanceAfter` (the heart), the service, and the command loop. Run both samples. Emphasise "state = facts, balances = derived", and the interest Strategy.

Self-check

- **Q:** Why ceil the EMI? → **A:** A floor would leave a shortfall at the end; rounding up guarantees N × 12 EMIs cover `A` (the final instalment may be smaller).
- **Q:** A `PAYMENT` after EMI 10 — does `BALANCE ... 5` include it? → **A:** No; only lump sums with `afterEmi ≤ 5` count.
- **Q:** Why is the balance computed rather than stored? → **A:** Queries may ask about any EMI number in any order; derivation is always consistent.
- **Q:** Where do you add compound interest (interest that is also charged on earlier interest)? → **A:** A new `InterestPolicy` implementation injected into `LedgerService`.

TL;DR

EMI = ⌈(P + PNR/100) / 12N⌉. Store loan facts and lump sums; derive paid and EMIs-left on demand with ceiling rounding. Interest rule is a Strategy; the driver is a tiny command loop.

21.7 Design an Inventory Management System

Problem & clarifying questions

A very common machine-coding and OOD question (asked in e-commerce — online shopping — and quick-commerce — groceries delivered in minutes — interviews): track **products** (SKUs — stock keeping units, one id per sellable item) stored in several **warehouses**; record every **stock movement** (goods received, shipped, transferred between warehouses, adjusted after a physical count); fulfil orders by choosing warehouses; and raise **reorder alerts** when stock runs low.

Question	Answer used
Is stock tracked per warehouse?	Yes: (SKU, warehouse) → quantity.
Can an order be split across warehouses?	Yes; the choice of warehouses is a pluggable strategy (nearest city first, or most stock first).
Partial shipments if stock is short?	No — all-or-nothing.
When is a reorder alert raised?	When *total* stock of a SKU drops to or below its reorder point. Once per dip (no alert spam); re-armed when stock recovers.
How much to reorder?	Up to the product's max level (`max − current`).
Negative stock?	Never.

Requirements

Functional

1. Add products (with reorder point and max level) and warehouses.
2. Receive stock (inbound) into a warehouse.
3. Ship an order quantity, allocating across warehouses by a strategy.
4. Transfer stock between warehouses.
5. Adjust stock after a physical count (cycle count).
6. Query stock per warehouse and total; list the movement history of a SKU.
7. Reorder alerts to subscribers.

Non-functional

- **No overselling** (selling more units than are in stock) under concurrent orders.
- Operations on different products run in parallel.
- Full audit trail (a complete record of every change; here every change is a movement).

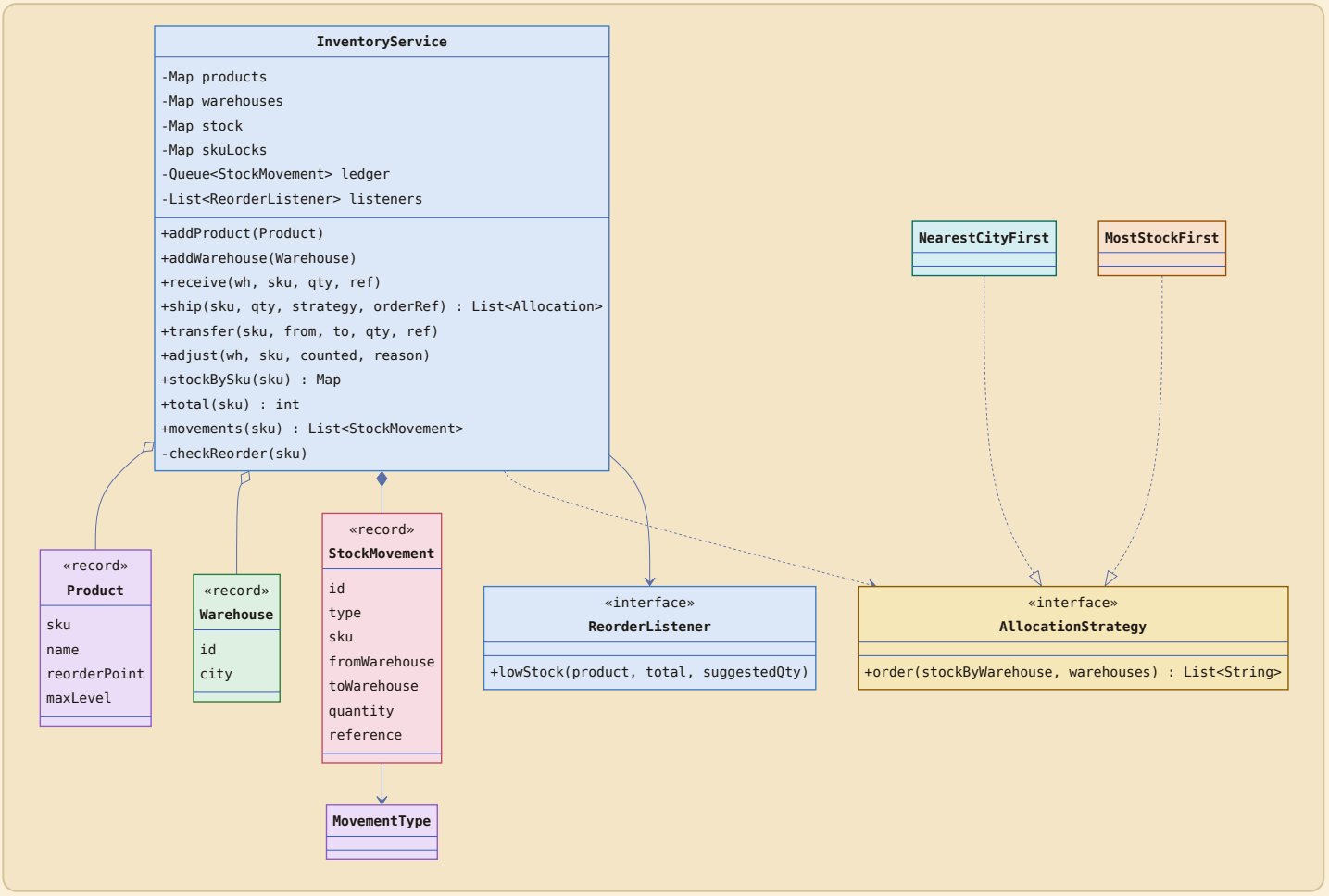
Use cases

Actor	Use cases
Warehouse staff	receive stock, transfer, cycle-count adjust
Order service (system)	ship / allocate order
Purchasing team (subscriber)	receives reorder alerts
Manager	views stock and movement history

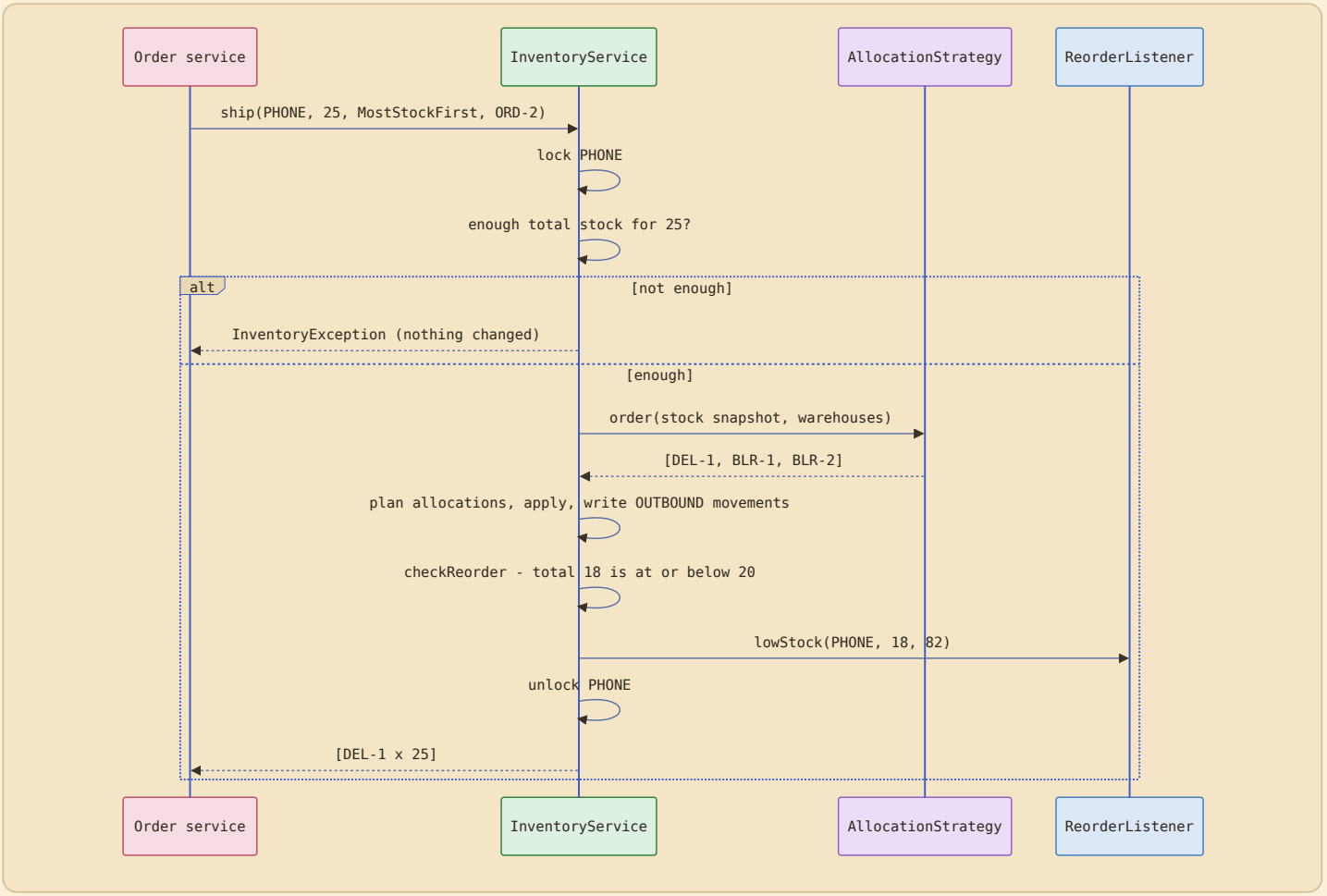
Core entities

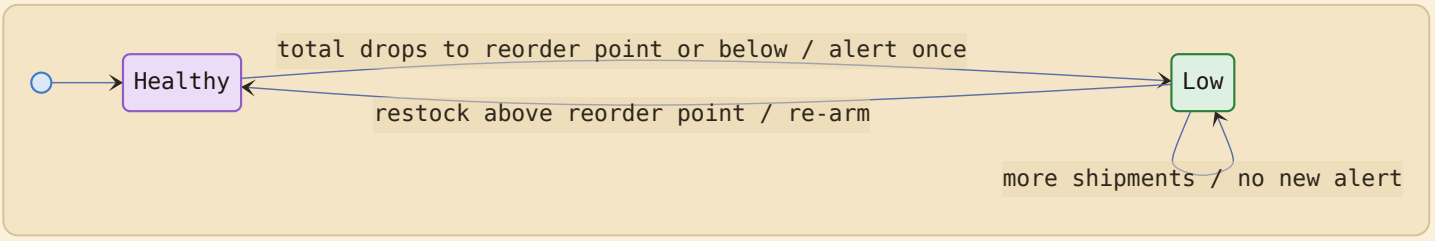
Class	Kind	Responsibility
`Product`	record	SKU, name, reorder point, max level (validated).
`Warehouse`	record	id, city.
`StockMovement`	record	Append-only ledger line (type, sku, from, to, qty, reference).
`MovementType`	enum	INBOUND, OUTBOUND, TRANSFER, ADJUSTMENT.
`Allocation`	record	"take q from warehouse w" — part of a shipment plan.
`AllocationStrategy` + `NearestCityFirst`, `MostStockFirst`	interface + strategies	Order in which warehouses are drained.
`ReorderListener`	interface	Observer for low-stock alerts.
`InventoryService`	class	Stock levels, per-SKU locking, movements, alerts.
`InventoryException`	exception	Readable failure.

Class diagram



Key flows


Order service
InventoryService
AllocationStrategy
ReorderListener
ship(PHONE, 25, MostStockFirst, ORD-2)
lock PHONE
enough total stock for 25?
alt
[not enough]
InventoryException (nothing changed)
[enough]
order(stock snapshot, warehouses)
[DEL-1, BLR-1, BLR-2]
plan allocations, apply, write OUTBOUND movements
checkReorder - total 18 is at or below 20
lowStock(PHONE, 18, 82)
unlock PHONE
[DEL-1 x 25]
Order service
InventoryService
AllocationStrategy
ReorderListener


Healthy
total drops to reorder point or below / alert once
restock above reorder point / re-arm
Low
more shipments / no new alert

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
One lock per SKU	Lock striping by key (a separate lock for each group of keys)	An order for one SKU touches several warehouses; a per-SKU lock covers all of them with no lock-ordering puzzle, while different SKUs never contend.	One lock per (SKU, warehouse) — needs lock ordering for multi-warehouse orders; one global lock — no parallelism.
Plan, then apply in `ship`	—	Check total first and build the full plan before changing anything → all-or-nothing.	Deduct warehouse by warehouse and roll back on shortage.
`AllocationStrategy`	Strategy	Business wants different policies (speed vs fewer split parcels); each is a small class.	`if (mode == ...)` in `ship`.
Append-only `StockMovement` ledger	Event log / audit	Every change is explainable; levels can be rebuilt by replaying; adjustments record the delta (the change, e.g. −3), not just the new number.	Updating quantities with no history.
`ReorderListener` **with once-per-dip**	Observer + edge-triggering (fire once when a value crosses a line)	Purchasing is notified once when stock crosses the threshold, not on every sale.	Level-triggered alerts (fire every time while the value is below the line) on every movement — spam.
Strategy gets a snapshot copy	Defensive copy (hand out a copy so the receiver cannot change your original)	A strategy cannot accidentally modify live stock.	Passing the live map.

Java implementation

```java
// File: lld/ch21/inventory/Product.java
package lld.ch21.inventory;

/** reorderPoint: raise an alert when total stock falls to this level or below. */
public record Product(String sku, String name, int reorderPoint, int maxLevel) {
    public Product {
        if (reorderPoint < 0 || maxLevel <= reorderPoint) throw new InventoryException("Need 0 <= reorderPoint <
maxLevel");
    }
}
```

```java
// File: lld/ch21/inventory/Warehouse.java
package lld.ch21.inventory;

public record Warehouse(String id, String city) { }
```

```java
// File: lld/ch21/inventory/MovementType.java
package lld.ch21.inventory;

public enum MovementType { INBOUND, OUTBOUND, TRANSFER, ADJUSTMENT }
```

```java
// File: lld/ch21/inventory/StockMovement.java
package lld.ch21.inventory;

/** Append-only ledger line. Stock levels can always be rebuilt by replaying these. */
public record StockMovement(int id, MovementType type, String sku, String fromWarehouse, String toWarehouse,
                            int quantity, String reference) {
    @Override public String toString() {
        return "#%d %-10s %s qty=%+d %s->%s (%s)".formatted(id, type, sku, quantity,
                fromWarehouse == null ? "-" : fromWarehouse, toWarehouse == null ? "-" : toWarehouse, reference);
    }
}
```

```java
// File: lld/ch21/inventory/Allocation.java
package lld.ch21.inventory;

public record Allocation(String warehouseId, int quantity) { }
```

```java
// File: lld/ch21/inventory/AllocationStrategy.java
package lld.ch21.inventory;

import java.util.List;
import java.util.Map;

/** Strategy: in which order should warehouses be drained to fulfil an order? */
public interface AllocationStrategy {
    List<String> order(Map<String, Integer> stockByWarehouse, Map<String, Warehouse> warehouses);
}
```

```java
// File: lld/ch21/inventory/MostStockFirst.java
package lld.ch21.inventory;

import java.util.Comparator;
import java.util.List;
import java.util.Map;

/** Fewest split shipments: take from the fullest warehouse first. */
public class MostStockFirst implements AllocationStrategy {
    @Override public List<String> order(Map<String, Integer> stock, Map<String, Warehouse> warehouses) {
        return stock.keySet().stream()
                .sorted(Comparator.comparing((String w) -> stock.get(w)).reversed().thenComparing(w -> w))
                .toList();
    }
}
```

```java
// File: lld/ch21/inventory/NearestCityFirst.java
package lld.ch21.inventory;

import java.util.Comparator;
import java.util.List;
import java.util.Map;

/** Ship from warehouses in the customer's city first (faster delivery), then the rest by stock. */
public class NearestCityFirst implements AllocationStrategy {
    private final String city;
    public NearestCityFirst(String city) { this.city = city; }

    @Override public List<String> order(Map<String, Integer> stock, Map<String, Warehouse> warehouses) {
        return stock.keySet().stream()
                .sorted(Comparator.comparing((String w) -> !warehouses.get(w).city().equalsIgnoreCase(city))
                        .thenComparing(w -> -stock.get(w)).thenComparing(w -> w))
                .toList();
    }
}
```

```java
// File: lld/ch21/inventory/ReorderListener.java
package lld.ch21.inventory;

/** Observer: purchasing, email, dashboards... anyone who reacts to low stock. */
public interface ReorderListener {
    void lowStock(Product product, int totalOnHand, int suggestedOrderQty);
}
```

```java
// File: lld/ch21/inventory/InventoryException.java
package lld.ch21.inventory;

public class InventoryException extends RuntimeException {
    public InventoryException(String message) { super(message); }
}
```

```java
// File: lld/ch21/inventory/InventoryService.java
package lld.ch21.inventory;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Queue;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.locks.ReentrantLock;

/**
 * Stock is kept per (sku, warehouse). Locking is per SKU: every operation on one product
 * (even across warehouses) is serialized, while different products proceed in parallel.
 */
public class InventoryService {
    private final Map<String, Product> products = new ConcurrentHashMap<>();
    private final Map<String, Warehouse> warehouses = new ConcurrentHashMap<>();
    private final Map<String, Map<String, Integer>> stock = new ConcurrentHashMap<>();   // sku -> (wh -> qty)
    private final Map<String, ReentrantLock> skuLocks = new ConcurrentHashMap<>();
    private final Set<String> alerted = ConcurrentHashMap.newKeySet();                   // SKUs already flagged low
    private final Queue<StockMovement> ledger = new ConcurrentLinkedQueue<>();              // append-heavy, lock-free
    private final List<ReorderListener> listeners = new CopyOnWriteArrayList<>();
    private final AtomicInteger movementSeq = new AtomicInteger();

    public void addListener(ReorderListener l) { listeners.add(l); }

    public void addProduct(Product p) {
        if (products.putIfAbsent(p.sku(), p) != null) throw new InventoryException("Duplicate SKU " + p.sku());
        stock.put(p.sku(), new LinkedHashMap<>());
        skuLocks.put(p.sku(), new ReentrantLock());
    }

    public void addWarehouse(Warehouse w) {
        if (warehouses.putIfAbsent(w.id(), w) != null) throw new InventoryException("Duplicate warehouse " + w.id());
    }

    public void receive(String warehouseId, String sku, int qty, String ref) {
        positive(qty);
        warehouse(warehouseId);
        withSku(sku, () -> {
            change(sku, warehouseId, qty);
            record(MovementType.INBOUND, sku, null, warehouseId, qty, ref);
        });
    }

    /** All-or-nothing: either the whole quantity is allocated across warehouses, or nothing moves. */
    public List<Allocation> ship(String sku, int qty, AllocationStrategy strategy, String orderRef) {
        positive(qty);
        return withSku(sku, () -> {
            Map<String, Integer> levels = stock.get(sku);
            int total = levels.values().stream().mapToInt(Integer::intValue).sum();
            if (total < qty) throw new InventoryException("Only " + total + " of " + sku + " available, order needs "
+ qty);
            List<Allocation> plan = new ArrayList<>();
            int remaining = qty;
            for (String wh : strategy.order(Map.copyOf(levels), warehouses)) {
                int take = Math.min(remaining, levels.get(wh));
                if (take > 0) { plan.add(new Allocation(wh, take)); remaining -= take; }
                if (remaining == 0) break;
            }
            for (Allocation a : plan) {
                change(sku, a.warehouseId(), -a.quantity());
                record(MovementType.OUTBOUND, sku, a.warehouseId(), null, -a.quantity(), orderRef);
            }
            return plan;
        });
    }
```

```
    public void transfer(String sku, String fromWh, String toWh, int qty, String ref) {
        positive(qty);
        warehouse(fromWh); warehouse(toWh);
        if (fromWh.equals(toWh)) throw new InventoryException("Source and target warehouse are the same");
        withSku(sku, () -> {
            int have = level(sku, fromWh);
            if (have < qty) throw new InventoryException(fromWh + " has only " + have + " of " + sku);
            change(sku, fromWh, -qty);
            change(sku, toWh, qty);
            record(MovementType.TRANSFER, sku, fromWh, toWh, qty, ref);
        });
    }

    /** Cycle count: set the level to what was physically counted and log the difference. */
    public void adjust(String warehouseId, String sku, int countedQty, String reason) {
        if (countedQty < 0) throw new InventoryException("Counted quantity cannot be negative");
        warehouse(warehouseId);
        withSku(sku, () -> {
            int delta = countedQty - level(sku, warehouseId);
            if (delta == 0) return;
            change(sku, warehouseId, delta);
            record(MovementType.ADJUSTMENT, sku, null, warehouseId, delta, reason);
        });
    }

    /** Sorted by warehouse id so output is stable. */
    public Map<String, Integer> stockBySku(String sku) {
        return withSku(sku, () -> java.util.Collections.unmodifiableMap(new java.util.TreeMap<>(stock.get(sku))));
    }

    public int total(String sku) { return withSku(sku, () -> totalUnlocked(sku)); }

    public List<StockMovement> movements(String sku) {
        return ledger.stream().filter(m -> m.sku().equals(sku)).toList();
    }

    // ------------------------------------------------------------ internals (called with the SKU lock held)

    private void change(String sku, String wh, int delta) {
        stock.get(sku).merge(wh, delta, Integer::sum);
    }

    private void record(MovementType type, String sku, String from, String to, int qty, String ref) {
        ledger.add(new StockMovement(movementSeq.incrementAndGet(), type, sku, from, to, qty, ref));
        checkReorder(sku);
    }

    /** Fires once when stock crosses down to the reorder point; re-arms when stock recovers. */
    private void checkReorder(String sku) {
        Product p = products.get(sku);
        int total = totalUnlocked(sku);
        if (total <= p.reorderPoint()) {
            if (alerted.add(sku)) listeners.forEach(l -> l.lowStock(p, total, p.maxLevel() - total));
        } else {
            alerted.remove(sku);
        }
    }

    private int totalUnlocked(String sku) { return stock.get(sku).values().stream().mapToInt(Integer::intValue).sum();
}
    private int level(String sku, String wh) { return stock.get(sku).getOrDefault(wh, 0); }

    private <T> T withSku(String sku, java.util.function.Supplier<T> action) {
        ReentrantLock lock = skuLocks.get(sku);
        if (lock == null) throw new InventoryException("Unknown SKU " + sku);
        lock.lock();
        try { return action.get(); } finally { lock.unlock(); }
    }

    private void withSku(String sku, Runnable action) { withSku(sku, () -> { action.run(); return null; }); }

    private void warehouse(String id) { if (!warehouses.containsKey(id)) throw new InventoryException("Unknown
warehouse " + id); }
    private static void positive(int q) { if (q <= 0) throw new InventoryException("Quantity must be positive: " + q);
```

```
}
}
```

```java
// File: lld/ch21/inventory/Main.java
package lld.ch21.inventory;

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    public static void main(String[] args) throws InterruptedException {
        InventoryService inv = new InventoryService();
        inv.addListener((p, total, suggest) ->
                System.out.println("  [ALERT] " + p.sku() + " low: " + total + " left, reorder " + suggest + " to
reach " + p.maxLevel()));

        inv.addWarehouse(new Warehouse("BLR-1", "Bengaluru"));
        inv.addWarehouse(new Warehouse("BLR-2", "Bengaluru"));
        inv.addWarehouse(new Warehouse("DEL-1", "Delhi"));
        inv.addProduct(new Product("PHONE", "Phone X", 20, 100));
        inv.addProduct(new Product("CASE", "Phone case", 10, 200));

        System.out.println("== receive stock");
        inv.receive("BLR-1", "PHONE", 30, "PO-1");
        inv.receive("DEL-1", "PHONE", 40, "PO-2");
        inv.receive("BLR-2", "PHONE", 5, "PO-3");
        System.out.println("  PHONE: " + inv.stockBySku("PHONE") + " total=" + inv.total("PHONE"));

        System.out.println("== ship 32 phones to a Bengaluru customer (nearest city first)");
        System.out.println("  " + inv.ship("PHONE", 32, new NearestCityFirst("Bengaluru"), "ORD-1"));
        System.out.println("  PHONE: " + inv.stockBySku("PHONE"));

        System.out.println("== transfer 15 phones DEL-1 -> BLR-1");
        inv.transfer("PHONE", "DEL-1", "BLR-1", 15, "TR-1");
        System.out.println("  PHONE: " + inv.stockBySku("PHONE"));

        System.out.println("== ship 25 more (most stock first) -> crosses reorder point");
        System.out.println("  " + inv.ship("PHONE", 25, new MostStockFirst(), "ORD-2"));
        System.out.println("  ship 1 more (no duplicate alert): " + inv.ship("PHONE", 1, new MostStockFirst(), "ORD-
3"));

        System.out.println("== cycle count finds 2 damaged units in BLR-1");
        inv.adjust("BLR-1", "PHONE", inv.stockBySku("PHONE").get("BLR-1") - 2, "damaged");
        System.out.println("  PHONE: " + inv.stockBySku("PHONE") + " total=" + inv.total("PHONE"));

        System.out.println("== restock re-arms the alert");
        inv.receive("BLR-1", "PHONE", 90, "PO-4");
        inv.ship("PHONE", 90, new MostStockFirst(), "ORD-4");

        System.out.println("== edge cases");
        attempt(() -> inv.ship("PHONE", 999, new MostStockFirst(), "ORD-X"));
        attempt(() -> inv.transfer("PHONE", "BLR-2", "BLR-2", 1, "TR-X"));
        attempt(() -> inv.receive("MUM-1", "PHONE", 5, "PO-X"));
        attempt(() -> inv.receive("BLR-1", "TABLET", 5, "PO-X"));
        attempt(() -> inv.receive("BLR-1", "PHONE", 0, "PO-X"));

        System.out.println("== movement ledger for PHONE");
        inv.movements("PHONE").forEach(m -> System.out.println("  " + m));

        System.out.println("== concurrency: 200 one-unit orders race for 50 cases");
        inv.receive("BLR-1", "CASE", 50, "PO-5");
        AtomicInteger ok = new AtomicInteger(), rejected = new AtomicInteger();
        ExecutorService pool = Executors.newFixedThreadPool(8);
        for (int i = 0; i < 200; i++) {
            pool.submit(() -> {
                try { inv.ship("CASE", 1, new MostStockFirst(), "web"); ok.incrementAndGet(); }
                catch (InventoryException e) { rejected.incrementAndGet(); }
            });
        }
        pool.shutdown();
        pool.awaitTermination(10, TimeUnit.SECONDS);
        System.out.println("  shipped=" + ok + " rejected=" + rejected + " left=" + inv.total("CASE"));
    }
```

```
    private static void attempt(Runnable r) {
        try { r.run(); System.out.println("  ok"); }
        catch (InventoryException e) { System.out.println("  error: " + e.getMessage()); }
    }
}
```

Output:

```
== receive stock
  PHONE: {BLR-1=30, BLR-2=5, DEL-1=40} total=75
== ship 32 phones to a Bengaluru customer (nearest city first)
  [Allocation[warehouseId=BLR-1, quantity=30], Allocation[warehouseId=BLR-2, quantity=2]]
  PHONE: {BLR-1=0, BLR-2=3, DEL-1=40}
== transfer 15 phones DEL-1 -> BLR-1
  PHONE: {BLR-1=15, BLR-2=3, DEL-1=25}
== ship 25 more (most stock first) -> crosses reorder point
  [ALERT] PHONE low: 18 left, reorder 82 to reach 100
  [Allocation[warehouseId=DEL-1, quantity=25]]
  ship 1 more (no duplicate alert): [Allocation[warehouseId=BLR-1, quantity=1]]
== cycle count finds 2 damaged units in BLR-1
  PHONE: {BLR-1=12, BLR-2=3, DEL-1=0} total=15
== restock re-arms the alert
  [ALERT] PHONE low: 15 left, reorder 85 to reach 100
== edge cases
  error: Only 15 of PHONE available, order needs 999
  error: Source and target warehouse are the same
  error: Unknown warehouse MUM-1
  error: Unknown SKU TABLET
  error: Quantity must be positive: 0
== movement ledger for PHONE
  #1 INBOUND    PHONE qty=+30 -->BLR-1 (PO-1)
  #2 INBOUND    PHONE qty=+40 -->DEL-1 (PO-2)
  #3 INBOUND    PHONE qty=+5 -->BLR-2 (PO-3)
  #4 OUTBOUND   PHONE qty=-30 BLR-1->- (ORD-1)
  #5 OUTBOUND   PHONE qty=-2 BLR-2->- (ORD-1)
  #6 TRANSFER   PHONE qty=+15 DEL-1->BLR-1 (TR-1)
  #7 OUTBOUND   PHONE qty=-25 DEL-1->- (ORD-2)
  #8 OUTBOUND   PHONE qty=-1 BLR-1->- (ORD-3)
  #9 ADJUSTMENT PHONE qty=-2 -->BLR-1 (damaged)
  #10 INBOUND    PHONE qty=+90 -->BLR-1 (PO-4)
  #11 OUTBOUND   PHONE qty=-90 BLR-1->- (ORD-4)
== concurrency: 200 one-unit orders race for 50 cases
  [ALERT] CASE low: 10 left, reorder 190 to reach 200
  shipped=50 rejected=150 left=0
```

Reading the output: the Bengaluru order drained BLR-1 then BLR-2 before touching Delhi; the reorder alert fired once when total PHONE stock fell to 18 (≤ 20) and not again at 17; after a restock pushed the total above 20 the alert re-armed and fired again when stock fell back; the ledger explains every unit; and with 200 concurrent one-unit orders for 50 cases, exactly 50 shipped — no overselling.

Concurrency & edge cases

- **Overselling:** check-total → plan → apply runs under the SKU's `ReentrantLock`; concurrent orders for the same SKU are serialised.
- **Parallelism:** different SKUs use different locks.
- **Ledger:** a `ConcurrentLinkedQueue` (a thread-safe queue that needs no lock) — lock-free appends from many SKU threads at once, insertion order kept, and iteration never throws `ConcurrentModificationException` (the error Java raises when a collection changes while it is being looped over). (A `CopyOnWriteArrayList` would copy the whole ledger on every movement — right for rarely-changing lists, wrong for an append-heavy log.)
- **Listeners:** `CopyOnWriteArrayList` — read on every alert, changed almost never, which is exactly its use case. Listener calls happen while holding the SKU lock — fine for quick callbacks; for slow ones (email), hand off to an executor (a pool of threads that runs submitted tasks) so the lock is not held during I/O (input/output, such as network or disk calls).
- **Edge cases handled:** unknown SKU/warehouse, non-positive quantities, negative counted quantity, transfer to the same warehouse, transfer or order larger than stock, invalid product thresholds, duplicate SKU/warehouse, "no change" adjustments (not logged).

Extensions & follow-up questions

- **Reservations (cart holds)?** Track `onHand` and `reserved` per (SKU, warehouse); available = onHand − reserved; reservations expire after a timeout; shipping converts a reservation into an OUTBOUND.
- **Batches / expiry dates (groceries, medicines)?** Stock per batch with expiry; allocate FEFO (first-expiry-first-out) — just another `AllocationStrategy`.
- **Distances instead of cities?** `NearestFirst` sorting by a `DistanceService` result.
- **Automatic purchase orders?** A `ReorderListener` that creates a PO (purchase order: a formal request to a supplier) for `max − current` and marks it pending so a second alert does not create a duplicate.
- **Multiple services / database?** Stock rows with optimistic versioning (`UPDATE ... SET qty = qty - ? WHERE qty >= ? AND version = ?`), and the movement ledger as an event stream (e.g. Kafka, a system that stores and delivers an ordered log of events) consumed by analytics.

Real-world example

Amazon's fulfilment network and Flipkart's warehouses decide which fulfilment centre ships each order based on factors like proximity and stock, and may split an order across centres — the job of `AllocationStrategy`. Shopify lets merchants hold inventory at multiple locations and set a fulfilment priority order among them. Quick-commerce apps (Blinkit, Zepto, Swiggy Instamart) run many small "dark stores" (warehouses closed to the public that only serve online orders), where accurate per-store stock and fast reorder alerts are essential.

In the interview

Start with the data shape: `stock[sku][warehouse] = qty` plus an append-only movement list. Code receive and ship (with the plan-then-apply and the per-SKU lock) first; they carry most of the marks. Then transfer and adjust (both tiny), then the reorder observer with once-per-dip, then allocation strategies. Run the 200-thread demo to prove "no overselling".

Self-check

- **Q:** Why does `ship` check the total before building the plan? → **A:** To reject short orders before any warehouse is touched (all-or-nothing).
- **Q:** Why is a per-SKU lock enough for a multi-warehouse order? → **A:** An order involves exactly one SKU, and all of that SKU's warehouses are covered by its one lock.
- **Q:** How do you add FEFO? → **A:** Add batch-level stock and a new `AllocationStrategy`; `ship` does not change.
- **Q:** Why record adjustments as movements? → **A:** So shrinkage/damage is auditable and levels can be rebuilt from the ledger.

TL;DR

Stock per (SKU, warehouse), every change is a ledger movement, a lock per SKU prevents overselling, allocation is a Strategy, and low-stock alerts are an edge-triggered Observer.

21.8 Design a Unix File Search (`find`)

Problem & clarifying questions

The Unix (the operating-system family that includes Linux and macOS) `find` command walks a directory tree (folders inside folders) and prints every entry that passes a set of tests, for example `find /var/log -type f -size +10M -mtime +7` ("regular files under /var/log bigger than 10 MiB, last changed more than 7 days ago"; 1 MiB = 1,048,576 bytes). The machine-coding version asks you to build a small `find`: a file-system model, a set of **filters**, a way to **combine** them with AND / OR / NOT, and a clean API for the search options. It is a favourite because the obvious solution (one method with ten optional parameters and a pile of `if`s) is exactly what the interviewer wants you to avoid.

Three words used below: a **predicate** (or *test*) is a yes/no question about one entry ("is its name `*.log`?"); a **glob** is a filename pattern with wildcards (`*` = any characters, `?` = one character, `[0-9]` = one character from a set); **depth** is how many levels below the start directory an entry sits (the start itself is depth 0).

Question	Answer used
Real disk or in-memory?	In-memory tree, so runs are identical; a `java.nio` (Java's file and I/O library) adapter (a class that wraps one API to fit another interface) is an extension.
Which tests?	Name glob (case-sensitive and insensitive), type (file / directory), size, modification age, owner, empty.
How are tests combined?	AND, OR, NOT and parentheses, with `find`'s precedence (which operator is applied first when there are no parentheses): NOT binds tightest, then AND, then OR. Two tests side by side mean AND.
Size semantics?	Like GNU `find` (the standard version on Linux, from the GNU project): the size is **rounded up** to the unit before comparing, so `-size -1M` matches only empty files. Size tests apply to files only.
Other options?	`mindepth`, `maxdepth`, a result limit, and **pruning** (skip whole directories such as `node_modules`, a JavaScript project's often huge folder of downloaded libraries).
API or command line?	Both: a fluent (written as one chain of method calls, `a().b().c()`) **Builder** API for code, and a parser for `find`-style command lines.
Output order?	Pre-order depth-first (go as deep as possible down one branch before the next; a directory is listed before its contents), children in name order.
Huge trees?	Stream results as they are found; no recursion (a method calling itself; deep trees must not overflow the stack — the limited memory that holds pending method calls); a parallel variant as an extension.

Requirements

Functional

1. Model files and directories (name, owner, modification time, size).
2. Search from a start path with any combination of tests joined by AND / OR / NOT.
3. Options: min/max depth, result limit, pruned directory names.
4. Parse `find` command lines into the same search criteria.
5. Clear errors for bad paths, unknown tests, malformed sizes and unbalanced parentheses.

Non-functional

- **Open/Closed**: a new test = one new class, no change to the search loop or the parser's structure.
- Memory: stream results; handle very deep trees.
- Thread safety: searching while files are being added must not throw.

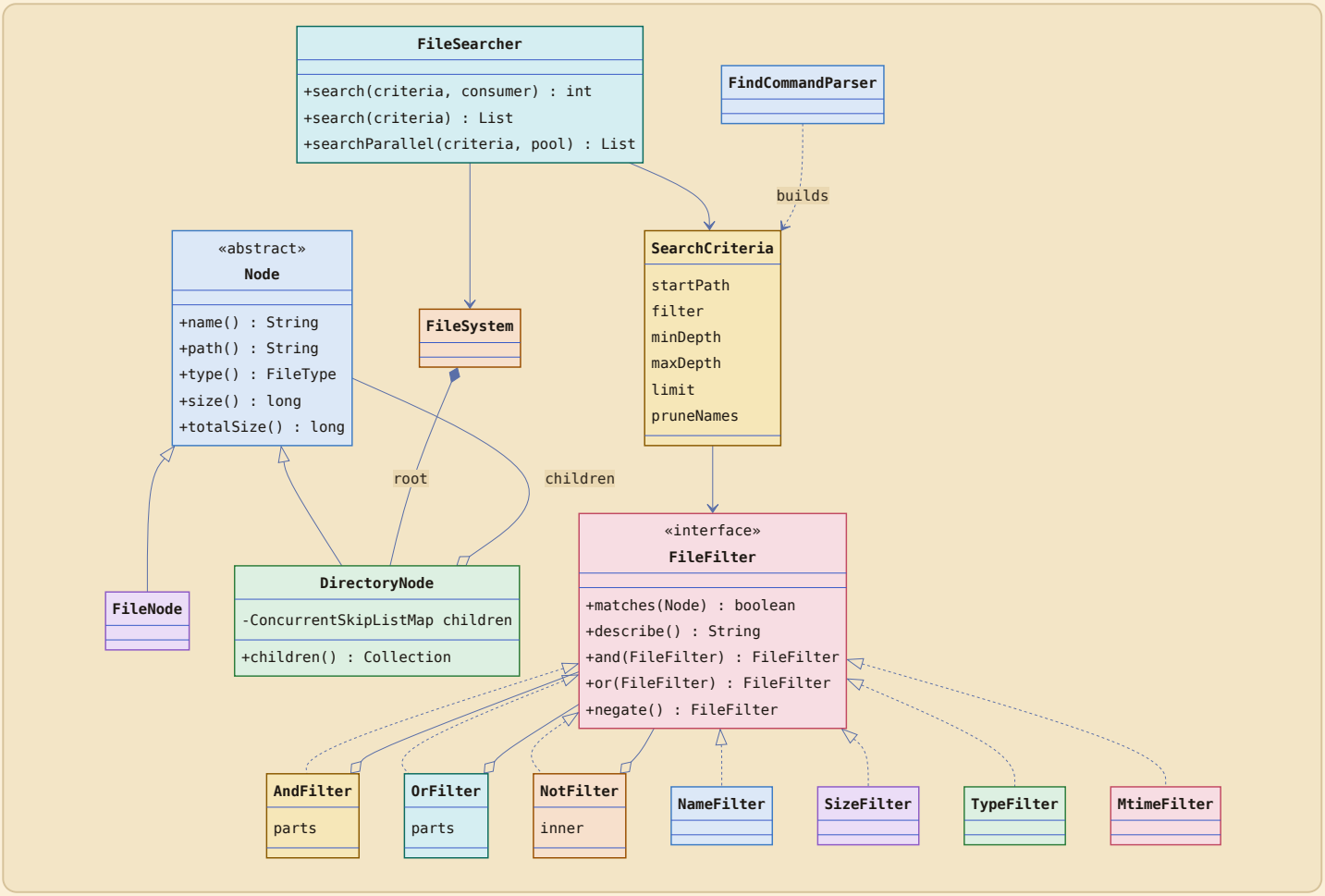
Use cases

Actor	Use cases
Developer (API user)	build criteria with the Builder, run a search, stream results
Shell user (someone typing commands into a terminal)	type a `find` command line, see matching paths or a clear error
Ops script (an operations team's automation script)	find big old logs (`-size +10M -mtime +7`) to archive, find empty directories to clean

Core entities

Class	Kind	Responsibility
`Node`	abstract sealed class (cannot be created directly, and only the listed subclasses may extend it)	**Composite** component: name, parent, owner, modified time, `path()`, `size()`, `totalSize()`.
`FileNode` / `DirectoryNode`	classes	Leaf and container; a directory keeps its children sorted in a `ConcurrentSkipListMap` (a sorted map that many threads can use at once).
`FileSystem`	class	Root, `mkdirs`, `createFile`, `resolve(path)`.
`FileFilter`	interface	**Specification**: `matches(node)`, `describe()`, and default `and` / `or` / `negate`.
`AndFilter`, `OrFilter`, `NotFilter`	records	**Composite** filters: a tree of filters that is itself a filter.
`Filters`	factory	Leaf tests: name/iname glob, type, size, mtime, owner, empty, match-all.
`SearchCriteria` + `Builder`	class	Immutable search description (start, filter, depths, limit, prune set), built fluently and validated once.
`FileSearcher`	class	Iterative DFS (depth-first search) with an explicit stack (a last-in, first-out list); streams results to a `Consumer`; optional fork/join version (Java's framework that splits a task into subtasks run in parallel, then joins the results).
`FindCommandParser`	class	Recursive-descent parser (one method per grammar rule, the methods calling each other) for `find` expressions → `SearchCriteria`.
`SearchException`	exception	Readable failure.

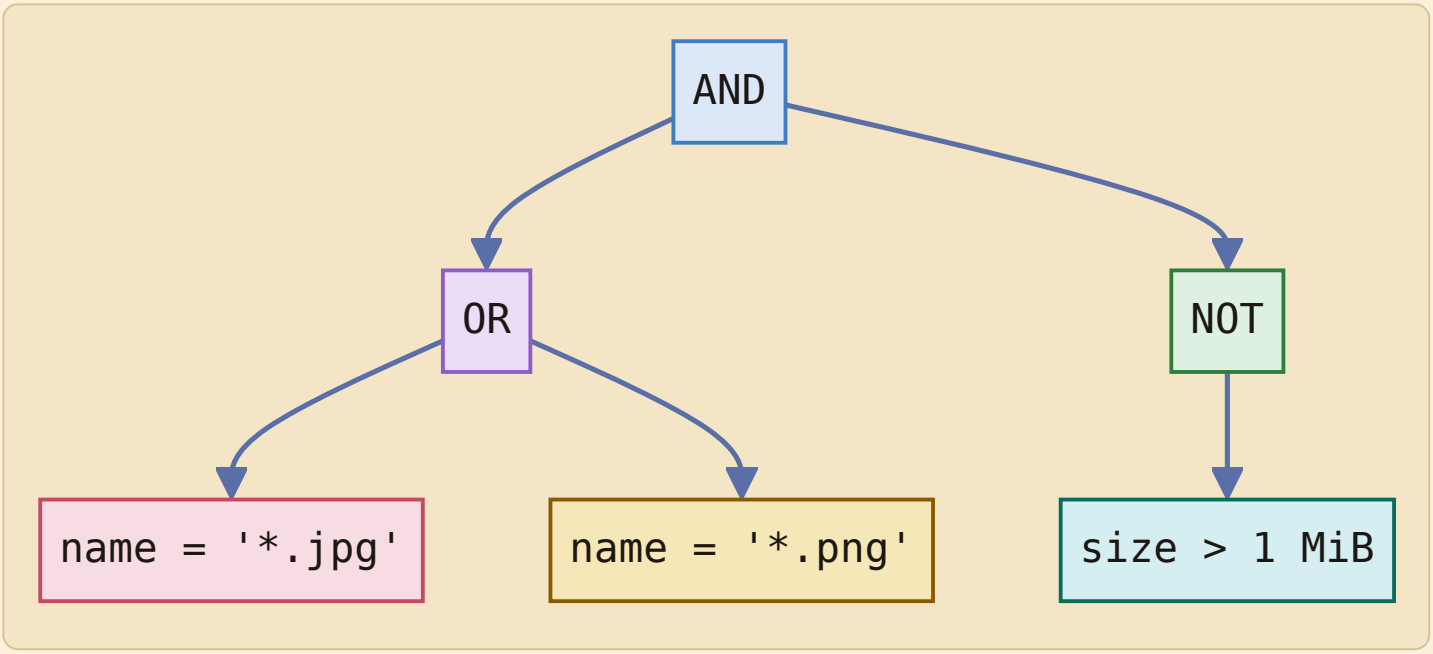
Class diagram



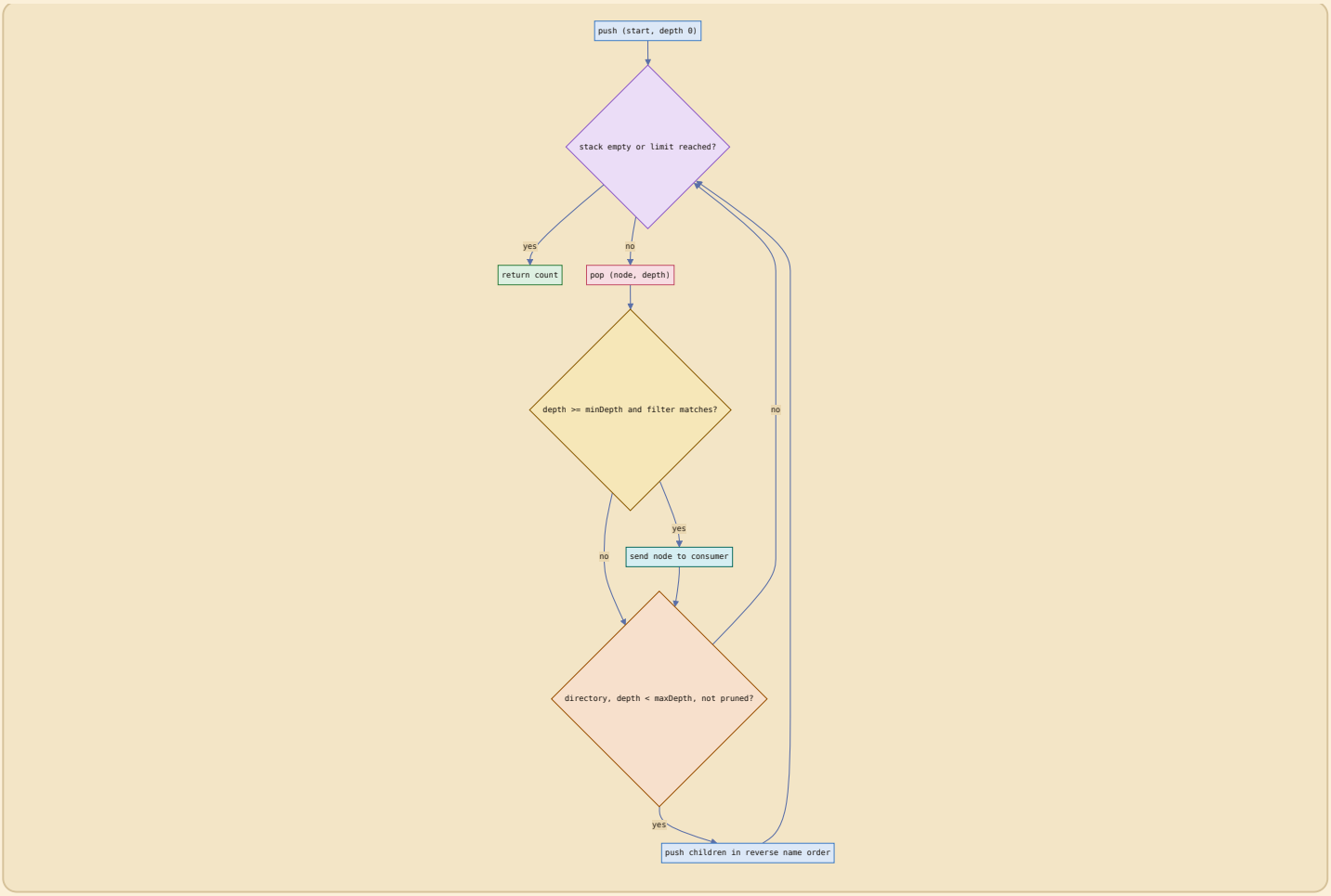
Two Composites sit side by side: the **file tree** (a directory is a node that contains nodes) and the **filter tree** (an AND is a filter that contains filters). The searcher walks the first and asks the second one question per entry.

Key flows

Parsing `find /home ( -name '*.jpg' -o -name '*.png' ) ! -size +1M` produces this filter tree:



The search loop (explicit stack, pre-order):



Operator precedence (from lowest to highest): `-o` (OR) < `-a` or nothing (AND) < `!` (NOT) < `( … )`. So `-name a -o -name b -type f` means `a OR (b AND file)` — a classic surprise, and the reason `find` users write parentheses.

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Each test is a small class with `matches(node)`	**Specification**	A new test is one record; the search loop never changes (Open/Closed)	`search(name, ext, minSize, maxSize, type, …)` with a growing list of `if`s
`AndFilter` / `OrFilter` / `NotFilter` hold other filters	**Composite**	Arbitrary nesting (`(a OR b) AND NOT c`) with one interface; the parser's output *is* this tree; `describe()` prints it	Lambdas (short anonymous functions) only (`a.and(b)`) — work, but cannot be printed or inspected (compare §21.1's `TaskFilter`)
`Node` / `FileNode` / `DirectoryNode`	**Composite**	The searcher and `totalSize()` treat files and directories uniformly	`instanceof` ("is this object of type X?") checks everywhere, or separate file and directory lists
`SearchCriteria.Builder`	**Builder**	Many optional settings; named calls; validation (`mindepth ≤ maxdepth`) once in `build()`; immutable result	Telescoping constructors or a mutable settings bag
Recursive-descent `FindCommandParser`	Interpreter-style parser	One method per precedence level makes precedence explicit and testable	Splitting on `-o` with string hacks
Iterative DFS with an explicit `ArrayDeque`	—	No `StackOverflowError` (the crash when the call stack runs out) on very deep trees; easy to add a limit	Recursion
Results go to a `Consumer<Node>`	Streaming / callback	Prints as it finds, like `find`; memory does not grow with the result count	Always building a full `List`
AND / OR short-circuit (stop as soon as the answer is known)	—	Put cheap tests first (`-name` before a size lookup on a real disk)	Evaluating every test
`ConcurrentSkipListMap` for children	Concurrent collection	Sorted (deterministic output) and safe to iterate while files are added	`TreeMap` (throws `ConcurrentModificationException`)

Java implementation

The file-system model:

```java
// File: lld/ch21/filesearch/FileType.java
package lld.ch21.filesearch;

public enum FileType { FILE, DIRECTORY }
```

```java
// File: lld/ch21/filesearch/SearchException.java
package lld.ch21.filesearch;

/** Bad path, bad option or bad expression: a readable message, never a stack trace for the user. */
public class SearchException extends RuntimeException {
    public SearchException(String message) { super(message); }
}
```

```java
// File: lld/ch21/filesearch/Node.java
package lld.ch21.filesearch;

import java.time.Instant;

/**
 * Composite pattern: a file-system entry is either a leaf (FileNode) or a container
 * (DirectoryNode) that holds more entries. Callers treat both through this one type.
 */
public abstract sealed class Node permits FileNode, DirectoryNode {
    private final String name;
    private final DirectoryNode parent;
    private final String owner;
    private final Instant modified;

    protected Node(String name, DirectoryNode parent, String owner, Instant modified) {
        this.name = name; this.parent = parent; this.owner = owner; this.modified = modified;
    }

    public String name() { return name; }
    public DirectoryNode parent() { return parent; }
    public String owner() { return owner; }
    public Instant modified() { return modified; }

    public abstract FileType type();

    /** Size in bytes of this entry alone (0 for a directory, as far as our filters care). */
    public abstract long size();

    /** du-style size: a file's own size; a directory's is the sum over everything inside it (Composite recursion). */
    public abstract long totalSize();

    public String path() {
        if (parent == null) return "/";
        String p = parent.path();
        return (p.equals("/") ? "" : p) + "/" + name;
    }

    @Override public String toString() { return path(); }
}
```

```java
// File: lld/ch21/filesearch/FileNode.java
package lld.ch21.filesearch;

import java.time.Instant;

public final class FileNode extends Node {
    private final long size;

    FileNode(String name, DirectoryNode parent, String owner, Instant modified, long size) {
        super(name, parent, owner, modified);
        if (size < 0) throw new SearchException("Negative size for " + name);
        this.size = size;
    }

    @Override public FileType type() { return FileType.FILE; }
    @Override public long size() { return size; }
    @Override public long totalSize() { return size; }
}
```

```java
// File: lld/ch21/filesearch/DirectoryNode.java
package lld.ch21.filesearch;

import java.time.Instant;
import java.util.Collection;
import java.util.concurrent.ConcurrentSkipListMap;

public final class DirectoryNode extends Node {
    /** Sorted by name (deterministic output) and safe to read while another thread adds files. */
    private final ConcurrentSkipListMap<String, Node> children = new ConcurrentSkipListMap<>();

    DirectoryNode(String name, DirectoryNode parent, String owner, Instant modified) {
        super(name, parent, owner, modified);
    }

    @Override public FileType type() { return FileType.DIRECTORY; }
    @Override public long size() { return 0; }
    @Override public long totalSize() { return children.values().stream().mapToLong(Node::totalSize).sum(); }

    public Collection<Node> children() { return children.values(); }
    public Node child(String name) { return children.get(name); }
    public boolean isEmpty() { return children.isEmpty(); }

    void add(Node n) {
        if (children.putIfAbsent(n.name(), n) != null)
            throw new SearchException(n.path() + " already exists");
    }
}
```

```java
// File: lld/ch21/filesearch/FileSystem.java
package lld.ch21.filesearch;

import java.time.Instant;

/** A small in-memory file system, so every demo run is identical. A java.nio adapter is an extension. */
public final class FileSystem {
    private final DirectoryNode root;

    public FileSystem(Instant created) { root = new DirectoryNode("", null, "root", created); }

    public DirectoryNode root() { return root; }

    /** Creates missing directories along the way, like mkdir -p. */
    public synchronized DirectoryNode mkdirs(String path, String owner, Instant modified) {
        DirectoryNode dir = root;
        for (String part : split(path)) {
            Node next = dir.child(part);
            if (next == null) { next = new DirectoryNode(part, dir, owner, modified); dir.add(next); }
            if (!(next instanceof DirectoryNode d)) throw new SearchException(next.path() + " is a file, not a
directory");
            dir = d;
        }
        return dir;
    }

    public synchronized FileNode createFile(String path, long size, String owner, Instant modified) {
        int slash = path.lastIndexOf('/');
        DirectoryNode dir = mkdirs(path.substring(0, Math.max(slash, 0)), owner, modified);
        FileNode f = new FileNode(path.substring(slash + 1), dir, owner, modified, size);
        dir.add(f);
        return f;
    }

    public Node resolve(String path) {
        Node n = root;
        for (String part : split(path)) {
            if (!(n instanceof DirectoryNode d) || (n = d.child(part)) == null)
                throw new SearchException("No such file or directory: " + path);
        }
        return n;
    }

    private static String[] split(String path) {
        String p = path.replaceAll("^/+|/+$", "");
        return p.isEmpty() ? new String[0] : p.split("/+");
    }
}
```

The filters — Specification leaves and Composite nodes:

```java
// File: lld/ch21/filesearch/FileFilter.java
package lld.ch21.filesearch;

/**
 * Specification pattern: one question, "does this entry match?". Leaf filters test one attribute;
 * AndFilter / OrFilter / NotFilter are Composite nodes that combine other filters into a tree.
 */
public interface FileFilter {
    boolean matches(Node n);

    /** Human-readable form of the expression tree, e.g. (name='*.log' AND size>+10M). */
    String describe();

    default FileFilter and(FileFilter other) { return AndFilter.of(this, other); }
    default FileFilter or(FileFilter other) { return OrFilter.of(this, other); }
    default FileFilter negate() { return new NotFilter(this); }
}
```

```java
// File: lld/ch21/filesearch/AndFilter.java
package lld.ch21.filesearch;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

/** Composite: matches when every child matches. Short-circuits like find's -a. */
public record AndFilter(List<FileFilter> parts) implements FileFilter {
    public AndFilter { parts = List.copyOf(parts); }

    /** Flattens nested ANDs so (a AND (b AND c)) is stored as (a AND b AND c). */
    static FileFilter of(FileFilter a, FileFilter b) {
        List<FileFilter> all = new ArrayList<>();
        for (FileFilter f : List.of(a, b)) if (f instanceof AndFilter and) all.addAll(and.parts()); else all.add(f);
        return new AndFilter(all);
    }

    @Override public boolean matches(Node n) {
        for (FileFilter f : parts) if (!f.matches(n)) return false;
        return true;
    }

    @Override public String describe() {
        return parts.stream().map(FileFilter::describe).collect(Collectors.joining(" AND ", "(", ")"));
    }
}
```

```java
// File: lld/ch21/filesearch/OrFilter.java
package lld.ch21.filesearch;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

/** Composite: matches when any child matches. Short-circuits like find's -o. */
public record OrFilter(List<FileFilter> parts) implements FileFilter {
    public OrFilter { parts = List.copyOf(parts); }

    static FileFilter of(FileFilter a, FileFilter b) {
        List<FileFilter> all = new ArrayList<>();
        for (FileFilter f : List.of(a, b)) if (f instanceof OrFilter or) all.addAll(or.parts()); else all.add(f);
        return new OrFilter(all);
    }

    @Override public boolean matches(Node n) {
        for (FileFilter f : parts) if (f.matches(n)) return true;
        return false;
    }

    @Override public String describe() {
        return parts.stream().map(FileFilter::describe).collect(Collectors.joining(" OR ", "(", ")"));
    }
}
```

```java
// File: lld/ch21/filesearch/NotFilter.java
package lld.ch21.filesearch;

/** Composite with one child: find's ! / -not. */
public record NotFilter(FileFilter inner) implements FileFilter {
    @Override public boolean matches(Node n) { return !inner.matches(n); }
    @Override public String describe() { return "NOT " + inner.describe(); }
}
```

```java
// File: lld/ch21/filesearch/Filters.java
package lld.ch21.filesearch;

import java.time.Duration;
import java.time.Instant;
import java.util.regex.Pattern;

/** Static factories for the leaf filters (one per find test). New test = one new record here; nothing else changes.
*/
public final class Filters {
    private Filters() { }

    public static FileFilter all() { return new MatchAll(); }
    public static FileFilter name(String glob) { return new NameFilter(glob, false); }
    public static FileFilter iname(String glob) { return new NameFilter(glob, true); }
    public static FileFilter type(FileType t) { return new TypeFilter(t); }
    public static FileFilter owner(String user) { return new OwnerFilter(user); }
    public static FileFilter empty() { return new EmptyFilter(); }
    /** spec like find's -size: "+10M", "-1k", "0c", "20" (no suffix = 512-byte blocks). */
    public static FileFilter size(String spec) { return SizeFilter.parse(spec); }
    /** spec like find's -mtime: "+7" (more than 7 whole days ago), "-1", "0". */
    public static FileFilter mtime(String spec, Instant now) { return MtimeFilter.parse(spec, now); }

    record MatchAll() implements FileFilter {
        @Override public boolean matches(Node n) { return true; }
        @Override public String describe() { return "true"; }
    }

    /** Glob on the last path component only, like find -name: * ? [abc] [!abc], and \ to escape. */
    record NameFilter(String glob, boolean ignoreCase, Pattern regex) implements FileFilter {
        NameFilter(String glob, boolean ignoreCase) {
            this(glob, ignoreCase, Pattern.compile(toRegex(glob), ignoreCase ? Pattern.CASE_INSENSITIVE |
Pattern.UNICODE_CASE : 0));
        }
        @Override public boolean matches(Node n) { return regex.matcher(n.name()).matches(); }
        @Override public String describe() { return (ignoreCase ? "iname='" : "name='") + glob + "'"; }

        static String toRegex(String glob) {
            StringBuilder re = new StringBuilder();
            for (int i = 0; i < glob.length(); i++) {
                char c = glob.charAt(i);
                switch (c) {
                    case '*' -> re.append(".*");
                    case '?' -> re.append('.');
                    case '\\' -> { if (++i < glob.length()) re.append(Pattern.quote(String.valueOf(glob.charAt(i)))); }
                    case '[' -> {
                        int end = glob.indexOf(']', i + 2);
                        if (end < 0) { re.append("\\["); break; }          // no closing ]: a literal [
                        String body = glob.substring(i + 1, end);
                        if (body.startsWith("!")) body = "^" + body.substring(1);
                        re.append('[').append(body.replace("\\", "\\\\")).append(']');
                        i = end;
                    }
                    default -> re.append(Pattern.quote(String.valueOf(c)));
                }
            }
            return re.toString();
        }
    }

    record TypeFilter(FileType type) implements FileFilter {
        @Override public boolean matches(Node n) { return n.type() == type; }
        @Override public String describe() { return "type=" + (type == FileType.FILE ? "f" : "d"); }
    }

    record OwnerFilter(String user) implements FileFilter {
        @Override public boolean matches(Node n) { return n.owner().equals(user); }
        @Override public String describe() { return "user=" + user; }
    }

    record EmptyFilter() implements FileFilter {
        @Override public boolean matches(Node n) {
```

```
            return n instanceof DirectoryNode d ? d.isEmpty() : n.size() == 0;
        }
        @Override public String describe() { return "empty"; }
    }

    /**
     * GNU find semantics: the file size is rounded UP to whole units, then compared.
     * So "-size -1M" means "fewer than 1 whole MiB after rounding up" = only empty files. A famous gotcha.
     * Size tests apply to files only in this model.
     */
    record SizeFilter(char cmp, long n, char unit, String spec) implements FileFilter {
        static SizeFilter parse(String spec) {
            var m = Pattern.compile("([+-]?)(\\d+)([cbkMG]?)").matcher(spec);
            if (!m.matches()) throw new SearchException("Invalid -size '" + spec + "' (use e.g. +10M, -1k, 100c)");
            char cmp = m.group(1).isEmpty() ? '=' : m.group(1).charAt(0);
            char unit = m.group(3).isEmpty() ? 'b' : m.group(3).charAt(0);
            return new SizeFilter(cmp, Long.parseLong(m.group(2)), unit, spec);
        }
        long unitBytes() {
            return switch (unit) { case 'c' -> 1; case 'b' -> 512; case 'k' -> 1024; case 'M' -> 1024L * 1024; default
-> 1024L * 1024 * 1024; };
        }
        @Override public boolean matches(Node node) {
            if (node.type() != FileType.FILE) return false;
            long units = (node.size() + unitBytes() - 1) / unitBytes();      // round up
            return switch (cmp) { case '+' -> units > n; case '-' -> units < n; default -> units == n; };
        }
        @Override public String describe() { return "size" + (cmp == '=' ? "=" : "") + spec; }
    }

    /** -mtime n: age in whole days (fraction dropped); +n = older than n days, -n = newer than n days. */
    record MtimeFilter(char cmp, long days, Instant now, String spec) implements FileFilter {
        static MtimeFilter parse(String spec, Instant now) {
            var m = Pattern.compile("([+-]?)(\\d+)").matcher(spec);
            if (!m.matches()) throw new SearchException("Invalid -mtime '" + spec + "' (use e.g. +7, -1, 0)");
            char cmp = m.group(1).isEmpty() ? '=' : m.group(1).charAt(0);
            return new MtimeFilter(cmp, Long.parseLong(m.group(2)), now, spec);
        }
        @Override public boolean matches(Node node) {
            long age = Duration.between(node.modified(), now).toDays();
            return switch (cmp) { case '+' -> age > days; case '-' -> age < days; default -> age == days; };
        }
        @Override public String describe() { return "mtime" + (cmp == '=' ? "=" : "") + spec; }
    }
}
```

Criteria, search and parser:

```java
// File: lld/ch21/filesearch/SearchCriteria.java
package lld.ch21.filesearch;

import java.util.LinkedHashSet;
import java.util.Set;

/**
 * Everything one search needs, immutable once built. Builder pattern: most options are optional,
 * so named builder calls replace a constructor with eight positional parameters.
 */
public final class SearchCriteria {
    private final String startPath;
    private final FileFilter filter;
    private final int minDepth, maxDepth, limit;
    private final Set<String> pruneNames;

    private SearchCriteria(Builder b) {
        startPath = b.startPath; filter = b.filter; minDepth = b.minDepth; maxDepth = b.maxDepth;
        limit = b.limit; pruneNames = Set.copyOf(b.pruneNames);
    }

    public static Builder from(String startPath) { return new Builder(startPath); }

    public String startPath() { return startPath; }
    public FileFilter filter() { return filter; }
    public int minDepth() { return minDepth; }
    public int maxDepth() { return maxDepth; }
    public int limit() { return limit; }
    public Set<String> pruneNames() { return pruneNames; }

    @Override public String toString() {
        return "from " + startPath + " where " + filter.describe()
                + (minDepth > 0 ? " mindepth=" + minDepth : "")
                + (maxDepth < Integer.MAX_VALUE ? " maxdepth=" + maxDepth : "")
                + (pruneNames.isEmpty() ? "" : " prune=" + pruneNames)
                + (limit < Integer.MAX_VALUE ? " limit=" + limit : "");
    }

    public static final class Builder {
        private final String startPath;
        private FileFilter filter = Filters.all();
        private boolean filterSet;
        private int minDepth = 0, maxDepth = Integer.MAX_VALUE, limit = Integer.MAX_VALUE;
        private final Set<String> pruneNames = new LinkedHashSet<>();

        private Builder(String startPath) {
            if (startPath == null || startPath.isBlank()) throw new SearchException("Start path is required");
            this.startPath = startPath;
        }

        /** Each call is ANDed with the previous ones, like consecutive tests on a find command line. */
        public Builder where(FileFilter f) { filter = filterSet ? filter.and(f) : f; filterSet = true; return this; }
        public Builder name(String glob) { return where(Filters.name(glob)); }
        public Builder type(FileType t) { return where(Filters.type(t)); }
        public Builder size(String spec) { return where(Filters.size(spec)); }
        public Builder minDepth(int d) { minDepth = d; return this; }
        public Builder maxDepth(int d) { maxDepth = d; return this; }
        public Builder limit(int n) { limit = n; return this; }
        /** Do not descend into directories with this name (like: -name node_modules -prune -o ...). */
        public Builder prune(String dirName) { pruneNames.add(dirName); return this; }

        public SearchCriteria build() {
            if (minDepth < 0 || maxDepth < 0) throw new SearchException("Depths must be >= 0");
            if (minDepth > maxDepth) throw new SearchException("mindepth " + minDepth + " > maxdepth " + maxDepth);
            if (limit <= 0) throw new SearchException("Limit must be positive");
            return new SearchCriteria(this);
        }
    }
}
```

```java
// File: lld/ch21/filesearch/FileSearcher.java
package lld.ch21.filesearch;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Deque;
import java.util.List;
import java.util.concurrent.ForkJoinPool;
import java.util.concurrent.RecursiveTask;
import java.util.function.Consumer;

/** Walks the tree and reports every entry that passes the filter. */
public final class FileSearcher {
    private final FileSystem fs;

    public FileSearcher(FileSystem fs) { this.fs = fs; }

    private record Frame(Node node, int depth) { }

    /**
     * Pre-order depth-first walk with an explicit stack (no recursion, so a very deep tree cannot
     * overflow the call stack). Results are streamed to the consumer as they are found, like find
     * printing line by line; nothing forces the whole result list into memory.
     */
    public int search(SearchCriteria c, Consumer<Node> out) {
        Deque<Frame> stack = new ArrayDeque<>();
        stack.push(new Frame(fs.resolve(c.startPath()), 0));
        int found = 0;
        while (!stack.isEmpty() && found < c.limit()) {
            Frame f = stack.pop();
            Node n = f.node();
            if (f.depth() >= c.minDepth() && c.filter().matches(n)) { out.accept(n); found++; }
            if (n instanceof DirectoryNode d && f.depth() < c.maxDepth() && !c.pruneNames().contains(d.name())) {
                List<Node> kids = new ArrayList<>(d.children());
                Collections.reverse(kids);                    // push in reverse so we pop in name order
                for (Node k : kids) stack.push(new Frame(k, f.depth() + 1));
            }
        }
        return found;
    }

    public List<Node> search(SearchCriteria c) {
        List<Node> out = new ArrayList<>();
        search(c, out::add);
        return out;
    }

    /** Extension: one fork/join task per directory. Same results in the same order; no limit support. */
    public List<Node> searchParallel(SearchCriteria c, ForkJoinPool pool) {
        return pool.invoke(new Walk(fs.resolve(c.startPath()), 0, c));
    }

    private static final class Walk extends RecursiveTask<List<Node>> {
        private final Node node; private final int depth; private final SearchCriteria c;
        Walk(Node node, int depth, SearchCriteria c) { this.node = node; this.depth = depth; this.c = c; }

        @Override protected List<Node> compute() {
            List<Node> out = new ArrayList<>();
            if (depth >= c.minDepth() && c.filter().matches(node)) out.add(node);
            if (node instanceof DirectoryNode d && depth < c.maxDepth() && !c.pruneNames().contains(d.name())) {
                List<Walk> tasks = new ArrayList<>();
                List<Object> slots = new ArrayList<>();         // files tested inline, sub-directories forked
                for (Node k : d.children()) {
                    if (k instanceof DirectoryNode) { Walk w = new Walk(k, depth + 1, c); tasks.add(w); slots.add(w);
}
                    else slots.add(k);
                }
                invokeAll(tasks);                               // fork all sub-directories, wait for all
                for (Object s : slots) {                        // collect in name order: same order as search()
                    if (s instanceof Walk w) out.addAll(w.join());
                    else if (depth + 1 >= c.minDepth() && c.filter().matches((Node) s)) out.add((Node) s);
                }
```

```
            }
            return out;
        }
    }
}
```

```java
// File: lld/ch21/filesearch/FindCommandParser.java
package lld.ch21.filesearch;

import java.time.Instant;
import java.util.ArrayList;
import java.util.List;
import java.util.regex.Pattern;

/**
 * Turns a find-style command line into SearchCriteria, with find's precedence:
 *   expr   := or
 *   or     := and ( ("-o" | "-or") and )*
 *   and    := unary ( ["-a" | "-and"] unary )*      -- two tests side by side mean AND
 *   unary  := ("!" | "-not") unary | "(" expr ")" | test
 *   test   := -name G | -iname G | -type f|d | -size N | -mtime N | -user U | -empty | -true
 * Global options -maxdepth N / -mindepth N may appear anywhere. This is recursive descent:
 * one method per grammar rule, each returning a FileFilter (the Composite tree).
 */
public final class FindCommandParser {
    private final Instant now;
    private List<String> tokens;
    private int pos;
    private SearchCriteria.Builder builder;

    public FindCommandParser(Instant now) { this.now = now; }

    /** tokens as the shell would pass them, e.g. ["find", "/home", "-name", "*.java", "-o", "-name", "*.md"]. */
    public SearchCriteria parse(List<String> args) {
        if (args.isEmpty() || !args.get(0).equals("find")) throw new SearchException("Command must start with
'find'");
        tokens = args; pos = 1;
        String start = pos < tokens.size() && !isOperatorOrTest(tokens.get(pos)) ? tokens.get(pos++) : ".";
        builder = SearchCriteria.from(start.equals(".") ? "/" : start);
        skipGlobalOptions();
        if (pos < tokens.size()) {                    // no tests at all = match everything
            builder.where(parseOr());
            if (pos < tokens.size()) throw new SearchException("Unexpected '" + tokens.get(pos) + "'");
        }
        return builder.build();
    }

    /** Convenience: split on spaces; quotes group words and are removed (enough for the demo). */
    public SearchCriteria parse(String commandLine) {
        List<String> out = new ArrayList<>();
        var m = Pattern.compile("'([^']*)'|\"([^\"]*)\"|(\\S+)").matcher(commandLine);
        while (m.find()) out.add(m.group(1) != null ? m.group(1) : m.group(2) != null ? m.group(2) : m.group(3));
        return parse(out);
    }

    private FileFilter parseOr() {
        FileFilter left = parseAnd();
        while (peekIs("-o", "-or")) { pos++; left = left.or(parseAnd()); }
        return left;
    }

    private FileFilter parseAnd() {
        FileFilter left = parseUnary();
        while (true) {
            skipGlobalOptions();
            if (pos >= tokens.size() || peekIs("-o", "-or", ")")) return left;
            if (peekIs("-a", "-and")) pos++;
            left = left.and(parseUnary());
        }
    }

    private FileFilter parseUnary() {
        skipGlobalOptions();
        if (pos >= tokens.size()) throw new SearchException("Expression ends too early");
        String t = tokens.get(pos++);
        switch (t) {
            case "!", "-not" -> { return parseUnary().negate(); }
            case "(" -> {
                FileFilter inner = parseOr();
```

```
                if (!peekIs(")")) throw new SearchException("Missing ')'");
                pos++;
                return inner;
            }
            case ")" -> throw new SearchException("Unexpected ')'");
            case "-empty" -> { return Filters.empty(); }
            case "-true" -> { return Filters.all(); }
            case "-name" -> { return Filters.name(arg(t)); }
            case "-iname" -> { return Filters.iname(arg(t)); }
            case "-user" -> { return Filters.owner(arg(t)); }
            case "-size" -> { return Filters.size(arg(t)); }
            case "-mtime" -> { return Filters.mtime(arg(t), now); }
            case "-type" -> {
                String v = arg(t);
                return switch (v) {
                    case "f" -> Filters.type(FileType.FILE);
                    case "d" -> Filters.type(FileType.DIRECTORY);
                    default -> throw new SearchException("-type takes f or d, not '" + v + "'");
                };
            }
            default -> throw new SearchException("Unknown predicate '" + t + "'");
        }
    }

    /** -maxdepth/-mindepth are options, not tests: record them and keep parsing the expression. */
    private void skipGlobalOptions() {
        while (peekIs("-maxdepth", "-mindepth")) {
            String opt = tokens.get(pos++);
            int n;
            try { n = Integer.parseInt(arg(opt)); } catch (NumberFormatException e) { throw new SearchException(opt +
" needs a number"); }
            if (opt.equals("-maxdepth")) builder.maxDepth(n); else builder.minDepth(n);
        }
    }

    private String arg(String option) {
        if (pos >= tokens.size()) throw new SearchException(option + " needs an argument");
        return tokens.get(pos++);
    }

    private boolean peekIs(String... options) {
        if (pos >= tokens.size()) return false;
        for (String o : options) if (tokens.get(pos).equals(o)) return true;
        return false;
    }

    private static boolean isOperatorOrTest(String t) { return t.startsWith("-") || t.equals("!") || t.equals("(") ||
t.equals(")"); }
}
```

```java
// File: lld/ch21/filesearch/Main.java
package lld.ch21.filesearch;

import java.time.Duration;
import java.time.Instant;
import java.util.List;
import java.util.concurrent.ForkJoinPool;

public class Main {
    static final Instant NOW = Instant.parse("2026-10-01T12:00:00Z");
    static final long KB = 1024, MB = 1024 * KB;

    public static void main(String[] args) {
        FileSystem fs = sampleTree();
        FileSearcher finder = new FileSearcher(fs);
        FindCommandParser parser = new FindCommandParser(NOW);

        System.out.println("== Builder API ==");
        show(finder, SearchCriteria.from("/").name("*.log").type(FileType.FILE).build());
        show(finder, SearchCriteria.from("/home").where(Filters.iname("*.jpg").and(Filters.size("+1M"))).build());
        show(finder, SearchCriteria.from("/").type(FileType.DIRECTORY).where(Filters.empty()).build());
        show(finder, SearchCriteria.from("/home").maxDepth(2).build());
        show(finder, SearchCriteria.from("/home/asha/projects")
                .where(Filters.name("*.java").or(Filters.name("*.js"))).prune("node_modules").build());
        show(finder, SearchCriteria.from("/").type(FileType.FILE).limit(3).build());

        System.out.println("\n== find command lines ==");
        for (String cmd : List.of(
                "find /var/log -type f -size +10M -mtime +7",
                "find /home -name '*.java' -o -name '*.md'",
                "find /home ( -name '*.jpg' -o -name '*.png' ) ! -size +1M",
                "find /home -size -1M",
                "find / -user ravi -type f",
                "find /home/asha -mindepth 2 -maxdepth 2 -type d",
                "find /home -iname 'b*'",
                "find /var/log -name 'app.log.[0-9]'"))
            show(finder, parser.parse(cmd), cmd);

        System.out.println("\n== bad input gives a clear message ==");
        for (String cmd : List.of("find / -size 10X", "find / ( -name a", "find /nope -name x",
                "find / -frobnicate", "find / -type x", "find / -maxdepth 1 -mindepth 3", "find / -name"))
            try { finder.search(parser.parse(cmd)); System.out.println("   (no error?) " + cmd); }
            catch (SearchException e) { System.out.printf("   %-34s -> %s%n", cmd, e.getMessage()); }

        System.out.println("\n== Composite sizes (du -s) ==");
        for (String p : List.of("/home/asha", "/home/asha/projects/app/node_modules", "/var/log"))
            System.out.printf("   %-40s %,d bytes%n", p, fs.resolve(p).totalSize());

        System.out.println("\n== Parallel search on a big generated tree ==");
        FileSystem big = new FileSystem(NOW);
        for (int d = 0; d < 200; d++)
            for (int f = 0; f < 100; f++)
                big.createFile("/data/d%03d/sub%d/f%03d.%s".formatted(d, f % 5, f, f % 3 == 0 ? "csv" : "txt"),
                        (long) f * 10 * KB, "etl", NOW.minus(Duration.ofDays(f % 30)));
        SearchCriteria q = new FindCommandParser(NOW).parse("find /data -name '*.csv' -size +500k -mtime -10");
        FileSearcher bigFinder = new FileSearcher(big);
        List<Node> seq = bigFinder.search(q);
        List<Node> par = bigFinder.searchParallel(q, ForkJoinPool.commonPool());
        System.out.println("   " + q);
        System.out.println("   sequential=" + seq.size() + " parallel=" + par.size() + " same order=" +
seq.equals(par)
                + " first=" + seq.get(0) + " last=" + seq.get(seq.size() - 1));
    }

    static void show(FileSearcher finder, SearchCriteria c) { show(finder, c, c.toString()); }

    static void show(FileSearcher finder, SearchCriteria c, String title) {
        System.out.println("$ " + title);
        if (!title.equals(c.toString())) System.out.println("  parsed: " + c);
        int n = finder.search(c, node -> System.out.println("   " + node + (node instanceof DirectoryNode ? "/" :
"")));
        if (n == 0) System.out.println("   (no matches)");
    }
```

```
    static FileSystem sampleTree() {
        FileSystem fs = new FileSystem(NOW);
        Instant today = NOW.minus(Duration.ofHours(3)), lastWeek = NOW.minus(Duration.ofDays(8)), old =
NOW.minus(Duration.ofDays(40));
        fs.createFile("/home/asha/notes.txt", 2 * KB, "asha", today);
        fs.createFile("/home/asha/photos/a.jpg", 3 * MB, "asha", lastWeek);
        fs.createFile("/home/asha/photos/b.JPG", 5 * MB, "asha", lastWeek);
        fs.createFile("/home/asha/photos/c.png", 800 * KB, "asha", old);
        fs.createFile("/home/asha/photos/d.jpg", 900 * KB, "asha", old);
        fs.createFile("/home/asha/projects/app/Main.java", 12 * KB, "asha", today);
        fs.createFile("/home/asha/projects/app/Util.java", 4 * KB, "asha", today);
        fs.createFile("/home/asha/projects/app/build.log", 3 * MB / 2, "asha", today);
        fs.createFile("/home/asha/projects/app/node_modules/lodash.js", 500 * KB, "asha", old);
        fs.createFile("/home/asha/projects/app/node_modules/react.js", MB, "asha", old);
        fs.mkdirs("/home/asha/projects/empty-dir", "asha", old);
        fs.createFile("/home/ravi/todo.md", 0, "ravi", today);
        fs.createFile("/home/ravi/report.pdf", 2 * MB, "ravi", lastWeek);
        fs.createFile("/var/log/syslog", 40 * MB, "root", old);
        fs.createFile("/var/log/app.log", 3 * MB, "root", today);
        fs.createFile("/var/log/app.log.1", 20 * MB, "root", lastWeek);
        fs.mkdirs("/tmp", "root", today);
        return fs;
    }
}
```

How the code works, part by part

- **Globs.** `NameFilter.toRegex` turns `*` into `.*`, `?` into `.`, keeps `[...]` sets (turning `[!...]` into `[^...]`) and quotes every other character, then compiles the regex once, when the filter is built. Like `find -name`, it matches only the last path component, and `*` also matches names that start with a dot.
- **Size.** `SizeFilter` parses `[+-]N[cbkMG]` (no suffix = 512-byte blocks, as in `find`), rounds the file size **up** to whole units, then compares: `+` means more than N units, `-` fewer, no sign exactly N.
- **Age.** `MtimeFilter` computes the age in whole days (the fraction is dropped) against an injected `now`, so the demo is repeatable.
- **Builder.** Each `where(...)` call is ANDed with earlier ones; `build()` checks depths and the limit and freezes everything into an immutable `SearchCriteria`.
- **Search.** `FileSearcher.search` pushes the start node, then pops, tests, emits and pushes children in reverse name order so they come off the stack in name order. It stops descending at `maxDepth` and at pruned directory names, and stops entirely at the limit.
- **Parser.** `parseOr` → `parseAnd` → `parseUnary`: one method per precedence level. `parseAnd` treats two tests side by side as AND; `-maxdepth`/`-mindepth` are options that can appear anywhere, so every level skips them first.
- **Parallel.** `searchParallel` forks one task per sub-directory (files are tested inline), waits for all, then joins results in name order — the same list, in the same order, as the sequential search.

Output:

```
== Builder API ==
$ from / where (name='*.log' AND type=f)
   /home/asha/projects/app/build.log
   /var/log/app.log
$ from /home where (iname='*.jpg' AND size+1M)
   /home/asha/photos/a.jpg
   /home/asha/photos/b.JPG
$ from / where (type=d AND empty)
   /home/asha/projects/empty-dir/
   /tmp/
$ from /home where true maxdepth=2
   /home/
   /home/asha/
   /home/asha/notes.txt
   /home/asha/photos/
   /home/asha/projects/
   /home/ravi/
   /home/ravi/report.pdf
   /home/ravi/todo.md
$ from /home/asha/projects where (name='*.java' OR name='*.js') prune=[node_modules]
   /home/asha/projects/app/Main.java
   /home/asha/projects/app/Util.java
$ from / where type=f limit=3
   /home/asha/notes.txt
   /home/asha/photos/a.jpg
   /home/asha/photos/b.JPG

== find command lines ==
$ find /var/log -type f -size +10M -mtime +7
  parsed: from /var/log where (type=f AND size+10M AND mtime+7)
   /var/log/app.log.1
   /var/log/syslog
$ find /home -name '*.java' -o -name '*.md'
  parsed: from /home where (name='*.java' OR name='*.md')
   /home/asha/projects/app/Main.java
   /home/asha/projects/app/Util.java
   /home/ravi/todo.md
$ find /home ( -name '*.jpg' -o -name '*.png' ) ! -size +1M
  parsed: from /home where ((name='*.jpg' OR name='*.png') AND NOT size+1M)
   /home/asha/photos/c.png
   /home/asha/photos/d.jpg
$ find /home -size -1M
  parsed: from /home where size-1M
   /home/ravi/todo.md
$ find / -user ravi -type f
  parsed: from / where (user=ravi AND type=f)
   /home/ravi/report.pdf
   /home/ravi/todo.md
$ find /home/asha -mindepth 2 -maxdepth 2 -type d
  parsed: from /home/asha where type=d mindepth=2 maxdepth=2
   /home/asha/projects/app/
   /home/asha/projects/empty-dir/
$ find /home -iname 'b*'
  parsed: from /home where iname='b*'
   /home/asha/photos/b.JPG
   /home/asha/projects/app/build.log
$ find /var/log -name 'app.log.[0-9]'
  parsed: from /var/log where name='app.log.[0-9]'
   /var/log/app.log.1

== bad input gives a clear message ==
   find / -size 10X                  -> Invalid -size '10X' (use e.g. +10M, -1k, 100c)
   find / ( -name a                  -> Missing ')'
   find /nope -name x                -> No such file or directory: /nope
   find / -frobnicate                -> Unknown predicate '-frobnicate'
   find / -type x                    -> -type takes f or d, not 'x'
   find / -maxdepth 1 -mindepth 3    -> mindepth 3 > maxdepth 1
   find / -name                      -> -name needs an argument

== Composite sizes (du -s) ==
   /home/asha                              13,281,280 bytes
   /home/asha/projects/app/node_modules     1,560,576 bytes
   /var/log                                66,060,288 bytes
```

```
== Parallel search on a big generated tree ==
   from /data where (name='*.csv' AND size+500k AND mtime-10)
   sequential=1600 parallel=1600 same order=true first=/data/d000/sub0/f060.csv last=/data/d199/sub4/f099.csv
```

Reading the output: `-size +1M` kept the two large photos and dropped the 900 KB one; the prune option skipped `node_modules`, so only the two Java files match `*.java OR *.js`; `-size -1M` found only the empty `todo.md` — the rounding gotcha, even though `notes.txt` is only 2 KB; `-mindepth 2 -maxdepth 2 -type d` lists directories exactly two levels below `/home/asha`; the bracket glob `app.log.[0-9]` matches the rotated log but not `app.log`. Every bad input produced a one-line message. The composite `totalSize()` gives `du`-style totals (`du` is the Unix disk-usage command), and on a generated tree of 20,000 files the sequential and parallel searches returned the same 1,600 paths in the same order.

Concurrency & edge cases

- **Adding files during a search:** children live in a `ConcurrentSkipListMap`, whose iterators are *weakly consistent* — iterators (the objects that step through a collection) never throw, and may or may not show an entry added after the iteration started (like `find` on a live disk).
- **Parallel search:** tasks only read the tree and each builds its own list; there is no shared mutable state, so no locks are needed.
- **Deep trees:** the sequential walk uses a heap-allocated stack (an ordinary object in main memory, not the limited call stack). (The fork/join version recurses one task per directory; fine for normal depths.)
- **Symbolic links / cycles** (real file systems): track visited `(device, inode)` pairs (an inode is the file system's internal id for a file; a symbolic link is a file that points to another path) or do not follow links by default — GNU `find` does not follow symbolic links unless asked (`-L`).
- **Edge cases handled:** start path missing or a file; `mindepth > maxdepth`; unknown predicate; missing argument; bad `-size`/`-mtime`/`-type` values; unbalanced parentheses; an unclosed `[` in a glob is treated as a literal; limit reached mid-walk.

Extensions & follow-up questions

- **Search a real disk?** An adapter that wraps `java.nio.file.Path` + `BasicFileAttributes` as `Node`s, or use `Files.walkFileTree` with a `FileVisitor` that returns `SKIP_SUBTREE` for pruned directories.
- **Actions (`-delete`, `-exec`)?** A `Consumer<Node>` per action (Command pattern); `-delete` must walk depth-first *post-order*, meaning children are visited before their directory.
- **Content search (`grep -r`, the Unix tool that searches inside files)?** A `ContainsTextFilter` — put it last in the AND so cheap name/size tests reject most files first.
- **Faster repeated searches?** Build an index (name → paths, like `locate`/`updatedb`, Unix tools that pre-build a filename database) and accept that it can be stale.
- **More tests (`-newer file`, `-perm`, `-regex`)?** One record each in `Filters` plus one `case` in the parser.
- **Sort by size or date?** Collect, then sort with a `Comparator` — a Strategy, as in PropertyHunt (§21.5).

Real-world example

GNU `find` (from GNU findutils) evaluates its expression left to right with exactly this precedence (`!`, then implicit or explicit `-a`, then `-o`) and short-circuits; its manual documents the "`-size -1M` only matches empty files" rounding and the default of not following symbolic links. Java's own `Files.find(start, maxDepth, BiPredicate)` and `Files.walk` take a depth limit and a predicate — the same split between *walking* and *testing* used here. IDE "Find in files" dialogs and desktop search boxes build the same kind of filter tree from checkboxes instead of a command line.

In the interview

Spend the first 10 minutes on the model (`Node`, `FileNode`, `DirectoryNode`) and the `FileFilter` interface with two leaves (name, size) — that alone lets you demo a search. Then add `AndFilter`/`OrFilter`/`NotFilter`, the Builder with depth options, and the iterative walk. Add the parser last, and only if time allows: interviewers mostly score the filter design and extensibility. Say the Open/Closed point out loud: "a new filter is one class; the searcher never changes".

Self-check

- **Q:** What does `find -name a -o -name b -type f` mean? → **A:** `a OR (b AND file)`, because AND binds tighter than OR.
- **Q:** Why does `-size -1M` match only empty files? → **A:** Sizes are rounded up to whole MiB, so any non-empty file is at least 1 unit, which is not less than 1.
- **Q:** Why are the AND/OR/NOT filters records rather than lambdas? → **A:** They can be printed (`describe()`), inspected and built by the parser as a visible tree.
- **Q:** How does pruning differ from filtering out a directory? → **A:** Pruning stops the walk from entering it (nothing inside is visited); a filter only hides the directory entry itself.
- **Q:** Why an explicit stack instead of recursion? → **A:** Very deep trees would overflow the call stack; an explicit stack also makes the limit easy.

TL;DR

Two Composites — the file tree and the filter tree. Filters are Specifications (`matches`) combined with AND / OR / NOT; the Builder describes the search once and validates it; an iterative DFS streams matches; a recursive-descent parser turns `find` command lines into the same criteria, with `!` > AND > OR precedence and GNU's rounded-up `-size`.

Chapter summary

Problem	Core entities	Key pattern(s)	Concurrency point	Trap to avoid
Task Management	Task, TaskStatus, TaskService	Specification filters, enum state machine, Observer	per-task `synchronized`	TODO → DONE jumps; filters as many parameters
Digital Wallet	Wallet, Transaction, Offer	Strategy offers, idempotency, funded cashback	lock both wallets in id order	`double` money; check-then-act on keys; cashback created from nothing
Event Calendar	TimeSlot, Event, EventStore	Strategy for representatives, plan-then-save	atomic check-and-book	closed intervals; trusting the sample blindly
Coding Blox	Contest, Level, User	Command map, Strategy (solver, scoring)	atomic rating update	unseeded randomness; running a contest twice
PropertyHunt	Property, SearchCriteria	Builder + predicate list, Strategy sort	synchronized mark-sold	comparing "90L" strings; stale shortlist copies
Ledger Co	Loan, LumpSum, Balance	Strategy interest, derived state	synchronized lump sums	storing a running balance; floor instead of ceil
Inventory	StockMovement, AllocationStrategy	Strategy allocation, Observer alerts	per-SKU lock	partial shipments; alert spam
Unix find	Node (Composite), FileFilter, SearchCriteria	Specification + Composite filters, Builder, recursive-descent parser	weakly consistent children map; read-only parallel walk	parameter-list search methods; `-o` vs AND precedence

The pattern behind all eight: entities guard their own rules, services orchestrate, strategies hold the "X or Y" rules, repositories hide storage, the driver parses and prints, and every error is a clear message — not a crash.

Chapter 22 — Revision: Cheat Sheets & Glossary

What you will learn in this chapter

- How to revise the whole handbook in 15 minutes, or in one evening the night before.
- The LLD (low-level design: the classes and code inside one program) answer framework on one page (Chapter 1).
- OOP, relationships and UML arrows on one page (Chapters 2–3).
- SOLID, the other design principles, and the clean-code rules as tables (Chapters 4–6).
- All 24 code smells with the refactorings that cure them (Chapter 7).
- All 23 GoF patterns (the classic catalogue by the "Gang of Four" authors) plus the extras, and a "which pattern?" table sorted by symptom (Chapters 8–11).
- The concurrency toolbox, a deadlock checklist and a "which `java.util.concurrent` class?" table (`java.util.concurrent` is Java's standard package of thread-safety tools) (Chapters 12–13).
- A map of every case study: entities, patterns, concurrency concern, and where to find it (Chapters 14–21).
- A machine-coding checklist, an edge-case checklist, and a glossary of about 200 terms.

Nothing in this chapter is new. Every row is a summary of a section in Chapters 1–21, and the right-hand column tells you where to go for the full explanation. This chapter has no code.

How to read the references. "§12.17" means section 12.17. Chapters 5 and 6 number their sections from 1 inside the chapter, so "Ch 5 §13" means section 13 of Chapter 5 (Fail Fast). Chapters 16, 17 and 19 have unnumbered case studies, so they are referenced by name, for example "Ch 16 · Car Rental".

New to a term? Every technical word in the tables below, from `API` and `JVM` to `TTL` and `SKU`, has a one-line definition in the glossary (22.11). A **TL;DR** ("too long; didn't read") heading marks a short summary.

22.1 How to revise

In 15 minutes (the morning of the interview)

Minutes	Read	Why
0–3	22.2 (the framework)	You will use it in the first minute of the interview
3–6	22.4 (SOLID one-pager) and the relationship table in 22.3	The two things asked about most often by name
6–10	"Which pattern?" table in 22.7	Patterns should come from a signal, not from memory
10–13	Concurrency cheat sheet and deadlock checklist in 22.8	Every booking/inventory problem has a race
13–15	The row(s) of 22.9 for the problems you expect	One sentence per problem: entities, pattern, the atomic step

The night before (about 2 hours)

1. **Read 22.2–22.8 slowly** (45 min). For each table row, say out loud *one example from the handbook*. If you cannot, open the referenced section.
2. **Pick three case studies** from 22.9 you have not practised recently (45 min). For each: write requirements, list classes, draw the class diagram, and name the one hard method, without looking. Then compare with the chapter's TL;DR.
3. **Scan the glossary** (22.11) and mark any term you cannot explain in one sentence (20 min). Read those sections only.
4. **Stop.** Sleep matters more than one more pattern.

Common mistakes when revising

- Re-reading chapters instead of **recalling** first and then checking. Recall is what the interview tests.
- Memorising all 23 patterns equally. Six come up in almost every round (22.7): Strategy, Factory, Observer, State, Singleton, Builder.
- Skipping the "when *not* to use it" column. Interviewers ask "would you always do that?" (§4.13).

22.2 The LLD interview answer framework (one page)

Source: Chapter 1 (§1.2, §1.4–§1.10, §1.12, §1.14).

TL;DR

Clarify → requirements → use cases → entities → relationships → class diagram → key flows → patterns/SOLID → code → edge cases/concurrency → extensions. Say each step out loud as you enter it.

The eleven steps

#	Step	Ask yourself	Output
1	Clarify	Who uses it? Single site or many?	Shared understanding
2	Requirements	Functional? Non-functional (thread-safe, extensible, in-memory)? **Out of scope?**	8–12 numbered lines + out-of-scope list
3	Use cases	What can each actor do?	Actor → use-case list
4	Entities	Which nouns hold state or behaviour? Which are just values?	Classes, one sentence each
5	Relationships	Uses / knows / has / owns / is-a? How many?	Typed lines with multiplicity
6	Class diagram	Fields, key methods, interfaces, enums	UML class diagram
7	Key flows	Steps of the 2–3 main use cases; lifecycle of the core object	Sequence / activity / state diagram
8	Patterns & SOLID	A rule that varies? A lifecycle? Many listeners? Two reasons to change?	1–2 named patterns + one rejected
9	Code	Interfaces, enums, models, services, driver	Running code for the main flows
10	Edge cases & concurrency	Bad input? Full? Two users, one seat?	Validation, exceptions, locks
11	Extensions	"What if X is added?"	Short answers, maybe one class

Time budgets

45-minute OOD (object-oriented design) round		90-minute machine-coding round (write runnable code)	
Clarify requirements	5–8 min	Read, requirements, assumptions	5–10 min
Actors and use cases	3–5 min	Model: entities, enums, service interfaces	10–15 min
Find the classes	8–10 min	Core flow end to end	30–40 min
Connect the classes	8–10 min	Remaining features, validation, edge cases	15–20 min
Write the code	12–15 min	Polish, one extension point, demo	5–10 min
		Buffer	5 min (never zero)

Rule of thumb: roughly **a quarter of the hour goes before you name a class**, and in machine coding you **get something running by the halfway mark** (§1.2).

Finding classes (§1.6)

- **Nouns → candidate classes; verbs → methods.** A verb with no class to live on means a class is missing.
- Cut candidates with five tests: is it a **value**? the **same thing** under another name? does it have **changing state**? does anything **call** it? does it differ in **behaviour or only data**?
- **Different data → field or enum. Different behaviour → class.**
- One sentence per class, **no "and"**. Put each method where its data is (Information Expert).
- Beginners most often forget the **record-keeping class** (ticket, loan, booking) and the **rule class** (rate, strategy).

What the interviewer judges (§1.2)

1. Did you **bound** the problem? 2. Does each class have **one job** you can say in a sentence? 3. Are the **relationships** right? 4. Does the **code match the diagram**? 5. Can you explain a **trade-off** (the choice and the rejected option)?

Machine coding adds: does it **run**, is it **modular**, **readable**, does it **handle bad input**, and (senior roles) is shared state **thread-safe**.

Code habits interviewers notice (§1.10)

Habit	Instead of
Pass time in (`long nowMinute`, `Instant now`, injected `Clock`)	Reading the clock inside logic
Money as `BigDecimal` from strings, or whole cents in a `long`	`double` (0.1 + 0.2 ≠ 0.3)
`IllegalArgumentException` / `IllegalStateException` / a custom exception	Silent failure or error codes
`Optional` when "not found" is normal	Returning `null`
`private final` fields	Public mutable fields
Domain names (`ParkingTicket`)	`Manager`, `Helper`, `Data2`

Skeleton: **Models → Repositories → Strategies → Services → Driver** (§1.10). The driver is the `main` class that feeds sample commands to the services and prints the results.

In the interview

> **30-second answer** (from §1.14): "I clarify and write 8 to 12 requirements with an out-of-scope list, list actors and use cases, pick classes from the nouns and give each one responsibility, connect them and draw a class diagram, sketch the key flow, run a quick SOLID check and pick one or two patterns where the signal is clear, then code the two or three core flows, handle edge cases and concurrency, and finish with extensions and trade-offs."

Common mistakes (§1.14)

Mistake	Fix
Coding in minute two	8–13 minutes on requirements and actors first
No out-of-scope list	Write it down and agree it
One subclass per value (`CompactSpot`, `LargeSpot`)	Different data → field/enum
God class (`ParkingLotManager`)	One sentence per class; split on "and"
Inheritance for reuse ("Car extends Engine")	Composition
Pattern stuffing	One or two patterns, each with its signal
All twelve classes at equal depth	Two or three flows fully
Ignoring two users on one resource	Lock or atomic operation on the shared state
Non-running code in machine coding	Thin end-to-end path running by halfway

---

22.3 OOP & relationships cheat sheet

Source: Chapter 2 (OOP) and Chapter 3 (relationships and UML).

The four pillars (§2.3–§2.6)

Pillar	One line	Java tools	Smell when missing
Encapsulation (§2.3)	Data is private; only the owning class's methods change it, and they enforce the rules (the *invariants*: facts that must always hold)	`private` fields, no setters, validation in constructors	Public fields; setters that break invariants
Abstraction (§2.4)	Expose only what callers need; hide how	Interfaces, abstract classes, small public API	Callers reach into internals
Inheritance (§2.5)	Child reuses and specialises parent — **only when "child is a parent" is truly correct**	`extends`, `super`, `@Override`	Subclass refuses or throws on parent methods
Polymorphism (§2.6)	One call, different behaviour per object; removes type switches	Overriding + dynamic dispatch (the method is picked from the object's real class at run time)	`switch` / `instanceof` chains on type

Encapsulation vs abstraction: hiding *data* behind rule-enforcing methods vs hiding *complexity* behind a minimal set of operations (Ch 2 self-check).

Other OOP facts worth one line each

Topic	Remember	Where
Overloading vs overriding	Compile time, by declared parameter types vs run time, by the actual object. Fields and `static` methods are not polymorphic (statics are hidden, not overridden)	§2.7
`this`, `static`, `final`	Current object; one per class; cannot be reassigned/overridden/extended	§2.2
Interfaces	Contracts; can have `default`, `static`, `private` methods; a functional interface has one abstract method (lambdas: short inline functions such as `x -> x * 2`)	§2.8
Enums	Fixed sets with fields and behaviour ("which spot types fit a truck" lives on `VehicleType`)	§2.10
Composition over inheritance	Avoids fragile base classes; allows run-time swapping	§2.11, Ch 5 §6
`equals` / `hashCode`	Equal objects must have equal hash codes; override both or hash collections break	§2.12
Immutability & records	Final class, private final fields, no setters, defensive copies. Records are only **shallowly** immutable — copy collections in the compact constructor	§2.13
Access modifiers	Default to `private`; package-private = same package only	§2.14
Generics	Type-safe reusable infrastructure; **PECS**: producer `extends`, consumer `super`	§2.15
Coupling & cohesion	Low coupling (between classes), high cohesion (inside a class)	§2.16
OOA → OOD → OOP	What the problem is → how, with objects → code. Connections that carry data become objects	§2.17

Abstract class vs interface (§2.9)

	Abstract class	Interface
Instantiable?	No	No
Instance fields (state)	Yes	No (only `public static final` constants)
Constructors	Yes	No
Method bodies	Any	`default`, `static`, `private` only
How many per class	**One** (`extends`)	**Many** (`implements`)
Relationship expressed	"is a" (shared identity and state)	"can do" (capability / role)
Typical use	Shared state + partial implementation for a close family (Template Method)	Contracts, roles, strategies, test seams

Rule of thumb: start with an interface; add an abstract class underneath when implementations share real state or a fixed algorithm (JDK, the Java Development Kit's standard library: `List` + `AbstractList` + `ArrayList`).

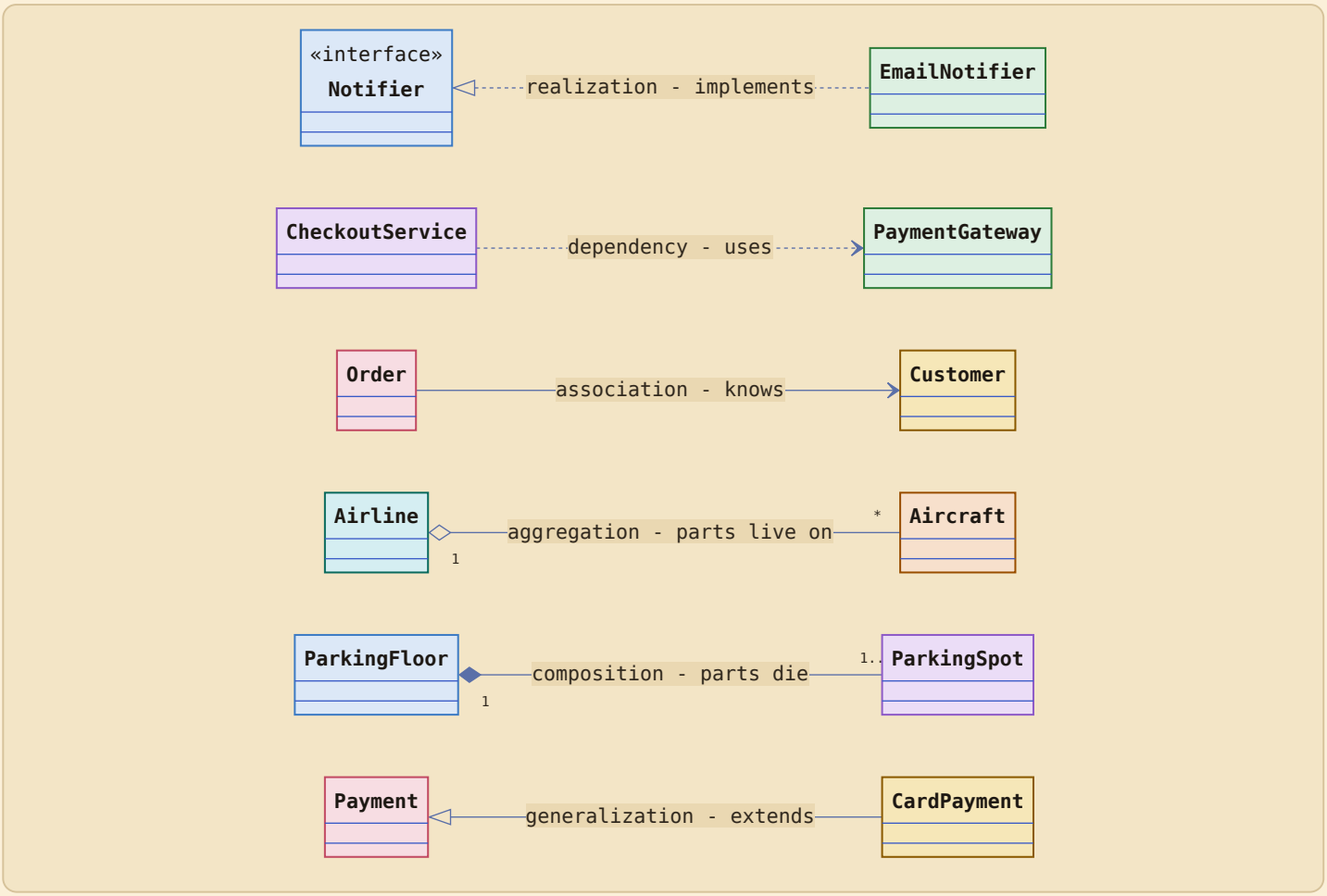
The six relationships (§3.2–§3.8)

Weakest to strongest coupling: **dependency, association, aggregation, composition, realization, generalization** (§3.16).

Relationship	Phrase	UML line	Java shape	Lifetime	Example
Dependency (§3.5)	A *uses* B	dashed, open arrow → B	B as parameter / local / return / static call	none	`CheckoutService` uses `PaymentGateway`
Association (§3.2)	A *knows* B	solid line (arrow for one-way)	field of type B, created elsewhere	independent	`Order` → `Customer`
Aggregation (§3.3)	A *has* B, parts live on	solid, hollow ◇ at A (the whole)	field/collection, parts passed in	B outlives A	`Airline` ◇— `Aircraft`
Composition (§3.4)	A *owns* B	solid, filled ◆ at A (the whole)	A creates B, hides it, B not shared	B dies with A	`ParkingFloor` ◆— `ParkingSpot`
Realization (§3.7)	A *implements* B	dashed, hollow △ at B	`implements`	—	`EmailNotifier` ⇢ `Notifier`
Generalization (§3.6)	A *is a* B	solid, hollow △ at B	`extends`	—	`CardPayment` → `Payment`

Choosing, in order (§3.8): interface A promises to fulfil → realization. Substitutable with no refused methods → generalization. Not kept in a field → dependency. Part that dies with the whole → composition. Part that outlives it → aggregation. Otherwise → association.

UML arrow legend (Mermaid)



Mermaid	Meaning
`A ..> B`	A depends on B
`A --> B`	A associated with B, navigable A to B
`A -- B`	association (both ways / unspecified)
`A o-- B`	aggregation, A is the whole
`A *-- B`	composition, A is the whole
`B <|.. A`	A realizes interface B
`B <|-- A`	A extends B
`A "1" *-- "1..*" B`	with multiplicities

Multiplicity → code (§3.17): `1` required field; `0..1` `Optional`/nullable; `1..*` non-empty collection checked in the constructor; exact numbers checked in constructors. Visibility: `+` public, `-` private, `#` protected, `~` package.

UML diagrams in the interview (§3.9–§3.15)

Diagram	Use it for	Budget in 45 min
Use case (§3.10)	Scope: actors + what each does. «include» = always; «extend» = sometimes	Usually spoken, 3–5 min
Class (§3.11)	**The answer.** 10–15 boxes, every line typed, with multiplicity	Always drawn, 10–15 min
Sequence (§3.12)	Call order between objects (3–4 objects, 6–8 messages)	One behaviour diagram only
Activity (§3.13)	A flow that branches; fork/join for parallel steps	"
State machine (§3.14)	One object's lifecycle; transition label `event [guard] / action`	"

UML has 14 diagram types: 7 structural, 7 behavioural (§3.9).

Common mistakes

- Diamond at the wrong end (it goes at the **whole**); triangle at the wrong end (it goes at the **parent/interface**) (§3.8).
- Lines without type or multiplicity (§3.8).
- Inheritance used to share data; subclasses that only pass a constant to `super` → one class with an enum field (§3.16).

22.4 SOLID one-pager

Source: Chapter 4 (§4.2, §4.7–§4.10, §4.12–§4.15), smells from Chapter 7 (§7.4).

TL;DR

SOLID = five names for the ways code becomes painful to change. **Use them to diagnose, not as a checklist**, apply a fix when a real change asks for it, and always say what the fix costs.

Principle	One-line rule	Smell that signals a violation	Fix	Patterns that help
S — Single Responsibility (§4.2)	A class should have one reason to change (one actor asks for its changes)	The responsibility sentence needs an "and"; *Divergent Change*, *Large Class*	Split by actor (Extract Class); each new class needs a name that stands alone	Command, Proxy, Decorator (each concern in its own class)
O — Open/Closed (§4.7)	Add new behaviour by adding code, not by editing working code	A `switch` / `if` chain over a type; *Repeated Switches*, *Shotgun Surgery*	Put the part that *has* varied behind an interface; add variants as classes (Replace Conditional with Polymorphism)	Strategy, Template Method, Decorator, Factory Method, Observer, Command, Visitor
L — Liskov Substitution (§4.8)	A subtype must be usable anywhere its parent is expected	Subclass throws, does nothing, or refuses a parent method; *Refused Bequest*	Better parent, a narrower interface, or composition (Replace Subclass with Delegate)	Composite, Null Object
I — Interface Segregation (§4.9)	No client should depend on methods it does not use	Empty method bodies or `UnsupportedOperationException` in implementers; fat interfaces	Split into **role interfaces**, one per kind of caller	Adapter
D — Dependency Inversion (§4.10)	Depend on abstractions, not concrete details; the policy owns the abstraction	A high-level class does `new` of (or names) a low-level one; cannot test without the real DB (database)/provider	Inject an interface owned by the high-level class; `main` wires the details	Strategy, Factory Method / Abstract Factory, Adapter, Observer

LSP in detail (§4.8, TL;DR): a subclass must accept at least as much, promise at least as much, keep invariants and history, and throw nothing new. Square/Rectangle is worse than `Penguin.fly()` throwing because it fails **silently** (wrong area, no error).

Five-minute pass over your diagram (§4.13, §1.8)

Run S → O → L → I → D over the diagram you just drew. **Two findings is a good pass; five means rework; zero means you did not look.** S and O earn the most credit: O is what interviewers probe with a late requirement ("what if trucks are charged differently?" → "add a class").

Related ideas (§4.3–§4.6, §4.14)

Idea	Question it asks
SRP	How many actors can ask this class to change?
Cohesion	Do the parts of this class belong together?
Coupling	How much does A depend on B?
Separation of Concerns	Is each area of work in its own module?
SRP vs ISP	SRP splits **classes** (guards the worker); ISP splits **interfaces** (guards the caller)
DIP vs DI vs IoC	DIP = principle (depend on abstractions); DI = technique (pass dependencies in); IoC = the broader idea that a framework calls your code; IoC containers (libraries such as Spring that create and wire objects for you) perform DI

Common mistakes: over-engineering (§4.15)

Warning sign	Principle misapplied	Do instead
`XHelper`, `XUtils`, `XManager2` after a split	SRP	Split by actor; names must stand alone
Six small classes that always change in the same commit	SRP over-split	Merge them back
Interface + factory + registry for one implementation	OCP (*Speculative Generality*)	Write the `if`; refactor at the second or third case (rule of three)
A subclass that "works" by silently doing nothing	LSP	Better parent or composition
One interface per method	ISP over-split	Group by caller into role interfaces
An interface in front of stable `String`-like classes	DIP	Invert only volatile dependencies
A DI container for a 200-line program	DIP	Wire by hand in `main`

YAGNI is the counterweight: SOLID says *what* a good seam looks like; YAGNI says *when* to cut it.

In the interview

Say **observation → cost → fix → price of the fix**, and describe before you label: "`Reporting` implements nine methods but calls two — an interface segregation problem" (§4.13).

22.5 Other design principles and Clean Code rules

Other principles (Chapter 5)

Principle	One-line idea	Smell it fixes	Key Java tool	Where
DRY	Each piece of knowledge has one authoritative home	Duplicated rules drifting apart	Value objects, constants, methods	Ch 5 §1
KISS	Choose the simplest design that works	Over-engineering	Plain code, `switch` expressions	Ch 5 §2
YAGNI	Don't build features before they are needed	Speculative Generality	Delete unused code	Ch 5 §3
Law of Demeter	Only talk to your immediate friends	Train wrecks `a.getB().getC().do()`	Intent-revealing delegate methods	Ch 5 §4
Tell, Don't Ask	Tell objects what to do; don't pull their data out and decide for them	Anemic model, Feature Envy	Command methods on the owner	Ch 5 §5
Composition over inheritance	Prefer "has-a" to "is-a" for reuse	Fragile base class	Forwarding / decorators	Ch 5 §6
Program to an interface	Depend on supertypes, choose the concrete class once at the edge	Hard-wired concrete dependencies	Interfaces + DI	Ch 5 §7
Encapsulate what varies	Isolate the part that changes behind an interface	Growing type switches	Strategy, lambdas	Ch 5 §8
Loose coupling	Interacting objects know little about each other	God orchestrators	Observer, events, DI	Ch 5 §9
Hollywood Principle / IoC	"Don't call us, we'll call you" — the framework owns the flow	Duplicated skeletons	Template Method, callbacks, DI	Ch 5 §10
Separation of Concerns	Each part handles one concern	Spaghetti code	Layers, MVC	Ch 5 §11, §4.5
Least Astonishment	Do what the name promises and nothing more	Hidden side effects	Honest names, `Optional`	Ch 5 §12
Fail fast	Detect problems early and loudly	Distant `NullPointerException`s	Guard clauses, `requireNonNull`	Ch 5 §13
Immutability	Objects never change after creation	Aliasing, races	Records, `List.copyOf`	Ch 5 §14
Design by Contract	Preconditions (caller), postconditions (method), invariants (always)	Magic error values	Checks at method entry/exit	Ch 5 §15
GRASP	Nine guidelines for "which class should do this?"	God classes	Expert, Creator, Controller…	Ch 5 §16
Domain-Driven Design	Model the business: bounded contexts, entities vs value objects, small aggregates, domain events, repositories per root	Anemic model, big ball of mud, primitive obsession	Records as value objects, aggregate roots, events + outbox (events saved in the same transaction as the data, published afterwards)	Ch 5 §17

GRASP in one line each (Ch 5 §16.1–§16.9):

Guideline	Rule
Information Expert	The class with the data does the work
Creator	The container (or the one with the initial data) creates
Controller	A thin non-UI object receives each system event and delegates
Low Coupling	Choose the design with fewer, more stable dependencies
High Cohesion	Each class does one related set of things
Polymorphism	Let each type implement the varying behaviour; don't switch on types
Pure Fabrication	Invent a helper class (e.g. a repository) when no domain class is a clean home
Indirection	Put a middleman between things that shouldn't know each other
Protected Variations	Wrap what's unstable behind a stable interface

Tensions to mention (Ch 5 summary): DRY vs loose coupling (sharing code couples its users); YAGNI vs encapsulate-what-varies (abstract only real or clearly predicted variation); Law of Demeter vs cohesion (too many delegates → Middle Man); Information Expert vs SoC (domain objects compute, fabrications persist).

Clean Code rules checklist (Chapter 6)

Area	Rule of thumb	Where
Names	Reveal intent; pronounceable; searchable; one word per concept; code reads like the requirement	Ch 6 §1
Functions	Small; do one thing; one level of abstraction; top-down (stepdown); ≤ 2–3 arguments; **no flag arguments**; no hidden side effects; commands *or* queries (CQS); exceptions, not error codes	Ch 6 §2
Comments	Say it in code; comment only the *why*, warnings, legal text, public contracts; delete commented-out code	Ch 6 §3
Formatting	Newspaper order (headline first); related things close; one automated team style	Ch 6 §4
Objects vs data structures	Objects hide data and expose behaviour; data structures expose data; no hybrids; no train wrecks	Ch 6 §5
Error handling	Unchecked exceptions with context; wrap vendor exceptions; empty collections / `Optional` / special case instead of `null`; never pass `null`; always clean up	Ch 6 §6
Boundaries	Keep third-party code behind your own interfaces; learn it with tests	Ch 6 §7
Unit tests	Three laws of TDD (Test-Driven Development: write a failing test first); one concept per test; Given–When–Then; **F.I.R.S.T.** (Fast, Independent, Repeatable, Self-validating, Timely)	Ch 6 §8
Classes	Small by responsibility; cohesive; open for extension; depend on abstractions	Ch 6 §9
Systems	Build objects in one place (composition root) and inject them; cross-cutting concerns in proxies/aspects	Ch 6 §10
Simple design	Beck: passes the tests → reveals intention → no duplication → fewest elements (*Clean Code* swaps the middle two)	Ch 6 §11
Concurrency	Separate concurrency code; share as little as possible; use the library; keep locks small; plan shutdown	Ch 6 §12
Smells	The C/E/F/G/J/N/T heuristics catalogue as a review vocabulary	Ch 6 §14
Code review	Correctness → design → readability → tests → security → performance → operability; automate style; small PRs (pull requests: proposed changes sent for review)	Ch 6 §15
Error strategy	Classify, translate at boundaries, retry only transient + idempotent (safe to repeat) with backoff (waiting longer between attempts), one global handler that logs once	Ch 6 §16
Testing strategy	Many unit, some integration, few E2E (end-to-end, whole-system); right test double (a stand-in such as a mock or fake); verify state over interactions; don't mock what you don't own	Ch 6 §17

Boy Scout Rule: leave the code a little cleaner than you found it (Ch 6 intro).

22.6 Code smells → refactorings (all 24)

Source: Chapter 7 (§7.2 smells, §7.3 catalogue, §7.4 lookup table). ***Bold*** *= the usual first move.*

#	Smell	Signal (one line)	Group	Refactorings that cure it
1	Mysterious Name (§7.2.1)	Name doesn't say what it does/holds (`data`, `proc()`, `Manager`)	2nd-ed extra	**Change Function Declaration**, Rename Variable, Rename Field
2	Duplicated Code (§7.2.2)	Same structure in more than one place	Dispensables	**Extract Function**, Slide Statements, Pull Up Method
3	Long Function (§7.2.3)	You want a comment to explain a block	Bloaters	**Extract Function**, Replace Temp with Query, Introduce Parameter Object, Preserve Whole Object, Replace Function with Command, Decompose Conditional, Replace Conditional with Polymorphism, Split Loop
4	Long Parameter List (§7.2.4)	Many parameters, same types, boolean flags	Bloaters	Replace Parameter with Query, **Preserve Whole Object**, **Introduce Parameter Object**, Remove Flag Argument, Combine Functions into Class
5	Global Data (§7.2.5)	Readable and writable from anywhere (incl. mutable singletons)	2nd-ed extra	**Encapsulate Variable** (then shrink its scope)
6	Mutable Data (§7.2.6)	Shared data changed after creation	2nd-ed extra	**Encapsulate Variable**, Split Variable, Slide Statements, Extract Function, Separate Query from Modifier, Remove Setting Method, Replace Derived Variable with Query, Combine Functions into Class/Transform, Change Reference to Value
7	Divergent Change (§7.2.7)	One class changed for many different reasons (SRP)	Change Preventers	**Split Phase**, Move Function, Extract Function, **Extract Class**
8	Shotgun Surgery (§7.2.8)	One change forces many small edits in many classes	Change Preventers	**Move Function**, **Move Field**, Combine Functions into Class/Transform, Split Phase, Inline Function, Inline Class
9	Feature Envy (§7.2.9)	Method uses another class's data more than its own	Couplers	**Move Function**, Extract Function
10	Data Clumps (§7.2.10)	Same group of values always travel together (`startDate, endDate`)	Bloaters	**Extract Class**, **Introduce Parameter Object**, Preserve Whole Object
11	Primitive Obsession (§7.2.11)	Money as `double`, status as `String`	Bloaters	**Replace Primitive with Object**, Replace Type Code with Subclasses, Replace Conditional with Polymorphism, Extract Class, Introduce Parameter Object
12	Repeated Switches (§7.2.12)	The *same* switch on a type code in several places	OO Abusers	**Replace Conditional with Polymorphism**
13	Loops (§7.2.13)	Classic loop that filters/maps/aggregates	2nd-ed extra	**Replace Loop with Pipeline**
14	Lazy Element (§7.2.14)	A class/function that doesn't pull its weight	Dispensables	**Inline Function**, **Inline Class**, Collapse Hierarchy

#	Smell	Signal (one line)	Group	Refactorings that cure it
15	Speculative Generality (§7.2.15)	Hooks "we might need someday"; abstract class with one subclass	Dispensables	Collapse Hierarchy, Inline Function, Inline Class, Change Function Declaration, Remove Dead Code
16	Temporary Field (§7.2.16)	Field set only in some situations, `null` otherwise	OO Abusers	**Extract Class**, Move Function, Introduce Special Case
17	Message Chains (§7.2.17)	`order.getCustomer().getAddress().getCity()`	Couplers	**Hide Delegate**, Extract Function, Move Function
18	Middle Man (§7.2.18)	Most methods just forward to another class	Couplers	**Remove Middle Man**, Inline Function, Replace Superclass/Subclass with Delegate
19	Insider Trading (§7.2.19)	Two modules poke at each other's internals	Couplers	**Move Function**, **Move Field**, Hide Delegate, Replace Subclass/Superclass with Delegate
20	Large Class (§7.2.20)	Too many fields/methods — a God Class	Bloaters	**Extract Class**, Extract Superclass, Replace Type Code with Subclasses
21	Alternative Classes with Different Interfaces (§7.2.21)	Same job, different method names (`sendMail` vs `dispatch`)	OO Abusers	**Change Function Declaration**, Move Function, Extract Superclass
22	Data Class (§7.2.22)	Only fields and getters/setters; others manipulate it	Dispensables	**Encapsulate Record**, Remove Setting Method, Move Function, Extract Function, Split Phase
23	Refused Bequest (§7.2.23)	Subclass doesn't want inherited methods, or throws on them (LSP)	OO Abusers	Push Down Method/Field, **Replace Subclass with Delegate**, **Replace Superclass with Delegate**
24	Comments (§7.2.24)	Comment used as deodorant for unclear code	Dispensables	**Extract Function**, Change Function Declaration, Introduce Assertion

Smell family → principle violated (§7.4)

Smell family	Principle
Divergent Change, Large Class	SRP
Repeated Switches, Shotgun Surgery	OCP
Refused Bequest	LSP
Long Parameter List with flags, fat interfaces	ISP / clean function design
Global Data, Mutable Data	Encapsulation
Hard-wired `new` of concrete dependencies	DIP
Duplicated Code	DRY
Speculative Generality	YAGNI
Message Chains, Insider Trading	Law of Demeter
Feature Envy, Data Class	Tell, Don't Ask

Refactoring toward patterns (§7.6)

Smell	Refactor toward
Switch/if-chain choosing an algorithm	Strategy
Methods that check and change a status field	State
`if (x == null)` at many call sites	Null Object / Special Case
Sibling subclasses with the same skeleton	Template Method
Many confusing constructors	Factory Method / creation methods
Complex object built by many ordered calls	Builder
Undo/queueing needed; huge function with shared locals	Command
Logging/caching mixed into a core class	Decorator
Objects polling for changes; hard-coded notification	Observer
Two classes, same job, different interfaces	Adapter
Subclass explosion from feature combinations	Strategy / Decorator / Bridge
A pattern that no longer earns its keep	Remove it (Inline Class, Collapse Hierarchy)

Refactoring = changing structure without changing behaviour, in small steps, with tests as the safety net. Wear one of **two hats** at a time: adding function or refactoring (§7.1.1).

22.7 Design patterns: all 23 GoF + extras

Source: Chapters 8–11. Catalogue in §8.3; comparisons in §8.15, §9.8, §10.7, §11.6, §11.8.

Count: 5 creational + 7 structural + 11 behavioral = **23**. **Class-scoped** (inheritance) patterns: Factory Method, class Adapter, Template Method, Interpreter; almost everything else is object-scoped (composition) (§8.3).

Creational (Chapter 8)

Pattern	Intent (one line)	Use when	Java / JDK example	Commonly confused with	Where
Singleton	Exactly one instance with a global access point	There really must be exactly one (config, pool, registry)	`Runtime.getRuntime()`; Spring (a popular Java framework) singleton-scoped beans	A global variable; a DI singleton scope (preferred for testing)	§8.8
Factory Method	Subclasses decide which concrete class to create via an overridable method	Shared procedure, varying product; others will add types	`Collection.iterator()`	Simple Factory (one method with a switch)	§8.10
Abstract Factory	Create *families* of related objects without naming concrete classes	Products must match; swap a whole set at once	`DocumentBuilderFactory`; JDBC (Java's database API) `Connection`	Factory Method (one product by inheritance vs a family by composition)	§8.11
Builder	Build a complex object step by step; one validating `build()`	Many optional fields, cross-field validation, immutable result	`StringBuilder`, `HttpClient.newBuilder()`	A plain constructor (fine for a handful of required fields); telescoping constructors	§8.12
Prototype	Create new objects by copying a configured instance	Setup is expensive, or only the interface is known	Copy constructors, `ArrayList.clone()`	Memento (a snapshot to restore, not a copy to use)	§8.13

Singleton notes (§8.8, §8.17): eager, holder-class or **`enum`** (most robust: blocks duplicates made through reflection, i.e. code calling private constructors by name, or through serialization, i.e. saving to bytes and reading back); double-checked locking needs a **`volatile`** field. It is global state that hurts testing — prefer injecting one shared instance. Many case studies reject it (Ch 14 self-check).

Structural (Chapter 9)

Pattern	Intent (one line)	Use when	Java / JDK example	Commonly confused with	Where
Adapter	Convert one interface into another that clients expect	"The vendor API doesn't match ours" (after the fact)	`Arrays.asList`, `InputStreamReader`	Bridge (same shape, but Bridge is designed up front); Facade	§9.1
Bridge	Split abstraction from implementation so both vary independently	Class names with two nouns; two dimensions multiplying classes	JDBC `Connection` + drivers	Adapter; Strategy (same shape)	§9.2
Composite	Treat single objects and trees uniformly	Part-whole trees (files, menus, UI)	`java.awt.Container` is a `Component` holding `Component`s	Decorator (also wraps a component, but adds behaviour to one)	§9.3
Decorator	Add responsibilities dynamically by wrapping; same interface, stackable	"Sometimes logging, sometimes caching, sometimes both"	`BufferedInputStream`, `Collections.synchronizedList`	Proxy (same shape; Proxy *controls access*); Chain of Responsibility (in CoR someone can stop the flow)	§9.4
Facade	One simple entry point to a complex subsystem	Every caller repeats the same multi-step sequence	SLF4J (a logging facade library) `LoggerFactory`, service layers	Adapter (converts one object's interface); Mediator	§9.5
Flyweight	Share intrinsic (immutable) state across huge numbers of objects	Millions of near-identical objects	`Integer.valueOf`, `String.intern()`	Object Pool (reuses objects one borrower at a time; flyweights are shared at once)	§9.6
Proxy	A stand-in that controls access (lazy, remote, protective, caching)	It's expensive, remote or restricted	`java.lang.reflect.Proxy`, Spring `@Transactional`, Hibernate (maps objects to database tables) lazy load	Decorator	§9.7

Memory hooks (§9.8): Adapter *make it fit* · Decorator *add to it* · Proxy *guard it* · Facade *simplify it* · Bridge *split it* · Composite *nest it* · Flyweight *share it*.

Behavioral (Chapters 10–11)

Pattern	Intent (one line)	Use when	Java / JDK example	Commonly confused with	Where
Chain of Responsibility	Pass a request along handlers until one handles (or stops) it	Several checks, any may reject; handler not known in advance	Servlet `Filter` chain; `java.util.logging` handlers	Decorator (every layer always runs)	§10.1
Command	Turn a request into an object: queue, log, retry, undo	Undo/redo, job queues, scheduling	`Runnable` / `Callable`; Swing `UndoManager`	Strategy (Strategy = *how*; Command = *what*)	§10.2
Iterator	Traverse a collection without exposing how it is stored	Walk a collection/tree/paged API	`Iterator`, `Spliterator`, JDBC `ResultSet`	Visitor (what to do at each node)	§10.3
Mediator	Centralise communication between peers in one coordinator	Many objects interact in tangled ways; order of reactions matters	Chat servers, UI controllers; `Executor` / `Timer` are a loose fit (§10.4)	Observer (broadcast, no coordination); Facade	§10.4
Memento	Capture and restore state without breaking encapsulation	Undo, checkpoints, rollback	A `record` snapshot; DB `ROLLBACK`, Git commits (real-world)	Command undo (store actions, not states); Prototype	§10.5
Interpreter	One class per grammar rule; evaluate sentences of a small language	User-written rules/expressions	`java.util.regex.Pattern`, SpEL (Spring's expression language)	Composite (same tree); Visitor (many operations over the AST, the abstract syntax tree a parser builds)	§10.6
Observer	Notify many dependents automatically when one object changes	An open-ended set of independent listeners	`PropertyChangeListener`, `java.util.concurrent.Flow`, Spring `@EventListener`	Mediator; pub-sub (Observer through a broker, §20.6)	§11.1
State	Behaviour changes with internal state, as if the class changed	Many states/transitions; methods repeat a `switch` on status	TCP connection (GoF's example), order lifecycles. `Thread.State` is only an enum of states, not the pattern	Strategy (same diagram; states know each other and advance themselves)	§11.2
Strategy	Make a family of algorithms interchangeable behind one interface	Several ways to do one job, chosen at run time	`Comparator`; `RejectedExecutionHandler` policies	State; Template Method; Decorator (Strategy *replaces* a rule, Decorator *wraps* one)	§11.3
Template Method	Fixed algorithm skeleton in a	Long fixed flow, a few varying steps	`AbstractList`, `InputStream.read(byte[],…)`,	Strategy (Template =	§11.4

Pattern	Intent (one line)	Use when	Java / JDK example	Commonly confused with	Where
	base class; subclasses fill in steps		`HttpServlet.service()`	inheritance, one step varies; Strategy = composition, whole algorithm varies)	
Visitor	Add operations to a stable set of classes without changing them (double dispatch)	Many new operations over a fixed type set (ASTs)	`javax.lang.model.element.ElementVisitor`; `FileVisitor` is named after it but is really a tree-walk callback	Iterator; in Java 21, **sealed types + pattern-matching** `switch` usually replace it	§11.5

Beyond GoF: extras you will meet

Pattern	Intent (one line)	Use when	Java / real example	Commonly confused with	Where
Simple Factory (idiom)	One `create(type)` method maps a type value to a class	Small, stable set of types chosen by config	`Calendar.getInstance()`, `EnumSet.noneOf`	Factory Method	§8.9
Object Pool	Borrow and return a bounded set of expensive objects	DB connections, limited resources	HikariCP (a database connection pool), `ThreadPoolExecutor`	Flyweight	§8.14
Null Object	A do-nothing implementation instead of `null`	`null` checks everywhere for an optional collaborator	`Collections.emptyList()`, `OutputStream.nullOutputStream()`	`Optional` (for lookups)	§11.7.1
Dependency Injection	Receive collaborators (as interfaces) through the constructor	Classes create their own collaborators; hard to test	Spring constructor injection; a hand-wired `main`	Service Locator (anti-pattern: the class asks for its dependencies)	§11.7.2
Repository	Collection-like interface over storage	Business logic mixed with storage code	Spring Data `CrudRepository`	DAO (table-oriented)	§11.7.3
Specification	A named, combinable business rule (`isSatisfiedBy`, and/or/not)	Same rule in queries and validation; finder-method explosion	`Predicate`; Spring Data JPA `Specification`	—	§11.7.4
MVC	Separate model, view and controller (Observer + Strategy + Composite)	UI, input handling and rules tangled	Spring MVC; Swing `JTable` + `TableModel`	MVP, MVVM (relatives)	§11.7.5
Event Sourcing (lite)	Store the events; derive current state by replaying them	Full history, audit, time travel	Append-only logs	Plain audit log	§11.7.6

"Which pattern?" by problem symptom

Combined from §8.5, §8.15, §9.8, §11.8 and §7.6.

If you see / need…	Use	Not to be confused with
One operation, several interchangeable rules chosen at run time (pricing, matching, routing)	**Strategy**	State, Template Method
The class to create depends on a type value; caller shouldn't know the concrete class	**Factory** (Simple Factory / Factory Method)	Abstract Factory
Products must always match as a family	**Abstract Factory**	Factory Method
Constructor has ~6+ arguments, many optional	**Builder**	—
Another one of *this*, cheaply or without knowing its class	**Prototype**	Memento
Exactly one instance must exist	**Singleton** (or DI singleton scope)	A convenient global handle
Expensive objects created and destroyed repeatedly	**Object Pool**	Flyweight
Existing class has the wrong interface	**Adapter**	Facade, Bridge
Two dimensions multiplying classes (`WindowsButton`, `MacButton`, …)	**Bridge**	Adapter
Tree of part-whole objects treated the same	**Composite**	Decorator
Add behaviour in combinations without editing the class	**Decorator**	Proxy
Every caller repeats the same five steps into a subsystem	**Facade**	Mediator
Millions of near-identical objects	**Flyweight**	Object Pool
Lazy load, access control, remote, caching (same interface)	**Proxy**	Decorator
A request must pass several checks; any may reject it	**Chain of Responsibility**	Decorator (all layers run)
Undo/redo, queue, schedule, retry or log *requested actions*	**Command**	Strategy
Evaluate user-written rules in a small language	**Interpreter** (+ Specification)	Visitor
Walk a collection/tree/paged API without exposing structure	**Iterator**	Visitor
Many objects depend on each other and the **order** of reactions matters	**Mediator**	Observer
Roll back to an earlier state without exposing internals	**Memento**	Command undo
One change must reach an open-ended set of listeners	**Observer** (or a broker)	Mediator
Three or more methods repeat one `switch` over a status; legal actions depend on it	**State** (or enum + guarded method for 3–4 states)	Strategy
Several classes share a long fixed flow, differ in a few steps	**Template Method**	Strategy
Many new operations over a stable set of types	**Visitor**, or sealed types + pattern matching	Iterator
`null` checks everywhere	**Null Object** / `Optional`	—
Classes create their own collaborators	**Dependency Injection**	Service Locator
Business logic mixed with storage	**Repository**	DAO
Same rule duplicated in queries and validation	**Specification**	—

Three questions for the famous look-alikes (§11.8):

1. Do the alternatives reference each other? Yes → **State**. No → **Strategy** (or Bridge if chosen once at wiring).
2. Can one participant stop the flow? Yes → **Chain of Responsibility**. Everyone always runs → **Decorator**. Everyone is told, nobody stops anyone → **Observer**.
3. Does a central object decide what happens next? Yes → **Mediator**. Each listener decides for itself → **Observer**.

The six that come up in almost every round (§1.9, §8.5)

Pattern	Signal	Do not use when
Strategy	One operation, several swappable rules	Only one rule exists and nothing hints at a second
Factory	Creation depends on a type value	Only one concrete class exists
Observer	Several listeners the source should not know	A single listener the source already references
State	Behaviour depends on status; many transitions	3–4 states handled by one guarded method
Singleton	Genuinely exactly one	You just want a global handle
Builder	Many optional fields; long constructor	A handful of required fields

Common mistakes (§8.6)

Anti-pattern	Fix
God Object (one class does everything)	Split by responsibility; push behaviour to the data owners
Golden Hammer (favourite pattern everywhere)	Start from the problem's signal
Pattern overuse ("patternitis") — factories for one class, single-implementation interfaces	YAGNI: add the pattern when the second variant arrives
Singletonitis (singletons as globals)	Dependency injection
Cargo-cult patterns (structure without intent)	Name the property the pattern gives you, or drop it

In the interview

Use **one or two** patterns in a 45-minute design, placed where their signal appears, and **name one you rejected** and why (§8.5, §8.16).

22.8 Concurrency cheat sheet

Source: Chapter 12 (concepts) and Chapter 13 (classic problems).

TL;DR (§12.28)

Races come from **shared mutable state**. Make the critical section atomic, or remove sharing/mutability. Visibility needs **happens-before** edges. Prevent deadlock with **lock ordering** or **tryLock + back-off**. In LLD: name the contended resource, keep **check + claim atomic**, lock fine-grained and in order, and say what changes with many servers.

Primitive | use for | gotcha

Primitive	Use for	Gotcha	Where
`synchronized`	Making a multi-step update atomic (mutex + visibility)	Don't lock on a changing reference, a boxed value or a string literal; no slow I/O while holding it; readers must use the same lock	§12.7
`ReentrantLock`	Same as `synchronized` plus try/timed/fair locking and `Condition`s	Call `lock()` **just before** `try`, `unlock()` in `finally`; fairness is noticeably slower	§12.8
`tryLock(timeout)`	Give up if the lock is busy; deadlock prevention with back-off	Never `unlock()` when `tryLock` returned `false`; don't retry in a tight loop (livelock)	§12.9
`ReentrantReadWriteLock`	Many readers, few writers	Cannot upgrade read → write (self-deadlock); don't modify under the read lock; writer starvation in non-fair mode	§12.10
`StampedLock`	Read-mostly data with optimistic reads	**Not reentrant**; `validate` the stamp before using values read optimistically	§12.10
`volatile`	Visibility of a flag or a published immutable snapshot	Gives visibility, **not** atomicity: `count++` is still a race	§12.6
`AtomicInteger` / `AtomicLong` / CAS	Lock-free counters and single-variable claims	Separate `get()` then `set()` is still a race — use `compareAndSet` / `updateAndGet`; no side effects in the lambda (may run several times); ABA → `AtomicStampedReference`	§12.15
`LongAdder`	Very hot counters (metrics)	Use instead of `AtomicLong` when contention is high	§12.15
`Semaphore(N)`	Limit concurrent access to N; signalling between threads	Release in `finally`; don't release when `acquire` never succeeded	§12.12
`wait/notifyAll`, `Condition`	Waiting for a condition on shared state	Always `while`, never `if` (spurious wakeups); hold the monitor; `notify()` can wake the wrong thread; state change without notify = lost wakeup	§12.13
`CountDownLatch`	Wait for N events once	One-shot; `countDown()` in `finally`, `await(timeout)`	§12.14
`CyclicBarrier` / `Phaser`	N threads meet at each phase (reusable)	Fewer threads than parties → everyone waits forever	§12.14
`ConcurrentHashMap`	Shared map	`containsKey` then `put` is a race — use `putIfAbsent` / `computeIfAbsent` / `merge`; no slow work or recursive updates inside `compute`; no `null` keys/values	§12.16
`CopyOnWriteArrayList`	Read-mostly lists (listener lists)	Every write copies the array	§12.16
`BlockingQueue` (bounded)	Handing work between threads (producer–consumer)	Unbounded queue → no back-pressure → memory blow-up; one poison pill per consumer	§12.22
`ExecutorService` / `ThreadPoolExecutor`	Running tasks on a bounded pool	Call `shutdown()`; `newFixedThreadPool` has an **unbounded** queue; exceptions in `submit`ted tasks are lost unless	§12.4, §12.21

Primitive	Use for	Gotcha	Where
		you call `get()`; tasks waiting on tasks in the same pool → thread-starvation deadlock	
Virtual threads	Many blocking I/O tasks	Pinning (Java 21–23: blocking inside `synchronized`); heavy `ThreadLocal`s waste memory	§12.4, §12.24
`ForkJoinPool` / parallel streams	Splitting CPU work	Use a threshold: tiny tasks cost more to schedule than to run; subtasks must touch disjoint data	§12.26, §13.10
`CompletableFuture`	Combining async results (`thenApply`, `thenCombine`, `allOf`)	Handle errors with `exceptionally` rather than calling `get()` at every step	§12.4
`ThreadLocal`	Per-thread state (thread confinement)	Clear it (`remove()`) in pooled threads, or values leak into the next task	§12.24
Immutability / confinement	Avoiding synchronization entirely	`final List` is not immutable if the list is mutable and exposed	§12.24

Which `java.util.concurrent` class for which job (§12.26)

I need to…	Use
Increment a shared counter	`AtomicInteger`/`AtomicLong`; `LongAdder` if very hot
Stop a loop from another thread	`volatile boolean` flag or `interrupt()`
Make a multi-step update atomic	`synchronized` or `ReentrantLock`
Give up if the lock is busy	`tryLock(timeout)`
Many readers, few writers	`ReentrantReadWriteLock`, `StampedLock`, or an immutable snapshot in a `volatile` field
Limit concurrent access to N	`Semaphore(N)`
Wait for a condition on shared state	`Lock` + `Condition` (or `wait/notifyAll`) in a `while` loop
Wait for N events once	`CountDownLatch`
N threads meet each phase	`CyclicBarrier` / `Phaser`
Hand work between threads	Bounded `BlockingQueue`
Shared map	`ConcurrentHashMap` with `putIfAbsent`/`compute`/`merge`
Read-mostly list	`CopyOnWriteArrayList`
Run tasks	`ExecutorService` (bounded); virtual threads for blocking I/O
Split CPU work	`ForkJoinPool` / parallel streams
Combine async results	`CompletableFuture`
Avoid synchronization entirely	Immutability, thread confinement

Deadlock checklist (§12.17–§12.19)

A deadlock needs **all four Coffman conditions**; break any one and it cannot happen.

#	Condition	How to break it
1	Mutual exclusion	Lock-free/immutable data, shareable resources (read locks)
2	Hold and wait	Acquire everything at once, or `tryLock` and release everything on failure
3	No preemption	Timeouts (`tryLock(timeout)`), leases that expire
4	Circular wait	**Global lock ordering** (always lock the lower id first)

- [ ] Do I ever hold two locks? If yes, is there **one global order** (account id, seat id), with a tie-breaker for equal keys?
- [ ] Do I call a **listener, callback or another object's synchronized method while holding my lock**? Move it outside the lock.
- [ ] Do tasks in a bounded pool **wait for other tasks in the same pool**? (thread-starvation deadlock)
- [ ] Are retries randomised (**jitter**) so two threads don't back off in lockstep? (livelock, §12.18)
- [ ] Could one thread wait forever while others progress? Consider fairness (§12.19).
- [ ] Detection: a thread dump (`jstack` / `jcmd`) prints "Found one Java-level deadlock"; `ThreadMXBean.findDeadlockedThreads()`.

Deadlock vs livelock vs starvation: nobody progresses and all wait / everybody runs but nobody progresses / some progress while one never does (§12.27).

The LLD concurrency recipe (§12.25)

1. **Name the shared mutable state** (free spots, seat → owner, stock).
2. **State the invariant** ("a seat has at most one booking", "stock ≥ 0").
3. **Find the critical operation** — usually a check-then-act (*find free → assign*).
4. **Pick the smallest tool that makes it atomic:** one item → `queue.poll()`, `putIfAbsent`, `compareAndSet`; several items all-or-nothing → per-item locks **in sorted order**; across servers → DB constraints/transactions, optimistic versioning, or a distributed lock.
5. **Choose granularity:** per floor / show / SKU, not one global lock.
6. **Handle time:** holds with expiry, timeouts, idempotent retries.

Classic problems (§13.12)

Problem	Key primitive	Invariant to state
FooBar (§13.1)	2 semaphores	Exactly one permit exists across both
Zero Even Odd (§13.2)	3 semaphores	Zero dispatches; one permit circulates
FizzBuzz (§13.3)	monitor + `notifyAll`	Predicates partition 1..n
H2O (§13.4)	2 semaphores + barrier	≤ 2 H and ≤ 1 O per round
Dining Philosophers (§13.5)	ordered locks / n−1 semaphore	No circular wait
TTL (time to live) cache (§13.6)	`ConcurrentHashMap` + `compute` + conditional remove	Never return or keep expired data
Striped map (§13.7)	lock array	Each bucket belongs to exactly one stripe
Blocking queue (§13.8)	lock + 2 conditions	0 ≤ count ≤ capacity; wait in loops
Bloom filter (§13.9)	atomic OR	Bits only go 0 → 1; no false negatives
Merge sort (§13.10)	Fork/Join	Subtasks touch disjoint ranges
Token-bucket rate limiter (§13.11)	lock or CAS on immutable state	Refill + take is one atomic step

Common mistakes (§12.25 and throughout Chapter 12)

- A global `synchronized` on `ParkingLot.park()` — correct, but serialises every gate.
- Checking availability with one call and booking with another (check-then-act).
- Locking seats in the order the user clicked them (deadlock between two multi-seat bookings).
- Forgetting that in-memory locks don't work across multiple app servers.
- Swallowing `InterruptedException` — rethrow, or restore the flag with `Thread.currentThread().interrupt()` (§12.3).
- Calling `run()` instead of `start()` (§12.3).

22.9 Case-study map

Source: Chapters 14–21. "Concurrency concern" is the race each chapter guards against, and "atomic step" is how.

Everyday objects (Chapter 14)

Problem	Key entities	Patterns used	Concurrency concern	Where
Parking lot	`ParkingLot`, `ParkingFloor`, `ParkingSpot`, `Vehicle`, `Ticket`, `DisplayBoard`, `SpotType` / `VehicleType` enums	Strategy (pricing, spot-finding), Factory (electric spots), Observer (boards); Singleton rejected	Two gates take one spot → atomic `pollFirst` + CAS; per-ticket lock for payment; `putIfAbsent` for plates	§14.1
Vending machine	`VendingMachine`, `Slot`, `Inventory`, `CashBox`, `Denomination`, `VendingState` + 5 states	**State** (the whole design)	`synchronized` machine; change-making by bounded coin-change DP (dynamic programming: build the answer from smaller amounts), not greedy (always take the biggest coin)	§14.2
Coffee machine	`CoffeeMachine`, `Recipe`, `RecipeBook`, `IngredientStore`, `OrderResult` (sealed)	Observer (low stock), recipes as data	Two orders take the last milk → one `ReentrantLock` around check + deduct; notify outside the lock	§14.3
ATM	`Atm`, `AtmState` + 4 states, `Transaction` (sealed), `NoteHandler`, `CashDispenser`, `BankService`, `Account`	State (session), Command (transactions), Chain of Responsibility (note cassettes)	Overdraw from two ATMs → per-account lock; transfers lock in id order	§14.4
Elevator	`Elevator`, `StopList` (two sorted sets), `Door`, `Dispatcher`, `DispatchStrategy`, `Building`	Strategy (dispatch) + modes; LOOK scheduling	Per-car lock, synchronized dispatcher; tick-driven `step()`	§14.5
Traffic signal	`Intersection`, `TrafficLight`, `Phase`, `SignalState` + 5 states, `TimingPolicy`, `SafetyMonitor`	State (timed), Strategy (timing), Observer (lights)	Synchronized tick; safe with a timer thread plus emergency threads	§14.6
Shipping locker	`LockerStation`, `Compartment`, `Booking`, `CompartmentSelector`, `PickupResult` (sealed), `LockerListener`, `LockerNetwork`	Strategy (door choice), Observer (SMS text messages), enum state machines, sealed results	Two carriers, one door → station lock around choose-and-mark; lazy expiry of holds and pickup windows; hashed single-use codes, keypad lockout	§14.7

Booking & inventory (Chapters 15–16)

Problem	Key entities	Patterns used	Concurrency concern	Where
Library	`Book` (title) vs `BookItem` (copy), `Member`, `BookLending`, `LendingService`, `Catalog`	Strategy (fines), notifier interface; FIFO (first in, first out) reservation queue per title	Last copy lent twice → lock member then copy (fixed order)	§15.1
Movie tickets (BookMyShow)	`Show`, `ShowSeat`, `Booking`, `SeatLockProvider`, `BookingService`, `CancellationPolicy`	Strategy (refund rule), gateway behind an interface	Book `ShowSeat`s; all-or-nothing hold with TTL → pay → atomic commit; refund if hold lost	§15.2
Concert tickets	`SeatedInventory`, `GaPool`, `SeatState` (sealed), `Hold`, `VirtualWaitingRoom`, `PurchaseLimiter`	Sealed seat states, waitlist	Extreme contention → CAS per seat, CAS counter for GA (general admission, unseated), FIFO waiting room, per-user limits, TTL holds with a reaper	§15.3
Hotel	`Hotel`, `Room`, `RoomBooking`, `Invoice`, `RoomCharge`, `Payment` subclasses, `FrontDesk`	Template Method (`RoomCharge`)	Room × night reserved atomically in `synchronized Room.tryReserve`	§15.4
Car rental	`Vehicle`, `DateRange`, `Reservation`, `Bill`/`BillItem`, `ReservationAddOn`, `PricingStrategy`	Template Method (add-ons), Strategy + Decorator (pricing), state machine	Availability is about **dates** → `Vehicle.tryBook(range)` synchronized	Ch 16 · Car Rental
Restaurant	`Menu`/`MenuSection`/`MenuItem`, `Table`, `Reservation`, `Order`, `Meal`/`MealItem`, `Kitchen`, `Check`	Observer (order ready), Strategy (chef assignment, payment), state machine	Two receptionists, one table → `Table.tryHold(time)` synchronized; listeners after the lock	Ch 16 · Restaurant
Airline	`Flight` vs `FlightInstance`, `Seat` vs `FlightSeat`, `FlightReservation`, `Itinerary`, `FareStrategy`	Strategy + Decorator (fares), Facade	Selling one seat twice → `FlightInstance.holdSeats` all-or-nothing under the instance lock	Ch 16 · Airline
Course registration	`Course` vs `Section`, `Student`, `RegistrationRule`s, `Registrar`	Chain of rules / Specification	Overselling a section → section lock inside a student lock (fixed order)	Ch 16 · Course Registration

Games (Chapter 17)

Problem	Key entities	Patterns used	Concurrency concern	Where
Blackjack + deck	`Card`, `Deck<T>`, `Shoe`, `Hand`, `BlackjackScoring`, `PlayerStrategy`, `DealerPolicy`	Strategy (player, dealer); injected randomness	One table = one thread; shared balances need an atomic wallet service	Ch 17 · Blackjack
Chess	`Piece` (sealed) + subclasses, `SlidingPiece`, `Board`, `Move`, `Rules`, `Game`	Polymorphic pieces, Factory Method, Command + Memento (undo)	`synchronized` game so a move and a resignation don't interleave	Ch 17 · Chess
Tic-tac-toe	`Board` (row/col/diagonal counters), `Game`, `Player`, `MoveStrategy`	Strategy (players)	Many games in a `ConcurrentHashMap`; per-game lock	Ch 17 · Tic-Tac-Toe
Snake & ladder	`Board`, `Jump`, `Dice`, `Rules`, `OverflowRule`, `WinStrategy`, `Game`	Strategy (dice, overflow, win)	Each game its own `Random` and its own thread of turns	Ch 17 · Snake and Ladder
2048	`LineMerger`, `Board`, `TileSpawner`, `Game`, `Direction`	Strategy (spawner); one pure merge function	Single-threaded; `synchronized move` if UI and input threads both call it	Ch 17 · 2048

Platforms (Chapters 18–19)

Problem	Key entities	Patterns used	Concurrency concern	Where
Stack Overflow	`Post` (sealed) → `Question` / `Answer` / `Comment`, `Member`, `Tag`, `Bounty`, `QuestionIndex`	Strategy (reputation), Observer (badges), Facade; inverted index	Two votes at once → `synchronized` post; `AtomicInteger` reputation	§18.1
Cricinfo	`Match` → `Innings` → `Over` → `Ball`, `Team`, `Playing11`, `StatQuery`	Template Method (stat queries), Observer (live pages)	Two scorers → `synchronized` match; observers in `CopyOnWriteArrayList`	§18.2
LinkedIn	`Member`, `Profile`, `SocialContent`, `ConnectionInvitation`, `ConnectionService`, `SearchIndex`, `FeedService`	State (invitation), Strategy (feed ranking, notifications), Facade	Symmetric connections → all graph changes synchronized on the service	§18.3
Facebook	`Member`, `Audience` (sealed), `PrivacyList`, `FriendService`, `VisibilityService`, `NewsFeedService`	Strategy (feed ranker), Observer (notifications)	Accept can't leave a one-way friendship → synchronized service, concurrent adjacency sets	§18.4
Amazon	`Product`, `ShoppingCart`, `Order`, `OrderLog`, `Payment`, `Shipment`, `Customer` → `Guest` / `Member`	Strategy (payment), Observer (notifications), order state machine, Facade	Last unit, two buyers → `synchronized Product.reserve`; all-or-nothing multi-line reservation	Ch 19 · Amazon
Stock brokerage + exchange	`Order` → Market/Limit/StopLoss/StopLimit, `OrderPart`, `StockLot`, `OrderBook`, `MatchingEngine`	Observer (fills), Template Method (statements), trade-price policy	Over-spending → reserved funds; one lock per symbol; price-then-time matching	Ch 19 · Stock Brokerage
Auction	`AuctionListing`, `Bid`, `BidResult`, `AuctionService`, `AuctionListener`	Observer, Specification (search), status state machine	Concurrent bids → all bid rules in one per-listing critical section; notify after unlocking	Ch 19 · Auction
Music streaming (Spotify)	`Song` / `Album` / `Artist`, `MusicLibrary`, `Playlist`, `MusicPlayer`, `PlaybackOrder`, `Recommender`	Strategy (play order, recommendations), Observer (plays), plan rules in an enum	Two devices press "next" → `synchronized` player methods	Ch 19 · Music Streaming
Food delivery (Swiggy) + FoodKart	`Restaurant`, `Cart`, `Order`, `DeliveryAgent`, `DeliveryAssignmentStrategy`, `RestaurantSelectionStrategy`	Strategy (assignment, selection, payment), Observer, order state machine	Kitchen capacity reserved atomically; riders claimed with CAS	Ch 19 · Food Delivery
Grocery store (POS, point of sale)	`Register`, `Sale`, `LineItem`, `PricingEngine`, `ItemPromotion` / `BasketPromotion`, `TaxPolicy`, `Tender` (sealed), `Receipt`, `Inventory`	Strategy (offers, tax, payment gateway), Facade (register), Observer (stock), sale state machine	Registers share stock → atomic `merge`, may go negative, edge-triggered alerts; returns claimed with `compute`	Ch 19 · Grocery Store

Pattern-driven components (Chapter 20)

Problem	Key entities	Patterns used	Concurrency concern	Where
Splitwise	`Expense`, `SplitStrategy` (Equal/Exact/Percent), `Share`, `Ledger`, `DebtSimplifier`	Strategy (splits)	One signed number per pair, one mutator; `BigDecimal`, no lost cents	§20.1
Notification service	`NotificationChannel` (abstract), channels, `SendRule`, `ChannelFactory`, `DeliveryListener`	Template Method (channels), Strategy (send rules), Observer, Simple Factory, Adapter	Rate-limit check-then-act gap; idempotency	§20.2
In-memory cache / LRU	`Cache<K,V>`, `Node`, `DoublyLinkedList`, `EvictionPolicy`, `Cache.Builder`, `ShardedCache`	Strategy (eviction), Observer (removals), Builder	One lock (a `get` is a write in LRU); listeners outside the lock; shard to scale	§20.3
Ride sharing (Uber)	`Rider`, `Driver`, `Trip`, `MatchingStrategy`, `PricingStrategy`, `SurgePricing`	Strategy ×3, Decorator (surge), trip state machine	Double-booking a driver → atomic claim (`AtomicBoolean`)	§20.4
Logging framework	`Logger`, `LogManager`, `LogRecord`, `Appender`, `AsyncAppender`, `Formatter`	Chain of Responsibility (hierarchy), Singleton (`LogManager`), Strategy (formatters), Template Method (appender), Decorator (async)	Thread-safe sinks; never block on a full queue; never crash the app	§20.5
Pub-sub	`Broker`, `Topic`, `Subscription`, `Subscriber`, `Message`	Observer through a broker	One dispatcher per subscription; ack after processing (at-least-once) → idempotent subscribers	§20.6
Rate limiter	`Limiter` (TokenBucket, FixedWindow, SlidingWindowLog/Counter), `CompositeLimiter`, `LimiterFactory`	Strategy, Simple Factory, Composite, Facade	Atomic refill-check-take; one limiter per key via `computeIfAbsent`; Redis (a shared in-memory store) + Lua (a script Redis runs atomically) across servers	§20.7

Machine-coding problems (Chapter 21)

Problem	Key entities	Patterns used	Concurrency concern	Where
Task management	`Task`, `TaskStatus`, `TaskService`, `TaskRepository`, `TaskFilter`	Specification (filters), Strategy (sorts), Observer, enum state machine	Per-task `synchronized`	§21.1
Digital wallet	`Wallet`, `Transaction`, `Money`, `Offer`, `WalletService`	Strategy (offers), idempotency key	Lock both wallets in a global (id) order; debit before credit; total conserved	§21.2
Event calendar (Flipkart)	`TimeSlot` (half-open), `Event`, `EventStore`, `RepresentativeSelector`, `CalendarService`	Strategy (representative), plan-then-save	Atomic check-and-book	§21.3
Coding Blox (Flipkart)	`Contest`, `Level`, `User`, `Question`, `SolveSimulator`, `CommandProcessor`	Command map, Strategy (solver, scoring)	Atomic rating update; don't run a contest twice	§21.4
PropertyHunt (ClearTrip)	`Property`, `SearchCriteria` (+ Builder), `PropertyService`	Builder + predicate list, Strategy (sort)	Synchronized mark-sold	§21.5
Ledger Co (Navi)	`Loan`, `LumpSum`, `Balance`, `InterestPolicy`, `LedgerService`	Strategy (interest), derived state, command loop	Synchronized lump sums	§21.6
Inventory management	`Product`, `Warehouse`, `StockMovement`, `AllocationStrategy`, `ReorderListener`	Strategy (allocation), Observer (edge-triggered alerts)	Overselling → lock per SKU (stock-keeping unit: one product variant)	§21.7
Unix file search (`find`)	`Node` → `FileNode` / `DirectoryNode`, `FileFilter`, `AndFilter` / `OrFilter` / `NotFilter`, `SearchCriteria` (+ Builder), `FileSearcher`, `FindCommandParser`	Specification + Composite (filters and file tree), Builder, recursive-descent parser	Weakly consistent children map; read-only fork/join walk	§21.8

> **TL;DR — what every case study teaches**
>
> - **Name the real unit of inventory** (copy, show seat, room-night, vehicle-day) (Ch 15 summary).
> - **Every "if free then take" is one atomic step** — a monitor, a CAS, or a conditional `UPDATE` (Ch 15 summary).
> - **Put rules on the object that owns the data**; the service only orders the steps (Ch 16, Ch 20 summaries).
> - **Strategy** for the rule most likely to change; **Observer** for "tell someone when it happens"; **State** (or an enum state machine) for lifecycles.
> - **Inject** time and randomness; money in `BigDecimal` or integer cents; call listeners **outside** locks.

22.10 Machine-coding and edge-case checklists

Source: §21.0 (and §1.10).

Machine-coding checklist

Before coding (0–20 min)

- [ ] Read the statement twice; mark **mandatory vs bonus**; ask clarifying questions.
- [ ] Put the statement's sample input into the driver as a `List<String>` of commands.

- [ ] Ask: "Where does it need to be thread-safe?" (wallets and inventory: always yes).
- [ ] Nouns → entities, verbs → service methods. Pick the 1–2 patterns the statement begs for ("can be sorted by X or Y" → Strategy).
- [ ] Create the six packages: **model, repository, service, strategy, exception, driver**.

While coding (20–75 min)

- [ ] Implement mandatory requirements **in the statement's order**; run the driver after each one (at least every 15 minutes).
- [ ] Entities guard their own invariants (e.g. `Task.changeStatus` rejects `TODO → DONE`).
- [ ] Services return data and throw domain exceptions; **no `System.out` in services**.
- [ ] Repository = interface + `ConcurrentHashMap` implementation; IDs from an `AtomicInteger`.
- [ ] Driver = `Map<String, Consumer<String[]>>` command processor; catch domain exceptions per command, **print and continue**.

Before you stop (75–90 min)

- [ ] Edge cases and validation done (see below); custom exceptions with readable messages.
- [ ] Thread-safety where money or stock is shared; say where the lock is and why.
- [ ] Bonus only after every mandatory feature runs.
- [ ] Clean names, delete dead code, run everything once more, prepare a 2-minute walkthrough.

Rubric the interviewer uses (§21.0)

#	Criterion	How to score well
1	Working, demo-able code	Thin end-to-end path in the first hour; never finish with code that doesn't compile
2	Functional completeness	Mandatory first, in order
3	Clean models	Enums for fixed sets, records for values, classes for things that change
4	Separation of concerns	Driver → Service → Repository
5	Extensibility	New rule = new class (Strategy, Specification)
6	Edge cases & failure handling	Custom exceptions; the driver keeps going
7	Readability	Names from the statement's nouns and verbs
8	Concurrency (when asked)	Say where the lock is; demo with a thread pool if time allows
9	Tests (bonus)	A tiny `assert`-style self-check in `main` is enough

Edge-case checklist (§1.10, §21.0, case studies)

Category	Questions to ask your code
Input	`null`, empty, negative, zero, duplicate IDs, unknown IDs, invalid numbers
Ownership	Wrong user acting on someone else's object; a player moving out of turn
Capacity	Full lot, zero seats left, empty playlist, max loans reached, party larger than any table
State	Paying twice, exiting without entering, cancelling a delivered order, moving after game over, running a contest twice
Time	Exit before entry, holds/bookings expiring, part hours (round up?), an event exactly at the end instant (half-open intervals), late returns
Money	`BigDecimal` scale and rounding; no lost cents in splits; compare with `compareTo`, not `equals` (Ch 4 money note)
Concurrency	Two threads booking the same seat/spot/rider; multi-item bookings locking in sorted order; retries that must be idempotent
Failure	Payment declined after a hold; hold lost before payment → compensating refund; listener throws
Randomness	Seeded `Random` or a scripted strategy so the demo is repeatable

Common mistakes (§21.0)

Mistake	Instead
20 minutes on a perfect class diagram	Enums/records first; the diagram lives in names
One `Manager` with 15 maps	One service per area + repositories
`System.out.println` inside a service	Services return data; the driver prints
`double` for money	`BigDecimal` with a fixed scale, or whole units in a `long`
Crashing on the first bad command	Catch domain exceptions in the driver loop
Bonus before mandatory	Mandatory in order; bonus last
`Random` without a seed	Seed it, or put randomness behind a Strategy
`LocalDate.now()` in logic	Inject a `java.time.Clock`

22.11 Glossary

Alphabetical. "Where" gives the chapter (and section) where the term is explained in full. Besides the design vocabulary, the list covers the everyday Java, tooling and domain words the case studies use (`JVM`, `Redis`, `SKU`, `UPI` …), so a newcomer can look up anything a chapter takes for granted. System-design terms such as latency, throughput, caching, message brokers and eventual consistency are explained in more depth in System Design Fundamentals.

Term	Plain meaning	Where
ABA problem	A value changes A → B → A, so a compare-and-swap cannot notice the change; fix with a version stamp (`AtomicStampedReference`)	Ch 12 §12.15
Abstract class	A class that cannot be instantiated; may hold state, constructors, finished and abstract methods	Ch 2 §2.9
Abstract Factory	Creational pattern that creates families of matching objects without naming their classes	Ch 8 §8.11
Abstraction	Exposing only what callers need and hiding how it is done (interface or abstract class)	Ch 2 §2.4
Access modifiers	`public`, `protected`, package-private, `private`: who can see a member	Ch 2 §2.14
Activity diagram	UML flowchart of a process, with decisions, swimlanes and fork/join	Ch 3 §3.13
Actor	Someone or something that uses the system (customer, admin, the system itself)	Ch 1 §1.5
Adapter	Structural pattern that converts one interface into the one the client expects	Ch 9 §9.1
Aggregate (DDD)	A cluster of objects changed as one unit; only its root is reachable from outside, and the root guards the invariants	Ch 5 §17
Aggregation	"Has-a" where the part can outlive the whole; hollow diamond at the whole	Ch 3 §3.3
Anemic domain model	Classes with data but no behaviour; the logic lives in services that pull the data out	Ch 5 §5
Anti-corruption layer	Translation layer at a bounded context's border that keeps a foreign or legacy model out of yours	Ch 5 §17
Anti-pattern	A common "solution" that reliably causes more problems than it solves	Ch 8 §8.6
AOP (aspect-oriented programming)	Adding cross-cutting behaviour (logging, transactions) around many methods without editing them, usually via generated proxies	Ch 5 §11, Ch 9 §9.7
API	Application Programming Interface: the set of public methods (or endpoints) other code is allowed to call	Ch 2 §2.6, §2.14
Appender / formatter	Logging roles: an appender writes a log record somewhere (console, file); a formatter turns it into text	Ch 20 §20.5
Append-only log	Messages are only ever added at the end; readers track their own offset	Ch 20 §20.6
Assertion	A check in code or a test that a condition is true, failing loudly if it is not	Ch 6 §8
Association	A class keeps a reference to another it does not own; solid line	Ch 3 §3.2
Association class	A class that holds data about a link itself (e.g. `Enrollment{grade}`)	Ch 3 §3.11
AST (abstract syntax tree)	The tree a parser builds from an expression or program, one node per piece of grammar	Ch 10 §10.6
Asynchronous vs synchronous	The caller carries on and gets the result later vs the caller waits until the work finishes	Ch 3 §3.12, Ch 20 §20.5
At-least-once delivery	A message may be delivered more than once, never lost; consumers must be idempotent	Ch 20 §20.6
Atomic variable	`AtomicInteger` / `AtomicLong` / `AtomicReference`: lock-free single-variable updates using CAS	Ch 12 §12.15

Term	Plain meaning	Where
Atomicity / atomic operation	An operation that happens completely or not at all, with no other thread able to see it half-done	Ch 12 §12.5
Audit log	An append-only record of who did what and when, kept for checking later	Ch 10 §10.2, Ch 21
Authentication vs authorization	Proving who you are (login, PIN) vs checking what you are allowed to do	Ch 6 §10, Ch 14 §14.4
Back-pressure	Slowing producers when consumers can't keep up, e.g. with a bounded queue	Ch 12 §12.22
BFS / DFS	Breadth-first search (visit neighbours level by level) vs depth-first search (go deep down one path, then back up)	Ch 18 §18.3, Ch 21 §21.8
`BigDecimal`	Exact decimal type used for money; build from strings, compare with `compareTo`	Ch 1 §1.10, Ch 4 (money note)
Big-O notation	Shorthand for how running time or memory grows with input size: `O(1)` constant, `O(log n)`, `O(n)` linear	Ch 20 §20.3
Blocking call	A call that makes the thread wait (for I/O, a lock or a queue) before it can continue	Ch 12 §12.4
Bloom filter	Probabilistic set: may say "maybe present" wrongly, never says "absent" wrongly	Ch 13 §13.9
Boilerplate	Repetitive code that must be written but carries no real logic (getters, `equals`, `hashCode`)	Ch 2 §2.13
Bounded context	Boundary inside which one domain model and one vocabulary apply consistently (often one team or service)	Ch 5 §17
Bounded queue	A queue with a fixed capacity; producers wait or are refused when it is full	Ch 12 §12.22
Boy Scout Rule	Leave the code a little cleaner than you found it	Ch 6 (intro)
Bridge	Structural pattern splitting an abstraction from its implementation so both vary independently	Ch 9 §9.2
Broker	A middleman service that receives messages from publishers and delivers them to subscribers	Ch 20 §20.6
Build tool	A program (Maven, Gradle) that downloads dependencies, compiles, tests and packages the code	Ch 3 §3.5
Builder	Creational pattern that builds a complex object step by step, validated in `build()`	Ch 8 §8.12
Busy waiting (spinning)	A thread repeatedly checking a condition in a loop instead of sleeping; wastes CPU	Ch 12 §12.6
Bytecode	The compact instruction format `javac` produces and the JVM runs	Ch 12
Callback	A function you hand to other code so it can call you back when something happens	Ch 5 §10
Caretaker / Originator	Memento roles: the originator makes and restores snapshots; the caretaker stores them without looking inside	Ch 10 §10.5
Carrier thread	The real OS thread that a virtual thread runs on while it is not blocked	Ch 12 §12.4
CAS (compare-and-swap)	Atomic "set to new value only if it still equals the expected value"	Ch 12 §12.15
Chain of Responsibility	Behavioral pattern passing a request along handlers until one handles or stops it	Ch 10 §10.1

Term	Plain meaning	Where
Characterisation test	A test that records what existing code does now, before you refactor it	Ch 7 §7.8
Checked vs unchecked exception	Checked exceptions must be declared or caught (compiler-enforced); unchecked ones (`RuntimeException` subclasses) need not be	Ch 2 §2.5, Ch 6 §6
Check-then-act	Read a condition, then act on it, as two steps: a race unless made atomic	Ch 12 §12.5, §12.25
Circuit breaker	Stops calling a failing dependency for a while after repeated errors, then tries again carefully	Ch 5 §13
Circular wait	T1 waits for T2's lock while T2 waits for T1's; one of the four deadlock conditions	Ch 12 §12.17
Class	A blueprint for objects: fields plus methods	Ch 2 §2.2
Class diagram	UML diagram of classes, fields, methods and relationships — "the answer" in an LLD round	Ch 3 §3.11
Class explosion	Subclasses multiplying across two dimensions (`WindowsButton`, `MacButton`, …)	Ch 9 §9.2
Classpath	The list of folders and JAR files where the JVM looks for classes	Ch 1 §1.3
Clean code	Code whose intent is obvious to the next reader and which is easy to change safely	Ch 6 (intro)
Coarse- vs fine-grained locking	One big lock for everything (simple, slow under load) vs many small locks per item (faster, harder to get right)	Ch 12 §12.11
Code review	A peer reads a change before merge: correctness first, then design, readability, tests, security, performance	Ch 6 §15
Code smell	A surface symptom that usually signals a deeper design problem; not a bug by itself	Ch 7 §7.2
Coffman conditions	Mutual exclusion, hold and wait, no preemption, circular wait: all four needed for deadlock	Ch 12 §12.17
Cohesion	How closely the things inside one class belong together; high is good	Ch 2 §2.16, Ch 4 §4.3
Coin-change DP	Finding the fewest coins (or whether change is possible) by building answers for smaller amounts first	Ch 14 §14.2
Collections framework	Java's built-in data structures (`List`, `Set`, `Map`, `Queue`) and their implementations	Ch 2
Command	Behavioral pattern turning a request into an object so it can be queued, logged or undone	Ch 10 §10.2
Command–Query Separation (CQS)	A method either changes state or returns a value, not both	Ch 6 §2
Commit / rollback	Make a transaction's changes permanent vs throw all of them away	Ch 10 §10.5, Ch 15 §15.2
`Comparator` / `Comparable`	An object that orders two values from outside vs a class's own natural order (`compareTo`)	Ch 1 §1.9, Ch 11 §11.3
Compensation	Undoing a completed step with a new action (e.g. a refund) because money can't be rolled back	Ch 15 §15.2
Compile time vs run time	When `javac` checks and translates the code vs when the program is actually executing	Ch 2 §2.7
`CompletableFuture`	A future you can chain (`thenApply`, `thenCombine`) to combine asynchronous results	Ch 12 §12.4

Term	Plain meaning	Where
Composite	Structural pattern that lets clients treat single objects and trees the same way	Ch 9 §9.3
Composition	"Owns" relationship: the part is created and hidden by the whole and dies with it; filled diamond	Ch 3 §3.4
Composition over inheritance	Reuse behaviour by holding and delegating rather than subclassing	Ch 2 §2.11, Ch 5 §6
Composition root	The one place (usually `main`) where objects are created and wired together	Ch 6 §10
Compound interest	Interest charged on the principal plus interest already added	Ch 21 §21.6
Concurrency	Several tasks in progress at once, possibly sharing data	Ch 12 §12.1
Concurrency vs parallelism	Many tasks in progress (structure) vs many executing at the same instant (hardware)	Ch 12 §12.1
`ConcurrentHashMap`	Thread-safe map with atomic compound operations (`putIfAbsent`, `compute`, `merge`)	Ch 12 §12.16
Condition variable	Lets a thread wait until shared state meets a condition (`wait/notify`, `Condition`)	Ch 12 §12.13
Constructor	Special method that runs when an object is created and sets up its fields	Ch 2 §2.2
Constructor chaining	One constructor calling another with `this(...)`	Ch 2 §2.2
Contention	Many threads competing for the same lock or variable at once, making them wait	Ch 12 §12.7, §12.11
Context switch	The OS pausing one thread and resuming another on a CPU core; costs time	Ch 12 §12.1
Controller (GRASP)	A thin non-UI object that receives each system event and delegates	Ch 5 §16.3
`CopyOnWriteArrayList`	Thread-safe list that copies on every write; good for read-mostly listener lists	Ch 12 §12.16
`CountDownLatch`	One-shot gate that opens after N `countDown()` calls	Ch 12 §12.14
Coupling	How much one class depends on another's details; low is good	Ch 2 §2.16, Ch 4 §4.4
CPU	The processor; each core runs one thread at a time	Ch 12 §12.1
CQRS	Command Query Responsibility Segregation: separate models for writing data and for reading it	Ch 5 §17
Creator (GRASP)	The class that contains, records or has the data for A should create A	Ch 5 §16.2
Critical section	Code that touches shared state and must run in one thread at a time	Ch 12 §12.5
Cron	A schedule format (minute, hour, day…) for running a job at fixed times	Ch 10 §10.6
Cross-cutting concern	Behaviour needed in many places (logging, security, transactions) that doesn't belong to one class	Ch 4 §4.5, Ch 6 §10
CRUD	Create, Read, Update, Delete: the four basic storage operations	Ch 5 §17
CSV	Comma-separated values: a plain-text table, one row per line	Ch 5 §3
`CyclicBarrier`	Reusable meeting point where N threads wait for each other	Ch 12 §12.14
Daemon thread	A background thread that does not keep the JVM alive once all normal threads end	Ch 12 §12.4

Term	Plain meaning	Where
DAO (Data Access Object)	A class that wraps the SQL for one table; closer to the database than a Repository	Ch 11 §11.7.3
Data Class (smell)	A class with only fields and getters/setters, manipulated by others	Ch 7 §7.2.22
Data Clumps (smell)	The same group of values always travelling together	Ch 7 §7.2.10
Database transaction	A group of database changes that succeed or fail together	Ch 12 §12.25, Ch 15
Dead code	Code that is never run or whose result is never used; delete it	Ch 6 §13, Ch 7
Dead-letter queue	Where messages go after retries are exhausted, for later inspection	Ch 20 §20.6
Deadlock	Two or more threads each wait forever for a resource another holds	Ch 12 §12.17
Debit / credit	Taking money out of an account vs putting money in	Ch 14 §14.4, Ch 21 §21.2
Decorator	Structural pattern adding behaviour by wrapping an object behind the same interface	Ch 9 §9.4
Deep vs shallow copy	Copying nested objects too vs sharing them with the original	Ch 8 §8.13
Defensive copy	Copying a mutable object passed in or handed out so outsiders can't change your internals	Ch 2 §2.13
Delegation	An object handing a job to another object it holds, instead of doing it itself	Ch 2 §2.11, Ch 5 §6
Dependency	"Uses" relationship: B appears only as a parameter, local, return type or static call; dashed arrow	Ch 3 §3.5
Dependency injection (DI)	Supplying a class's collaborators from outside (usually via the constructor)	Ch 4 §4.10, Ch 11 §11.7.2
Dependency Inversion Principle	High- and low-level code both depend on abstractions owned by the high-level policy	Ch 4 §4.10
`Deque` / `ArrayDeque`	Double-ended queue: add and remove at both ends; the usual Java stack and queue	Ch 7 §7.7
Derive, don't store	Compute counts and totals from the underlying records so they can't drift	Ch 18 §18.5
Design by Contract	Preconditions (caller's duty), postconditions (method's promise), invariants (always true)	Ch 5 §15
Design pattern	A named, reusable solution to a recurring design problem	Ch 8 §8.1
Director	Optional Builder role that runs a fixed sequence of building steps	Ch 8 §8.12
Distributed lock	A lock shared by several servers, kept in an external store (database, Redis)	Ch 12 §12.25
Divergent Change (smell)	One class changed for many different reasons	Ch 7 §7.2.7
Domain event	Past-tense record of a business fact (`OrderPlaced`) that other parts of the system react to	Ch 5 §17
Domain exception	A custom exception named after a business rule (`InsufficientBalanceException`)	Ch 21 §21.0
Domain-Driven Design (DDD)	Modelling software on the business: bounded contexts, entities, value objects, aggregates, domain events, repositories	Ch 5 §17
Double dispatch	Choosing a method by both the element's and the visitor's type (Visitor's `accept`)	Ch 11 §11.5

Term	Plain meaning	Where
Double-checked locking	Lazy singleton that checks, locks, checks again; the field must be `volatile`	Ch 8 §8.8
Driver (machine coding)	The `main` class that feeds commands to the services and prints results	Ch 1 §1.10, Ch 21 §21.0
DRY	Don't Repeat Yourself: each piece of knowledge has one authoritative home	Ch 5 §1
DTO (Data Transfer Object)	A plain data holder used only to carry values across a boundary (API, layer)	Ch 2 §2.3
Dynamic dispatch	The JVM picks an overridden method from the object's actual class at run time	Ch 2 §2.6–§2.7
Dynamic programming (DP)	Solving a problem by combining saved answers to smaller sub-problems	Ch 14 §14.2
Edge-triggered alert	Fires once when a value crosses a threshold, not on every update while it stays past it	Ch 19 · Grocery Store, Ch 21 §21.7
EMI	Equated monthly instalment: the fixed amount repaid each month on a loan	Ch 21 §21.6
Encapsulate what varies	Isolate the part that changes behind an interface	Ch 5 §8
Encapsulation	Data kept private; only the owning class's methods change it and enforce its rules	Ch 2 §2.3
Entity (DDD)	Object defined by an identity that survives changes; two entities are equal when their ids are equal	Ch 5 §17
Enum (with behaviour)	A fixed set of named constants that can carry fields and methods	Ch 2 §2.10
`EnumMap` / `EnumSet`	Very fast map and set implementations whose keys are the constants of one enum	Ch 2 §2.10
`equals`/`hashCode` contract	Equal objects must have equal hash codes	Ch 2 §2.12
Event Sourcing	Store the stream of changes (events); rebuild state by replaying them	Ch 11 §11.7.6
Event-driven	Parts of the system react to published events instead of calling each other directly	Ch 5 §9
Eventual consistency	Copies of data may disagree for a short time but converge once updates stop	Ch 5 §17
Eviction	Removing entries from a cache to make room (LRU, LFU, FIFO)	Ch 20 §20.3
Exception	An object thrown to signal an error, which unwinds the call stack until something catches it	Ch 6 §6
`ExecutorService`	Runs submitted tasks on a pool of threads; separates what to run from how	Ch 12 §12.4
Exponential backoff	Waiting longer after each failed retry (1 s, 2 s, 4 s…), usually with jitter	Ch 6 §16
Expression problem	Visitor makes adding operations cheap but adding types expensive (and the reverse for plain polymorphism)	Ch 11 §11.5
Facade	Structural pattern giving one simple entry point to a complex subsystem	Ch 9 §9.5
Factory Method	Creational pattern where subclasses decide which product to create	Ch 8 §8.10
Fail fast	Detect problems early and loudly (guard clauses, `requireNonNull`)	Ch 5 §13

Term	Plain meaning	Where
Fairness	Serving waiting threads roughly in order so none starves; costs throughput	Ch 12 §12.19
Fan-out	One event or post copied to many receivers (followers' feeds, subscribers)	Ch 18 §18.4, Ch 20 §20.6
Feature Envy (smell)	A method more interested in another class's data than its own	Ch 7 §7.2.9
Field	A variable that belongs to an object (instance field) or to the class (`static` field)	Ch 2 §2.2
FIFO / LIFO	First in, first out (a queue) vs last in, first out (a stack)	Ch 12 §12.8, Ch 15
F.I.R.S.T.	Clean tests are Fast, Independent, Repeatable, Self-validating, Timely	Ch 6 §8
Flag argument	A boolean parameter that makes a function do two things; split it instead	Ch 6 §2
Flyweight	Structural pattern sharing intrinsic state among many fine-grained objects	Ch 9 §9.6
Fork/Join	Split a task recursively into subtasks run in parallel, then combine results	Ch 12 §12.26, Ch 13 §13.10
Fragile base class	A change in a parent class silently breaks subclasses	Ch 5 §6
Framework	A library that owns the program's flow and calls your code at set points (Spring, JUnit)	Ch 5 §10
Functional interface	An interface with one abstract method, usable with lambdas	Ch 2 §2.8
Functional requirement	A behaviour the system must have ("a customer takes a ticket")	Ch 1 §1.5
GA (general admission)	Unnumbered standing or seating tickets sold as a count, not as named seats	Ch 15 §15.3
Garbage collection (GC)	The JVM automatically freeing memory of objects nothing refers to any more	Ch 12
Generalization	"Is-a" (inheritance) relationship; solid line, hollow triangle at the parent	Ch 3 §3.6
Generics	Type parameters (`List<T>`) for type-safe reusable code	Ch 2 §2.15
Geohash	Encodes a latitude/longitude as a short string; nearby places share a prefix, so it works as a search key	Ch 20 §20.4
Getter / setter	Methods that read (`getX`) or change (`setX`) a field	Ch 2 §2.3
Git	The version-control tool that records every change to the code as commits	Ch 4 §4.2
Glob	Filename wildcard pattern: `*` any characters, `?` one character, `[0-9]` one character from a set	§21.8
Global Data (smell)	Data readable and writable from anywhere, including mutable singletons	Ch 7 §7.2.5
God class / God Object	One class that knows and does almost everything	Ch 8 §8.6, Ch 7 §7.2.20
GoF	"Gang of Four": the four authors of the 1994 book that catalogued the 23 classic design patterns	Ch 8 §8.2
Golden Hammer	Applying a favourite tool or pattern to every problem	Ch 8 §8.6
Graph	Nodes joined by edges (people and connections, stops and routes)	Ch 18 §18.3
GRASP	Larman's nine guidelines for assigning responsibilities to classes	Ch 5 §16

Term	Plain meaning	Where
Greedy algorithm	Always take the best-looking choice right now; fast, but not always correct (e.g. coin change)	Ch 14 §14.2
GST	Goods and Services Tax, India's sales tax	Ch 19 · Grocery Store
Guard clause	An early check-and-return/throw at the top of a method instead of nested `if`s	Ch 7 §7.3.5, Ch 5 §13
Half-open interval	A time range `[start, end)` that includes the start but not the end, so back-to-back bookings don't clash	Ch 15 §15.4, Ch 21 §21.3
Happens-before	Java Memory Model rule guaranteeing one action's effects are visible to another	Ch 12 §12.6
Hash function / hash code	Turns a value into a number used to pick a bucket; equal values must give equal hashes	Ch 2 §2.12
Hash table / `HashMap`	Key-value store that finds an entry in about constant time by hashing the key	Ch 2 §2.12
Heap vs stack (memory)	Where objects live (shared by all threads) vs each thread's own area for local variables and calls	Ch 12 §12.2
Heuristic	A rule of thumb that is usually right and cheap to apply, not a guarantee	Ch 6 §14
Hexagonal architecture	Ports and adapters: the domain core defines interfaces; databases, UIs and vendors plug in from outside	Ch 20 §20.4
High Cohesion (GRASP)	Assign responsibilities so each class does related work	Ch 5 §16.5
HLD vs LLD	Many services and machines vs classes inside one application	Ch 1 §1.2
Hold with TTL	A temporary claim (e.g. seats during payment) that expires automatically	Ch 15 §15.2
Holder-class idiom	Lazy singleton created inside a nested static class; the JVM's class loading makes it thread-safe	Ch 8 §8.8
Hollywood Principle	"Don't call us, we'll call you": the framework calls your hooks	Ch 5 §10
HTTP	The request/response protocol browsers and web services use	Ch 3 §3.12
IDE	Integrated development environment (IntelliJ, Eclipse): editor plus compiler, debugger and refactoring tools	Ch 7
Idempotency	Running an operation twice has the same effect as once; an idempotency key enforces it	Ch 21 §21.2, Ch 20 §20.6
Identity vs equality	Same object in memory (`==`) vs same value (`equals`)	Ch 2 §2.12
Immutability	Objects that never change after creation; safe to share between threads	Ch 2 §2.13, Ch 5 §14
Indirection (GRASP)	Put an intermediate object between two things that shouldn't know each other	Ch 5 §16.8
Information Expert (GRASP)	The class that has the data does the work	Ch 5 §16.1
Inheritance	A child class reuses and specialises a parent (`extends`); only for true is-a	Ch 2 §2.5
In-memory	Data kept only in the program's RAM, lost when it stops; the default in LLD rounds	Ch 1 §1.4
Insider Trading (smell)	Two modules exchanging too much private information	Ch 7 §7.2.19
Instance	One object created from a class	Ch 2 §2.2

Term	Plain meaning	Where
Integration test	Tests several real parts together (e.g. service plus database), slower than a unit test	Ch 6 §17
Interface	A contract of methods; a class can implement many	Ch 2 §2.8
Interface Segregation Principle	No client should depend on methods it does not use	Ch 4 §4.9
Interning	Keeping one shared copy of each equal value (`String.intern()`)	Ch 9 §9.6
Interpreter	Behavioral pattern with one class per grammar rule, evaluating sentences of a small language	Ch 10 §10.6
Interrupt	A polite request to a thread to stop what it is doing; blocking calls react by throwing `InterruptedException`	Ch 12 §12.3
Intrinsic vs extrinsic state	Flyweight's shared, immutable part vs the per-use part passed in by the caller	Ch 9 §9.6
Invariant	A fact about an object that must always be true ("balance is never negative")	Ch 5 (key terms), §15
Inversion of Control (IoC)	The framework controls the flow and calls your code; DI is one way to achieve it	Ch 5 §10, Ch 4 §4.14
Inverted index	A map from each word/tag to the list of items containing it, for search	Ch 18 §18.1
Invoker / Receiver	Command roles: the invoker triggers commands; the receiver does the real work	Ch 10 §10.2
I/O	Input/output: reading or writing files, network or database; slow compared with CPU work	Ch 12 §12.4
Iterator	Behavioral pattern for traversing a collection without exposing its storage	Ch 10 §10.3
JAR	Java archive: a zip file of compiled classes and resources	Ch 4 §4.7
Java Memory Model (JMM)	Rules for when one thread's writes become visible to another	Ch 12 §12.6
`java.time`	Java's date-time API: `Instant` (a moment), `Duration`, `LocalDate`, and `Clock` (injectable "now")	Ch 1 §1.10
JDBC	Java's standard API for talking to relational databases	Ch 8 §8.11
JDK	Java Development Kit: compiler, tools and the standard library	Ch 2
JIT compiler	Part of the JVM that turns hot bytecode into fast machine code while the program runs	Ch 12
Jitter	Random extra delay added to retries so many clients don't retry at the same instant	Ch 6 §16, Ch 12 §12.18
JPA / Hibernate	Java's standard for mapping objects to database tables, and its most popular implementation	Ch 9 §9.7
JSON	A text format for structured data: objects in `{}`, lists in `[]`	Ch 3 §3.2
JUnit	The standard Java unit-testing framework	Ch 6 §8
JVM	Java Virtual Machine: runs compiled Java bytecode and manages memory and threads	Ch 2, Ch 12
Kafka	A distributed append-only log used as a durable message broker	Ch 20 §20.6
KISS	Keep It Simple: choose the simplest design that works	Ch 5 §2
KYC	Know Your Customer: identity checks a financial app must do before allowing higher limits	Ch 21 §21.2

Term	Plain meaning	Where
Lambda	A short anonymous function, e.g. `x -> x * 2`, usable wherever a functional interface is expected	Ch 2 §2.8
Large Class (smell)	A class trying to do too much; a God Class	Ch 7 §7.2.20
Latency	How long one operation takes from start to finish	Ch 1 §1.4
Law of Demeter	Only talk to your immediate friends; avoid `a.getB().getC()` chains	Ch 5 §4
Lazy Element (smell)	A class or function that doesn't pull its weight	Ch 7 §7.2.14
Lazy expiry	Deleting an expired entry only when someone next touches it, instead of by a timer	Ch 13 §13.6, Ch 14 §14.7
Lazy initialization	Creating an object only when it is first needed	Ch 8 §8.8
Lease	A lock or hold that expires automatically after a set time unless renewed	Ch 12 §12.9
Least Astonishment	Code should do what its name promises and nothing more	Ch 5 §12
Ledger	An append-only list of money movements from which balances are derived	Ch 20 §20.1, Ch 21
Legacy code	Existing code, often without tests, that is risky to change	Ch 7 §7.8
LFU / MRU	Cache eviction that removes the least frequently used vs the most recently used entry	Ch 20 §20.3
Linked list	A chain of nodes where each points to the next (doubly linked: also the previous)	Ch 20 §20.3
`LinkedHashMap`	A `HashMap` that also remembers insertion or access order; the shortcut for an LRU cache	Ch 20 §20.3
Linter / static analysis	Tools that read code without running it and flag likely bugs or style issues	Ch 6 §15
Liskov Substitution Principle	A subtype must be usable anywhere its parent is expected, with no surprises	Ch 4 §4.8
Listener	An object registered to be told when an event happens (the observer in Observer)	Ch 11 §11.1
Livelock	Threads keep reacting to each other and running, but none makes progress	Ch 12 §12.18
Lock ordering	Always acquire locks in one global order to prevent circular wait	Ch 12 §12.17
Lock striping	Guarding a map with an array of locks, each covering some buckets	Ch 13 §13.7
Lombok	A library that generates boilerplate (getters, builders) from annotations at compile time	Ch 8 §8.12
Long Function (smell)	A function too long to understand at a glance	Ch 7 §7.2.3
Long Parameter List (smell)	Many parameters, often of the same type or boolean flags	Ch 7 §7.2.4
`LongAdder`	Counter that scales better than `AtomicLong` under heavy contention	Ch 12 §12.15
LOOK scheduling	Elevator keeps moving in one direction while stops remain ahead, then reverses	Ch 14 §14.5
Loops (smell)	A classic loop where a stream pipeline would state the intent more clearly	Ch 7 §7.2.13
Loose coupling	Interacting objects know as little about each other as possible	Ch 5 §9

Term	Plain meaning	Where
Lost wakeup	A signal sent before the waiter started waiting, so the waiter sleeps forever	Ch 12 §12.13
Low Coupling (GRASP)	Assign responsibilities to keep dependencies few and stable	Ch 5 §16.4
LRU cache	Evicts the least recently used entry; hash map + doubly linked list gives O(1)	Ch 20 §20.3
Lua	A small scripting language; Redis runs a Lua script as one atomic step	Ch 20 §20.7
Machine-coding round	LLD round where working, runnable in-memory code is the answer (90–120 min)	Ch 1 §1.2, Ch 21 §21.0
Market vs limit order	Buy or sell now at the best available price vs only at a given price or better	Ch 19 · Stock Brokerage
Matching engine	Per-symbol order book matching buy and sell orders by best price, then oldest first	Ch 19 · Stock Brokerage
MDC	Mapped Diagnostic Context: per-thread key-values (request id, user) automatically added to log lines	Ch 20 §20.5
Mediator	Behavioral pattern centralising communication between peers in one coordinator	Ch 10 §10.4
Memento	Behavioral pattern capturing and restoring state without exposing internals	Ch 10 §10.5
Memory leak	Memory that is never freed because something still holds references to objects no longer needed	Ch 11 §11.1, Ch 12 §12.24
Message Chains (smell)	`a.getB().getC().getD()` navigation	Ch 7 §7.2.17
Message queue	A buffer service where producers leave messages for consumers to process later	Ch 20 §20.6
Method	A function that belongs to a class	Ch 2 §2.2
Method reference	Shorthand for a lambda that just calls a method, e.g. `String::length`	Ch 2 §2.8
Microservice vs monolith	Many small independently deployed services vs one deployable application	Ch 5 §17
Middle Man (smell)	A class whose methods mostly forward to another class	Ch 7 §7.2.18
Mixin	A small interface with `default` methods that adds a capability to many classes	Ch 4 §4.3
Mock / stub / fake	Test doubles: a mock checks calls were made, a stub returns canned answers, a fake is a simple working version	Ch 6 §17
Mockito	The most common Java library for creating mocks in tests	Ch 6 §8
Module / package	A group of related classes; in Java a package is a folder-level namespace	Ch 2 §2.14
Monitor	The lock built into every Java object, used by `synchronized` and `wait/notify`	Ch 12 §12.7
Multiplicity	How many objects take part at each end of a relationship (`1`, `0..1`, `*`, `1..*`)	Ch 3 §3.2
Mutable Data (smell)	Shared data changed after creation, causing surprises elsewhere	Ch 7 §7.2.6
Mutation testing	Deliberately breaking the code in small ways to check the tests notice	Ch 6 §17
Mutex	A lock that lets only one thread into a critical section	Ch 12 §12.7

Term	Plain meaning	Where
MVC	Model–View–Controller; a compound of Observer, Strategy and Composite	Ch 11 §11.7.5
Mysterious Name (smell)	A name that doesn't say what the thing does or holds	Ch 7 §7.2.1
Non-functional requirement	A quality: thread safety, extensibility, in-memory storage, latency	Ch 1 §1.5
Null Object	A do-nothing implementation used instead of `null`	Ch 11 §11.7.1
`NullPointerException` (NPE)	Thrown when you call a method or read a field through a `null` reference	Ch 5 §13
Object	One instance of a class with real values	Ch 2 §2.2
Object Pool	Borrow and return a bounded set of expensive objects	Ch 8 §8.14
Observer	Behavioral pattern notifying many dependents when one object changes	Ch 11 §11.1
Offset	A position number in a log or list; each reader remembers its own	Ch 20 §20.6
OOA / OOD / OOP	Analysis (what) → design (how, with objects) → programming (code)	Ch 2 §2.17
Open/Closed Principle	Open for extension, closed for modification: add code, don't edit working code	Ch 4 §4.7
Optimistic locking	Read with a version; the write succeeds only if the version is unchanged	Ch 12 §12.10, §12.25
`Optional`	A container that may be empty; return it when "not found" is normal	Ch 1 §1.10
Order book	Per-stock lists of waiting buy and sell orders, sorted by price then time	Ch 19 · Stock Brokerage
OTP	One-time password: a short code valid for a single use, usually sent by SMS	Ch 5 §13, Ch 20 §20.2
Outbox (transactional)	Save events in a table in the same transaction as the data, and publish them afterwards, so neither is lost	Ch 5 §17
`OutOfMemoryError`	Thrown when the JVM cannot find memory for a new object	Ch 12 §12.2
Out-of-scope list	Written list of what you will not build; shrinks the problem	Ch 1 §1.5
Overloading	Same method name, different parameters; chosen at compile time	Ch 2 §2.7
Overriding	Subclass replaces a parent's method; chosen at run time	Ch 2 §2.7
Payment gateway	An external service that actually charges cards or UPI; hidden behind your own interface	Ch 15 §15.2, Ch 19
PECS	Producer `extends`, consumer `super` — the rule for generic wildcards	Ch 2 §2.15
Pessimistic locking	Lock a row before changing it (e.g. `SELECT … FOR UPDATE`) so nobody else can touch it meanwhile	Ch 12 §12.5, Ch 15 §15.2
Phaser	A reusable, flexible barrier for threads meeting in phases	Ch 12 §12.14, §12.26
PII	Personally identifiable information (names, phone numbers) that must not leak into logs	Ch 6 §15
Pinning	A virtual thread stuck on its carrier thread while it blocks	Ch 12 §12.4
Pipeline	Data flowing through a sequence of steps, each feeding the next (streams, filters, middleware)	Ch 7 §7.2.13

Term	Plain meaning	Where
PLU code	Four-digit price look-up code for loose produce, keyed at the till together with the scale weight	Ch 19 · Grocery Store
Plugin	A piece of code added to a system through a known interface without editing the system	Ch 4 §4.7
PNR	Passenger name record: the booking reference for an airline reservation	Ch 16 · Airline
Poison pill	A special message telling a consumer to stop; one per consumer	Ch 12 §12.22
Polymorphism	One call behaves differently depending on the object; removes type switches	Ch 2 §2.6, Ch 5 §16.6
POS	Point of sale: the till where a sale is rung up and paid	Ch 19 · Grocery Store
Precondition / postcondition	What must be true before a call (caller's duty) / what the method promises after it	Ch 4 §4.8, Ch 5 §15
Primitive Obsession (smell)	Using `double`, `String`, `int` for domain concepts like money or status	Ch 7 §7.2.11
`PriorityQueue`	A queue that always hands out the smallest (or highest-priority) element first; a binary heap inside	Ch 16 · Course Registration
Process	A running program with its own memory; it contains one or more threads	Ch 12 §12.2
Producer–consumer	Threads hand work through a (bounded) queue	Ch 12 §12.22
Program to an interface	Depend on supertypes; pick the concrete class once, at the edge	Ch 5 §7
Protected Variations (GRASP)	Wrap points of instability behind a stable interface	Ch 5 §16.9
Prototype	Creational pattern creating new objects by copying a configured one	Ch 8 §8.13
Proxy	Structural pattern: a stand-in controlling access to another object	Ch 9 §9.7
Pub-sub	Observer through a broker: publishers and subscribers don't know each other	Ch 20 §20.6
Pure Fabrication (GRASP)	An invented helper class (e.g. a repository) when no domain class fits	Ch 5 §16.7
Pure function	A function whose result depends only on its inputs and which changes nothing else	Ch 11 §11.7.6, Ch 17 · 2048
Qualified association	An association looked up by a key (`Library [isbn] → Book`, i.e. a `Map`)	Ch 3 §3.11
Race condition	The result depends on the timing of threads touching shared mutable state	Ch 12 §12.5
Rate limiter	Caps how many requests a key may make per time window (token bucket, windows)	Ch 13 §13.11, Ch 20 §20.7
Read-write lock	Many readers at once, or one writer	Ch 12 §12.10
Realization	A class implements an interface; dashed line, hollow triangle at the interface	Ch 3 §3.7
Record	Compact Java class for immutable data; only shallowly immutable	Ch 2 §2.13
Recursion	A function that solves a problem by calling itself on a smaller part	Ch 9 §9.3

Term	Plain meaning	Where
Recursive-descent parser	A parser with one method per grammar rule, each calling the others	Ch 10 §10.6, Ch 21 §21.8
Redis	A fast shared in-memory key-value store, often used for caches, counters and distributed locks	Ch 20 §20.7
Reentrancy	A thread may re-acquire a lock it already holds	Ch 12 §12.8
`ReentrantLock`	Explicit lock with try/timed/fair locking and conditions	Ch 12 §12.8
Refactoring	Changing internal structure without changing observable behaviour	Ch 7 §7.1
Reflection	Code inspecting or calling classes, fields and methods at run time by name	Ch 8 §8.8
Refused Bequest (smell)	A subclass that doesn't want, or throws on, inherited methods	Ch 7 §7.2.23
Regex	Regular expression: a pattern language for matching text	Ch 10 §10.6
Repeated Switches (smell)	The same `switch` on a type code in several places	Ch 7 §7.2.12
Repository	Collection-like interface that hides storage from business logic	Ch 11 §11.7.3
Requirement	One thing the system must do, written so you can check it	Ch 1 §1.5
REST	A style of HTTP API built around resources (URLs) and verbs (GET, POST…)	Ch 5 §1
Retry	Trying a failed operation again; safe only if the operation is idempotent	Ch 6 §16
Role interface	A small interface serving one kind of caller (the ISP fix)	Ch 4 §4.9
Rule of three	Refactor to an abstraction at the second or third case, not the first	Ch 4 §4.15
`RuntimeException`	Base class of unchecked exceptions	Ch 2 §2.5
Saga	A long business process split into local steps, each with a compensating action if a later step fails	Ch 3 §3.12, Ch 15
Scalability	How well a system keeps working as users, data or load grow	Ch 1 §1.14
`ScheduledExecutorService`	Runs tasks after a delay or repeatedly at a fixed rate	Ch 13 §13.6
Schema	The defined structure of stored data (tables, columns, types)	Ch 2 §2.17
SDK	Software development kit: a vendor's ready-made library for calling its service	Ch 9 §9.1
Sealed type	A type whose permitted subtypes are fixed, enabling exhaustive `switch`	Ch 11 §11.5
Seam	A place where you can swap behaviour (e.g. an interface) without editing the code around it; used for testing	Ch 2 §2.9, Ch 7 §7.7
Seed	The starting number for a pseudo-random generator; the same seed gives the same sequence	Ch 17
Semaphore	A counter of permits that limits concurrent access or signals between threads	Ch 12 §12.12
Separation of Concerns	Each part of the program handles one concern	Ch 4 §4.5, Ch 5 §11
Sequence diagram	UML diagram of messages between objects over time	Ch 3 §3.12
Serialization	Turning an object into bytes or text (and back) for storing or sending	Ch 8 §8.8
Service Locator	Anti-pattern where a class asks a registry for its dependencies, hiding them	Ch 11 §11.7.2

Term	Plain meaning	Where
Sharding	Splitting data across independent parts (each with its own lock) to scale	Ch 20 §20.3
Shotgun Surgery (smell)	One change forces many small edits across many classes	Ch 7 §7.2.8
Side effect	Anything a function does besides returning a value (changing a field, writing a file)	Ch 5 §12, Ch 6 §2
Signaling	One thread tells another that something has happened (semaphore, latch)	Ch 12 §12.20
Simple Factory	One method mapping a type value to a class; an idiom, not GoF	Ch 8 §8.9
Single Responsibility Principle	A class should have one reason (one actor) to change	Ch 4 §4.2
Singleton	Creational pattern ensuring exactly one instance with global access	Ch 8 §8.8
SKU	Stock-keeping unit: one sellable product variant tracked in inventory	Ch 21 §21.7
SLF4J	A logging facade: code logs through its API and any logging library can sit underneath	Ch 9 §9.5
Sliding window	Rate-limit algorithm counting requests in a moving time window (log or counter)	Ch 20 §20.7
SMS / SMTP	Text messages to phones / the protocol for sending email	Ch 20 §20.2
Snapshot	A copy of state at one moment, used to restore it or to read it safely	Ch 10 §10.5
SOLID	SRP, OCP, LSP, ISP, DIP: five guidelines that keep one change from spreading	Ch 4 §4.1
Specification	Named, combinable business-rule object with `isSatisfiedBy`	Ch 11 §11.7.4
Speculative Generality (smell)	Hooks and abstractions added "in case" but never used	Ch 7 §7.2.15
Split tender	Paying one bill with several payment methods (e.g. part UPI, part cash)	Ch 19 · Grocery Store
Spring	The most widely used Java application framework; creates and wires objects (DI), web, data access	Ch 4 §4.14, Ch 11 §11.7.2
Spurious wakeup	A waiting thread wakes with no signal; why `wait()` sits in a `while` loop	Ch 12 §12.13
SQL	The language for querying and updating relational databases	Ch 11 §11.7.3
Stack trace	The list of method calls active when an exception was thrown; shows where it came from	Ch 6 §6
`StampedLock`	Read-write lock with optimistic reads; not reentrant	Ch 12 §12.10
Starvation	One thread never gets the resource while others keep progressing	Ch 12 §12.19
State (pattern)	Behavioral pattern where an object's behaviour changes with its internal state	Ch 11 §11.2
State machine diagram	UML diagram of one object's states and transitions (`event [guard] / action`)	Ch 3 §3.14
Stateless vs stateful	Keeps no data between calls (safe to share) vs remembers data between calls	Ch 1 §1.6
`static`	Belongs to the class, one per class, not to each object	Ch 2 §2.2

Term	Plain meaning	Where
Stop-loss / stop-limit	Orders that wake up when the price hits a trigger, then sell at market / at a limit price	Ch 19 · Stock Brokerage
Strategy	Behavioral pattern making algorithms interchangeable behind one interface	Ch 11 §11.3
Stream (Java)	A pipeline over a collection (`filter`, `map`, `collect`) that states what, not how	Ch 2 §2.8, Ch 7 §7.2.13
Subclass / superclass	The child class that `extends` / the parent it extends	Ch 2 §2.5
`synchronized`	Java keyword that runs a block or method under an object's monitor: one thread at a time, with visibility	Ch 12 §12.7
TCP	The internet protocol that delivers a reliable, ordered byte stream between two machines	Ch 11 §11.2
Technical debt	Shortcuts in code that make future changes slower until someone cleans them up	Ch 7 §7.1
Telescoping constructors	A ladder of constructors each adding one more parameter; Builder replaces it	Ch 8 §8.12
Tell, Don't Ask	Tell objects what to do instead of pulling their data out to decide	Ch 5 §5
Template Method	Behavioral pattern: fixed algorithm skeleton in a base class, steps in subclasses	Ch 11 §11.4
Temporary Field (smell)	A field set only in some situations and empty otherwise	Ch 7 §7.2.16
Test double	Anything standing in for a real dependency in a test: dummy, stub, spy, mock, fake	Ch 6 §17
Test-Driven Development (TDD)	Failing test → make it pass → refactor, in very short cycles	Ch 6 §8
Thread	An independent path of execution inside a process, sharing its memory	Ch 12 §12.2
Thread confinement	Keeping data inside one thread so it never needs locking	Ch 12 §12.24
Thread dump	A snapshot of every thread's state and call stack (`jstack`, `jcmd`) used to diagnose hangs	Ch 12 §12.3
Thread pool	A fixed set of reusable threads fed from a task queue	Ch 12 §12.21
Thread states	NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED	Ch 12 §12.3
Thread-safe	Gives correct results even when many threads call it at the same time	Ch 12 §12.5
Thread-starvation deadlock	Tasks in a bounded pool wait for tasks queued behind them in the same pool	Ch 12 §12.17, §12.21
Throughput	How much work gets done per unit of time (requests per second)	Ch 12 §12.11
Timeout	A limit on how long to wait before giving up	Ch 12 §12.9
TL;DR	"Too long; didn't read": a short summary at the end of each section or chapter	Every chapter
Token bucket	Rate-limit algorithm: tokens refill at a fixed rate; each request takes one; allows bursts	Ch 13 §13.11
`toString`	Method returning a readable text form of an object, used in logs and printing	Ch 2 §2.12
`TreeMap` / `TreeSet`	Sorted map and set (red-black tree inside); support "next higher" and range queries	Ch 14 §14.5

Term	Plain meaning	Where
`tryLock`	Acquire a lock only if free (or within a timeout); otherwise give up	Ch 12 §12.9
TTL (time to live)	How long an entry or hold stays valid before it expires	Ch 13 §13.6, Ch 15 §15.2
Two hats	While coding you are either adding function or refactoring, never both at once	Ch 7 §7.1.1
Type erasure	Java removes generic type arguments after compiling, so `List<String>` and `List<Integer>` are the same class at run time	Ch 2 §2.15
Ubiquitous language	One vocabulary shared by domain experts and the code (`place()`, not `setStatus(2)`)	Ch 5 §17
UI	User interface: the screens or controls a person interacts with	Ch 11 §11.7.5
UML	Unified Modeling Language: 14 diagram types, 7 structural and 7 behavioural	Ch 3 §3.9
Unit test	A fast test of one small piece of code in isolation	Ch 6 §8
UPI	Unified Payments Interface: India's instant bank-to-bank mobile payment system	Ch 19, Ch 21 §21.2
Use case	One thing an actor can do with the system	Ch 1 §1.5, Ch 3 §3.10
Use-case diagram	UML diagram of actors, use cases and the system boundary («include», «extend»)	Ch 3 §3.10
UTC	Coordinated Universal Time: the time-zone-free reference clock for storing timestamps	Ch 16 · Airline
Value object	A small immutable object defined by its values (money, date range), usually a record	Ch 2 §2.13, Ch 5 §1, Ch 5 §17
Virtual threads	Lightweight JVM-managed threads for large numbers of blocking tasks (Java 21)	Ch 12 §12.4
Virtual waiting room	FIFO queue that admits buyers gradually during a ticket rush	Ch 15 §15.3
Visibility (memory)	Whether a write by one thread can be seen by another; not guaranteed without `volatile`, locks or other happens-before edges	Ch 12 §12.6
Visitor	Behavioral pattern adding operations to a stable set of classes via double dispatch	Ch 11 §11.5
`volatile`	Makes writes to a field visible to other threads; does not make `x++` atomic	Ch 12 §12.6
Webhook	A URL a service calls to notify you that something happened (e.g. "payment succeeded")	Ch 15 §15.2
Wildcard (generics)	`?` in a generic type: `? extends T` (read from it), `? super T` (write to it)	Ch 2 §2.15
Work stealing	Idle threads in a fork/join pool take queued tasks from busy threads	Ch 12 §12.26
Wrapper	An object that holds another and adds or changes behaviour around it (decorators, adapters, boxed primitives)	Ch 9 §9.4
Write-through / write-back	Cache writes go to storage immediately vs later in batches	Ch 20 §20.3
YAGNI	You Aren't Gonna Need It: don't build features before they are needed	Ch 5 §3

Self-check

- **Q:** Name the eleven framework steps in order. **A:** Clarify, requirements, use cases, entities, relationships, class diagram, key flows, patterns & SOLID, code, edge cases & concurrency, extensions.
- **Q:** Where does the diamond go, and what does filled mean? **A:** At the whole; filled = composition (the part dies with the whole).
- **Q:** A `switch (vehicleType)` computes fees. Which principle, smell and pattern? **A:** OCP; Repeated Switches (if it repeats); Strategy.
- **Q:** Decorator vs Proxy vs Chain of Responsibility? **A:** Decorator adds behaviour and every layer runs; Proxy controls access; in CoR one handler can stop the flow.
- **Q:** State vs Strategy? **A:** States know each other and advance themselves; strategies are chosen from outside.
- **Q:** Four Coffman conditions, and the usual fix? **A:** Mutual exclusion, hold and wait, no preemption, circular wait; global lock ordering.
- **Q:** Why is `volatile` not enough for `count++`? **A:** Visibility, not atomicity; use `AtomicInteger` or a lock.
- **Q:** The six machine-coding packages? **A:** model, repository, service, strategy, exception, driver.
- **Q:** What makes every booking problem safe? **A:** Naming the real unit of inventory and making "if free then take" one atomic step (plus holds that expire).

TL;DR

- **Framework:** clarify → requirements → use cases → entities → relationships → diagram → flows → patterns/SOLID → code → edge cases/concurrency → extensions.
- **Classes:** different data → field; different behaviour → class; one sentence per class; composition over inheritance.
- **SOLID** diagnoses change pain; always state the cost of the fix, and know when not to apply it.
- **Patterns** come from signals: Strategy (rules vary), Factory (type value → class), Observer (many listeners), State (lifecycle), Singleton (exactly one), Builder (many optional fields). Use one or two, name a rejected one.
- **Concurrency:** find the shared state, state the invariant, make check-then-act atomic with the smallest tool, lock in a fixed order, notify outside locks.
- **Machine coding:** run early and often, mandatory first, services return data, the driver prints and keeps going.

Chapter 23 — Interview Question Bank

How to use this bank

This chapter collects the low-level design (LLD) and machine-coding questions that come up again and again in interviews, worded the way they usually appear on LeetCode Discuss interview-experience posts and in the popular public question lists (see the sources at the end). Use it as a checklist so that nothing surprises you.

- ⋆ marks a question that is asked very often. Do those first.
- **Part A** is the "Design X" prompts (parking lot, BookMyShow, Splitwise …). When this book has a full case study, the entry is a short reminder of what to say plus a link to the full design. When it doesn't, the entry is a complete outline you can build from.
- **Part B** is the concept questions (OOP, UML, SOLID, patterns, concurrency, Java specifics). Each has a short answer you can say out loud and a link to the full explanation.
- **Part C** is a minute-by-minute plan for a 60–90 minute machine-coding round.

How to drill: read the question, cover the answer, say your answer out loud (or sketch the classes), then compare. Mark the ones you stumbled on and come back to them the next day.

Part A — "Design X" (LLD and machine coding) questions

These are the "Design X" prompts that show up again and again in object-oriented design (OOD) rounds and in machine-coding rounds. An OOD round is a whiteboard-style talk about classes; a machine-coding round is 90–150 minutes of writing working code. ⋆ marks prompts asked very often, and every machine-coding favourite also gets a ⋆. When this book has a full case study, the entry here is a short reminder of what to say, followed by a link. When the book has no case study, the entry is a complete outline you can build from.

Everyday objects

Q1. ⋆ Design a Parking Lot

Asked at: Flipkart (SDE2 LLD round)

- Entities: `ParkingLot` → `Level` → `ParkingSpot` (sized: small, compact, large), `Vehicle`, `Ticket`, `EntryGate` / `ExitGate`, `FeeCalculator`.
- Patterns: Strategy for spot allocation and for pricing; Singleton (or one injected instance) for the lot; Factory for vehicles.
- What interviewers probe: two cars racing for the last free spot (claim the spot atomically), hourly vs flat pricing, and adding EV charging spots without editing old code. → *Full design:* Ch 14 — 14.1 Design a Parking Lot

Q2. ⋆ Design an Elevator / Lift System

- Entities: `ElevatorController`, `Elevator` (current floor, direction, door state), `Request` (hall call or car call), `Dispatcher`.
- Patterns: State (moving up, moving down, idle, doors open), Strategy for dispatching (nearest car, SCAN / LOOK).
- What interviewers probe: the scheduling algorithm (why plain first-come-first-served makes cars zig-zag), keeping requests consistent when many buttons are pressed at once, and overload or maintenance modes. → *Full design:* Ch 14 — 14.5 Design an Elevator System

Q3. ⋆ Design a Vending Machine

- Entities: `VendingMachine`, `Inventory`, `Product`, `Coin` / `Note`, `ChangeDispenser`.
- Pattern: State (idle, has money, dispensing, sold out) is the whole point of this question.
- What interviewers probe: making change with a limited coin stock, refunding on cancel, and what happens when a product sells out while money is inserted. → *Full design:* Ch 14 — 14.2 Design a Vending Machine

Q4. Design a Coffee Vending Machine

- Entities: `CoffeeMachine`, `Recipe` (ingredient → quantity), `IngredientStore`, `Beverage`, add-ons.
- Patterns: Decorator for add-ons (extra shot, milk), Factory for drinks, State for the machine.
- What interviewers probe: checking and deducting every ingredient as one atomic step, and refill alerts (Observer). → *Full design:* Ch 14 — 14.3 Design a Coffee Vending Machine

Q5. ⋆ Design an ATM

- Entities: `ATM`, `Card`, `Account`, `BankService`, `CashDispenser`, `Transaction` (withdraw, deposit, balance enquiry).
- Patterns: State (no card, card inserted, PIN verified), Chain of Responsibility for dispensing notes by denomination.
- What interviewers probe: what to do if cash is debited but not dispensed (reversal), PIN retry limits, and two ATMs withdrawing from one account at the same time. → *Full design:* Ch 14 — 14.4 Design an ATM

Q6. Design a Traffic Signal Control System

- Entities: `Intersection`, `TrafficLight`, `SignalPhase`, `SignalController`, `Timer`.
- Patterns: State for the light cycle, Strategy for timing plans (fixed, rush hour, sensor-driven), Observer for emergency vehicles.
- What interviewers probe: never letting conflicting directions be green together, and handling emergency override safely. → *Full design:* Ch 14 — 14.6 Design a Traffic Signal Control System

Q7. Design an Amazon Locker / shipping locker

- Entities: `LockerLocation`, `Locker` (by size), `Package`, `PickupCode`, `LockerService`.
- Patterns: Strategy for choosing a locker size, State for the locker (free, reserved, occupied).
- What interviewers probe: code expiry and returning the package to the sender, and two deliveries claiming the same locker. → *Full design:* Ch 14 — 14.7 Design a Shipping Locker System

Q8. Design a toll booth / parking-fee calculator (pricing strategies)

- This is the pricing part of the parking lot pulled out on its own: a `FeeCalculator` interface with hourly, flat, slab and weekend implementations, chosen by vehicle type and time.
- Patterns: Strategy (one class per pricing rule), optionally Decorator for surcharges and discounts layered on top.
- What interviewers probe: time zones and midnight boundaries, rounding (charge per started hour?), and keeping money as integer paise or `BigDecimal`, never `double`. → *Full design:* Ch 14 — 14.1 Design a Parking Lot (pricing section)

Q9. Design a home automation system

Requirements. A user owns devices (lights, fans, thermostats, door locks, sensors) made by different vendors. The user can switch a device on or off, set values (brightness, temperature), group actions into a "scene" ("Good night" turns off lights and locks doors), undo the last action, and create rules ("if motion after 23:00, turn on the hall light"). Schedules ("at 07:00") are nice to have.

Main classes.

- `Device` (interface): `id()`, `status()`. Subtypes `Light`, `Thermostat`, `Lock`, `MotionSensor`.
- `VendorAdapter`: wraps each vendor's protocol (Zigbee, Wi-Fi API) behind the `Device` interface.
- `DeviceCommand`: one action with `execute()` and `undo()`, for example `SetBrightness(light, 40)` remembers the old brightness.
- `Scene`: a list of commands run as one; it is itself a command.
- `Hub`: registry of devices, runs commands, keeps an undo stack.
- `Rule`: a trigger (sensor event, time) plus a condition plus a command.
- `EventBus`: sensors publish events; rules subscribe.

Patterns. Adapter (vendor protocols), Command (actions with undo), Composite (a scene is a command made of commands), Observer (sensor events reach rules), Strategy for the condition check.

Concurrency and edge cases. Send commands to one device in order: give each device its own single-threaded executor (or a lock), while different devices run in parallel on virtual threads. A device may be offline: queue the command with a timeout and report failure instead of hanging. Avoid rule loops (rule A turns on a light, which fires rule B, which turns it off) by tracking which rule caused an event and limiting the depth.

Extension question. "Support user roles: a guest can unlock the front door only between 10:00 and 18:00." Add an `AccessPolicy` checked by the `Hub` before running any command (a Proxy around devices works well).

Q10. Design a Jackpot / slot machine

Requirements. A player inserts credit, places a bet, and pulls the lever. Three (or five) reels spin and stop on symbols. A pay table decides the win. The machine tracks credit, pays out, and refuses to play when it is out of service or cannot cover the payout. Optional: a progressive jackpot shared by many machines.

Main classes.

- `SlotMachine`: holds the credit balance and the current state, and exposes `insert`, `bet`, `spin` and `cashOut`.
- `MachineState` (sealed interface): `Idle`, `HasCredit`, `Spinning`, `OutOfService`. Each state allows only its own actions.
- `Reel`: a list of symbols with weights, so a cherry can be more common than a seven.
- `RandomSource`: an interface wrapping `SecureRandom`. Tests swap in a fixed sequence, so results are repeatable.
- `PayTable`: maps a symbol combination to a multiplier. A Strategy, so different machines can pay differently.
- `JackpotPool`: one shared progressive pool. Each bet adds a small cut.

Patterns. State (the machine lifecycle), Strategy (pay rules, random source), Singleton-like shared `JackpotPool` (inject one instance rather than a static).

Concurrency and edge cases. Two machines can hit the jackpot at the same moment. Take the pool with one atomic step, `AtomicLong.getAndSet(seed)`, so only one machine wins the full amount. Use whole numbers for money. Reject a bet bigger than the credit. Go out of service if the machine's cash cannot cover the largest possible win. Log every spin (seed, result) for audits.

Extension question. "Add a bonus round after three scatter symbols." Add a `BonusRound` state that the machine moves into after the normal payout. No existing state class needs to change.

Q11. Design a music playlist

- Entities: `Song`, `Playlist` (ordered list), `Player` (current position, play mode), `User`.
- Patterns: Iterator (sequential, shuffle, repeat as different iterators) or Strategy for play order.
- What interviewers probe: shuffle without repeats until all songs played, and editing the playlist while it is playing. → *Full design:* Ch 1 — 1.3 A complete answer in five minutes

Booking and reservations

Q12. ⋆ Design BookMyShow / a movie ticket booking system

Asked at: Flipkart

- Entities: `Cinema` → `Screen` → `Seat`, `Movie`, `Show`, `ShowSeat` (seat state per show), `Booking`, `Payment`.
- Patterns: State for a seat (available, held, booked), Strategy for pricing, Observer for confirmations.
- What interviewers probe: two users picking the same seat (hold with expiry, then confirm), releasing holds when payment fails or times out. → *Full design:* Ch 15 — 15.2 Design a Movie Ticket Booking System

Q13. ⋆ Design a Library Management System

- Entities: `Book` (title info) vs `BookItem` (physical copy), `Member`, `Loan`, `Reservation`, `Fine`, `Catalog` (search).
- Patterns: Strategy for fine rules, Observer for "your reserved book is back".
- What interviewers probe: the book vs copy split, borrowing limits, and the reservation queue. → *Full design:* Ch 15 — 15.1 Design a Library Management System

Q14. ⋆ Design a Hotel Management System

- Entities: `Hotel`, `Room` (type, rate), `Guest`, `Reservation` (date range), `Invoice`, `HousekeepingTask`.
- Patterns: State for reservation and room status, Strategy for room pricing.
- What interviewers probe: date-range overlap checks, overbooking rules, and late check-out charges. → *Full design:* Ch 15 — 15.4 Design a Hotel Management System

Q15. Design a concert ticket booking system

- Like BookMyShow, but one big event with a sudden rush: seat sections, general admission, and a waiting room.
- Patterns: State for seats, plus a queue in front of booking.
- What interviewers probe: thousands of buyers at the same second, hold timeouts, and per-user ticket limits. → *Full design:* Ch 15 — 15.3 Design a Concert Ticket Booking System

Q16. ⋆ Design a car / vehicle rental system (Zoomcar)

Asked at: Flipkart (SDE2, 2024)

- Entities: `Branch`, `Vehicle`, `Reservation` (pickup and drop time and branch), `Customer`, `PricingPolicy`, `Invoice`.
- Patterns: Strategy for pricing and for vehicle selection, State for reservation lifecycle.
- What interviewers probe: availability across overlapping time windows, one-way rentals, and damage or late-return charges. → *Full design:* Ch 16 — Design a Car Rental System

Q17. Design a restaurant management system (table booking / waiting queue)

- Entities: `Table`, `Reservation`, `WaitlistEntry`, `Order`, `MenuItem`, `Bill`, `Kitchen`.
- Patterns: Command for orders sent to the kitchen, Observer for "your table is ready", State for orders.
- What interviewers probe: fitting party sizes to tables, walk-ins vs reservations, and splitting the bill. → *Full design:* Ch 16 — Design a Restaurant Management System

Q18. Design an airline management / flight booking system

- Entities: `Flight` (route) vs `FlightInstance` (a dated departure), `Aircraft`, `Seat`, `Itinerary`, `Passenger`, `Booking`.
- Patterns: State for booking, Strategy for fare rules.
- What interviewers probe: the route vs instance split, multi-leg trips booked all-or-nothing, and seat holds. → *Full design:* Ch 16 — Design an Airline Management System

Q19. Design a course registration system

- Entities: `Course`, `Section` (time slot, capacity), `Student`, `Enrollment`, `Prerequisite`, `Waitlist`.
- Patterns: Specification or Chain of Responsibility for eligibility checks (prerequisites, clashes, credit limit).
- What interviewers probe: the last seat taken by two students at once, and automatic waitlist promotion. → *Full design:* Ch 16 — Design a Course Registration System

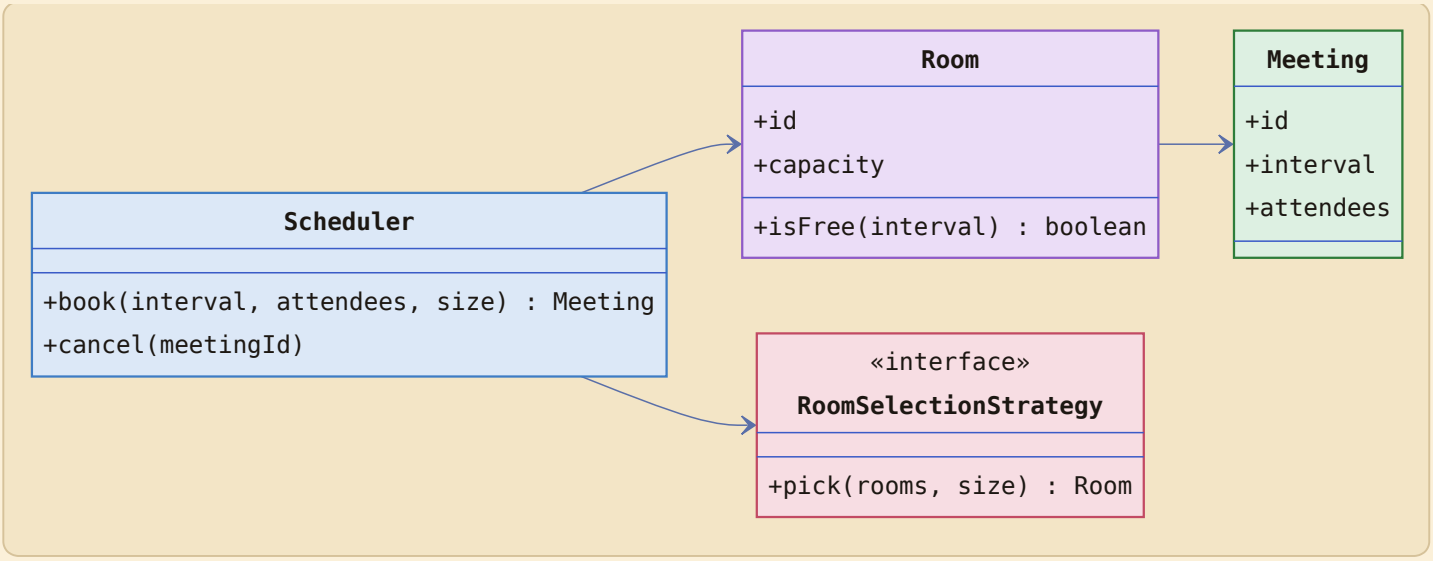
Q20. Design a learning management system / online exam portal

- Start from course registration and add `Exam`, `Question` (single choice, multiple choice, text), `Attempt`, `Grader`.
- Patterns: Strategy for grading each question type, State for an attempt (started, submitted, graded), and a timer that auto-submits.
- What interviewers probe: saving answers often so a dropped connection loses nothing, and randomising question order per student. → *Related design:* Ch 16 — Design a Course Registration System

Q21. ⋆ Design a Meeting Scheduler / meeting room booking

Requirements. An office has rooms, each with a capacity. A user books a room for a time interval with a list of attendees. The system picks a free room large enough, rejects clashes, supports cancellation, shows a room's or a person's calendar, and notifies attendees. A common follow-up asks for the first free slot that suits every attendee.

Main classes.



- `Interval`: a record `(start, end)` with `overlaps(other)`. Treat it as half-open, so 10:00–11:00 and 11:00–12:00 do not clash.
- `Room`: keeps its bookings in a `TreeMap<start, Meeting>`. To check a clash, look only at the neighbours (`floorEntry`, `ceilingEntry`), which takes O(log n).
- `RoomSelectionStrategy`: picks the smallest room that fits, or the nearest room.
- `NotificationService`: tells attendees about invites, changes and cancellations (Observer).

Concurrency and edge cases. Two people can book the same room for overlapping times. Lock per room, then check and insert under that lock. If you try rooms one by one, release each lock before taking the next, so the code cannot deadlock. Reject `end <= start`. A recurring meeting expands into instances, and the booking fails if any instance clashes (or the system offers to skip it).

Extension question. "Find the earliest 30-minute slot when all attendees are free." Collect every attendee's busy intervals, sort them, merge the overlapping ones, then scan the gaps. This is the classic merge-intervals technique.

Q22. ⋆ Design a doctor appointment booking system

Asked at: Flipkart (SDE2, 2021)

Requirements. Doctors register with a speciality (cardiologist, dermatologist). Each doctor declares available one-hour slots for the day. Patients search by speciality and see free slots, sorted by start time by default. A patient books, cancels, and views their bookings. A patient may not hold two appointments at the same time. When a slot is full, the patient can join its waitlist, and a cancellation promotes the first person waiting.

Main classes.

- `Doctor` (id, name, speciality, slots) and `Patient` (id, name, appointments).
- `Slot`: doctor, start time, status (`FREE`, `BOOKED`), booked patient, and a waitlist (a `Deque<Patient>`).
- `Appointment`: a record (id, patient, slot).
- `BookingService`: `book`, `cancel` and `joinWaitlist`. It enforces "no double booking for the patient".
- `SearchService` with a `SlotRankingStrategy`. Order by start time by default, or by doctor rating later.
- Repositories: in-memory maps, as the machine-coding round expects.

Patterns. Strategy (ranking), Observer (tell a waitlisted patient they were promoted), a simple State on `Slot`.

Concurrency and edge cases. Two patients may book the last slot together. Guard the slot status with a per-slot lock, or a compare-and-set on an `AtomicReference<SlotStatus>`. Check the patient's own clash under the patient's lock. Lock patient then slot, always in that order, so no deadlock can form. Validate that slots are whole hours inside working hours. Cancelling an already-cancelled booking is a no-op.

Extension question. "Show the trending doctor: the one with the most bookings today." Keep a counter per doctor that `book` updates, and read it through a max-heap or a sorted view.

Games

Q23. ⋆ Design Snake and Ladder

- Entities: `Board` (size, snakes and ladders as jumps), `Player`, `Dice`, `Game` (turn order queue).
- Patterns: Strategy for dice (one die, two dice, a loaded die in tests), a simple game loop.
- What interviewers probe: input-driven setup in machine coding, exact-roll-to-win rules, and chained jumps. → *Full design:* Ch 17 — Design Snake and Ladder

Q24. ⋆ Design Chess

- Entities: `Board`, `Square`, `Piece` subclasses, `Move`, `Player`, `Game` (status, history).
- Patterns: Strategy (or polymorphism) for each piece's moves, Command for moves with undo, State for game status.
- What interviewers probe: check / checkmate detection (simulate the move, then test the king), castling and en passant, and why moves should be objects. → *Full design:* Ch 17 — Design Chess

Q25. ⋆ Design a 2048 game

Asked at: Flipkart (2021)

- Entities: `Board` (grid), `Tile`, `Game` (score, state), `MoveDirection`, `TileSpawner`.
- Pattern: write one "slide and merge a row left" routine and rotate the grid for the other three directions.
- What interviewers probe: each tile merging only once per move, spawning only after a real change, and detecting game over. → *Full design:* Ch 17 — Design 2048

Q26. ⋆ Design Tic-Tac-Toe

- Entities: `Board`, `Player`, `Game`, `WinChecker`.
- Patterns: Strategy for players (human, AI), and per-row / column / diagonal counters so a win check is O(1).
- What interviewers probe: N×N boards, the O(1) win check, and keeping input handling out of the game rules. → *Full design:* Ch 17 — Design Tic-Tac-Toe

Q27. Design a deck of cards / Blackjack

- Entities: `Card` (rank, suit enums), `Deck`, `Shoe`, `Hand`, `Player`, `Dealer`, `Game`.
- Patterns: Template Method or Strategy for the dealer's rule, and a generic deck reused by other card games.
- What interviewers probe: ace counted as 1 or 11, and making a deck that works for any card game. → *Full design:* Ch 17 — Design Blackjack and a Deck of Cards

Q28. Design Minesweeper

Requirements. A rows × cols grid hides a number of mines. The player reveals a cell or toggles a flag. Revealing a mine loses the game. Revealing a number shows how many of the eight neighbours are mines. Revealing a zero opens all connected empty cells and their numbered borders. The player wins when every non-mine cell is revealed. The first click must never be a mine.

Main classes.

- `Cell`: `mine` (boolean), `adjacentMines`, and `state` (`HIDDEN`, `REVEALED`, `FLAGGED`).
- `Board`: the grid, `neighbours(r, c)`, `revealedCount`, `mineCount`.
- `MinePlacer` (interface): random placement with a seed. Tests use a fixed layout. Mines are placed *after* the first click, avoiding that cell (and its neighbours, if you like).
- `Game`: `reveal(r, c)`, `toggleFlag(r, c)`, and `status` (`PLAYING`, `WON`, `LOST`).
- `Renderer`: prints the board. Keep it separate from the rules.

Patterns. Strategy (mine placement), State for game status (no moves after a win or loss), and Observer if a UI must redraw.

Algorithm. For reveal, use a breadth-first search with a queue, not recursion. A recursive flood fill on a 1000×1000 empty board can overflow the stack. Mark each cell revealed when you add it to the queue, so no cell is processed twice. Flagged cells are not opened by the flood. Win check: `revealedCount == rows * cols - mineCount`, an O(1) test.

Edge cases. Ignore clicks outside the board, on revealed cells, or after the game ended. Reject more mines than cells minus one.

Extension question. "Add chording: clicking a number whose flags are all placed opens its other neighbours." Write it as one more method that reuses `reveal`, so no existing rule changes.

Q29. Design a Bowling Alley scoring system

- Entities: `Game`, `Frame` (rolls), `TenthFrame` (up to three rolls), `Player`, `Lane`.
- Key logic: a strike adds the next two rolls, a spare adds the next one. Score from the stored list of rolls instead of keeping running totals, so bonuses resolve naturally.
- What interviewers probe: the tenth frame, partial scores while bonuses are pending, and validating pin counts per frame.
- The scoring core for a finished game (validate rolls first: 0–10 each, at most 10 per frame outside the tenth):

```
static int score(List<Integer> rolls) {
    int total = 0, i = 0;
    for (int frame = 1; frame <= 10; frame++) {
        if (rolls.get(i) == 10) {                                  // strike
            total += 10 + rolls.get(i + 1) + rolls.get(i + 2); i += 1;
        } else if (rolls.get(i) + rolls.get(i + 1) == 10) {        // spare
            total += 10 + rolls.get(i + 2); i += 2;
        } else { total += rolls.get(i) + rolls.get(i + 1); i += 2; }
    }
    return total;   // 12 strikes -> 300; 21 rolls of 5 -> 150
}
```

This book has no Bowling Alley case study, so this outline is the whole answer here.

Q30. Design an online game engine

Requirements. Provide the core that any 2D game can build on: a loop that updates and draws the world at a steady rate, game objects with position and behaviour, keyboard and mouse input, collision detection, events between objects, loading images and sounds once, and screens (menu, playing, paused).

Main classes.

- `GameLoop`: runs a fixed-timestep update (say 60 times a second) and renders as often as possible. It catches up with several updates if a frame was slow.
- `Entity`: an id that holds `Component`s (`Position`, `Velocity`, `Sprite`, `Collider`). This is the Entity-Component-System (ECS) idea: behaviour comes from parts, not from a deep inheritance tree.
- `System`: `MovementSystem`, `CollisionSystem` and `RenderSystem`. Each one walks the entities that have the components it needs.
- `InputHandler`: maps keys to `Command` objects (`Jump`, `MoveLeft`), so key bindings can change.
- `EventBus`: lets objects publish "player died" or "coin collected" without knowing each other.
- `AssetManager`: loads each texture once and shares it.
- `SceneManager`: holds a stack of `Scene`s. Each scene is a state, such as `Menu`, `Level` or `Pause`.

Patterns. Component (composition over inheritance), Command (input), Observer (events), Flyweight (shared assets), State (scenes), Game Loop.

Concurrency and edge cases. Keep the world single-threaded. Input arrives from other threads into a `ConcurrentLinkedQueue`, and the loop drains it at the start of each tick. Asset loading can run on virtual threads, publishing "loaded" events. Use a fixed time step, so physics gives the same result on fast and slow machines.

Extension question. "Make it multiplayer." The server owns the true state and runs ticks. Clients send commands, predict movement locally, and correct themselves when the server state arrives.

Platforms and marketplaces

Q31. ⋆ Design a cab booking / ride-sharing service (Uber; Flipkart "offer and select a ride")

Asked at: Flipkart

- Entities: `Rider`, `Driver`, `Vehicle`, `Location`, `RideRequest`, `Trip`, `FareCalculator`, `MatchingService`.

- Patterns: Strategy for matching (nearest, best rated) and for fares (surge), State for the trip, Observer for live updates.
- What interviewers probe: two riders matched to one driver, cancellation rules, and the carpool variant (offer a ride with seats, select the best ride). → *Full design:* Ch 20 — 20.4 Design a Ride Sharing Service (Uber)

Q32. ⋆ Design a food delivery system (Swiggy / Zomato; Flipkart "FoodKart")

Asked at: Flipkart

- Entities: `Restaurant`, `Menu`, `Item`, `Cart`, `Order`, `DeliveryAgent`, `RestaurantSelectionStrategy`.
- Patterns: State for the order, Strategy for choosing the restaurant (lowest cost, highest rating) and assigning agents, Observer for tracking.
- What interviewers probe: restaurant capacity (max orders at once) freed when an order completes, and one-restaurant-per-cart rules. → *Full design:* Ch 19 — Design an Online Food Delivery Service (Swiggy) + FoodKart

Q33. ⋆ Design CricInfo / a cricket scoreboard (Udaan "Cricket Match Dashboard")

Asked at: Udaan

- Entities: `Match`, `Team`, `Player`, `Innings`, `Over`, `Ball` (the event that everything derives from), `Scorecard`.
- Patterns: Template Method for match formats, Observer for live updates, "derive stats from balls, don't store totals".
- What interviewers probe: extras (wide, no-ball), strike rotation at the end of an over, and the machine-coding version that prints the scorecard after every over. → *Full design:* Ch 18 — 18.2 Design Cricinfo

Q34. ⋆ Design a stock exchange / order matching engine (LLD)

- Entities: `Order` (buy or sell, limit or market, quantity, price, time), `OrderBook` per symbol, `Trade`, `MatchingEngine`.
- Data structure: for each side, a `TreeMap<price, Deque<Order>>`. The best bid is the highest key and the best ask is the lowest. Match by price first, then by time.
- What interviewers probe: partial fills, cancelling an order sitting in the book, and a single writer thread per symbol so no locks are needed. → *Related design:* Ch 19 — Design an Online Stock Brokerage System

Q35. Design an online stock brokerage system

- Entities: `Account`, `Portfolio`, `Position`, `Order` (market, limit, stop), `Exchange` adapter, `Watchlist`.
- Patterns: Strategy/Factory for order types, State for order lifecycle, Observer for price alerts, Adapter for the exchange.
- What interviewers probe: blocking funds when an order is placed, partial fills, and order status kept in step with the exchange. → *Full design:* Ch 19 — Design an Online Stock Brokerage System

Q36. Design a cryptocurrency exchange

- Same core as Q34 (order book and matching) plus wallets per currency, deposits that wait for blockchain confirmations, and withdrawals that need approval.
- Patterns: State for deposits and withdrawals, Strategy for fees.
- What interviewers probe: exact decimal amounts (`BigDecimal` with each coin's scale), and never letting a balance go negative under concurrent trades. → *Related design:* Ch 19 — Design an Online Stock Brokerage System

Q37. Design Stack Overflow

- Entities: `User` (reputation, badges), `Question`, `Answer`, `Comment`, `Vote`, `Tag`, `Bounty`.
- Patterns: Observer for reputation changes and notifications, Strategy for search and ranking.
- What interviewers probe: one vote per user per post, accepting an answer, and permissions that depend on reputation. → *Full design:* Ch 18 — 18.1 Design Stack Overflow

Q38. Design LinkedIn

- Entities: `Member`, `Profile`, `Connection`, `Invitation`, `Post`, `Job`, `Message`.
- Patterns: State for invitations, BFS for 1st/2nd/3rd degree connections, Strategy for feed ranking.

- What interviewers probe: two-sided operations (accepting adds both sides), and degree computation limits. → *Full design:* Ch 18 — 18.3 Design LinkedIn

Q39. Design Facebook / a social network

- Entities: `User`, `Friendship`, `Follow`, `Post`, `Audience` (privacy), `Page`, `Group`, `NewsFeed`.
- Patterns: one visibility policy used by feed and search, Strategy for feed ranking, pull-model feed.
- What interviewers probe: friends vs followers, privacy checks on every read, and friend suggestions. → *Full design:* Ch 18 — 18.4 Design Facebook

Q40. ⋆ Design Twitter (post / follow / feed; LeetCode 355 "Design Twitter")

- The LeetCode version: `postTweet`, `getNewsFeed` (the 10 newest tweets from self and followees), `follow`, and `unfollow`.
- Data: a map from user to followees, and per user a list of tweets with a global increasing timestamp. Build the feed by merging the users' newest-first lists with a max-heap, k-way merge style, and stop after 10.
- What interviewers probe: why you pull (merge on read) rather than push here, self-follow, and unfollowing yourself. → *Related design:* Ch 18 — 18.4 Design Facebook (feed and follow)

Q41. ⋆ Design Amazon (online shopping system / cart / checkout)

- Entities: `Product`, `Catalog`, `Cart`, `Order`, `OrderItem`, `Payment`, `Shipment`, `Inventory`.
- Patterns: State for orders, Strategy for payment and shipping, Observer for status emails.
- What interviewers probe: reserving stock at checkout vs at payment, price changing while the item sits in the cart, and partial cancellations. → *Full design:* Ch 19 — Design Amazon — Online Shopping System

Q42. Design an online auction system (eBay; live IPL auction)

- Entities: `Auction` (item, reserve price, end time), `Bid`, `Bidder`, `Seller`, `AuctionHouse`.
- Patterns: State (scheduled, live, closed), Observer for outbid alerts, Strategy for auction type (English, sealed bid).
- What interviewers probe: two bids arriving at the same moment (serialise per auction), last-second sniping and extending the end time. → *Full design:* Ch 19 — Design an Online Auction System

Q43. Design Spotify / a music streaming service (LLD)

- Entities: `Track`, `Album`, `Artist`, `Playlist`, `User`, `Subscription`, `Player`, `PlayQueue`.
- Patterns: Strategy for recommendations and shuffle, State for the player, Iterator for queues.
- What interviewers probe: free vs premium limits, collaborative playlists, and the play-next queue vs the playlist order. → *Full design:* Ch 19 — Design a Music Streaming Service (Spotify)

Q44. Design a grocery store / POS / checkout (BigBasket)

- Entities: `Product` (priced by unit or by weight), `Inventory`, `Basket`, `Receipt`, `Discount`, `Payment`.
- Patterns: Strategy and Composite for discounts (buy-2-get-1, percent off a category), Decorator for tax.
- What interviewers probe: the order in which discounts apply, and returns. → *Full design:* Ch 19 — Design a Grocery Store (POS & Checkout)

Q45. Design a subscription-based sports score website

- Reuse the Cricinfo model for matches and add `Subscription` (plan, expiry), `Entitlement` checks and live score pushes.
- Patterns: Observer (subscribers to a match), Proxy for checking entitlements before content is served.
- What interviewers probe: free vs paid content per request, and fan-out of score updates to many subscribers. → *Related design:* Ch 18 — 18.2 Design Cricinfo

Q46. Design a logistics / shipment tracking system

- Entities: `Shipment`, `Parcel`, `Hub`, `Route`, `TrackingEvent`, `Courier`.
- Patterns: State for shipment status, Observer for customer updates, Strategy for route choice.
- What interviewers probe: events that arrive out of order (sort by event time, not arrival), and returns. → *Related design:* Ch 14 — 14.7 Design a Shipping Locker System

Q47. Design a newsletter service

- Entities: `Publication`, `Issue`, `Subscriber`, `Subscription` (topics, frequency), `Delivery`.

- Patterns: Observer / Pub-Sub (subscribers per topic), Strategy for channel (email, in-app), Template Method for rendering.
- What interviewers probe: unsubscribe taking effect at once, retries without double delivery, and daily digests. → *Related design:* Ch 20 — 20.6 Design a Pub-Sub System

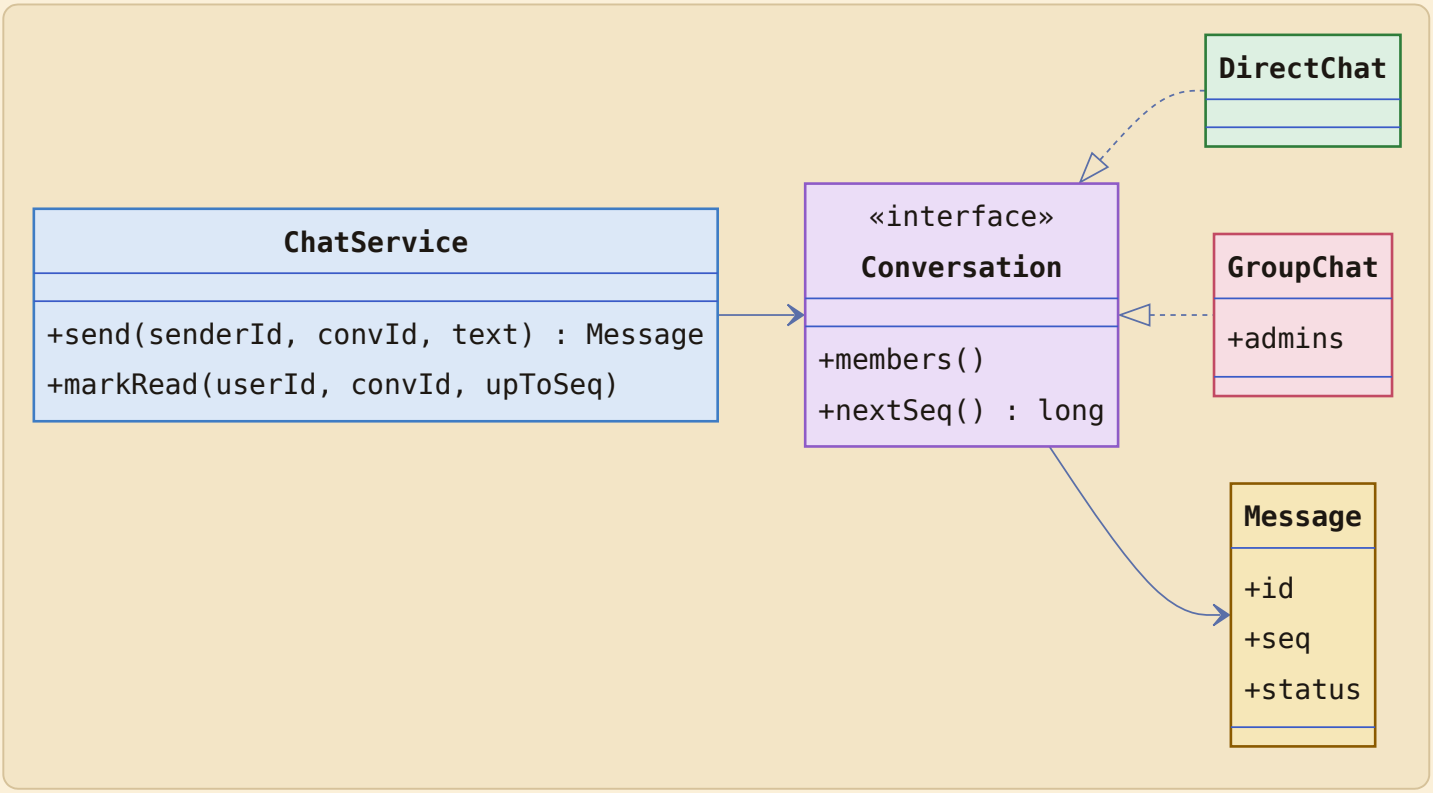
Q48. Design a maps navigator client (walk / car / bus / bike)

- Entities: `Navigator`, `Route`, `RouteStrategy` (one per travel mode), `Location`.
- Pattern: Strategy is the whole question. The navigator swaps strategies at runtime without `if` chains.
- What interviewers probe: adding a new travel mode without touching old code, and combining modes (walk + bus). → *Related design:* Ch 11 — 11.3 Strategy

Q49. Design a chat server / WhatsApp (LLD)

Requirements. Users send text messages one-to-one and in groups. Messages show sent, delivered and read ticks. Offline users receive their messages when they come back. Each conversation keeps its order. Groups have admins who add and remove members. Online and last-seen status is a nice extra.

Main classes.



- `Conversation` (sealed interface): `DirectChat` (two members) and `GroupChat` (members, admins).
- `Message`: a record (id, conversationId, senderId, seq, text, sentAt). Its delivery status is kept per recipient, because in a group each person reads at a different time.
- `Inbox`: per user, a queue of messages not yet delivered.
- `PresenceService`: online or offline, and last seen.
- `ConnectionRegistry`: the user's live sessions. Delivery listeners hang off it (Observer).

Patterns. Observer (deliver to connected sessions), Strategy (delivery channel: socket or push notification), and Mediator (`ChatService` coordinates instead of users talking to each other).

Concurrency and edge cases. Give each conversation an `AtomicLong` sequence. Sort messages by sequence, not by clock time, so senders on two phones cannot reorder history. Make delivery idempotent: a client acknowledges the highest sequence it has seen, and re-sent messages are ignored. A removed group member must not receive later messages, so check membership when the message is sent.

Extension question. "Add message editing and deletion." Store edits as new events that point at the original id, and let clients apply them, so the order stays clean.

Q50. Design Truecaller

Requirements. Users sign up with a phone number and may upload their contacts. Anyone can look up a number to see the most likely name and a spam score. Users report numbers as spam and block numbers. Users can

search by name. A user can mark their own number as private.

Main classes.

- `PhoneNumber`: a value object. Its constructor normalises the input to E.164 format (for example `+919876543210`), so the same number can never be stored two ways.
- `User`: an id, an own number, contacts and a block list.
- `Contact`: a record (number, savedName).
- `GlobalDirectory`: maps each number to a name-count table (name → how many users saved it under that name). A lookup returns the most frequent name, and a registered user's own name always wins.
- `SpamService`: counts reports per number and gives a score. A number is "likely spam" past a threshold, weighted by how trustworthy each reporter is.
- `CallerIdService`: answers `lookup(number)` with `CallerInfo(name, spamScore, blocked)`.
- `NameSearchIndex`: a trie (prefix tree) from name prefixes to numbers.

Patterns. Value object, Strategy (spam scoring, name ranking), Observer (a new contact upload updates the directory and the index), Facade (`CallerIdService`).

Concurrency and edge cases. Uploads and reports arrive at the same time from many users. Use `ConcurrentHashMap<PhoneNumber, ConcurrentHashMap<String, LongAdder>>` for name counts and `LongAdder` for spam counts, so updates never block reads. Count one report per reporter per number, so repeat reports don't inflate the score. A private user's name should not be served from the crowd data.

Extension question. "Mark verified business numbers." Add a `VerifiedBusiness` record checked before the crowd data, so a verified name always beats crowd guesses.

Q51. Design a mentorship / mock-interview platform (Pramp)

Requirements. Users sign up with the topics they practise (data structures, system design) and their available time slots. The platform pairs two users who share a topic and a free slot. Each person interviews the other for half the session. Afterwards both leave feedback. Users can cancel, and they are reminded before a session.

Main classes.

- `User`: id, topics, a calendar of `AvailabilitySlot`s, and a rating.
- `AvailabilitySlot`: a record (start, end).
- `MatchRequest`: user, topic, preferred slots.
- `MatchingService`: keeps the open requests per topic and slot, and pairs requests through a `MatchingStrategy` (first come first served, similar rating).
- `Session`: two participants, topic, slot, question set, and a state (`SCHEDULED`, `IN_PROGRESS`, `COMPLETED`, `CANCELLED`, `NO_SHOW`).
- `Feedback`: a record (from, to, scores, notes).
- `ReminderScheduler`: uses a `ScheduledExecutorService` to fire reminders.

Patterns. Strategy (matching), State (session), Observer (send notifications on state change), Builder for sessions.

Concurrency and edge cases. One user must never be matched twice for the same slot. Run the matching in one step: lock both users' calendars in a fixed order (lowest id first) to avoid deadlock, check both are still free, then book both. A cancellation within an hour counts against the user's reliability, and the partner goes back into the pool. A no-show is detected when a session never moves to `IN_PROGRESS`.

Extension question. "Add paid sessions with expert mentors." Add a `Mentor` role with a price. Payment must succeed before the session moves to `SCHEDULED`. The rest of the flow stays the same.

Q52. Design an Application Tracking System (Greenhouse)

Requirements. A company posts jobs. Each job has a pipeline of stages (Applied → Phone screen → Onsite → Offer → Hired). Candidates apply, and recruiters move applications forward or reject them. Interviewers submit scorecards. Hiring managers see a dashboard of each job's pipeline. Every action is audited.

Main classes.

- `Job`: id, title, department, status (`OPEN`, `ON_HOLD`, `CLOSED`), and its own `Pipeline` (an ordered list of `Stage`s).
- `Candidate`: a person, who may have several `Application`s.

- `Application`: candidate + job, the current stage, a status (`ACTIVE`, `REJECTED`, `WITHDRAWN`, `HIRED`), and a history of stage changes.
- `Interview`: application, interviewers and time. `Scorecard`: interviewer, ratings and a recommendation.
- `ApplicationService`: `advance`, `reject` and `hire`. It checks the rules (no skipping to offer without scorecards, unless overridden).
- `PermissionPolicy`: decides what a recruiter, an interviewer or a hiring manager may see or do.
- `AuditLog` and `NotificationService`.

Patterns. State (application status), Template Method or configurable stage lists (each job has its own pipeline), Chain of Responsibility (the checks before advancing), Observer (emails and audit entries on each change).

Concurrency and edge cases. Two recruiters may move the same application at the same time. Use optimistic locking: a version number on `Application`, where an update fails if the version changed. Duplicate applications (same candidate, same job) are merged or rejected. Closing a job should not delete applications, only stop new ones.

Extension question. "Auto-reject candidates who fail a knock-out question." Add a rule to the advance chain that runs on `Applied`.

Q53. Design a call center

Requirements. The center has three levels of employee: respondents, managers and directors. An incoming call goes to a free respondent. If the respondent can't handle it, or none is free, it escalates to a manager, then to a director. If nobody is free, the call waits in a queue. When an employee finishes a call, they take the next waiting call at their level.

Main classes.

- `Employee` (abstract or sealed): id, `Rank`, and a current call. Subtypes `Respondent`, `Manager`, `Director`. Each can `canHandle(call)`.
- `Call`: caller, required minimum rank (it rises when escalated), state (`WAITING`, `IN_PROGRESS`, `ESCALATED`, `ENDED`).
- `CallDispatcher`: holds a queue of free employees per rank (`BlockingQueue<Employee>`) and a queue of waiting calls per rank. `dispatch(call)` tries the call's rank and above.
- `EscalationHandler`: a chain from respondent to manager to director.

Patterns. Chain of Responsibility (escalation), Strategy (routing: least busy, skills-based), State (call lifecycle).

Concurrency and edge cases. Calls arrive and end on many threads. Use `poll()` on the free-employee queues, so taking an employee is atomic. When an employee becomes free, first check the waiting-call queue for their rank (and lower ranks), and only then return to the free pool. Do these two steps under one lock, or a call could be missed. If a call is dropped mid-escalation, end it cleanly and free the employee. A call that waits too long can be escalated or offered a callback.

Extension question. "Add priority customers." Change the waiting queue to a `PriorityBlockingQueue` ordered by priority, then by arrival time.

Q54. Design a Survey / Google Forms system

Requirements. An owner builds a form with a title and questions of several types: short text, paragraph, single choice, multiple choice, rating, and date. Questions can be required and can have validation (email format, number range). The form can show or skip a section based on an earlier answer. Respondents submit responses. The owner sees a summary per question and can close the form.

Main classes.

- `Form`: id, owner, settings (one response per user, deadline), an ordered list of `Section`s, a version, and a status.
- `Question` (sealed interface): `TextQuestion`, `ChoiceQuestion` (options, multiSelect), `RatingQuestion` (scale), `DateQuestion`. Each has `validate(Answer)`.
- `ValidationRule`: composable rules (required, regex, range).
- `BranchRule`: "if the answer to question 3 is 'No', go to section 5".
- `Response`: form id, form version, respondent, and `Map<questionId, Answer>`.
- `FormBuilder`: builds a form step by step and checks it is valid.
- `Summary`: per question type, a different aggregation (counts, average, word list).

Patterns. Builder (forms), Composite (form → sections → questions), Strategy/Specification (validation), Visitor or a pattern-matching `switch` over the sealed `Question` types for summaries.

Edge cases. Editing a form after responses exist: create a new version, so old responses still point at the questions they answered. Enforce one response per user with a set of respondent ids, updated atomically with `ConcurrentHashMap.newKeySet().add(...)`. Validate every answer on the server, never trusting the client.

Extension question. "Add a quiz mode with scores." Add a correct answer and points to each question, plus a `Grader` that walks the response. Forms that are not quizzes are unaffected.

Q55. Design an employee management / payroll system

Requirements. Store employees with department, role and manager. Show the org chart and everyone who reports (directly or indirectly) to a manager. Track leave balances and requests. Run monthly payroll for salaried, hourly and commission employees, with deductions (tax, provident fund) and payslips.

Main classes.

- `Employee`: id, name, department, role, managerId, join date, and a `PayPolicy`.
- `OrgChart`: manager id → list of direct reports. `allReports(id)` walks the tree breadth-first.
- `PayPolicy` (interface): `SalariedPolicy` (monthly amount, prorated for part months), `HourlyPolicy` (rate × timesheet hours, overtime), `CommissionPolicy` (base + percent of sales).
- `Deduction` (interface): `IncomeTax`, `ProvidentFund`, `LeaveWithoutPay`, applied in order.
- `PayrollRun`: a month and the list of `Payslip`s it produced. Its status is `DRAFT`, `APPROVED` or `PAID`.
- `LeaveService`: balances per leave type, and requests that a manager approves.

Patterns. Strategy (pay policies), Chain of Responsibility or a list of deductions, Composite (org tree), State (payroll run and leave request).

Edge cases. Hold money as `long` paise or `BigDecimal`, and round only once, at the end. Mid-month joiners and leavers get prorated pay. A manager cycle (A reports to B, B to A) must be rejected when a manager is assigned. Once a payroll run is `PAID`, it is never edited. Corrections go into next month as adjustments.

Extension question. "Change an employee's salary from the middle of the month." Store pay policies with an effective date, and prorate between the two policies.

Components and infrastructure

Q56. ⋆ Design a Rate Limiter (LLD)

Asked at: Atlassian (P4 LLD round)

- Entities: `RateLimiter` interface, `TokenBucket`, `SlidingWindowLog`, `FixedWindowCounter`, a `RateLimiterRegistry` (one limiter per client key), and a config.
- Patterns: Strategy for the algorithm, Factory to build from config.
- What interviewers probe: thread safety (per-key lock or CAS), lazy refill based on elapsed time instead of a timer thread, and the burst behaviour of each algorithm. → *Full design:* Ch 20 — 20.7 Design a Rate Limiter; also Ch 13 — 13.11 Rate limiter — token bucket

Q57. ⋆ Design an LRU Cache

- Data structure: a `HashMap<key, Node>` plus a doubly linked list. Every `get` or `put` moves the node to the front, and eviction removes the tail. Both operations are O(1).
- In Java you can also show `LinkedHashMap` with `accessOrder = true` and an overridden `removeEldestEntry`.
- What interviewers probe: writing the linked list by hand with dummy head and tail nodes, and making it thread-safe. → *Full design:* Ch 20 — 20.3 Design an In-Memory Cache (and an LRU Cache from scratch)

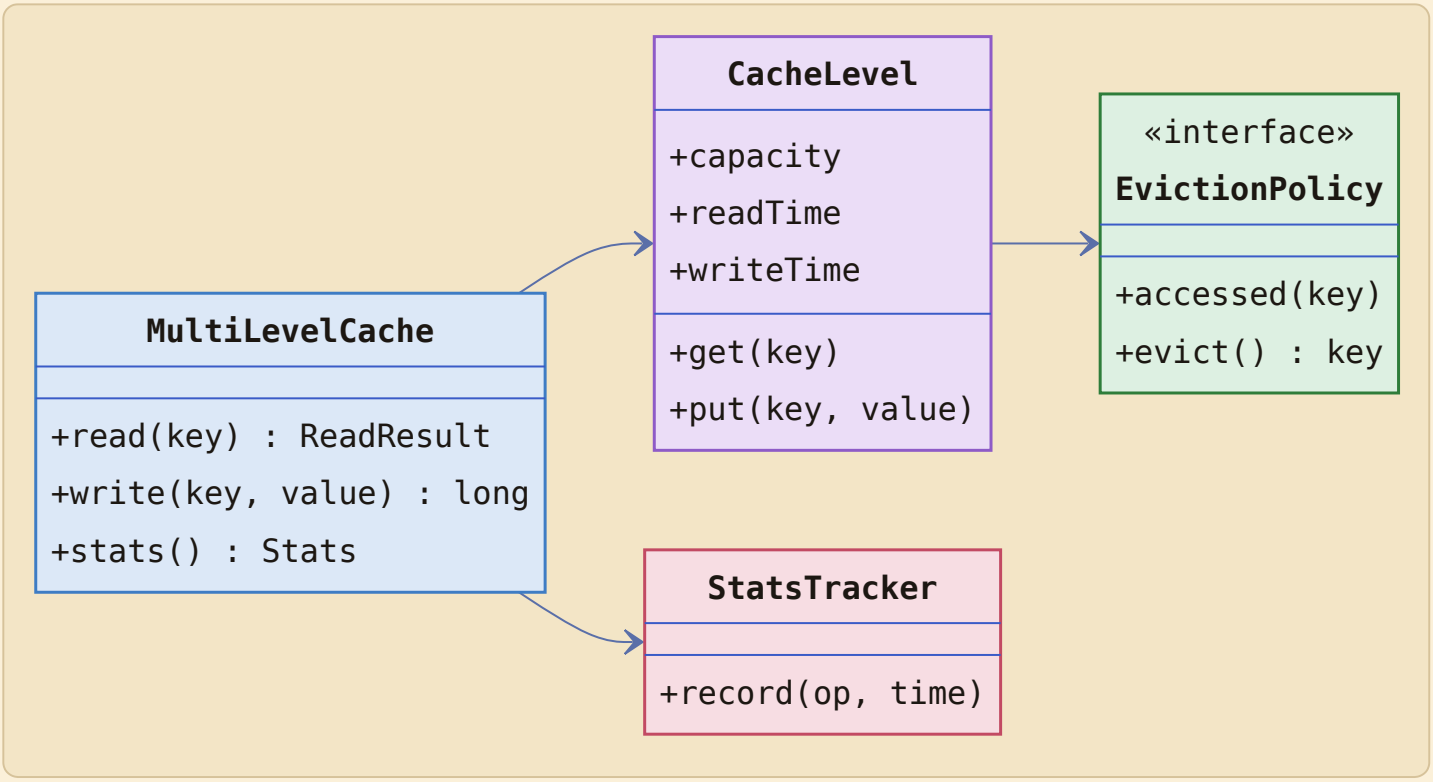
Q58. ⋆ Design a cache with pluggable eviction policies (LRU / LFU)

- Entities: `Cache<K, V>`, `Storage`, `EvictionPolicy` (`keyAccessed`, `evictKey`) with LRU, LFU and FIFO implementations.
- Pattern: Strategy for eviction. LFU in O(1) uses a frequency → linked set map plus a `minFreq` pointer (LeetCode 460).
- What interviewers probe: tie-breaking in LFU, and keeping the policy and the storage in step. → *Full design:* Ch 20 — 20.3 Design an In-Memory Cache

Q59. ⋆ Design a multi-level cache

Requirements. A cache has N levels (L1, L2, … Ln). Each level has a capacity, a read time and a write time. Reading a key checks L1 first, then each lower level, until the key is found. When found at level i, write the value into every level above i. Writing a key writes it to every level, skipping a level that already holds the same value. Report the time taken by every read and write, plus the average read and write time over the last N operations. Show usage per level.

Main classes.



- `CacheLevel`: one level, with its own storage and `EvictionPolicy` (LRU by default).
- `MultiLevelCache`: holds the ordered list of levels and runs the read and write paths. Time is the sum of each level's read time visited, plus the write times added.
- `StatsTracker`: a ring buffer of the last N timings for reads and for writes, with running sums so averages are O(1).

Patterns. Chain of Responsibility (a level passes a miss to the next one), Strategy (eviction), Builder (configure levels).

Concurrency and edge cases. In a machine-coding round, guard the whole cache with one lock (`synchronized` or a `ReentrantLock`). A read-write lock does not help here, because a read is not read-only: it updates each level's LRU order and copies the value into the levels above, so every read would need the write lock anyway. For more throughput, lock levels one at a time, always top-down, so no deadlock can form. An eviction at L1 does not remove the key from L2. Reject capacity 0 or an empty list of levels.

Extension question. "Make it write-back instead of write-through." Writes go to L1 only and are marked dirty. They reach lower levels when evicted, and a flush is forced on shutdown.

Q60. ⋆ Design an in-memory key-value store (with transactions / TTL / versioning)

- Base: `ConcurrentHashMap<String, Entry>` where `Entry` holds value and expiry; expired entries are removed lazily on read plus by a periodic sweeper.
- Transactions (`begin`, `set`, `get`, `commit`, `rollback`, nestable): keep a stack of change maps. Reads check the top map, then the maps below it, then the base store. `commit` merges the top map down, and `rollback` pops it.
- Versioning: store a list of `(version, value)` per key and answer `get(key, version)` by binary search.
- What interviewers probe: isolation between clients (one transaction stack per session), and deletes inside a transaction (store a tombstone marker). → *Related design:* Ch 20 — 20.3 Design an In-Memory Cache; Ch 13 — 13.6 Thread-safe cache with TTL

Q61. ⋆ Design a Logging Framework / Logger library

- Entities: `Logger`, `LogLevel`, `LogMessage`, `Appender` (console, file, database), `Formatter`, `LoggerConfig`.

- Patterns: Singleton or factory for loggers, Strategy for appenders and formatters, Chain of Responsibility for level filtering.
- What interviewers probe: async logging through a queue without losing messages on shutdown, and per-package log levels. → *Full design:* Ch 20 — 20.5 Design a Logging Framework

Q62. ⋆ Design a Pub-Sub system / message queue (LLD)

- Entities: `Broker`, `Topic`, `Message`, `Publisher`, `Subscriber`, `Subscription` (with offset).
- Patterns: Observer, and a per-subscriber offset so slow consumers don't block fast ones.
- What interviewers probe: at-least-once delivery with acks and retries, ordering within a topic, and replay from an offset. → *Full design:* Ch 20 — 20.6 Design a Pub-Sub System

Q63. ⋆ Design a Notification System (LLD)

Asked at: Flipkart

- Entities: `Notification`, `Channel` (email, SMS, push), `Template`, `UserPreference`, `NotificationService`, `RetryPolicy`.
- Patterns: Strategy per channel, Template Method or Builder for messages, Observer for events, Decorator for retries.
- What interviewers probe: user opt-outs, retries with backoff, falling back to another channel, and not sending duplicates. → *Full design:* Ch 20 — 20.2 Design a Notification Service

Q64. Design a HashMap

- Array of buckets; each bucket is a linked list (Java turns long chains into balanced trees); index = `hash(key) & (capacity - 1)`; resize and rehash when size exceeds capacity × load factor (0.75).
- For the concurrent version: lock striping (a lock per group of buckets) or CAS on empty buckets.
- What interviewers probe: `equals`/`hashCode` contract, why capacity is a power of two, and resizing under concurrency. → *Full design:* Ch 13 — 13.7 Design a (simplified) Concurrent HashMap

Q65. ⋆ Design a thread-safe blocking queue / producer-consumer

- A bounded buffer: a lock with two conditions, `notFull` and `notEmpty`. `put` waits while the queue is full, and `take` waits while it is empty. Always wait inside a `while` loop, never an `if`.
- Alternative: two `Semaphore`s (free slots, filled slots) plus a mutex.
- What interviewers probe: spurious wakeups, `signal` vs `signalAll`, and shutdown with a poison pill. → *Full design:* Ch 13 — 13.8 Thread-safe bounded Blocking Queue; Ch 12 — 12.22 Producer-consumer pattern

Q66. Design a thread-safe cache with TTL

- `ConcurrentHashMap` of entries with expiry time; lazy expiry on `get`; a `ScheduledExecutorService` or a `DelayQueue` for active cleanup.
- What interviewers probe: get-or-compute without two threads loading the same key (`computeIfAbsent`), and clock choice (`System.nanoTime` for durations). → *Full design:* Ch 13 — 13.6 Thread-safe cache with TTL

Q67. Design a concurrent Bloom filter

- A bit array with k hash functions. "Might contain" can give false positives but never false negatives.
- Concurrent version: an `AtomicLongArray` with CAS on each word, so adds are lock-free.
- What interviewers probe: choosing the size and k for a target false-positive rate, and why you cannot delete (use a counting Bloom filter for that). → *Full design:* Ch 13 — 13.9 Concurrent Bloom Filter

Q68. Design a thread pool / executor

- Entities: a work queue (`BlockingQueue<Runnable>`), worker threads looping on `take()`, and `submit` returning a `Future`. Shutdown has a graceful mode and an immediate mode.
- What interviewers probe: what happens when the queue is full (rejection policy), worker death on exceptions, and when virtual threads make a pool unnecessary. → *Full design:* Ch 12 — 12.21 Thread pool pattern

Q69. Dining philosophers / print FooBar alternately / building H2O (LeetCode concurrency)

- Short puzzles about making threads take turns or meet at a point. The tools are semaphores, `CyclicBarrier`, lock ordering, and conditions.
- Dining philosophers: break the deadlock by always taking the lower-numbered fork first, or by letting at most four philosophers sit at once.

- What interviewers probe: proving there is no deadlock and no starvation. → *Full design:* Ch 13 — 13.1–13.5

Q70. Design a connection pool / object pool

- Entities: `Pool` with idle and in-use sets, `acquire(timeout)` and `release`, max size, validation on borrow, and idle eviction.
- Pattern: Object Pool, with a `Semaphore` capping how many objects can be borrowed at once.
- What interviewers probe: leaked connections (borrow timeouts, try-with-resources wrapper), and broken connections returned to the pool. → *Full design:* Ch 8 — 8.14 Bonus — Object Pool

Q71. Design a payment gateway (Razorpay) — LLD

- Entities: `PaymentRequest`, `Payment` (state: created, authorised, captured, failed, refunded), `PaymentMethod` (card, UPI, net banking), `ProcessorAdapter` per bank, `Refund`, `WebhookNotifier`.
- Patterns: State (payment lifecycle), Strategy/Adapter (methods and processors), Observer (webhooks to merchants).
- What interviewers probe: an idempotency key so retries never double-charge, timeouts with an unknown result (check status before retrying), and refunds that are partial. → *Related design:* Ch 21 — 21.2 Design a Digital Wallet Service

Q72. Design a text editor with undo / redo (Notion / Google Docs LLD)

Asked at: Notion

- Entities: `Document` (a buffer, such as a list of lines or a gap buffer), `Cursor`, `EditCommand` (insert, delete), and `History` (an undo stack and a redo stack).
- Patterns: Command (each edit knows how to undo itself) or Memento (save snapshots). Any new edit clears the redo stack.
- What interviewers probe: Command vs Memento memory cost, grouping keystrokes into one undo step, and collaborative editing (operational transforms) as a stretch. → *Full design:* Ch 10 — 10.2 Command and 10.5 Memento

Q73. Design a business rule engine

- Entities: `Rule` (condition + action, priority), `Condition` (composable with and / or / not), `Fact` (input data), `RuleEngine` (evaluate in priority order, first-match or all-match).
- Patterns: Specification (composable conditions), Chain of Responsibility (rule order), Interpreter if rules come from a text language.
- What interviewers probe: loading rules from config without redeploying, and conflicts when two rules match. → *Related design:* Ch 11 — 11.7 Beyond GoF (Specification); Ch 10 — 10.1 Chain of Responsibility

Q74. Design a circuit breaker (LLD)

- States: `Closed` (count failures), `Open` (fail fast until a cool-down ends), `HalfOpen` (let a few trial calls through; success closes, failure re-opens).
- Patterns: State, plus Proxy/Decorator wrapping the protected call.
- What interviewers probe: a sliding window of failure rate vs a plain count, and thread-safe state changes (`AtomicReference` with CAS). → *Related design:* Ch 20 — 20.2 Design a Notification Service (provider circuit breaker)

Q75. Design a Hit Counter (LeetCode 362)

Requirements. `hit(timestamp)` records a hit at a time given in seconds. `getHits(timestamp)` returns how many hits happened in the past 300 seconds, including the current second. Timestamps arrive in non-decreasing order. Follow-ups: what if there are millions of hits per second, and what if it must be thread-safe?

Main classes.

- `HitCounter` (interface) with `hit` and `getHits`.
- `QueueHitCounter`: a deque of timestamps. `getHits` drops entries older than `t - 299` and returns the size. It is simple, but memory grows with the number of hits.
- `BucketHitCounter`: two arrays of size 300, `times[]` and `counts[]`. The index is `t % 300`. If `times[i] != t`, the bucket is stale, so reset it to `(t, 1)`; otherwise increment. `getHits` sums the buckets whose `times[i] > t - 300`. Memory is fixed, and `getHits` costs O(300).
- `HitCounterRegistry`: one counter per key (per URL or per user), in a `ConcurrentHashMap`.

Patterns. Strategy (swap implementations), with the sliding window as the core idea. This is the same window as a rate limiter, and interviewers often link the two.

Concurrency and edge cases. For thread safety, guard each bucket's reset-or-increment as one step. Lock per bucket (stripes), or store `(time, count)` packed into one `AtomicLong` updated with CAS. For very high hit rates, use a `LongAdder` per bucket. Hits from the future should not happen, but clamp them anyway. If time jumps forward by more than 300 seconds, the stale check already clears old buckets.

Extension question. "Make the window configurable and also report per minute and per hour." Keep several bucket arrays at different resolutions (seconds, minutes), like a time-series database's roll-ups.

Q76. Design a URL shortener (LLD / class design)

Requirements. `shorten(longUrl)` returns a short code such as `aZ3k9Q`. `resolve(code)` returns the long URL. Users may request a custom alias. Links can expire. The service counts clicks. The same long URL may get the same code or a new one; ask which.

Main classes.

- `ShortUrl`: a record (code, longUrl, ownerId, createdAt, expiresAt).
- `CodeGenerator` (interface): `CounterBase62Generator` (an `AtomicLong` counter encoded in base 62, short with no collisions, but guessable), `HashGenerator` (the first 7 characters of a hash of the URL, retrying with a salt on collision), and `RandomGenerator`.
- `UrlRepository`: `saveIfAbsent(ShortUrl)`, `find(code)`. The in-memory version is a `ConcurrentHashMap`.
- `UrlShortenerService`: validates the URL, applies the alias or generates a code, and handles expiry.
- `ClickTracker`: a `LongAdder` per code, with events sent on asynchronously for analytics.

Patterns. Strategy (code generation), Repository, Facade (the service), Observer (click events).

Concurrency and edge cases. Two users may ask for the same custom alias. `putIfAbsent` decides the winner atomically, and the loser gets "alias taken". The counter generator never collides, while hash and random generators must retry. Reject non-HTTP(S) URLs and links to the shortener itself (loops). Expired codes return "not found". Remove them lazily, plus a sweeper. Keep codes case-sensitive and avoid look-alike characters if humans type them.

Extension question. "Hide the order of codes, so people cannot enumerate links." Scramble the counter with a reversible mixing function before encoding. It stays collision-free but no longer looks sequential.

Q77. Design a JSON / CSV parser

Requirements. Parse JSON text into an in-memory tree: objects, arrays, strings (with escapes and `\uXXXX`), numbers (sign, fraction, exponent), `true`, `false` and `null`. Give clear errors with line and column. Serialise the tree back to text. For CSV: split rows into fields, where quoted fields may contain commas, newlines and doubled quotes (`""`).

Main classes.

- `JsonValue` (sealed interface) with records `JsonObject(Map)`, `JsonArray(List)`, `JsonString`, `JsonNumber(BigDecimal)`, `JsonBool`, `JsonNull`.
- `Lexer`: turns characters into `Token`s (`LBRACE`, `STRING`, `NUMBER`, …) and tracks position.
- `Parser`: recursive descent. `parseValue` looks at the next token and calls `parseObject`, `parseArray` and so on. One method per grammar rule.
- `JsonWriter`: walks the tree with a pattern-matching `switch` over the sealed types. The compiler (Java 21 and later) checks that every case is covered.
- `ParseException`: message, line, column.
- `CsvReader`: a small state machine with the states `FIELD_START`, `IN_UNQUOTED`, `IN_QUOTED`, `QUOTE_IN_QUOTED`.

Patterns. Interpreter/Composite (the value tree), Visitor-style traversal (here a sealed `switch`), State (the CSV reader), Builder for writing.

Edge cases. Duplicate keys (last wins, or an error). Very deep nesting, so add a depth limit against stack overflow. Leading zeros (`01` is invalid in JSON). Trailing commas (invalid). Big numbers (use `BigDecimal`). Text left over after the root value (an error). For huge inputs, offer a streaming, event-based API (`startObject`, `key`, `value`) instead of building the whole tree.

Extension question. "Map JSON onto Java records." Use reflection on record components (`getRecordComponents()`) to match keys to constructor parameters.

Q78. Design a configuration management system (LLD)

Requirements. Services read typed config values (`int`, `boolean`, `duration`) by key. Values can be set at several layers: defaults, then environment (dev, staging, prod), then service, then instance. The most specific layer wins. Every change creates a new version and records who changed what. Users can roll back to an earlier version. Running services are told about changes without restarting.

Main classes.

- `ConfigKey`: a record (namespace, name), plus a `ConfigSchema` that holds the type, default, and validation per key.
- `ConfigSnapshot`: an immutable map for one version: version number, author, time, values.
- `ConfigStore`: keeps the list of snapshots per scope, and an `AtomicReference<ConfigSnapshot>` for the current one. `set` builds a new snapshot (copy-on-write) and swaps it in with CAS.
- `ConfigResolver`: looks a key up through the layers in order (instance → service → environment → default).
- `ConfigWatcher` (Observer): a client registers a callback on keys or prefixes.
- `AuditLog`: who, what, when, old value and new value.

Patterns. Chain of Responsibility (layered lookup), Observer (change listeners), Memento/immutable snapshots (versions and rollback), Strategy for value parsers.

Concurrency and edge cases. Readers never lock: they read the current snapshot reference. Writers race through CAS and retry on conflict, or use compare-by-version so a stale edit is rejected. Validate against the schema before publishing, so a bad value never goes live. A listener that throws must not stop the other listeners.

Extension question. "Add feature flags rolled out to 10% of users." Add a `FlagRule` with a percentage. Hash the user id into 0–99 and compare it, so each user gets a stable result.

Q79. Design AWS Lambda (serverless) — LLD

Requirements. Users register a function: name, runtime (Java, Python), memory, timeout, code location. `invoke(name, payload)` runs it synchronously and returns the result, or runs it asynchronously with retries. Warm environments are reused, and new ones are started (cold start) when none is free. A per-function limit caps how many run at once. Idle environments shut down after a while.

Main classes.

- `FunctionConfig`: a record (name, runtime, memoryMb, timeout, maxConcurrency, codeRef).
- `FunctionRegistry`: create, update and delete functions, with versions.
- `ExecutionEnvironment`: one sandbox. Its state goes `INITIALISING` → `READY` → `BUSY` → `READY` …, and finally `TERMINATED`.
- `RuntimeFactory`: builds an environment for each runtime type.
- `EnvironmentPool`: per function, a deque of warm environments (newest first, so old ones age out).
- `Invoker`: takes a concurrency permit, borrows or creates an environment, runs with a timeout, and returns the environment to the pool.
- `AsyncQueue` + `RetryPolicy` + `DeadLetterQueue` for asynchronous calls.
- `IdleReaper`: a scheduled task that terminates environments idle longer than a limit.

Patterns. Object Pool (warm environments), Factory (runtimes), State (environment lifecycle), Proxy (the `Invoker` hides pooling and limits), Strategy (retry backoff).

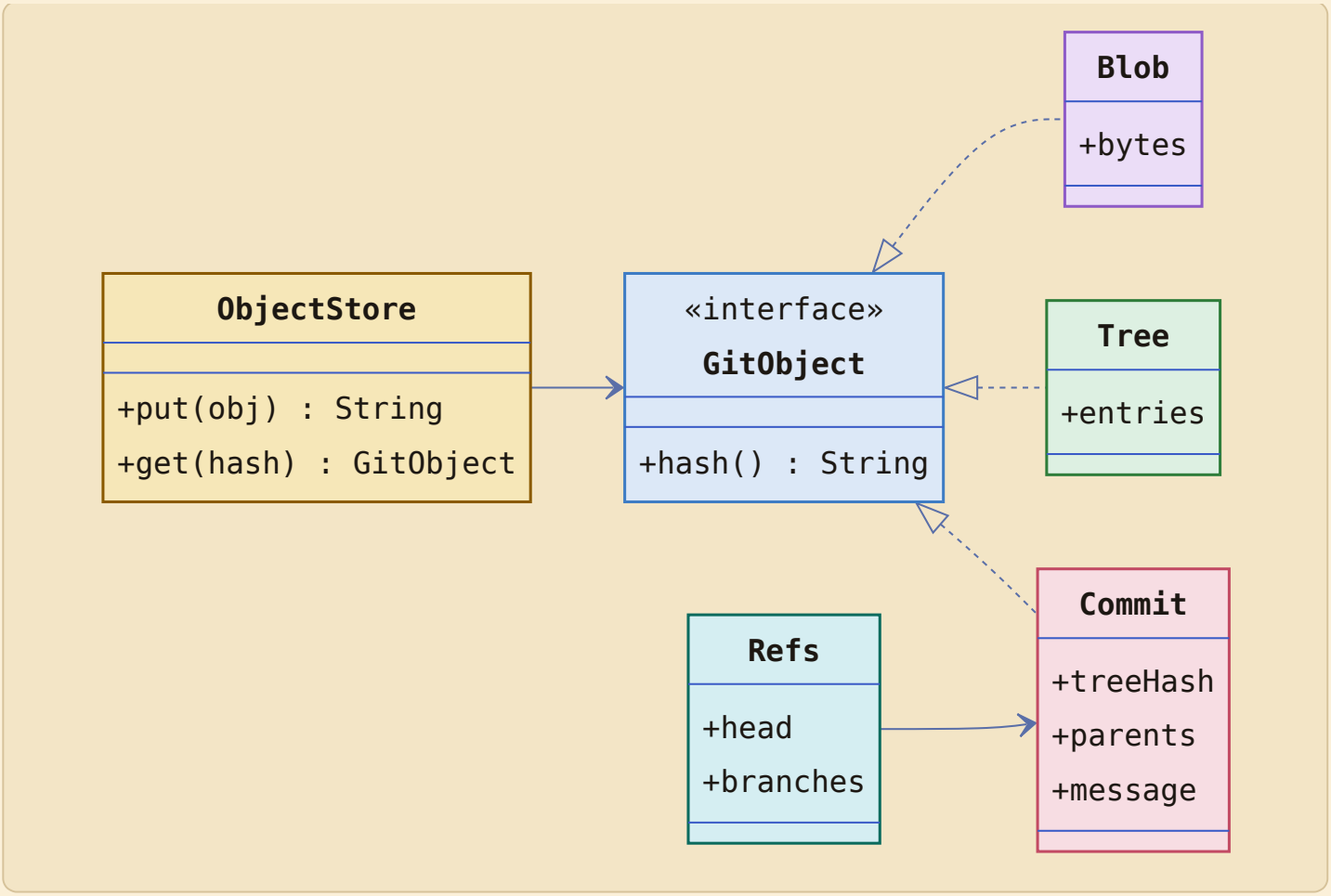
Concurrency and edge cases. A per-function `Semaphore` enforces `maxConcurrency`. Throttle by failing fast, or queue async calls. Run the handler on a virtual thread and wait with `Future.get(timeout)`. On timeout, kill and discard that environment, never reuse it. A crashed environment is removed, not returned. An update to the code marks old warm environments as stale.

Extension question. "Add provisioned concurrency." Pre-create N environments at deploy time, and keep the reaper from removing them.

Q80. Design a version-control / Git-like system

Requirements. Support `init`, `add` (stage files), `commit`, `log`, `branch`, `checkout`, `diff`, and `merge` for a single local repository. Identical content is stored once. History is never changed by normal commands.

Main classes.



- `Blob`: file content. `Tree`: a directory with name → (blob or tree) hash entries. `Commit`: tree hash, parent hashes, author, time, message. All of them are immutable records.
- `ObjectStore`: content-addressed storage, a map from hash (SHA-256 of the content) to object. The same content always gets the same hash, so duplicates cost nothing.
- `Index` (staging area): path → blob hash for the next commit.
- `Refs`: branch name → commit hash, and `HEAD` → a branch name (or a commit, the "detached" state).
- `Repository`: a facade exposing the commands, with each command as a `Command` object for the CLI.

Patterns. Composite (trees of trees), Command (CLI operations), immutable value objects, Facade.

Algorithms and edge cases. `log` follows parent links. `merge` first finds the merge base, the lowest common ancestor of the two commits (BFS from both sides). If one side is the base, just move the branch pointer (a fast-forward). Otherwise do a three-way merge per file and mark conflicts. `checkout` with uncommitted changes must refuse, or warn. Moving a branch is one atomic reference update.

Extension question. "Add `revert`." Create a new commit that undoes the changes of an earlier one. Never rewrite history.

Machine-coding round favourites

Q81. ⋆ Design Splitwise (expense sharing)

Asked at: Swiggy (SDE1; SDE2, twice)

- Entities: `User`, `Group`, `Expense` with a `Split` type (equal, exact, percent), `BalanceSheet` (who owes whom), `SettlementService`.
- Patterns: Strategy/Factory for split types, plus validation that the splits add up to the total.
- What interviewers probe: rounding (give the leftover paise to one person), simplifying debts (greedy matching of the biggest creditor with the biggest debtor), and the command-line input format. → *Full design:* Ch 20 — 20.1 Design Splitwise

Q82. ⋆ Design a task scheduler / distributed task scheduler (LLD)

Asked at: Flipkart (SDE2, Jan 2026)

- Entities: `Task` (id, payload, run-at time, recurrence, retries), `Scheduler` (`schedule`, `cancel`), a `DelayQueue` or `PriorityQueue` ordered by run time, a `Worker` pool, and an `ExecutionRecord`.
- Design: one dispatcher thread waits for the next due task (a `Condition` with a timed wait, woken early when a sooner task arrives) and hands it to a worker pool. A recurring task schedules its next run after finishing.
- What interviewers probe: cancelling a queued task, retries with backoff, and the distributed version (lease a task in storage with a timeout, so a crashed worker's task is picked up again). → *Related design:* Ch 12 — 12.21 Thread pool pattern

Q83. ⋆ Design an in-memory file system (mkdir / ls / find)

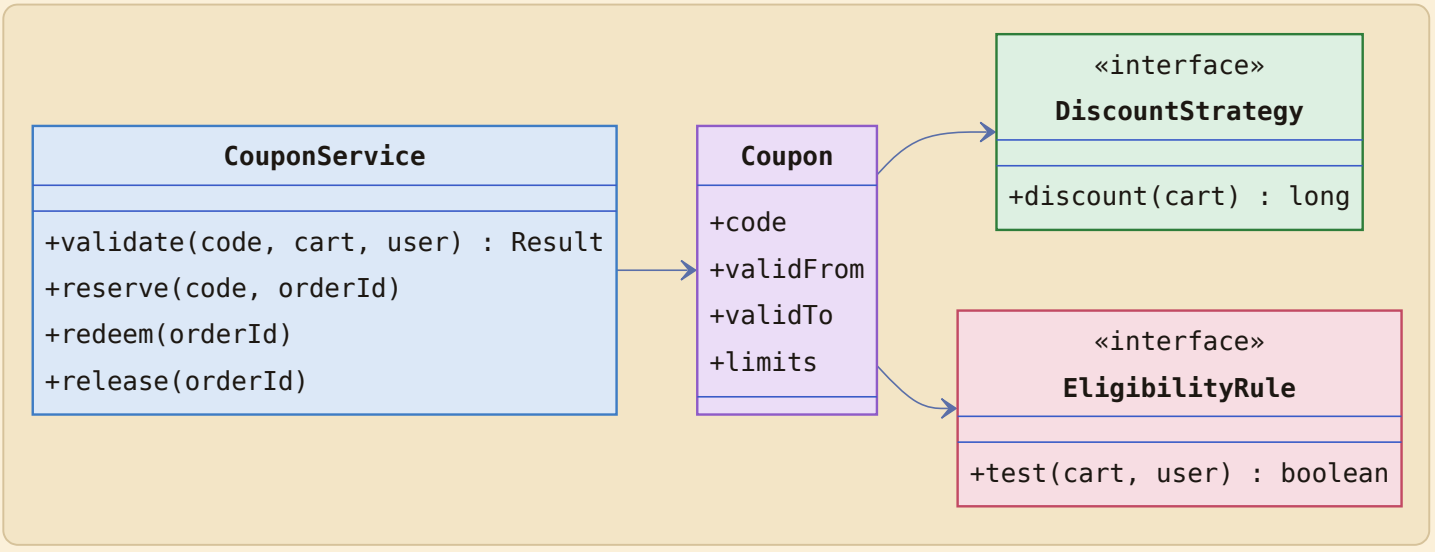
Asked at: ClearTrip and others (Unix `find`)

- Entities: `FileSystemNode` (sealed: `File`, `Directory`), `Path` parsing, and `FileSystem` (`mkdir -p`, `ls`, `addContent`, `readContent`).
- Patterns: Composite for the tree, Specification/Strategy for `find` filters (name, extension, size) combined with and / or.
- What interviewers probe: adding a new filter without changing existing ones, and walking a deep tree without recursion limits. → *Full design:* Ch 21 — 21.8 Design a Unix File Search (`find`); Ch 9 — 9.3 Composite Pattern

Q84. ⋆ Design a coupon / promo-code system

Requirements. Admins create coupons: a code, a discount type (flat ₹100 off, 20% off up to ₹150, buy-one-get-one), a validity window, a total usage limit, and a per-user limit. Eligibility rules include a minimum cart value, certain categories, first order only, and certain user segments. At checkout the user applies a code. The system validates it, computes the discount, and redeems it when payment succeeds. Coupons can or cannot be combined (stacking).

Main classes.



- `DiscountStrategy`: `FlatDiscount`, `PercentDiscount(cap)`, `BuyXGetY`. The result never exceeds the eligible amount.
- `EligibilityRule`: small rules joined with `and` / `or` (Specification). Each failure gives a reason the user can read.
- `Redemption`: a record (code, userId, orderId, status: `RESERVED`, `REDEEMED`, `RELEASED`).
- `UsageCounter`: total and per-user counts.

Patterns. Strategy (discounts), Specification / Chain of Responsibility (eligibility), Factory (build a coupon from an admin request).

Concurrency and edge cases. A "first 1,000 users" coupon must not be used 1,001 times. At checkout, *reserve* a use: increment the counter atomically only while it is below the limit (a CAS loop on an `AtomicInteger`). Move it to `REDEEMED` when payment succeeds, and *release* it on failure or timeout. Make redeeming idempotent per order id. Codes are case-insensitive. Check validity windows with a time zone in mind.

Extension question. "Auto-apply the best coupon." Evaluate every eligible coupon's discount against the cart and pick the biggest, and allow stacking only within the stacking rules.

Q85. ⋆ Design an Event Calendar (Flipkart; Google Calendar)

Asked at: Flipkart

- Entities: `User`, `Event` (time range, organiser, participants), `Team`, `Invite` (accept / reject), `Calendar` view.
- Patterns: Observer for invites, and an interval-overlap check for conflicts.
- What interviewers probe: finding a common free slot for a team, recurring events, and the per-user agenda for a day. → *Full design:* Ch 21 — 21.3 Design an Event Calendar (Flipkart)

Q86. ⋆ Design a LeetCode-like coding platform ("Coding Blox", Flipkart)

Asked at: Flipkart

- Entities: `User` (score), `Contest`, `Question` (difficulty level, score), `Submission`, `Leaderboard`.
- Patterns: Strategy for scoring and question selection, Observer for leaderboard updates.
- What interviewers probe: contest states (created, running, ended), only the creator can start a contest, and ordering the leaderboard. → *Full design:* Ch 21 — 21.4 Design a LeetCode-like Coding Platform

Q87. ⋆ Design a property listing site ("PropertyHunt", ClearTrip)

Asked at: ClearTrip

- Entities: `User`, `Property` (location, price, size, type, status), `Shortlist`, `SearchQuery`.
- Patterns: Specification/Builder for search filters, Strategy for sorting.
- What interviewers probe: combining many optional filters cleanly, and marking a property sold so it leaves search results. → *Full design:* Ch 21 — 21.5 Design a Property Listing Site

Q88. ⋆ Design a loan ledger ("Ledger Co", Navi)

Asked at: Navi

- Entities: `Loan` (principal, rate, years, EMI), `Payment` (lump sum after EMI n), `Bank`, `LedgerService`, and commands `LOAN`, `PAYMENT` and `BALANCE`.
- Patterns: Command for each input line, and pure functions for the EMI and balance maths.
- What interviewers probe: rounding EMIs up, how a lump sum shortens the remaining EMIs, and parsing the input file robustly. → *Full design:* Ch 21 — 21.6 Design a Loan Ledger

Q89. ⋆ Design a Task Management System / Jira / task planner

- Entities: `Task` (title, assignee, status, due date, priority), `Story`, `Bug`, `Sprint`, `User`, `TaskService`.
- Patterns: State for status transitions, Factory for task types, Observer for assignment notifications.
- What interviewers probe: valid transitions per task type, subtasks, and filtering or sorting by many fields. → *Full design:* Ch 21 — 21.1 Design a Task Management System

Q90. ⋆ Design a To-Do list app

Asked at: PhonePe

- A smaller task manager: `TodoItem` (text, done, due date, tags), `TodoList`, add / edit / complete / delete, filters, and an activity log.
- Patterns: Command for actions (gives undo and the activity log for free), Strategy for sorting.
- What interviewers probe: soft delete, undo, and recurring tasks. → *Related design:* Ch 21 — 21.1 Design a Task Management System

Q91. ⋆ Design a Digital Wallet (Paytm / PhonePe)

- Entities: `User`, `Wallet` (balance), `Transaction` (credit, debit, transfer), `Offer`, `Statement`.
- Patterns: Strategy for offers and cashback, Command or a ledger of immutable transactions.
- What interviewers probe: transfers between two wallets without deadlock (lock in id order), money as whole paise, and idempotent transfers. → *Full design:* Ch 21 — 21.2 Design a Digital Wallet Service

Q92. ⋆ Design an Inventory / Warehouse Management System

- Entities: `Product` / `SKU`, `Warehouse`, `StockLevel`, `StockMovement` (in, out, transfer), `Reservation`, `ReorderRule`.

- Patterns: Observer for low-stock alerts, Strategy for picking a warehouse, and reserve-then-commit for orders.
- What interviewers probe: preventing overselling under concurrent orders, and keeping a movement log so stock can be audited. → *Full design:* Ch 21 — 21.7 Design an Inventory Management System

Part B — Concept questions

These are the "explain X" and "X vs Y" questions that come up in a low-level design (LLD) round, usually in the first 10 minutes or while you are coding. Answer in two to four sentences, then give a tiny example from your design.

OOP basics

Q93. ⋆ What are the four pillars of OOP? Explain each with an example. Encapsulation keeps an object's data private and changes it only through methods that check the rules (a `BankAccount` whose `withdraw` refuses overdrafts). **Abstraction** shows *what* something does and hides *how* (a `PaymentGateway` interface with `pay()`). **Inheritance** lets a subclass reuse and specialise a parent (`SavingsAccount extends Account`). **Polymorphism** means one call runs different code depending on the real object type (`account.monthlyFee()`). → *More:* Ch 02 — 2.3–2.6 The four pillars

Q94. ⋆ Encapsulation vs abstraction: what is the difference? Encapsulation is about *protecting state*: fields are private and every change goes through a method that keeps the object valid. Abstraction is about *hiding complexity*: callers depend on a simple contract (an interface or a small public API) and never see the details. Encapsulation is done inside one class, while abstraction is a design decision about what to expose at all. → *More:* Ch 02 — 2.3 Encapsulation, 2.4 Abstraction

Q95. ⋆ Compile-time vs runtime polymorphism; overloading vs overriding. Overloading means several methods share one name but have different parameter lists. The compiler picks one from the declared argument types, so this is compile-time (static) polymorphism. **Overriding** means a subclass redefines an inherited instance method with the same signature. The JVM picks the version from the object's actual class while the program runs (dynamic dispatch), so this is runtime polymorphism. Return types may be covariant (a subtype) when overriding, and the override cannot reduce visibility or throw broader checked exceptions. → *More:* Ch 02 — 2.7 Overloading vs overriding

Q96. Can you override a static or a private method? No. A `static` method with the same signature in a subclass *hides* the parent's method, and the call is chosen by the declared type, not the object. A `private` method is invisible to subclasses, so a method with the same name there is a brand-new method. `final` methods cannot be overridden either. → *More:* Ch 02 — 2.7 Overloading vs overriding

Q97. ⋆ Interface vs abstract class: when do you use which? Use an **interface** for a capability that unrelated classes can share (`Comparable`, `PaymentMethod`). A class can implement many interfaces, and an interface has no instance state. Use an **abstract class** when closely related classes share state or a fixed algorithm skeleton (constructor, protected fields, Template Method). A class can extend only one class. In LLD rounds, default to interfaces and add an abstract base class only when real shared code appears. → *More:* Ch 02 — 2.9 Abstract classes vs interfaces

Q98. What are default, static and private methods in interfaces? A **default** method (Java 8) has a body that implementing classes inherit, so you can add methods to an interface without breaking existing implementers. A **static** interface method is a helper called as `Shape.unit()` and is not inherited. **Private** interface methods (Java 9) share code between default methods. Interfaces still cannot hold instance fields, only constants. → *More:* Ch 02 — 2.8 Interfaces

Q99. Can an abstract class have a constructor? Can it be instantiated? It can have constructors, and they run when a subclass is created (via `super(...)`) to initialise the shared fields. You cannot write `new` on it directly, although an anonymous subclass works. Since Java 25 (flexible constructor bodies, JEP 513), a constructor may run statements *before* calling `super(...)`: it can validate or prepare arguments and assign its own fields, but it cannot read fields, call instance methods or otherwise use `this`. → *More:* Ch 02 — 2.2 Classes, objects, constructors

Q100. ⋆ Composition vs inheritance: why "favor composition"? Inheritance ties a subclass to its parent's internals at compile time, so a parent change can break every child (the "fragile base class"), and Java allows only one parent. Composition means an object *holds* collaborators behind interfaces and delegates to them. You can swap them at runtime and combine behaviours without an explosion of subclasses. Use

inheritance only for a true, stable IS-A relationship where the subclass honours the parent's contract. → *More:* Ch 02 — 2.11 Composition over inheritance

Q101. IS-A vs HAS-A. IS-A is inheritance or interface implementation: a `Car` *is a* `Vehicle`, so it can be used wherever a `Vehicle` is expected. HAS-A is a field reference (association, aggregation or composition): a `Car` *has an* `Engine`. If you can't say "every X is a Y and can stand in for it" truthfully, use HAS-A. → *More:* Ch 03 — 3.8 Decision guide

Q102. ⋆ What is the diamond problem? How does Java handle multiple inheritance? The diamond problem appears when class D inherits the same method from two parents B and C that both override A's version, so it is unclear which one D gets. Java avoids it for state by allowing only one superclass. With interfaces, if two default methods clash, the class *must* override the method and may pick one with `B.super.method()`. A class method always beats an interface default. → *More:* Ch 02 — 2.8 Interfaces

Q103. ⋆ Explain the `equals()` and `hashCode()` contract. Two objects that are `equals` must return the same `hashCode`. The reverse is not required, because collisions are allowed. Hash collections find the bucket with `hashCode` and then compare with `equals`. If you override only `equals`, a `HashSet` will hold "duplicates" and `map.get` will miss keys. Build both from the same fields, never from mutable fields of a key while it sits in a map, and use records for value objects. → *More:* Ch 02 — 2.12 equals/hashCode contract

Q104. ⋆ How do you make a class immutable? What do records give you? Make the class `final`, all fields `private final`, set them once in the constructor, provide no setters, and make defensive copies of mutable inputs and outputs (`List.copyOf`). A **record** gives final fields, a canonical constructor, accessors, `equals`, `hashCode` and `toString` in one line. You still copy mutable components in a compact constructor.

```
record Order(String id, List<String> items) {
    Order { items = List.copyOf(items); }   // defensive copy, also rejects nulls
}
```

→ *More:* Ch 02 — 2.13 Immutability and records

Q105. Explain the access modifiers, and `static` vs `final`. `private` means this class only. *No modifier* (package-private) means the same package. `protected` means the package plus subclasses. `public` means everyone. `static` means "belongs to the class, not to each object" (one shared counter, a factory method). `final` means "cannot change": a final variable is assigned once, a final method cannot be overridden, a final class cannot be extended. They are independent, so `static final` is a class-level constant. → *More:* Ch 02 — 2.2 and 2.14

Q106. ⋆ Coupling vs cohesion. Cohesion measures how closely the parts of one class belong together. Aim high: one clear job. **Coupling** measures how much one class depends on others' details. Aim low: depend on small interfaces. High cohesion plus low coupling means a change stays in one place and is easy to test. These two yardsticks sit behind almost every principle and pattern. → *More:* Ch 02 — 2.16 Coupling and cohesion

Relationships and UML

Q107. ⋆ Association vs aggregation vs composition. Association means two classes know about each other (`Doctor`–`Patient`). **Aggregation** is a weak whole–part link where the part can live on its own and be shared (`Team` has `Player`s). **Composition** is a strong whole–part link where the part is created and destroyed with the whole and belongs to only one owner (`Order` owns its `OrderLine`s). In UML, aggregation is a hollow diamond and composition is a filled diamond on the whole's side. → *More:* Ch 03 — 3.2–3.4

Q108. Dependency vs association. A **dependency** is a short-lived use: the class receives another as a method parameter, local variable or return value (a dashed arrow in UML). An **association** is a lasting structural link stored in a field (a solid line). If deleting the field removes the link, it was an association. → *More:* Ch 03 — 3.5 Dependency

Q109. What is UML? Class vs sequence vs activity vs state diagrams. UML (Unified Modeling Language) is a standard notation for drawing software designs. **Class diagrams** show static structure: classes, fields, methods, relationships. **Sequence diagrams** show which objects call which, in time order, for one scenario. **Activity diagrams** are flowcharts of a process. **State machine diagrams** show an object's states and the events that move it between them, as for a vending machine or an order. → *More:* Ch 03 — 3.9–3.14

Q110. Draw a class diagram (or use-case diagram) for X. Start with a **use-case diagram** only if actors matter: actors as stick figures, use cases as ovals ("Park vehicle", "Pay fee"). For the class diagram, list the nouns from the requirements as classes, put two or three key fields and methods in each, and draw only the relationships that carry meaning, with multiplicities (`1`, `0..*`). Mark interfaces and enums. Keep it to what you will actually code. → *More:* Ch 03 — 3.10 Use-case, 3.11 Class diagrams

Q111. ⋆ How do you approach an LLD interview, and how much UML should you draw? First clarify the requirements and agree on the scope (core use cases, what is out of scope). Then find the entities, then draw a lean class diagram with the key interfaces. Walk through one main flow, and only then write code with the patterns that fit. Add concurrency and extensibility last. Draw just enough UML to agree on the shape, usually one class diagram and maybe one sequence or state sketch. Interviewers judge the reasoning and the code, not notation perfection. → *More:* Ch 01 — 1.4 The step-by-step answer framework

Q112. ⋆ LLD vs HLD: what is the difference? HLD (high-level design) chooses the services, databases, caches, queues and how they scale across machines, with boxes and arrows. LLD (low-level design) works inside one service: classes, interfaces, relationships, patterns, method signatures and thread safety, often ending in working code. HLD asks "which components and how do they scale?", LLD asks "how is this component built cleanly?" → *More:* Ch 01 — 1.1 What this round is

Q113. ⋆ Machine-coding round: how do you structure code in 90 minutes, and what do evaluators look for? They want working, runnable code that shows the core flows end to end with a demo `main`. They also look for clean separation into models, services and strategies, sensible names, input validation with clear errors, and an easy path to the extension they will ask for. Keep the code in memory (maps for storage), get a vertical slice working early, then add features one at a time. Part C below gives a minute-by-minute plan. → *More:* Ch 21 — 21.0 How the machine-coding round works

SOLID

Q114. ⋆ Explain SOLID with examples. **S**ingle Responsibility: one reason to change. **O**pen/Closed: add behaviour with new classes, not by editing tested ones. **L**iskov Substitution: subtypes must work wherever the parent is expected. **I**nterface Segregation: small role-based interfaces. **D**ependency Inversion: high-level code depends on abstractions, not on concrete details. Together they keep changes local and code testable. Apply them where change is likely, not everywhere. → *More:* Ch 04 — SOLID principles

Q115. ⋆ Single Responsibility Principle with an example. A class should have one reason to change, meaning one actor or concern drives its edits. An `Invoice` that calculates totals, prints PDFs and saves to the database changes for finance, design and DBA reasons. Split it into `Invoice` (rules), `InvoicePrinter` and `InvoiceRepository`. The test: describe the class without the word "and". → *More:* Ch 04 — 4.2 SRP

Q116. ⋆ Open/Closed Principle with an example. Code should be open for extension but closed for modification. A `FareCalculator` with a growing `switch (vehicleType)` must be edited for every new type. Instead, define a `PricingStrategy` interface and add `EvPricing` as a new class while the calculator stays untouched. Strategy, Decorator and Factory registries are the usual tools. → *More:* Ch 04 — 4.7 OCP

Q117. ⋆ Liskov Substitution: explain the rectangle/square problem. LSP says a subtype must keep the parent's promises: no stronger preconditions, no weaker postconditions, no surprise exceptions. If `Square extends Rectangle`, then `setWidth(5)` silently changes the height too, so code that expects area = width × height breaks. The fix is not to make Square a subtype of a mutable Rectangle. Use a common `Shape` interface or immutable shapes instead. → *More:* Ch 04 — 4.8 LSP

Q118. Interface Segregation with an example. Clients should not depend on methods they don't use. A fat `Machine` interface with `print`, `scan` and `fax` forces a basic printer to throw `UnsupportedOperationException`. Split it into `Printer`, `Scanner` and `Fax`, and let a multifunction device implement all three. → *More:* Ch 04 — 4.9 ISP

Q119. ⋆ Dependency Inversion vs Dependency Injection vs Inversion of Control. **DIP** is a principle: high-level policy and low-level details both depend on an abstraction that the high-level side owns (`OrderService` → `PaymentGateway` interface ← `StripeGateway`). **Dependency Injection** is a technique: dependencies are handed in from outside, usually through the constructor, instead of being created with `new`. **IoC** is the broader idea that a framework or container calls your code and wires objects ("don't call us, we'll call you"). DI is one way to do IoC, and both help you apply DIP. → *More:* Ch 04 — 4.10 DIP

Q120. Can SOLID be overdone? How do you use it in an interview? Yes. Interfaces with a single implementation that will never vary, factories for trivial objects and ten-class hierarchies for a two-case `if` are over-engineering. In an interview, apply SOLID at the points the interviewer is likely to

extend (pricing, notification channels, strategies) and say why. Leave stable parts simple. → *More:* Ch 04 — 4.15 Over-engineering

Design principles and clean code

Q121. ⋆ What are DRY, KISS and YAGNI? DRY (Don't Repeat Yourself): each piece of knowledge, such as a rule or formula, lives in one place. **KISS** (Keep It Simple): choose the simplest design that meets the need. **YAGNI** (You Aren't Gonna Need It): don't build features or hooks for imagined future needs. They balance each other, because too much DRY or flexibility hurts simplicity. → *More:* Ch 05 — §1–3

Q122. Law of Demeter and Tell, Don't Ask. The **Law of Demeter** says a method should talk only to its direct friends (its fields, its parameters, objects it creates), not to strangers through chains like `order.getCustomer().getWallet().debit(x)`. **Tell, Don't Ask** says tell an object to do the work (`order.pay(x)`) instead of pulling its data out and deciding for it. Both keep logic next to its data and cut coupling. → *More:* Ch 05 — §4–5

Q123. What does "program to an interface, not an implementation" mean? Declare fields, parameters and return types with the abstraction (`List`, `NotificationSender`), and create the concrete class in one place (a factory or the composition root). Callers then work with any implementation, including test fakes, without changes. → *More:* Ch 05 — §7

Q124. What does "encapsulate what varies" mean? Find the parts of the design that are likely to change, such as pricing rules, payment providers or eviction policies. Hide each one behind an interface so that the stable code around it doesn't change when they do. Most behavioural patterns (Strategy, State, Observer) are this idea applied to one kind of change. → *More:* Ch 05 — §8

Q125. What are the GRASP principles? GRASP (General Responsibility Assignment Software Patterns) are nine guidelines for deciding which class gets which job. **Information Expert**: give the job to the class that has the data. **Creator**: the class that holds or aggregates B creates B. **Controller**: one object receives system events. **Low Coupling** and **High Cohesion**. **Polymorphism**. **Pure Fabrication**: invent a service class when no domain class fits. **Indirection**. **Protected Variations**. → *More:* Ch 05 — §16 GRASP

Q126. DDD basics: entity vs value object, and what is an aggregate? An **entity** has an identity that lasts while its attributes change (a `Customer` with an ID). A **value object** is defined only by its values, is immutable and is compared by value (`Money`, `Address`). An **aggregate** is a cluster of objects changed as one unit through a single **root** (`Order` with its lines), which protects the cluster's rules. Outsiders hold references only to the root. → *More:* Ch 05 — §17 DDD essentials

Q127. What does "fail fast" mean? Detect invalid input or state as early as possible and stop with a clear exception, instead of carrying bad data forward until it fails somewhere confusing. Examples are `Objects.requireNonNull` in constructors, validating in a record's compact constructor and rejecting an invalid state transition right away. → *More:* Ch 05 — §13 Fail Fast

Q128. ⋆ What are anti-patterns? What is a God class? What is over-engineering? An **anti-pattern** is a common "solution" that causes more problems than it solves. A **God class** knows and does too much (a `ParkingLotManager` with 40 methods), so every change touches it and it can't be tested in isolation. **Over-engineering** adds layers, patterns and configurability nobody asked for. Other anti-patterns are Singleton abuse, anemic models, magic numbers and copy-paste code. → *More:* Ch 08 — 8.6 Anti-patterns

Q129. ⋆ What are code smells? Name a few and their refactorings. A **code smell** is a surface sign of a deeper design problem. **Long Method** → Extract Method. **Large Class** → Extract Class. **Long Parameter List** → Introduce Parameter Object. **Switch on type** → Replace Conditional with Polymorphism. **Feature Envy** (a method that uses another class's data more than its own) → Move Method. **Duplicate Code** → Extract and reuse. **Refactoring** means changing structure in small, behaviour-preserving steps, with tests as the safety net. → *More:* Ch 07 — 7.2 Code smells, 7.4 Lookup table

Q130. What makes code "clean" (naming, functions, error handling)? Names reveal intent (`activeBookings`, not `list2`). Functions are small, do one thing at one level of abstraction and take few arguments. Errors are reported with exceptions that carry context, not with error codes or `null`. Comments explain *why*, not *what*. Clean code reads like the problem description. → *More:* Ch 06 — Clean code

Q131. Checked vs unchecked exceptions: which do you use? Checked exceptions extend `Exception` (but not `RuntimeException`). The compiler forces callers to catch or declare them, which suits recoverable conditions from outside the program, such as `IOException`. **Unchecked** exceptions extend `RuntimeException` and usually signal programming errors or broken rules (`IllegalArgumentException`, a custom `SeatAlreadyBookedException`).

Modern Java design prefers unchecked domain exceptions with clear names, plus checked ones only where the caller can really recover. → *More:* Ch 06 — §6 Error Handling

Q132. Unit testing: mocks vs stubs, and how do you make a design testable? A **stub** returns canned answers so the code under test can run. A **mock** also *verifies* interactions (for example, "`send` was called once"). A **fake** is a working lightweight version, such as an in-memory repository. Designs are testable when dependencies are injected through interfaces, time and randomness are injected (`Clock`), and logic sits apart from I/O. → *More:* Ch 06 — §17 Testing strategies

Creational patterns

Q133. ⋆ What is a design pattern? What are the three GoF categories? A design pattern is a named, reusable solution to a recurring design problem. It is a template you adapt, not a library you import. The "Gang of Four" (GoF) book lists 23 patterns in three groups. **Creational** patterns deal with how objects are made (Singleton, Factory Method, Abstract Factory, Builder, Prototype). **Structural** patterns deal with how objects are composed (Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy). **Behavioral** patterns deal with how objects communicate and share work (Strategy, Observer, State, Command and others). → *More:* Ch 08 — 8.1–8.3

Q134. ⋆ Singleton: how do you make it thread-safe? Use an **enum** (simplest, and safe against reflection and serialization), or the **holder idiom**: a static nested class that the JVM loads, thread-safely, on first use. **Double-checked locking** also works but needs a `volatile` field. Without `volatile`, another thread can see a half-constructed object.

```
final class Config {
    private Config() {}
    private static class Holder { static final Config INSTANCE = new Config(); }
    static Config get() { return Holder.INSTANCE; }
}
```

→ *More:* Ch 08 — 8.8 Singleton

Q135. How can a Singleton be broken? Reflection can call the private constructor. Deserialization creates a new instance unless you add `readResolve`. Cloning breaks it if the class is `Cloneable`. Each class loader can also hold its own copy. An enum singleton blocks the first three. → *More:* Ch 08 — 8.8 Singleton

Q136. ⋆ Why is Singleton often called an anti-pattern? It is global mutable state in disguise. It hides dependencies (callers reach out with `getInstance()` instead of receiving them), it makes unit tests hard because you can't swap or reset it, and it creates concurrency hot spots. Prefer creating one instance at the composition root (or a DI container's singleton scope) and injecting it. → *More:* Ch 08 — 8.8 Singleton

Q137. ⋆ Simple Factory vs Factory Method vs Abstract Factory. A **Simple Factory** is one method with a `switch` that returns the right subtype (`VehicleFactory.create(type)`). It is an idiom, not a GoF pattern. **Factory Method** lets subclasses decide which product to create by overriding a creation method. **Abstract Factory** creates *families* of related products that must match (a `WindowsUiFactory` makes `WinButton` + `WinCheckbox`). In interviews, a simple factory or a `Map<Type, Supplier<T>>` registry is usually enough. → *More:* Ch 08 — 8.9–8.11, 8.15

Q138. ⋆ When would you use Builder? Use it when an object has many parameters, several of them optional (the "telescoping constructor" problem), or when you want to validate everything before building an immutable object. The builder collects values with readable chained calls and `build()` checks them once. Records with a few fields don't need one.

```
var pizza = Pizza.builder().size(LARGE).cheese(true).topping("olive").build();
```

→ *More:* Ch 08 — 8.12 Builder

Q139. Prototype pattern; shallow vs deep copy. Prototype creates new objects by copying a configured existing one, which helps when construction is expensive or the configuration is complex. A **shallow copy** copies fields as they are, so the copy shares nested mutable objects with the original. A **deep copy** also copies those nested objects, so changes don't leak between them. In Java, prefer copy constructors or factory methods over `clone()`. → *More:* Ch 08 — 8.13 Prototype

Q140. What is an object pool, and when is it worth it? A pool keeps a fixed set of expensive, reusable objects (database connections, threads). It lends one out and takes it back, so you avoid repeated

creation and cap the total. It needs a thread-safe borrow/return (often a `BlockingQueue` or a `Semaphore`) and must reset each object's state on return. For cheap objects it only adds bugs. → *More:* Ch 08 — 8.14 Object Pool

Structural patterns

Q141. ⋆ Adapter: when would you use it? Adapter vs Facade vs Bridge. An **Adapter** converts one interface into another that the client expects. A typical use is wrapping a third-party `LegacyPaymentApi` so it implements your `PaymentGateway`. A **Facade** gives a *simpler* front to a whole subsystem (`HomeTheater.watchMovie()`) without changing the interfaces. A **Bridge** splits an abstraction from its implementation up front, so the two can vary independently (`Shape` × `Renderer`). Adapter fixes a mismatch after the fact, Facade simplifies, and Bridge is planned in from the start. → *More:* Ch 09 — 9.1, 9.2, 9.5, 9.8

Q142. ⋆ Decorator vs Proxy (and vs inheritance). Both wrap an object behind the same interface. A **Decorator** *adds* behaviour, and decorators can be stacked at runtime (`new Logging(new Compressing(stream))`), which avoids a subclass for every combination. A **Proxy** *controls access* to the same behaviour: lazy loading, caching, security checks, remote calls. Usually the proxy, not the client, decides whether to create the real object. → *More:* Ch 09 — 9.4 Decorator, 9.7 Proxy

Q143. ⋆ Composite pattern (file system / org chart). Composite treats single objects and groups of objects through one interface, so the client doesn't care which it has. A `File` and a `Directory` both implement `FileSystemNode.size()`, and the directory sums its children recursively. Use it for any tree: menus, org charts, expression trees. → *More:* Ch 09 — 9.3 Composite

Q144. What is the Flyweight pattern? Flyweight saves memory when there are huge numbers of similar objects. It shares the *intrinsic* (unchanging, shareable) state and passes the *extrinsic* (per-use) state in from outside. Examples: one shared `Glyph` per character in a text editor, with the position stored separately, or tree types shared across a million trees in a game. `Integer.valueOf` caching small values is a JDK example. → *More:* Ch 09 — 9.6 Flyweight

Q145. How does Java support Proxy at runtime (dynamic proxies, Spring AOP)? `java.lang.reflect.Proxy` creates, at runtime, a class that implements given interfaces and sends every call to an `InvocationHandler`. That is how logging, transactions and retries get added around calls. Spring AOP builds either a JDK proxy (interfaces only) or a generated subclass (CGLIB). Plain Spring uses a JDK proxy when the bean has an interface, while Spring Boot defaults to CGLIB subclasses for everything. Either way the advice lives in the proxy, so a `@Transactional` method called from inside the same class (through `this`) bypasses it. → *More:* Ch 09 — 9.7 Proxy

Behavioral patterns

Q146. ⋆ Strategy pattern: where did you use it? Strategy puts each interchangeable algorithm behind an interface and lets the context pick one at runtime. Examples are a parking fee per vehicle type, Splitwise's equal/exact/percent splits, cache eviction (LRU/LFU) and payment methods. It removes `switch` statements and follows Open/Closed. In modern Java a strategy is often just a lambda.

```
interface SplitStrategy { Map<User, Money> split(Money total, List<User> users); }
```

→ *More:* Ch 11 — 11.3 Strategy

Q147. ⋆ Strategy vs State. They have the same class shape but a different intent. With **Strategy**, the *client* picks the algorithm, and strategies don't know about each other. With **State**, the *object itself* changes behaviour as its internal state changes, and the states usually trigger the transitions (`IdleState` → `HasCoinState`). Ask "who chooses?": the caller means Strategy, the object's own lifecycle means State. → *More:* Ch 11 — 11.8 Choose the right behavioral pattern

Q148. Strategy vs Template Method. **Template Method** fixes the steps of an algorithm in a base class and lets subclasses fill in some of them (inheritance, chosen at compile time). **Strategy** swaps the whole algorithm through composition, at runtime. Prefer Strategy when variants change independently or need to be combined. Template Method suits a stable skeleton with small hooks. → *More:* Ch 11 — 11.4 Template Method

Q149. ⋆ Observer pattern; Observer vs Pub-Sub. In **Observer**, a subject keeps a list of observers and calls them directly when its state changes (a stock price notifying displays). The subject knows its observers' interface, and delivery is usually synchronous and in-process. **Pub-Sub** puts a broker or event bus between them: publishers send to a topic without knowing the subscribers, and delivery can be asynchronous and cross-process. Pub-Sub decouples more, at the cost of extra infrastructure. → *More:* Ch 11 — 11.1 Observer

Q150. ⋆ Command pattern: how does it give you undo/redo? Command turns a request into an object with `execute()` and `undo()`, so it can be queued, logged, retried or reversed. For undo/redo, push each executed command on an undo stack. Undo pops it, calls `undo()` and pushes it on the redo stack. Any new command clears the redo stack. Text editors, remote controls and job queues use it. → *More:* Ch 10 — 10.2 Command

Q151. Chain of Responsibility (middleware, approval flows). A request passes along a chain of handlers. Each one either handles it or passes it to the next, so the sender doesn't know who will deal with it. Examples are expense approval (manager → director → CFO by amount), ATM cash dispensing by note value, and servlet filters or HTTP middleware (auth → rate limit → logging). Handlers can be added or reordered without changing the client. → *More:* Ch 10 — 10.1 Chain of Responsibility

Q152. Iterator, Mediator, Memento, Interpreter, Visitor: when would you use each? Iterator: walk a collection without exposing how it is stored. **Mediator**: objects talk through a hub instead of to each other (an air-traffic control tower, a chat room). **Memento**: save and restore an object's state without breaking encapsulation (snapshots for undo). **Interpreter**: evaluate sentences of a small grammar (rule expressions). **Visitor**: add new operations to a stable class hierarchy without editing it (export or tax calculation over node types). In modern Java, sealed types with pattern-matching `switch` often replace Visitor. → *More:* Ch 10 — 10.3–10.6, Ch 11 — 11.5 Visitor

Q153. ⋆ State pattern (vending machine / ATM / elevator). Each state becomes a class that implements the same interface (`insertCoin`, `selectItem`, `dispense`). The machine delegates to its current state object, and states switch the machine to the next state. This replaces large `if (state == ...)` blocks, makes illegal actions explicit per state, and lets you add a state without touching the others. A small enum-based state machine is fine when the states have little behaviour. → *More:* Ch 11 — 11.2 State

Q154. What are Null Object, Repository, Specification and MVC? Null Object: a do-nothing implementation (`NoDiscount`) used instead of `null`, which removes null checks. **Repository**: a collection-like interface for storing and finding aggregates (`BookingRepository.findById`) that hides the storage. **Specification**: a business rule as a combinable object (`isAvailable.and(inCity)`). **MVC** (Model–View–Controller): the model holds data and rules, the view displays it, and the controller turns input into model updates. → *More:* Ch 11 — 11.7 Beyond GoF

Q155. ⋆ Which patterns would you use for a parking lot, an elevator or a vending machine? Parking lot: Strategy for pricing and spot allocation, Factory for vehicles or spots, Singleton or injected for the lot, Observer for display boards. **Elevator**: State for each car (moving/idle/doors open), Strategy for the dispatch algorithm, Command for requests. **Vending machine**: State for idle/has-money/dispensing, and Chain of Responsibility for giving change. Name a pattern only when you can say what change it absorbs. → *More:* Ch 22 — 22.9 Case-study map

Q156. How have lambdas and modern Java changed classic patterns? Patterns with a single method, such as Strategy, Command, Observer callbacks and Template Method hooks, are often just functional interfaces with lambdas (`Comparator`, `Runnable`, `Function`). Records cover simple value objects, and sealed interfaces with pattern-matching `switch` give exhaustive case handling in place of many Visitor uses. The *ideas* stay the same, with less boilerplate. → *More:* Ch 11 — 11.6 Quick-reference summary

Concurrency

Q157. ⋆ Process vs thread. A **process** is a running program with its own memory space. Processes are isolated and costly to create, and they communicate through IPC (inter-process communication: pipes, sockets). A **thread** is a path of execution *inside* a process. Threads share the heap with their siblings but each has its own stack, so they are cheap to switch and communicate through shared memory, which is why they need synchronisation. → *More:* Ch 12 — 12.2

Q158. ⋆ Concurrency vs parallelism. Concurrency means dealing with many tasks over the same period by interleaving them. It is about structure, and it works on one core. **Parallelism** means literally running tasks at the same instant on several cores. It is about speed. A web server handling 10,000 waiting connections is concurrent, while a multi-core merge sort is parallel. → *More:* Ch 12 — 12.1

Q159. What are the states in a thread's lifecycle? `Thread.State` has six values. **NEW** means created but not started. **RUNNABLE** means running or ready to run. **BLOCKED** means waiting to enter a `synchronized` monitor. **WAITING** means waiting without a time limit (`wait()`, `join()`, `LockSupport.park`). **TIMED_WAITING** means waiting with a timeout (`sleep(ms)`, `wait(ms)`). **TERMINATED** means finished. → *More:* Ch 12 — 12.3

Q160. Why call `start()` and not `run()`? `start()` asks the JVM to create a new thread, which then calls `run()`. Calling `run()` directly just executes the method on the *current* thread, so nothing runs concurrently. Calling `start()` twice on the same thread throws `IllegalThreadStateException`. → *More:* Ch 12 — 12.4

Q161. ⋆ Runnable vs Callable; ExecutorService; CompletableFuture. A `Runnable` returns nothing and can't throw checked exceptions. A `Callable<V>` returns a value and may throw. An `ExecutorService` runs tasks on a managed pool and hands back a `Future` for the result. `CompletableFuture` adds non-blocking chaining and combining (`thenApply`, `thenCombine`, `allOf`) and error handling. Always shut executors down. Since Java 19, `ExecutorService` is `AutoCloseable`, so try-with-resources works: `close()` shuts the executor down and waits for submitted tasks to finish. → *More:* Ch 12 — 12.4

Q162. ⋆ What are virtual threads, and when should you not use them? Virtual threads (final in Java 21) are cheap JVM-managed threads. When one blocks on I/O it unmounts from its OS carrier thread, so you can run one per request or task, even millions. They help blocking, I/O-heavy work, not CPU-bound work. Don't pool them, and limit access to scarce resources with a `Semaphore`. Since JDK 24 (JEP 491), blocking inside `synchronized` or in `Object.wait()` no longer pins the carrier. Pinning remains in rare cases: blocking while a native frame is on the stack, and blocking in class loading or a class initializer. → *More:* Ch 12 — 12.4

Q163. ⋆ What is a race condition? What is a critical section? A **race condition** happens when the result depends on the timing of threads touching shared mutable data. A classic case is `count++`, which is read–modify–write: two threads read 5 and both write 6. Another is "check-then-act", where two users both see seat A1 free. A **critical section** is the code that touches the shared state and must run by one thread at a time (or be made atomic). → *More:* Ch 12 — 12.5

Q164. ⋆ What does `volatile` do? What is happens-before (the Java Memory Model)? The Java Memory Model (JMM) defines when one thread's writes become visible to another. **Happens-before** is its guarantee: if A happens-before B, then B sees A's effects. Releasing a lock happens-before the next acquire of it, `Thread.start` and `join` create the edge too, and so do volatile writes and reads. `volatile` gives visibility and ordering for one variable but **not** atomicity, so `volatile int count; count++` is still a race. It is good for flags like `running = false`. → *More:* Ch 12 — 12.6

Q165. ⋆ `synchronized` vs `ReentrantLock`. Both give mutual exclusion and are **reentrant**, meaning the thread holding the lock can enter it again. `synchronized` is simpler and releases automatically. `ReentrantLock` adds `tryLock` with a timeout, interruptible locking, optional fairness and several `Condition`s, but you must `unlock()` in `finally`. Start with `synchronized` and switch when you need those features.

```
lock.lock();
try { /* critical section */ } finally { lock.unlock(); }
```

→ *More:* Ch 12 — 12.7, 12.8

Q166. ReadWriteLock and StampedLock: when do you use them? A `ReentrantReadWriteLock` lets many readers in at once but a writer gets exclusive access, which helps when reads far outnumber writes (a config cache). `StampedLock` adds **optimistic reads**: read without locking, then `validate(stamp)`, and retry under a real read lock if a write happened. It is faster for read-heavy data but not reentrant and harder to use correctly. → *More:* Ch 12 — 12.10

Q167. ⋆ Semaphore vs mutex. A **mutex** (lock) lets exactly one thread in and has an owner: only the locker should unlock. A **semaphore** holds N permits, so up to N threads proceed at once (limiting a pool to 10 connections). It has no owner, so one thread can release a permit another acquired, which is useful for signalling. A binary semaphore looks like a mutex but lacks ownership and reentrancy. → *More:* Ch 12 — 12.12 Semaphores

Q168. `wait()`/`notify()` vs `Condition`; `notify()` vs `notifyAll()`. `wait()` releases the object's monitor and sleeps until notified. It must be called while holding the monitor (`synchronized`), and always inside a `while` loop that re-checks the condition, because of spurious wakeups. `notify()` wakes one arbitrary waiter, `notifyAll()` wakes all. `notifyAll` is safer when waiters wait for different conditions. A `Condition` from a `Lock` does the same job, but one lock can have several conditions (`notFull`, `notEmpty`), so you wake only the right group. → *More:* Ch 12 — 12.13 Condition variables

Q169. ⋆ **`wait()` vs `sleep()`.** **`Object.wait()` releases** the monitor, must be called inside `synchronized` (otherwise it throws `IllegalMonitorStateException`), and returns on `notify`/`notifyAll`, a timeout, an interrupt or a spurious wakeup. Use it to wait for a condition. `Thread.sleep()` is a static method that pauses the current thread for a time and **keeps** any locks it holds. Use it for delays, not coordination. Both throw `InterruptedException`. → *More:* Ch 12 — 12.3, 12.13

Q170. ⋆ **`CountDownLatch` vs `CyclicBarrier`.** A `CountDownLatch` lets threads wait until a count reaches zero. It is one-shot, and the threads counting down are usually different from the waiters ("start when all 5 services are ready"). A `CyclicBarrier` makes a fixed group of threads wait for *each other* at a point and can be reused round after round, with an optional barrier action ("each simulation step"). `Phaser` is the flexible, dynamic version. → *More:* Ch 12 — 12.14

Q171. ⋆ **What is CAS? How do atomics like `AtomicInteger` work?** CAS (compare-and-swap) is a CPU instruction: "set the value to new **only if** it still equals expected", which is atomic and lock-free. Atomic updates are a CAS loop: read, compute, CAS, and retry if another thread changed the value (`updateAndGet` works exactly like this, and on x86 the JIT turns `incrementAndGet` into a single atomic add instruction). That beats locks under low contention, and `LongAdder` spreads hot counters across cells under high contention. Watch for the **ABA problem** (a value changes A→B→A unnoticed). `AtomicStampedReference` solves it with a version stamp. → *More:* Ch 12 — 12.15

Q172. ⋆ **How does `ConcurrentHashMap` work? How is it different from `Hashtable` and `synchronizedMap`?** `Hashtable` and `Collections.synchronizedMap` lock the whole map for every call. Since Java 8, `ConcurrentHashMap` uses CAS to place the first node in an empty bucket and otherwise locks only that bucket's head node. Reads take no lock at all, and writes to different buckets run in parallel. It forbids `null` keys and values. Its iterators are weakly consistent and never throw `ConcurrentModificationException`. Use `compute`/`merge`/`putIfAbsent` for atomic read-modify-write. → *More:* Ch 12 — 12.16, Ch 13 — 13.7

Q173. ⋆ **What is a deadlock? What are its conditions, and how do you prevent and detect it?** A deadlock happens when threads wait on each other forever, for example T1 holds lock A and wants B while T2 holds B and wants A. The four Coffman conditions must all hold: mutual exclusion, hold-and-wait, no preemption and circular wait. To prevent it, break one condition. The most common way is to always acquire locks in a global order (for example by account ID). Others are `tryLock` with a timeout and backoff, or holding fewer locks. To detect it, take a thread dump (`jstack`), use `ThreadMXBean.findDeadlockedThreads()`, or use a wait-for graph. → *More:* Ch 12 — 12.17 Deadlock

Q174. Livelock vs starvation. In a **livelock**, threads are active but make no progress because they keep reacting to each other, like two people stepping aside the same way in a corridor. The fix is random backoff. In **starvation**, a thread never gets the CPU or the lock because others keep winning. The fix is fair locks (`new ReentrantLock(true)`), bounded queues and avoiding priority tricks. → *More:* Ch 12 — 12.18, 12.19

Q175. ⋆ **Explain the producer-consumer problem and how to solve it.** Producers put items into a bounded buffer and consumers take them out. Producers must wait when the buffer is full, and consumers must wait when it is empty, without busy-spinning or racing. The easy solution is a `BlockingQueue` (`ArrayBlockingQueue`), whose `put` and `take` block. Hand-rolled versions use a lock with `notFull`/`notEmpty` conditions, or `wait`/`notifyAll` in `while` loops. → *More:* Ch 12 — 12.22, Ch 13 — 13.8

Q176. The reader-writer problem. Many readers may read shared data at once, but a writer needs exclusive access. A naive solution can starve writers while readers keep arriving. In Java, use `ReentrantReadWriteLock` (with the fair option if writer starvation matters), `StampedLock`, or copy-on-write structures when writes are rare. → *More:* Ch 12 — 12.23

Q177. ⋆ **How do you prevent double booking of a seat or parking spot?** The danger is check-then-act: two requests both see the seat free and both book it. In memory, make the check and the claim one atomic step, for example `seatHolds.putIfAbsent(seatId, userId)` on a `ConcurrentHashMap`, or lock per show or seat (fine-grained), acquiring several seats in sorted order. With a database, use **optimistic locking** (a version column, and the update fails if the version changed, so you retry or report "taken"), **pessimistic locking** (`SELECT ... FOR UPDATE`) or a unique constraint. Add temporary holds with a timeout during payment. → *More:* Ch 12 — 12.25

Q178. Optimistic vs pessimistic locking. Pessimistic locking takes the lock *before* working, assuming conflicts are likely. It is safe but blocks others. **Optimistic** locking works without a lock and checks at commit that nobody changed the data (a version number or CAS), retrying if they did. It is fast when conflicts are rare. Use pessimistic locking for hot, contended rows (the last seats of a flash sale) and optimistic locking for most other updates. → *More:* Ch 12 — 12.25

Q179. Coarse-grained vs fine-grained locking. **Coarse-grained** means one lock guards a lot (the whole parking lot). It is simple and deadlock-free but serialises everything. **Fine-grained** means many locks guard small parts (per floor, per spot, per bucket). Throughput goes up, but so do complexity and deadlock risk. Start coarse, measure, and split only the hot path. Lock striping is the middle ground. → *More:* Ch 12 — 12.11

Q180. Name the ways to make a class thread-safe. In rough order of preference: make it **immutable** (records, final fields). **Confine** the state to one thread (local variables, `ThreadLocal`, or a single-threaded owner reached via a queue). Use **thread-safe building blocks** (`ConcurrentHashMap`, atomics, `BlockingQueue`). Last, **lock** carefully around every invariant that spans several fields. Document which one you chose. → *More:* Ch 12 — 12.24 Immutability & thread confinement

Java specifics for LLD rounds

Q181. ⋆ Why is `String` immutable in Java? Strings are shared everywhere, so immutability makes that safe. Literals live in the **string pool** and are reused, which would be dangerous if one holder could change them. Immutable strings are automatically **thread-safe**. Their `hashCode` can be **cached**, which makes them fast, stable `HashMap` keys. They are also **secure** for values like file paths, URLs and class names, which cannot be changed after they are checked. "Modifying" methods such as `concat` and `replace` return new strings. → *More:* Ch 02 — 2.13 Immutability

Q182. ⋆ Is Java pass-by-value or pass-by-reference? Always **pass-by-value**. For objects, the value copied is the *reference*. So a method can change the object's fields through its copy (`list.add(x)` is visible to the caller), but reassigning the parameter (`list = new ArrayList<>()`) does not affect the caller's variable. That is why a `swap(a, b)` method can't swap the caller's objects.

```
void reset(StringBuilder sb) { sb.setLength(0); sb = new StringBuilder("x"); }
// the caller's sb is now empty, not "x"
```

→ *More:* Ch 02 — 2.2 Classes and objects

Q183. ⋆ `==` vs `equals()` for strings (and the string pool). `==` compares references (is this the same object?), while `equals` compares contents. `"ab" == "ab"` is true because string literals (and compile-time constants such as `"a" + "b"`) are interned into the pool. `new String("ab") == "ab"` is false, and so is a string built at runtime unless you call `intern()`. Always compare strings with `equals` (or `Objects.equals` for null-safety). → *More:* Ch 02 — 2.12 Identity vs equality

Q184. `String` vs `StringBuilder` vs `StringBuffer`. `String` is immutable, so concatenating in a loop creates many temporary objects. `StringBuilder` is mutable and fast but not thread-safe, which makes it the default for building text. `StringBuffer` is the old synchronised version and is rarely needed, since builders are almost always used inside one thread. → *More:* Ch 02 — 2.13 Immutability

Q185. `final` vs `finally` vs `finalize`. `final` is a modifier for a constant variable, a non-overridable method or a non-extendable class. `finally` is the block after `try` that always runs, used for cleanup, though try-with-resources is better. `finalize()` was a GC cleanup hook. It has been deprecated for removal since JDK 18 (JEP 421) because it is unpredictable and slow. Use `AutoCloseable` or `java.lang.ref.Cleaner` instead. → *More:* Ch 02 — 2.2 static and final

Q186. ⋆ How does `HashMap` work internally? `HashMap` keeps an array of buckets. It spreads the key's `hashCode` bits and masks the result to a bucket index, then compares with `equals` inside the bucket. Collisions form a linked list. When a bucket already holding 8 entries (`TREEIFY_THRESHOLD`) gets another, the list becomes a red-black tree, but only if the table has at least 64 slots (`MIN_TREEIFY_CAPACITY`); a smaller table is resized instead. A tree shrinks back to a list at 6 entries or fewer. When the size exceeds capacity × load factor (0.75 by default), the table doubles and entries are redistributed. It allows one `null` key and is not thread-safe. → *More:* Ch 13 — 13.7 Concurrent HashMap

Q187. `Comparable` vs `Comparator`. `Comparable` gives a class its single natural order through `compareTo`, for example `Money` by amount. A `Comparator` is an external, swappable ordering, which makes it a Strategy (`Comparator.comparing(Ride::fare).thenComparing(Ride::eta)`). Keep `compareTo` consistent with `equals`, or sorted sets and maps will behave oddly. → *More:* Ch 11 — 11.3 Strategy

Q188. Fail-fast vs fail-safe iterators. **Fail-fast** iterators (`ArrayList`, `HashMap`) throw `ConcurrentModificationException` if the collection is structurally changed during iteration, other than through the iterator's own `remove`. **Fail-safe** (weakly consistent or snapshot) iterators (`ConcurrentHashMap`,

`CopyOnWriteArrayList`) never throw, but may not show the latest changes. To remove while looping, use `iterator.remove()` or `removeIf`. → *More:* Ch 12 — 12.16 Concurrent collections

Q189. ⋆ Sealed classes, records and pattern matching: how do they help in LLD? A `sealed` interface lists its only allowed implementations (Java 17). Combined with records and pattern-matching `switch` (Java 21), the compiler checks that every case is handled, with no `default` needed. This is a clean way to model closed sets like payment results or vehicle types.

```
sealed interface Payment permits Card, Upi {}
record Card(String last4) implements Payment {}
record Upi(String vpa) implements Payment {}
String describe(Payment p) { return switch (p) {
    case Card c -> "Card " + c.last4();
    case Upi u  -> "UPI " + u.vpa(); }; }
```

→ *More:* Ch 02 — 2.13 Immutability and records

Q190. Generics: what are wildcards and PECS? `List<? extends Shape>` accepts lists of any Shape subtype and lets you *read* Shapes, but you can't add to it. `List<? super Circle>` lets you *add* Circles. **PECS** ("Producer Extends, Consumer Super") is the rule: use `extends` for sources you read from and `super` for sinks you write to, as in `Collections.copy(List<? super T> dst, List<? extends T> src)`. Generics are erased at runtime, so `new T()` is impossible. → *More:* Ch 02 — 2.15 Generics basics

Q191. When should you use `Optional`, and how do you avoid `null` in a design? Return `Optional<T>` from lookups that may find nothing (`findSpot()`), so callers must handle absence. Don't use it for fields, parameters or collections; return an empty collection instead. Elsewhere, avoid `null` with validated constructors, the Null Object pattern and `Objects.requireNonNull`. → *More:* Ch 11 — 11.7.1 Null Object

Part C — How to run a 60–90 minute machine-coding round

A machine-coding round asks you to build a small working system, such as Splitwise, a parking lot or a ride-sharing service, in one sitting, usually in an IDE. It is judged on whether the code runs, how it is structured, and how easily it absorbs a change. The plan below assumes 90 minutes. For a 60-minute round, roughly halve each step.

Minutes 0–10: clarify and cut scope. Read the problem twice. Write down the must-have features (usually 3–5 commands), the input/output format, and what you will leave out (persistence, auth, UI). Ask about the tricky rules, such as rounding, ties, invalid input and concurrent users. Confirm that in-memory storage and a console demo are fine.

Minutes 10–20: model. List the entities (nouns) and the services (verbs). Decide the few interfaces where change is likely, such as a pricing, split or allocation strategy. Sketch the packages: `model`, `service`, `strategy`, `repository` (in-memory maps), `exception`, and `Main` for the demo. Don't draw full UML. A short list of classes is enough.

Minutes 20–60: build a vertical slice first. Get the main flow working end to end (create → act → show) before adding anything else. Use records and enums for the models, constructor injection for the services, and clear custom exceptions for invalid operations. Commit or save each working state. Write code that compiles at every step, and don't leave a half-built feature that breaks the demo.

Minutes 60–75: complete the features and handle edge cases. Add the remaining commands one by one. Validate input, handle "not found" and duplicates, and make any shared state thread-safe if concurrency was asked (`ConcurrentHashMap`, per-entity locks).

Minutes 75–85: demo and test. Run `Main` on the sample input plus two or three edge cases. If time allows, add a few unit tests for the core rule, such as the split math or the fare calculation.

Minutes 85–90: walk through and extend. Explain the structure in one minute, then show how the likely extension fits: "a new split type is one new class". Be honest about anything you skipped.

Common ways to lose marks: one giant class, no working demo, over-engineering before the basics run, ignoring input validation, and staying silent. Talk while you code.

→ *More:* Ch 01 — The LLD interview approach, Ch 21 — 21.0 How the machine-coding round works

Sources of the question lists

Question wording and frequency only; every answer above is written for this book.

- LeetCode Discuss interview-experience posts (Flipkart SDE2, Swiggy SDE1/SDE2, Atlassian P4, PhonePe, Meta E5) and the "How do you prepare for machine coding rounds" thread
- prasadgujar/low-level-design-primer — `questions.md` (LeetCode-Discuss-sourced list)
- ashishps1/awesome-low-level-design — "LLD Interview Problems" and concurrency problems
- kumaransg/LLD — machine-coding questions tagged Flipkart / Udaan / Navi / ClearTrip
- donnemartin/system-design-primer — "Object-oriented design interview questions"
- workat.tech machine-coding practice problems
- Course tables of contents (titles only): Grokking the Object-Oriented Design Interview